

Функциональное программирование

Ярон Мински
Анил Мадхавапедди
Джейсон Хикки

Программирование

на языке OCaml

Ярон Мински, Анил Мадхавапедди и Джейсон Хикки

Программирование на языке OCaml

Bruce A. Tate

Real World OCaml

O'REILLY®

Ярон Мински, Анил Мадхавапедди и Джейсон Хикки

Программирование на языке OCaml



Москва, 2014

УДК 004.6
ББК 32.973.26
М57

Мински Я., Мадхавapedди А., Хикки Дж.
М57 Программирование на языке OCaml / пер. с англ. А. Н. Киселева. – М.: ДМК
Пресс, 2014. – 536 с.: ил.

ISBN 978-5-97060-539-6

Эта книга введет вас в мир OCaml, надежный язык программирования, обладающий большой выразительностью, безопасностью и быстродействием. Пройдя через множество примеров, вы быстро поймете, что OCaml – это превосходный инструмент, позволяющий писать быстрый, компактный и надежный системный код.

Вы познакомитесь с основными понятиями языка, узнаете о приемах и инструментах, помогающих превратить OCaml в эффективное средство разработки практических приложений. В конце книги вы сможете углубиться в изучение тонких особенностей инструментов компилятора и среды выполнения OCaml.

УДК 004.6
ББК 32.973.26

Все права защищены. Любая часть этой книги не может быть воспроизведена в какой бы то ни было форме и какими бы то ни было средствами без письменного разрешения владельцев авторских прав.

Материал, изложенный в данной книге, многократно проверен. Но поскольку вероятность технических ошибок все равно существует, издательство не может гарантировать абсолютную точность и правильность приводимых сведений. В связи с этим издательство не несет ответственности за возможные ошибки, связанные с использованием книги.

ISBN 978-1-449-32391-2 (анг.)

ISBN 978-5-97060-539-6 (рус.)

© 2014 Yaron Minsky, Anil Madhavapeddy,
Jason Hickey

© Оформление, перевод, ДМК Пресс, 2014

Моей Лайзе, верящей в силу слов и помогшей мне найти меня. – Ярон

*Моим маме и папе, отведшим меня в библиотеку
и освободившим мое воображение. – Анил*

Моей Нобу, наполняющей каждый мой день новыми событиями. – Джейсон

Содержание

Вступление	14
Часть I. Основы языка	22
Глава 1. Введение	23
ОSam1 как калькулятор	23
Функции и автоматический вывод типов	25
Автоматический вывод типов	27
Автоматический вывод обобщенных типов	28
Кортежи, списки, необязательные значения и сопоставление с образцом.....	30
Кортежи	30
Списки	31
Необязательные значения	38
Записи и варианты	40
Императивное программирование.....	42
Массивы	42
Изменяемые поля записей	43
Ссылки	45
Циклы for и while	46
Законченная программа	48
Компиляция и запуск.....	48
Что дальше	49
Глава 2. Переменные и функции	50
Переменные.....	50
Сопоставление с образцом и let.....	53
Функции.....	54
Анонимные функции	55
Функции нескольких аргументов.....	57
Рекурсивные функции.....	59
Префиксные и инфиксные операторы.....	60
Объявление функций с помощью ключевого слова function.....	65
Аргументы с метками.....	66
Необязательные аргументы	69
Глава 3. Списки и образцы	77
Основы списков.....	77
Использование сопоставления с образцом для извлечения данных из списка	78
Ограничения (и благословения) сопоставления с образцом	80
Производительность	81
Определение ошибок	83

Эффективное использование модуля List	84
Другие полезные функции из модуля List	88
Хвостовая рекурсия	91
Компактность и скорость сопоставления с образцом	93
Глава 4. Файлы, модули программы	98
Программы в единственном файле	98
Программы и модули из нескольких файлов	101
Сигнатуры и абстрактные типы	103
Конкретные типы в сигнатурах	106
Вложенные модули	107
Открытие модулей	109
Подключение модулей	111
Типичные ошибки при работе с модулями	113
Несовпадение типов	113
Отсутствие определений	114
Несоответствие определений типов	114
Циклические зависимости	115
Проектирование с применением модулей	117
Старайтесь не экспортировать конкретные типы	117
Продумывайте синтаксис вызовов	117
Создавайте однородные интерфейсы	118
Определяйте интерфейсы до реализации	119
Глава 5. Записи	120
Сопоставление с образцом и полнота	122
Уплотнение полей	124
Повторное использование имен полей	125
Функциональные обновления	129
Изменяемые поля	131
Поля первого порядка	132
Глава 6. Варианты	137
Универсальные образцы и рефакторинг	139
Объединение записей и вариантов	141
Варианты и рекурсивные структуры данных	145
Полиморфные варианты	149
Пример: и снова о цветных терминалах	151
Когда следует использовать полиморфные варианты	157
Глава 7. Обработка ошибок	159
Типы возвращаемых значений с признаком ошибки	159
Кодирование ошибок в результате	160
Error и Or_error	161

Функция bind и другие идиомы обработки ошибок.....	163
Исключения.....	165
Вспомогательные функции для возбуждения исключений.....	167
Обработчики исключений.....	169
Восстановление работоспособности после исключений.....	169
Перехват определенных исключений.....	170
Трассировка стека.....	172
От исключений к типам с информацией об ошибках и обратно	174
Выбор стратегии обработки ошибок.....	175
Глава 8. Императивное программирование.....	177
Пример: императивные словари.....	177
Элементарные изменяемые данные	182
Данные в формах, подобных массивам.....	182
Изменяемые поля записей и объектов и ссылочные ячейки	183
Внешние функции.....	184
Циклы for и while	184
Пример: двусвязные списки.....	186
Изменение списка	188
Итеративные функции	189
Отложенные вычисления и другие благоприятные эффекты	190
Мемоизация и динамическое программирование	192
Ввод и вывод.....	200
Терминальный ввод/вывод.....	200
Форматированный вывод с помощью printf.....	202
Файловый ввод/вывод	204
Порядок вычислений.....	207
Побочные эффекты и слабый полиморфизм	209
Ограничение значений	210
Частичное применение и ограничение значения	211
Ослабление ограничения значений	212
В заключение.....	215
Глава 9. Функторы.....	216
Простейший пример	216
Более практичный пример: вычисления с применением интервалов.....	218
Создание абстрактных функторов	222
Совместно используемые ограничения	223
Деструктивная подстановка	225
Использование нескольких интерфейсов.....	227
Расширение модулей	231
Глава 10. Модули первого порядка.....	235
Приемы работы с модулями первого порядка.....	235

Пример: фреймворк обработки запросов	241
Реализация обработчика запросов	242
Диспетчеризация запросов по нескольким обработчикам	244
Загрузка и выгрузка обработчиков запросов.....	248
Жизнь без модулей первого порядка	252
Глава 11. Объекты	253
Объекты OCaml	253
Полиморфизм объектов.....	255
Неизменяемые объекты	257
Когда следует использовать объекты.....	258
Подтипизация	259
Подтипизация в ширину	259
Подтипизация в глубину	260
Вариантность	261
Сужение	265
Подтипизация и рядный полиморфизм	267
Глава 12. Классы	269
Классы в OCaml	269
Параметры класса и полиморфизм.....	270
Типы объектов и интерфейсы.....	272
Функциональные итераторы.....	274
Наследование	276
Типы классов	277
Открытая рекурсия	278
Скрытые методы.....	280
Бинарные методы.....	281
Виртуальные классы и методы.....	285
Создание простых фигур	285
Инициализаторы.....	288
Множественное наследование.....	288
Как выполняется разрешение имен	289
Примеси	290
Отображение анимированных фигур.....	293
Часть II. Инструменты и технологии	295
Глава 13. Отображения и хэш-таблицы	296
Отображения	297
Создание отображений с компараторами	298
Деревья.....	301
Полиморфные компараторы.....	302
Множества	304

Соответствие интерфейсу Comparable.S	304
Хэш-таблицы	307
Соответствие интерфейсу Hashable.S	310
Выбор между отображениями и хэш-таблицами.....	311
Глава 14. Анализ командной строки.....	315
Простейший анализ командной строки	315
Анонимные аргументы	316
Определение простых команд.....	317
Выполнение простых команд.....	317
Типы аргументов	319
Определение собственных типов аргументов	320
Необязательные аргументы и аргументы по умолчанию.....	321
Последовательности аргументов	324
Добавление поддержки передачи именованных флагов в командной строке.....	325
Группировка подкоманд	327
Расширенное управление парсингом.....	329
Типы в основе Command.Spec	330
Объединение фрагментов спецификаций	331
Интерактивный запрос ввода.....	333
Добавление аргументов с метками в функции обратного вызова.....	335
Автодополнение командной строки средствами Bash	336
Создание фрагментов автодополнения	336
Установка фрагмента автодополнения	337
Альтернативные парсеры командной строки.....	338
Глава 15. Обработка данных JSON	339
Основы JSON	339
Парсинг данных в формате JSON с помощью Yojson	340
Выборка значений из структур JSON	343
Конструирование значений JSON	346
Использование нестандартных расширений JSON	348
Автоматическое отображение JSON в типы OCaml.....	350
Основы ATD.....	350
Аннотации ATD	351
Компиляция спецификаций ATD в код на OCaml.....	352
Пример: запрос информации об организации в GitHub	353
Глава 16. Парсинг с помощью OCamllex и Menhir	357
Лексический анализ и парсинг	358
Определение парсера.....	360
Описание грамматики.....	360
Парсинг последовательностей	362
Определение лексического анализатора.....	364

Вступление.....	364
Регулярные выражения.....	365
Лексические правила	366
Рекурсивные правила	367
Объединяем все вместе	368

Глава 17. Сериализация данных с применением

s-выражений	371
Основы использования	372
Преобразование типов OCaml в s-выражения	374
Формат Sexpr	376
Сохранение инвариантов	377
Вывод информативных сообщений об ошибках.....	380
Директивы sexpr-преобразований	382
sexpr_opaque.....	383
sexpr_list	384
sexpr_option	385
Определение значений по умолчанию.....	385

Глава 18. Конкурентное программирование

с помощью Async	388
Основы Async	389
Ivar и uvar	393
Примеры: эхо-сервер.....	395
Усовершенствование эхо-сервера	399
Пример: поиск определений с помощью DuckDuckGo.....	401
Обработка URI	402
Парсинг строк JSON.....	402
Выполнение запроса HTTP	403
Обработка исключений.....	406
Мониторы.....	408
Пример: обработка исключений при работе с DuckDuckGo.....	410
Тайм-ауты, отмена и выбор.....	413
Работа с системными потоками	416
Защищенность данных в потоках и блокировки	419

Часть III. Система времени выполнения..... 421

Глава 19. Интерфейс внешних функций..... 422

Пример: интерфейс к терминалу	423
Простые скалярные типы языка C.....	427
Указатели и массивы.....	429
Выделение памяти для указателей.....	430
Использование представлений для отображения составных значений.....	431

Структуры и объединения	432
Определение структуры	432
Добавление полей в структуры	433
Незавершенные определения структур	433
Определение массивов	437
Передача функций в код на С	438
Пример: быстрая сортировка в командной строке	439
Дополнительная информация о взаимодействии с кодом на С	441
Организация структур в памяти	442
Глава 20. Представление значений в памяти	444
Блоки и значения OCaml	445
Различение целых чисел и указателей во время выполнения	445
Блоки и значения	447
Целые числа, символы и другие простые типы	448
Кортежи, записи и массивы	448
Вещественные числа и массивы	449
Варианты и списки	450
Полиморфные варианты	452
Строковые значения	453
Нестандартные блоки памяти	454
Управление внешней памятью средствами Bigarray	454
Глава 21. Сборка мусора	456
Алгоритм сборки мусора	456
Сборка мусора с разделением на поколения	457
Быстрая вспомогательная куча	457
Выделение памяти во вспомогательной куче	458
Основная куча долгоживущих блоков	459
Выделение памяти в основной куче	460
Стратегии распределения памяти	461
Маркировка и сканирование кучи	462
Компактификация кучи	463
Указатели между поколениями	464
Подключение функций-финализаторов к значениям	467
Глава 22. Компиляторы: парсинг и контроль типов	470
Обзор инструментов компилятора	470
Парсинг исходного кода	472
Синтаксические ошибки	473
Автоматическое оформление отступов в исходном коде	473
Автоматическое создание документации на основе интерфейсов	475
Препроцессинг исходного кода	477
Использование Camlp4 в интерактивной оболочке	479

Запуск Camlp4 из командной строки	480
Препроцессинг сигнатур модулей	482
Дополнительные источники информации о Camlp4	483
Статическая проверка типов	483
Демонстрация типов, выводимых компилятором	484
Вывод типов	486
Модули и отдельная компиляция	491
Упаковка модулей вместе	493
Сокращение путей к модулям в сообщениях об ошибках	495
Типизированное синтаксическое дерево	496
Использование ocp-index для поддержки автодополнения	496
Непосредственное исследование типизированного синтаксического дерева	497
Глава 23. Компиляторы: байт-код и машинный код	501
Нетипизированная lambda-форма	501
Оптимизация сопоставлений с образцом	501
Оценка производительности сопоставления с образцом	504
Переносимый байт-код	506
Компиляция и компоновка байт-кода	507
Выполнение байт-кода	508
Встраивание байт-кода OCaml в программы на C	509
Компиляция быстрого машинного кода	511
Исследование ассемблерного кода	511
Отладка двоичных выполняемых файлов	515
Профилирование машинного кода	519
Встраивание машинного кода в программы на C	521
Сводка по расширениям имен файлов	522
Алфавитный указатель	523

Вступление

Почему именно OCaml?

Выбор языка программирования играет важную роль. Он влияет на надежность, безопасность и эффективность программ, а также простоту чтения кода, его рефакторинга и расширения. Языки способны также влиять на образ мышления программиста и приемы проектирования программ, даже когда они не используются.

Мы решили написать эту книгу, потому что верим в важность выбора языка программирования и особенно в важность изучения OCaml. Каждый из нас имеет более чем 15-летний опыт использования OCaml в академической практике и профессиональной карьере, и к настоящему времени все мы уверены, что этот язык действительно является мощным инструментом для создания сложных программных систем. Наша цель – сделать этот инструмент более доступным для широкого круга программистов, рассказав о нем все, что необходимо для эффективного использования OCaml в повседневной практике.

Что делает OCaml особенным, так это то, что он занимает золотую середину в пространстве языков программирования. Он обладает уникальной комбинацией эффективности, выразительности и практичности, которую вы не найдете ни в каком другом языке. В значительной степени это обусловлено превосходным сочетанием некоторых ключевых особенностей OCaml, перечисленных ниже, которые продолжают развиваться уже более 40 лет.

- *Механизм сборки мусора*, обеспечивающий автоматическое управление памятью. В наши дни этой особенностью обладают многие современные языки высокого уровня.
- *Функции первого порядка (first-class functions)*¹, которые можно передавать как самые обычные значения. Аналогичной особенностью обладают, например, JavaScript, Common Lisp и C#.
- *Статический контроль типов* для увеличения производительности и снижения числа ошибок, проявляющихся во время выполнения, как в Java и C#.
- *Параметрический полиморфизм*, позволяющий конструировать абстракции, применимые к данным разных типов. Похожая особенность существует в Java и C# в виде поддержки обобщенных типов (generics) и в C++? в виде шаблонов.
- Великолепная поддержка *неизменяемых (immutable) данных*, обеспечивающих возможность создания программ, не вносящих подчас разрушительных

¹ Термин «first-class functions» не имеет устоявшегося перевода на русский язык. В Интернете часто можно встретить такие толкования, как «функции первого рода», «функции первого класса» и даже «первоклассные функции». Однако, чтобы не вносить путаницу и подчеркнуть, что речь идет не о типах, а о свойстве функциональных языков, по согласованию с практиками, имеющими многолетний опыт функционального программирования, было решено использовать толкование «функции первого порядка». — *Прим. перев.*

изменений в структуры данных. Эта особенность является традиционным свойством функциональных языков, таких как Scheme, и поддерживается крупными фреймворками распределенных вычислений, такими как Hadoop.

- Механизм *автоматического вывода типов*, позволяющий избежать необходимости кропотливо объявлять тип каждой переменной в программе и автоматически определяющий типы на основе используемых значений. Эта особенность в ограниченном виде присутствует в C# в виде поддержки неявно типизированных локальных переменных (*implicitly typed local variables*) и в C++11 в виде ключевого слова `auto`.
- *Алгебраические типы данных* и механизм *сопоставления с образцом*, дающие возможность манипулировать сложными структурами данных. Подобные особенности доступны также в Scala и F#.

Некоторые из вас уже знают и с удовольствием используют эти особенности, для других они станут настоящим открытием, но большинство из вас будет встречать *некоторые* из них и в других языках. Как будет демонстрироваться на протяжении всей книги, наличие всех этих особенностей в одном языке придает ему особую силу. Несмотря на их важность, эти идеи нашли лишь ограниченное применение в господствующих языках, но, даже проникнув в эти языки, например функции первого порядка в C# или параметрический полиморфизм в Java, они обычно принимают ограниченную и нелепую форму. Единственными языками, воплотившими эти идеи в полном объеме, являются *функциональные языки со статическим контролем типов*, такие как OCaml, F#, Haskell, Scala и Standard ML.

В ряду этих достойных языков OCaml стоит особняком, потому что ему удастся обеспечить большую власть, оставаясь при этом весьма практичным языком. Компилятор OCaml использует довольно простую стратегию компиляции и производит высокоэффективный код, не требующий сложных оптимизаций и применения дорогостоящей динамической компиляции (Just-in-Time, JIT). Это, наряду со строгой моделью вычислений в языке OCaml, делает поведение программ легко предсказуемым. Сборщик мусора использует *инкрементальный* алгоритм, исключая возможность появления длительных пауз на сборку мусора, и обеспечивает высокую точность, в том смысле, что надежно соберет все неиспользуемые данные (в отличие от сборщиков мусора, использующих алгоритмы на основе подсчета ссылок).

Все вышперечисленное делает OCaml превосходным выбором для программистов, желающих освоить лучший язык программирования и в то же время продолжать решать практические задачи.

Краткая история развития

OCaml был написан группой разработчиков, в состав которой вошли Ксавье Леруа (Xavier Leroy), Джером Вуййон (Jérôme Vouillon), Дамиан Долигес (Damien Doligez) и Дидье Реми (Didier Rémy), в 1996 году в институте INRIA (Франция). Он явился результатом многолетних исследований языков семейства ML, разработка которых началась в 60-х годах и которые имеют глубокие связи с академическим сообществом.

Язык ML (Meta Language) был создан в 1972 году Робинот Милнерот (Robin Milner) (работавшим сначала в Стэнфордском, а потом в Кембриджском университете) как метаязык логики вычислимых функций (Logic for Computable Functions, LCF) для построения формальных доказательств. Со временем ML был преобразован в компилятор с целью упростить использование LCF на компьютерах с разной архитектурой и к началу 80-х постепенно превратился в полноценную, развитую систему.

Первая реализация языка Caml появилась в 1987 году. Она была создана Аскандером Суаресом (Ascánder Suárez) и продолжена Пьером Вейссом (Pierre Weis) и Мишелем Мони (Michel Mauny). В 1990 году Ксавье Леруа и Дамиан Долигес создали новую реализацию под названием Caml Light, выполненную в виде интерпретатора байт-кода с быстрым, последовательным сборщиком мусора. В течение следующих нескольких лет были разработаны практичные библиотеки, такие как инструменты управления синтаксисом, написанные Мишелем Мони, что способствовало продвижению Caml в академические и исследовательские круги.

Ксавье Леруа продолжил работу над языком Caml Light, дополнив его новыми возможностями, что привело к выходу в 1995 году версии Caml Special Light. Она обеспечивала существенно более высокую производительность за счет собственного компилятора, сопоставимую с такими языками, как C++. Система модулей, реализованная в духе Standard ML, также обладала мощными возможностями и упрощала создание крупномасштабных программных продуктов.

Современный язык OCaml появился в 1996 году, когда Дидье Реми и Джером Вуйион добавили в него мощную и изящную объектную систему. Примечательной особенностью этой объектной системы была поддержка многих типичных объектно-ориентированных идиом в сочетании со статическим контролем типов, тогда как поддержка тех же идиом в других языках, таких как C++ и Java, требовала дополнительных проверок во время выполнения. В 2000 году Жак Гарриг (Jacques Garrigue) добавил в OCaml еще несколько новых особенностей, таких как полиморфные методы (polymorphic methods), варианты, а также аргументы с метками (labeled arguments) и необязательные аргументы.

В последнее десятилетие отмечался значительный рост популярности OCaml и постоянное его совершенствование с целью поддержать растущий коммерческий и академический интерес. Модули первого порядка, обобщенные алгебраические типы данных (Generalized Algebraic Data Types, GADT) и динамическое связывание повысили гибкость языка. Появилась также поддержка аппаратных архитектур x86_64, ARM, PowerPC и Sparc, что превратило OCaml в отличный выбор для систем, где потребление ресурсов, предсказуемость и производительность являются важными факторами.

Стандартная библиотека Core

Одного языка недостаточно для практического его применения. Для разработки прикладных программ необходим также богатый набор библиотек. Ограниченный объем возможностей стандартной библиотеки OCaml, распространяемой вместе

с компилятором, часто является причиной разочарований при изучении OCaml. Именно поэтому стандартную библиотеку не следует рассматривать как универсальный инструмент – она создавалась только для поддержки компилятора и потому целенаправленно сохранялась небольшой, с маленьким количеством функций.

К счастью, в мире открытого программного обеспечения ничто не мешает созданию альтернативных библиотек в дополнение к стандартной, и именно они включаются в состав базового дистрибутива.

В недрах Jane Street – компании, использующей OCaml уже более десяти лет, – для внутреннего применения была разработана стандартная библиотека Core. Она изначально проектировалась с прицелом на звание универсальной стандартной библиотеки. Как и сам язык OCaml, библиотека Core создавалась с учетом требований к безошибочности, надежности и производительности.

Библиотека Core распространяется вместе с дополнительными синтаксическими расширениями, добавляющими новые возможности в язык OCaml, и другими библиотеками, такими как библиотека Async поддержки асинхронных сетевых взаимодействий, обеспечивающая возможность создания сложных распределенных систем только средствами библиотеки Core. Все эти библиотеки распространяются под свободной лицензией Apache 2, что дает возможность беспрепятственно использовать ее в любительских, академических и коммерческих разработках.

Платформа OCaml

Core – всеобъемлющая и эффективная стандартная библиотека, но, кроме нее, существует еще масса программного обеспечения на OCaml. С момента выхода первой версии OCaml в 1996 году сформировалось обширное сообщество программистов, которым было создано множество полезных библиотек и инструментов. Мы представим некоторые из этих библиотек при рассмотрении примеров в данной книге.

Установка этих сторонних библиотек выполняется легко и просто, с помощью инструмента управления пакетами, известного как ОРАМ. Мы подробнее расскажем об этом инструменте далее в книге, а сейчас просто отметим, что он составляет основу платформы, образуемую инструментами и библиотеками, наряду с компилятором OCaml, которая позволяет быстро и эффективно создавать прикладные программы.

Мы также воспользуемся этим инструментом для установки *utop* – интерфейса командной строки. Это современная интерактивная оболочка, поддерживающая историю команд, интерпретацию макросов, дополнение имен модулей и другие приятные мелочи, которые делают работу с языком намного удобнее. Мы будем использовать *utop* на протяжении всей книги для опробования примеров.

Об этой книге

Данная книга предназначена для программистов, имеющих некоторый опыт использования обычных языков программирования, причем необязательно функциональных. В зависимости от уровня подготовки многие понятия, рассматривае-

мые здесь, окажутся для вас незнакомыми, включая названия таких традиционных инструментов функционального программирования, как функции высшего порядка и неизменяемые типы данных, а также некоторые особенности мощной системы типов OCaml и системы модулей.

Если вы уже знакомы с языком OCaml, эта книга может преподнести вам немало сюрпризов. Библиотека Core переопределяет многие стандартные пространства имен с целью наделить систему модулей OCaml новыми особенностями и сделать доступными по умолчанию некоторые мощные структуры данных. Старый код на OCaml все еще сможет взаимодействовать с библиотекой Core, но вам может потребоваться изменить его, чтобы получить максимальную выгоду. Весь новый код, который еще только предстоит написать, изначально будет использовать Core, и мы уверены, что время на изучение модели Core не будет потрачено впустую – она с успехом используется многими программами и помогает устранить препятствия, возникающие при создании сложных приложений на OCaml.

Код, использующий традиционную стандартную библиотеку компилятора, всегда будет существовать, однако для изучения ее особенностей существует масса ресурсов в Интернете. Книга «Практический OCaml», напротив, описывает приемы, используемые авторами в своей работе, для создания масштабируемых, надежных программных систем.

План книги

Данная книга делится на три части:

- первая часть описывает сам язык, начиная с общего обзора OCaml. Не отчаивайтесь, если при чтении этой части что-то останется для вас непонятным. Основная задача данной части – дать вам представление о различных аспектах языка, а идеи, представленные здесь, более подробно будут описаны в последующих главах.
После начального знакомства с языком мы перейдем к более сложным особенностям, таким как модули, функторы и объекты, обзор которых займет достаточно много времени. Знание этих понятий играет важную роль. Эти идеи послужат вам хорошим фундаментом, даже при использовании других новейших языков, отличных от OCaml, многие из которых основаны на идеях, заложенных в ML;
- во второй части закладываются основы разработки типичных приложений на примерах использования инструментов и приемов программирования, от анализа аргументов командной строки до реализации асинхронных сетевых взаимодействий. Попутно вы увидите, как объединяются некоторые понятия, представленные в первой части, в действующие библиотеки и инструменты, сочетающие в себе различные возможности языка;
- в третьей части обсуждаются система времени выполнения OCaml и набор инструментов компилятора. Эта тема достаточно проста, если сравнивать с реализациями других языков (такими как виртуальная машина Java или .NET CLR). В этой части вы получите все знания, необходимые для соз-

дания высокопроизводительных систем и интерфейсов с библиотеками на языке C. Здесь мы также поговорим о приемах профилирования и отладки с применением таких инструментов, как GNU *gdb*.

Инструкции по установке

Далее мы будем использовать некоторые инструменты, созданные нами в процессе работы над этой книгой. Некоторые из них направлены на улучшение компилятора OCaml, а это означает, что вам нужно будет обновить свою среду разработки (использовать компилятор версии 4.01). Процесс установки протекает преимущественно в автоматическом режиме, при использовании диспетчера пакетов OPAM. Инструкции по установке и перечень устанавливаемых пакетов можно найти по адресу: <https://github.com/realworldocaml/book/wiki/Installation-Instructions>.

К моменту публикации книги библиотека Core не поддерживала операционную систему Windows, поэтому ее работоспособность может быть гарантирована только в Mac OS X, Linux, FreeBSD и OpenBSD. Если вы пользуетесь ОС Windows, обращайтесь на страницу с инструкциями по установке, указанную выше (возможно, когда вы будете читать эту книгу, что-то изменится), или установите Linux в виртуальную машину, чтобы следовать за примерами в книге, поскольку они стоят того.

Эта книга не является справочным пособием. Ее цель – познакомить вас с языком, библиотеками, инструментами и приемами программирования, которые помогут вам стать эффективным программистом на OCaml. Но она не может служить заменой документации с описанием API и справочных руководств по языку OCaml. Ссылки на документацию с описанием всех библиотек и инструментов, упоминаемых в книге, вы найдете по адресу: <https://ocaml.janestreet.com/ocaml-core/latest/doc/>.

Примеры программного кода

Все примеры в книге свободно доступны на условиях общественной лицензии. Вы можете копировать и использовать любые фрагменты примеров в своих разработках, без каких-либо ограничений и упоминания авторства.

Репозиторий с программным кодом доступен по адресу: <https://github.com/realworldocaml/examples>. Все фрагменты кода в книге снабжены заголовками, из которых вы сможете узнать имена файлов с исходным кодом, сценариями командной оболочки или вспомогательными данными.

Если вам покажется, что использование вами примеров нарушит условия, описанные выше, обращайтесь с вопросами к нам по адресу: permissions@oreilly.com.

Safari® Books Online

Safari Books Online (www.safaribooksonline.com) – это виртуальная библиотека, содержащая авторитетную информацию в виде книг и видеоматериалов, созданных ведущими специалистами в области технологий и бизнеса. Профессионалы

в области технологии, разработчики программного обеспечения, веб-дизайнеры, а также бизнесмены и творческие работники используют Safari Books Online как основной источник информации для проведения исследований, решения проблем, обучения и подготовки к сертификационным испытаниям.

Библиотека Safari Books Online предлагает широкий выбор продуктов и тарифов для организаций, правительственных учреждений и физических лиц. Подписчики имеют доступ к поисковой базе данных, содержащей информацию о тысячах книг, видеоматериалов и рукописей от таких издателей, как O'Reilly Media, Prentice Hall Professional, Addison-Wesley Professional, Microsoft Press, Sams, Que, Peachpit Press, Focal Press, Cisco Press, John Wiley & Sons, Syngress, Morgan Kaufmann, IBM Redbooks, Packt, Adobe Press, FT Press, Apress, Manning, New Riders, McGraw-Hill, Jones & Bartlett, Course Technology и десятков других. За подробной информацией о Safari Books Online обращайтесь по адресу: <http://www.safaribooksonline.com/>.

Как с нами связаться

С вопросами и предложениями, касающимися этой книги, обращайтесь в издательство:

O'Reilly Media

1005 Gravenstein Highway North

Sebastopol, CA 95472

(800) 998-9938 (в Соединенных Штатах Америки или в Канаде)

(707) 829-0515 (международный)

(707) 829-0104 (факс)

Список опечаток, файлы с примерами и другую дополнительную информацию вы найдете на сайте книги:

<http://oreil.ly/realworldOCaml>.

Свои пожелания и вопросы технического характера отправляйте по адресу:

bookquestions@oreilly.com.

Дополнительную информацию о книгах, обсуждения, конференции и новости вы найдете на веб-сайте издательства: <http://www.oreilly.com>.

Ищите нас на Facebook: <http://facebook.com/oreilly>.

Следуйте за нами в Твиттере: <http://twitter.com/oreillymedia>.

Смотрите нас на YouTube: <http://www.youtube.com/oreillymedia>.

Благодарности

Нам хотелось бы выразить слова благодарности всем, кто помогал улучшать книгу «Практический OCaml»:

- Лео Уайт (Leo White) участвовал в работе над текстом глав 11 и 12 и предоставил примеры для них;

- Джереми Яллоп (Jeremy Yallop) создал и описал библиотеку Ctypes, представленную в главе 19;
- Стивен Уикс (Stephen Weeks) занимался разработкой модульной архитектуры библиотеки Core, и его многочисленные примечания послужили основой для глав 20 и 21;
- Джереми Димино (Jeremie Dimino) является автором *utop* – интерактивного интерфейса командной строки, используемого на протяжении всей книги. Мы особенно благодарны ему за изменения, которые он внес, чтобы улучшить работу *utop* в контексте этой книги;
- члены сообщества пользователей OCaml прислали более 2400 комментариев к рукописи книги, выложенной в Интернете для обсуждения. Благодаря им было выявлено и устранено огромное число ошибок и неточностей.

ОСНОВЫ ЯЗЫКА

Первая часть охватывает основные понятия языка, которые необходимо знать, чтобы создавать программы на языке OCaml. Она начинается обзором использования интерактивной оболочки. Последующие главы содержат более подробное описание сведений, представленных в первой главе, включая детальное изложение подходов к императивному программированию на OCaml.

В последних нескольких главах будут представлены мощные средства абстракции, имеющиеся в языке OCaml. Сначала мы познакомимся с функторами и используем их для создания библиотеки поддержки интервалов, а затем исследуем особенности применения модулей для построения системы плагинов с поддержкой контроля типов. Язык OCaml поддерживает также объектно-ориентированный стиль программирования, и мы закончим первую часть двумя главами, охватывающими объектную систему: сначала мы покажем, как напрямую использовать объекты OCaml, а затем продемонстрируем особенности использования системы классов и расскажем о некоторых дополнительных возможностях, таких как наследование. В этих же главах мы обсудим проект простой объектно-ориентированной графической библиотеки.

Глава 1

Введение

В этой главе дается обзор языка OCaml посредством знакомства с серией небольших примеров, демонстрирующих основные особенности языка. Ее цель — дать вам представление о возможностях OCaml без глубокого погружения в каждую отдельно взятую тему.

На протяжении всей книги мы будем использовать библиотеку *Core*, более богатую возможностями замену стандартной библиотеки OCaml. Также мы будем использовать *utop*, интерактивную оболочку, которая позволяет вводить выражения и получать результаты в диалоговом режиме. *utop* — это более простая в использовании версия стандартной интерактивной оболочки OCaml (которую, кстати, можно запустить командой *ocaml*). Следующие далее инструкции предполагают использование именно инструмента *utop*.

Прежде чем продолжить, убедитесь, что у вас имеется действующая версия OCaml, чтобы вы могли опробовать примеры в процессе чтения данной главы.

OCaml как калькулятор

Первое, что необходимо предпринять, чтобы включить в работу библиотеку *Core*, — открыть *Core.Std*:

OCaml utop

<https://github.com/realworldocaml/examples/blob/v1/code/guided-tour/main.topscript>

```
$ utop
```

```
# open Core.Std;;
```

Эта инструкция открывает доступ к определениям в библиотеке *Core*, необходимым во многих примерах в этой и в последующих главах.

Теперь попробуем выполнять простейшие арифметические операции:

OCaml utop (part 1)

<https://github.com/realworldocaml/examples/blob/v1/code/guided-tour/main.topscript>

```
# 3 + 4;;  
- : int = 7  
# 8 / 3;;  
- : int = 2  
# 3.5 +. 6.;;  
3  
- : float = 9.5  
# 30_000_000 / 300_000;;
```



```
- : int = 100
# sqrt 9.;;
- : float = 3.
```

По большому счету, многое из того, что можно наблюдать здесь, имеется в любых других языках программирования, тем не менее сделаем сразу несколько замечаний:

- чтобы сообщить интерактивной оболочке, что она должна вычислить выражение, следует завершить инструкцию двумя символами `;;`. Это — особенность самой интерактивной оболочки. В программах так делать не следует (хотя иногда полезно включить `;;`, чтобы получить более подробное сообщение об ошибке, явно обозначив конец объявления верхнего уровня);
- после вычисления выражения интерактивная оболочка сначала печатает тип результата, а затем сам результат;
- аргументы отделяются от имен вызываемых функций пробелами, а не скобками или запятыми, что делает OCaml больше похожим на командную оболочку UNIX, чем на традиционные языки программирования, такие как C или Java;
- OCaml позволяет вставлять символы подчеркивания в числовые литералы для улучшения читаемости. Имейте в виду, что подчеркивания могут находиться в любом месте числового литерала, а не только через каждые три разряда;
- OCaml различает типы `float` (тип представления вещественных чисел) и `int` (тип представления целых чисел). Литералы разных типов отличаются (6. вместо 6), так же отличаются инфиксные операторы, предназначенные для обработки разных типов (+. вместо +), а кроме того, OCaml не выполняет автоматического приведения между этими типами. Возможно, это не очень удобно, но в этом есть и свои преимущества, поскольку данная особенность препятствует появлению некоторых видов ошибок, часто возникающих в других языках из-за различий между целочисленным и вещественным типами. Например, во многих языках выражение `1 / 3` вернет нуль, а выражение `1 / 3.0` вернет одну треть. OCaml требует явно указывать, какая операция должна быть выполнена.

Можно также создать переменную, чтобы присвоить имя результату выражения. Делается это с помощью ключевого слова `let`. Эта операция называется *let-связывание* (`let binding`):

OCaml utop (part 2)

<https://github.com/realworldocaml/examples/blob/v1/code/guided-tour/main.topscript>

```
# let x = 3 + 4;;
val x : int = 7
# let y = x + x;;
val y : int = 14
```

После создания новой переменной интерактивная оболочка сообщает ее имя (x или y), тип (int) и значение (7 или 14).

Отметьте, что не каждый идентификатор может играть роль имени переменной. Имена переменных не могут включать знаки пунктуации, кроме символа подчеркивания (`_`) и одиночной кавычки (`'`), и должны начинаться с буквы в нижнем регистре или с символа подчеркивания. То есть следующие имена являются допустимыми:

OCaml `utop` (part 3)

<https://github.com/realworldocaml/examples/blob/v1/code/guided-tour/main.topscript>

```
# let x7 = 3 + 4;;
val x7 : int = 7
# let x_plus_y = x + y;;
val x_plus_y : int = 21
# let x' = x + 1;;
val x' : int = 8
# let _x' = x' + x';;
# _x';;
- : int = 16
```

Обратите внимание, что по умолчанию *utop* не заботится о выводе имен переменных, начинающихся с символа подчеркивания.

Следующие имена являются недопустимыми:

OCaml `utop` (part 4)

<https://github.com/realworldocaml/examples/blob/v1/code/guided-tour/main.topscript>

```
# let Seven = 3 + 4;;
Characters 4-9:
Error: Unbound constructor Seven
(Ошибка: Несвязанный конструктор Seven)
# let 7x = 7;;
Characters 5-10:
Error: This expression should not be a function, the expected type is
int
(Ошибка: Это выражение не должно быть функцией, ожидается тип int)
# let x-plus-y = x + y;;
Characters 4-5:
Error: Parse error: [fun_binding] expected after [ipatt] (in [let_binding])
(Ошибка: Синтаксическая ошибка: [fun_binding] ожидается после [ipatt]
(в [let_binding]))
```

Эти сообщения об ошибках немного сбивают с толку, но они станут более понятны, когда вы поближе познакомитесь с языком.

ФУНКЦИИ И АВТОМАТИЧЕСКИЙ ВЫВОД ТИПОВ

Ключевое слово `let` можно также использовать для определения функций:

OCaml `utop` (part 5)

<https://github.com/realworldocaml/examples/blob/v1/code/guided-tour/main.topscript>

```
# let square x = x * x ;;
val square : int -> int = <fun>
```

```
# square 2;;
- : int = 4
# square (square 2) ;;
- : int = 16
```

Функции в языке OCaml являются самыми обычными значениями, как любые другие, именно поэтому связывание имени переменной с функцией выполняется с помощью ключевого слова `let`, как если бы мы использовали `let` для связывания имени переменной с простым значением, например целым числом. Когда `let` применяется для определения функции, первый идентификатор, следующий за `let`, интерпретируется как имя функции, а каждый последующий – как отдельный аргумент функции. То есть `square` в данном примере – это имя функции, принимающей единственный аргумент.

Теперь, после создания такого интересного значения, как функция, типы, что выводит интерактивная оболочка, также начинают становиться все интереснее и интереснее. Здесь `int -> int` представляет тип функции. В данном случае он сообщает, что мы имеем дело с функцией, принимающей аргумент типа `int` и возвращающей значение типа `int`. Имеется также возможность определять функции, принимающие несколько аргументов. (Обратите внимание, что следующий пример не будет работать, если не открыть `Core.Std`, как предлагалось выше.)

OCaml utop (part 6)

<https://github.com/realworldocaml/examples/blob/v1/code/guided-tour/main.topscript>

```
# let ratio x y =
    Float.of_int x /. Float.of_int y
;;
val ratio : int -> int -> float = <fun>
# ratio 4 7;;
- : float = 0.571428571429
```

Так получилось, что этот пример одновременно является нашим первым модулем. Конструкция `Float.of_int` здесь – это ссылка на функцию `of_int` в модуле `Float`. Такой прием несколько отличается от используемого в объектно-ориентированных языках, где форма записи с точкой обычно применяется для обращения к методу объекта. Отметьте, что имена модулей всегда начинаются с большой буквы.

Первое время форма представления типов функций с несколькими аргументами может казаться непривычной, но мы объясним, чем это обусловлено, когда доберемся до карринга функций в разделе «Функции с несколькими аргументами» в главе 2. А пока просто считайте стрелки простыми разделителями в списке типов аргументов, где последняя стрелка отделяет тип возвращаемого значения. То есть конструкция `int -> int -> float` описывает функцию, принимающую два аргумента типа `int` и возвращающую значение типа `float`.

Можно также написать функцию, принимающую другие функции в аргументах. Ниже приводится пример функции с тремя аргументами: функцией `test` и двумя целочисленными аргументами. Функция возвращает сумму целых чисел, прошедших проверку с помощью функции `test`:

OCaml utop (part 7)

<https://github.com/realworldocaml/examples/blob/v1/code/guided-tour/main.topscript>

```
# let sum_if_true test first second =
  (if test first then first else 0)
  + (if test second then second else 0)
;;
val sum_if_true : (int -> bool) -> int -> int -> int = <fun>
```

Если внимательнее взглянуть на сигнатуру, выведенную компилятором, можно заметить, что первый аргумент является функцией, принимающей целое число и возвращающей логическое значение, а остальные два аргумента – целые числа. Ниже приводится пример использования этой функции:

OCaml utop (part 8)

<https://github.com/realworldocaml/examples/blob/v1/code/guided-tour/main.topscript>

```
# let even x =
  x mod 2 = 0 ;;
val even : int -> bool = <fun>
# sum_if_true even 3 4;;
- : int = 4
# sum_if_true even 2 4;;
- : int = 6
```

Обратите внимание, что в определении функции `even` мы дважды использовали знак равно (`=`): первый из них является частью `let`-привязки и отделяет определяемый объект от самого определения, а второй выполняет проверку на равенство, сравнивая результат выражения `x mod 2` с нулем. Это совершенно разные операции, несмотря на то что в коде они выглядят одинаково.

Автоматический вывод типов

По мере усложнения типов, которые будут встречаться нам на пути, у многих из вас появится вопрос: «Как OCaml распознает их, учитывая, что мы никак явно не обозначаем типы?»

OCaml определяет тип выражения, используя прием под названием *автоматический вывод типа* (type inference), когда тип выражения определяется (выводится) по имеющейся информации о типах компонентов, составляющих выражение.

Для примера рассмотрим процесс вывода типа функции `sum_if_true`:

1. В языке OCaml требуется, чтобы обе ветви инструкции `if` имели одинаковый тип, соответственно для выражения `if test first then first else 0` требуется, чтобы значение `first` имело тот же тип, что и число `0`, то есть значение `first` должно иметь тип `int`. Аналогично из выражения `if test second then second else 0` можно заключить, что значение `second` имеет тип `int`.
2. Функции `test` передается значение `first` в качестве аргумента. Так как `first` имеет тип `int`, тип входного аргумента функции `test` также должен иметь тип `int`.
3. `test first` используется как условие в инструкции `if`, соответственно, `test` должна возвращать значение типа `bool`.

4. Из того факта, что `+` возвращает значение типа `int`, следует, что `sum_if_true` так же должна возвращать значение типа `int`.

Эта цепочка рассуждений закрепляет типы за всеми переменными и определяет общий тип функции `sum_if_true`.

Со временем вы получите более полное понимание, как действует механизм автоматического вывода типов в OCaml, что позволит вам проще тянуть нить рассуждений через свои программы. Впрочем, вы можете упростить себе эту задачу, добавляя явные аннотации типов. Эти аннотации не влияют на поведение программ на языке OCaml, но они могут играть роль дополнительного документирующего фактора, а также отлавливать непредвиденные изменения типов. Они могут также пригодиться в выяснении причин, вызывающих ошибки компиляции.

Ниже приводится аннотированная версия функции `sum_if_true`:

OCaml utop (part 9)

<https://github.com/realworldocaml/examples/blob/v1/code/guided-tour/main.topscript>

```
# let sum_if_true (test : int -> bool) (x : int) (y : int) : int =
  (if test x then x else 0)
  + (if test y then y else 0)
;;
val sum_if_true : (int -> bool) -> int -> int -> int = <fun>
```

Здесь каждый аргумент функции сопровождается аннотацией типа, а заключительная аннотация сообщает тип значения, возвращаемого функцией. Такие аннотации типов можно включать в любые выражения на языке OCaml.

Автоматический вывод обобщенных типов

Иногда информации оказывается недостаточно, чтобы точно определить конкретный тип некоторого значения. Взгляните на следующую функцию.

OCaml utop (part 10)

<https://github.com/realworldocaml/examples/blob/v1/code/guided-tour/main.topscript>

```
# let first_if_true test x y =
  if test x then x else y
;;
val first_if_true : ('a -> bool) -> 'a -> 'a -> 'a = <fun>
```

Функция `first_if_true` принимает в виде аргументов функцию `test` и два значения, `x` и `y`. Если условие `if test x` выполняется, она возвращает `x`, иначе возвращается `y`. Сможете ли вы определить тип `first_if_true`? Здесь нет никаких очевидных подсказок, таких как арифметические операторы или литералы, на основе которых можно было бы вывести типы `x` и `y`. Исходя из такого объявления, можно заключить, что функция `first_if_true` способна принимать значения любых типов.

И действительно, если взглянуть на тип, который вернула интерактивная оболочка, можно увидеть, что вместо конкретного типа OCaml ввел *переменный тип* (type variable) `'a`, указывающий, что выражение имеет обобщенный тип. (Тот факт, что это переменный тип, отмечает начальная одиночная кавычка.) В частно-

сти, аргумент `test` имеет тип `('a -> bool)`, означающий, что `test` является функцией с одним аргументом, возвращающей значение типа `bool` и принимающей аргумент любого типа `'a`. Но независимо от того, каким будет тип `'a`, он должен совпадать с типами двух других аргументов, `x` и `y`, а также с типом возвращаемого значения функции `first_if_true`. Такого рода обобщенность называется *параметрическим полиморфизмом* (parametric polymorphism), потому что суть заключается в параметризации типа с помощью переменной типа. Это очень похоже на обобщенные типы (generics) в C# и Java.

Обобщенность функции `first_if_true` позволяет писать, к примеру, такой код:

OCaml utop (part 11)

<https://github.com/realworldocaml/examples/blob/v1/code/guided-tour/main.topscript>

```
# let long_string s = String.length s > 6;;
val long_string : string -> bool = <fun>
# first_if_true long_string "short" "loooooong";;
- : string = "loooooong"
```

Или такой:

OCaml utop (part 12)

<https://github.com/realworldocaml/examples/blob/v1/code/guided-tour/main.topscript>

```
# let big_number x = x > 3;;
val big_number : int -> bool = <fun>
# first_if_true big_number 4 3;;
- : int = 4
```

Здесь `long_string` и `big_number` – это функции, и каждая из них пересылается в вызов функции `first_if_true` вместе с двумя аргументами соответствующего типа (строки в первом примере и целые числа во втором). Но мы не сможем смешать два разных конкретных типа для `'a` в одном вызове `first_if_true`:

OCaml utop (part 13)

<https://github.com/realworldocaml/examples/blob/v1/code/guided-tour/main.topscript>

```
# first_if_true big_number "short" "loooooong";;
```

Characters 25-32:

```
Error: This expression has type string but an expression was expected of type
      int
(Ошибка: Это выражение имеет тип string, тогда как ожидалось выражение типа
      int)
```

В данном примере функция `big_number` требует, чтобы параметр `'a` конкретизировался как `int`, тогда как аргументы `"short"` и `"loooooong"` требуют, чтобы параметр `'a` конкретизировался как `string`, что невозможно получить одновременно.

Ошибки и исключения

В OCaml (как и в любом другом компилирующем языке) ошибки времени компиляции и времени выполнения имеют существенные отличия. Как показывает практика разработки программного обеспечения, чем раньше обнаруживается ошибка в процессе разработки, тем лучше, соответственно, этап компиляции является самым лучшим в этом смысле.

Работа в интерактивной оболочке иногда делает малозаметной грань между ошибками времени компиляции и времени выполнения, но в действительности она никуда не исчезает. Обычно сообщения, указывающие на ошибку в типе, например такое:

OCaml utop (part 14)

<https://github.com/realworldocaml/examples/blob/v1/code/guided-tour/main.topscrip>

```
# let add_potato x =  
  x + "potato";;
```

Characters 28-36:

```
Error: This expression has type string but an expression was expected of type  
      int  
(Ошибка: Это выражение имеет тип string, тогда как ожидалось выражение типа  
      int)
```

являются свидетельством ошибок времени компиляции (потому что `+` требует, чтобы оба аргумента имели тип `int`). Ошибки, которые не могут быть выявлены системой типов, такие как деление на ноль, вызывают исключения во время выполнения:

OCaml utop (part 15)

<https://github.com/realworldocaml/examples/blob/v1/code/guided-tour/main.topscrip>

```
# let is_a_multiple x y =  
  x mod y = 0 ;;  
val is_a_multiple : int -> int -> bool = <fun>  
# is_a_multiple 8 2;;  
- : bool = true  
# is_a_multiple 8 0;;  
Exception: Division_by_zero.
```

Разница в том, что ошибки в типах не позволят даже запустить программу. В первом случае само определение `add_potato` содержит ошибку, во втором – функция `is_a_multiple` терпит неудачу только после вызова и только когда ей передаются аргументы, вызывающие исключение.

Кортежи, списки, необязательные значения и сопоставление с образцом

Кортежи

К настоящему моменту мы познакомились с простыми типами данных, такими как `int`, `float` и `string`, а также с типами функций, такими как `string -> int`. Но мы пока ничего не говорили о структурах данных. Для начала мы познакомимся с особенно простой структурой – кортежем. Кортеж – это упорядоченная коллекция значений, причем в одной коллекции могут находиться значения разных типов. Создать кортеж можно простым перечислением значений через запятую:

OCaml utop (part 16)

<https://github.com/realworldocaml/examples/blob/v1/code/guided-tour/main.topscrip>

```
# let a_tuple = (3, "three");;  
val a_tuple : int * string = (3, "three")  
# let another_tuple = (3, "four", 5.);;  
val another_tuple : int * string * float = (3, "four", 5.)
```

(Для читателей с математической подготовкой отметим, что символ $*$ используется для обозначения множества всех пар типов $t * s$ и соответствует декартову произведению множества элементов типа t и множества элементов типа s .)

Извлекать элементы кортежей в языке OCaml можно с помощью сопоставления с образцом, как показано ниже:

OCaml utop (part 17)

<https://github.com/realworldocaml/examples/blob/v1/code/guided-tour/main.topscript>

```
# let (x,y) = a_tuple;;
val x : int = 3
val y : string = "three"
```

Здесь конструкция (x,y) в левой части выражения let-привязки представляет образец. Этот образец создает новые переменные x и y и связывает каждую из них с соответствующим компонентом сопоставляемого значения. После этого вновь созданные переменные могут использоваться в последующих выражениях:

OCaml utop (part 18)

<https://github.com/realworldocaml/examples/blob/v1/code/guided-tour/main.topscript>

```
# x + String.length y;;
- : int = 8
```

Обратите внимание, что для конструирования кортежей и сопоставления кортежей с образцом используется один и тот же синтаксис.

Сопоставление с образцом можно также заметить в определениях аргументов функций. Следующая функция вычисляет расстояние между двумя точками на плоскости, где каждая точка представлена парой вещественных чисел. Синтаксис сопоставления с образцом дает возможность извлекать необходимые значения с минимумом усилий:

OCaml utop (part 19)

<https://github.com/realworldocaml/examples/blob/v1/code/guided-tour/main.topscript>

```
# let distance (x1,y1) (x2,y2) =
  sqrt ((x1 -. x2) ** 2. +. (y1 -. y2) ** 2.)
;;
val distance : float * float -> float * float -> float = <fun>
```

Оператор $**$ в примере выше выполняет возведение вещественных чисел в степень.

Это был лишь первый пример использования сопоставления с образцом, которое является одним из самых часто используемых инструментов в языке OCaml, и, как будет показано дальше, оно обладает удивительной широтой возможностей.

Списки

В отличие от кортежей, позволяющих составлять коллекции из фиксированного числа элементов, возможно разных типов, списки могут содержать произвольное число элементов одного типа. Взгляните на следующий пример:

OCaml utop (part 20)

<https://github.com/realworldocaml/examples/blob/v1/code/guided-tour/main.topscript>

```
# let languages = ["OCaml"; "Perl"; "C"];;
val languages : string list = ["OCaml"; "Perl"; "C"]
```

Отметьте, что, в отличие от кортежей, списки не могут содержать элементов разных типов:

OCaml utop (part 21)

<https://github.com/realworldocaml/examples/blob/v1/code/guided-tour/main.topscript>

```
# let numbers = [3;"four";5];;
Characters 17-23:
Error: This expression has type string but an expression was expected of type
      int
(Ошибка: Это выражение имеет тип string, тогда как ожидалось выражение типа
      int)
```

Модуль List

В состав библиотеки Core входит модуль List, содержащий богатую коллекцию функций для работы со списками. Обращаться к компонентам модулей можно с помощью точечной нотации. Например, ниже показано, как определить длину списка:

OCaml utop (part 22)

<https://github.com/realworldocaml/examples/blob/v1/code/guided-tour/main.topscript>

```
# List.length languages;;
- : int = 3
```

Следующий пример немного сложнее. В нем определяются длины всех слов в списке languages:

OCaml utop (part 23)

<https://github.com/realworldocaml/examples/blob/v1/code/guided-tour/main.topscript>

```
# List.map languages ~f:String.length;;
- : int list = [5; 4; 1]
```

Функция List.map принимает два аргумента: список и функцию преобразования элементов этого списка. Она возвращает новый список с преобразованными элементами, не изменяя при этом оригинального списка.

Особенно отметьте, что функция, которая передается в вызов List.map, передается как аргумент с меткой (labeled argument) ~f. Аргументы с метками определяются их именами (метками), а не позициями в списке аргументов, что дает возможность менять порядок следования аргументов в вызовах функций, не влияя на их поведение:

OCaml utop (part 24)

<https://github.com/realworldocaml/examples/blob/v1/code/guided-tour/main.topscript>

```
# List.map ~f:String.length languages;;
- : int list = [5; 4; 1]
```

Более подробно об аргументах с метками и об их важности будет рассказываться в главе 2.

Конструирование списков с помощью ::

Помимо квадратных скобок, для создания списков можно также использовать оператор `::`, добавляющий новый элемент в начало списка:

OCaml utop (part 25)

<https://github.com/realworldocaml/examples/blob/v1/code/guided-tour/main.topscript>

```
# "French" :: "Spanish" :: languages;;
- : string list = ["French"; "Spanish"; "OCaml"; "Perl"; "C"]
```

Здесь мы создали новый, расширенный список, не изменив при этом первоначального списка, в чем легко убедиться:

OCaml utop (part 26)

<https://github.com/realworldocaml/examples/blob/v1/code/guided-tour/main.topscript>

```
# languages;;
- : string list = ["OCaml"; "Perl"; "C"]
```

Точки с запятой и запятые

В отличие от многих других языков, в OCaml для разделения элементов в списках используется не запятая, а точка с запятой. Запятые используются для разделения элементов в кортежах. Если попытаться использовать запятую в литерале списка, код скомпилируется, но результат получится совсем не тот, что вы могли бы ожидать:

OCaml utop (part 27)

<https://github.com/realworldocaml/examples/blob/v1/code/guided-tour/main.topscript>

```
# ["OCaml", "Perl", "C"];;
- : (string * string * string) list = [("OCaml", "Perl", "C")]
```

В данном случае вместо списка с тремя строками получился одноэлементный список, содержащий кортеж с тремя строками.

Этот пример доказывает, что запятые создают кортеж даже в отсутствие окружающих круглых скобок. То есть кортеж целых чисел, например, можно записать так:

OCaml utop (part 28)

<https://github.com/realworldocaml/examples/blob/v1/code/guided-tour/main.topscript>

```
# 1,2,3;;
- : int * int * int = (1, 2, 3)
```

Вообще говоря, такой стиль не приветствуется, и его следует избегать.

Форма записи списков в квадратных скобках, по сути, является всего лишь синтаксическим сахаром для оператора `::`. То есть все следующие объявления совершенно эквивалентны:

OCaml utop (part 29)

<https://github.com/realworldocaml/examples/blob/v1/code/guided-tour/main.topscript>

```
# [1; 2; 3];;
- : int list = [1; 2; 3]
```

```
# 1 :: (2 :: (3 :: []));;
- : int list = [1; 2; 3]
# 1 :: 2 :: 3 :: [];;
- : int list = [1; 2; 3]
```

Обратите внимание, что для представления пустого списка используется пара скобок [], а оператор :: является правоассоциативным.

С помощью оператора :: можно добавить только один элемент и только в начало списка, при этом, чтобы создать новый список с нуля, определение должно завершаться пустым списком [].

Существует также оператор конкатенации списков @, объединяющий два списка:

OCaml utop (part 30)

<https://github.com/realworldocaml/examples/blob/v1/code/guided-tour/main.topscript>

```
# [1;2;3] @ [4;5;6];;
- : int list = [1; 2; 3; 4; 5; 6]
```

Важно помнить, что, в отличие от оператора ::, этот оператор имеет непостоянное время выполнения. Время выполнения операции объединения двух списков пропорционально длине первого списка.

Образцы списков в операциях сопоставления

Извлекать элементы списков в языке OCaml можно с помощью сопоставления с образцом. Образцы списков формируются на основе двух конструкторов, [] и ::, например:

OCaml utop (part 31)

<https://github.com/realworldocaml/examples/blob/v1/code/guided-tour/main.topscript>

```
# let my_favorite_language (my_favorite :: the_rest) =
    my_favorite
;;
```

Characters 25-69:

Warning 8: this pattern-matching is not exhaustive.

Here is an example of a value that is not matched:

```
[]
```

(Предупреждение 8: это сопоставление не является исчерпывающим.

Ниже следует пример значения, не соответствующего сопоставлению:

```
[])
```

```
val my_favorite_language : 'a list -> 'a = <fun>
```

Организовав сопоставление с применением ::, мы выделили и дали имя первому элементу списка (my_favorite) и остатку списка (the_rest). Знакомые с языком Lisp или Scheme заметят в этом сходство с функциями car и cdr, которые используются для разбиения списков на первый элемент и остаток («голову» и «хвост»).

Однако, как показано в примере, интерактивной оболочке не понравилось такое определение, и она тут же вывела предупреждение о том, что образец не явля-

ется исчерпывающим. Это означает, что существуют значения рассматриваемого типа, не соответствующие образцу. В тексте предупреждения даже приводится пример такого значения: [], пустой список. Если попробовать вызывать функцию `my_favorite_language`, можно заметить, что она замечательно работает с непустыми списками, но терпит неудачу, если передать ей пустой список:

OCaml utop (part 32)

<https://github.com/realworldocaml/examples/blob/v1/code/guided-tour/main.topscript>

```
# my_favorite_language ["English";"Spanish";"French"];;
- : string = "English"
# my_favorite_language [];;
Exception: (Match_failure //toplevel// 0 25).
```

Избегать подобных предупреждений и, что особенно важно, гарантировать безошибочную работу кода во всех возможных ситуациях можно с помощью инструкции `match`.

Можно сказать, что инструкция `match` является усовершенствованной версией инструкции `switch` в языках C и Java. По сути, она дает возможность перечислить образцы через символ вертикальной черты (`|`). (Причем первый символ `|` перед первым образцом можно опустить.) Встретив такую инструкцию, компилятор направит поток выполнения к первому совпавшему образцу. Как мы уже видели, образец может создать сразу несколько переменных, соответствующих подструктурам в сопоставляемом значении.

Ниже приводится новая версия `my_favorite_language`, использующая сопоставление и не вызывающая предупреждений компилятора:

OCaml utop (part 33)

<https://github.com/realworldocaml/examples/blob/v1/code/guided-tour/main.topscript>

```
# let my_favorite_language languages =
  match languages with
  | first :: the_rest -> first
  | [] -> "OCaml" (* Отличное значение по умолчанию! *)
;;
val my_favorite_language : string list -> string = <fun>
# my_favorite_language ["English";"Spanish";"French"];;
- : string = "English"
# my_favorite_language [];;
- : string = "OCaml"
```

Этот пример также содержит наш первый комментарий. Комментарии в языке OCaml заключаются в пары символов (`* и *`), могут вкладываться друг в друга произвольным образом и охватывать несколько строк. В OCaml нет эквивалента однострочных комментариев в стиле C++, которые начинаются с `//`.

Первый образец `first :: the_rest` охватывает случай, когда список `languages` содержит хотя бы один элемент, так как любой список, кроме пустого, можно записать с помощью одного или более операторов `::`. Второй образец `[]` соответствует только пустому списку. В комплексе эти два образца являются исчерпывающими,

потому что каждый список является либо пустым, либо содержит хотя бы один элемент.

Рекурсивные функции для списков

Рекурсивные функции, или функции, вызывающие самих себя, занимают важное место в OCaml и в любом другом функциональном языке. Типичный подход к созданию рекурсивной функции заключается в том, чтобы разделить логику на множество *базовых случаев*, которые могут быть вычислены непосредственно, и множество *индуктивных случаев*, когда функция разбивает решение задачи на более мелкие фрагменты и затем вызывает себя для решения этих более мелких фрагментов задачи.

При создании рекурсивных функций для обработки списков разделение на базовые и индуктивные случаи часто выполняется с помощью сопоставления с образцом. Ниже приводится пример простой функции, вычисляющей сумму элементов списка:

OCaml utop (part 34)

<https://github.com/realworldocaml/examples/blob/v1/code/guided-tour/main.topscript>

```
# let rec sum l =
  match l with
  | [] -> 0 (* базовый случай *)
  | hd :: tl -> hd + sum tl (* индуктивный случай *)
;;
val sum : int list -> int = <fun>
# sum [1;2;3];;
- : int = 6
```

Следуя типичной идиоме языка OCaml, мы использовали `hd` для ссылки на голову списка и `tl` — для ссылки на хвост списка. Обратите внимание: чтобы функция могла вызывать саму себя, нам пришлось использовать ключевое слово `rec`. Как видите, базовый и индуктивный случаи являются разными ветвями инструкции сопоставления.

Логически вычисление простой рекурсивной функции, такой как `sum`, близко напоминает математическое уравнение, для нахождения значения которого требуется выполнить поэтапное его развертывание:

OCaml: guided-tour/recursion.ml

<https://github.com/realworldocaml/examples/blob/v1/code/guided-tour/recursion.ml>

```
sum [1;2;3]
= 1 + sum [2;3]
= 1 + (2 + sum [3])
= 1 + (2 + (3 + sum []))
= 1 + (2 + (3 + 0))
= 1 + (2 + 3)
= 1 + 5
= 6
```

Это достаточно точная модель происходящего, когда OCaml выполняет рекурсивную функцию.

Можно также использовать более сложные образцы списков. Например, ниже приводится функция, удаляющая повторяющиеся элементы, следующие друг за другом:

OCaml utop (part 35)

<https://github.com/realworldocaml/examples/blob/v1/code/guided-tour/main.topscript>

```
# let rec destutter list =
  match list with
  | [] -> []
  | hd1 :: hd2 :: t1 ->
    if hd1 = hd2 then destutter (hd2 :: t1)
    else hd1 :: destutter (hd2 :: t1)

;;
```

Characters 29-171:

Warning 8: this pattern-matching is not exhaustive.

Here is an example of a value that is not matched:

```
_::[]
```

(Предупреждение 8: это сопоставление не является исчерпывающим.

Ниже следует пример значения, не соответствующего сопоставлению:

```
_::[])
```

```
val destutter : 'a list -> 'a list = <fun>
```

И снова первая ветвь сопоставления является базовым случаем, а вторая – индуктивным. К сожалению, в этом коде имеется проблема, о чем свидетельствует предупреждение. В частности, здесь не предусмотрена обработка списка с единственным элементом. Исправить эту проблему можно, добавив еще один образец в сопоставление:

OCaml utop (part 36)

<https://github.com/realworldocaml/examples/blob/v1/code/guided-tour/main.topscript>

```
# let rec destutter list =
  match list with
  | [] -> []
  | [hd] -> [hd]
  | hd1 :: hd2 :: t1 ->
    if hd1 = hd2 then destutter (hd2 :: t1)
    else hd1 :: destutter (hd2 :: t1)

;;
val destutter : 'a list -> 'a list = <fun>
# destutter ["hey"; "hey"; "hey"; "man!"];;
- : string list = ["hey"; "man!"]
```

Обратите внимание, что в этой реализации используется еще один образец для сопоставления со списком `[hd]`, которому соответствует список с единственным элементом. Мы можем использовать этот прием для сопоставления со списками, имеющими фиксированное число элементов. Например, образцу `[x; y; z]` будет соответствовать любой список, содержащий точно три элемента и связывающий эти элементы с переменными `x`, `y` и `z`.

В последних нескольких примерах использовалось множество рекурсивных функций. На практике это не является чем-то необходимым. Чаще вы будете ис-

пользовать итеративные функции из модуля `List`. Но совсем нелишним будет знать, как использовать прием рекурсии, если возникнет такая необходимость.

Необязательные значения

Еще одну часто используемую структуру данных в языке OCaml представляет тип `option`. Он используется для выражения того факта, что значение может отсутствовать. Например:

OCaml utop (part 37)

<https://github.com/realworldocaml/examples/blob/v1/code/guided-tour/main.topscript>

```
# let divide x y =
  if y = 0 then None else Some (x/y) ;;
val divide : int -> int -> int option = <fun>
```

Функция `divide` может вернуть `None`, если делитель окажется равен нулю, или `Some` с результатом в противном случае. `Some` и `None` — это конструкторы, позволяющие конструировать необязательные значения, подобно тому, как `::` и `[]` позволяют конструировать списки. Значение типа `option` можно воспринимать как специальный список, способный содержать только нуль или один элемент.

Извлечь содержимое из необязательного значения можно с помощью сопоставления с образцом, по аналогии с кортежами и списками. Взгляните на следующую функцию, создающую запись в журнале на основе необязательного времени и строки с текстом сообщения. Если время не указано (то есть если `time` имеет значение `None`), функция вычисляет текущее время и использует его:

OCaml utop (part 38)

<https://github.com/realworldocaml/examples/blob/v1/code/guided-tour/main.topscript>

```
# let log_entry maybe_time message =
  let time =
    match maybe_time with
    | Some x -> x
    | None -> Time.now ()
  in
  Time.to_sec_string time ^ " -- " ^ message
;;
val log_entry : Time.t option -> string -> string = <fun>
# log_entry (Some Time.epoch) "A long long time ago";;
- : string = "1970-01-01 01:00:00 -- A long long time ago"
# log_entry None "Up to the minute";;
- : string = "2013-08-18 14:48:08 -- Up to the minute"
```

Для работы со значениями времени этот пример использует модуль `Time` из библиотеки `Core` и оператор `^`, выполняющий конкатенацию строк. Оператор конкатенации определяется в модуле `Pervasives`, который автоматически подключается любой программой на языке OCaml.

Вложение выражений связывания с помощью `let` и `in`

Функция `log_entry` стала первой в этой книге, использующей `let` для определения новой переменной в теле функции. Инструкцию `let` в паре с `in` можно использовать для созда-

ния новой привязки внутри любой локальной области видимости, в том числе и в теле функции. Инструкция `in` отмечает начало области видимости, где может использоваться новая переменная. То есть мы могли бы написать:

OCaml utoп

https://github.com/realworldocaml/examples/blob/v1/code/guided-tour/local_let.topscript

```
# let x = 7 in
  x + x
;;
- : int = 14
```

Отметьте, что область видимости `let`-привязки ограничивается двумя точками с запятой, поэтому за ними значение `x` больше недоступно:

OCaml utoп (part 1)

https://github.com/realworldocaml/examples/blob/v1/code/guided-tour/local_let.topscript

```
# x;;
Characters -1-1:
Error: Unbound value x
(Ошибка: Несвязанное значение x)
```

Допускается вкладывать друг в друга множество инструкций `let`, каждая из которых добавляет новую связанную переменную к предыдущей:

OCaml utoп (part 2)

https://github.com/realworldocaml/examples/blob/v1/code/guided-tour/local_let.topscript

```
# let x = 7 in
  let y = x * x in
    x + y
;;
- : int = 56
```

Такой прием вложения `let`-привязок часто используется для построения сложных выражений, где каждый компонент определяется отдельно, перед их объединением в конечное выражение.

Необязательные значения играют важную роль, потому что обеспечивают стандартный способ выражения значения, которого может не быть. В OCaml нет такой вещи, как исключение `NullPointerException`. Этим он отличается от многих других языков, включая Java и C#, где большинство (если не все) типов данных *могут иметь пустое значение*, то есть независимо от типа любая конкретная переменная в этих языках может содержать пустое значение `null`. В таких языках пустой указатель (`null`) может прятаться где угодно.

Однако в OCaml отсутствующие значения являются явными. Значение типа `string * string` всегда будет содержать два действительных значения типа `string`. Если необходимо, например, допустить возможность отсутствия первого значения в этой паре, тип необходимо изменить на `string option * string`. Как будет показано в главе 7, такая явность позволяет компилятору дополнительно убедиться, что вы правильно обрабатываете возможность отсутствия данных.

Записи и варианты

Пока что мы рассматривали структуры данных, такие как списки и кортежи, предопределенные в самом языке. Но OCaml позволяет также определять новые типы данных. Ниже приводится игрушечный пример типа данных, представляющего точку в двумерном пространстве:

OCaml utop (part 41)

<https://github.com/realworldocaml/examples/blob/v1/code/guided-tour/main.topscript>

```
# type point2d = { x : float; y : float };;
type point2d = { x : float; y : float; }
```

`point2d` – это тип записи, который можно рассматривать как кортеж с именованными элементами. Записи конструируются очень просто:

OCaml utop (part 42)

<https://github.com/realworldocaml/examples/blob/v1/code/guided-tour/main.topscript>

```
# let p = { x = 3.; y = -4. };;
val p : point2d = {x = 3.; y = -4.}
```

Извлекать содержимое из данных этого типа можно с помощью сопоставления с образцом:

OCaml utop (part 43)

<https://github.com/realworldocaml/examples/blob/v1/code/guided-tour/main.topscript>

```
# let magnitude { x = x_pos; y = y_pos } =
  sqrt (x_pos ** 2. +. y_pos ** 2.);;
val magnitude : point2d -> float = <fun>
```

Сопоставление с образцом здесь связывает переменную `x_pos` со значением в поле `x`, а переменную `y_pos` – со значением в поле `y`.

Этот код можно сократить, воспользовавшись приемом *уплотнения полей* (field punning). В частности, если имя поля и имя связываемой с ним переменной совпадают, имя поля можно опустить. То есть нашу функцию `magnitude` можно переписать так:

OCaml utop (part 44)

<https://github.com/realworldocaml/examples/blob/v1/code/guided-tour/main.topscript>

```
# let magnitude { x; y } = sqrt (x ** 2. +. y ** 2.);;
val magnitude : point2d -> float = <fun>
```

Для обращения к полям записей можно также использовать точечную нотацию:

OCaml utop (part 45)

<https://github.com/realworldocaml/examples/blob/v1/code/guided-tour/main.topscript>

```
# let distance v1 v2 =
  magnitude { x = v1.x -. v2.x; y = v1.y -. v2.y };;
val distance : point2d -> point2d -> float = <fun>
```

И, конечно же, новые типы можно использовать в определениях более крупных типов. Ниже приводятся некоторые типы, моделирующие различные геометрические объекты, содержащие значения типа `point2d`:

OCaml utop (part 46)

<https://github.com/realworldocaml/examples/blob/v1/code/guided-tour/main.topscript>

```
# type circle_desc = { center: point2d; radius: float }
  type rect_desc   = { lower_left: point2d; width: float; height: float }
  type segment_desc = { endpoint1: point2d; endpoint2: point2d } ;;

type circle_desc = { center : point2d; radius : float; }
type rect_desc   = { lower_left : point2d; width : float; height : float; }
type segment_desc = { endpoint1 : point2d; endpoint2 : point2d; }
```

Теперь представьте, что вам захотелось объединить несколько объектов этих типов вместе, чтобы описать сцену с несколькими объектами. Для этого необходим некоторый универсальный способ представления этих разнотипных объектов в виде единственного типа. Один из таких способов заключается в использовании *вариантного типа* (variant type):

OCaml utop (part 47)

<https://github.com/realworldocaml/examples/blob/v1/code/guided-tour/main.topscript>

```
# type scene_element =
  | Circle of circle_desc
  | Rect of rect_desc
  | Segment of segment_desc
;;

type scene_element =
  Circle of circle_desc
  | Rect of rect_desc
  | Segment of segment_desc
```

Символ `|` разделяет разные случаи вариантного значения (первый символ `|` можно опустить), а каждый случай начинается с имени, в котором первая буква – заглавная, например `Circle`, `Rect` или `Segment`, чтобы их можно было отличать друг от друга.

Ниже показано, как можно реализовать функцию, определяющую, находится ли заданная точка в пределах некоторого элемента списка `scene_elements`:

OCaml utop (part 48)

<https://github.com/realworldocaml/examples/blob/v1/code/guided-tour/main.topscript>

```
# let is_inside_scene_element point scene_element =
  match scene_element with
  | Circle { center; radius } ->
    distance center point < radius
  | Rect { lower_left; width; height } ->
    point.x > lower_left.x && point.x < lower_left.x +. width
    && point.y > lower_left.y && point.y < lower_left.y +. height
  | Segment { endpoint1; endpoint2 } -> false
;;

val is_inside_scene_element : point2d -> scene_element -> bool = <fun>
# let is_inside_scene point scene =
```

```

List.exists scene
  ~f:(fun el -> is_inside_scene_element point el)
;;
val is_inside_scene : point2d -> scene_element list -> bool = <fun>
# is_inside_scene {x=3.;y=7.}
  [ Circle {center = {x=4.;y= 4.}; radius = 0.5 } ];;
- : bool = false
# is_inside_scene {x=3.;y=7.}
  [ Circle {center = {x=4.;y= 4.}; radius = 5.0 } ];;
- : bool = true

```

Вы наверняка обратили внимание, что инструкция `match` в этом примере напоминает инструкцию `match` с необязательными значениями и списками. И это не случайно: необязательные значения (типа `option`) и списки в действительности являются частными случаями вариантных типов и достаточно важными, чтобы их определить отдельно в стандартной библиотеке (а в случае списков даже добавить специальный синтаксис).

В этом примере мы также впервые использовали *анонимную функцию* в вызове `List.exists`. Анонимные функции объявляются с помощью ключевого слова `fun` и не нуждаются в явном имени. Такие функции часто используются в языке OCaml, особенно в вызовах итеративных функций, таких как `List.exists`.

Цель функции `List.exists` состоит в том, чтобы проверить наличие в рассматриваемом списке элементов, для которых указанная функция возвращает `true`. В данном случае мы применили `List.exists`, чтобы выявить элемент, внутри которого находится точка с указанными координатами.

Императивное программирование

Код, который мы писали до сих пор, является почти *исключительно функциональным*. Это, грубо говоря, означает, что код не изменяет переменных или значений в процессе выполнения. В действительности почти все структуры данных, встречавшиеся нам, относятся к разряду *неизменяемых* (*immutable*), в том смысле, что в языке отсутствуют инструменты, которые позволяли бы изменять их. Такой стиль в корне отличается от императивного стиля программирования, когда вычисления строятся как последовательность инструкций, изменяющих состояние программы.

Функциональный стиль является основным при программировании на языке OCaml. При использовании этого стиля переменные и большинство структур данных остаются неизменными. Но OCaml поддерживает и императивный стиль программирования, позволяя изменять структуры данных, такие как массивы и хэш-таблицы, и контролировать поток выполнения с помощью управляющих конструкций, таких как циклы `for` и `while`.

Массивы

Самой простой изменяемой структурой данных в OCaml являются, пожалуй, массивы. Массивы в OCaml близко напоминают массивы в других языках, таких как C: индексы начинаются с 0, а операции доступа или изменения элементов массива

имеют постоянное время выполнения. Массивы более компактны в терминах занимаемого объема памяти, чем большинство других структур данных в OCaml, включая и списки. Например:

OCaml utop (part 49)

<https://github.com/realworldocaml/examples/blob/v1/code/guided-tour/main.topscript>

```
# let numbers = [| 1; 2; 3; 4 |];;
val numbers : int array = [|1; 2; 3; 4|]
# numbers.(2) <- 4;;
- : unit = ()
# numbers;;
- : int array = [|1; 2; 4; 4|]
```

Для ссылки на элемент массива используется синтаксис `.(i)`, а для его изменения используется синтаксис `<-`. Так как отсчет элементов массивов начинается с нуля, элемент `.(2)` – это третий элемент массива.

Тип `unit`, который мы видели в предыдущем фрагменте, интересен тем, что имеет только одно возможное значение, записываемое как `()`. Это означает, что значения типа `unit` вообще не несут никакой информации и обычно используются в качестве заполнителей. То есть тип `unit` используется, чтобы вернуть значение операции, такой как установка изменяемого поля, и является скорее побочным эффектом, чем возвращаемым значением. Он также используется для передачи функциям аргументов, для которых не требуется значение. Это напоминает роль, которую играет тип `void` в языках C и Java.

Изменяемые поля записей

Массив – важная изменяемая структура данных, но не единственная. Записи, неизменяемые по умолчанию, могут содержать поля, явно объявленные изменяемыми. Ниже приводится маленький пример структуры данных для накопления статистической информации о коллекции чисел.

OCaml utop (part 50)

<https://github.com/realworldocaml/examples/blob/v1/code/guided-tour/main.topscript>

```
# type running_sum =
  { mutable sum: float;
    mutable sum_sq: float; (* сумма квадратов *)
    mutable samples: int;
  }
;;
type running_sum = {
  mutable sum : float;
  mutable sum_sq : float;
  mutable samples : int;
}
```

Поля в `running_sum` объявлены так, что их легко можно изменять, накапливая в них информацию. На их основе можно вычислить среднее и стандартное отклонение, как показано в следующем примере. Обратите внимание, что там исполь-

зуются две `let`-привязки без двух точек с запятой между ними. Это обусловлено тем, что две точки с запятой требуются, только чтобы сообщить интерактивной оболочке *utop*, что она может приступить к обработке ввода, но не для отделения двух объявлений:

OCaml utop (part 51)

<https://github.com/realworldocaml/examples/blob/v1/code/guided-tour/main.topscript>

```
# let mean rsum = rsum.sum /. float rsum.samples
  let stdev rsum =
    sqrt (rsum.sum_sq /. float rsum.samples
          -. (rsum.sum /. float rsum.samples) ** 2.) ;;
val mean : running_sum -> float = <fun>
val stdev : running_sum -> float = <fun>
```

Здесь мы использовали функцию `float`, которая является удобным эквивалентом функции `Float.of_int` из библиотеки `Pervasives`.

Нам также необходимы функции для создания и изменения `running_sums`:

OCaml utop (part 52)

<https://github.com/realworldocaml/examples/blob/v1/code/guided-tour/main.topscript>

```
# let create () = { sum = 0.; sum_sq = 0.; samples = 0 }
  let update rsum x =
    rsum.samples <- rsum.samples + 1;
    rsum.sum <- rsum.sum +. x;
    rsum.sum_sq <- rsum.sum_sq +. x *. x
  ;;
val create : unit -> running_sum = <fun>
val update : running_sum -> float -> unit = <fun>
```

Функция `create` возвращает запись `running_sum`, соответствующую пустому множеству, а `update rsum x` изменяет `rsum`, отражая добавление значения `x` во множество путем изменения числа элементов во множестве, суммы значений и суммы квадратов значений.

Обратите внимание, что операции в последовательности отделяются друг от друга одним символом точки с запятой. Когда мы использовали функциональный стиль, в этом не было необходимости, но требования изменились при переходе к императивному стилю.

Ниже приводится пример использования функций `create` и `update`. Отметим, что в этом примере применяется функция `List.iter`, которая вызывает функцию `~f` для каждого элемента указанного списка:

OCaml utop (part 53)

<https://github.com/realworldocaml/examples/blob/v1/code/guided-tour/main.topscript>

```
# let rsum = create ();;
val rsum : running_sum = {sum = 0.; sum_sq = 0.; samples = 0}
# List.iter [1.;3.;2.;-7.;4.;5.] ~f:(fun x -> update rsum x);;
- : unit = ()
# mean rsum;;
- : float = 1.3333333333333
# stdev rsum;;
- : float = 3.94405318873
```

Следует отметить, что предыдущий алгоритм слишком прост и имеет плохую точность из-за эффекта потери значимости. В статье по адресу http://en.wikipedia.org/wiki/Algorithms_for_calculating_variance можно найти дополнительную информацию об алгоритмах вычисления дисперсии, где особое внимание уделяется взвешенному инкрементальному и параллельному алгоритмам.

Ссылки

Одиночное изменяемое значение можно создать с помощью объявления `ref`. Тип `ref` определяется в стандартной библиотеке, но в нем нет ничего особенного. Это простой тип записи с единственным изменяемым полем `contents`:

OCaml utop (part 54)

<https://github.com/realworldocaml/examples/blob/v1/code/guided-tour/main.topscript>

```
# let x = { contents = 0 };;
val x : int ref = {contents = 0}
# x.contents <- x.contents + 1;;
- : unit = ()
# x;;
- : int ref = {contents = 1}
```

Существует несколько удобных функций и операторов для работы со ссылками:

OCaml utop (part 55)

<https://github.com/realworldocaml/examples/blob/v1/code/guided-tour/main.topscript>

```
# let x = ref 0 (* создает ссылку, то есть { contents = 0 } *) ;;
val x : int ref = {contents = 0}
# !x (* возвращает содержимое ссылки, то есть x.contents *) ;;
- : int = 0
# x := !x + 1 (* присваивание, то есть x.contents <- ... *) ;;
- : unit = ()
# !x ;;
- : int = 1
```

В этих операторах нет ничего таинственного. Вы можете реализовать ту же самую функциональность всего в нескольких строках кода:

OCaml utop (part 56)

<https://github.com/realworldocaml/examples/blob/v1/code/guided-tour/main.topscript>

```
# type 'a ref = { mutable contents : 'a }

  let ref x = { contents = x }
  let (!) r = r.contents
  let (:=) r x = r.contents <- x
  ;;
type 'a ref = { mutable contents : 'a; }
val ref : 'a -> 'a ref = <fun>
val ( ! ) : 'a ref -> 'a = <fun>
val ( := ) : 'a ref -> 'a -> unit = <fun>
```

Комбинация символов `'a` перед `ref` указывает, что тип `ref` является полиморфным по аналогии со списками, то есть может хранить значение любого типа.

Круглые скобки вокруг `!` и `:=` необходимы, так как это – операторы, а не обычные функции.

Даже при том, что `ref` является всего лишь типом записи, он играет важную роль, потому что реализует стандартный способ имитации поведения традиционных изменяемых переменных, поддерживаемых большинством других языков. Например, можно императивно просуммировать элементы списка, передав функции `List.iter` другую, простую функцию, которая будет вызвана для каждого элемента списка и сохранит сумму в ссылке:

OCaml utop (part 57)

<https://github.com/realworldocaml/examples/blob/v1/code/guided-tour/main.topscript>

```
# let sum list =
  let sum = ref 0 in
  List.iter list ~f:(fun x -> sum := !sum + x);
  !sum
;;
val sum : int list -> int = <fun>
```

Это не самый идиоматический способ суммирования элементов списка, но он показывает возможность использования ссылок вместо изменяемых переменных.

Циклы `for` и `while`

Язык OCaml поддерживает также традиционные императивные управляющие конструкции, такие как циклы `for` и `while`. Ниже приводится пример реализации случайной перестановки элементов массива с использованием цикла `for`. В качестве источника случайности мы использовали модуль `Random`. Модуль `Random` всегда начинает вычисления с одного и того же начального значения по умолчанию, но его можно изменить вызовом `Random.self_init` для установки другого начального значения:

OCaml utop (part 58)

<https://github.com/realworldocaml/examples/blob/v1/code/guided-tour/main.topscript>

```
# let permute array =
  let length = Array.length array in
  for i = 0 to length - 2 do
    (* выбрать j для перестановки *)
    let j = i + Random.int (length - i) in
    (* Поменять местами i и j *)
    let tmp = array.(i) in
    array.(i) <- array.(j);
    array.(j) <- tmp
  done
;;
val permute : 'a array -> unit = <fun>
```

С точки зрения синтаксиса, обратите внимание на ключевые слова, отличающие цикл `for`: `for`, `to`, `do` и `done`.

Ниже приводится пример использования этого кода:

OCaml utop (part 59)

<https://github.com/realworldocaml/examples/blob/v1/code/guided-tour/main.topscript>

```
# let ar = Array.init 20 ~f:(fun i -> i);;
val ar : int array =
  [|0; 1; 2; 3; 4; 5; 6; 7; 8; 9; 10; 11; 12; 13; 14; 15; 16; 17; 18; 19|]
# permute ar;;
- : unit = ()
# ar;;
- : int array =
  [|1; 2; 4; 6; 11; 7; 14; 9; 10; 0; 13; 16; 19; 12; 17; 5; 3; 18; 8; 15|]
```

Кроме того, OCaml поддерживает циклы `while`, как показано в определении следующей функции поиска первого отрицательного элемента в массиве. Отметим, что `while` (как и `for`) является ключевым словом:

OCaml utop (part 60)

<https://github.com/realworldocaml/examples/blob/v1/code/guided-tour/main.topscript>

```
# let find_first_negative_entry array =
  let pos = ref 0 in
  while !pos < Array.length array && array.(!pos) >= 0 do
    pos := !pos + 1
  done;
  if !pos = Array.length array then None else Some !pos
;;
val find_first_negative_entry : int array -> int option = <fun>
# find_first_negative_entry [|1;2;0;3|];;
- : int option = None
# find_first_negative_entry [|1;-2;0;3|];;
- : int option = Some 1
```

Заметим также, что предыдущий код использует тот факт, что оператор `&&` в OCaml (выполняет логическую операцию «И») вычисляется по короткой схеме. В частности, в выражении вида `expr1 && expr2` подвыражение `expr2` вычисляется, только если подвыражение `expr1` вернет `true`. Если бы это было не так, предыдущая функция генерировала бы ошибку выхода за границы массива. В действительности мы могли бы получить эту ошибку, переписав функцию так, чтобы исключить вычисление по короткой схеме:

OCaml utop (part 61)

<https://github.com/realworldocaml/examples/blob/v1/code/guided-tour/main.topscript>

```
# let find_first_negative_entry array =
  let pos = ref 0 in
  while
    let pos_is_good = !pos < Array.length array in
    let element_is_non_negative = array.(!pos) >= 0 in
    pos_is_good && element_is_non_negative
  do
    pos := !pos + 1
  done;
  if !pos = Array.length array then None else Some !pos
;;
```



```
val find_first_negative_entry : int array -> int option = <fun>
# find_first_negative_entry [|1;2;0;3|];;
Exception: (Invalid_argument "index out of bounds").
```

Оператор «ИЛИ», `||`, тоже выполняет вычисления по короткой схеме.

Законченная программа

До сих пор мы знакомились с самыми основными особенностями языка, используя для этого интерактивную оболочку *utop*. Теперь мы покажем вам, как создать простую самостоятельную программу. В данном разделе мы создадим программу, суммирующую числа из списка, прочитанного со стандартного ввода.

Сохраните следующий код в файле с именем *sum.ml*. Обратите внимание, что не нужно завершать выражения последовательностью `;;`, потому что этого не требуется за пределами интерактивной оболочки:

OCaml

<https://github.com/realworldocaml/examples/blob/v1/code/guided-tour/sum.ml>

```
open Core.Std

let rec read_and_accumulate accum =
  let line = In_channel.input_line In_channel.stdin in
  match line with
  | None -> accum
  | Some x -> read_and_accumulate (accum +. Float.of_string x)

let () =
  printf "Total: %F\n" (read_and_accumulate 0.)
```

Это наш первый опыт использования подпрограмм ввода/вывода в языке OCaml. Функция `read_and_accumulate` является рекурсивной и использует `In_channel.input_line` для чтения строк, одну за одной, со стандартного ввода, вызывая саму себя в каждой итерации и передавая накопленную сумму. Отметьте, что `input_line` возвращает необязательное значение, где `None` служит признаком конца входного потока.

После того как `read_and_accumulate` вернет накопленную сумму, ее необходимо вывести. Это делается с помощью команды `printf`, которая обеспечивает поддержку строк формата, подобных тем, что можно встретить во многих языках. Строка формата анализируется компилятором и используется для определения количества и типов остальных аргументов. В данном случае имеется всего одна директива форматирования, `%F`, поэтому `printf` ожидает получить только один дополнительный аргумент типа `float`.

Компиляция и запуск

Скомпилировать программу можно с помощью *corebuild*, небольшой обертки вокруг *ocamlbuild*, инструмента сборки, распространяемого вместе с компилятором OCaml. Сценарий *corebuild* устанавливается вместе с библиотекой *Core* и служит

для передачи компилятору флагов, необходимых для сборки программы вместе с библиотекой Core.

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/guided-tour/build_sum.out

```
$ corebuild sum.native
```

Расширение файла *.native* указывает, что мы создаем выполняемый файл с машинным кодом – подробнее об этом мы поговорим в главе 4. После окончания сборки можно запустить получившийся выполняемый файл, как любую другую утилиту командной строки. Мы можем передать программе *sum.native* исходные числа, вводя их по одному в строке и нажав **Ctrl-D** по окончании:

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/guided-tour/build_sum.out

```
$ ./sum.native
```

```
1
```

```
2
```

```
3
```

```
94.5
```

```
Total: 100.5
```

Чтобы создать более удобную утилиту командной строки, придется приложить дополнительные усилия и реализовать надлежащий интерфейс командной строки, а также предусмотреть обработку ошибок, но об этом мы поговорим в главе 14.

Что дальше

Вот и закончился ознакомительный тур! Впереди предстоит узнать еще очень многое, но мы надеемся, что вы составили себе некоторое мнение о языке OCaml и теперь будете чувствовать себя более комфортно.

Глава 2

Переменные и функции

Переменные и функции являются фундаментальными идеями практически во всех языках программирования. Однако в OCaml используется несколько иной подход к реализации этих понятий, чем в большинстве других языков, с которыми вы, вероятно, знакомы. Поэтому в данной главе мы поближе рассмотрим особенности функций и переменных в языке OCaml, начиная с самого простого – как определить переменную – и заканчивая такими сложными особенностями функций, как аргументы с метками и необязательные аргументы.

Не пугайтесь некоторых подробностей, особенно тех, что приводятся ближе к концу главы. Концепции, что описываются здесь, играют важную роль, но если что-то останется для вас непонятным после первого прочтения, просто вернитесь к этой главе, после того как получите более полное представление о других особенностях языка.

Переменные

В самом простом понимании переменная – это идентификатор, связанный с некоторым значением. В языке OCaml такие связи часто создаются с помощью ключевого слова `let`, в общем случае имеющего следующий синтаксис (обратите внимание, что имя переменной должно начинаться с буквы нижнего регистра или символа подчеркивания):

Syntax

<https://github.com/realworldocaml/examples/blob/v1/code/variables-and-functions/let.syntax>

```
let <variable> = <expr>
```

В главе 4, когда мы будем исследовать систему поддержки модулей, вы увидите, что этот же синтаксис используется для создания `let`-привязок на верхнем уровне модулей.

Каждая привязка к переменной имеет определенную *область видимости* – область программы, где можно сослаться на эту привязку. В интерактивной оболочке *utop* область видимости любой `let`-привязки верхнего уровня распространяется на весь код, следующий ниже, который будет введен в течение сеанса. Когда привязка осуществляется внутри модуля, ее действие простирается до конца этого модуля.

Например:

OCaml utop

<https://github.com/realworldocaml/examples/blob/v1/code/variables-and-functions/main.topscript>

```
# let x = 3;;
val x : int = 3
# let y = 4;;
val y : int = 4
# let z = x + y;;
val z : int = 7
```

Ключевое слово `let` можно также использовать для создания связанных переменных, область видимости которых ограничивается определенным выражением, с использованием следующего синтаксиса:

Syntax

https://github.com/realworldocaml/examples/blob/v1/code/variables-and-functions/let_in.syntax

```
let <variable> = <expr1> in <expr2>
```

Здесь сначала будет вычислено выражение `expr1`, а затем выражение `expr2`, внутри которого будет доступна переменная `variable`, связанная со значением, произведенным выражением `expr1`. Вот как это выглядит на практике:

OCaml utop (part 1)

<https://github.com/realworldocaml/examples/blob/v1/code/variables-and-functions/main.topscript>

```
# let languages = "OCaml,Perl,C++,C";;
val languages : string = "OCaml,Perl,C++,C"
# let dashed_languages =
  let language_list = String.split languages ~on:', ' in
    String.concat ~sep:"- " language_list
  ;;
val dashed_languages : string = "OCaml-Perl-C++-C"
```

Обратите внимание, что область видимости переменной `language_list` ограничивается выражением `String.concat ~sep:"- " language_list`, и она недоступна на верхнем уровне, в чем можно убедиться, попытавшись получить ее значение:

OCaml utop (part 2)

<https://github.com/realworldocaml/examples/blob/v1/code/variables-and-functions/main.topscript>

```
# language_list;;
Characters ~1-13:
Error: Unbound value language_list
(Ошибка: Несвязанное значение language_list)
```

`let`-привязки во вложенной области видимости могут *затенять*, или скрывать, определения во внешней области видимости. Так, мы могли бы записать пример `dashed_languages` иначе:

OCaml utop (part 3)

<https://github.com/realworldocaml/examples/blob/v1/code/variables-and-functions/main.topscript>

```
# let languages = "OCaml,Perl,C++,C";;
val languages : string = "OCaml,Perl,C++,C"
# let dashed_languages =
  let languages = String.split languages ~on:', ' in
  String.concat ~sep:"- " languages
;;
val dashed_languages : string = "OCaml-Perl-C++-C"
```

На этот раз во внутренней области видимости мы связали список строк с именем `languages` вместо `language_list`, скрыв тем самым оригинальное определение `languages`. Но за границами определения `dashed_languages` действие внутренней области видимости заканчивается, и вновь начинает действовать первоначальное определение `languages`:

OCaml utop (part 4)

<https://github.com/realworldocaml/examples/blob/v1/code/variables-and-functions/main.topscript>

```
# languages;;
- : string = "OCaml,Perl,C++,C"
```

Для выполнения сложных вычислений часто принято использовать последовательность вложенных выражений `let/in`. Например, можно записать такое выражение:

OCaml utop (part 5)

<https://github.com/realworldocaml/examples/blob/v1/code/variables-and-functions/main.topscript>

```
# let area_of_ring inner_radius outer_radius =
  let pi = acos (-1.) in
  let area_of_circle r = pi *. r *. r in
  area_of_circle outer_radius -. area_of_circle inner_radius
;;
val area_of_ring : float -> float -> float = <fun>
# area_of_ring 1. 3.;;
- : float = 25.1327412287
```

Важно не путать последовательность `let`-привязок с модификацией изменяемых переменных. Например, посмотрите, как будет работать `area_of_ring`, если преднамеренно добавить небольшой кусочек запутывающего кода:

OCaml utop (part 6)

<https://github.com/realworldocaml/examples/blob/v1/code/variables-and-functions/main.topscript>

```
# let area_of_ring inner_radius outer_radius =
  let pi = acos (-1.) in
  let area_of_circle r = pi *. r *. r in
  let pi = 0. in
```

```

    area_of_circle outer_radius -. area_of_circle inner_radius
;;
Characters 126-128:
Warning 26: unused variable pi.
(Предупреждение 26: неиспользуемая переменная pi.)
val area_of_ring : float -> float -> float = <fun>

```

Здесь мы повторно связали имя `pi` с нулевым значением после определения функции `area_of_circle`. Кому-то может показаться, что в результате вычислений должен получиться нуль, но в действительности поведение функции не изменится. Причина в том, что первоначальное определение `pi` не изменилось – оно просто было скрыто новым определением. Это означает, что при попытке сослаться на `pi` ниже мы увидим новое определение – значение 0, но выше этого определения значение переменной не изменится. Однако ниже нового определения `pi` эта переменная не используется, поэтому оно не оказывает никакого влияния. Этим же объясняется предупреждение, выданное интерактивной оболочкой, сообщающее о неиспользуемом определении `pi`.

В OCaml `let`-привязки относятся к категории неизменяемых. В языке поддерживается множество различных изменяемых значений, о которых рассказывается в главе 8, но в нем нет изменяемых переменных.

Почему переменные не могут изменяться?

Факт неизменяемости переменных является одним из источников недопонимания для тех, кто только начинает осваивать OCaml. Это кажется удивительным даже с точки зрения терминологии. Разве суть переменной не в том, чтобы изменяться?

Ответ на этот вопрос прост: переменные в OCaml (и вообще в функциональных языках) в действительности больше напоминают переменные в уравнениях, чем переменные в императивных языках. Представьте себе математическое тождество $x(y + z) = xy + xz$. Переменные x , y и z в нем нельзя изменить. Они изменяются в том смысле, что вы можете подставить в это тождество разные числа, для которых оно будет выполняться. То же справедливо и для переменных в функциональных языках. Функциям можно передать разные входные значения, вследствие чего переменные в них будут принимать разные значения без всякого изменения.

Сопоставление с образцом и `let`

Другим полезным свойством `let`-привязки является поддержка *образцов* (patterns) с левой стороны. Взгляните на следующий код, использующий `List.unzip`, функцию для преобразования списка пар в пару списков:

OCaml `utop` (part 7)

<https://github.com/realworldocaml/examples/blob/v1/code/variables-and-functions/main.topscript>

```

# let (ints,strings) = List.unzip [(1,"one"); (2,"two"); (3,"three")];;
val ints : int list = [1; 2; 3]
val strings : string list = ["one"; "two"; "three"]

```

Здесь конструкция `(ints,strings)` – это образец, и `let` связывает оба идентификатора в шаблоне с возвращаемыми значениями. Фактически образец является описанием структуры данных, где отдельные компоненты являются иденти-

фикаторами, которые требуется связать. Как было показано в разделе «Кортежи, списки, необязательные значения и сопоставление с образцом» в главе 1, OCaml поддерживает образцы для самых разных типов данных.

Применение образца в let-привязке имеет больше смысла для бесспорных образцов, то есть когда любое значение рассматриваемого типа гарантированно будет соответствовать образцу. Образцы кортежей и записей являются бесспорными, а образцы списков – нет. Взгляните на следующий пример, реализующий функцию для перевода в верхний регистр первого элемента списка значений, разделенных запятыми:

OCaml utop (part 8)

<https://github.com/realworldocaml/examples/blob/v1/code/variables-and-functions/main.topscript>

```
# let upcase_first_entry line =
  let (first :: rest) = String.split ~on:', ' line in
    String.concat ~sep:", " (String.uppercase first :: rest)
;;
Characters 40-53:
Warning 8: this pattern-matching is not exhaustive.
Here is an example of a value that is not matched:
[]
(Предупреждение 8: это сопоставление не является исчерпывающим.
Ниже следует пример значения, не соответствующего сопоставлению:
[])
val upcase_first_entry : string -> string = <fun>
```

На практике данная ситуация невозможна, потому что `String.split` всегда возвращает список как минимум с одним элементом. Но компилятор не знает об этом и поэтому вывел предупреждение. Вообще говоря, такие случаи лучше обрабатывать явно, применяя инструкцию `match`:

OCaml utop (part 9)

<https://github.com/realworldocaml/examples/blob/v1/code/variables-and-functions/main.topscript>

```
# let upcase_first_entry line =
  match String.split ~on:', ' line with
  | [] -> assert false (* String.split возвращает не менее одного элемента *)
  | first :: rest -> String.concat ~sep:", " (String.uppercase first :: rest)
;;
val upcase_first_entry : string -> string = <fun>
```

Обратите внимание: здесь мы впервые использовали инструкцию `assert`, которую удобно применять для отметки случаев, которые никогда не должны выполняться. Подробнее об инструкции `assert` рассказывается в главе 7.

Функции

Учитывая, что OCaml является функциональным языком, неудивительно, что функции в нем играют важную роль. Фактически функции присутствовали прак-

тически во всех примерах, которые мы видели. Этот раздел содержит более подробную информацию о функциях и рассказывает о том, как действуют функции в языке OCaml. Как будет показано далее, функции в OCaml имеют множество отличий от функций в основных языках программирования.

Анонимные функции

Начнем со знакомства с наиболее типичным способом объявления функций в языке OCaml: *анонимными функциями*. Анонимная функция – это функция, объявленная без имени. Такие функции могут объявляться с помощью ключевого слова `fun`, как показано ниже:

OCaml utop (part 10)

<https://github.com/realworldocaml/examples/blob/v1/code/variables-and-functions/main.topscript>

```
# (fun x -> x + 1) ;;
- : int -> int = <fun>
```

Анонимные функции действуют практически так же, как именованные. Например, анонимную функцию можно применить к аргументу:

OCaml utop (part 11)

<https://github.com/realworldocaml/examples/blob/v1/code/variables-and-functions/main.topscript>

```
# (fun x -> x + 1) 7;;
- : int = 8
```

Или передать другой функции. Передача функций итеративным функциям, таким как `List.map`, является, пожалуй, наиболее типичным случаем использования анонимных функций:

OCaml utop (part 12)

<https://github.com/realworldocaml/examples/blob/v1/code/variables-and-functions/main.topscript>

```
# List.map ~f:(fun x -> x + 1) [1;2;3];;
- : int list = [2; 3; 4]
```

Ими можно даже наполнять структуры данных:

OCaml utop (part 13)

<https://github.com/realworldocaml/examples/blob/v1/code/variables-and-functions/main.topscript>

```
# let increments = [ (fun x -> x + 1); (fun x -> x + 2) ] ;;
val increments : (int -> int) list = [<fun>; <fun>]
# List.map ~f:(fun g -> g 5) increments;;
- : int list = [6; 7]
```

Стоит приостановиться на минуту и разобрать этот пример, поскольку такой способ применения функций первое время может казаться непонятным. Обра-

тите внимание, что `(fun g -> g 5)` – это функция, принимающая другую функцию в виде аргумента и применяющая ее к числу 5. Вызов `List.map` поочередно применяет `(fun g -> g 5)` ко всем элементам списка `increments` (которые сами являются функциями) и возвращает список с результатами применения всех этих функций.

Важно понять, что функции в языке OCaml являются самыми обычными значениями и с ними можно выполнять любые операции, допустимые для обычных значений, включая передачу другим функциям и возврат из функций, а также сохранение в структурах данных. Даже имена присваиваются функциям точно так же, как любым другим значениям, – с помощью `let`-привязки:

OCaml utop (part 14)

<https://github.com/realworldocaml/examples/blob/v1/code/variables-and-functions/main.topscript>

```
# let plusone = (fun x -> x + 1);;
val plusone : int -> int = <fun>
# plusone 3;;
- : int = 4
```

Определение именованных функций настолько типично, что для этого даже имеется синтаксический сахар. Например, следующее определение функции `plusone` эквивалентно предыдущему:

OCaml utop (part 15)

<https://github.com/realworldocaml/examples/blob/v1/code/variables-and-functions/main.topscript>

```
# let plusone x = x + 1;;
val plusone : int -> int = <fun>
```

Это – наиболее часто используемый и удобный способ объявления функций, но, помимо синтаксических тонкостей, эти два стиля определения функций совершенно эквивалентны.

let и fun

Функции и `let`-привязки имеют непосредственное отношение друг к другу. Параметры функции можно рассматривать как переменные, которые будут связаны со значениями, переданными вызывающей программой. Например, следующие два выражения почти эквивалентны:

OCaml utop (part 16)

<https://github.com/realworldocaml/examples/blob/v1/code/variables-and-functions/main.topscript>

```
# (fun x -> x + 1) 7;;
- : int = 8
# let x = 7 in x + 1;;
- : int = 8
```

Такая взаимосвязь играет важную роль, и мы еще вернемся к ней, когда будем изучать стиль программирования на основе монад в главе 18.

Функции нескольких аргументов

Конечно же, OCaml поддерживает функции нескольких аргументов, такие как:

OCaml utop (part 17)

<https://github.com/realworldocaml/examples/blob/v1/code/variables-and-functions/main.topscript>

```
# let abs_diff x y = abs (x - y);;
val abs_diff : int -> int -> int = <fun>
# abs_diff 3 4;;
- : int = 1
```

Кому-то сигнатура функции `abs_diff` со всеми ее стрелками может показаться немного сложной. Чтобы понять, почему она получилась такой, давайте перепишем `abs_diff`, представив ее в эквивалентной форме с помощью ключевого слова `fun`:

OCaml utop (part 18)

<https://github.com/realworldocaml/examples/blob/v1/code/variables-and-functions/main.topscript>

```
# let abs_diff =
  (fun x -> (fun y -> abs (x - y)));;
val abs_diff : int -> int -> int = <fun>
```

Такая форма записи более явно показывает, что `abs_diff` фактически является функцией одного аргумента, возвращающей другую функцию одного аргумента, которая, в свою очередь, возвращает конечный результат. Так как функции вложены друг в друга, внутреннее выражение `abs (x - y)` имеет доступ к обоим значениям — к значению `x`, связанному применением внешней функции, и к значению `y`, связанному применением внутренней функции.

Функции такого вида называются *каррированными функциями* (curried function). (Название «карринг» было принято в честь математика и логика Хаскелла Карри (Haskell Curry), оказавшего большое влияние на теорию и архитектуру языков программирования.) Для правильной интерпретации сигнатуры типа каррированной функции важно помнить, что стрелка `->` является правоассоциативной. Соответственно, в сигнатуру `abs_diff` можно добавить круглые скобки, как показано ниже:

OCaml: variables-and-functions/abs_diff.mli

https://github.com/realworldocaml/examples/blob/v1/code/variables-and-functions/abs_diff.mli

```
val abs_diff : int -> (int -> int)
```

Скобки не изменяют сути сигнатуры, но с их помощью легче увидеть каррирование.

Карринг — это намного больше, чем теоретический курьез. Каррирование можно использовать для специализации функции передачей ей некоторых из аргументов. Например, в следующем фрагменте создается специализированная версия функции `abs_diff`, вычисляющей расстояние до заданного числа от числа 3:

OCaml utop (part 19)

<https://github.com/realworldocaml/examples/blob/v1/code/variables-and-functions/main.topscript>

```
# let dist_from_3 = abs_diff 3;;
val dist_from_3 : int -> int = <fun>
# dist_from_3 8;;
- : int = 5
# dist_from_3 (-1);;
- : int = 4
```

Прием применения некоторых аргументов каррированной функции, чтобы получить новую функцию, называется *частичным применением* (partial application).

Обратите внимание, что ключевое слово `fun` поддерживает карринг, предоставляя собственный синтаксис, поэтому следующее определение функции `abs_diff` эквивалентно предыдущему.

OCaml utop (part 20)

<https://github.com/realworldocaml/examples/blob/v1/code/variables-and-functions/main.topscript>

```
# let abs_diff = (fun x y -> abs (x - y));;
val abs_diff : int -> int -> int = <fun>
```

Возможно, вам кажется, что каррирование функций является жутко дорогим удовольствием, но это не так. В OCaml вызов каррированных функций со всеми их аргументами не влечет никаких накладных расходов. (Частичное применение сопряжено с небольшими накладными расходами, что, впрочем, неудивительно.)

Каррирование – не единственный способ определения функций нескольких аргументов в OCaml. В этом языке допускается также использовать в роли аргументов разные элементы кортежа. То есть мы вполне могли бы написать такое определение функции:

OCaml utop (part 21)

<https://github.com/realworldocaml/examples/blob/v1/code/variables-and-functions/main.topscript>

```
# let abs_diff (x,y) = abs (x - y);;
val abs_diff : int * int -> int = <fun>
# abs_diff (3,4);;
- : int = 1
```

При использовании этой формы определения функций компилятор OCaml выполняет некоторые оптимизации. В частности, он не создает в памяти новый кортеж с единственной целью передать аргументы функции. Но вы не сможете использовать прием частичного применения с функциями, объявленными таким способом.

Каждый из этих стилей имеет свои преимущества, но в общем случае следует придерживаться каррирования, потому что этот стиль является стилем по умолчанию в мире OCaml.

Рекурсивные функции

Функция считается *рекурсивной*, если она ссылается на саму себя. Рекурсия – важный прием в любом языке программирования, но в функциональных языках он имеет особую значимость, потому что дает возможность создавать циклические конструкции. (Как будет рассказываться в главе 8, OCaml поддерживает также императивные конструкции циклов, такие как `for` и `while`, но они могут применяться только при использовании императивных свойств языка OCaml.)

Чтобы определить рекурсивную функцию, необходимо пометить `let`-привязку как рекурсивную с помощью ключевого слова `rec`, как показано в следующем примере определения функции, выполняющей поиск первого повторяющегося элемента в списке:

OCaml utop (part 22)

<https://github.com/realworldocaml/examples/tree/v1/code/variables-and-functions/main.topscript>

```
# let rec find_first_stutter list =
  match list with
  | [] | [_] ->
    (* нуль или один элемент - нет повторяющихся элементов *)
    None
  | x :: y :: tl ->
    if x = y then Some x else find_first_stutter (y::tl)
;;

val find_first_stutter : 'a list -> 'a option = <fun>
```

Обратите внимание в этом примере на образец `| [] | [_]` – это то, что называют *ИЛИ-образцом* (or-pattern). Он представляет собой дизъюнкцию двух образцов, совпадение с которой будет считаться состоявшимся, если совпадет хотя бы один из образцов. В данном случае образцу `[]` соответствует пустой список, а образцу `[_]` – любой список с одним элементом. Здесь используется символ подчеркивания (`_`), что избавляет нас от необходимости явно указывать имя для этого единственного элемента.

Можно также определить множество взаимно рекурсивных значений, используя `let rec` в сочетании с ключевым словом `and`. Пример (неоправданно неэффективный) приводится ниже:

OCaml utop (part 23)

<https://github.com/realworldocaml/examples/blob/v1/code/variables-and-functions/main.topscript>

```
# let rec is_even x =
  if x = 0 then true else is_odd (x - 1)
and is_odd x =
  if x = 0 then false else is_even (x - 1)
;;

val is_even : int -> bool = <fun>
val is_odd : int -> bool = <fun>
# List.map ~f:is_even [0;1;2;3;4;5];;
- : bool list = [true; false; true; false; true; false]
```

```
# List.map ~f:is_odd [0;1;2;3;4;5];;
- : bool list = [false; true; false; true; false; true]
```

OCaml различает нерекурсивные (с использованием `let`) и рекурсивные (с использованием `let rec`) определения в основном по техническим причинам: алгоритм вывода типов должен знать, когда определяется множество взаимно рекурсивных функций, а также по некоторым другим причинам, которые отсутствуют в исключительно функциональных языках, таких как Haskell, поэтому они должны отмечаться программистом явно.

Но это решение имеет и положительные стороны. С одной стороны, рекурсивные (и особенно взаимно рекурсивные) определения сложнее в восприятии, чем нерекурсивные. Соответственно, отсутствие ключевого слова `rec` позволяет предположить, что `let`-привязка является нерекурсивной и может опираться только на предшествующие привязки.

Кроме того, наличие нерекурсивной формы упрощает создание нового определения, расширяющего и замещающего существующее с помощью приема затенения.

Префиксные и инфиксные операторы

До сих пор мы рассматривали примеры функций, используемых в обоих стилях записи, префиксном и инфиксном:

OCaml utop (part 24)

<https://github.com/realworldocaml/examples/blob/v1/code/variables-and-functions/main.topscript>

```
# Int.max 3 4 (* префиксный стиль *);;
- : int = 4
# 3 + 4      (* инфиксный стиль *);;
- : int = 7
```

Возможно, вы даже не предполагали, что во втором случае имеете дело с обычной функцией, но это действительно так. Инфиксные операторы, такие как `+`, в действительности отличаются от других функций только синтаксисом. Если заключить инфиксный оператор в круглые скобки, его можно будет использовать как обычную префиксную функцию:

OCaml utop (part 25)

<https://github.com/realworldocaml/examples/blob/v1/code/variables-and-functions/main.topscript>

```
# (+) 3 4;;
- : int = 7
# List.map ~f:(+) 3 [4;5;6];;
- : int list = [7; 8; 9]
```

Во втором выражении мы частично применили `(+)` для создания функции, увеличивающей свой единственный аргумент на 3.

Функция синтаксически интерпретируется как оператор, если имя этой функции принадлежит множеству специальных идентификаторов. Это множество включает любые идентификаторы, состоящие из следующих символов:

Syntax

<https://github.com/realworldocaml/examples/blob/v1/code/variables-and-functions/operators.syntax>

```
! $ % & * + - . / : < = > ? @ ^ | ~
```

или являющиеся предопределенными строками, включая `mod`, оператор деления по модулю, и `lsl` (от «logical shift left» – логический сдвиг влево), оператор поразрядного сдвига.

Мы можем определить (или переопределить) значение оператора. Ниже приводится пример простого оператора сложения двухэлементных векторов, содержащих целые числа:

OCaml utop (part 26)

<https://github.com/realworldocaml/examples/blob/v1/code/variables-and-functions/main.topscript>

```
# let (!!) (x1,y1) (x2,y2) = (x1 + x2, y1 + y2);;
val ( !! ) : int * int -> int * int -> int * int = <fun>
# (3,2) !! (-2,4);;
- : int * int = (1, 6)
```

Обратите внимание, что операторы, содержащие `*`, требуют проявления особой осторожности при обращении с ними. Взгляните на следующий пример:

OCaml utop (part 27)

<https://github.com/realworldocaml/examples/blob/v1/code/variables-and-functions/main.topscript>

```
# let (***) x y = (x ** y) ** y;;
Characters 17-18:
Error: This expression has type int but an expression was expected of type
      float
(Ошибка: Это выражение имеет тип int, тогда как ожидалось выражение типа
      float)
```

Проблема в том, что `(***)` не интерпретируется как оператор; он воспринимается как комментарий! Чтобы избавиться от проблемы, следует добавить пробелы между оператором и скобками:

OCaml utop (part 28)

<https://github.com/realworldocaml/examples/blob/v1/code/variables-and-functions/main.topscript>

```
# let ( ***) x y = (x ** y) ** y;;
val ( ***) : float -> float -> float = <fun>
```

Синтаксическая роль оператора обычно определяется по одному-двум первым символам, однако из этого правила есть несколько исключений. В табл. 2.1 перечислены группы операторов и других синтаксических форм в порядке убывания приоритетов. Форма записи `!...` обозначает класс операторов, начинающихся с `!`.

Таблица 2.1. Приоритеты и ассоциативность операторов

Префикс оператора	Ассоциативность
!..., ?..., ~...	Префиксный
.., .(, .[	–
Применение функции, конструктор, assert, lazy	Левая
-, -.	Префиксный
**..., lsl, lsr, asr	Правая
*..., /..., %..., mod, land, lor, lxor	Левая
+..., -...	Левая
::	Правая
@..., ^...	Правая
=..., <..., >..., ..., &..., \$...	Левая
&, &&	Правая
or,	Правая
,	–
<-, :=	Правая
if	–
;	Правая

Существует одно важное исключение: операторы `-` и `-.` , выполняющие целочисленное и вещественное вычитание, могут выступать в качестве и префиксных (отрицание), и инфиксных (вычитание) операторов. То есть допустимыми являются оба выражения: `-x` и `x - y`. Важно также запомнить, что оператор отрицания имеет более низкий приоритет, чем применение функции, это означает, что при передаче отрицательного значения его необходимо заключить в круглые скобки, как в следующем примере:

OCaml utop (part 29)

<https://github.com/realworldocaml/examples/blob/v1/code/variables-and-functions/main.topscript>

```
# Int.max 3 (~4);;
- : int = 3
# Int.max 3 -4;;
Characters ~1-9:
Error: This expression has type int -> int
      but an expression was expected of type int
(Ошибка: Это выражение имеет тип int -> int,
      тогда как ожидалось выражение типа int)
```

Здесь компилятор OCaml интерпретировал второе выражение, как показано ниже:

OCaml utop (part 30)

<https://github.com/realworldocaml/examples/blob/v1/code/variables-and-functions/main.topscript>

```
# (Int.max 3) - 4;;
Characters 1-10:
Error: This expression has type int -> int
```

but an expression was expected of type int
 (Ошибка: Это выражение имеет тип int -> int,
 тогда как ожидалось выражение типа int)

что, очевидно, не имеет смысла.

Ниже приводится пример очень полезного оператора из стандартной библиотеки, чье поведение полностью зависит от правил приоритетов операций, перечисленных выше:

OCaml utop (part 31)

<https://github.com/realworldocaml/examples/blob/v1/code/variables-and-functions/main.topscript>

```
# let (|>) x f = f x ;;
val ( |> ) : 'a -> ('a -> 'b) -> 'b = <fun>
```

Цель этого оператора сначала кажется неочевидной: он просто принимает значение и функцию и применяет функцию к значению. Несмотря на такое описание, не сообщающее ничего особенного, этот оператор играет роль упорядочивающего оператора, напоминая оператор конвейера (|) в командной оболочке UNIX. Взгляните, например, на следующий фрагмент кода, осуществляющий вывод отдельных элементов переменной окружения PATH. Обратите внимание, что List.dedup удаляет повторяющиеся элементы из списка, сортируя список с помощью указанной функции сравнения:

OCaml utop (part 32)

<https://github.com/realworldocaml/examples/blob/v1/code/variables-and-functions/main.topscript>

```
# let path = "/usr/bin:/usr/local/bin:/bin:/sbin";;
val path : string = "/usr/bin:/usr/local/bin:/bin:/sbin"
# String.split ~on:'/' path
|> List.dedup ~compare:String.compare
|> List.iter ~f:print_endline
;;
/bin
/sbin
/usr/bin
/usr/local/bin
- : unit = ()
```

Отметьте, что мы могли бы обойтись без оператора |>, но реализация получилась более запутанной:

OCaml utop (part 33)

<https://github.com/realworldocaml/examples/blob/v1/code/variables-and-functions/main.topscript>

```
# let split_path = String.split ~on:'/' path in
  let deduped_path = List.dedup ~compare:String.compare split_path in
  List.iter ~f:print_endline deduped_path
;;
/bin
/sbin
```



```
/usr/bin
/usr/local/bin
- : unit = ()
```

Важной частью в этом примере является частичное применение. Например, `List.iter` обычно принимает два аргумента: функцию для применения к каждому элементу списка и сам список. Мы можем вызвать `List.iter` со всеми аргументами:

OCaml utop (part 34)

<https://github.com/realworldocaml/examples/blob/v1/code/variables-and-functions/main.topscript>

```
# List.iter ~f:print_endline ["Two"; "lines"];;
Two
lines
- : unit = ()
```

Или передать только аргумент-функцию, получив в результате функцию вывода списка строк:

OCaml utop (part 35)

<https://github.com/realworldocaml/examples/blob/v1/code/variables-and-functions/main.topscript>

```
# List.iter ~f:print_endline;;
- : string list -> unit = <fun>
```

Именно эта последняя форма используется в предыдущем конвейере `|>`.

В данном случае можно использовать только оператор `|>`, потому что он является левоассоциативным. Давайте посмотрим, что случится, если попробовать использовать правоассоциативный оператор, такой как `(^>)`:

OCaml utop (part 36)

<https://github.com/realworldocaml/examples/blob/v1/code/variables-and-functions/main.topscript>

```
# let (^>) x f = f x;;
val ( ^> ) : 'a -> ('a -> 'b) -> 'b = <fun>
# Sys.getenv_exn "PATH"
^> String.split ~on:'.' path
^> List.dedup ~compare:String.compare
^> List.iter ~f:print_endline
;;
```

Characters 98-124:

```
Error: This expression has type string list -> unit
      but an expression was expected of type
      (string list -> string list) -> 'a
Type string list is not compatible with type
string list -> string list
```

(Ошибка: Это выражение имеет тип `list -> unit`, тогда как ожидалось выражение типа `(string list -> string list) -> 'a`. Тип `string list` несовместим с типом `string list -> string list`)

На первый взгляд, сообщение об ошибке кажется обескураживающим. Проблема в том, что оператор `^>` является правоассоциативным. Он пытается передать значение `List.dedup ~compare:String.compare` функции `List.iter ~f:print_endline`. Но функция `List.iter ~f:print_endline` ожидает получить список строк, а не функцию.

Этот пример с ошибкой наглядно показывает важность выбора оператора, и особенно это касается его ассоциативности.

Объявление функций с помощью ключевого слова `function`

Другой способ определить функцию – воспользоваться ключевым словом `function`. Вместо синтаксической поддержки объявления функций нескольких аргументов (каррирования) ключевое слово `function` предлагает встроенную поддержку сопоставления с образцом. Например:

OCaml utop (part 37)

<https://github.com/realworldocaml/examples/blob/v1/code/variables-and-functions/main.topscript>

```
# let some_or_zero = function
  | Some x -> x
  | None -> 0
;;
val some_or_zero : int option -> int = <fun>
# List.map ~f:some_or_zero [Some 3; None; Some 4];;
- : int list = [3; 0; 4]
```

Такое объявление эквивалентно комбинации обычного определения функции с инструкцией `match`:

OCaml utop (part 38)

<https://github.com/realworldocaml/examples/blob/v1/code/variables-and-functions/main.topscript>

```
# let some_or_zero num_opt =
  match num_opt with
  | Some x -> x
  | None -> 0
;;
val some_or_zero : int option -> int = <fun>
```

Разные стили объявления функций могут комбинироваться без каких-либо ограничений, как показано в следующем примере, где определяется функция двух аргументов (каррированная), второй аргумент которой анализируется с помощью выражения сопоставления с образцом:

OCaml utop (part 39)

<https://github.com/realworldocaml/examples/blob/v1/code/variables-and-functions/main.topscript>

```
# let some_or_default default = function
  | Some x -> x
```

```

    | None -> default
;;
val some_or_default : 'a -> 'a option -> 'a = <fun>
# some_or_default 3 (Some 5);;
- : int = 5
# List.map ~f:(some_or_default 100) [Some 3; None; Some 4];;
- : int list = [3; 100; 4]

```

Обратите также внимание на использование приема частичного применения для создания функции, которая затем передается в вызов `List.map`. Иными словами, `some_or_default 100` – это функция, созданная передачей единственного, первого аргумента функции `some_or_default`.

Аргументы с метками

До сих пор все наши функции различали свои аргументы по их позициям, то есть порядок передачи аргументов функциям играл важную роль. Однако язык OCaml поддерживает также аргументы с метками (*labeled arguments*), позволяя функциям идентифицировать аргументы по именам. На самом деле мы уже встречали функции из библиотеки `Core`, такие как `List.map`, использующие аргументы с метками. Определения таких аргументов начинаются со знака тильды (`~`), за которым следует метка (завершающаяся двоеточием), которая в теле функции будет служить именем аргумента. Например:

OCaml utop (part 40)

<https://github.com/realworldocaml/examples/blob/v1/code/variables-and-functions/main.topscript>

```

# let ratio ~num ~denom = float num /. float denom;;
val ratio : num:int -> denom:int -> float = <fun>

```

Передавать аргументы таким функциям можно, используя похожее соглашение. Как показано ниже, аргументы с метками допускается передавать в произвольном порядке:

OCaml utop (part 41)

<https://github.com/realworldocaml/examples/blob/v1/code/variables-and-functions/main.topscript>

```

# ratio ~num:3 ~denom:10;;
- : float = 0.3
# ratio ~denom:10 ~num:3;;
- : float = 0.3

```

Язык OCaml поддерживает также *уплотнение меток* (*label punning*), то есть, если имя метки и имя переменной совпадают, текст после двоеточия (`:`) можно отбросить. Фактически мы уже использовали этот прием в объявлении функции `ratio`. Следующий пример демонстрирует, как прием уплотнения меток можно использовать в вызове функции:

OCaml utop (part 42)

<https://github.com/realworldocaml/examples/blob/v1/code/variables-and-functions/main.topscript>

```
# let num = 3 in
  let denom = 4 in
    ratio ~num ~denom;;
- : float = 0.75
```

Аргументы с метками с успехом можно использовать в разных ситуациях.

- Когда определяется функция с большим числом аргументов. Выше определенного порога аргументы проще запоминать по именам, чем по позициям.
- Когда назначение того или иного аргумента неочевидно из сигнатуры. Представьте функцию, создающую хэш-таблицу, в первом аргументе которой передается начальный размер массива, лежащего в основе хэш-таблицы, а во втором — логический флаг, указывающий, должен ли сокращаться объем массива при удалении элементов:

OCaml

https://github.com/realworldocaml/examples/blob/v1/code/variables-and-functions/htable_sig1.ml

```
val create_hashtable : int -> bool -> ('a,'b) Hashtable.t
```

По одной только сигнатуре сложно понять назначение аргументов, но, добавив метки, можно сделать наши намерения более очевидными:

OCaml

https://github.com/realworldocaml/examples/blob/v1/code/variables-and-functions/htable_sig2.ml

```
val create_hashtable :
  init_size:int -> allow_shrinking:bool -> ('a,'b) Hashtable.t
```

Особенно важную роль играет выбор имен меток в отношении логических значений, поскольку порой трудно понять смысл значения `true` — «разрешить» или «запретить» ту или иную особенность.

- Когда определяется функция с несколькими аргументами, которые легко спутать друг с другом. Эта проблема особенно характерна для аргументов одного типа. Например, взгляните на сигнатуру функции, извлекающей подстроку:

OCaml

https://github.com/realworldocaml/examples/blob/v1/code/variables-and-functions/substring_sig1.ml

```
val substring: string -> int -> int -> string
```

Здесь имеются два целочисленных аргумента, определяющих начальную позицию подстроки в строке и длину подстроки, соответственно. Мы можем обозначить их более очевидным образом, добавив метки в сигнатуру:

OCaml

https://github.com/realworldocaml/examples/blob/v1/code/variables-and-functions/substring_sig2.ml

```
val substring: string -> pos:int -> len:int -> string
```

Следование этому правилу улучшает читаемость сигнатур и клиентского кода и снижает вероятность путаницы позиции и длины подстроки.

- Когда требуется более высокая гибкость в порядке следования аргументов при вызове функции. Представьте себе функцию, такую как `List.iter`, которая принимает два аргумента: функцию и список элементов, к которым будет применяться эта функция. На практике часто используется прием частичного применения `List.iter` за счет передачи ей одной только функции, как показано в следующем примере:

OCaml utop (part 43)

<https://github.com/realworldocaml/examples/blob/v1/code/variables-and-functions/main.topscript>

```
# String.split ~on:'.' path
    |> List.dedup ~compare:String.compare
    |> List.iter ~f:print_endline
;;
/bin
/sbin
/usr/bin
/usr/local/bin
- : unit = ()
```

При таком подходе аргумент с функцией должен следовать в объявлении первым. Однако иногда удобнее, когда аргумент с функцией передается вторым. Одна из типичных причин перемены аргументов местами – удобочитаемость кода. В частности, когда функция нескольких аргументов передается другой функции, код проще читать, если эта функция передается последней.

Функции высшего порядка и метки

Аргументы с метками имеют один досадный недостаток: когда вызывается функция, имеющая аргументы с метками, порядок их следования действительно не имеет значения, но он имеет значение в контексте высшего порядка, например когда функция с такими аргументами передается другой функции. Например:

OCaml utop (part 44)

<https://github.com/realworldocaml/examples/blob/v1/code/variables-and-functions/main.topscript>

```
# let apply_to_tuple f (first,second) = f ~first ~second;;
val apply_to_tuple : (first:'a -> second:'b -> 'c) -> 'a * 'b -> 'c = <fun>
```

Здесь определяется функция `apply_to_tuple`, принимающая в первом аргументе функцию двух аргументов с метками `first` и `second`, следующими именно в таком

порядке. Мы могли бы определить `apply_to_tuple` иначе, изменив порядок следования аргументов с метками:

OCaml utop (part 45)

<https://github.com/realworldocaml/examples/blob/v1/code/variables-and-functions/main.topscript>

```
# let apply_to_tuple_2 f (first,second) = f ~second ~first;;
val apply_to_tuple_2 : (second:'a -> first:'b -> 'c) -> 'b * 'a -> 'c = <fun>
```

Как оказывается, порядок имеет значение. В частности, если определить функцию с аргументами, следующими в ином порядке:

OCaml utop (part 46)

<https://github.com/realworldocaml/examples/blob/v1/code/variables-and-functions/main.topscript>

```
# let divide ~first ~second = first / second;;
val divide : first:int -> second:int -> int = <fun>
```

можно обнаружить, что ее нельзя передать функции `apply_to_tuple_2`.

OCaml utop (part 47)

<https://github.com/realworldocaml/examples/blob/v1/code/variables-and-functions/main.topscript>

```
# apply_to_tuple_2 divide (3,4);;
Characters 17-23:
Error: This expression has type first:int -> second:int -> int
      but an expression was expected of type second:'a -> first:'b -> 'c
(Ошибка: Это выражение имеет тип first:int -> second:int -> int,
      тогда как ожидалось выражение типа second:'a -> first:'b -> 'c)
```

Но ее можно передать первоначальной версии `apply_to_tuple`:

OCaml utop (part 48)

<https://github.com/realworldocaml/examples/blob/v1/code/variables-and-functions/main.topscript>

```
# let apply_to_tuple f (first,second) = f ~first ~second;;
val apply_to_tuple : (first:'a -> second:'b -> 'c) -> 'a * 'b -> 'c = <fun>
# apply_to_tuple divide (3,4);;
- : int = 0
```

Как результат при передаче функций, имеющих аргументы с метками, в виде аргументов другим функциям необходимо позаботиться о передаче аргументов с метками в правильном порядке.

Необязательные аргументы

Необязательный аргумент чем-то напоминает аргумент с меткой. Вызывающий код может по выбору передавать или не передавать такой аргумент в вызов функции. Необязательные аргументы передаются с применением того же синтаксиса, что и аргументы с метками, и, подобно аргументам с метками, могут указываться в любом порядке.

Ниже приводится пример функции, выполняющей конкатенацию строк с необязательным разделителем. Она использует оператор `^` для попарного объединения строк:

OCaml utop (part 49)

<https://github.com/realworldocaml/examples/blob/v1/code/variables-and-functions/main.topscript>

```
# let concat ?sep x y =
  let sep = match sep with None -> "" | Some x -> x in
  x ^ sep ^ y
;;
val concat : ?sep:string -> string -> string -> string = <fun>
# concat "foo" "bar" (* без необязательного аргумента *) ;;
- : string = "foobar"
# concat ~sep:":" "foo" "bar" (* с необязательным аргументом *) ;;
- : string = "foo:bar"
```

Здесь знак вопроса (?) в определении функции используется, чтобы пометить аргумент `sep` как необязательный. Если вызывающий код передаст в аргументе `sep` значение типа `string`, внутри функции аргумент `sep` будет доступен как значение типа `string option`, в противном случае он будет доступен как значение `None`.

В предыдущем примере потребовалось писать шаблонный код для выбора разделителя по умолчанию, когда вызывающий код не передает аргумент `sep`. Это настолько распространенный шаблон программирования, что для него был добавлен явный синтаксис определения значения по умолчанию, позволяющий записать реализацию `concat` более компактно:

OCaml utop (part 50)

<https://github.com/realworldocaml/examples/blob/v1/code/variables-and-functions/main.topscript>

```
# let concat ?(sep="") x y = x ^ sep ^ y ;;
val concat : ?sep:string -> string -> string -> string = <fun>
```

Необязательные аргументы очень удобны, но не следует злоупотреблять ими. Главное преимущество необязательных аргументов состоит в том, что они позволяют писать функции нескольких аргументов, которые пользователи могут просто игнорировать в большинстве ситуаций и вспоминать о них, только когда требуется передать значения, отличные от значений по умолчанию. Они также позволяют дополнять API новыми функциональными возможностями, не изменяя использующего его кода.

Недостаток же заключается в том, что пользователь может не узнать о возможности выбора и из-за этого невольно (и ошибочно) использовать поведение по умолчанию. Вообще говоря, использовать необязательные аргументы имеет смысл, только когда преимущество компактности кода перевешивает недостаток отсутствия явности.

Это означает, что редко используемые функции не должны иметь необязательных аргументов. Лучше вообще стараться избегать использования необязатель-

ных аргументов при определении внутренних функций модуля, то есть функций, которые не включены в интерфейс модуля или в `mli`-файл. Подробнее о `mli`-файлах рассказывается в главе 4.

Явная передача необязательных аргументов

За кулисами, когда вызывающий код не передает необязательного аргумента, функция получит в этом аргументе значение `None`, `Some` – в противном случае. Значения `Some` и `None` обычно не передаются явно.

Но иногда бывает желательно передать `Some` или `None` явно. OCaml дает такую возможность, но при этом аргумент должен быть помечен знаком вопроса (?) вместо тильды (~). Например, следующие две строки представляют эквивалентные способы передачи аргумента `sep` функции `concat`:

OCaml utop (part 51)

<https://github.com/realworldocaml/examples/blob/v1/code/variables-and-functions/main.topscript>

```
# concat ~sep:" " "foo" "bar" (* передать необязательный аргумент *);;
- : string = "foo:bar"
# concat ?sep:(Some " ") "foo" "bar" (* явно передать [Some] *);;
- : string = "foo:bar"
```

А следующие две строки представляют эквивалентные способы вызова `concat` без аргумента `sep`:

OCaml utop (part 52)

<https://github.com/realworldocaml/examples/blob/v1/code/variables-and-functions/main.topscript>

```
# concat "foo" "bar" (* необязательный аргумент не передается *);;
- : string = "foobar"
# concat ?sep:None "foo" "bar" (* явно передается 'None' *);;
- : string = "foobar"
```

Этот прием можно использовать, когда требуется определить функцию-обертку, передающую необязательные аргументы обертываемой функции. Например, представьте, что потребовалось создать функцию с именем `uppercase_concat`, которая действует так же, как `concat`, за исключением того, что преобразует первую строку в верхний регистр. Эту функцию можно было бы определить, как показано ниже:

OCaml utop (part 53)

<https://github.com/realworldocaml/examples/blob/v1/code/variables-and-functions/main.topscript>

```
# let uppercase_concat ?(sep="") a b = concat ~sep (String.uppercase a) b ;;
val uppercase_concat : ?sep:string -> string -> string -> string = <fun>
# uppercase_concat "foo" "bar";;
- : string = "FOObar"
# uppercase_concat "foo" "bar" ~sep:"";;
- : string = "FOO:bar"
```


При таком подходе мы вынуждены принять решение о значении разделителя по умолчанию. Из-за этого, если позднее поведение по умолчанию функции `concat` изменится, нам нужно не забыть привести `uppercase_concat` в соответствие с этим.

Чтобы избавить себя от этих сложностей, можно просто заставить `uppercase_concat` передавать необязательный аргумент функции `concat` с использованием синтаксиса ?:

OCaml utop (part 54)

<https://github.com/realworldocaml/examples/blob/v1/code/variables-and-functions/main.topscript>

```
# let uppercase_concat ?sep a b = concat ?sep (String.uppercase a) b ;;
val uppercase_concat : ?sep:string -> string -> string -> string = <fun>
```

Если теперь кто-то вызовет `uppercase_concat` без аргумента, она явно передаст `None` функции `concat`, оставив за ней право решать, какое значение по умолчанию использовать.

Вывод типов аргументов с метками и необязательных аргументов

Аргументы с метками и необязательные аргументы имеют один тонкий аспект – порядок автоматического определения их типов. Взгляните на следующий пример, вычисляющий числовые производные функции двух вещественных аргументов. Функция вычисления производных принимает аргумент `delta`, определяющий диапазон, в котором вычисляются производные; значения `x` и `y`, определяющие координаты точки, где вычисляется производная; и функцию `f`, для которой вычисляется производная. Сама функция `f` принимает два аргумента с метками, `x` и `y`. Обратите внимание, что в именах переменных допускается использовать апострофы (`'`), то есть `x'` и `y'` здесь являются самыми обычными переменными:

OCaml utop (part 55)

<https://github.com/realworldocaml/examples/blob/v1/code/variables-and-functions/main.topscript>

```
# let numeric_deriv ~delta ~x ~y ~f =
  let x' = x +. delta in
  let y' = y +. delta in
  let base = f ~x ~y in
  let dx = (f ~x:x' ~y -. base) /. delta in
  let dy = (f ~x ~y:y' -. base) /. delta in
  (dx,dy)
;;
val numeric_deriv :
  delta:float ->
  x:float -> y:float -> f:(x:float -> y:float -> float) -> float * float =
  <fun>
```

Здесь порядок следования аргументов функции `f` неочевиден. Так как аргументы с метками могут передаваться в произвольном порядке, складывается ощущение, что он может быть и `y:float -> x:float -> float`, и `x:float -> y:float -> float`.

Хуже того, функцию `f` вполне можно было бы объявить с одним необязательным аргументом вместо аргумента с меткой, что могло бы привести к такой сигнатуре функции `numeric_deriv`:

OCaml

https://github.com/realworldocaml/examples/blob/v1/code/variables-and-functions/numerical_deriv_alt_sig.mli

```
val numeric_deriv :
  delta:float ->
  x:float -> y:float -> f:(?x:float -> y:float -> float) -> float * float
```

Поскольку появляется множество возможных вариантов типов на выбор, компилятор OCaml должен применить некоторые эвристики, чтобы выбрать наиболее подходящий. Компилятор в данных ситуациях отдает предпочтение аргументам с метками перед необязательными аргументами и выбирает порядок аргументов, соответствующий порядку в исходном коде.

Имейте в виду, что в разных точках исходного кода применяемые эвристики могут давать разные варианты сигнатуры. Ниже приводится версия `numeric_deriv`, где в разных вызовах функции `f` аргументы передаются в разном порядке:

OCaml utop (part 56)

<https://github.com/realworldocaml/examples/blob/v1/code/variables-and-functions/main.topscript>

```
# let numeric_deriv ~delta ~x ~y ~f =
  let x' = x +. delta in
  let y' = y +. delta in
  let base = f ~x ~y in
  let dx = (f ~y ~x:x' -. base) /. delta in
  let dy = (f ~x ~y:y' -. base) /. delta in
  (dx,dy)
;;
```

Characters 130-131:

Error: This function is applied to arguments
in an order different from other calls.

This is only allowed when the real type is known.

*(Ошибка: Эта функция применяется к аргументам,
следующим в разном порядке в разных вызовах.*

Такое допустимо, только когда фактический тип известен.)

Как следует из сообщения об ошибке, мы можем заставить OCaml принять тот факт, что аргументы могут передаваться функции `f` в разном порядке, если явно указать информацию о типах. Так, следующий код компилируется без ошибок благодаря аннотациям типов в определении функции `f`:

OCaml utop (part 57)

<https://github.com/realworldocaml/examples/blob/v1/code/variables-and-functions/main.topscript>

```
# let numeric_deriv ~delta ~x ~y ~(f: x:float -> y:float -> float) =
  let x' = x +. delta in
  let y' = y +. delta in
```

```

    let base = f ~x ~y in
    let dx = (f ~y ~x:x' -. base) /. delta in
    let dy = (f ~x ~y:y' -. base) /. delta in
    (dx,dy)
;;
val numeric_deriv :
  delta:float ->
  x:float -> y:float -> f:(x:float -> y:float -> float) -> float * float =
  <fun>

```

Необязательные аргументы и частичное применение

Необязательные аргументы могут вызывать некоторые сложности при использовании приема частичного применения. Конечно, необязательные аргументы можно применять частично, если указывать их явно:

OCaml utop (part 58)

<https://github.com/realworldocaml/examples/blob/v1/code/variables-and-functions/main.topscript>

```

# let colon_concat = concat ~sep:"";;
val colon_concat : string -> string -> string = <fun>
# colon_concat "a" "b";;
- : string = "a:b"

```

Но что случится, если частично применить первый аргумент?

OCaml utop (part 59)

<https://github.com/realworldocaml/examples/blob/v1/code/variables-and-functions/main.topscript>

```

# let prepend_pound = concat "# ";;
val prepend_pound : string -> string = <fun>
# prepend_pound "a BASH comment";;
- : string = "# a BASH comment"

```

Необязательный аргумент `?sep` в этом случае оказался скрытым, или недоступным. Действительно, если теперь попытаться передать необязательный аргумент частично примененной функции, он будет отвергнут:

OCaml utop (part 60)

<https://github.com/realworldocaml/examples/blob/v1/code/variables-and-functions/main.topscript>

```

# prepend_pound "a BASH comment" ~sep:"";;
Characters 1-13:
Error: This function has type string -> string
      It is applied to too many arguments; maybe you forgot a `;'.
(Ошибка: Эта функция имеет тип string -> string.
      Ей передано слишком много аргументов; возможно, вы забыли ';'.)

```

Так что же заставило OCaml решить сделать необязательный аргумент недоступным?

Здесь действует следующее правило: необязательный аргумент становится недоступным, как только будет определен первый позиционный аргумент (то есть

без метки и обязательный), следующий за необязательным. Оно объясняет поведение `prepend_pound`. Но если бы мы в определении функции `concat` поставили необязательный аргумент во вторую позицию:

OCaml utop (part 61)

<https://github.com/realworldocaml/examples/blob/v1/code/variables-and-functions/main.topscript>

```
# let concat x ?(sep="") y = x ^ sep ^ y ;;
val concat : string -> ?sep:string -> string -> string = <fun>
```

применение первого аргумента не привело бы к сокрытию необязательного аргумента.

OCaml utop (part 62)

<https://github.com/realworldocaml/examples/blob/v1/code/variables-and-functions/main.topscript>

```
# let prepend_pound = concat "# ";;
val prepend_pound : ?sep:string -> string -> string = <fun>
# prepend_pound "a BASH comment";;
- : string = "# a BASH comment"
# prepend_pound "a BASH comment" ~sep:"--- ";;
- : string = "# --- a BASH comment"
```

Однако если передать в вызов функции все имеющиеся аргументы сразу, это не приводит к сокрытию необязательных аргументов, что сохраняет за нами возможность передавать необязательные аргументы в любой позиции в списке аргументов. То есть мы можем выполнить вызов:

OCaml utop (part 63)

<https://github.com/realworldocaml/examples/blob/v1/code/variables-and-functions/main.topscript>

```
# concat "a" "b" ~sep:"=";;
- : string = "a=b"
```

Необязательный аргумент в принципе не может стать недоступным, если за ним не следуют позиционные аргументы, однако в этом случае компилятор выводит предупреждение:

OCaml utop (part 64)

<https://github.com/realworldocaml/examples/blob/v1/code/variables-and-functions/main.topscript>

```
# let concat x y ?(sep="") = x ^ sep ^ y ;;
Characters 15-38:
Warning 16: this optional argument cannot be erased.
val concat : string -> string -> ?sep:string -> string = <fun>
(Предупреждение 16: этот необязательный аргумент не может быть скрыт.
val concat : string -> string -> ?sep:string -> string = <fun>)
```

И действительно, если этой функции передать два позиционных аргумента, необязательный аргумент `sep` не будет скрыт, вместо этого будет возвращена частично примененная функция, ожидающая получить аргумент `sep`:

OCaml utop (part 65)

<https://github.com/realworldocaml/examples/blob/v1/code/variables-and-functions/main.topscript>

```
# concat "a" "b";;  
- : ?sep:string -> string = <fun>
```

Как видите, поддержка аргументов с метками и необязательных аргументов в языке OCaml имеет некоторые сложности. Но все не так плохо, и эти две особенности действительно остаются очень полезными. Аргументы с метками и необязательные аргументы являются весьма эффективными инструментами создания более удобных и более безопасных API, а время, потраченное на изучение приемов эффективного их применения, не пропадет даром.

Глава 3

Списки и образцы

В этой главе мы сосредоточимся на двух элементах, чаще других используемых при программировании на языке OCaml: списках и сопоставлении с образцом. Оба они обсуждались в главе 1, но здесь мы исследуем их более подробно, используя их совместно, чтобы проиллюстрировать особенности друг друга.

ОСНОВЫ СПИСКОВ

Списки в языке OCaml являются неизменяемыми, конечными последовательностями элементов одного типа. Как вы уже знаете, списки в OCaml можно создавать с помощью квадратных скобок и точек с запятой:

OCaml utop

<https://github.com/realworldocaml/examples/tree/v1/code/lists-and-patterns/main.topscript>

```
# [1;2;3];;  
- : int list = [1; 2; 3]
```

и с использованием эквивалентной нотации :::

OCaml utop (part 1)

<https://github.com/realworldocaml/examples/tree/v1/code/lists-and-patterns/main.topscript>

```
# 1 :: (2 :: (3 :: [])) ;;  
- : int list = [1; 2; 3]  
# 1 :: 2 :: 3 :: [] ;;  
- : int list = [1; 2; 3]
```

Как видите, оператор :: является правоассоциативным, а это означает, что списки можно конструировать без применения круглых скобок. Пустой список [] используется здесь для обозначения конца списка. Отметьте, что пустой список является полиморфным, то есть может использоваться с элементами любого типа, например:

OCaml utop (part 2)

<https://github.com/realworldocaml/examples/tree/v1/code/lists-and-patterns/main.topscript>

```
# let empty = [];;  
val empty : 'a list = []  
# 3 :: empty;;  
- : int list = [3]  
# "three" :: empty;;  
- : string list = ["three"]
```

Способ, каким оператор `::` присоединяет элементы в начало списка, отражает тот факт, что списки в языке OCaml являются односвязными. На рис. 3.1 схематически изображено, как выглядит список `1 :: 2 :: 3 :: []` в виде структуры данных. Заключительная стрелка (из прямоугольника с числом 3) указывает на пустой список.



Рис. 3.1 ❖ Односвязный список как структура данных

Каждый оператор `::` фактически добавляет новый блок в цепочку. Каждый блок состоит из двух компонентов: ссылки на данные, составляющие элемент списка, и ссылки на оставшуюся часть списка. Это объясняет, как оператор `::` может наращивать список, не изменяя его; новый элемент списка сохраняется отдельно, без изменения существующих:

OCaml utop (part 3)

<https://github.com/realworldocaml/examples/tree/v1/code/lists-and-patterns/main.topscript>

```
# let l = 1 :: 2 :: 3 :: [];;
val l : int list = [1; 2; 3]
# let m = 0 :: 1;;
val m : int list = [0; 1; 2; 3]
# l;;
- : int list = [1; 2; 3]
```

Использование сопоставления с образцом для извлечения данных из списка

Читать данные из списка можно с помощью инструкции `match`. Ниже приводится простой пример рекурсивной функции, определяющей сумму всех элементов списка:

OCaml utop (part 4)

<https://github.com/realworldocaml/examples/tree/v1/code/lists-and-patterns/main.topscript>

```
# let rec sum l =
  match l with
  | [] -> 0
  | hd :: tl -> hd + sum tl
;;
val sum : int list -> int = <fun>
# sum [1;2;3];;
- : int = 6
# sum [];;
- : int = 0
```

Этот код следует соглашению об использовании имени `hd` для представления первого элемента (или головы (`head`)) списка и имени `tl` для представления оставшейся части (или хвоста (`tail`)) списка.

Инструкция `match` в функции `sum` в действительности выполняет две операции: во-первых, она действует как инструмент выбора, выделяя различные возможные варианты, и, во-вторых, дает возможность присвоить имена отдельным элементам исходной структуры данных. В данном случае переменные `hd` и `tl` связываются вторым образом в инструкции `match`. Переменные, связанные таким способом, можно использовать в выражении справа от стрелки, следующей за текущим образом.

Тот факт, что инструкцию `match` можно использовать для связывания новых переменных, может быть источником недопонимания. Давайте посмотрим, как. Представьте, что нам требуется написать функцию, отфильтровывающую из списка элементы, равные некоторому значению. У вас наверняка появится желание написать код, как показано ниже, но при попытке скомпилировать его компилятор сразу же выдаст предупреждение:

OCaml utop (part 5)

<https://github.com/realworldocaml/examples/blob/v1/code/lists-and-patterns/main.topscript>

```
# let rec drop_value l to_drop =
  match l with
  | [] -> []
  | to_drop :: tl -> drop_value tl to_drop
  | hd :: tl -> hd :: drop_value tl to_drop
;;
```

Characters 114-122:

Warning 11: this match case is unused.

```
val drop_value : 'a list -> 'a -> 'a list = <fun>
```

(Предупреждение 11: этот образец не используется)

```
val drop_value : 'a list -> 'a -> 'a list = <fun>
```

Более того, функция будет работать неправильно, отфильтровывая все элементы, а не только те, что равны указанному значению:

OCaml utop (part 6)

<https://github.com/realworldocaml/examples/blob/v1/code/lists-and-patterns/main.topscript>

```
# drop_value [1;2;3] 2;;
- : int list = []
```

Что не так?

Здесь важно понимать, что присутствие `to_drop` во втором образце не подразумевает сравнения первого элемента списка со значением аргумента `to_drop`, переданного функции `drop_value`. Вместо этого просто создается новая переменная `to_drop`. Она будет связана с первым элементом списка и скроет прежнее определение `to_drop`. Третий образец окажется неиспользуемым, потому что фактически тот же самый шаблон используется во втором образце.

Лучший выход из этой ситуации – вообще не использовать сопоставление с образцом для сравнения первого элемента списка с аргументом `to_drop`, а применить обычную инструкцию `if`:

OCaml utop (part 7)

<https://github.com/realworldocaml/examples/blob/v1/code/lists-and-patterns/main.topscript>

```
# let rec drop_value l to_drop =
  match l with
  | [] -> []
  | hd :: tl ->
    let new_tl = drop_value tl to_drop in
    if hd = to_drop then new_tl else hd :: new_tl
;;
val drop_value : 'a list -> 'a -> 'a list = <fun>
# drop_value [1;2;3] 2;;
- : int list = [1; 3]
```

Обратите внимание, что в случае, когда требуется отбросить конкретное литеральное значение (а не значение, переданное в переменной), можно использовать нечто, подобное первоначальной версии `drop_value`:

OCaml utop (part 8)

<https://github.com/realworldocaml/examples/blob/v1/code/lists-and-patterns/main.topscript>

```
# let rec drop_zero l =
  match l with
  | [] -> []
  | 0 :: tl -> drop_zero tl
  | hd :: tl -> hd :: drop_zero tl
;;
val drop_zero : int list -> int list = <fun>
# drop_zero [1;2;0;3];;
- : int list = [1; 2; 3]
```

Ограничения (и благословения) сопоставления с образцом

Предыдущий пример наглядно демонстрирует, что сопоставление с образцом не может использоваться для выражения произвольных условий. Образцы могут описывать организацию структуры данных и даже включать литералы, как в примере реализации функции `drop_zero`, но на этом круг их возможностей ограничивается. С помощью образца можно проверить, содержит ли список два элемента, но нельзя проверить равенство первых двух элементов.

Образцы можно рассматривать как специализированный подязык, на котором можно выразить ограниченное (но достаточно богатое) множество условий. Ограниченность языка образцов одновременно является и благословением, упрощая их поддержку и оптимизацию в компиляторе. В частности, эффективность

инструкций `match` и способность компилятора обнаруживать ошибки во многом обусловлены ограниченной природой образцов.

Производительность

По простоте душевной можно было бы подумать, что инструкция `match` последовательно проверяет все образцы, чтобы найти совпадение. В ситуациях, когда образцы снабжаются произвольным ограничивающим кодом, так и есть. Но чаще компилятор OCaml оказывается в состоянии сгенерировать машинный код, сразу выполняющий переход к нужной ветви, опираясь на множество простых проверок во время выполнения.

Например, взгляните на следующие, немного дурацкие функции, увеличивающие целое число на единицу. Первая реализована на основе инструкции `match`, а вторая – в виде последовательности инструкций `if`:

OCaml utop (part 9)

<https://github.com/realworldocaml/examples/blob/v1/code/lists-and-patterns/main.topscript>

```
# let plus_one_match x =
  match x with
  | 0 -> 1
  | 1 -> 2
  | 2 -> 3
  | _ -> x + 1

let plus_one_if x =
  if x = 0 then 1
  else if x = 1 then 2
  else if x = 2 then 3
  else x + 1
;;

val plus_one_match : int -> int = <fun>
val plus_one_if : int -> int = <fun>
```

Обратите внимание на образец `_` в первой функции. Это шаблонный образец – он совпадает с любым значением, но не связывает совпавшее значение ни с каким именем переменной.

Если проверить производительность этих функций, можно заметить, что `plus_one_if` выполняется намного медленнее, чем `plus_one_match`, причем превосходство возрастает с увеличением возможных вариантов. Следующий фрагмент определяет производительность этих функций с помощью библиотеки `core_bench`, которую можно установить командой `opam install core_bench`, выполнив ее в командной строке:

OCaml utop (part 10)

<https://github.com/realworldocaml/examples/blob/v1/code/lists-and-patterns/main.topscript>

```
# #require "core_bench";;
# open Core_bench.Std;;
```

```
# let run_bench tests =
  Bench.bench
    ~ascii_table:true
    ~display:Textutils.Ascii_table.Display.column_titles
    tests
;;
val run_bench : Bench.Test.t list -> unit = <fun>
# [ Bench.Test.create ~name:"plus_one_match" (fun () ->
  ignore (plus_one_match 10))
  ; Bench.Test.create ~name:"plus_one_if" (fun () ->
  ignore (plus_one_if 10)) ]
|> run_bench
;;
Estimated testing time 20s (change using -quota SECS).
```

Name	Time (ns)	% of max
plus_one_match	46.81	68.21
plus_one_if	68.63	100.00

```
- : unit = ()
```

Вот вам другой, менее искусственный пример. Мы можем реализовать функцию `sum`, о которой писалось выше в этой главе, с использованием инструкции `if` вместо `match`, и применить в ней функции `is_empty`, `hd_exn` и `tl_exn` из модуля `List` для деконструкции списка:

OCaml utop (part 11)

<https://github.com/realworldocaml/examples/blob/v1/code/lists-and-patterns/main.topscript>

```
# let rec sum_if l =
  if List.is_empty l then 0
  else List.hd_exn l + sum_if (List.tl_exn l)
;;
val sum_if : int list -> int = <fun>
```

Давайте еще раз проверим производительность, чтобы увидеть разницу:

OCaml utop (part 12)

<https://github.com/realworldocaml/examples/blob/v1/code/lists-and-patterns/main.topscript>

```
# let numbers = List.range 0 1000 in
[ Bench.Test.create ~name:"sum_if" (fun () -> ignore (sum_if numbers))
  ; Bench.Test.create ~name:"sum" (fun () -> ignore (sum numbers)) ]
|> run_bench
;;
Estimated testing time 20s (change using -quota SECS).
```

Name	Time (ns)	% of max
sum_if	110_535	100.00
sum	22_361	20.23

```
- : unit = ()
```

На этот раз преимущество реализации на основе инструкции `match` оказалось намного разительнее. Большая разница обусловлена необходимостью выполнять одну и ту же работу много раз, так как при каждом вызове функции приходится повторно анализировать первый элемент списка, чтобы определить, является ли он пустой ячейкой. В инструкции `match` эта работа выполняется точно один раз на элемент списка.

Вообще говоря, сопоставление с образцом действует эффективнее любого другого кода, который вы только сможете придумать. Известным исключением является сопоставление со строками, которые проверяются последовательно, из-за чего инструкция `match` с длинной последовательностью строк может проигрывать приему на основе хэш-таблицы. Но в большинстве других случаев сопоставление с образцом выходит победителем.

Определение ошибок

Однако более важной особенностью, чем производительность, является способность инструкции `match` обнаруживать ошибки. Мы уже видели пример способности языка OCaml обнаруживать проблемы в операциях сопоставления с образцом: в ошибочной реализации `drop_value`, когда компилятор OCaml предупредил нас, что последний образец является избыточным. Не существует алгоритмов, которые могли бы определить избыточность предиката, написанного на универсальном языке, но в контексте образцов эта задача решается достаточно надежно.

Компилятор OCaml также проверяет инструкции `match` на полноту. Взгляните, что произойдет, если в реализации `drop_zero` удалить одну из ветвей:

OCaml utop (part 13)

<https://github.com/realworldocaml/examples/blob/v1/code/lists-and-patterns/main.topscript>

```
# let rec drop_zero l =
  match l with
  | [] -> []
  | 0 :: tl -> drop_zero tl
;;
```

Characters 26-84:

Warning 8: this pattern-matching is not exhaustive.

Here is an example of a value that is not matched:

```
1::_
```

(Предупреждение 8: это сопоставление не является исчерпывающим.

Нижe следует пример значения, не соответствующего сопоставлению:

```
1::_)
```

```
val drop_zero : int list -> 'a list = <fun>
```

Как видите, компилятор сгенерировал предупреждение, сообщив, что мы пропустили возможный вариант, а также привел пример вероятного образца.

Даже в таких простых ситуациях, как эта, проверка на полноту может оказаться весьма полезной. Но, как будет показано в главе 6, ценность этой проверки увеличивается еще больше с ростом сложности примеров, особенно когда в работу включаются типы данных, определяемые пользователем. Помимо вылавливания

ошибок, эти проверки могут играть роль своеобразного инструмента рефакторинга, направляя вас к фрагментам кода, где требуется изменить код, чтобы привести его в соответствие с изменением типов.

Эффективное использование модуля List

К настоящему моменту мы написали немало кода, использующего сопоставление с образцом и рекурсивные функции для обработки списков. Но в реальной жизни вы чаще будете использовать для этих целей модуль `List`, который битком набит функциями, реализующими наиболее типичные операции со списками.

Давайте разберем конкретный пример и посмотрим, как применять этот модуль на практике. Напишем функцию `render_table`, принимающую список заголовков столбцов и список строк, которая будет выводить их в виде отформатированной таблицы:

OCaml utop (part 69)

<https://github.com/realworldocaml/examples/blob/v1/code/lists-and-patterns/main.topscript>

```
# printf "%s\n"
(render_table
  ["language"; "architect"; "first release"]
  [ ["Lisp" ; "John McCarthy" ; "1958"] ;
    ["C"    ; "Dennis Ritchie" ; "1969"] ;
    ["ML"   ; "Robin Milner"   ; "1973"] ;
    ["OCaml"; "Xavier Leroy"   ; "1996"] ;
  ]);;
| language | architect      | first release |
|-----+-----+-----|
| Lisp     | John McCarthy | 1958          |
| C        | Dennis Ritchie | 1969          |
| ML       | Robin Milner  | 1973          |
| OCaml    | Xavier Leroy  | 1996          |
- : unit = ()
```

Первое, что нужно сделать, – написать функцию, определяющую максимальную ширину каждой колонки. Для этого можно преобразовать заголовок и все строки данных в список целых чисел, отражающих длину, а затем выбрать самый большой элемент. Реализовать весь необходимый код вручную будет довольно трудоемкой задачей, но мы можем избавить себя от лишних сложностей, воспользовавшись тремя функциями из модуля `List`: `map`, `map2_exn` и `fold`.

Описать действие функции `List.map` проще простого. Она принимает список и функцию для преобразования элементов этого списка и возвращает новый список с преобразованными элементами. То есть мы можем записать:

OCaml utop (part 14)

<https://github.com/realworldocaml/examples/blob/v1/code/lists-and-patterns/main.topscript>

```
# List.map ~f:String.length ["Hello"; "World!"];;
- : int list = [5; 6]
```

Функция `List.map2_exn` похожа на `List.map`, за исключением того, что она принимает два списка и функцию для их объединения. То есть мы можем записать:

OCaml utop (part 15)

<https://github.com/realworldocaml/examples/blob/v1/code/lists-and-patterns/main.topscript>

```
# List.map2_exn ~f:Int.max [1;2;3] [3;2;1];;
- : int list = [3; 2; 3]
```

Суффикс `_exn` здесь обозначает, что функция возбудит исключение, если списки будут иметь разную длину:

OCaml utop (part 16)

<https://github.com/realworldocaml/examples/blob/v1/code/lists-and-patterns/main.topscript>

```
# List.map2_exn ~f:Int.max [1;2;3] [3;2;1;0];;
Exception: (Invalid_argument "length mismatch in rev_map2_exn: 3 <> 4").
```

Функция `List.fold` самая сложная из этой троицы. Она принимает три аргумента: обрабатываемый список, начальное значение аккумулятора и функцию для обновления аккумулятора. Функция `List.fold` выполняет обход элементов списка слева направо, на каждом шаге обновляет аккумулятор и в конце возвращает получившееся значение аккумулятора. Кое-что из этого можно увидеть, взглянув на сигнатуру функции `fold`:

OCaml utop (part 17)

<https://github.com/realworldocaml/examples/blob/v1/code/lists-and-patterns/main.topscript>

```
# List.fold;;
- : 'a list -> init:'accum -> f:('accum -> 'a -> 'accum) -> 'accum = <fun>
```

Функцию `List.fold` можно использовать как своеобразный сумматор для списка:

OCaml utop (part 18)

<https://github.com/realworldocaml/examples/blob/v1/code/lists-and-patterns/main.topscript>

```
# List.fold ~init:0 ~f:(+) [1;2;3;4];;
- : int = 10
```

Этот пример получился таким простым потому, что аккумулятор и элементы списка принадлежат одному типу. Но функция `fold` не ограничена такими ситуациями. Функцию `fold` можно, к примеру, задействовать для перестановки элементов списка в обратном порядке, в этом случае аккумулятор сам будет являться списком:

OCaml utop (part 19)

<https://github.com/realworldocaml/examples/blob/v1/code/lists-and-patterns/main.topscript>

```
# List.fold ~init:[] ~f:(fun list x -> x :: list) [1;2;3;4];;
- : int list = [4; 3; 2; 1]
```

Давайте объединим эти три функции для поиска максимальной ширины каждой колонки:

OCaml utop (part 20)

<https://github.com/realworldocaml/examples/blob/v1/code/lists-and-patterns/main.topscript>

```
# let max_widths header rows =
  let lengths l = List.map ~f:String.length l in
  List.fold rows
    ~init:(lengths header)
    ~f:(fun acc row ->
      List.map2_exn ~f:Int.max acc (lengths row))
;;
val max_widths : string list -> string list list -> int list = <fun>
```

С помощью `List.map` мы определили функцию `lengths`, которая преобразует список строк в список целочисленных длин. Затем мы использовали `List.fold` для обхода строк и с помощью `map2_exn` определили максимальное значение аккумулятора длин значений в каждой строке таблицы, при этом аккумулятор инициализировался длинами заголовков.

Теперь, когда ширина каждой колонки известна, можно приступить к коду, генерирующему строку, отделяющую заголовок от остальной части таблицы. Мы сделаем это с помощью `String.make`, отобразив целочисленные значения в строки из дефисов соответствующей длины. Затем объединим эти строки дефисов с помощью `String.concat`, которая выполняет конкатенацию строк через необязательный разделитель, и с помощью оператора `^` добавим внешние ограничители колонок:

OCaml utop (part 21)

<https://github.com/realworldocaml/examples/blob/v1/code/lists-and-patterns/main.topscript>

```
# let render_separator widths =
  let pieces = List.map widths
    ~f:(fun w -> String.make (w + 2) '-')
  in
  "|" ^ String.concat ~sep:"+" pieces ^ "|"
;;
val render_separator : int list -> string = <fun>
# render_separator [3;6;2];;
- : string = "|-----+-----+-----|"
```

Обратите внимание, что строку дефисов мы сделали на два символа длиннее, чтобы обеспечить наличие не менее одного пробела с каждой стороны записи в таблице.

Производительность `String.concat` и `^`

В предыдущем примере мы объединяли строки двумя разными способами: с помощью функции `String.concat`, которая принимает список строк, и оператора `^`, осуществляющего объединение пары строк. Старайтесь избегать использования оператора `^` для объединения большого числа строк, потому что каждый раз он размещает в памяти новую строку. То есть следующий код

```
# let s = "." ^ "." ^ "." ^ "." ^ "." ^ "." ^ "." ^ ".";;
val s : string = "....."
```

создаст строки с длиной 2, 3, 4, 5, 6 и 7 символов, тогда как следующий код

<https://github.com/realworldocaml/examples/blob/v1/code/lists-and-patterns/main.topscript>

```
# let s = String.concat [".";".";".";".";".";".";"."];;
val s : string = "....."
```

создаст единственную строку длиной 7 символов, а также список с 7 элементами. При таких небольших размерах разница в производительности оказывается небольшой, но при сборке длинных строк применение оператора может существенно снизить скорость выполнения программы.

Теперь нам нужно реализовать вывод одной строки в таблице. Сначала напомним функцию с именем `pad` для дополнения строки пробелами до указанной длины плюс по одному пробелу с каждой стороны:

<https://github.com/realworldocaml/examples/blob/v1/code/lists-and-patterns/main.topscript>

```
# let pad s length =
  " " ^ s ^ String.make (length - String.length s + 1) ' '
;;
val pad : string -> int -> string = <fun>
# pad "hello" 10;;
- : string = " hello "
```

Сконструировать строку таблицы можно путем объединения отдельных строк в колонках. И снова воспользуемся функцией `List.map2_exn` для объединения списка с данными в строке таблицы со списком размеров колонок:

<https://github.com/realworldocaml/examples/blob/v1/code/lists-and-patterns/main.topscript>

```
# let render_row row widths =
  let padded = List.map2_exn row widths ~f:pad in
  ~| " ^ String.concat ~sep:"|" padded ^ "|"
;;

val render_row : string list -> int list -> string = <fun>
# render_row ["Hello";"World"] [10;15];;
- : string = "| Hello | World |"
```

Теперь можно объединить все вместе в одну функцию, осуществляющую отображение таблицы:

<https://github.com/realworldocaml/examples/blob/v1/code/lists-and-patterns/main.topscrip>

```
# let render_table header rows =
  let widths = max widths header rows in
```



```

String.concat ~sep:"\n"
(render_row header widths
:: render_separator widths
:: List.map rows ~f:(fun row -> render_row row widths)
)
;;
val render_table : string list -> string list list -> string = <fun>

```

Другие полезные функции из модуля List

В предыдущем примере мы использовали всего три функции из модуля List. Мы не можем позволить себе привести здесь полное описание всего интерфейса модуля (для этого вам следует обратиться к электронной документации), но некоторые особенно полезные функции отметим.

Объединение элементов списка с помощью List.reduce

Функция List.fold, описанная выше, — очень мощная и универсальная функция. Но иногда желательно иметь инструмент попроще. Одним из таких инструментов является функция List.reduce. По сути, это специализированная версия функции List.fold, не требующая явно указывать начальное значение. Она создает аккумулятор, тип которого соответствует типу элементов обрабатываемого списка.

Вот как выглядит ее сигнатура:

OCaml utop (part 27)

<https://github.com/realworldocaml/examples/blob/v1/code/lists-and-patterns/main.topscript>

```

# List.reduce;;
- : 'a list -> f:('a -> 'a -> 'a) -> 'a option = <fun>

```

Функция reduce имеет необязательное возвращаемое значение, то есть для пустого списка она вернет None.

Теперь посмотрим, как действует функция reduce:

OCaml utop (part 28)

<https://github.com/realworldocaml/examples/blob/v1/code/lists-and-patterns/main.topscript>

```

# List.reduce ~f:(+) [1;2;3;4;5];;
- : int option = Some 15
# List.reduce ~f:(+) [];;
- : int option = None

```

Фильтрация с помощью List.filter и List.filter_map

Очень часто при обработке списков бывает необходимо сосредоточить внимание лишь на подмножестве элементов списка. Такую возможность дает функция List.filter:

OCaml utop (part 29)

<https://github.com/realworldocaml/examples/blob/v1/code/lists-and-patterns/main.topscript>

```

# List.filter ~f:(fun x -> x mod 2 = 0) [1;2;3;4;5];;
- : int list = [2; 4]

```

Обратите внимание, что `mod` используется здесь в качестве инфиксного оператора, как описывалось в главе 2.

Иногда может потребоваться одновременно преобразовать и отфильтровать элементы списка. В этом случае вам поможет функция `List.filter_map`. Функция, передаваемая в вызов `List.filter_map`, должна возвращать необязательное значение, а `List.filter_map` отбросит все элементы, для которых будет получено значение `None`.

Следующее выражение составляет список расширений имен файлов, присутствующих в текущем каталоге, и передает его функции `List.dedup` для удаления повторяющихся значений. Обратите внимание, что в этом примере используются также функции из других модулей, включая `Sys.ls_dir`, возвращающую список содержимого каталога, и `String.rsplit2`, разбивающую строку по самому правому указанному символу:

OCaml utop (part 30)

<https://github.com/realworldocaml/examples/blob/v1/code/lists-and-patterns/main.topscript>

```
# List.filter_map (Sys.ls_dir ".") ~f:(fun fname ->
  match String.rsplit2 ~on:'.' fname with
  | None | Some ("",_) -> None
  | Some (_,ext) ->
    Some ext)
|> List.dedup
;;
- : string list = ["ascii"; "ml"; "mli"; "topscript"]
```

Предыдущий код может также служить примером применения ИЛИ-шаблона, позволяющего указывать несколько образцов в одной ветви инструкции `match`. В данном случае таким шаблоном является образец `None | Some ("",_)`. Как будет показано далее, ИЛИ-шаблоны могут присутствовать в любом месте в больших образцах.

Деление списков с помощью `List.partition_tf`

Еще одной полезной операцией, тесно связанной с фильтрацией, является операция деления списка. Функция `List.partition_tf` принимает список и функцию, возвращающую логическое значение для каждого элемента списка, и создает два списка. Суффикс `tf` в имени служит напоминанием для забывчивых пользователей, что элементы, для которых указанная функция вернула `true`, помещаются в первый из возвращаемых списков, а элементы, получившие `false`, — во второй. Например:

OCaml utop (part 31)

<https://github.com/realworldocaml/examples/blob/v1/code/lists-and-patterns/main.topscript>

```
# let is_ocaml_source s =
  match String.rsplit2 s ~on:'.' with
  | Some (_,("ml"|"mli")) -> true
  | _ -> false
```

```
;;
val is_ocaml_source : string -> bool = <fun>
# let (ml_files, other_files) =
    List.partition_tf (Sys.ls_dir ".") ~f:is_ocaml_source;;
val ml_files : string list = ["example.mli"; "example.ml"]
val other_files : string list = ["main.topscrip"; "lists_layout.ascii"]
```

Комбинирование списков

Другая распространенная операция над списками – конкатенация. В действительности модуль `List` позволяет выполнять эту операцию несколькими разными способами. Во-первых, существует функция `List.append`, выполняющая объединение двух списков:

OCaml utop (part 32)

<https://github.com/realworldocaml/examples/tree/v1/code/lists-and-patterns/main.topscrip>

```
# List.append [1;2;3] [4;5;6];;
- : int list = [1; 2; 3; 4; 5; 6]
```

Имеется также оператор `@`, действующий подобно функции `List.append`:

OCaml utop (part 33)

<https://github.com/realworldocaml/examples/tree/v1/code/lists-and-patterns/main.topscrip>

```
# [1;2;3] @ [4;5;6];;
- : int list = [1; 2; 3; 4; 5; 6]
```

Конкатенацию списков в списке можно выполнить с помощью функции `List.concat`:

OCaml utop (part 34)

<https://github.com/realworldocaml/examples/tree/v1/code/lists-and-patterns/main.topscrip>

```
# List.concat [[1;2];[3;4;5];[6];[]];;
- : int list = [1; 2; 3; 4; 5; 6]
```

Ниже приводится пример использования функции `List.concat` в паре с `List.map` для создания рекурсивного списка дерева каталогов:

OCaml utop (part 35)

<https://github.com/realworldocaml/examples/tree/v1/code/lists-and-patterns/main.topscrip>

```
# let rec ls_rec s =
    if Sys.is_file_exn ~follow_symlinks:true s
    then [s]
    else
        Sys.ls_dir s
        |> List.map ~f:(fun sub -> ls_rec (s ^/ sub))
        |> List.concat
;;
val ls_rec : string -> string list = <fun>
```

Обратите внимание на инфиксный оператор `^/` из библиотеки `Core`. Он добавляет новый элемент в строку пути к файлу. Этот оператор является эквивалентом функции `Filename.concat` из библиотеки `Core`.

Комбинация функций `List.map` и `List.concat`, представленная выше, используется настолько часто, что специально была создана функция `List.concat_map`, объединяющая эти две функции в одну, но действующая более эффективно:

OCaml utop (part 36)

<https://github.com/realworldocaml/examples/tree/v1/code/lists-and-patterns/main.topscript>

```
# let rec ls_rec s =
  if Sys.is_file_exn ~follow_symlinks:true s
  then [s]
  else
    Sys.ls_dir s
    |> List.concat_map ~f:(fun sub -> ls_rec (s ^/ sub))
;;
val ls_rec : string -> string list = <fun>
```

Хвостовая рекурсия

Единственный способ найти длину списка в языке OCaml – выполнить обход списка от начала до конца. Как результат продолжительность такой операции находится в линейной зависимости от длины списка. Ниже приводится пример реализации функции, выполняющей эту операцию:

OCaml utop (part 37)

<https://github.com/realworldocaml/examples/tree/v1/code/lists-and-patterns/main.topscript>

```
# let rec length = function
  | [] -> 0
  | _ :: t1 -> 1 + length t1
;;
val length : 'a list -> int = <fun>
# length [1;2;3];;
- : int = 3
```

Она выглядит достаточно незамысловатой, но, применив ее к очень длинному списку, вы можете столкнуться с проблемой, как показано в следующем примере:

OCaml utop (part 38)

<https://github.com/realworldocaml/examples/tree/v1/code/lists-and-patterns/main.topscript>

```
# let make_list n = List.init n ~f:(fun x -> x);;
val make_list : int -> int list = <fun>
# length (make_list 10);;
- : int = 10
# length (make_list 10_000_000);;
Stack overflow during evaluation (looping recursion?).
```

Этот пример создает списки с помощью `List.init`, которая принимает целое число `n` и функцию `f` и создает список длиной `n`, каждый элемент которого создается вызовом `f` с индексом этого элемента.

Чтобы понять, где кроется ошибка в примере выше, нужно знать немного больше о том, как происходят вызовы функций. Обычно в момент вызова функции в памяти резервируется некоторое пространство, где сохраняется информация о вызове, такая как значения переданных аргументов или ссылка на код, которому следует передать управление по завершении функции. Для поддержки вложенных вызовов такая информация, как правило, организована в виде стека, где для каждого вложенного вызова выделяется новый *кадр стека*, который затем освобождается по завершении работы функции.

Именно в выделении новых кадров стека и заключена проблема, проявляющаяся в функции `length`: она пытается выделить 10 миллионов кадров стека, полностью исчерпывая доступную память стека. К счастью, эта проблема имеет решение. Взгляните на следующую, альтернативную реализацию:

OCaml utop (part 39)

<https://github.com/realworldocaml/examples/tree/v1/code/lists-and-patterns/main.topscript>

```
# let rec length_plus_n l n =
  match l with
  | [] -> n
  | _ :: tl -> length_plus_n tl (n + 1)
;;
val length_plus_n : 'a list -> int -> int = <fun>
# let length l = length_plus_n l 0 ;;
val length : 'a list -> int = <fun>
# length [1;2;3;4];;
- : int = 4
```

Эта реализация опирается на вспомогательную функцию `length_plus_n`, которая вычисляет длину указанного списка и прибавляет указанное число `n`. Число `n` выступает здесь в роли аккумулятора, в котором шаг за шагом накапливается ответ. Благодаря такому подходу мы можем выполнять сложение в цикле вместо раскручивания последовательности вложенных вызовов функций, как в первой реализации `length`.

Преимущество такого решения состоит в том, что рекурсивный вызов в `length_plus_n` оказывается последней инструкцией в функции, или, как говорят, «хвостовым» вызовом. Подробнее о том, что это означает, мы расскажем чуть ниже, но самое важное следствие этого – отсутствие необходимости выделять новый кадр стека для хвостового вызова благодаря так называемой *оптимизации хвостовых вызовов*. Говорят, что функция выполняет хвостовую рекурсию, если все рекурсивные вызовы в ней являются хвостовыми. Функция `length_plus_n` действительно выполняет хвостовую рекурсию, и как результат `length` может обрабатывать очень длинные списки без угрозы исчерпать пространство на стеке:

OCaml utop (part 40)

<https://github.com/realworldocaml/examples/tree/v1/code/lists-and-patterns/main.topscript>

```
# length (make_list 10_000_000) ;;
- : int = 10000000
```

Итак, какие вызовы считаются хвостовыми? Представьте ситуацию, когда одна функция (*вызывающая*) вызывает другую (*вызываемую*). Вызов считается хвостовым, если вызывающая функция ничего не делает со значением, возвращаемым вызываемой функцией, кроме того что просто возвращает его. Суть оптимизации хвостовых вызовов заключается в том, что когда вызывающая функция производит хвостовой вызов, отпадает необходимость сохранять ее кадр стека. Поэтому вместо выделения нового кадра стека для вызываемой функции компилятор использует кадр стека вызывающей функции.

Хвостовая рекурсия важна не только при обработке списков. Обычная нехвостовая рекурсия имеет смысл при обработке структур данных, таких как двоичные деревья, когда глубина дерева является логарифмической функцией от объема данных. Но при работе с линейными структурами, когда глубина рекурсии линейно зависит от объема обрабатываемых данных, хвостовая рекурсия часто обеспечивает более оптимальное решение.

Компактность и скорость сопоставления с образцом

Теперь, когда мы знаем намного больше об особенностях работы со списками и применения сопоставления с образцом, посмотрим, как можно усовершенствовать пример из раздела «Рекурсивные функции для списков» в главе 1: реализацию функции `destutter`, которая удаляет смежные повторяющиеся значения из списка. Ниже приводится реализация, представленная раньше:

OCaml utop (part 41)

<https://github.com/realworldocaml/examples/tree/v1/code/lists-and-patterns/main.topscript>

```
# let rec destutter list =
  match list with
  | [] -> []
  | [hd] -> [hd]
  | hd :: hd' :: tl ->
    if hd = hd' then destutter (hd' :: tl)
    else hd :: destutter (hd' :: tl)
;;
val destutter : 'a list -> 'a list = <fun>
```

Рассмотрим несколько способов сделать этот код более компактным и более эффективным.

Сначала займемся увеличением эффективности. Одна из проблем реализации `destutter`, представленной выше, состоит в том, что в некоторых ветках инструк-

ции `match` она повторно создает значения справа от стрелок, уже имеющиеся слева. То есть образец `[hd] -> [hd]` фактически размещает в памяти новый элемент списка, хотя достаточно было бы просто вернуть совпавший список. Мы можем уменьшить число операций выделения памяти, воспользовавшись образцом `as`, который дает возможность объявить имя для совпадения с образцом. Пока мы еще не ушли далеко, добавим также ключевое слово `function`, чтобы избавиться от необходимости явно использовать инструкцию `match`:

OCaml utop (part 42)

<https://github.com/realworldocaml/examples/tree/v1/code/lists-and-patterns/main.topscript>

```
# let rec destutter = function
  | [] as l -> l
  | [_] as l -> l
  | hd :: (hd' :: _ as tl) ->
    if hd = hd' then destutter tl
    else hd :: destutter tl
;;
val destutter : 'a list -> 'a list = <fun>
```

Можно пойти еще дальше и объединить две первые ветви в одну, применив ИЛИ-шаблон:

OCaml utop (part 43)

<https://github.com/realworldocaml/examples/tree/v1/code/lists-and-patterns/main.topscript>

```
# let rec destutter = function
  | [] | [_] as l -> l
  | hd :: (hd' :: _ as tl) ->
    if hd = hd' then destutter tl
    else hd :: destutter tl
;;
val destutter : 'a list -> 'a list = <fun>
```

Этот код можно сделать немного быстрее, добавив в него выражение `when`. Выражение `when` позволяет добавлять в образцы дополнительные предварительные условия в виде произвольных выражений на языке OCaml. В данном случае с помощью `when` можно реализовать проверку равенства двух первых элементов:

OCaml utop (part 44)

<https://github.com/realworldocaml/examples/tree/v1/code/lists-and-patterns/main.topscript>

```
# let rec destutter = function
  | [] | [_] as l -> l
  | hd :: (hd' :: _ as tl) when hd = hd' -> destutter tl
  | hd :: tl -> hd :: destutter tl
;;
val destutter : 'a list -> 'a list = <fun>
```

Полиморфное сравнение

В предыдущем примере функции `destutter` мы использовали тот факт, что OCaml позволяет использовать оператор `=` для сравнения значений любого типа. То есть мы можем записать:

OCaml utoп (part 45)

<https://github.com/realworldocaml/examples/tree/v1/code/lists-and-patterns/main.topscript>

```
# 3 = 4;;
- : bool = false
# [3;4;5] = [3;4;5];;
- : bool = true
# [Some 3; None] = [None; Some 3];;
- : bool = false
```

И действительно, если посмотреть на тип оператора сравнения, можно увидеть, что он является полиморфным:

OCaml utoп (part 46)

<https://github.com/realworldocaml/examples/tree/v1/code/lists-and-patterns/main.topscript>

```
# (=);;
- : 'a -> 'a -> bool = <fun>
```

В языке OCaml имеется целое семейство полиморфных операторов сравнения, включая стандартные инфиксные операторы `<`, `>=` и другие, а также функцию `compare`, возвращающую `-1`, `0` или `1`, если первый операнд меньше, равен или больше второго операнда соответственно.

Кто-то из вас может спросить: «Как можно было бы определить подобные функции самостоятельно, если бы они отсутствовали в языке OCaml?» Как оказывается, это невозможно. Полиморфные функции сравнения в языке OCaml встроены в окружение времени выполнения (runtime) и реализованы на низкоуровневом языке. Полиморфизм этих функций сравнения достигается за счет практически полного игнорирования информации о типах сравниваемых значений и принятия во внимание только их организации в памяти.

Полиморфное сравнение имеет некоторые ограничения. Например, оно вызывает ошибку при попытке сравнения функций:

OCaml utoп (part 47)

<https://github.com/realworldocaml/examples/tree/v1/code/lists-and-patterns/main.topscript>

```
# (fun x -> x + 1) = (fun x -> x + 1);;
Exception: (Invalid_argument "equal: functional value").
```

Точно так же ошибку вызовет попытка сравнить значения, находящиеся за пределами области динамической памяти (heap) OCaml, например значения в библиотеках на C. Но для других видов значений сравнение выполняется безупречно.

Для простых атомарных типов полиморфное сравнение имеет вполне ожидаемую семантику: для вещественных и целых чисел полиморфное сравнение соответствует числовому сравнению. Для строк применяется лексикографическое сравнение.

Однако иногда игнорирование типа при полиморфном сравнении может вызывать проблемы, особенно когда у вас имеются собственные представления о равенстве и о порядке следования значений, которые вам хотелось бы выразить. Мы обсудим эту и некоторые другие проблемы полиморфного сравнения в главе 13.

Имейте в виду, что `when` имеет некоторые недостатки. Как отмечалось выше, статические проверки, выполняемые при сопоставлении с образцами, опираются на тот факт, что язык определения образцов довольно ограничен. Стоит только

добавить поддержку произвольных условий в образцах, как будет что-то утрачено. Например, в число утрат может попасть возможность компилятора проверять полноту охвата возможных вариантов или избыточность некоторых образцов.

Рассмотрим следующую функцию, принимающую список необязательных значений (значений типа `option`) и возвращающую число значений `Some`. Поскольку в этой реализации используются предложения `when`, компилятор оказывается не в состоянии надежно определить, что код является исчерпывающим:

OCaml utop (part 48)

<https://github.com/realworldocaml/examples/tree/v1/code/lists-and-patterns/main.topscript>

```
# let rec count_some list =
  match list with
  | [] -> 0
  | x :: tl when Option.is_none x -> count_some tl
  | x :: tl when Option.is_some x -> 1 + count_some tl
;;
```

Characters 30-169:

Warning 8: this pattern-matching is not exhaustive.

Here is an example of a value that is not matched:

```
_::__
(However, some guarded clause may match this value.)
(Предупреждение 8: это сопоставление не является исчерпывающим.
Ниже следует пример значения, не соответствующего сопоставлению:
_::__
(Однако некоторые образцы с ограничителями могут совпадать с этим значением.))
val count_some : 'a option list -> int = <fun>
```

Несмотря на предупреждение, функция действует великолепно:

OCaml utop (part 49)

<https://github.com/realworldocaml/examples/tree/v1/code/lists-and-patterns/main.topscript>

```
# count_some [Some 3; None; Some 4];;
- : int = 2
```

Если добавить еще один образец без ограничивающего выражения `when`, компилятор перестанет выдавать предупреждение, но и не будет сообщать об избыточности кода.

OCaml utop (part 50)

<https://github.com/realworldocaml/examples/tree/v1/code/lists-and-patterns/main.topscript>

```
# let rec count_some list =
  match list with
  | [] -> 0
  | x :: tl when Option.is_none x -> count_some tl
  | x :: tl when Option.is_some x -> 1 + count_some tl
  | x :: tl -> -1 (* никогда не будет выполняться *)
;;
val count_some : 'a option list -> int = <fun>
```

Пожалуй, лучшим решением в данной ситуации будет просто отбросить второе выражение `when`:

OCaml utop (part 51)

<https://github.com/realworldocaml/examples/tree/v1/code/lists-and-patterns/main.topscript>

```
# let rec count_some list =
  match list with
  | [] -> 0
  | x :: tl when Option.is_none x -> count_some tl
  | _ :: tl -> 1 + count_some tl
;;
val count_some : 'a option list -> int = <fun>
```

Такое решение менее очевидно, чем решение, основанное на непосредственном сопоставлении, где каждый образец был описан явно:

OCaml utop (part 52)

<https://github.com/realworldocaml/examples/tree/v1/code/lists-and-patterns/main.topscript>

```
# let rec count_some list =
  match list with
  | [] -> 0
  | None :: tl -> count_some tl
  | Some _ :: tl -> 1 + count_some tl
;;
val count_some : 'a option list -> int = <fun>
```

Из всего сказанного следует, что, несмотря на удобство выражений `when`, все же следует отдавать предпочтение простым образцам, если их вполне достаточно.

И последнее замечание. Реализация функции `count_some`, представленная выше, длиннее, чем необходимо; хуже того, в ней используется обычная (нехвостовая) рекурсия. На практике лучше использовать для этих же целей функцию `List.count` из библиотеки `Core`:

OCaml utop (part 53)

<https://github.com/realworldocaml/examples/tree/v1/code/lists-and-patterns/main.topscript>

```
# let count_some l = List.count ~f:Option.is_some l;;
val count_some : 'a option list -> int = <fun>
```

Глава 4

Файлы, модули программы

До сих пор мы исследовали возможности языка OCaml только в интерактивной оболочке. По мере движения в направлении от примеров к действующим программам вам придется покинуть интерактивную оболочку и начать строить программы из файлов. Файлы – это нечто большее, чем просто удобный способ хранения программного кода; в языке OCaml они соответствуют модулям, которые делят программу на концептуальные единицы.

В этой главе мы покажем вам, как конструировать программы на OCaml из коллекции файлов, а также познакомим с основами работы с модулями и их сигнатурами.

Программы в единственном файле

Начнем с примера: напишем утилиту, которая будет читать ввод пользователя со стандартного ввода и вычислять частоту встречаемости строк. В конце она будет выводить 10 наиболее часто встречаемых строк. Для начала создадим простую реализацию, которую сохраним в файле с именем *freq.ml*.

В этой реализации будут использоваться две функции из модуля `List.Assoc`, содержащего вспомогательные функции для работы с ассоциативными списками (association lists), то есть со списками пар ключ/значение. В частности, мы будем использовать функцию `List.Assoc.find`, осуществляющую поиск заданного ключа в ассоциативном списке, и функцию `List.Assoc.add`, добавляющую новую привязку в ассоциативный список, как показано ниже:

OCaml utop

<https://github.com/realworldocaml/examples/tree/v1/code/files-modules-and-programs/intro.topscript>

```
# let assoc = [("one", 1); ("two", 2); ("three", 3)] ;;
val assoc : (string * int) list = [("one", 1); ("two", 2); ("three", 3)]
# List.Assoc.find assoc "two" ;;
- : int option = Some 2
# List.Assoc.add assoc "four" 4 (* добавить новый ключ *) ;;
- : (string, int) List.Assoc.t =
[("four", 4); ("one", 1); ("two", 2); ("three", 3)]
# List.Assoc.add assoc "two" 4 (* переопределить существующий ключ *) ;;
- : (string, int) List.Assoc.t = [("two", 4); ("one", 1); ("three", 3)]
```

Отметьте, что функция `List.Assoc.add` не модифицирует исходного списка, а создает новый, куда добавляет указанную пару ключ/значение.

Теперь наполним сам файл *freq.ml*:

OCaml

<https://github.com/realworldocaml/examples/tree/v1/code/files-modules-and-programs-freq/freq.ml>

```
open Core.Std
```

```
let build_counts () =
  In_channel.fold_lines stdin ~init:[] ~f:(fun counts line ->
    let count =
      match List.Assoc.find counts line with
      | None -> 0
      | Some x -> x
    in
    List.Assoc.add counts line (count + 1)
  )

let () =
  build_counts ()
  |> List.sort ~cmp:(fun (_,x) (_,y) -> Int.descending x y)
  |> (fun l -> List.take l 10)
  |> List.iter ~f:(fun (line,count) -> printf "%3d: %s\n" count line)
```

Функция `build_counts` читает строки со стандартного ввода, конструирует из этих строк ассоциативный список, запоминая в нем частоту встречаемости каждой строки. Эту работу она выполняет с помощью функции `In_channel.fold_lines` (похожей на функцию `List.fold`, описанную в главе 3), которая читает строки по одной и вызывает указанную функцию `fold` для каждой строки, чтобы обновить аккумулятор. Первоначально аккумулятор инициализируется пустым списком.

Вслед за определением функции `build_counts` мы вызываем ее, чтобы создать ассоциативный список, сортируем этот список по частоте встречаемости в порядке убывания, извлекаем из списка 10 первых элементов и затем выполняем по ним итерации, чтобы вывести на экран. Все эти операции объединены в единую цепь с помощью оператора `|>`, описанного в главе 2.

Где находится функция `main`?

В отличие от программ на языке C, программы на OCaml не имеют уникальной функции `main`. При выполнении программы на OCaml все инструкции, имеющиеся в ней, выполняются в порядке их следования. Файлы реализации могут содержать не только определения функций, но и произвольные выражения. В данном примере роль функции `main` играет объявление, начинающееся с последовательности `let () =`, которая запускает обработку. Но в действительности весь файл выполняется с самого начала, поэтому весь файл в некотором смысле является одной большой функцией `main`.

Кому-то идиома запуска вычислений инструкцией вида `let () =` может показаться странной, но в ней есть определенный смысл. `let`-привязка здесь играет роль сопоставления с образцом для значения типа `unit`, которая требует, чтобы выражение справа возвращало значение `unit`, что характерно в первую очередь для функций, производящих побочные эффекты.

Если бы мы не использовали в программе библиотеку Core или любую другую внешнюю библиотеку, то могли бы скомпилировать выполняемый файл, как показано ниже:

Terminal

```
https://github.com/realworldocaml/examples/tree/v1/code/
files-modules-and-programs-freq/simple_build_fail.out
```

```
$ ocamlc freq.ml -o freq.byte
File "freq.ml", line 1, characters 0-13:
Error: Unbound module Core
(Ошибка: Несвязанный модуль Core)
```

Но, как видите, в данном случае компиляция завершилась ошибкой из-за того, что компилятору не удалось найти библиотеку Core. Чтобы исправить проблему, необходимо использовать чуть более сложную команду, связывающую программу с библиотекой Core:

Terminal

```
https://github.com/realworldocaml/examples/tree/v1/code/
files-modules-and-programs-freq/simple_build.out
```

```
$ ocamlfind ocamlc -linkpkg -thread -package core freq.ml -o freq.byte
```

В ней используется инструмент *ocamlfind*, который автоматически вызывает другие утилиты компилятора OCaml (в данном случае *ocamlc*) с соответствующими флагами для связывания с определенными библиотеками и пакетами. Здесь флаг *-package core* предписывает утилите *ocamlfind* связать выполняемый файл с библиотекой Core; флаг *-linkpkg* требует от *ocamlfind* подключить все необходимые пакеты, а флаг *-thread* включает поддержку многопоточной модели выполнения (требуется для библиотеки Core).

Этот прием достаточно прост для проекта из одного файла, но для более сложных проектов необходим дополнительный инструмент, который управляет бы процессом сборки. Одним из таких инструментов является *ocamlbuild*, поставляемый вместе с компилятором OCaml. Мы еще вернемся к *ocamlbuild* в главе 22, а пока будем пользоваться простой оберткой вокруг *ocamlbuild*, которая называется *corebuild*. Она устанавливает параметры, необходимые для сборки проектов с библиотекой Core и другими связанными с ней библиотеками:

Terminal

```
https://github.com/realworldocaml/examples/tree/v1/code/
files-modules-and-programs-freq-obuild/build.out
```

```
$ corebuild freq.byte
```

Если вызвать утилиту *corebuild* и передать ей целевой файл *freq.native* вместо *freq.byte*, мы получим выполняемый файл программы с машинным кодом.

Получившийся файл можно запустить из командной строки. Следующая команда извлечет строки из двоичного файла *ocamlcpt* и выведет наиболее часто встречающиеся. Обратите внимание, что конкретные результаты могут отличаться на разных платформах, поскольку сами двоичные файлы имеют разный формат на разных платформах:

Terminal

<https://github.com/realworldocaml/examples/tree/v1/code/files-modules-and-programs-freq-obuild/test.out>

```
$ strings `which ocamlc` | ./freq.byte
6: +pci_expr =
6: -pci_params =
6: .pci_virt = %a
4: #lsr
4: #lsl
4: $lxor
4: #lor
4: $land
4: #mod
3: 6 .section .rdata,"dr"
```

Байт-код и машинный код

В составе дистрибутива OCaml распространяются два компилятора: компилятор в байт-код `ocamlc` и компилятор в машинный код `ocamlopt`. Программы, скомпилированные с помощью `ocamlc`, интерпретируются виртуальной машиной, а программы, скомпилированные с помощью `ocamlopt`, содержат машинный код и могут выполняться непосредственно под управлением операционной системы на данной аппаратной архитектуре. Утилита `ocamlbuild` создает выполняемые файлы с байт-кодом, если имя целевого файла имеет расширение `.byte`, а для целевых файлов с расширением `.native` она генерирует машинный код.

Помимо разной производительности, во всем остальном одна и та же программа в байт-коде и в машинном коде имеет совершенно идентичное поведение. Есть несколько важных моментов, которые вам следует знать и помнить. Во-первых, компилятор байт-кода может использоваться на большем числе архитектур и поддерживает некоторые возможности, недоступные для машинного кода. Например, отладчик OCaml работает только с байт-кодом (впрочем, машинный код, произведенный компилятором `ocamlopt`, можно отлаживать с помощью `gdb`, отладчика GNU Debugger). Кроме того, компилятор байт-кода работает быстрее, чем компилятор машинного кода. С другой стороны, чтобы запустить выполняемый файл с байт-кодом, на компьютере обычно должен быть установлен OCaml. Впрочем, это требование не является строгим, поскольку есть возможность скомпилировать выполняемый файл с байт-кодом со встроенной поддержкой времени выполнения, для чего следует использовать флаг компилятора `-custom`.

Как правило, программы для промышленного использования должны компилироваться в машинный код, но для создания отладочных сборок есть смысл использовать байт-код. И конечно, байт-код оказывается единственной альтернативой на платформах, не поддерживаемых компилятором машинного кода. Более подробно мы поговорим о компиляторах в главе 23.

Программы и модули из нескольких файлов

Файлы с исходным кодом на языке OCaml связываются в систему модулей, где каждый файл компилируется в модуль с именем, соответствующим имени файла. Мы уже сталкивались с модулями выше, когда использовали функции, такие как `find` и `add` из модуля `List.Assoc`. Модули можно считать коллекциями определений, хранящимися в пространствах имен.

Давайте посмотрим, как можно использовать поддержку модулей для рефакторинга реализации *freq.ml*. Как вы наверняка помните, переменная `counts` хранит

ассоциативный список со счетчиками обработанных строк. Но время, необходимое на обновление ассоциативного списка, линейно зависит от его длины, то есть временная сложность обработки файла является квадратичной функцией от числа отдельных строк в этом файле.

Мы можем исправить данный недостаток, заменив ассоциативный список более эффективной структурой данных. Но для этого сначала нужно вынести базовую функциональность в отдельный модуль с явным интерфейсом. Как только мы получим ясный программный интерфейс, то сможем приступить к рассмотрению альтернативных (и более эффективных) реализаций.

Начнем с создания файла *counter.ml*, куда поместим логику управления ассоциативным списком, используемым для представления счетчиков. Ключевая функция с именем *touch* увеличивает счетчик для указанной строки на единицу:

OCaml

<https://github.com/realworldocaml/examples/blob/v1/code/files-modules-and-programs-freq-with-counter/counter.ml>

```
open Core.Std

let touch t s =
  let count =
    match List.Assoc.find t s with
    | None -> 0
    | Some x -> x
  in
  List.Assoc.add t s (count + 1)
```

Файл *counter.ml* будет скомпилирован в модуль с именем *Counter*, где имя модуля автоматически определяется из имени файла. Первая буква в имени модуля заглавная, несмотря на то что в имени файла она прописная. В действительности имена модулей всегда начинаются с заглавной буквы.

Теперь можно переписать файл *freq.ml* и задействовать в нем модуль *Counter*. Отметьте, что полученный код все еще можно скомпилировать с помощью утилиты *ocamlbuild*, которая автоматически определит все зависимости и скомпилирует файл *counter.ml*:

OCaml

<https://github.com/realworldocaml/examples/blob/v1/code/files-modules-and-programs-freq-with-counter/freq.ml>

```
open Core.Std

let build_counts () =
  In_channel.fold_lines stdin ~init:[] ~f:Counter.touch

let () =
  build_counts ()
  |> List.sort ~cmp:(fun (_,x) (_,y) -> Int.descending x y)
  |> (fun l -> List.take l 10)
  |> List.iter ~f:(fun (line,count) -> printf "%3d: %s\n" count line)
```

Сигнатуры и абстрактные типы

Несмотря на то что мы перенесли часть логики в модуль `Counter`, код в файле *freq.ml* все еще зависит от особенностей реализации `Counter`. И действительно, если взглянуть на определение `build_counts`, можно увидеть, что начальное пустое множество частот строк в этой функции создается как пустой список. Было бы неплохо устранить такую зависимость и получить возможность изменять реализацию модуля `Counter` без необходимости изменять клиентский код, такой как в файле *freq.ml*.

Тонкости реализации модуля можно спрятать за дополнительным *интерфейсом*. (Имейте в виду, что в языке OCaml такие термины, как *интерфейс*, *сигнатура* и *тип модуля*, являются взаимозаменяемыми.) Ограничить доступ к элементам модуля, реализованного в файле *filename.ml*, можно с помощью сигнатуры в файле с именем *filename.mli*.

В файле *counter.mli* мы определим интерфейс, описывающий доступные в настоящий момент элементы, объявленные в файле *counter.ml*, никак не ограничивая доступа. Для определения значений в сигнатуре используются объявления `val`, имеющие следующий синтаксис:

Syntax

<https://github.com/realworldocaml/examples/blob/v1/code/files-modules-and-programs/val.syntax>

```
val <identifier> : <type>
```

Используя этот синтаксис, сигнатуру файла *counter.ml* можно записать так:

OCaml

<https://github.com/realworldocaml/examples/blob/v1/code/files-modules-and-programs-freq-with-sig/counter.mli>

```
open Core.Std
```

```
(** Нарастивает счетчик для указанной строки. *)
val touch : (string * int) list -> string -> (string * int) list
```

Отметьте, что *ocamlbuild* автоматически определяет присутствие файла *.mli* и включает его в процесс сборки.

Автоматическое создание файлов .mli

Если вы не горите желанием вручную создавать файлы *.mli*, можете попросить OCaml сгенерировать его автоматически, используя в качестве основы файл с исходным программным кодом. После этого вы сможете подправить сигнатуру в соответствии со своими потребностями. Ниже показано, как это можно сделать с помощью *corebuild*:

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/files-modules-and-programs-freq-with-counter/infer_mli.out

```
$ corebuild counter.inferred.mli
$ cat _build/counter.inferred.mli
val touch :
  ('a, int) Core.Std.List.Assoc.t -> 'a -> ('a, int) Core.Std.List.Assoc.t
```


Сгенерированный файл `.mli` получится практически эквивалентным тому, что мы написали вручную, но не таким аккуратным, более пространным и, конечно же, без комментариев. Вообще говоря, к автоматическому созданию файлов `.mli` имеет смысл прибегать только на начальном этапе разработки. Файлы `.mli` в языке OCaml являются основным местом объявления и описания интерфейсов, и никакие самые лучшие автоматизированные средства не смогут заменить внимательное отношение человека к редактированию и организации содержимого файлов.

Чтобы скрыть тот факт, что счетчики хранятся в ассоциативных списках, нам необходимо сделать тип счетчиков *абстрактным*. Тип считается абстрактным, если его имя экспортируется посредством интерфейса, а определение – нет. Ниже представлен абстрактный интерфейс для модуля `Counter`:

OCaml

<https://github.com/realworldocaml/examples/blob/v1/code/files-modules-and-programs-freq-with-sig-abstract/counter.mli>

```
open Core.Std

(** Коллекция счетчиков строк *)
type t

(** Пустая коллекция счетчиков строк *)
val empty : t

(** Нарастивает счетчик для указанной строки. *)
val touch : t -> string -> t

(** Преобразует коллекцию счетчиков в ассоциативный список. Строки хранятся
в единственном экземпляре, а счетчики имеют значения >= 1. *)
val to_list : t -> (string * int) list
```

Обратите внимание, что нам пришлось добавить в интерфейс `Counter` определения `empty` и `to_list`, потому что иначе не было бы возможности создать коллекцию `Counter.t` или получить данные из нее.

Мы также воспользовались возможностью добавить описание модуля. Файлы `.mli` – это место, где определяются интерфейсы модулей, и естественное место для их описания. Наши комментарии начинаются с двух символов звездочки. Это сделано специально, чтобы их можно было выбирать с помощью *ocamldoc*, инструмента, автоматически генерирующего документацию с описанием API. Подробнее об инструменте *ocamldoc* будет рассказываться в главе 22.

Ниже приводится версия *counter.ml*, переписанная в соответствии с содержимым файла *counter.mli*:

OCaml

<https://github.com/realworldocaml/examples/blob/v1/code/files-modules-and-programs-freq-with-sig-abstract/counter.ml>

```
open Core.Std

type t = (string * int) list

let empty = []
```

```
let to_list x = x

let touch t s =
  let count =
    match List.Assoc.find t s with
    | None -> 0
    | Some x -> x
  in
  List.Assoc.add t s (count + 1)
```

Если теперь попробовать скомпилировать *freq.ml*, мы получим следующую ошибку:

Terminal

<https://github.com/realworldocaml/examples/tree/v1/code/files-modules-and-programs-freq-with-sig-abstract/build.out>

```
$ corebuild freq.byte
```

```
File "freq.ml", line 4, characters 42-55:
```

```
Error: This expression has type Counter.t -> string -> Counter.t
      but an expression was expected of type 'a list -> string -> 'a list
      Type Counter.t is not compatible with type 'a list
```

```
Command exited with code 2.
```

```
(Ошибка: Это выражение имеет тип Counter.t -> string -> Counter.t,
  тогда как ожидалось выражение типа 'a list -> string -> 'a list
  Тип Counter.t несовместим с типом 'a list.
```

```
Команда завершилась с кодом 2.)
```

Эта ошибка обусловлена зависимостью реализации в файле *freq.ml* от того факта, что счетчики представлены в виде ассоциативных списков, который мы только что скрыли за интерфейсом. Чтобы исправить проблему, достаточно в функции *build_counts* использовать *Counter.empty* вместо *[]* и задействовать *Counter.to_list* для получения ассоциативного списка перед его обработкой и выводом. Окончательная реализация представлена ниже:

OCaml

<https://github.com/realworldocaml/examples/blob/v1/code/files-modules-and-programs-freq-with-sig-abstract-fixed/freq.ml>

```
open Core.Std
```

```
let build_counts () =
  In_channel.fold_lines stdin ~init:Counter.empty ~f:Counter.touch
```

```
let () =
  build_counts ()
  |> Counter.to_list
  |> List.sort ~cmp:(fun (_,x) (_,y) -> Int.descending x y)
  |> (fun counts -> List.take counts 10)
  |> List.iter ~f:(fun (line,count) -> printf "%3d: %s\n" count line)
```

Теперь можно приступить к оптимизации реализации модуля *Counter*. Ниже приводится альтернативная и более эффективная реализация, основанная на структуре данных *Map* из библиотеки *Core*:

OCaml

<https://github.com/realworldocaml/examples/blob/v1/code/files-modules-and-programs-freq-fast/counter.ml>

```
open Core.Std

type t = int String.Map.t

let empty = String.Map.empty

let to_list t = Map.to_alist t

let touch t s =
  let count =
    match Map.find t s with
    | None -> 0
    | Some x -> x
  in
  Map.add t ~key:s ~data:(count + 1)
```

Отметьте, что здесь в одних местах мы используем `String.Map`, а в других `Map`. Это обусловлено необходимостью иметь доступ к специализированной информации о типах в некоторых операциях, таких как операция создания `Map.t`, а в других операциях, таких как поиск в `Map.t`, в этом нет необходимости. Подробнее об этом будет рассказываться в главе 13.

Конкретные типы в сигнатурах

В примере подсчета частоты встречаемости строк для представления коллекции счетчиков в модуле `Counter` был определен абстрактный тип `Counter.t`. Но иногда на практике бывает необходимо указать в интерфейсе *конкретный* тип, включив в него определение типа.

Например, представьте, что в модуль `Counter` потребовалось добавить функцию, возвращающую строку со срединным (*median*) значением счетчика. Для четного количества строк, когда нет точно срединного значения, функция могла бы возвращать две строки — до и после середины списка. Для отражения того факта, что может быть возвращено два значения, мы могли бы определить собственный тип данных. Ниже представлена одна из возможных реализаций:

OCaml (part 1)

<https://github.com/realworldocaml/examples/tree/v1/code/files-modules-and-programs-freq-median/counter.ml>

```
type median = | Median of string
              | Before_and_after of string * string

let median t =
  let sorted_strings = List.sort (Map.to_alist t)
    ~cmp:(fun (_, x) (_, y) -> Int.descending x y)
  in
  let len = List.length sorted_strings in
```

```

if len = 0 then failwith "median: empty frequency count";
let nth n = fst (List.nth_exn sorted_strings n) in
if len mod 2 = 1
then Median (nth (len/2))
else Before_and_after (nth (len/2 - 1), nth (len/2));;

```

В этой реализации мы использовали `failwith`, чтобы возбудить исключение в случае, когда список пуст. Мы еще вернемся к исключениям в главе 7. Отметьте также, что функция `fst` просто возвращает первый элемент любого кортежа из двух элементов.

Теперь, чтобы экспортировать эту функцию в интерфейсе, необходимо вместе с ней экспортировать и тип `median` с его определением. Обратите внимание, что значения (примерами которых могут служить функции) и типы находятся в разных пространствах имен, благодаря чему отсутствуют конфликты имен. Поставленную задачу решают следующие две строки в файле *counter.mli*:

OCaml (part 1)

<https://github.com/realworldocaml/examples/blob/v1/code/files-modules-and-programs-freq-median/counter.mli>

```

(** Представляет середину коллекции строк, упорядоченных по частоте
    встречаемости. В случае четного числа элементов в коллекции
    возвращаются два элемента, до и после середины выборки. *)
type median = | Median of string
              | Before_and_after of string * string

val median : t -> median

```

Выбор между абстрактным и конкретным типами является важным решением. Абстрактные типы дают более полный контроль над операциями создания и доступа к значениям и упрощают обеспечение инвариантности, так как она обеспечивается самим типом; конкретные типы позволяют экспортировать в клиентский код дополнительные тонкости и структуру данных. Выбор того или иного решения во многом зависит от конкретной ситуации.

Вложенные модули

До настоящего момента мы видели только модули, соответствующие файлам, таким как *counter.ml*. Но модули (и их сигнатуры) могут вкладываться внутрь других модулей. Рассмотрим простой пример. Вообразите программу, в которой придется иметь дело со множеством идентификаторов, таких как имена пользователей и имена хостов. Если представить их в виде последовательностей, вы сможете легко перепутать их друг с другом.

Более удачное решение заключается в определении абстрактных типов для каждой разновидности идентификаторов, реализованных как строки. При таком подходе система типов не позволит вам спутать имя пользователя с именем хоста, а если потребуется, вы сможете явно преобразовать тот или иной идентификатор в строку или из строки.

Ниже показано, как можно было бы объявить подобный абстрактный тип внутри вложенного модуля:

OCaml

https://github.com/realworldocaml/examples/blob/v1/code/files-modules-and-programs/abstract_username.ml

```
open Core.Std
```

```
module Username : sig
  type t
  val of_string : string -> t
  val to_string : t -> string
end = struct
  type t = string
  let of_string x = x
  let to_string x = x
end
```

Обратите внимание, что функции `to_string` и `of_string` выше реализованы как простые функции тождественности, в том смысле, что они не производят никакого эффекта. Эти функции являются просто частью дисциплины, проводимой кодом через систему типов.

В общем виде объявление модуля имеет следующий синтаксис:

Syntax

<https://github.com/realworldocaml/examples/blob/v1/code/files-modules-and-programs/module.syntax>

```
module <ИМЯ> : <СИГНАТУРА> = <РЕАЛИЗАЦИЯ>
```

Можно было бы переписать модуль иначе, определив сигнатуру в отдельном объявлении `module type` верхнего уровня и обеспечив тем самым возможность создавать множество самостоятельных типов с одной и той же реализацией в их основе:

OCaml

https://github.com/realworldocaml/examples/blob/v1/code/files-modules-and-programs/session_info.ml

```
open Core.Std
```

```
module type ID = sig
  type t
  val of_string : string -> t
  val to_string : t -> string
end
```

```
module String_id = struct
  type t = string
  let of_string x = x
  let to_string x = x
end
```

```
module Username : ID = String_id
module Hostname : ID = String_id
```

```

type session_info = { user: Username.t;
                      host: Hostname.t;
                      when_started: Time.t;
}

let sessions_have_same_user s1 s2 =
  s1.user = s2.host

```

В предыдущем коде есть одна ошибка: в нем сравнивается имя пользователя в одном сеансе с именем хоста в другом, тогда как должны сравниваться имена пользователей. Однако благодаря нашим объявлениям типов компилятор обнаружит эту ошибку:

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/files-modules-and-programs/build_session_info.out

```

$ corebuild session_info.native
File "session_info.ml", line 24, characters 12-19:
Error: This expression has type Hostname.t
      but an expression was expected of type Username.t
Command exited with code 2.
(Ошибка: Это выражение имеет тип Hostname.t,
      тогда как ожидалось выражение типа Username.t
Команда завершилась с кодом 2.)

```

Это был тривиально простой пример, но путаница идентификаторов разного рода очень часто является источником ошибок в реальной жизни, и подход на основе абстрактных типов для идентификаторов разных классов эффективно решает такого рода проблемы.

Открытие модулей

Чаще всего обращения к значениям и типам внутри модуля осуществляются через указание имени этого модуля. Например, вызов функции `map` из модуля `List` записывается как `List.map`. Однако иногда бывает желательно иметь возможность ссылаться на элементы модуля без явной квалификации их имен. В таких ситуациях можно использовать директиву `open`.

Мы уже встречали директиву `open` выше, например когда писали `open Core.Std`, чтобы получить доступ к стандартным определениям в библиотеке `Core`. Если говорить простым языком, операция открытия модуля добавляет его содержимое в окружение, где компилятор сможет найти определения различных идентификаторов. Например:

OCaml utop

<https://github.com/realworldocaml/examples/blob/v1/code/files-modules-and-programs/main.topscript>

```

# module M = struct let foo = 3 end;;
module M : sig val foo : int end
# foo;;

```

```

Characters -1-3:
Error: Unbound value foo
(Ошибка: несвязанное значение foo)
# open M;;
# foo;;
- : int = 3

```

Директива `open` является основной, когда требуется изменить окружение, добавив в него библиотеку, такую как `Core`, но вообще считается хорошим тоном открывать как можно меньше модулей. Открытие модуля является своеобразным компромиссом между краткостью кода и его явностью – чем больше модулей открывается, тем меньше квалифицирующих имен приходится использовать и тем сложнее определять, откуда взялся тот или иной идентификатор.

Ниже приводится несколько простых советов по открытию модулей:

- открывать на верхнем уровне следует небольшое число модулей, и только те, что специально предназначены для этого, как, например, `Core.Std` или `Option.Monad_infix`;
- если потребуется открыть модуль, лучше делать это *локально*; ниже приводятся два примера локального открытия модулей:

OCaml utop (part 1)

<https://github.com/realworldocaml/examples/blob/v1/code/files-modules-and-programs/main.topscript>

```

# let average x y =
    let open Int64 in
    x + y / of_int 2;;
val average : int64 -> int64 -> int64 = <fun>

```

Здесь операторы `of_int` и `/` импортированы из модуля `Int64`.

Существует и другой, более простой способ локального открытия модуля, который удобно использовать в небольших выражениях:

OCaml utop (part 2)

<https://github.com/realworldocaml/examples/blob/v1/code/files-modules-and-programs/main.topscript>

```

# let average x y =
    Int64.(x + y / of_int 2);;
val average : int64 -> int64 -> int64 = <fun>

```

- альтернативой локальному открытию модулей, которая позволяет сделать код более компактным, но без потери явности, является локальное переименование модуля, например фрагмент, использующий тип `Counter.median`:

OCaml (part 1)

https://github.com/realworldocaml/examples/blob/v1/code/files-modules-and-programs-freq-median/use_median_1.ml

```

let print_median m =
  match m with
  | Counter.Median string -> printf "True median:\n  %s\n" string
  | Counter.Before_and_after (before, after) ->
    printf "Before and after median:\n  %s\n  %s\n" before after

```

можно переписать так:

OCaml (part 1)

https://github.com/realworldocaml/examples/blob/v1/code/files-modules-and-programs-freq-median/use_median_2.ml

```
let print_median m =
  let module C = Counter in
  match m with
  | C.Median string -> printf "True median:\n    %s\n" string
  | C.Before_and_after (before, after) ->
    printf "Before and after median:\n    %s\n    %s\n" before after
```

Поскольку имя `C` модуля определено только в ограниченной области видимости, его легко понять и запомнить, что оно означает. Переименование модулей на верхнем уровне программы обычно расценивается как ошибка.

Подключение модулей

Если операция открытия модуля оказывает влияние на окружение, в котором компилятор выполняет поиск идентификаторов, то подключение дает возможность фактически добавить новые идентификаторы в текущий модуль. Взгляните на следующий простой модуль, представляющий диапазон целочисленных значений:

OCaml utop (part 3)

<https://github.com/realworldocaml/examples/blob/v1/code/files-modules-and-programs/main.topscript>

```
# module Interval = struct
  type t = | Interval of int * int
          | Empty

  let create low high =
    if high < low then Empty else Interval (low,high)
end;;

module Interval :
  sig type t = Interval of int * int | Empty val create : int -> int -> t end
```

Для создания новой, расширенной версии модуля `Interval` мы могли бы использовать директиву `include`:

OCaml utop (part 4)

<https://github.com/realworldocaml/examples/blob/v1/code/files-modules-and-programs/main.topscript>

```
# module Extended_interval = struct
  include Interval

  let contains t x =
    match t with
    | Empty -> false
    | Interval (low,high) -> x >= low && x <= high
  end;;

module Extended_interval :
```



```

sig
  type t = Interval.t = Interval of int * int | Empty
  val create : int -> int -> t
  val contains : t -> int -> bool
end
# Extended_interval.contains (Extended_interval.create 3 10) 4;;
- : bool = true

```

В отличие от `open`, директива `include` не просто изменяет порядок поиска идентификаторов – она изменяет содержимое модуля. Если бы мы использовали `open`, то получили бы совершенно иной результат:

OCaml utop (part 5)

<https://github.com/realworldocaml/examples/blob/v1/code/files-modules-and-programs/main.topscript>

```

# module Extended_interval = struct
  open Interval

  let contains t x =
    match t with
    | Empty -> false
    | Interval (low,high) -> x >= low && x <= high
  end;;
module Extended_interval :
  sig val contains : Extended_interval.t -> int -> bool end
# Extended_interval.contains (Extended_interval.create 3 10) 4;;
Characters 28-52:
Error: Unbound value Extended_interval.create
(Ошибка: Несвязанное значение Extended_interval.create)

```

Рассмотрим более реалистичный пример. Представьте, что вам потребовалось реализовать расширенную версию модуля `List`, добавив в нее некоторые возможности, отсутствующие в модуле из библиотеки `Core`. Директива `include` дает вам такую возможность:

OCaml

https://github.com/realworldocaml/examples/blob/v1/code/files-modules-and-programs/ext_list.ml

```

open Core.Std

(* Новая функция *)
let rec intersperse list el =
  match list with
  | [] | [ _ ] -> list
  | x :: y :: tl -> x :: el :: intersperse (y::tl) el

(* Остальное содержимое из модуля list *)
include List

```

Но как теперь написать интерфейс для этого нового модуля? Как оказывается, директива `include` работает и с сигнатурами, поэтому при создании собственного файла `.mli` можно использовать практически тот же самый прием. Единственная проблема – необходимо вручную подключить сигнатуру для модуля `List`. Сделать

это можно с помощью инструкции `module type of`, которая возвращает сигнатуру указанного модуля:

OCaml

https://github.com/realworldocaml/examples/blob/v1/code/files-modules-and-programs/ext_list.mli

```
open Core.Std
```

```
(* Подключить интерфейс модуля List из библиотеки Core *)
include (module type of List)
```

```
(* Сигнатура добавленной нами функции *)
val intersperse : 'a list -> 'a -> 'a list
```

Отметьте, что порядок объявлений в файле *.mli* необязательно должен совпадать с порядком объявлений в файле *.ml*. Порядок объявлений в файле *.ml* имеет значение лишь потому, что он определяет, какие значения будут затеняться. Если вам потребуется заменить функцию из модуля `List` новой реализацией с тем же именем, объявление этой функции в файле *.ml* должно следовать после объявления `include List`.

Теперь мы можем использовать модуль `Ext_list` взамен модуля `List`. Если потребуется обеспечить предпочтительное использование `Ext_list` в рамках всего проекта, можно создать файл с общими определениями:

OCaml

<https://github.com/realworldocaml/examples/blob/v1/code/files-modules-and-programs/common.ml>

```
module List = Ext_list
```

и добавить директиву `open Common` после директивы `open Core.Std` в начало каждого файла, после чего все ссылки на модуль `List` автоматически превратятся в ссылки на модуль `Ext_list`.

Типичные ошибки при работе с модулями

Когда компилятор OCaml выполняет сборку программы, состоящей из файлов *.ml* и *.mli*, при обнаружении расхождений между ними он сообщит об ошибке. Ниже описываются некоторые типичные ошибки, с которыми вы можете столкнуться.

Несовпадение типов

Самая простая разновидность ошибок – несовпадение типов, указанных в сигнатуре и в реализации модуля. Например, если изменить объявление `val` в файле *counter.mli*, поменяв местами типы первого и второго аргументов:

OCaml (part 1)

<https://github.com/realworldocaml/examples/blob/v1/code/files-modules-and-programs-freq-with-sig-mismatch/counter.mli>

```
(** Нарастивает счетчик для указанной строки. *)
val touch : string -> t -> t
```

и попытаться скомпилировать программу, мы получим ошибку:

Terminal

<https://github.com/realworldocaml/examples/tree/v1/code/files-modules-and-programs-freq-with-sig-mismatch/build.out>

```
$ corebuild freq.byte
```

```
File "freq.ml", line 4, characters 53-66:
```

```
Error: This expression has type string -> Counter.t -> Counter.t
      but an expression was expected of type
```

```
      Counter.t -> string -> Counter.t
```

```
Type string is not compatible with type Counter.t
```

```
Command exited with code 2.
```

(Ошибка: Это выражение имеет тип string -> Counter.t -> Counter.t,

тогда как ожидалось выражение типа

Counter.t -> string -> Counter.t

Тип string несовместим с типом Counter.t

Команда завершилась с кодом 2.)

Отсутствие определений

Мы могли бы решить добавить в модуль Counter новую функцию, извлекающую частоту встречаемости указанной строки, и вставили бы в файл *.mli* следующую строку:

OCaml (part 1)

<https://github.com/realworldocaml/examples/blob/v1/code/files-modules-and-programs-freq-with-missing-def/counter.mli>

```
val count : t -> string -> int
```

Если теперь попробовать скомпилировать программу, не добавив фактической реализации функции, мы получим ошибку:

Terminal

<https://github.com/realworldocaml/examples/tree/v1/code/files-modules-and-programs-freq-with-missing-def/build.out>

```
$ corebuild freq.byte
```

```
File "counter.ml", line 1:
```

```
Error: The implementation counter.ml
```

```
      does not match the interface counter.cmi:
```

```
      The field `count' is required but not provided
```

```
Command exited with code 2.
```

(Ошибка: Реализация counter.ml не соответствует

интерфейсу counter.cmi:

Поле 'count' является обязательным, но оно отсутствует

Команда завершилась с кодом 2.)

Отсутствие определений типов приводит к появлению аналогичных сообщений.

Несоответствие определений типов

Определения типов, присутствующие в файле *.mli*, должны совпадать с соответствующими определениями в файле *.ml*. Вернемся к примеру определения типа

`median`. Порядок определения вариантов в нем имеет значение для компилятора OCaml, поэтому определение типа `median` в реализации с иным порядком следования вариантов:

OCaml (part 1)

<https://github.com/realworldocaml/examples/blob/v1/code/files-modules-and-programs-freq-with-type-mismatch/counter.mli>

```
(** Представляет середину коллекции строк, упорядоченных по частоте
    встречаемости. В случае четного числа элементов в коллекции
    возвращаются два элемента, до и после середины выборки. *)
type median = | Before_and_after of string * string
              | Median of string
```

вызовет сообщение об ошибке во время компиляции:

Terminal

<https://github.com/realworldocaml/examples/tree/v1/code/files-modules-and-programs-freq-with-type-mismatch/build.out>

```
$ corebuild freq.byte
```

```
File "counter.ml", line 1:
```

```
Error: The implementation counter.ml
```

```
does not match the interface counter.cmi:
```

```
Type declarations do not match:
```

```
type median = Median of string | Before_and_after of string * string
is not included in
```

```
type median = Before_and_after of string * string | Median of string
```

```
File "counter.ml", line 18, characters 5-84: Actual declaration
```

```
Fields number 1 have different names, Median and Before_and_after.
```

```
Command exited with code 2.
```

(Ошибка: Реализация counter.ml

не соответствует интерфейсу counter.mli:

Не совпадают объявления типа:

*type median = Median of string | Before_and_after of string * string*
не включено в

*type median = Before_and_after of string * string | Median of string*
Файл "counter.ml", строка 18, позиции 5-84: Фактическое объявление

полей с номером 1 имеют разные имена, Median и Before_and_after.

Команда завершилась с кодом 2.)

Порядок также важен и для других объявлений типов, включая порядок следования полей в записях и порядок аргументов в функциях (в том числе аргументов с метками и необязательных аргументов).

Циклические зависимости

В большинстве случаев OCaml не позволяет циклических зависимостей, то есть коллекции определений не могут ссылаться друг на друга. Если все же надобности в таких определениях не избежать, их обычно помечают особым образом. Например, когда определяется множество взаиморекурсивных значений (таких как функции `is_even` и `is_odd` в разделе «Рекурсивные функции» в главе 2), для их определения следует использовать `let rec` вместо `let`.

То же относится и к модулям. По умолчанию циклические зависимости между модулями недопустимы, а циклические зависимости между файлами недопустимы в принципе. И все же есть возможность объявления рекурсивных модулей, но в очень редких случаях, поэтому мы не будем развивать эту тему дальше.

Простейшим примером недопустимой циклической зависимости является ссылка в модуле на самого себя. То есть если попытаться добавить ссылку на модуль `Counter` в файл `counter.ml`:

OCaml (part 1)

<https://github.com/realworldocaml/examples/blob/v1/code/files-modules-and-programs-freq-cyclic1/counter.ml>

```
let singleton l = Counter.touch Counter.empty
```

компилятор выведет следующее сообщение об ошибке:

Terminal

<https://github.com/realworldocaml/examples/tree/v1/code/files-modules-and-programs-freq-cyclic1/build.out>

```
$ corebuild freq.byte
File "counter.ml", line 18, characters 18-31:
Error: Unbound module Counter
Command exited with code 2.
(Ошибка: Несвязанный модуль Counter
Команда завершилась с кодом 2.)
```

Немного по-иному эта проблема проявляется, если создать циклические ссылки между файлами.

Воспроизвести эту ситуацию можно добавлением ссылки на модуль `Freq` из `counter.ml`, как показано ниже:

OCaml (part 1)

<https://github.com/realworldocaml/examples/blob/v1/code/files-modules-and-programs-freq-cyclic2/counter.ml>

```
let _build_counts = Freq.build_counts
```

В данном случае компилятор `ocamlbuild` (вызывается сценарием `corebuild`) обнаружит ошибку и явно сообщит о циклической зависимости:

Terminal

<https://github.com/realworldocaml/examples/tree/v1/code/files-modules-and-programs-freq-cyclic2/build.out>

```
$ corebuild freq.byte
Circular dependencies: "freq.cmo" already seen in
[ "counter.cmo"; "freq.cmo" ]
(Циклические зависимости: "freq.cmo" уже замечен в
[ "counter.cmo"; "freq.cmo" ])
```

Проектирование с применением модулей

Модули являются одним из основных инструментов структурирования программ на языке OCaml. Поэтому мы завершим эту главу некоторыми советами по эффективной организации модульной структуры программ.

Старайтесь не экспортировать конкретные типы

При проектировании файлов *.mli* вам часто придется выбирать, экспортировать свои определения конкретных типов или оставить их абстрактными. Часто абстрактные типы предпочтительнее по двум причинам: они улучшают гибкость архитектуры и дают возможность обеспечить инвариантность использования вашего модуля.

Абстракции повышают гибкость за счет ограничения числа способов взаимодействия с вашими типами и, соответственно, уменьшения зависимости клиентского кода от особенностей реализации типа. Если вы экспортируете тип явно, клиентский код будет зависеть от любых особенностей типа по вашему выбору. Если тип остается абстрактным, вам достаточно будет экспортировать лишь конкретные операции. Это означает, что вы можете свободно изменять реализацию, не опасаясь оказать разрушительное влияние на клиентский код, при условии что семантика экспортируемых операций сохраняется неизменной.

Аналогично абстракция позволяет обеспечить инвариантность ваших типов. Если тип экспортируется явно, пользователи модуля смогут создавать новые экземпляры этого типа (или, если он является изменяемым, модифицировать существующие экземпляры) способом, который поддерживается типом, лежащим в основе. Это может нарушать желаемую инвариантность — свойство вашего типа, которое, как предполагается, всегда остается желательным. Оставляя тип абстрактным, вы можете защитить его инвариантность, экспортировав только те функции, которые способствуют этому.

Несмотря на описанные выгоды, иногда их цена может оказаться слишком высокой. В частности, экспортирование конкретных типов делает возможным применение к значениям этих типов сопоставления с образцом, которое, как было показано в главе 3, является мощным и важным инструментом. Вообще говоря, реализации конкретных типов следует экспортировать, только когда возможность сопоставления с образцом становится особенно ценной, а инвариантность типа обеспечивается самой его реализацией.

Продумывайте синтаксис вызовов

При разработке интерфейса необходимо подумать не только о том, насколько он будет понятен для тех, кто будет читать описание интерфейса в файле *.mli*, но и о том, насколько будут понятны обращения к интерфейсу для тех, кто будет читать код.

Дело в том, что в большинстве своем программисты взаимодействуют со сторонними API, читая и адаптируя код, использующий его, откладывая чтение опре-

деления интерфейса в долгий ящик. Делая свой API максимально очевидным, вы упрощаете жизнь своим пользователям.

Существует множество способов повышения читаемости вызовов. Один из примеров – аргументы с метками (обсуждаются в разделе «Аргументы с метками» в главе 2), которые могут служить своеобразной документацией.

Удобочитаемость можно также улучшить простым выбором удачных имен функций, вариантов и полей записей. Удачные имена не всегда оказываются длинными. Если вы хотите написать анонимную функцию, удваивающую числа: `(fun x -> x * 2)`, для имени переменной отлично подойдет такое короткое имя, как `x`. Как правило, короткие имена следует выбирать, когда они имеют ограниченную область видимости, а длинные – когда они имеют глобальную область видимости. Например, имена функций в интерфейсе модуля должны быть более длинными и более явными.

Разумеется, всегда нужно стремиться найти золотую середину между явностью вашего интерфейса и длиной имен, которые его составляют. Существует еще одно хорошее правило: чем реже используется имя, тем длиннее и очевиднее оно должно быть, потому чем реже используется имя, тем большую важность приобретает его очевидность.

Создавайте однородные интерфейсы

Проектирование интерфейса модуля не является какой-то обособленной задачей. Интерфейсы, присутствующие в вашем коде, должны гармонично сочетаться друг с другом. Также следует особое внимание уделять стандартизации интерфейсов.

Создатели библиотеки `Core` приложили немало усилий, чтобы создать однородные интерфейсы. Ниже перечислены некоторые рекомендации, которым следовали разработчики `Core`.

- *Отдельный модуль для (почти) каждого типа.* Создавайте отдельные модули для каждого своего типа в программе, а основному типу модуля следует давать имя `t`.
- *Передавать `t` первым.* Если имеется модуль `M`, основным типом в котором является `M.t`, функции в `M`, принимающие значение типа `M.t`, должны принимать его в первом аргументе.
- Имена функций, возбуждающих исключения при определенных условиях, должны оканчиваться на `_exn`. Об иных ошибках должно сообщаться в возвращаемых значениях типа `option` или `Or_error.t` (оба рассматриваются в главе 7).

Разработчиками библиотеки `Core` были также выработаны стандарты, определяющие сигнатуры некоторых функций. Например, сигнатура функции `map` всегда остается неизменной, независимо от типов значений, к которым она применяется. Такая однородность API достигается за счет использования *включаемых сигнатур* (`signature includes`), позволяющих разным модулям совместно использовать компоненты их интерфейсов. Данный прием описывается в разделе «Использование нескольких интерфейсов» в главе 9.

Стандарты, определенные разработчиками Core, могут подходить или не подходить для ваших проектов, но в любом случае ваши проекты только выиграют, если вы будете следовать каким-либо стандартам.

Определяйте интерфейсы до реализации

Краткость и гибкость определений типов на языке OCaml дают возможность использовать подход к проектированию программного обеспечения, основанный на типах. Такой подход предусматривает разработку системы типов будущего приложения перед его реализацией.

Такой подход с успехом может использоваться при создании программ на любых языках и предполагает создание определений типов перед разработкой логики вычислений, а на уровне модулей – создание первого варианта файла *.mli* до создания файла *.ml*.

Конечно, процесс проектирования обычно движется в двух направлениях одновременно. Вам часто придется возвращаться к типам и изменять их в процессе работы над реализацией. Но типы и сигнатуры позволяют построить скелет проекта и одновременно яснее понять цели и намерения еще до того, как вы потратите уйму сил и времени на реализацию конкретных решений.

Глава 5

Записи

Одна из замечательных особенностей OCaml – краткость и выразительность системы объявления новых типов данных, и записи являются ключевым компонентом этой системы. Мы уже касались записей в главе 1, но в этой главе мы подробнее узнаем, как действуют записи, а также дадим вам несколько советов по эффективному их использованию.

Запись – это коллекция значений, хранящихся как единое целое, в которой каждый элемент идентифицируется собственным именем поля. Ниже представлен базовый синтаксис определения записей:

Syntax

<https://github.com/realworldocaml/examples/blob/v1/code/records/record.syntax>

```
type <имя-записи> =  
  { <поле> : <тип> ;  
    <поле> : <тип> ;  
    ...  
  }
```

Отметьте, что имена полей в записях должны начинаться с буквы нижнего регистра.

Далее приводится простой пример записи `host_info`, предназначенной для хранения сводной информации о компьютере:

OCaml utop

<https://github.com/realworldocaml/examples/blob/v1/code/records/main.topscript>

```
# type host_info =  
  { hostname : string;  
    os_name  : string;  
    cpu_arch : string;  
    timestamp : Time.t;  
  };;  
type host_info = {  
  hostname : string;  
  os_name  : string;  
  cpu_arch : string;  
  timestamp : Time.t;  
}
```

Создать новую запись типа `host_info` в программе не составляет никакого труда. Следующий фрагмент извлекает всю необходимую информацию о текущем компьютере с помощью модуля `Shell` из библиотеки `Core_extended`, который пере-

дает команды командной оболочке. В нем также используется функция `Time.now` из модуля `Time` в библиотеке `Core`:

OCaml utop (part 1)

<https://github.com/realworldocaml/examples/blob/v1/code/records/main.topscript>

```
# #require "core_extended";;
# open Core_extended.Std;;
# let my_host =
  let sh = Shell.sh_one_exn in
  { hostname = sh "hostname";
    os_name = sh "uname -s";
    cpu_arch = sh "uname -p";
    timestamp = Time.now ();
  };;
val my_host : host_info =
{hostname = "ocaml-wwwl"; os_name = "Linux"; cpu_arch = "unknown";
 timestamp = 2013-08-18 14:50:48.986085+01:00}
```

Кто-то из вас может задаться вопросом: «Как компилятор определил, что `my_host` имеет тип `host_info`?» Секрет прост: компилятор использовал в качестве подсказок имена полей записи. Далее в этой главе мы поговорим о том, что случится, если в области видимости компилятора окажется несколько типов записей с одинаковыми именами полей.

После получения экземпляра записи из него можно извлекать значения полей, используя точечную нотацию:

OCaml utop (part 2)

<https://github.com/realworldocaml/examples/blob/v1/code/records/main.topscript>

```
# my_host.cpu_arch;;
- : string = "unknown"
```

Определяя тип, вы всегда можете параметризовать его полиморфным типом. Записи не являются исключением. Так, например, следующий тип можно использовать для хранения любых элементов вместе с отметкой времени:

OCaml utop (part 3)

<https://github.com/realworldocaml/examples/blob/v1/code/records/main.topscript>

```
# type 'a timestamped = { item: 'a; time: Time.t };;
type 'a timestamped = { item : 'a; time : Time.t; }
```

Объявив такой тип, можно написать полиморфную функцию, оперирующую этим параметризованным типом:

OCaml utop (part 4)

<https://github.com/realworldocaml/examples/blob/v1/code/records/main.topscript>

```
# let first_timestamped list =
  List.reduce list ~f:(fun a b -> if a.time < b.time then a else b)
;;
val first_timestamped : 'a timestamped list -> 'a timestamped option = <fun>
```

Сопоставление с образцом и полнота

Другой способ извлечь информацию из записи – воспользоваться сопоставлением с образцом, как, например, в следующем определении функции `host_info_to_string`:

OCaml utop (part 5)

<https://github.com/realworldocaml/examples/blob/v1/code/records/main.topscript>

```
# let host_info_to_string { hostname = h; os_name = os;
                           cpu_arch = c; timestamp = ts;
                         } =
    sprintf "%s (%s / %s, on %s)" h os c (Time.to_sec_string ts);;
val host_info_to_string : host_info -> string = <fun>
# host_info_to_string my_host;;
- : string = "ocaml-wwwl (Linux / unknown, on 2013-08-18 14:50:48)"
```

Обратите внимание, что мы использовали образец с единственным вариантом вместо нескольких, перечисленных через вертикальную черту (`|`). Единственного образца оказалось достаточно просто потому, что образцы для записей являются *однозначными*, то есть образец для записи никогда не потерпит неудачу во время выполнения. Такая однозначность обусловлена тем, что набор полей в записях остается неизменным. Вообще говоря, образцы для типов с фиксированной структурой, таких как записи и кортежи, всегда являются однозначными, в отличие от типов с переменной структурой, таких как списки и варианты.

Еще одной важной характеристикой образцов записей является допустимость неполноты; образец может включать лишь подмножество полей записи. Это может приносить дополнительные удобства, но может также служить источником ошибок. Например, при добавлении новых полей в запись необходимо также обновить программный код, потому что компилятор будет игнорировать возможные несоответствия.

Представьте, к примеру, что нам потребовалось добавить в запись `host_info` новое поле с именем `os_release`:

OCaml utop (part 6)

<https://github.com/realworldocaml/examples/blob/v1/code/records/main.topscript>

```
# type host_info =
  { hostname   : string;
    os_name    : string;
    cpu_arch   : string;
    os_release : string;
    timestamp  : Time.t;
  } ;;
type host_info = {
  hostname : string;
  os_name  : string;
  cpu_arch : string;
  os_release : string;
  timestamp : Time.t;
}
```

Функция `host_info_to_string` при этом будет по-прежнему компилироваться без ошибок. В данном конкретном случае совершенно очевидно, что функцию `host_info_to_string` желательно бы изменить, чтобы включить вывод значения поля `os_release`, и было бы здорово, если бы система типов могла предупредить нас о желательных изменениях.

К счастью, в языке OCaml имеется возможность организовать вывод предупреждений в случае обнаружения отсутствующих полей в образцах. Если включить вывод этих предупреждений (с помощью директивы верхнего уровня `#warnings "+9"`), компилятор будет выводить соответствующие предупреждения:

OCaml utop (part 7)

<https://github.com/realworldocaml/examples/blob/v1/code/records/main.topscript>

```
# #warnings "+9";;
# let host_info_to_string { hostname = h; os_name = os;
                           cpu_arch = c; timestamp = ts;
                         } =
    sprintf "%s (%s / %s, on %s)" h os c (Time.to_sec_string ts);;
Characters 24-139:
Warning 9: the following labels are not bound in this record pattern:
os_release
Either bind these labels explicitly or add ';' '_' to the pattern.
(Предупреждение 9: следующие имена отсутствуют в образце записи:
os_release
Свяжите эти имена явно или добавьте ';' '_' в образец.)
val host_info_to_string : host_info -> string = <fun>
```

Мы можем отключить вывод предупреждения для данного образца, явно сообщив, что другие поля игнорируются сознательно. Делается это путем добавления символа подчеркивания в образец:

OCaml utop (part 8)

<https://github.com/realworldocaml/examples/blob/v1/code/records/main.topscript>

```
# let host_info_to_string { hostname = h; os_name = os;
                           cpu_arch = c; timestamp = ts; _
                         } =
    sprintf "%s (%s / %s, on %s)" h os c (Time.to_sec_string ts);;
val host_info_to_string : host_info -> string = <fun>
```

Старайтесь всегда включать вывод предупреждений о неполноте образцов записей и явно отключать их с помощью `_` там, где это необходимо.

Предупреждения компилятора

Компилятор OCaml до краев наполнен всевозможными полезными предупреждениями, которые можно включать и выключать по отдельности. Их описания встроены в сам компилятор, поэтому информацию о любом предупреждении можно получить, как показано ниже, на примере предупреждения с кодом 9:

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/records/warn_help.out

```
$ ocaml -warn-help | egrep '\b9\b'
9 Missing fields in a record pattern.
R Synonym for warning 9.
```

Считайте предупреждения OCaml мощным множеством дополнительных инструментов статического анализа и обязательно включайте их в своем окружении сборки. Обычно не требуется включать все предупреждения – во многих случаях вполне достаточно тех, что включены по умолчанию.

Предупреждения, использовавшиеся для сборки примеров в этой книге, определялись с помощью следующего флага: `-w @A-4-33-41-42-43-34-44`.

Синтаксис определения флага можно узнать, выполнив команду `ocaml -help`, но на всякий случай отметим, что флаг выше включает все предупреждения, придавая им статус ошибок, запрещая только предупреждения с кодами, явно перечисленными после A.

Интерпретация предупреждений как ошибок (то есть остановка компиляции сразу после вывода предупреждения) – отличная практика на этапе разработки, потому что в противном случае разработчики часто игнорируют их. Однако когда пакет готовится к распространению, такой подход не сулит ничего хорошего, потому что список предупреждений может расти с каждой новой версией компилятора, из-за чего компиляция вашего пакета с новой версией компилятора может неожиданно потерпеть неудачу.

Уплотнение полей

Когда имя переменной совпадает с именем поля записи, OCaml позволяет использовать более краткий синтаксис. Например, сопоставление с образцом в следующей функции свяжет все указанные поля с переменными, имеющими те же имена. Это называется *уплотнение полей* (field punning):

OCaml utop (part 9)

<https://github.com/realworldocaml/examples/blob/v1/code/records/main.topscript>

```
# let host_info_to_string { hostname; os_name; cpu_arch; timestamp; _ } =
  sprintf "%s (%s / %s) <%s>" hostname os_name cpu_arch
    (Time.to_string timestamp);;
val host_info_to_string : host_info -> string = <fun>
```

Данный прием можно также использовать при создании экземпляров записей. Взгляните на следующий код, генерирующий запись `host_info`:

OCaml utop (part 10)

<https://github.com/realworldocaml/examples/blob/v1/code/records/main.topscript>

```
# let my_host =
  let sh cmd = Shell.sh_one_exn cmd in
  let hostname = sh "hostname" in
  let os_name = sh "uname -s" in
  let cpu_arch = sh "uname -p" in
  let os_release = sh "uname -r" in
  let timestamp = Time.now () in
  { hostname; os_name; cpu_arch; os_release; timestamp };;
val my_host : host_info =
{hostname = "ocaml-www1"; os_name = "Linux"; cpu_arch = "unknown";
 os_release = "3.2.0-1-amd64";
 timestamp = 2013-08-18 14:50:55.287342+01:00}
```

Здесь сначала определяются переменные с именами, соответствующими полям записи, а затем в инструкции создания записи просто перечисляются поля, которые нужно включить.

В определениях функций можно одновременно использовать и уплотнение полей, и уплотнение меток для создания записей из значений аргументов с метками:

OCaml utop (part 11)

<https://github.com/realworldocaml/examples/blob/v1/code/records/main.topscript>

```
# let create_host_info ~hostname ~os_name ~cpu_arch ~os_release =
  { os_name; cpu_arch; os_release;
    hostname = String.lowercase hostname;
    timestamp = Time.now () };;

val create_host_info :
  hostname:string ->
  os_name:string -> cpu_arch:string -> os_release:string -> host_info = <fun>
```

Этот код выглядит намного короче версии, где не использовался прием уплотнения:

OCaml utop (part 12)

<https://github.com/realworldocaml/examples/blob/v1/code/records/main.topscript>

```
# let create_host_info
  ~hostname:hostname ~os_name:os_name
  ~cpu_arch:cpu_arch ~os_release:os_release =
  { os_name = os_name;
    cpu_arch = cpu_arch;
    os_release = os_release;
    hostname = String.lowercase hostname;
    timestamp = Time.now () };;

val create_host_info :
  hostname:string ->
  os_name:string -> cpu_arch:string -> os_release:string -> host_info = <fun>
```

Все вместе – аргументы с метками, имена полей, а также приемы уплотнения – способствуют применению стиля, когда на протяжении всего кода используются одни и те же имена. В общем и целом это считается хорошей практикой, поскольку способствует единообразию в именовании, которое, в свою очередь, упрощает навигацию по исходным текстам программ.

Повторное использование имен полей

Определение разных записей с одинаковыми именами полей может стать источником проблем. Рассмотрим простой пример: создание типов для представления протокола, используемого при регистрации на сервере.

Мы опишем три типа сообщений: `log_entry`, `heartbeat` и `logon`. Сообщение `log_entry` используется для доставки регистрационной информации на сервер; сообщение `logon` отправляется для инициации соединения, оно включает идентификатор пользователя и информацию для аутентификации; и сообщение `heartbeat`

отправляется клиентом периодически для поддержания соединения в открытом состоянии. Все эти сообщения включают идентификатор сеанса и время отправки сообщения:

OCaml utop (part 13)

<https://github.com/realworldocaml/examples/blob/v1/code/records/main.topscript>

```
# type log_entry =
  { session_id: string;
    time: Time.t;
    important: bool;
    message: string;
  }

type heartbeat =
  { session_id: string;
    time: Time.t;
    status_message: string;
  }

type logon =
  { session_id: string;
    time: Time.t;
    user: string;
    credentials: string;
  }

;;

type log_entry = {
  session_id : string;
  time : Time.t;
  important : bool;
  message : string;
}

type heartbeat = {
  session_id : string;
  time : Time.t;
  status_message : string;
}

type logon = {
  session_id : string;
  time : Time.t;
  user : string;
  credentials : string;
}
```

Повторное использование имен полей может привести к некоторой неоднозначности. Например, представьте, что мы пишем функцию для извлечения `session_id` из записи, значение какого типа мы получим?

OCaml utop (part 14)

<https://github.com/realworldocaml/examples/blob/v1/code/records/main.topscript>

```
# let get_session_id t = t.session_id;;
val get_session_id : logon -> string = <fun>
```

В данном случае компилятор OCaml просто выбрал самое последнее определение поля записи. Мы можем вынудить OCaml предположить, что используется другой тип (например, `heartbeat`), добавив аннотацию с описанием типа:

OCaml utop (part 15)

<https://github.com/realworldocaml/examples/blob/v1/code/records/main.topscript>

```
# let get_heartbeat_session_id (t:heartbeat) = t.session_id;;
val get_heartbeat_session_id : heartbeat -> string = <fun>
```

Несмотря на возможность использования аннотаций для разрешения неоднозначности в именах полей, неоднозначность может проявиться даже там, где ее, казалось бы, не должно быть. Взгляните на следующую функцию, извлекающую идентификатор сеанса и код состояния из экземпляра записи типа `heartbeat`:

OCaml utop (part 16)

<https://github.com/realworldocaml/examples/blob/v1/code/records/main.topscript>

```
# let status_and_session t = (t.status_message, t.session_id);;
val status_and_session : heartbeat -> string * string = <fun>
# let session_and_status t = (t.session_id, t.status_message);;
Characters 44-58:
Error: The record type logon has no field status_message
(Ошибка: Запись типа logon не имеет поля с именем status_message)

# let session_and_status (t:heartbeat) = (t.session_id, t.status_message);;
val session_and_status : heartbeat -> string * string = <fun>
```

Почему первое определение скомпилировалось успешно без аннотаций типов, а второе – нет? Разница в том, что в первом случае механизм контроля типов первым встретил поле `status_message` и заключил, что запись имеет тип `heartbeat`. Когда порядок полей поменялся и первым встретилось поле `session_id`, компилятор решил, что имеет дело с типом `logon` и выражение `t.status_message` не имеет никакого смысла.

Мы можем избавиться от неоднозначности, либо используя отличающиеся имена полей, либо, что в общем случае предпочтительнее, создавая отдельный модуль для каждого типа. Упаковывание типов в модули является довольно полезной идиомой (и широко применяется в библиотеке `Core`), обеспечивающей отдельное пространство имен для каждого типа. При использовании такого стиля определяемым типам принято давать имя `t`, то есть мы можем написать такой код:

OCaml utop (part 17)

<https://github.com/realworldocaml/examples/blob/v1/code/records/main.topscript>

```
# module Log_entry = struct
  type t =
    { session_id: string;
      time: Time.t;
      important: bool;
      message: string;
    }
end
```



```

end
module Heartbeat = struct
  type t =
    { session_id: string;
      time: Time.t;
      status_message: string;
    }
end
module Logon = struct
  type t =
    { session_id: string;
      time: Time.t;
      user: string;
      credentials: string;
    }
end;;
module Log_entry :
sig
  type t = {
    session_id : string;
    time : Time.t;
    important : bool;
    message : string;
  }
end
module Heartbeat :
sig
  type t = { session_id : string; time : Time.t; status_message : string; }
end
module Logon :
sig
  type t = {
    session_id : string;
    time : Time.t;
    user : string;
    credentials : string;
  }
end
end

```

Теперь функцию создания `log_entry` можно записать так:

OCaml utop (part 18)

<https://github.com/realworldocaml/examples/blob/v1/code/records/main.topscript>

```

# let create_log_entry ~session_id ~important message =
  { Log_entry.time = Time.now (); Log_entry.session_id;
    Log_entry.important; Log_entry.message }
;;
val create_log_entry :
session_id:string -> important:bool -> string -> Log_entry.t = <fun>

```

Квалификация имен полей именем модуля `Log_entry` является обязательным требованием, потому что данная функция находится за пределами модуля `Log_entry`, где определена запись. Однако компилятор OCaml требует квалифициро-

вать только одно имя поля записи, поэтому эту функцию можно записать более кратко. Отметим, что допускается вставлять пробельные символы между путем к модулю и именем поля:

OCaml utop (part 19)

<https://github.com/realworldocaml/examples/blob/v1/code/records/main.topscript>

```
# let create_log_entry ~session_id ~important message =
  { Log_entry.
    time = Time.now (); session_id; important; message }
;;

val create_log_entry :
  session_id:string -> important:bool -> string -> Log_entry.t = <fun>
```

Этот прием можно использовать не только для создания экземпляров записей. Тот же трюк можно применять и в выражениях сопоставления с образцом:

OCaml utop (part 20)

<https://github.com/realworldocaml/examples/blob/v1/code/records/main.topscript>

```
# let message_to_string { Log_entry.important; message; _ } =
  if important then String.uppercase message else message
;;

val message_to_string : Log_entry.t -> string = <fun>
```

Используя точечную нотацию для доступа к полям записей, допускается квалифицировать имена полей именами модулей прямо в выражении:

OCaml utop (part 21)

<https://github.com/realworldocaml/examples/blob/v1/code/records/main.topscript>

```
# let is_important t = t.Log_entry.important;;
val is_important : Log_entry.t -> bool = <fun>
```

Первое время такой синтаксис может казаться непривычным. Поэтому здесь важно понимать, что точка может использоваться в двух ипостасях. Во-первых, точка – это оператор обращения к полю записи. Все, что находится правее, считается именем поля. Во-вторых, точка – это оператор доступа к содержимому модуля. В данном случае она позволяет сослаться на поле `important` в модуле `Log_entry`. Тот факт, что имя `Log_entry` начинается с большой буквы и потому никак не может быть именем поля, устраняет неоднозначность между этими двумя ипостасями.

Для функций, определяемых внутри модуля, где объявлена данная запись, квалификация именем модуля вообще не нужна.

Функциональные обновления

Очень часто приходится определять новые записи, отличающиеся от существующих лишь узким подмножеством полей. Например, представьте, что наша служба регистрации имеет тип записи для представления состояния клиента, включая отметку времени, когда последний раз от клиента было принято сообщение `heartbeat`. Ниже приводится определение типа для представления этой инфор-

мации, а также функция, обновляющая информацию о клиенте, когда поступает новое сообщение heartbeat:

OCaml utop (part 22)

<https://github.com/realworldocaml/examples/blob/v1/code/records/main.topscript>

```
# type client_info =
  { addr: Unix.Inet_addr.t;
    port: int;
    user: string;
    credentials: string;
    last_heartbeat_time: Time.t;
  };;

type client_info = {
  addr : UnixLabels.inet_addr;
  port : int;
  user : string;
  credentials : string;
  last_heartbeat_time : Time.t;
}

# let register_heartbeat t hb =
  { addr = t.addr;
    port = t.port;
    user = t.user;
    credentials = t.credentials;
    last_heartbeat_time = hb.Heartbeat.time;
  };;

val register_heartbeat : client_info -> Heartbeat.t -> client_info = <fun>
```

Такое определение выглядит слишком многословным, учитывая, что требуется изменить только одно поле, а все остальные просто копируются из `t`. Для большей краткости мы можем использовать синтаксис *функционального обновления* (functional update), поддерживаемого языком OCaml. Этот синтаксис имеет следующий вид:

Syntax

https://github.com/realworldocaml/examples/blob/v1/code/records/functional_update.syntax

```
{ <запись> with <поле> = <значение>;
  <поле> = <значение>;
  ...
}
```

Цель приема функционального обновления – создать новую запись на основе имеющейся, с изменением значений некоторых полей.

Теперь функцию `register_heartbeat` можно записать более кратко:

OCaml utop (part 23)

<https://github.com/realworldocaml/examples/blob/v1/code/records/main.topscript>

```
# let register_heartbeat t hb =
  { t with last_heartbeat_time = hb.Heartbeat.time };;

val register_heartbeat : client_info -> Heartbeat.t -> client_info = <fun>
```

Функциональные обновления делают код независимым от идентичности полей в записи, которые не изменяются. Зачастую это именно то, что вам требуется, но у этого приема есть и свои недостатки. В частности, если изменится определение записи и в нее добавятся дополнительные поля, система типов не предложит вам пересмотреть свой код, чтобы определить, нужно ли его адаптировать с учетом появления новых полей. Взгляните, что произойдет, если мы решим добавить поле с состоянием последнего принятого сообщения `heartbeat`:

OCaml utop (part 24)

<https://github.com/realworldocaml/examples/blob/v1/code/records/main.topscript>

```
# type client_info =
  { addr: Unix.Inet_addr.t;
    port: int;
    user: string;
    credentials: string;
    last_heartbeat_time: Time.t;
    last_heartbeat_status: string;
  };;

type client_info = {
  addr : UnixLabels.inet_addr;
  port : int;
  user : string;
  credentials : string;
  last_heartbeat_time : Time.t;
  last_heartbeat_status : string;
}
```

Теперь первоначальная реализация функции `register_heartbeat` может оказаться ошибочной, и компилятор вполне мог бы предупредить нас о появлении неуученного нового поля. Но версия, использующая прием функционального обновления, продолжает компилироваться как ни в чем не бывало, даже при том, что она ошибочно игнорирует новое поле. Чтобы исправить эту проблему, следовало бы обновить код, как показано ниже:

OCaml utop (part 25)

<https://github.com/realworldocaml/examples/blob/v1/code/records/main.topscript>

```
# let register_heartbeat t hb =
  { t with last_heartbeat_time = hb.Heartbeat.time;
    last_heartbeat_status = hb.Heartbeat.status_message;
  };;

val register_heartbeat : client_info -> Heartbeat.t -> client_info = <fun>
```

Изменяемые поля

Как и большинство значений в языке OCaml, записи по умолчанию являются неизменяемыми. Однако отдельные поля можно объявлять изменяемыми. В следующем примере два последних поля в записи `client_info` объявлены изменяемыми:

OCaml utop (part 26)

<https://github.com/realworldocaml/examples/blob/v1/code/records/main.topscript>

```
# type client_info =
  { addr: Unix.Inet_addr.t;
    port: int;
    user: string;
    credentials: string;
    mutable last_heartbeat_time: Time.t;
    mutable last_heartbeat_status: string;
  };;

type client_info = {
  addr : UnixLabels.inet_addr;
  port : int;
  user : string;
  credentials : string;
  mutable last_heartbeat_time : Time.t;
  mutable last_heartbeat_status : string;
}
```

Изменение значений таких полей выполняется с помощью оператора <-. Версию register_heartbeat с побочным эффектом можно было бы реализовать так:

OCaml utop (part 27)

<https://github.com/realworldocaml/examples/blob/v1/code/records/main.topscript>

```
# let register_heartbeat t hb =
  t.last_heartbeat_time <- hb.Heartbeat.time;
  t.last_heartbeat_status <- hb.Heartbeat.status_message
;;

val register_heartbeat : client_info -> Heartbeat.t -> unit = <fun>
```

Обратите внимание, что оператор <- не требуется для инициализации, потому что все поля записи, включая изменяемые, определяются в момент создания записи.

Политика «неизменяемость по умолчанию», применяемая в языке OCaml, является благом, но императивный стиль также является важной частью программирования на OCaml. Подробнее о том, как (и когда) следует использовать императивные особенности OCaml, мы рассказывали в разделе «Императивное программирование» в главе 1.

Поля первого порядка

Рассмотрим функцию извлечения имен пользователей из списка сообщений типа Logon:

OCaml utop (part 28)

<https://github.com/realworldocaml/examples/blob/v1/code/records/main.topscript>

```
# let get_users logons =
  List.dedup (List.map logons ~f:(fun x -> x.Logon.user));;

val get_users : Logon.t list -> string list = <fun>
```

Здесь для доступа к полю `user` мы определили небольшую функцию (`fun x -> x.Logon.user`). Такой прием реализации функций доступа настолько часто используется в практике, что со временем была добавлена возможность автоматического создания таких функций. Эта возможность реализована в библиотеке `fieldslib` расширения синтаксиса, распространяемой в составе `Core`.

Аннотация `with fields` в конце объявления типа записи заставит компилятор применить расширение к данному объявлению типа. Например, мы могли бы объявить тип `Logon`, как показано ниже:

OCaml utop

<https://github.com/realworldocaml/examples/blob/v1/code/records/main-29.rawscript>

```
# module Logon = struct
  type t =
    { session_id: string;
      time: Time.t;
      user: string;
      credentials: string;
    }
  with fields
end;;
module Logon :
sig
  type t = {
    session_id : string;
    time : Time.t;
    user : string;
    credentials : string;
  }
  val credentials : t -> string
  val user : t -> string
  val time : t -> Time.t
  val session_id : t -> string
  module Fields :
  sig
    val names : string list
    val credentials :
      ([< `Read | `Set_and_create ], t, string) Field.t_with_perm
    val user :
      ([< `Read | `Set_and_create ], t, string) Field.t_with_perm
    val time :
      ([< `Read | `Set_and_create ], t, Time.t) Field.t_with_perm
    val session_id :
      ([< `Read | `Set_and_create ], t, string) Field.t_with_perm

    [ ... еще множество определений опущено ... ]
  end
end
```

Обратите внимание, что компилятор сгенерировал довольно *длинный* вывод из-за того, что `fieldslib` добавила в определение типа множество вспомогательных функций для работы с полями записи. Мы рассмотрим только некоторые из них; с назначением остальных вы сможете ознакомиться в документации к `fieldslib`.

Одна из функций, которые мы получили, называется `Logon.user`. Мы будем использовать ее для извлечения поля `user` из сообщения `logon`:

OCaml utop (part 30)

<https://github.com/realworldocaml/examples/blob/v1/code/records/main.topscript>

```
# let get_users logons = List.dedup (List.map logons ~f:Logon.user) ;;
val get_users : Logon.t list -> string list = <fun>
```

Помимо функций доступа к полям, `fieldslib` создала также вложенный модуль с именем `Fields`, содержащий представления всех полей в форме значений типа `Field.t`. Модуль `Field` предоставляет следующие функции:

- **Field.name** – возвращает имя поля;
- **Field.get** – возвращает значение поля;
- **Field.fset** – реализует функциональное обновление поля;
- **Field.setter** – возвращает `None`, если поле является неизменяемым, и `Some f` в противном случае, где `f` – это функция для изменения данного поля.

Тип `Field.t` имеет два параметра типа: первый определяет тип записи, а второй – тип текущего поля. То есть поле `Logon.Fields.session_id` имеет тип `(Logon.t, string) Field.t`, а поле `Logon.Fields.time` имеет тип `(Logon.t, Time.t) Field.t`. Соответственно, если вызвать функцию `Field.get` для поля `Logon.Fields.user`, она вернет функцию извлечения поля `user` из `Logon.t`:

OCaml utop (part 31)

<https://github.com/realworldocaml/examples/blob/v1/code/records/main.topscript>

```
# Field.get Logon.Fields.user;;
- : Logon.t -> string = <fun>
```

Здесь первый параметр `Field.t` соответствует записи, переданной функции `get`, а второй параметр соответствует значению, содержащемуся в поле, которое также является типом возвращаемого значения `get`.

Тип функции `Field.get` оказался немного сложнее, чем можно было бы ожидать:

OCaml utop (part 32)

<https://github.com/realworldocaml/examples/blob/v1/code/records/main.topscript>

```
# Field.get;;
- : ('b, 'r, 'a) Field.t_with_perm -> 'r -> 'a = <fun>
```

Как видите, вместо `Field.t` используется тип `Field.t_with_perm`, потому что доступ к полям регулируется механизмом доступа, который вступает в работу в некоторых специальных случаях, когда мы экспортируем возможность читать поля из записей, но не создавать новые записи. Поэтому мы лишаемся возможности использовать прием функционального обновления.

Мы можем использовать поля первого порядка, например, чтобы реализовать обобщенную функцию вывода значения поля записи:

OCaml utop (part 33)

<https://github.com/realworldocaml/examples/blob/v1/code/records/main.topscript>

```
# let show_field field to_string record =
  let name = Field.name field in
```

```

    let field_string = to_string (Field.get field record) in
    name ^ ": " ^ field_string
;;
val show_field :
('a, 'b, 'c) Field.t_with_perm -> ('c -> string) -> 'b -> string = <fun>

```

Она принимает три аргумента: `Field.t`, функцию преобразования содержимого поля в строку и запись, в которой находится поле.

Ниже приводится пример применения функции `show_field`:

OCaml utop (part 34)

<https://github.com/realworldocaml/examples/blob/v1/code/records/main.topscript>

```

# let logon = { Logon.
    session_id = "26685";
    time = Time.now ();
    user = "yminsky";
    credentials = "Xy2d9W"; }
;;
val logon : Logon.t =
{Logon.session_id = "26685"; time = 2013-08-18 14:51:00.509463+01:00;
 user = "yminsky"; credentials = "Xy2d9W"}
# show_field Logon.Fields.user Fn.id logon;;
- : string = "user: yminsky"
# show_field Logon.Fields.time Time.to_string logon;;
- : string = "time: 2013-08-18 14:51:00.509463+01:00"

```

Отметьте, кстати, что в этом примере мы впервые использовали модуль `Fn` (сокращенно от «function»), содержащий коллекцию интересных примитивов для работы с функциями. Например, `Fn.id` – это функция идентичности.

Библиотека `fieldslib` также реализует операторы высшего порядка, такие как `Fields.fold` и `Fields.iter`, позволяющие осуществлять обход полей записи. Так, например, в случае с записью `Logon.t` итератор полей имеет следующий тип:

OCaml utop (part 35)

<https://github.com/realworldocaml/examples/blob/v1/code/records/main.topscript>

```

# Logon.Fields.iter;;
- : session_id:([< `Read | `Set_and_create ], Logon.t, string)
    Field.t_with_perm -> 'a) ->
    time:([< `Read | `Set_and_create ], Logon.t, Time.t) Field.t_with_perm ->
    'b) ->
    user:([< `Read | `Set_and_create ], Logon.t, string) Field.t_with_perm ->
    'c) ->
    credentials:([< `Read | `Set_and_create ], Logon.t, string)
    Field.t_with_perm -> 'd) ->
    'd
= <fun>

```

Выглядит немного устрашающе, в основном из-за маркеров управления доступом, но сама структура достаточно проста. Каждый аргумент с меткой является функцией, принимающей поле первого порядка требуемого типа. Обратите внимание, что `iter` передает каждой из этих функций обратного вызова тип `Field.t`, а

не содержимое поля записи. Тем не менее содержимое поля можно получить с помощью комбинации записи и `Field.t`.

Теперь давайте задействуем оператор `Logon.Fields.iter` и функцию `show_field` для вывода всех полей в записи `Logon`:

OCaml utop (part 36)

<https://github.com/realworldocaml/examples/blob/v1/code/records/main.topscript>

```
# let print_logon logon =
  let print_to_string field =
    printf "%s\n" (show_field field to_string logon)
  in
  Logon.Fields.iter
    ~session_id:(print Fn.id)
    ~time:(print Time.to_string)
    ~user:(print Fn.id)
    ~credentials:(print Fn.id)
;;
val print_logon : Logon.t -> unit = <fun>
# print_logon logon;;
session_id: 26685
time: 2013-08-18 14:51:00.509463+01:00
user: yminsky
credentials: Xy2d9W
- : unit = ()
```

Этот прием имеет один замечательный побочный эффект — он помогает адаптировать программный код при изменении набора полей в записи. Если так случится, что в запись `Logon.t` будет добавлено новое поле, тип `Logon.Fields.iter` изменится соответственно, получив новый параметр. Никакой код, использующий `Logon.Fields.iter`, не будет компилироваться, пока не будет исправлен с учетом нового аргумента.

Итераторы по полям записей могут пригодиться при решении самых разных задач, где используются записи, от создания функций проверки допустимости значений в полях записей до автоматического создания определений веб-форм на основе типа записи. Такие приложения способны гарантировать учет всех полей записи.

Глава 6

Варианты

Вариантные типы являются одной из самых полезных возможностей языка OCaml, а также одной из самых необычных. Они позволяют представлять данные, которые могут принимать самые разные формы, где каждая форма явно помечается явным тегом. Как будет показано в этой главе, при объединении с сопоставлением с образцом варианты обеспечивают удобный способ представления комплексных данных и организации их анализа.

Ниже приводится базовый синтаксис объявления вариантного типа:

Syntax

<https://github.com/realworldocaml/examples/blob/v1/code/variants/variant.syntax>

```
type <вариант> =  
  | <Тег> [ of <тип> [* <тип>]... ]  
  | <Тег> [ of <тип> [* <тип>]... ]  
  | ...
```

Каждая строка описывает одну из форм представления вариантного типа. Для каждой формы определяется тег и дополнительно может определяться последовательность полей, где каждое поле имеет указанный тип.

Рассмотрим конкретный пример использования вариантов. Практически все терминалы поддерживают восемь основных цветов, и мы можем организовать их представление с помощью варианта. Все цвета объявляются как простые теги и разделяются символом вертикальной черты (|). Обратите внимание, что теги вариантов должны начинаться с большой буквы:

OCaml utop

<https://github.com/realworldocaml/examples/blob/v1/code/variants/main.topscript>

```
# type basic_color =  
  | Black | Red | Green | Yellow | Blue | Magenta | Cyan | White ;;  
type basic_color =  
  Black  
  | Red  
  | Green  
  | Yellow  
  | Blue  
  | Magenta  
  | Cyan  
  | White  
# Cyan ;;  
- : basic_color = Cyan  
# [Blue; Magenta; Red] ;;  
- : basic_color list = [Blue; Magenta; Red]
```

Следующая функция использует сопоставление с образцом для преобразования `basic_color` в соответствующее целое число. Проверка полноты выражения сопоставления с образцом, выполняемая компилятором, гарантирует вывод предупреждения, если мы пропустим какой-то цвет:

OCaml utop (part 1)

<https://github.com/realworldocaml/examples/blob/v1/code/variants/main.topscript>

```
# let basic_color_to_int = function
  | Black -> 0 | Red      -> 1 | Green -> 2 | Yellow -> 3
  | Blue  -> 4 | Magenta -> 5 | Cyan  -> 6 | White -> 7 ;;
val basic_color_to_int : basic_color -> int = <fun>
# List.map ~f:basic_color_to_int [Blue;Red];;
- : int list = [4; 1]
```

С помощью этой функции мы сможем сгенерировать соответствующие экранированные последовательности (escape-последовательности) для изменения цвета строки при отображении ее в терминале:

OCaml utop

<https://github.com/realworldocaml/examples/blob/v1/code/variants/main-2.rawscript>

```
# let color_by_number number text =
  sprintf "\027[38;5;%dm%s\027[0m" number text;;
val color_by_number : int -> string -> string = <fun>
# let blue = color_by_number (basic_color_to_int Blue) "Blue";;
val blue : string = "\027[38;5;4mBlue\027[0m"
# printf "Hello %s World!\n" blue;;
Hello Blue World!
```

В большинстве терминалов слово «Blue» будет выведено синим цветом.

В этом примере формы варианта являются простыми тегами, с которыми не связаны никакие данные. В таком виде вариантный тип действует подобно перечислениям в языке C и Java. Но, как мы увидим ниже, варианты способны на большее, чем представлять простые перечисления. Вообще говоря, простого перечисления недостаточно для эффективного описания полного множества цветов, поддерживаемого современными терминалами. Многие терминалы, включая почтенный `xterm`, поддерживают 256 разных цветов, разбивая их на следующие группы:

- восемь базовых цветов для простого и жирного шрифтов;
- цветовой куб RGB размером 6×6×6;
- 24-уровневая палитра оттенков серого цвета.

Мы также реализуем представление этого, более сложного цветового пространства в виде вариантного типа, но на этот раз разные теги будут снабжаться аргументами, описывающими данные. Имейте в виду, что варианты могут иметь множество аргументов, разделенных звездочками (*):

OCaml utop (part 3)

<https://github.com/realworldocaml/examples/blob/v1/code/variants/main.topscript>

```
# type weight = Regular | Bold
type color =
  | Basic of basic_color * weight (* базовые цвета, обычный и жирный шрифты *)
```

```

| RGB of int * int * int      (* цветовой куб 6x6x6 *)
| Gray of int                  (* 24-уровневая палитра оттенков серого *)
;;
type weight = Regular | Bold
type color =
  Basic of basic_color * weight
  | RGB of int * int * int
  | Gray of int
# [RGB (250,70,70); Basic (Green, Regular)];;
- : color list = [RGB (250, 70, 70); Basic (Green, Regular)]

```

Как и прежде, для преобразования цвета в соответствующее целое число будет использоваться сопоставление с образцом. Но в данном случае сопоставление будет делать намного больше, чем просто определять ту или иную форму вариантного типа, — с его помощью мы будем извлекать данные, ассоциированные с тегами:

OCaml utop (part 4)

<https://github.com/realworldocaml/examples/blob/v1/code/variants/main.topscript>

```

# let color_to_int = function
  | Basic (basic_color, weight) ->
    let base = match weight with Bold -> 8 | Regular -> 0 in
    base + basic_color_to_int basic_color
  | RGB (r,g,b) -> 16 + b + g * 6 + r * 36
  | Gray i -> 232 + i ;;
val color_to_int : color -> int = <fun>

```

Теперь мы имеем возможность выводить текст, используя полное множество доступных цветов:

OCaml utop

<https://github.com/realworldocaml/examples/blob/v1/code/variants/main-5.rawscript>

```

# let color_print color s =
  printf "%s\n" (color_by_number (color_to_int color) s) ;;
val color_print : color -> string -> unit = <fun>
# color_print (Basic (Red,Bold)) "A bold red!";;
A bold red!
# color_print (Gray 4) "A muted gray...";;
A muted gray...

```

Универсальные образцы и рефакторинг

Система типов в языке OCaml может действовать как инструмент рефакторинга, предупреждая о необходимости приведения фрагментов кода в соответствие с изменившимся интерфейсом. Это особенно ценно в отношении вариантных типов.

Взгляните, что случится, если изменить определение `color`, как показано ниже:

OCaml utop (part 1)

https://github.com/realworldocaml/examples/blob/v1/code/variants/catch_all.topscript

```

# type color =
  | Basic of basic_color      (* базовые цвета *)
  | Bold of basic_color      (* базовые цвета, жирный шрифт *)

```

```

| RGB of int * int * int (* цветовой куб 6х6х6 *)
| Gray of int             (* 24-уровневая палитра оттенков серого *)
;;
type color =
  Basic of basic_color
| Bold of basic_color
| RGB of int * int * int
| Gray of int

```

Мы, по сути, разбили базовый случай `Basic` на два, `Basic` и `Bold`, и теперь вариант `Basic` имеет единственный аргумент вместо двух. Однако функция `color_to_int` все еще ожидает получить вариантный тип со старой структурой, поэтому если попробовать скомпилировать ее, компилятор заметит несоответствие:

OCaml utop (part 2)

https://github.com/realworldocaml/examples/blob/v1/code/variants/catch_all.topscript

```

# let color_to_int = function
  | Basic (basic_color, weight) ->
    let base = match weight with Bold -> 8 | Regular -> 0 in
    base + basic_color_to_int basic_color
  | RGB (r,g,b) -> 16 + b + g * 6 + r * 36
  | Gray i -> 232 + i ;;

```

Characters 34-60:

```

Error: This pattern matches values of type 'a * 'b
      but a pattern was expected which matches values of type basic_color
(Ошибка: Этому образцу соответствуют значения типа 'a * 'b, тогда как
      ожидается образец, соответствующий значениям типа basic_color)

```

Здесь компилятор заметил, что тер `Basic` используется с неверным числом аргументов. Но стоит исправить эту проблему, как компилятор сразу же заметит другую – отсутствие образца для нового тег `Bold`:

OCaml utop (part 3)

https://github.com/realworldocaml/examples/blob/v1/code/variants/catch_all.topscript

```

# let color_to_int = function
  | Basic basic_color -> basic_color_to_int basic_color
  | RGB (r,g,b) -> 16 + b + g * 6 + r * 36
  | Gray i -> 232 + i ;;

```

Characters 19-154:

```

Warning 8: this pattern-matching is not exhaustive.
Here is an example of a value that is not matched:
Bold _
(Предупреждение 8: это сопоставление не является исчерпывающим.
Ниже следует пример значения, не соответствующего сопоставлению:
Bold _ )
val color_to_int : color -> int = <fun>

```

Исправив эту проблему, мы получим правильную реализацию:

OCaml utop (part 4)

https://github.com/realworldocaml/examples/blob/v1/code/variants/catch_all.topscript

```

# let color_to_int = function
  | Basic basic_color -> basic_color_to_int basic_color

```

```
| Bold basic_color -> 8 + basic_color_to_int basic_color
| RGB (r,g,b) -> 16 + b + g * 6 + r * 36
| Gray i -> 232 + i ;;

val color_to_int : color -> int = <fun>
```

Как видите, ошибки в типах идентифицируют участки кода, которые следует поправить, чтобы завершить рефакторинг. Это фантастически удобное свойство компилятора, но, чтобы оно работало надежно, необходимо писать код так, чтобы у компилятора были все шансы помочь нам в поиске ошибок. С этой целью избегайте использования универсальных образцов в выражениях сопоставления.

Ниже приводится пример, иллюстрирующий влияние универсального образца на проверку полноты охвата в выражении сопоставления. Представьте, что нам нужна версия функции `color_to_int`, способная работать со старыми терминалами, поддерживающими только первые 16 цветов (восемь `basic_colors` для обычного и жирного шрифтов), и с новыми, поддерживающими все остальные цвета, но эти цвета должны интерпретироваться как белый. Такую функцию можно написать, как показано ниже:

OCaml utop (part 5)

https://github.com/realworldocaml/examples/blob/v1/code/variants/catch_all.topscript

```
# let oldschool_color_to_int = function
  | Basic (basic_color,weight) ->
    let base = match weight with Bold -> 8 | Regular -> 0 in
    base + basic_color_to_int basic_color
  | _ -> basic_color_to_int White;;
Characters 44-70:
Error: This pattern matches values of type 'a * 'b
but a pattern was expected which matches values of type basic_color
(Ошибка: Этому образцу соответствуют значения типа 'a * 'b', тогда как
ождается образец, соответствующий значениям типа basic_color)
```

Но из-за того, что универсальный образец соответствует всем возможным случаям, система типов больше не будет предупреждать об отсутствии образца, соответствующего новому варианту `Bold`, когда мы включим его в новое определение типа. Но мы можем вернуть проверку обратно, исключив универсальный образец и добавив теги, которые будут игнорироваться.

Объединение записей и вариантов

Для описания коллекции типов, включающей варианты, записи и кортежи, часто используется термин *алгебраические типы данных*. Алгебраические типы данных действуют как особенно удобный и мощный язык описания данных. Основу их удобства составляют две разновидности типов: *типы произведений* (product types), такие как кортежи и записи, которые способны объединять разные типы и с математической точки зрения напоминают декартовы произведения; и *суммарные типы* (sum types), такие как варианты, которые позволяют составлять комбинации из множества разных вариантов в рамках одного типа и с математической точки зрения напоминают несвязные объединения (disjoint unions).

Широта возможностей алгебраических типов данных во многом обусловлена способностью представлять конструкции из многослойных комбинаций суммарных типов и типов произведений. Давайте посмотрим, чего можно достичь с их помощью на примере типов, применявшихся для реализации регистрации на сервере в главе 5. Для начала вспомним определение `Log_entry.t`:

OCaml utop (part 1)

<https://github.com/realworldocaml/examples/blob/v1/code/variants/logger.topscript>

```
# module Log_entry = struct
  type t =
    { session_id: string;
      time: Time.t;
      important: bool;
      message: string;
    }
end
;;
module Log_entry :
sig
  type t = {
    session_id : string;
    time : Time.t;
    important : bool;
    message : string;
  }
end
```

Этот тип записи объединяет множество разнотипных полей данных в единое значение. В частности, единственное значение типа `Log_entry.t` хранит идентификатор сеанса `session_id`, *и* время `time`, *и* флаг `important`, *и* сообщение `message`. Проще говоря, типы записей можно считать конъюнкцией (произведением) типов. Варианты, напротив, являются дизъюнкцией (суммой) типов и позволяют представить множество вариантов (форм) единственного значения, как показано в следующем примере:

OCaml utop (part 2)

<https://github.com/realworldocaml/examples/blob/v1/code/variants/logger.topscript>

```
# type client_message = | Logon of Logon.t
                        | Heartbeat of Heartbeat.t
                        | Log_entry of Log_entry.t
;;
type client_message =
  Logon of Logon.t
| Heartbeat of Heartbeat.t
| Log_entry of Log_entry.t
```

Значением типа `client_message` может быть значение типа `Logon`, *или* `Heartbeat`, *или* `Log_entry`. Если потребуется написать обобщенный код обработки любых сообщений, нам как раз пригодится тип `client_message`, позволяющий манипулировать различными допустимыми формами (вариантами) сообщений. В своем коде мы

сможем выполнить сопоставление для значения типа `client_message`, чтобы определить конкретный тип текущего обрабатываемого сообщения.

Вы можете повысить точность своих типов, используя варианты для представления различных форм и записи для представления общей структуры. Взгляните на следующую функцию, принимающую список значений типа `client_message` и возвращающую все сообщения, сгенерированные указанным пользователем. Данная функция выполняет операцию свертки списка сообщений, где роль аккумулятора играет пара:

- множество идентификаторов сеансов для данного пользователя;
- множество сообщений, сгенерированных данным пользователем.

Вот конкретная реализация:

OCaml utop (part 3)

<https://github.com/realworldocaml/examples/blob/v1/code/variants/logger.topscript>

```
# let messages_for_user user messages =
  let (user_messages, _) =
    List.fold messages ~init:([],String.Set.empty)
      ~f:(fun ((messages,user_sessions) as acc) message ->
        match message with
        | Logon m ->
          if m.Logon.user = user then
            (message::messages, Set.add user_sessions m.Logon.session_id)
          else acc
        | Heartbeat _ | Log_entry _ ->
          let session_id = match message with
          | Logon m -> m.Logon.session_id
          | Heartbeat m -> m.Heartbeat.session_id
          | Log_entry m -> m.Log_entry.session_id
          in
          if Set.mem user_sessions session_id then
            (message::messages,user_sessions)
          else acc
      )
  in
  List.rev user_messages
;;
val messages_for_user : string -> client_message list -> client_message list =
<fun>
```

В этой реализации довольно неуклюже смотрится определение идентификатора сеанса. Имеются также повторяющиеся участки кода для каждой из возможных форм сообщений (включая вариант `Logon`, который фактически является невозможным в этой точке), извлекающие идентификатор сеанса. Такой способ обработки сообщений выглядит избыточным, потому что поле идентификатора сеанса присутствует во всех формах сообщений.

Мы можем оптимизировать этот код, произведя рефакторинг наших типов, явно обозначив, какая информация имеется во всех типах сообщений. Сначала сократим определения записей для каждого типа сообщений, оставив в них только уникальные поля:

OCaml utop (part 4)

<https://github.com/realworldocaml/examples/blob/v1/code/variants/logger.topscript>

```
# module Log_entry = struct
  type t = { important: bool;
            message: string;
            }
end
module Heartbeat = struct
  type t = { status_message: string; }
end
module Logon = struct
  type t = { user: string;
            credentials: string;
            }
end ;;

module Log_entry : sig type t = { important : bool; message : string; } end
module Heartbeat : sig type t = { status_message : string; } end
module Logon : sig type t = { user : string; credentials : string; } end
```

Затем определим вариантный тип, объединяющий все эти типы:

OCaml utop (part 5)

<https://github.com/realworldocaml/examples/blob/v1/code/variants/logger.topscript>

```
# type details =
  | Logon of Logon.t
  | Heartbeat of Heartbeat.t
  | Log_entry of Log_entry.t
;;

type details =
  Logon of Logon.t
  | Heartbeat of Heartbeat.t
  | Log_entry of Log_entry.t
```

Нам также потребуется запись, содержащая поля, общие для всех сообщений:

OCaml utop (part 6)

<https://github.com/realworldocaml/examples/blob/v1/code/variants/logger.topscript>

```
# module Common = struct
  type t = { session_id: string;
            time: Time.t;
            }
end ;;

module Common : sig type t = { session_id : string; time : Time.t; } end
```

Теперь полное сообщение можно представить как пару типов `Common.t` и `details`. С учетом описанных изменений можно переписать функцию, как показано ниже:

OCaml utop (part 7)

<https://github.com/realworldocaml/examples/blob/v1/code/variants/logger.topscript>

```
# let messages_for_user user messages =
  let (user_messages, _) =
    List.fold messages ~init:([],String.Set.empty)
```

```

~f:(fun ((messages,user_sessions) as acc) ((common,details) as message) ->
  let session_id = common.Common.session_id in
  match details with
  | Logon m ->
    if m.Logon.user = user then
      (message::messages, Set.add user_sessions session_id)
    else acc
  | Heartbeat _ | Log_entry _ ->
    if Set.mem user_sessions session_id then
      (message::messages,user_sessions)
    else acc
  )
in
List.rev user_messages
;;
val messages_for_user :
  string -> (Common.t * details) list -> (Common.t * details) list = <fun>

```

Как видите, код, извлекающий идентификатор сеанса, можно заменить простым выражением `common.Common.session_id`.

Помимо всего прочего, такая организация дает возможность сразу же приводить данные к конкретному типу сообщения, как только он становится известен, и затем передавать их коду, обрабатывающему сообщения данного типа. В частности, для представления произвольного сообщения можно использовать тип `Common.t * details`, а для представления сообщения инициации соединения – тип `Common.t * Logon.t`. То есть, если бы у нас имелись функции для обработки сообщений каждого типа, мы могли бы написать функцию передачи сообщений этим функциям, как показано ниже:

OCaml utop (part 8)

<https://github.com/realworldocaml/examples/blob/v1/code/variants/logger.topscript>

```

# let handle_message server_state (common,details) =
  match details with
  | Log_entry m -> handle_log_entry server_state (common,m)
  | Logon m -> handle_logon server_state (common,m)
  | Heartbeat m -> handle_heartbeat server_state (common,m)
;;

```

В результате функция `handle_log_entry` получала бы только сообщения типа `Log_entry`, функция `handle_logon` – только сообщения `Logon` и т. д.

Варианты и рекурсивные структуры данных

Другое типичное применение вариантов – представление древовидных, рекурсивных структур данных. Продемонстрируем это на примере реализации простого языка логических выражений. Такой язык может пригодиться везде, где требуется организовать фильтрацию, – от средств анализа пакетов до программ – клиентов электронной почты.

Выражение на этом языке будет представлено вариантным типом `expr`, где для каждого элемента выражения определяется свой тег:

OCaml utop

<https://github.com/realworldocaml/examples/blob/v1/code/variants/blang.topscript>

```
# type 'a expr =
  | Base of 'a
  | Const of bool
  | And of 'a expr list
  | Or of 'a expr list
  | Not of 'a expr
;;

type 'a expr =
  Base of 'a
  | Const of bool
  | And of 'a expr list
  | Or of 'a expr list
  | Not of 'a expr
```

Отметьте, что определение типа `expr` является рекурсивным, в том смысле что значение типа `expr` может содержать другие значения типа `expr`. Кроме того, тип `expr` параметризован полиморфным типом `'a`, используемым для определения типа значения, соответствующего тегу `Base`.

Назначение каждого тега достаточно очевидно и без лишних слов. Теги `And`, `Or` и `Not` — это операторы логических выражений, а тег `Const` позволяет вводить в выражения константы `true` и `false`.

Тег `Base` — это связующее звено между `expr` и вашим приложением, дающее возможность указывать элементы некоторого базового типа предиката, истинность или ложность которого определяются приложением. Для языка фильтров, используемого в приложении обработки электронной почты, например, базовые предикаты могли бы определять проверки электронных писем, как показано ниже:

OCaml utop (part 1)

<https://github.com/realworldocaml/examples/blob/v1/code/variants/blang.topscript>

```
# type mail_field = To | From | CC | Date | Subject
  type mail_predicate = { field: mail_field;
                        contains: string }
;;

type mail_field = To | From | CC | Date | Subject
type mail_predicate = { field : mail_field; contains : string; }
```

На основе определений выше можно сконструировать простое выражение с `mail_predicate` в качестве базового предиката:

OCaml utop (part 2)

<https://github.com/realworldocaml/examples/blob/v1/code/variants/blang.topscript>

```
# let test field contains = Base { field; contains };;
val test : mail_field -> string -> mail_predicate expr = <fun>
# And [ Or [ test To "doligez"; test CC "doligez" ];
        test Subject "runtime";
      ]
;;
- : mail_predicate expr =
```

```
And
[Or
  [Base {field = To; contains = "doligez"};
   Base {field = CC; contains = "doligez"}];
Base {field = Subject; contains = "runtime"}]
```

Однако одной возможности конструировать такие выражения недостаточно; необходима еще возможность вычислять их. Этим как раз и занимается следующая функция:

OCaml utop (part 3)

<https://github.com/realworldocaml/examples/blob/v1/code/variants/blang.topscript>

```
# let rec eval expr base_eval =
  (* поддерживается сокращенный синтаксис, благодаря которому не требуется
    повторно и явно передавать [base_eval] в [eval] *)
  let eval' expr = eval expr base_eval in
  match expr with
  | Base base   -> base_eval base
  | Const bool  -> bool
  | And exprs   -> List.for_all exprs ~f:eval'
  | Or exprs    -> List.exists exprs ~f:eval'
  | Not expr    -> not (eval' expr)
;;
val eval : 'a expr -> ('a -> bool) -> bool = <fun>
```

Код имеет простую и понятную структуру — мы просто сопоставляем структуру данных с образцом, выполняя соответствующие вычисления, опираясь на имена тегов. Чтобы задействовать полученные функции в конкретном приложении, достаточно просто написать функцию `base_eval`, реализующую базовый предикат.

Другой интересной операцией с выражениями является упрощение. Ниже приводится множество функций, упрощающих конструкцию выражений и отражающих поддерживаемые теги в типе `expr`:

OCaml utop (part 4)

<https://github.com/realworldocaml/examples/blob/v1/code/variants/blang.topscript>

```
# let and_1 =
  if List.mem 1 (Const false) then Const false
  else
    match List.filter 1 ~f:((<>) (Const true)) with
    | [] -> Const true
    | [ x ] -> x
    | 1 -> And 1

let or_1 =
  if List.mem 1 (Const true) then Const true
  else
    match List.filter 1 ~f:((<>) (Const false)) with
    | [] -> Const false
    | [ x ] -> x
    | 1 -> Or 1

let not_ = function
```

```

    | Const b -> Const (not b)
    | e -> Not e
;;
val and_ : 'a expr list -> 'a expr = <fun>
val or_ : 'a expr list -> 'a expr = <fun>
val not_ : 'a expr -> 'a expr = <fun>

```

Теперь можно написать процедуру упрощения, опирающуюся на предыдущие функции.

OCaml utop (part 5)

<https://github.com/realworldocaml/examples/blob/v1/code/variants/blang.topscript>

```

# let rec simplify = function
  | Base _ | Const _ as x -> x
  | And l -> and_ (List.map ~f:simplify l)
  | Or l -> or_ (List.map ~f:simplify l)
  | Not e -> not_ (simplify e)
;;
val simplify : 'a expr -> 'a expr = <fun>

```

Попробуем применить ее к логическому выражению и посмотрим, насколько хорошо она справится со своей задачей упрощения:

OCaml utop (part 6)

<https://github.com/realworldocaml/examples/blob/v1/code/variants/blang.topscript>

```

# simplify (Not (And [ Or [Base "it's snowing"; Const true];
                      Base "it's raining"]));;
- : string expr = Not (Base "it's raining")

```

Она корректно преобразовала ветку `Or` в `Const true` и полностью устранила операцию `And`, поскольку имеет лишь один нетривиальный операнд.

Однако не все упрощения подвластны этой функции. В частности, посмотрите, что случится, если добавить двойное отрицание:

OCaml utop (part 7)

<https://github.com/realworldocaml/examples/blob/v1/code/variants/blang.topscript>

```

# simplify (Not (And [ Or [Base "it's snowing"; Const true];
                      Not (Not (Base "it's raining"))]));;
- : string expr = Not (Not (Not (Base "it's raining")))

```

Она не смогла устранить двойное отрицание, и несложно догадаться – почему. Функция `not_` включает универсальный образец, поэтому она игнорирует все, кроме единственного случая, описанного явно, – отрицания константы. Применение универсальных образцов не всегда является наилучшим выходом, и если сделать код более явным, тогда причины пропуска двойного отрицания станут более очевидными:

OCaml utop (part 8)

<https://github.com/realworldocaml/examples/blob/v1/code/variants/blang.topscript>

```

# let not_ = function
  | Const b -> Const (not b)
  | (Base _ | And _ | Or _ | Not _) as e -> Not e

```

```
;;
val not_ : 'a expr -> 'a expr = <fun>
```

Конечно, эту проблему легко можно исправить, добавив явный случай двойного отрицания:

OCaml utop (part 9)

<https://github.com/realworldocaml/examples/blob/v1/code/variants/blang.topscript>

```
# let not_ = function
  | Const b -> Const (not b)
  | Not e -> e
  | (Base _ | And _ | Or _ ) as e -> Not e
;;
val not_ : 'a expr -> 'a expr = <fun>
```

Пример реализации логических выражений – это не бесполезная игрушка. В библиотеке Core существует действующий модуль, написанный в том же духе, который называется Blang (сокращенно от «Boolean language» – язык логики), нашедший широкое практическое применение в самых разных приложениях. Алгоритм упрощения может пригодиться, например, когда вам потребуется организовать вычисление выражений, где результаты вычисления некоторых базовых предикатов уже известны.

Вообще говоря, применение вариантов для создания рекурсивных структур данных является распространенной практикой, и примеры такого их использования можно встретить где угодно, от реализаций простых языков до конструирования сложных структур данных.

Полиморфные варианты

Помимо простых вариантов, которые мы видели до сих пор, OCaml поддерживает также так называемые *полиморфные варианты* (polymorphic variants). Как будет показано далее, полиморфные варианты являются более гибкими и более легковесными, чем обычные варианты, но дополнительные возможности имеют свою цену.

Синтаксически полиморфные варианты отличаются от обычных начальным обратным штрихом. И, в отличие от обычных вариантов, полиморфные варианты могут использоваться без явного объявления типа:

OCaml utop (part 6)

<https://github.com/realworldocaml/examples/blob/v1/code/variants/main.topscript>

```
# let three = `Int 3;;
val three : [> `Int of int ] = `Int 3
# let four = `Float 4.;;
val four : [> `Float of float ] = `Float 4.
# let nan = `Not_a_number;;
val nan : [> `Not_a_number ] = `Not_a_number
# [three; four; nan];;
- : [> `Float of float | `Int of int | `Not_a_number ] list =
[ `Int 3; `Float 4.; `Not_a_number ]
```

Как видите, типы полиморфных вариантов выводятся автоматически, и когда мы объединяем варианты с разными тегами, компилятор выводит новый тип, которому известны все эти теги. Обратите внимание, что в этом примере имена тегов (например, ``Int`) совпадают с именами типов (`int`). Это – общепринятое соглашение в OCaml.

Система типов будет сообщать об ошибке, встретив противоречивое применение одного и того же тега:

OCaml utop (part 7)

<https://github.com/realworldocaml/examples/blob/v1/code/variants/main.topscript>

```
# let five = `Int "five";;
val five : [> `Int of string ] = `Int "five"
# [three; four; five];;
Characters 14-18:
Error: This expression has type [> `Int of string ]
      but an expression was expected of type
      [> `Float of float | `Int of int ]
Types for tag `Int are incompatible
(Ошибка: Это выражение имеет тип [> `Int of string ],
  тогда как ожидалось выражение типа
  [> `Float of float | `Int of int ]
  Типы тегов `Int несовместимы)
```

Символ `>` в начале вариантных типов выше играет важную роль, потому что он отмечает типы как открытые комбинации вариантных типов. Конструкция `[> `Int of string | `Float of float]` читается как «описание варианта, в число тегов которого входят ``Int of string` и ``Float of float`, но в него могут также включаться другие теги». Иными словами, символ `>` можно интерпретировать как «эти и, возможно, другие теги».

В некоторых ситуациях компилятор OCaml может определять вариантные типы, начинающиеся с символа `<`, сообщающего: «эти теги, и не более того», – как показано в следующем примере:

OCaml utop (part 8)

<https://github.com/realworldocaml/examples/blob/v1/code/variants/main.topscript>

```
# let is_positive = function
| `Int x -> x > 0
| `Float x -> x > 0.
;;
val is_positive : [< `Float of float | `Int of int ] -> bool = <fun>
```

Здесь присутствие символа `<` объясняется тем, что `is_positive` не предусматривает возможности обработки значений с другими тегами, отличными от ``Float of float` или ``Int of int`.

Метки `<` и `>` можно рассматривать как индикаторы верхней и нижней границ множества тегов. Если один и тот же набор тегов имеет и нижнюю, и верхнюю границы, в результате получается *точный* полиморфный вариантный тип, не имеющий ни той, ни другой метки. Например:

OCaml utop (part 9)

<https://github.com/realworldocaml/examples/blob/v1/code/variants/main.topscript>

```
# let exact = List.filter ~f:is_positive [three;four];;
val exact : [ `Float of float | `Int of int ] list = [ `Int 3; `Float 4.]
```

Как ни удивительно, но имеется также возможность создавать полиморфные вариантные типы, имеющие разные верхнюю и нижнюю границы. Отметим, что `Ok` и `Error` в следующем примере являются значениями типа `Result.t` из библиотеки `Core`:

OCaml utop (part 10)

<https://github.com/realworldocaml/examples/blob/v1/code/variants/main.topscript>

```
# let is_positive = function
  | `Int x -> Ok (x > 0)
  | `Float x -> Ok (x > 0.)
  | `Not_a_number -> Error "not a number";;
val is_positive :
  [< `Float of float | `Int of int | `Not_a_number ] ->
  (bool, string) Result.t = <fun>
# List.filter [three; four] ~f:(fun x ->
  match is_positive x with Error _ -> false | Ok b -> b);;
- : [< `Float of float | `Int of int | `Not_a_number > `Float `Int ] list =
[ `Int 3; `Float 4.]
```

Здесь механизм вывода типов сообщает, что в число тегов может входить не более трех тегов — ``Float`, ``Int` и ``Not_a_number` — и не менее двух — ``Float` и ``Int`. Как вы уже наверняка поняли, полиморфные варианты могут приводить к выводу весьма сложных типов.

Пример: и снова о цветных терминалах

Чтобы увидеть особенности применения полиморфных вариантов на практике, вернемся к цветным терминалам. Представьте, что у нас появился терминал нового типа, поддерживающий еще большее число цветов, например за счет добавления поддержки альфа-канала, благодаря чему появилась возможность определять полупрозрачные цвета. Мы могли бы смоделировать расширенный набор цветов с помощью обычного варианта, как показано ниже:

OCaml utop (part 11)

<https://github.com/realworldocaml/examples/blob/v1/code/variants/main.topscript>

```
# type extended_color =
  | Basic of basic_color * weight (* базовые цвета, обычный и жирный шрифты *)
  | RGB of int * int * int (* пространство цветов 6x6x6 *)
  | Gray of int (* 24-уровневая палитра оттенков серого *)
  | RGBA of int * int * int * int (* пространство цветов 6x6x6x6 *)
;;
type extended_color =
  Basic of basic_color * weight
  | RGB of int * int * int
  | Gray of int
  | RGBA of int * int * int * int
```


Нам требуется написать функцию `extended_color_to_int`, действующую подобно `color_to_int` для всех старых цветов и добавляющую новую логику для обработки цветов, включающих значение альфа-канала. Можно было бы попытаться записать эту функцию так:

OCaml utop (part 12)

<https://github.com/realworldocaml/examples/blob/v1/code/variants/main.topscript>

```
# let extended_color_to_int = function
  | RGBA (r,g,b,a) -> 256 + a * 6 + g * 36 + r * 216
  | (Basic _ | RGB _ | Gray _) as color -> color_to_int color
;;
```

Characters 154-159:

```
Error: This expression has type extended_color
      but an expression was expected of type color
```

*(Ошибка: Это выражение имеет тип extended_color,
тогда как ожидалось выражение типа color)*

Реализация выглядит вполне адекватно, но компилятор сообщает об ошибке, потому что функции `extended_color` и `color` с его (компилятора) точки зрения имеют разные и несовместимые типы. Компилятор, к примеру, не распознал сходства между тегами `Basic` в двух типах.

Нам нужно объединить теги двух разных вариантных типов, и полиморфные варранты допускают такую возможность. Для начала перепишем `basic_color_to_int` и `color_to_int` с применением полиморфных вариантов. Это совсем несложно:

OCaml utop (part 13)

<https://github.com/realworldocaml/examples/blob/v1/code/variants/main.topscript>

```
# let basic_color_to_int = function
  | `Black -> 0 | `Red -> 1 | `Green -> 2 | `Yellow -> 3
  | `Blue -> 4 | `Magenta -> 5 | `Cyan -> 6 | `White -> 7

let color_to_int = function
  | `Basic (basic_color,weight) ->
    let base = match weight with `Bold -> 8 | `Regular -> 0 in
    base + basic_color_to_int basic_color
  | `RGB (r,g,b) -> 16 + b + g * 6 + r * 36
  | `Gray i -> 232 + i
;;
```

```
val basic_color_to_int :
```

```
[< `Black | `Blue | `Cyan | `Green | `Magenta | `Red | `White | `Yellow ] ->
int = <fun>
```

```
val color_to_int :
```

```
[< `Basic of
  [< `Black
  | `Blue
  | `Cyan
  | `Green
  | `Magenta
  | `Red
  | `White
```

```

        | `Yellow ] *
[< `Bold | `Regular ]
    | `Gray of int
    | `RGB of int * int * int ] ->
int = <fun>

```

Теперь можно попробовать написать `extended_color_to_int`. Основная проблема заключается в том, что `extended_color_to_int` должна вызывать `color_to_int` с более узким типом, то есть включающим меньшее число тегов. Такое сужение может быть достигнуто с помощью сопоставления с образцом. В частности, в следующем фрагменте тип переменной `color` включает только теги ``Basic`, ``RGB` и ``Gray` и не включает ``RGBA`:

OCaml utop (part 14)

<https://github.com/realworldocaml/examples/blob/v1/code/variants/main.topscript>

```

# let extended_color_to_int = function
  | `RGBA (x,g,b,a) -> 256 + a + b * 6 + g * 36 + r * 216
  | (`Basic _ | `RGB _ | `Gray _) as color -> color_to_int color
;;
val extended_color_to_int :
  [< `Basic of
    [< `Black
      | `Blue
      | `Cyan
      | `Green
      | `Magenta
      | `Red
      | `White
      | `Yellow ] *
    [< `Bold | `Regular ]
  | `Gray of int
  | `RGB of int * int * int
  | `RGBA of int * int * int * int ] ->
int = <fun>

```

Этот пример сбалансирован более изящно, чем можно было бы представить. В частности, если мы используем универсальный образец вместо явного перечисления, сужение типа не произойдет, и компилятор сообщит об ошибке:

OCaml utop (part 15)

<https://github.com/realworldocaml/examples/blob/v1/code/variants/main.topscript>

```

# let extended_color_to_int = function
  | `RGBA (x,g,b,a) -> 256 + a + b * 6 + g * 36 + r * 216
  | color -> color_to_int color
;;
Characters 125-130:
Error: This expression has type [> `RGBA of int * int * int * int ]
      but an expression was expected of type
      [< `Basic of
        [< `Black
          | `Blue
          | `Cyan
          | `Green

```

```

    | `Magenta
    | `Red
    | `White
    | `Yellow ] *
  [< `Bold | `Regular ]
  | `Gray of int
  | `RGB of int * int * int ]

```

The second variant type does not allow tag(s) `RGBA

(Ошибка: Это выражение имеет тип [> `RGBA of int * int * int * int],

тогда как ожидается выражение типа

```

  [< `Basic of
    [< `Black
      | `Blue
      | `Cyan
      | `Green
      | `Magenta
      | `Red
      | `White
      | `Yellow ] *
    [< `Bold | `Regular ]
  | `Gray of int
  | `RGB of int * int * int ]

```

Второй вариантный тип не поддерживает тег(и) `RGBA)

Полиморфные варианты и универсальные образцы

Как мы видели на примере определения функции `is_positive`, инструкция `match` может вызвать вывод типа верхней границы варианта, ограничивающий множество допустимых тегов только теми, что могут быть ею обработаны. Если в инструкцию `match` добавить универсальный образец, мы получим тип с нижней границей:

OCaml utop (part 16)

<https://github.com/realworldocaml/examples/blob/v1/code/variants/main.topscript>

```

# let is_positive_permissive = function
  | `Int x -> Ok (x > 0)
  | `Float x -> Ok (x > 0.)
  | _ -> Error "Unknown number type"
;;

val is_positive_permissive :
  [> `Float of float | `Int of int ] -> (bool, string) Result.t = <fun>
# is_positive_permissive (`Int 0);;
- : (bool, string) Result.t = Ok false
# is_positive_permissive (`Ratio (3,4));;
- : (bool, string) Result.t = Error "Unknown number type"

```

Универсальные образцы часто являются источником ошибок, даже при использовании обычных вариантов, а для полиморфных вариантов это особенно характерно. Это объясняется отсутствием возможности ограничить множество тегов, которые может обрабатывать функция. Такой код особенно чувствителен к опечаткам. Например, если в коде, вызывающем `is_positive_permissive`, допустить опечатку и вместо `Float` записать `Floot`, ошибочный код будет благополучно скомпилирован:

OCaml utop (part 17)

<https://github.com/realworldocaml/examples/blob/v1/code/variants/main.topscript>

```

# is_positive_permissive (`Floot 3.5);;
- : (bool, string) Result.t = Error "Unknown number type"

```

При использовании обычных вариантов такие опечатки будут опознаны компилятором как попытка передать неизвестный тег. Как правило, следует проявлять особую осторожность при смешивании универсальных образцов с полиморфными вариантами.

Давайте посмотрим, как можно превратить наш код в полноценную библиотеку с реализацией в файле *.ml* и интерфейсом в отдельном файле *.mli*, о которых рассказывалось в главе 4. Начнем с определения интерфейса *.mli*:

OCaml: OCaml

https://github.com/realworldocaml/examples/blob/v1/code/variants-termcol/terminal_color.mli

```
open Core.Std

type basic_color =
  [ `Black   | `Blue | `Cyan | `Green
    | `Magenta | `Red  | `White | `Yellow ]

type color =
  [ `Basic of basic_color * [ `Bold | `Regular ]
    | `Gray of int
    | `RGB of int * int * int ]

type extended_color =
  [ color
    | `RGBA of int * int * int * int ]

val color_to_int          : color -> int
val extended_color_to_int : extended_color -> int
```

Здесь тип `extended_color` явно определен как расширение типа `color`. Кроме того, обратите внимание, что все эти типы определены как точные варианты. Ниже приводится реализация данной библиотеки:

OCaml: OCaml

https://github.com/realworldocaml/examples/blob/v1/code/variants-termcol/terminal_color.ml

```
open Core.Std

type basic_color =
  [ `Black | `Blue | `Cyan | `Green
    | `Magenta | `Red | `White | `Yellow ]

type color =
  [ `Basic of basic_color * [ `Bold | `Regular ]
    | `Gray of int
    | `RGB of int * int * int ]

type extended_color =
  [ color
    | `RGBA of int * int * int * int ]

let basic_color_to_int = function
```

```

| `Black -> 0 | `Red      -> 1 | `Green -> 2 | `Yellow -> 3
| `Blue  -> 4 | `Magenta -> 5 | `Cyan  -> 6 | `White  -> 7

let color_to_int = function
  | `Basic (basic_color, weight) ->
    let base = match weight with `Bold -> 8 | `Regular -> 0 in
    base + basic_color_to_int basic_color
  | `RGB (r,g,b) -> 16 + b + g * 6 + r * 36
  | `Gray i -> 232 + i

let extended_color_to_int = function
  | `RGBA (r,g,b,a) -> 256 + a + b * 6 + g * 36 + r * 216
  | `Grey x -> 2000 + x
  | (`Basic _ | `RGB _ | `Gray _) as color -> color_to_int color

```

Обратите внимание на определение функции `extended_color_to_int`, здесь мы особо подчеркнули некоторые недостатки полиморфных вариантов. В частности, мы добавили собственную обработку серого цвета, вместо того чтобы переложить эту работу на `color_to_int`. Но при этом допустили опечатку, записав `Gray` как `Grey`. Эту ошибку компилятор точно должен обнаружить при использовании обычных вариантов, но когда применяются полиморфные варианты, компилятор компилирует код без вывода сообщений об ошибках. Как результат компилятор выведет более широкий тип для `extended_color_to_int`, который по стечению обстоятельств совместим с более узким типом, объявленным в файле *.mli*.

Если добавить в код явную аннотацию типа (не полагаясь на определение в *.mli*), компилятор получит достаточный объем информации, чтобы вывести предупреждение:

OCaml: OCaml (part 1)

https://github.com/realworldocaml/examples/blob/v1/code/variants-termcol-annotated/terminal_color.ml

```

let extended_color_to_int : extended_color -> int = function
  | `RGBA (r,g,b,a) -> 256 + a + b * 6 + g * 36 + r * 216
  | `Grey x -> 2000 + x
  | (`Basic _ | `RGB _ | `Gray _) as color -> color_to_int color

```

В частности, он сообщит о неиспользуемом образце ``Grey`:

Terminal

<https://github.com/realworldocaml/examples/tree/v1/code/variants-termcol-annotated/build.out>

```
$ corebuild terminal_color.native
```

```
File "terminal_color.ml", line 30, characters 4-11:
```

```
Error: This pattern matches values of type [? `Grey of 'a ]
      but a pattern was expected which matches values of type extended_color
      The second variant type does not allow tag(s) `Grey
Command exited with code 2.
```

(Ошибка: Этот образец соответствует значению типа [? `Grey of 'a ], тогда как ожидается, что он должен соответствовать значениям типа `extended_color`. Второй вариантный тип не допускает тег(и) ``Grey`
Команда завершилась с кодом 2.)

Как только в нашем распоряжении оказываются определения типов, мы можем пересмотреть реализацию сопоставления с образцом, чтобы сузить тип. В частности, можно явно указать имя типа в образце, снабдив его префиксом #:

OCaml: OCaml (part 1)

https://github.com/realworldocaml/examples/blob/v1/code/variants-termcol-fixed/terminal_color.ml

```
let extended_color_to_int : extended_color -> int = function
  | `RGBA (r,g,b,a) -> 256 + a + b * 6 + g * 36 + r * 216
  | #color as color -> color_to_int color
```

Этот прием может пригодиться, когда необходимо сузить до типа с более длинным определением, но нет желания явно указывать все теги в сопоставлении.

Когда следует использовать полиморфные варианты

На первый взгляд, полиморфные варианты выглядят как улучшенная версия обычных вариантов. С их использованием можно делать все то же самое, что и с применением обычных вариантов, плюс ко всему этому вы получаете дополнительную гибкость и более краткий синтаксис. Так что же не так?

В действительности обычные варианты в большинстве ситуаций являют собой более прагматичный выбор, потому что гибкость полиморфных вариантов заставляет платить слишком высокую цену. Ниже перечислены некоторые недостатки:

- **Сложность.** Как мы имели возможность убедиться, правила типизации для полиморфных вариантов намного сложнее тех же правил для обычных вариантов. Это означает, что широкое использование полиморфных вариантов может заставить вас крепко чесать затылок, когда вы будете пытаться понять, почему данный фрагмент кода компилируется или, наоборот, не компилируется. Это может также вынудить вас проводить долгие часы в расшифровывании странных сообщений об ошибках. Фактически краткость на уровне программного кода часто покупается ценой детального описания типов.
- **Поиск ошибок.** Полиморфные варианты также подвергаются строгому контролю типов, но терпимость к опечаткам из-за высокой гибкости снижает шансы компилятора найти ошибки в программе.
- **Эффективность.** Хотя это и не самый важный фактор, но полиморфные варианты требуют больше вычислительных ресурсов, чем обычные варианты, и компилятор OCaml не способен генерировать код сопоставления для полиморфных вариантов, такой же эффективный, как для обычных вариантов.

И тем не менее, несмотря на все вышесказанное, полиморфные варианты по-прежнему остаются полезной и мощной особенностью языка, просто надо знать и понимать их ограничения и особенности эффективного их использования.

Вероятно, лучше всего полиморфные варианты использовать там, где достаточно было бы обычных вариантов, но они оказываются слишком тяжеловесными синтаксически. Например, часто бывает желательно определить вариантный тип для кодирования ввода или вывода функции, и нет никакого желания объявлять

отдельный тип. Полиморфные варианты в этом случае очень пригодятся вам. И пока вы будете использовать аннотации типов, чтобы ограничить их явно, все будет работать прекрасно.

Наибольшие проблемы варианты доставляют именно там, где нужна вся их мощь; в частности, когда вы хотите воспользоваться способностью полиморфных вариантов иметь перекрывающиеся множества поддерживаемых тегов. Это затрагивает поддержку подтипов в OCaml. Как мы обсудим далее, когда будем рассматривать объекты в главе 11, подтипы влекут за собой массу сложностей, и практически всегда эти сложности являются весьма нежелательными.

Глава 7

Обработка ошибок

Никому не нравится заниматься обработкой ошибок. Это утомительно, легко сделать что-то не так, и обычно не доставляет такого удовольствия, как планирование будущих успехов программы. Но обработка ошибок играет важную роль, и хотя вам не нравится думать об этом аспекте, сбои в работе программ из-за ненадлежащей обработки ошибок обычно являют собой гораздо большую неприятность.

К счастью, в арсенале OCaml имеются мощные инструменты, позволяющие организовать надежный заслон перед ошибками с минимумом усилий. В этой главе мы обсудим некоторые из подходов к обработке ошибок в программах на языке OCaml и дадим ряд советов, касающихся проектирования интерфейсов, упрощающих обработку ошибок.

Для начала рассмотрим два простых подхода к реализации индикации ошибок: типы возвращаемых значений с признаком ошибки и исключения.

Типы возвращаемых значений с признаком ошибки

Лучший способ в OCaml сообщить об ошибке – включить признак ошибки в возвращаемое значение. Взгляните на тип функции `find` из модуля `List`:

OCaml utop

<https://github.com/realworldocaml/examples/blob/v1/code/error-handling/main.topscript>

```
# List.find;;  
- : 'a list -> f:('a -> bool) -> 'a option = <fun>
```

Необязательное значение `option` в возвращаемом типе указывает, что функция может потерпеть неудачу при поиске указанного элемента:

OCaml utop (part 1)

<https://github.com/realworldocaml/examples/blob/v1/code/error-handling/main.topscript>

```
# List.find [1;2;3] ~f:(fun x -> x >= 2) ;;  
- : int option = Some 2  
# List.find [1;2;3] ~f:(fun x -> x >= 10) ;;  
- : int option = None
```

Включение признака ошибки в возвращаемое значение требует, чтобы вызывающий код явно обрабатывал его, что, в свою очередь, дает вызывающему коду возможность выбирать между обработкой ошибки своими силами или передачей ее вверх по стеку вызовов.

Взгляните на следующую функцию `compute_bounds`. Она принимает список и функцию сравнения и возвращает верхнюю и нижнюю границы значений элементов в списке путем определения наибольшего и наименьшего элементов списка. Функции `List.hd` и `List.last`, возвращающие `None` для пустого списка, используются для извлечения наибольшего и наименьшего элементов:

OCaml utop (part 2)

<https://github.com/realworldocaml/examples/blob/v1/code/error-handling/main.topscript>

```
# let compute_bounds ~cmp list =
  let sorted = List.sort ~cmp list in
  match List.hd sorted, List.last sorted with
  | None, _ | _, None -> None
  | Some x, Some y -> Some (x,y)
;;

val compute_bounds : cmp:(('a -> 'a -> int) -> 'a list -> ('a * 'a) option) =
  <fun>
```

Для обработки ошибочных ситуаций используется инструкция `match`. В данном случае она передает значение `None`, полученное от `hd` или `last`, в возвращаемое значение `compute_bounds`.

С другой стороны, в функции `find_mismatches`, следующей ниже, ошибки, встреченные в ходе вычислений, не передаются дальше в возвращаемом значении. `find_mismatches` принимает две хэш-таблицы в аргументах и ищет одинаковые ключи, имеющие разные значения в этих двух таблицах. При этом присутствие ключа в одной таблице и отсутствие в другой вообще не считается ошибкой:

OCaml utop (part 3)

<https://github.com/realworldocaml/examples/blob/v1/code/error-handling/main.topscript>

```
# let find_mismatches table1 table2 =
  Hashtbl.fold table1 ~init:[] ~f:(fun ~key ~data mismatches ->
    match Hashtbl.find table2 key with
    | Some data' when data' <> data -> key :: mismatches
    | _ -> mismatches
  )
;;

val find_mismatches : ('a, 'b) Hashtbl.t -> ('a, 'b) Hashtbl.t -> 'a list =
  <fun>
```

Применение необязательных значений (типа `option`) для возврата признака ошибки подчеркивает тот факт, что порой сложно с первого взгляда сказать, является ли конкретный результат правильным результатом или, например, признаком отсутствия искомого элемента в списке. Это зависит от общего контекста программы и потому не является чем-то, о чем может догадываться универсальная библиотека. Одно из преимуществ типов возвращаемых значений с признаком ошибки состоит в том, что они отлично работают в обеих ситуациях.

Кодирование ошибок в результате

Тип `option` не всегда является достаточно выразительным для индикации ошибок. В частности, когда ошибка представлена единственным возможным значением `None`, никто не сможет с полной уверенностью сказать что-либо о ее природе.

Тип `Result.t` специально проектировался, чтобы восполнить этот недостаток. Он имеет следующее определение:

OCaml

<https://github.com/realworldocaml/examples/tree/v1/code/error-handling/result.mli>

```
module Result : sig
  type ('a,'b) t = | Ok of 'a
                  | Error of 'b
end
```

По сути, `Result.t` аналогичен типу `option`, с той лишь разницей, что может нести информацию о природе ошибки. Конструкторы `Ok` и `Error`, которые можно считать аналогами `Some` и `None`, определены на верхнем уровне `Core.Std`. Благодаря этому их можно использовать:

OCaml utop (part 4)

<https://github.com/realworldocaml/examples/tree/v1/code/error-handling/main.topscrip>

```
# [ Ok 3; Error "abject failure"; Ok 4 ];;
- : (int, string) Result.t list = [Ok 3; Error "abject failure"; Ok 4]
```

без предварительного открытия модуля `Result`.

Error и Or_error

Тип `Result.t` дает полную свободу в выборе типов значений, представляющих ошибки, но часто полезно прийти к какому-то одному стандарту. Такой подход, кроме всего прочего, упрощает создание вспомогательных функций для автоматизации обработки наиболее распространенных ошибок.

Но какой тип выбрать? Не лучше ли будет представлять ошибки строками с их описанием? Или лучше использовать более структурированное представление, например, в формате XML? Или использовать какое-то иное представление?

Библиотека `Core` отвечает на этот вопрос типом `Error.t`, в котором предпринята попытка соблюсти баланс между эффективностью, удобством и полнотой контроля над представлением ошибок.

Для тех, кому непонятно, почему в этом списке присутствует упоминание эффективности, заметим, что генерирование сообщений об ошибках является довольно дорогостоящей операцией. Создание строкового представления ошибки может отнимать много времени, особенно если при этом требуется выполнять преобразование числовых данных.

Тип `Error.t` решает эту проблему за счет применения отложенных вычислений. В частности, он позволяет отложить генерацию строки с текстом сообщения до момента, когда она действительно потребуется, а это означает, что во многих ситуациях эти строки вообще никогда не будут создаваться. Конечно, вы можете создавать сообщения из уже готовых строк:

OCaml utop (part 5)

<https://github.com/realworldocaml/examples/tree/v1/code/error-handling/main.topscrip>

```
# Error.of_string "something went wrong";;
- : Error.t = something went wrong
```

Но можно также конструировать значения типа `Error.t` с помощью *переходника* (`thunk`), то есть функции, принимающей единственный аргумент типа `unit`:

OCaml utop (part 6)

<https://github.com/realworldocaml/examples/tree/v1/code/error-handling/main.topscrip>

```
# Error.of_thunk (fun () ->
  sprintf "something went wrong: %f" 32.3343) ;;
- : Error.t = something went wrong: 32.334300
```

В данном случае мы можем пользоваться всеми выгодами отложенных вычислений, предоставляемыми модулем `Error`, поскольку переходник не будет вызван до самого момента, когда потребуется преобразовать значение `Error.t` в строку.

Чаше всего значения типа `Error.t` создаются с помощью *s-выражений*. S-выражение – это сбалансированное по круглым скобкам выражение, где роль листьев играют строки¹. Ниже приводится простой пример:

Scheme

<https://github.com/realworldocaml/examples/tree/v1/code/error-handling/sexpr.scm>

(Это (самое настоящее) (s-выражение))

S-выражения поддерживаются посредством пакета `Sexplib`, распространяемого вместе с библиотекой `Core`, и представляют наиболее используемый формат сериализации из поддерживаемых библиотекой `Core`. Действительно, большинство типов в `Core` включают встроенные средства преобразования s-выражений. Ниже приводится пример создания объекта ошибки с применением `sexpr`-преобразователя для значений времени `Time.sexp_of_t`:

OCaml utop (part 7)

<https://github.com/realworldocaml/examples/tree/v1/code/error-handling/main.topscrip>

```
# Error.create "Something failed a long time ago" Time.epoch Time.sexp_of_t ;;
- : Error.t =
Something failed a long time ago: (1970-01-01 01:00:00.000000+01:00)
```

Обратите внимание, что значение времени в действительности не сериализуется в s-выражение до момента вывода ошибки.

Подобный способ передачи информации об ошибке не ограничивается только встроенными типами. Мы еще будем подробно обсуждать эту тему в главе 17, тем не менее замечу, что пакет `Sexplib` включает расширение языка, способное автоматически генерировать `sexpr`-преобразователи для новых типов:

OCaml utop (part 8)

<https://github.com/realworldocaml/examples/blob/v1/code/error-handling/main.topscrip>

```
# let custom_to_sexp = <:sexp_of<float * string list * int>>;
val custom_to_sexp : float * string list * int -> Sexp.t = <fun>
# custom_to_sexp (3.5, ["a";"b";"c"], 6034) ;;
- : Sexp.t = (3.5 (a b c) 6034)
```

¹ <http://ru.wikipedia.org/wiki/S-выражение>. – Прим. перев.

Ту же идиому можно использовать для определения своих ошибок:

OCaml utop (part 9)

<https://github.com/realworldocaml/examples/blob/v1/code/error-handling/main.topscript>

```
# Error.create "Something went terribly wrong"
  (3.5, ["a";"b";"c"], 6034)
  <:sexp_of<float * string list * int>> ;;
- : Error.t = Something went terribly wrong: (3.5 (a b c) 6034)
```

Модуль `Error` также поддерживает операции преобразования ошибок. Например, часто бывает удобно добавить в значение ошибки информацию о контексте, в котором она возникла, или объединить несколько ошибок вместе. Этой цели служат `Error.tag` и `Error.of_list`:

OCaml utop (part 10)

<https://github.com/realworldocaml/examples/blob/v1/code/error-handling/main.topscript>

```
# Error.tag
  (Error.of_list [ Error.of_string "Your tires were slashed";
                  Error.of_string "Your windshield was smashed" ])
  "over the weekend"
;;
- : Error.t =
over the weekend: Your tires were slashed; Your windshield was smashed
```

Тип `'a Or_error.t` — это просто более краткая форма для `('a, Error.t) Result.t`, и после типа `option` это самый распространенный способ возврата информации об ошибках в библиотеке `Core`.

Функция `bind` и другие идиомы обработки ошибок

По мере накопления опыта обработки ошибок в программах на языке OCaml вы обнаружите, что снова и снова пишете один и тот же шаблонный код. Многие из этих шаблонов уже реализованы в виде функций в таких модулях, как `Option` и `Result`. Один из особенно примечательных шаблонов построен на основе функции `bind`, которая одновременно является и обычной функцией, и инфиксным оператором `>>=`. Ниже показано определение функции `bind` для типа `option`:

OCaml utop (part 11)

<https://github.com/realworldocaml/examples/blob/v1/code/error-handling/main.topscript>

```
# let bind option f =
  match option with
  | None -> None
  | Some x -> f x
;;
val bind : 'a option -> ('a -> 'b option) -> 'b option = <fun>
```

Как видите, вызов `bind None f` вернет `None`, не вызывая `f`, а вызов `bind (Some x) f` вернет `f x`. Функция `bind` может использоваться как инструмент для вызова последовательности функций, способных возвращать ошибки, причем первая же ошибка прервет выполнение всей последовательности. Ниже приводится измененная версия `compute_bounds`, использующая последовательность вложенных вызовов `bind`:

OCaml utop (part 12)

<https://github.com/realworldocaml/examples/blob/v1/code/error-handling/main.topscript>

```
# let compute_bounds ~cmp list =
  let sorted = List.sort ~cmp list in
  Option.bind (List.hd sorted) (fun first ->
    Option.bind (List.last sorted) (fun last ->
      Some (first,last)))
;;
val compute_bounds : cmp:('a -> 'a -> int) -> 'a list -> ('a * 'a) option =
  <fun>
```

Однако такой код сложно воспринимается на глаз. Его можно сделать более простым для чтения и выбросить из него некоторые круглые скобки, задействовав форму инфиксного оператора `bind`, к которому можно получить доступ, открыв модуль `Option.Monad_infix` локально. Модуль называется `Monad_infix` потому, что оператор `bind` является частью вложенного интерфейса с именем `Monad`, который подробнее будет рассматриваться в главе 18:

OCaml utop (part 13)

<https://github.com/realworldocaml/examples/blob/v1/code/error-handling/main.topscript>

```
# let compute_bounds ~cmp list =
  let open Option.Monad_infix in
  let sorted = List.sort ~cmp list in
  List.hd sorted >>= fun first ->
  List.last sorted >>= fun last ->
  Some (first,last)
;;
val compute_bounds : cmp:('a -> 'a -> int) -> 'a list -> ('a * 'a) option =
  <fun>
```

Ситуация улучшилась, но не существенно. И действительно, для таких небольших примеров, как этот, непосредственное сопоставление со значениями типа `option` обычно выглядит предпочтительнее, чем использование `bind`. Но для больших и сложных примеров с многоэтажной обработкой ошибок идиома применения `bind` выглядит яснее.

Существуют и другие интересные и практичные идиомы, реализованные в виде функций в модуле `Option`. Примером таких функций может служить `Option.both`, которая принимает два значения типа `option` и возвращает новую пару типа `option`, которая получит значение `None`, если хотя бы один из аргументов имеет значение `None`. С помощью `Option.both` можно еще больше сократить нашу функцию `compute_bounds`:

OCaml utop (part 14)

<https://github.com/realworldocaml/examples/blob/v1/code/error-handling/main.topscript>

```
# let compute_bounds ~cmp list =
  let sorted = List.sort ~cmp list in
  Option.both (List.hd sorted) (List.last sorted)
;;
val compute_bounds : cmp:('a -> 'a -> int) -> 'a list -> ('a * 'a) option =
  <fun>
```

Эти функции бесценны, потому что позволяют выражать обработку ошибок явно и кратко. Мы познакомились только с функциями в контексте модуля `Option`, но еще больше функций можно найти в модулях `Result` и `Or_error`.

Исключения

Исключения в OCaml ничем не отличаются от исключений во многих других языках, таких как Java, C# и Python. Исключения позволяют прервать вычисления и сообщить об ошибке. При этом имеется механизм, дающий возможность перехватить и обработать исключение (и, может быть, даже восстановить работоспособность), вызванное вложенными вычислениями.

Возбудить исключение можно, к примеру, попыткой выполнить деление на нуль:

OCaml utop (part 15)

<https://github.com/realworldocaml/examples/blob/v1/code/error-handling/main.topscript>

```
# 3 / 0;;
Exception: Division_by_zero.
```

Исключение может вызвать завершение программы независимо от глубины вложенности вычислений:

OCaml utop (part 16)

<https://github.com/realworldocaml/examples/blob/v1/code/error-handling/main.topscript>

```
# List.map ~f:(fun x -> 100 / x) [1;3;0;4];;
Exception: Division_by_zero.
```

Если вставить вызов `printf` в середину вычислений, можно увидеть, что работа `List.map` прерывается на полпути и ей так и не удается добраться до конца списка:

OCaml utop (part 17)

<https://github.com/realworldocaml/examples/blob/v1/code/error-handling/main.topscript>

```
# List.map ~f:(fun x -> printf "%d\n!" x; 100 / x) [1;3;0;4];;
1
3
0
Exception: Division_by_zero.
```

В дополнение к встроенным исключениям, таким как `Divide_by_zero`, OCaml позволяет определять собственные:

OCaml utop (part 18)

<https://github.com/realworldocaml/examples/blob/v1/code/error-handling/main.topscript>

```
# exception Key_not_found of string;;
exception Key_not_found of string
# raise (Key_not_found "a");;
Exception: Key_not_found("a").
```

Исключения — это обычные значения, и ими можно оперировать, как любыми другими значениями в языке OCaml:

OCaml utop (part 19)

<https://github.com/realworldocaml/examples/blob/v1/code/error-handling/main.topscript>

```
# let exceptions = [ Not_found; Division_by_zero; Key_not_found "b" ];;
val exceptions : exn list = [Not_found; Division_by_zero; Key_not_found("b")]
# List.filter exceptions ~f:(function
  | Key_not_found _ | Not_found -> true
  | _ -> false);;
- : exn list = [Not_found; Key_not_found("b")]
```

Все исключения принадлежат одному и тому же типу `exn`. Тип `exn` являет собой особый случай в системе типов OCaml. Он напоминает вариантные типы, с которыми мы познакомились в главе 6, с той лишь разницей, что является *открытым*, то есть он не определен полностью ни в какой точке программы. В частности, в него можно добавлять новые теги (новые исключения) в разных частях программы. Это существенное отличие исключений от обычных вариантных типов, которые определены в замкнутой вселенной доступных тегов. Как результат для типа `exn` нельзя построить исчерпывающее сопоставление с образцом, поскольку полный набор возможных исключений неизвестен.

Следующая функция использует исключение `Key_not_found`, объявленное нами выше, чтобы сообщить об ошибке:

OCaml utop (part 20)

<https://github.com/realworldocaml/examples/blob/v1/code/error-handling/main.topscript>

```
# let rec find_exn alist key = match alist with
  | [] -> raise (Key_not_found key)
  | (key', data) :: tl -> if key = key' then data else find_exn tl key
;;
val find_exn : (string * 'a) list -> string -> 'a = <fun>
# let alist = [("a", 1); ("b", 2)];;
val alist : (string * int) list = [("a", 1); ("b", 2)]
# find_exn alist "a";;
- : int = 1
# find_exn alist "c";;
Exception: Key_not_found("c").
```

Теперь мы дали функции имя `find_exn`, чтобы предупредить пользователя, что для нее свойственно возбуждать исключения. Это соглашение об именовании широко используется в библиотеке `Core`.

Функция `raise` в предыдущем примере возбуждает исключение и тем самым прерывает вычисления. Тип этой функции выглядит немного странным на первый взгляд:

OCaml utop (part 21)

<https://github.com/realworldocaml/examples/blob/v1/code/error-handling/main.topscript>

```
# raise;;
- : exn -> 'a = <fun>
```

Тип `'a` возвращаемого значения дает повод думать, что `raise` способна производить возвращаемые значения любого типа, без каких-либо ограничений. Это ка-

жется невозможным, но факт есть факт. В действительности `raise` имеет такой тип возвращаемого значения просто потому, что она вообще ничего и никогда не возвращает. Подобным поведением обладает не только функция `raise`, возбуждающая исключение. Ниже приводится пример еще одной функции, ничего не возвращающей:

OCaml utop (part 22)

<https://github.com/realworldocaml/examples/blob/v1/code/error-handling/main.topscript>

```
# let rec forever () = forever ();;
val forever : unit -> 'a = <fun>
```

Функция `forever` ничего не возвращает по другой причине: она образует бесконечный цикл.

Очень важно понимать эту особенность, потому что это означает, что тип значения, возвращаемого функцией `raise`, может быть любым, вписывающимся в контекст вызова. То есть система типов позволит вам возбудить исключение в любой точке программы.

Объявление исключений с использованием `sexp`

OCaml не всегда генерирует удобное для восприятия текстовое представление исключения. Например:

OCaml utop (part 23)

<https://github.com/realworldocaml/examples/blob/v1/code/error-handling/main.topscript>

```
# exception Wrong_date of Date.t;;
exception Wrong_date of Date.t
# Wrong_date (Date.of_string "2011-02-23");;
- : exn = Wrong_date(_)
```

Но если исключение объявлено с помощью `sexp` (и составляющие его типы имеют `sexp`-преобразователи), есть возможность получить больше информации:

OCaml utop (part 24)

<https://github.com/realworldocaml/examples/blob/v1/code/error-handling/main.topscript>

```
# exception Wrong_date of Date.t with sexp;;
exception Wrong_date of Date.t
# Wrong_date (Date.of_string "2011-02-23");;
- : exn = (//toplevel//.Wrong_date 2011-02-23)
```

Присутствие точки перед `Wrong_date` объясняется тем, что сгенерированное представление включает полный путь к модулю, где было определено исключение. В данном случае строка `//toplevel//` указывает, что исключение было объявлено на верхнем уровне, а не в модуле.

Все это является частью поддержки `s`-выражений в библиотеке `Sexplib`, которая более подробно описывается в главе 17.

Вспомогательные функции для возбуждения исключений

В OCaml и Core имеется множество вспомогательных функций, упрощающих задачу возбуждения исключений. Самой простой из них является функция `failwith`, которую можно было бы объявить, как показано ниже:

OCaml utop (part 25)

<https://github.com/realworldocaml/examples/blob/v1/code/error-handling/main.topscript>

```
# let failwith msg = raise (Failure msg);;
val failwith : string -> 'a = <fun>
```

Существует еще несколько удобных функций для возбуждения исключений, описание которых можно найти в документации к модулям `Common` и `Exp` в библиотеке `Core`.

Другой важный способ возбуждения исключений – директива `assert`. Эта директива используется в ситуациях, когда нарушение указанного условия свидетельствует об ошибке. Взгляните на следующий фрагмент, реализующий объединение двух списков:

OCaml utop (part 26)

<https://github.com/realworldocaml/examples/blob/v1/code/error-handling/main.topscript>

```
# let merge_lists xs ys ~f =
  if List.length xs <> List.length ys then None
  else
    let rec loop xs ys =
      match xs,ys with
      | [],[] -> []
      | x::xs, y::ys -> f x y :: loop xs ys
      | _ -> assert false
    in
      Some (loop xs ys)
;;

val merge_lists : 'a list -> 'b list -> f:('a -> 'b -> 'c) -> 'c list option =
  <fun>
# merge_lists [1;2;3] [-1;1;2] ~f:(+);;
- : int list option = Some [0; 3; 5]
# merge_lists [1;2;3] [-1;1] ~f:(+);;
- : int list option = None
```

Здесь используется директива `assert false`, которая гарантированно срабатывает. Вообще говоря, в директиву `assert` можно включить проверку любого условия.

В данном случае `assert` никогда не будет вызвана, потому что перед вызовом `loop` мы выполняем проверку равенства длин списков. Если изменить код, как показано ниже, где мы отбросили эту проверку, он сможет возбудить исключение с помощью `assert`:

OCaml utop (part 27)

<https://github.com/realworldocaml/examples/blob/v1/code/error-handling/main.topscript>

```
# let merge_lists xs ys ~f =
  let rec loop xs ys =
    match xs,ys with
    | [],[] -> []
    | x::xs, y::ys -> f x y :: loop xs ys
    | _ -> assert false
  in
    loop xs ys
;;
```

```
val merge_lists : 'a list -> 'b list -> f:('a -> 'b -> 'c) -> 'c list = <fun>
# merge_lists [1;2;3] [-1] ~f:(+);;
Exception: (Assert_failure //toplevel// 5 13).
```

Этот пример демонстрирует некоторые особенности директивы `assert`: она сохраняет номер строки и номер символа в строке в исходном коде, где была выполнена проверка.

Обработчики исключений

До сих пор мы видели только примеры, когда исключения прерывают вычисления. Но часто бывает желательно дать программе возможность среагировать на исключение и исправить ошибку. Такая возможность достигается с помощью *обработчиков исключений*.

В OCaml обработчики исключений объявляются с применением инструкции `try/with`. Ее синтаксис приводится ниже.

Syntax

https://github.com/realworldocaml/examples/blob/v1/code/error-handling/try_with.syntax

```
try <выражение> with
| <образец1> -> <выражение1>
| <образец2> -> <выражение2>
...
```

Инструкция `try/with` сначала вычисляет свое тело — выражение. Если оно не возбудит исключение, результат выражения станет общим результатом инструкции `try/with`.

Но если при вычислении выражения возникнет исключение, оно будет передано инструкциям сопоставления с образцом, следующим за `with`. Если найдено совпадение с каким-нибудь образцом, исключение считается перехваченным, и результатом всей конструкции `try/with` станет результат вычисления выражения справа от соответствующего образца.

В противном случае исходное исключение продолжит подъем по стеку вызовов, где сможет быть обработано следующим обработчиком исключений. Если исключение нигде не будет перехвачено, оно завершит выполнение программы.

Восстановление работоспособности после исключений

Одна из проблем, связанных с исключениями, состоит в том, что они могут прерывать вычисления в самых неожиданных местах, оставляя программу в противоречивом состоянии. Взгляните на следующую функцию, выполняющую загрузку файла с напоминаниями, оформленными в виде s-выражений:

OCaml utop (part 28)

<https://github.com/realworldocaml/examples/blob/v1/code/error-handling/main.topscript>

```
# let reminders_of_sexp =
  <:of_sexp<(Time.t * string) list>>
;;
val reminders_of_sexp : Sexp.t -> (Time.t * string) list = <fun>
```

```
# let load_reminders filename =
  let inc = In_channel.create filename in
  let reminders = reminders_of_sexp (Sexp.input_sexp inc) in
  In_channel.close inc;
  reminders
;;
val load_reminders : string -> (Time.t * string) list = <fun>
```

Проблема, собственно, в том, что функция, загружающая s-выражение и преобразующая его в список пар значений `Time.t/string`, может возбудить исключение, если файл содержит синтаксические ошибки. В этом случае открытый файл `In_channel.t` не будет закрыт, что приведет к утечке файловых дескрипторов.

Исправить эту проблему можно с помощью функции `protect` из библиотеки `Core`, которая принимает два аргумента: переходник (`thunk`) `f` – код, выполняющий основные вычисления; и переходник `finally`, который вызывается после вызова `f`, независимо от того, как завершился этот вызов – нормально или в результате исключения. Этим она напоминает конструкцию `try/finally`, имеющуюся во многих языках программирования, но реализована в виде библиотечной функции, а не как встроенный примитив. Ниже показано, как можно было бы использовать эту функцию для исправления `load_reminders`:

OCaml utop (part 29)

<https://github.com/realworldocaml/examples/blob/v1/code/error-handling/main.topscript>

```
# let load_reminders filename =
  let inc = In_channel.create filename in
  protect ~f:(fun () -> reminders_of_sexp (Sexp.input_sexp inc))
    ~finally:(fun () -> In_channel.close inc)
;;
val load_reminders : string -> (Time.t * string) list = <fun>
```

Это настолько типичная проблема, что для ее решения в составе `In_channel` предусмотрена функция `with_file`, автоматизирующая этот шаблон:

OCaml utop (part 30)

<https://github.com/realworldocaml/examples/blob/v1/code/error-handling/main.topscript>

```
# let reminders_of_sexp filename =
  In_channel.with_file filename ~f:(fun inc ->
    reminders_of_sexp (Sexp.input_sexp inc))
;;
val reminders_of_sexp : string -> (Time.t * string) list = <fun>
```

Функция `In_channel.with_file` реализована поверх `protect`, благодаря чему она способна «прибрать за собой» в случае исключения.

Перехват определенных исключений

Система обработки исключений в OCaml позволяет настраивать логику обработки ошибок на определенные исключения. Например, как известно, `List.find_exn` возбуждает исключение `Not_found` в случае неудачного поиска элемента в списке. Давайте разберем пример, как можно воспользоваться этим знанием. Взгляните на следующую функцию:

OCaml utop (part 31)

<https://github.com/realworldocaml/examples/blob/v1/code/error-handling/main.topscript>

```
# let lookup_weight ~compute_weight alist key =
  try
    let data = List.Assoc.find_exn alist key in
    compute_weight data
  with
    Not_found -> 0. ;;
val lookup_weight :
  compute_weight:('a -> float) -> ('b, 'a) List.Assoc.t -> 'b -> float =
  <fun>
```

Как следует из сигнатуры, `lookup_weight` принимает ассоциативный список, ключ для поиска значения и функцию вычисления вещественного веса найденного значения. Если искомый ключ не будет найден, функция вернет вес, равный 0..

Однако использование исключений в таком коде влечет за собой некоторые проблемы. В частности, что случится, если `compute_weight` возбудит исключение? В идеале `lookup_weight` должна передать это исключение дальше, если только это не исключение `Not_found`, но в действительности этот код работает совсем не так:

OCaml utop (part 32)

<https://github.com/realworldocaml/examples/blob/v1/code/error-handling/main.topscript>

```
# lookup_weight ~compute_weight:(fun _ -> raise Not_found)
  ["a",3; "b",4] "a" ;;
- : float = 0.
```

Источники подобных проблем очень сложно выявлять заранее, потому что система типов не сообщает о том, какие исключения могут возбуждаться функцией. По этой причине лучше не полагаться на идентификацию исключений в попытках выяснить природу ошибки. Более практичный подход заключается в сужении области действия обработчика исключений, чтобы при их появлении легко можно было понять, в каком месте случилась ошибка:

OCaml utop (part 33)

<https://github.com/realworldocaml/examples/blob/v1/code/error-handling/main.topscript>

```
# let lookup_weight ~compute_weight alist key =
  match
    try Some (List.Assoc.find_exn alist key)
    with _ -> None
  with
    | None -> 0.
    | Some data -> compute_weight data ;;
val lookup_weight :
  compute_weight:('a -> float) -> ('b, 'a) List.Assoc.t -> 'b -> float =
  <fun>
```

Совершенно очевидно, что здесь лучше было бы использовать функцию `List.Assoc.find`, не возбуждающую исключений:

OCaml utop (part 34)

<https://github.com/realworldocaml/examples/blob/v1/code/error-handling/main.topscrip>

```
# let lookup_weight ~compute_weight alist key =
  match List.Assoc.find alist key with
  | None -> 0.
  | Some data -> compute_weight data ;;
val lookup_weight :
  compute_weight:(('a -> float) -> ('b, 'a) List.Assoc.t -> 'b -> float =
  <fun>
```

Трассировка стека

Самой большой ценностью исключений является возможность получить из них полезную отладочную информацию в форме трассировки стека. Рассмотрим следующую простую программу:

OCaml

https://github.com/realworldocaml/examples/blob/v1/code/error-handling/blow_up.ml

```
open Core.Std
exception Empty_list

let list_max = function
  | [] -> raise Empty_list
  | hd :: tl -> List.fold tl ~init:hd ~f:(Int.max)

let () =
  printf "%d\n" (list_max [1;2;3]);
  printf "%d\n" (list_max [])
```

Скомпилировав и запустив эту программу, мы получим трассировочную информацию, содержащую некоторые сведения о месте, где возникла ошибка, и цепочку вызовов функций, имевшую место на момент ошибки:

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/error-handling/build_blow_up.out

```
$ corebuild blow_up.byte
$ ./blow_up.byte
3
Fatal error: exception Blow_up.Empty_list
Raised at file "blow_up.ml", line 5, characters 16-26
Called from file "blow_up.ml", line 10, characters 17-28
```

Получить трассировку стека внутри программы можно вызовом функции `Exn.backtrace`, которая возвращает трассировку для самого последнего исключения. Это может пригодиться для сообщения подробной информации об ошибках, которые не заставили программу завершиться аварийно.

Однако этот прием действует, только если трассировка разрешена, что бывает не всегда. В действительности по умолчанию трассировка стека в OCaml отключена, и даже если включить ее во время выполнения программы, вы не сможете получить трассировку, если программа не скомпилирована с отладочной информацией. Библиотека полностью изменяет настройки по умолчанию, поэтому, если

программа скомпонована с библиотекой Core, она получит включенную поддержку обратной трассировки.

Но даже если программа скомпонована с библиотекой Core и скомпилирована с отладочной информацией, у вас всегда есть возможность отключить трассировку, записав в переменную окружения OCAMLRUNPARAM пустое значение:

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/error-handling/build_blow_up_notrace.out

```
$ corebuild blow_up.byte
$ OCAMLRUNPARAM= ./blow_up.byte
3
Fatal error: exception Blow_up.Empty_list
```

Получившееся сообщение об ошибке значительно менее информативно. Выключить трассировку программно можно вызовом `Backtrace.Exn.set_recording false`.

Есть вполне законные причины отключать трассировку, и важнейшей из них является скорость. Исключения в OCaml действуют очень быстро, но они будут работать еще быстрее, если запретить трассировку. Ниже приводятся результаты простого хронометража, выполненного с помощью пакета `core_bench`:

OCaml

https://github.com/realworldocaml/examples/blob/v1/code/error-handling/exn_cost.ml

```
open Core.Std
open Core_bench.Std

let simple_computation () =
  List.range 0 10
  |> List.fold ~init:0 ~f:(fun sum x -> sum + x * x)
  |> ignore

let simple_with_handler () =
  try simple_computation () with Exit -> ()

let end_with_exn () =
  try
    simple_computation ();
    raise Exit
  with Exit -> ()

let () =
  [ Bench.Test.create ~name:"simple computation"
    (fun () -> simple_computation ());
    Bench.Test.create ~name:"simple computation w/handler"
    (fun () -> simple_with_handler ());
    Bench.Test.create ~name:"end with exn"
    (fun () -> end_with_exn ());
  ]
  |> Bench.make_command
  |> Command.run
```

Мы тестируем три ситуации: простые вычисления без исключений; те же самые вычисления с обработчиком исключений, но не возбуждающие исключений; и наконец, те же самые вычисления, где исключения используются для возврата управления вызывающему коду.

Мы запустили программу с включенной поддержкой трассировки и получили следующие результаты:

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/error-handling/run_exn_cost.out

```
$ corebuild -pkg core_bench exn_cost.native
$ ./exn_cost.native -ascii cycles
Estimated testing time 30s (change using -quota SECS).
```

Name	Cycles	Time (ns)	% of max
-----	-----	-----	-----
simple computation	279	117	76.40
simple computation w/handler	308	129	84.36
end with exn	366	153	100.00

Здесь видно, что мы потеряли что-то около 30 циклов за счет добавления обработчика исключений и еще почти 60 из-за фактического возбуждения исключений. Затем мы отключили поддержку трассировки и вновь провели тестирование:

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/error-handling/run_exn_cost_notrace.out

```
$ OCAMLRUNPARAM= ./exn_cost.native -ascii cycles
Estimated testing time 30s (change using -quota SECS).
```

Name	Cycles	Time (ns)	% of max
-----	-----	-----	-----
simple computation	279	116	83.50
simple computation w/handler	308	128	92.09
end with exn	334	140	100.00

Здесь цена обработчика осталась прежней, но вот цена самих исключений теперь составила всего 25 дополнительных циклов в противовес 60. Следует, однако, отметить, что все вышесказанное имеет значение только в ситуациях, когда исключения используются как инструмент управления потоком выполнения, что в большинстве случаев считается стилистической ошибкой.

От исключений к типам с информацией об ошибках и обратно

И исключения, и типы возвращаемых значений с информацией об ошибках являются неотъемлемыми компонентами программирования на OCaml. Вам часто придется выбирать между этими двумя мирами. К счастью, в состав Core входит несколько вспомогательных функций, которые помогут вам в этом. Например,

если имеется фрагмент кода, способный возбуждать исключения, вы сможете перехватывать их и заключать в тип `option`, как показано ниже:

OCaml utop (part 35)

<https://github.com/realworldocaml/examples/tree/v1/code/error-handling/main.topscript>

```
# let find alist key =
  Option.try_with (fun () -> find_exn alist key) ;;
val find : (string * 'a) list -> string -> 'a option = <fun>
# find ["a",1; "b",2] "c";;
- : int option = None
# find ["a",1; "b",2] "b";;
- : int option = Some 2
```

Оба модуля, `Result` и `Or_error`, имеют похожие функции `try_with`. Соответственно, мы можем записать:

OCaml utop (part 36)

<https://github.com/realworldocaml/examples/tree/v1/code/error-handling/main.topscript>

```
# let find alist key =
  Or_error.try_with (fun () -> find_exn alist key) ;;
val find : (string * 'a) list -> string -> 'a Or_error.t = <fun>
# find ["a",1; "b",2] "c";;
- : int Or_error.t = Core_kernel.Result.Error ("Key_not_found(\"c\")")
```

И затем повторно возбудить исключение:

OCaml utop (part 37)

<https://github.com/realworldocaml/examples/tree/v1/code/error-handling/main.topscript>

```
# Or_error.ok_exn (find ["a",1; "b",2] "b");;
- : int = 2
# Or_error.ok_exn (find ["a",1; "b",2] "c");;
Exception: ("Key_not_found(\"c\")").
```

Выбор стратегии обработки ошибок

Язык OCaml поддерживает и исключения, и типы возвращаемых значений с информацией об ошибках, так какой же способ предпочтительнее? В зависимости от того, что важнее — краткость или явность.

Исключения более компактны, потому что они позволяют перенести обработку ошибок в более обширную область видимости, а также потому, что не загромождают ваших типов данных. Но их краткость имеет свою цену: исключения слишком легко игнорируются. Типы возвращаемых значений с информацией об ошибках, напротив, полностью объявляются в ваших типах, благодаря чему ошибки оказываются более явными и их невозможно игнорировать.

Правильный выбор во многом зависит от самого приложения. Если вы пишете прикладную версию программы, когда важно получить действующую модель как можно быстрее, а ошибки не так дороги, возможно, правильнее будет использовать исключения. Напротив, при разработке промышленного ПО, где ошибки

обходятся очень дорого, лучше все-таки определить типы возвращаемых значений, содержащие информацию об ошибках.

Однако нет смысла полностью избегать исключений. Вы с успехом можете следовать принципу: «исключения – для исключительных ситуаций». Если ошибка маловероятна, то выбор исключения для ее обозначения часто оказывается более оправданным.

Кроме того, если ошибка является вездесущей, реализация ее обработки с применением типов значений с информацией об ошибках может оказаться убийственно трудоемкой задачей. Отличным примером может служить ошибка исчерпания памяти, которая способна возникнуть в любой точке программы и из-за чего придется использовать типы значений с информацией об ошибках повсюду. Ситуация, когда каждая операция помечается как способная сгенерировать ошибку, ничуть не лучше, чем менее явный прием обработки на основе исключений.

Проще говоря, подход на основе типов значений с информацией об ошибках лучше применять для обработки типичных ошибок, которые часто могут возникать в процессе работы, но которые не являются вездесущими.

Императивное программирование

Большая часть кода, который мы видели до сих пор, как и большая часть кода на языке OCaml в целом, является *чисто функциональной*. Чисто функциональный код не изменяет внутреннего состояния программы, не выполняет операций ввода/вывода, не читает значения системных часов и вообще никак не взаимодействует с изменчивым миром. То есть чистые функции действуют на манер математических функций, всегда возвращая одно и то же значение для одного и того же набора аргументов и никогда не воздействуя на окружающий мир иначе, как через возвращаемое значение. Императивный код, напротив, действует с побочными эффектами, изменяя внутреннее состояние программы или взаимодействуя с внешним миром. Императивная функция может иметь побочные эффекты и возвращать разные результаты для одного и того же набора аргументов.

По умолчанию на OCaml пишется чисто функциональный код, и это имеет свои преимущества – в целом функциональный код легче анализировать, в нем ниже вероятность появления ошибок, и он более читабельный. Но императивный код имеет фундаментальное значение для практического программирования, потому что для решения практических задач требуется взаимодействовать с окружающим миром, который по своей природе является императивным. Кроме того, императивное программирование обеспечивает создание более производительных программ. Чисто функциональный код на OCaml достаточно эффективен, и все же существует масса алгоритмов, которые могут быть эффективно реализованы только с применением императивных приемов.

Язык OCaml предлагает замечательный компромисс в этом отношении, давая простой и естественный способ создания программ в чисто функциональном стиле и предоставляя великолепную поддержку императивного стиля программирования. В этой главе мы пройдемся по императивным особенностям языка OCaml и покажем вам, как использовать их на полную мощь.

Пример: императивные словари

Начнем с реализации простого императивного словаря, то есть с изменяемого отображения ключей в значения. Этот словарь мы создадим исключительно в иллюстративных целях; и библиотека Core, и стандартная библиотека компилятора включают собственные реализации императивных словарей, которые и следует

использовать в практическом программировании. Дополнительные рекомендации по использованию реализации из библиотеки Core вы найдете в главе 13.

Словарь, который мы собираемся описать, напоминает словари из библиотеки Core и стандартной библиотеки и будет реализован как хэш-таблица. В частности, мы будем использовать *открытую схему хэширования*, где хэш-таблица будет представлена массивом ячеек, содержащих списки пар ключ/значение и адресуемых хэш-кодами.

Ниже приводится интерфейс, на основе которого будет конструироваться реализация. Тип `('a, 'b) t` представляет словарь с ключами типа `'a` и данными типа `'b`:

OCaml (part 1)

<https://github.com/realworldocaml/examples/tree/v1/code/imperative-programming/dictionary.mli>

```
(* файл: dictionary.mli *)
open Core.Std
```

```
type ('a, 'b) t
```

```
val create : unit -> ('a, 'b) t
val length : ('a, 'b) t -> int
val add    : ('a, 'b) t -> key:'a -> data:'b -> unit
val find   : ('a, 'b) t -> 'a -> 'b option
val iter   : ('a, 'b) t -> f:(key:'a -> data:'b -> unit) -> unit
val remove : ('a, 'b) t -> 'a -> unit
```

Файл *.mli* с определением интерфейса также включает коллекцию объявлений вспомогательных функций, назначение и поведение которых должны быть очевидны из их названий и сигнатур. Отметим, что некоторые функции, в особенности те, что изменяют словарь (такие как `add`), возвращают значение типа `unit`. Это типично для функций, действующих с побочными эффектами.

Теперь пройдемся по реализации (в соответствующем файле *.ml*) и шаг за шагом исследуем встречающиеся там различные императивные конструкции.

Наш первый шаг – определение типа словаря в виде записи с двумя полями:

OCaml (part 1)

<https://github.com/realworldocaml/examples/tree/v1/code/imperative-programming/dictionary.ml>

```
(* файл: dictionary.ml *)
open Core.Std
```

```
type ('a, 'b) t = { mutable length: int;
                   buckets: ('a * 'b) list array;
}
```

Первое поле `length` объявлено как изменяемое. По умолчанию записи в языке OCaml являются неизменяемыми, но отдельные поля могут быть изменяемыми, если пометить их ключевым словом `mutable`. Второе поле `buckets` является неизменяемым, но оно содержит массив, который сам по себе является изменяемой структурой данных.

Теперь перейдем к основным функциям управления словарем:

OCaml (part 2)

<https://github.com/realworldocaml/examples/tree/v1/code/imperative-programming/dictionary.ml>

```
let num_buckets = 17

let hash_bucket key = (Hashtbl.hash key) mod num_buckets

let create () =
  { length = 0;
    buckets = Array.create ~len:num_buckets [];
  }

let length t = t.length

let find t key =
  List.find_map t.buckets.(hash_bucket key)
    ~f:(fun (key',data) -> if key' = key then Some data else None)
```

Обратите внимание, что `num_buckets` — это константа, то есть массив `bucket` имеет фиксированную длину. В практической реализации могло бы потребоваться обеспечить возможность увеличения размера массива с увеличением числа элементов в словаре, но мы не будем предусматривать это, чтобы упростить пример.

Функция `hash_bucket` будет использоваться в модуле повсеместно, с целью определения позиции элемента массива для заданного ключа в массиве. Она реализована на основе функции `Hashtbl.hash` — хэш-функции в библиотеке OCaml, которая может применяться к значениям любого типа. То есть ее сигнатура имеет вид: `'a -> int`.

Другие функции, представленные выше, имеют довольно простые реализации:

- `create` — создает пустой словарь;
- `length` — извлекает значение длины из соответствующего поля записи, то есть возвращает число элементов, хранящихся в словаре;
- `find` — отыскивает ключ в таблице и возвращает соответствующее ему значение в виде значения типа `option`.

Еще одной важной чертой императивного синтаксиса, которую можно наблюдать в функции `find`, является форма записи `array.(index)`, применяемая для извлечения значения из массива. Функция `find` также использует функцию `List.find_map`, тип которой можно узнать, просто введя ее имя в интерактивной оболочке:

OCaml utop (part 1)

<https://github.com/realworldocaml/examples/blob/v1/code/imperative-programming/examples.topscript>

```
# List.find_map;;
- : 'a list -> f:('a -> 'b option) -> 'b option = <fun>
```

Функция `List.find_map` выполняет итерации по элементам списка, вызывая `f` для каждого из них, пока `f` не вернет значение типа `Some`, которое и возвращается вы-

зывающему коду. Если `f` вернет `None` для всех значений, вызывающему коду также будет возвращено `None`.

Теперь рассмотрим реализацию `iter`:

OCaml (part 3)

<https://github.com/realworldocaml/examples/blob/v1/code/imperative-programming/dictionary.ml>

```
let iter t ~f =
  for i = 0 to Array.length t.buckets - 1 do
    List.iter t.buckets.(i) ~f:(fun (key, data) -> f ~key ~data)
  done
```

Функция `iter` предназначена для обхода всех элементов словаря. В частности, `iter t ~f` вызовет `f` для каждой пары ключ/значение в словаре `t`. Отметьте, что `f` должна возвращать значение типа `unit`, поскольку ожидается, что она будет производить побочные эффекты, а не возвращать что-то полезное. И сама функция `iter` также возвращает значение типа `unit`.

В теле `iter` используются две формы итераций: цикл `for` выполняет обход элементов массива; а внутри этого цикла вызывается `List.iter` для обхода значений в этом элементе. Вместо внешнего цикла `for` обход элементов массива можно было бы реализовать посредством рекурсивных вызовов, но цикл `for` синтаксически удобнее и идиоматически более подходит в императивных контекстах.

Следующий код реализует добавление элементов в словарь и их удаление:

OCaml (part 4)

<https://github.com/realworldocaml/examples/blob/v1/code/imperative-programming/dictionary.ml>

```
let bucket_has_key t i key =
  List.exists t.buckets.(i) ~f:(fun (key', _) -> key' = key)

let add t ~key ~data =
  let i = hash_bucket key in
  let replace = bucket_has_key t i key in
  let filtered_bucket =
    if replace then
      List.filter t.buckets.(i) ~f:(fun (key', _) -> key' <> key)
    else
      t.buckets.(i)
  in
  t.buckets.(i) <- (key, data) :: filtered_bucket;
  if not replace then t.length <- t.length + 1

let remove t key =
  let i = hash_bucket key in
  if bucket_has_key t i key then (
    let filtered_bucket =
      List.filter t.buckets.(i) ~f:(fun (key', _) -> key' <> key)
    in
    t.buckets.(i) <- filtered_bucket;
    t.length <- t.length - 1
  )
```

Код выше гораздо сложнее из-за необходимости проверять существование элемента перед добавлением и удалением, чтобы знать, следует ли изменять `t.length`. С этой целью используется функция `bucket_has_key`.

Еще одной важной чертой императивного синтаксиса, которую можно наблюдать в обеих функциях — `add` и `remove`, — является использование оператора `<-` для изменения элементов массива (в форме: `array.(i) <- expr`) и для изменения полей записей (в форме: `record.field <- expression`).

Мы также использовали оператор упорядочения (`;`), чтобы определить последовательность выполнения императивных операций. То же самое можно было бы сделать с помощью `let`-привязок:

OCaml (part 1)

<https://github.com/realworldocaml/examples/blob/v1/code/imperative-programming/dictionary2.ml>

```
let () = t.buckets.(i) <- (key, data) :: filtered_bucket in
  if not replace then t.length <- t.length + 1
```

но оператор `;` более идиоматичен и краток. В общем случае конструкция

Syntax

<https://github.com/realworldocaml/examples/blob/v1/code/imperative-programming/semicolon.syntax>

```
<выражение1>;
<выражение2>;
...
<выражениеN>
```

эквивалентна:

Syntax

<https://github.com/realworldocaml/examples/blob/v1/code/imperative-programming/let-unit.syntax>

```
let () = <выражение1> in
let () = <выражение2> in
...
<выражениеN>
```

При вычислении последовательности выражений `expr1`; `expr2` выражение `expr1` вычисляется первым, а выражение `expr2` — вторым. Выражение `expr1` должно иметь тип `unit` (впрочем, это скорее пожелание, чем жесткое ограничение; превратить его в жесткое ограничение можно с помощью флага `-strict-sequence` компилятора, что можно только приветствовать), а значением всей последовательности становится значение выражения `expr2`. Например, последовательность `print_string "hello world"; 1 + 2` сначала выведет строку `"hello world"`, а затем вернет целое число 3.

Отметьте также, что все операции, производящие побочные эффекты, мы выполняем в самом конце каждой функции. Это общепринятая практика, потому что она снижает вероятность прерывания таких операций исключениями и оставления структур данных в противоречивом состоянии.

Элементарные изменяемые данные

Теперь, когда мы познакомились с законченным примером, давайте более последовательно рассмотрим императивное программирование на OCaml. Выше мы встретились с двумя разными формами изменяемых данных: записями с изменяемыми полями и массивами. Давайте обсудим их более подробно, наряду с другими элементарными формами изменяемых данных в OCaml.

Данные в формах, подобных массивам

OCaml поддерживает множество структур данных, подобных массивам; то есть изменяемых контейнеров, индексируемых целыми числами, которые обеспечивают постоянное время доступа к своим элементам. Некоторые из них мы рассмотрим в этом разделе.

Обычные массивы

Универсальные полиморфные массивы представлены типом `array`. Модуль `Array` содержит разнообразные вспомогательные функции для выполнения операций с массивами, в том числе и операции, вызывающие изменения. К их числу относятся `Array.set` для изменения отдельных элементов и `Array.blit` для эффективного копирования значений из одного диапазона индексов в другой.

Массивы также поддерживают специальный синтаксис для извлечения элементов из массива:

Syntax

<https://github.com/realworldocaml/examples/blob/v1/code/imperative-programming/array-get.syntax>

```
<выражение_массив>.(<выражение_индекс>)
```

и для их изменения:

Syntax

<https://github.com/realworldocaml/examples/blob/v1/code/imperative-programming/array-set.syntax>

```
<выражение_массив>.(<выражение_индекс>) <- <выражение_значение>
```

Обращение к индексам за границами массива (и любых других структур данных, подобных массивам) вызывает исключение.

Литералы массивов записываются с использованием последовательностей символов `[]` и `||` в качестве ограничителей. То есть `[| 1; 2; 3 |]` – это литерал массива целых чисел.

Строки

Строки фактически являются массивами байтов, которые часто используются для представления текстовых данных. Основное преимущество типа `string` перед `Char.t array` (`Char.t` – это 8-битный символ) состоит в том, что первый более экономно расходует память; для представления одного символа в массиве расходуют

ся 8 байт памяти в 64-битной системе, тогда как в строках каждый символ представлен всего одним байтом.

Строки также имеют свой, уникальный синтаксис для извлечения и изменения данных:

Syntax

<https://github.com/realworldocaml/examples/blob/v1/code/imperative-programming/string.syntax>

```
<строковое_выражение>.[<выражение_индекс>]
<строковое_выражение>.[<выражение_индекс>] <- <выражение_символ>
```

А строковые литералы заключаются в кавычки. Имеется также модуль `String`, где вы найдете вспомогательные функции для работы со строками.

Большие массивы

Тип `Bigarray.t` применяется для хранения блоков памяти за пределами кучи (области динамической памяти) OCaml. Основное назначение этого типа – использование для взаимодействий с библиотеками на C или Fortran. Более подробно он обсуждается в главе 20. Для работы с большими массивами также используется свой синтаксис:

Syntax

<https://github.com/realworldocaml/examples/blob/v1/code/imperative-programming/bigarray.syntax>

```
<выражение_большого_массива>.{<выражение_индекс>}
<выражение_большого_массива>.{<выражение_индекс>} <- <выражение_значения>
```

Изменяемые поля записей и объектов и ссылочные ячейки

Как мы уже видели, записи являются неизменяемыми, но отдельные их поля могут объявляться изменяемыми. Запись значений в эти изменяемые поля можно выполнить с помощью оператора `<-`, например: `record.field <- expr`.

Как будет рассказываться в главе 11, поля объектов тоже могут объявляться изменяемыми и изменяться точно так же, как поля записей.

Ссылочные ячейки

Переменные в OCaml никогда не изменяются – они могут ссылаться на изменяемые данные, но сами переменные, указывающие на эти данные, не изменяются. Однако иногда бывает желательно иметь возможность определять отдельные изменяемые переменные, в точности похожие на переменные в других языках. В OCaml это достигается с помощью типа `ref`, который фактически является контейнером единственного полиморфного изменяемого поля.

Ниже показано, как определить значение типа `ref`:

OCaml utop (part 1)

<https://github.com/realworldocaml/examples/blob/v1/code/imperative-programming/ref.topscrip>

```
# type 'a ref = { mutable contents : 'a };;
type 'a ref = { mutable contents : 'a; }
```


Стандартная библиотека определяет следующие операторы для работы с типом `ref`.

- `ref expr` – создает ссылочную ячейку (reference cell), содержащую значение выражения `expr`.
- `!refcell` – возвращает содержимое ссылочной ячейки.
- `refcell := expr` – замещает содержимое ссылочной ячейки.

Взгляните, как они действуют:

OCaml utop (part 3)

<https://github.com/realworldocaml/examples/blob/v1/code/imperative-programming/ref.topscript>

```
# let x = ref 1;;
val x : int ref = {contents = 1}
# !x;;
- : int = 1
# x := !x + 1;;
- : unit = ()
# !x;;
- : int = 2
```

Это всего лишь обычные функции на языке OCaml, которые можно было бы определить так:

OCaml utop (part 2)

<https://github.com/realworldocaml/examples/blob/v1/code/imperative-programming/ref.topscript>

```
# let ref x = { contents = x };;
val ref : 'a -> 'a ref = <fun>
# let (!) r = r.contents;;
val ( ! ) : 'a ref -> 'a = <fun>
# let (:=) r x = r.contents <- x;;
val ( := ) : 'a ref -> 'a -> unit = <fun>
```

Внешние функции

Еще один источник императивных операций в OCaml – ресурсы из внешних библиотек, получаемые посредством интерфейса внешних функций (Foreign Function Interface, FFI). Интерфейс FFI открывает программам на OCaml доступ к императивным конструкциям, которые экспортируются системными вызовами или другими внешними библиотеками. Многие из них являются встроенными, например доступ к системному вызову `write` или `clock`, а другие поступают из пользовательских библиотек, таких как LAPACK. Интерфейс FFI в языке OCaml обсуждается в главе 19.

Циклы `for` и `while`

В языке OCaml имеется поддержка традиционных императивных циклов, в частности циклов `for` и `while`. Ни одна из этих конструкций не является по-настоящему необходимой, потому что их легко можно эмулировать с помощью рекурсивных

функций. Тем не менее циклы `for` и `while` позволяют писать более краткий и идиоматичный императивный код.

Цикл `for` является самым простым из этой пары. На самом деле мы уже видели цикл `for` в действии – в функции `iter` (в модуле `Dictionary`). Ниже приводится еще один простой пример цикла `for`:

OCaml utop (part 1)

<https://github.com/realworldocaml/examples/blob/v1/code/imperative-programming/for.topscript>

```
# for i = 0 to 3 do printf "i = %d\n" i done;;
i = 0
i = 1
i = 2
i = 3
- : unit = ()
```

Как видите, верхняя и нижняя границы участвуют в цикле. Итерации в обратном направлении можно организовать с помощью ключевого слова `downto`:

OCaml utop (part 2)

<https://github.com/realworldocaml/examples/blob/v1/code/imperative-programming/for.topscript>

```
# for i = 3 downto 0 do printf "i = %d\n" i done;;
i = 3
i = 2
i = 1
i = 0
- : unit = ()
```

Обратите внимание, что переменная цикла `for`, в данном случае `i`, является неизменяемой в области видимости цикла, а также локальной по отношению к циклу, то есть она недоступна за пределами цикла.

В OCaml также поддерживаются циклы `while`, состоящие из условного выражения и тела. Цикл сначала вычисляет условное выражение, а затем, если результат имеет истинное значение, выполняет тело и снова переходит в начало цикла. Ниже приводится простой пример функции для перестановки элементов массива в обратном порядке:

OCaml utop (part 3)

<https://github.com/realworldocaml/examples/blob/v1/code/imperative-programming/for.topscript>

```
# let rev_inplace ar =
  let i = ref 0 in
  let j = ref (Array.length ar - 1) in
  (* завершить, когда верхний и нижний индексы встретятся *)
  while !i < !j do
    (* поменять элементы местами *)
    let tmp = ar.(!i) in
    ar.(!i) <- ar.(!j);
    ar.(!j) <- tmp;
```

```

    (* передвинуть индексы *)
    incr i;
    decr j
  done
;;
val rev_inplace : 'a array -> unit = <fun>
# let nums = [|1;2;3;4;5|];;
val nums : int array = [|1; 2; 3; 4; 5|]
# rev_inplace nums;;
- : unit = ()
# nums;;
- : int array = [|5; 4; 3; 2; 1|]

```

В этом примере использовались функции `incr` и `decr`, которые являются встроенными функциями, выполняющими увеличение и уменьшение значений типа `int ref` на единицу соответственно.

Пример: двусвязные списки

Еще одну распространенную императивную структуру данных представляют двусвязные списки. Двусвязные списки позволяют выполнять итерации в обоих направлениях, а добавление элементов и их удаление выполняются за постоянное время. Библиотека `Core` определяет свой тип двусвязных списков (в модуле `Doubly_linked`), но мы не будем его трогать и напишем свою библиотеку для работы со связанными списками для иллюстрации.

Ниже приводится содержимое файла `.mli` для нашего будущего модуля:

OCaml

<https://github.com/realworldocaml/examples/blob/v1/code/imperative-programming/dlist.mli>

```

(* файл: dlist.mli *)
open Core.Std

type 'a t
type 'a element

(** Базовые операции со списком *)
val create : unit -> 'a t
val is_empty : 'a t -> bool

(** Навигация с использованием [элемента](ов) *)
val first : 'a t -> 'a element option
val next : 'a element -> 'a element option
val prev : 'a element -> 'a element option
val value : 'a element -> 'a

(** Обход всей структуры данных *)
val iter : 'a t -> f:('a -> unit) -> unit
val find_el : 'a t -> f:('a -> bool) -> 'a element option

(** Изменение *)

```

```
val insert_first : 'a t -> 'a -> 'a element
val insert_after : 'a element -> 'a -> 'a element
val remove : 'a t -> 'a element -> unit
```

Обратите внимание на определения двух типов: 'a t, тип списка, и 'a element, тип элемента. Элементы действуют подобно указателям — они позволяют осуществлять навигацию по списку и указывают на данные, к которым можно применить операцию изменения.

Теперь перейдем к реализации. Сначала определим типы 'a element и 'a t:

OCaml (part 1)

<https://github.com/realworldocaml/examples/blob/v1/code/imperative-programming/dlist.ml>

```
(* файл: dlist.ml *)
open Core.Std

type 'a element =
  { value : 'a;
    mutable next : 'a element option;
    mutable prev : 'a element option
  }

type 'a t = 'a element option ref
```

Тип 'a element — это запись, содержащая хранимое значение (value) данного узла, а также необязательные (и изменяемые) поля, указывающие на предыдущий (prev) и следующий (next) элементы в списке. В первой записи в списке поле prev хранит значение None, аналогично в последней записи поле next хранит значение None.

Сам список имеет тип 'a t — изменяемая ссылка на необязательный элемент. Если список пуст, эта ссылка будет хранить значение None, и Some — в противном случае.

Теперь определим несколько простых функций для работы со списком и его элементами:

OCaml (part 2)

<https://github.com/realworldocaml/examples/blob/v1/code/imperative-programming/dlist.ml>

```
let create () = ref None
let is_empty t = !t = None

let value elt = elt.value

let first t = !t
let next elt = elt.next
let prev elt = elt.prev
```

Их реализации напрямую вытекают из определений типов.

Циклические структуры данных

Двусвязные списки — это циклическая структура данных, в том смысле, что возможно, следуя по последовательности указателей, вернуться в ее начало. Вообще говоря, при создании циклической структуры данных побочные эффекты неизбежны. Эти эффекты

выражаются в создании элементов данных с последующим их включением в циклическую структуру.

Впрочем, из этого правила есть исключение: циклическую структуру данных фиксированного размера можно создать с использованием `let rec`:

OCaml utop (part 2)

<https://github.com/realworldocaml/examples/blob/v1/code/imperative-programming/examples.topscript>

```
# let rec endless_loop = 1 :: 2 :: 3 :: endless_loop;;
val endless_loop : int list =
  [1; 2; 3; 1; 2; 3; 1; 2; 3;
   1; 2; 3; 1; 2; 3; 1; 2; 3;
   1; 2; 3; 1; 2; 3; 1; 2; 3;
   ...]
```

Однако этот подход имеет существенные ограничения. Циклические структуры данных должны быть изменяемыми.

Изменение списка

Теперь приступим к изучению операций, изменяющих список, и начнем с функции `insert_first`, которая вставляет элемент в начало списка:

OCaml (part 3)

<https://github.com/realworldocaml/examples/blob/v1/code/imperative-programming/dlist.ml>

```
let insert_first t value =
  let new_elt = { prev = None; next = !t; value } in
  begin match !t with
  | Some old_first -> old_first.prev <- Some new_elt
  | None -> ()
  end;
  t := Some new_elt;
  new_elt
```

Функция `insert_first` сначала определяет новый элемент `new_elt`, затем добавляет его в список и, наконец, записывает в указатель на список ссылку на `new_elt`. Имейте в виду, что приоритет выражения сопоставления очень низок, поэтому, чтобы отделить его от следующего присваивания (`t := Some new_elt`), мы заключили сопоставление в операторные скобки `begin ... end`. Тот же эффект можно было бы получить с помощью круглых скобок. Без скобок, операторных или круглых, заключительное присваивание оказалось бы частью случая `None` в выражении сопоставления.

Для вставки нового элемента после определенного служит функция `insert_after`. Она принимает в качестве аргументов элемент, после которого нужно вставить новый узел, и значение для нового элемента:

OCaml (part 4)

<https://github.com/realworldocaml/examples/blob/v1/code/imperative-programming/dlist.ml>

```
let insert_after elt value =
  let new_elt = { value; prev = Some elt; next = elt.next } in
```

```
begin match elt.next with
| Some old_next -> old_next.prev <- Some new_elt
| None -> ()
end;
elt.next <- Some new_elt;
new_elt
```

Наконец, нам нужна функция `remove`:

OCaml (part 5)

<https://github.com/realworldocaml/examples/blob/v1/code/imperative-programming/dlist.ml>

```
let remove t elt =
  let { prev; next; _ } = elt in
  begin match prev with
  | Some prev -> prev.next <- next
  | None -> t := next
  end;
  begin match next with
  | Some next -> next.prev <- prev;
  | None -> ()
  end;
  elt.prev <- None;
  elt.next <- None
```

Обратите внимание, как осторожно выполняется изменение указателя `prev` следующего элемента и указателя `next` — предыдущего, если они существуют. Если предыдущий элемент отсутствует, изменяется указатель на сам список. В любом случае указатели `next` и `prev` самого удаляемого элемента устанавливаются в значение `None`.

Эти функции более хрупкие, чем может показаться. В частности, неправильное использование интерфейса может привести к повреждению данных. Например, если попытаться дважды удалить один и тот же элемент, это приведет к тому, что ссылка на список получит значение `None`, то есть список будет очищен полностью. Аналогичные проблемы возникнут при удалении элемента, не принадлежащего списку.

Это не должно быть неожиданностью. Комплексные императивные структуры данных могут оказаться намного сложнее в обслуживании, чем их функциональные эквиваленты. Проблемы, описанные выше, можно решить, более тщательно осуществляя проверку на ошибки, и подобные проверки уже реализованы в модулях библиотеки `Coq`, таких как `Doubly_linked`. Всегда, когда это возможно, используйте императивные структуры данных только из проверенных библиотек. А когда это невозможно, уделите особое внимание обработке ошибок.

Итеративные функции

При определении контейнеров, таких как списки, словари и деревья, обычно бывает необходимо реализовать множество итеративных функций, таких как `iter`, `map` и `fold`, которые позволяли бы кратко выражать типичные шаблоны выполнения итераций.

Модуль `Dlist` включает две такие функции: `iter`, цель которой – вызвать функцию, возвращающую `unit` для каждого элемента списка; и `find_el`, которая вызывает указанную функцию проверки для каждого значения, хранящегося в списке, и возвращает первый же элемент, прошедший эту проверку. Обе функции, `iter` и `find_el`, реализованы с использованием простых рекурсивных циклов, использующих указатель `next` для перехода от элемента к элементу и значение `value`, извлеченное из элемента:

OCaml (part 6)

<https://github.com/realworldocaml/examples/blob/v1/code/imperative-programming/dlist.ml>

```
let iter t ~f =
  let rec loop = function
    | None -> ()
    | Some el -> f (value el); loop (next el)
  in
  loop !t

let find_el t ~f =
  let rec loop = function
    | None -> None
    | Some elt ->
      if f (value elt) then Some elt
      else loop (next elt)
  in
  loop !t
```

На этом мы завершаем исследование реализации, но, вообще говоря, чтобы библиотека получила хоть сколько-нибудь значимую практическую ценность, над ней еще работать и работать. Как отмечалось выше, для практических нужд лучше все-таки использовать проверенные библиотеки, такие как модуль `Doubly_linked` в библиотеке `Coq`, который имеет более совершенный интерфейс и осуществляет проверку ошибочных ситуаций. Тем не менее данный пример может служить отличной демонстрацией некоторых приемов по созданию нетривиальных императивных структур данных на языке OCaml, а также некоторых ловушек.

Отложенные вычисления и другие благоприятные эффекты

Существует масса ситуаций, когда предпочтительнее использовать чисто функциональный стиль, но при этом желательно производить побочные эффекты в ограниченном объеме, чтобы повысить производительность кода. Такие эффекты иногда называют *благоприятными*, или *доброкачественными*. Они дают возможность использовать некоторые сильные стороны императивных возможностей OCaml, получая при этом основные преимущества, которые дает функциональное программирование.

Одним из примеров таких благоприятных эффектов являются *отложенные*, или «ленивые», вычисления. Ленивые значения не вычисляются, пока они дей-

ствительно не потребуются. В OCaml ленивые значения создаются с использованием ключевого слова `lazy`, которое можно применять для преобразования выражений произвольного типа `s` в ленивое значение типа `s Lazy.t`. Вычисление такого выражения будет отложено до вызова `Lazy.force`:

OCaml utop (part 1)

<https://github.com/realworldocaml/examples/blob/v1/code/imperative-programming/lazy.topscript>

```
# let v = lazy (print_string "performing lazy computation\n"; sqrt 16.);;
val v : float lazy_t = <lazy>
# Lazy.force v;;
performing lazy computation
- : float = 4.
# Lazy.force v;;
- : float = 4.
```

Как видно из вывода в примере выше, фактическое вычисление значения выражения осуществляется только один раз и только после вызова `force`.

Чтобы лучше разобраться, как осуществляются отложенные вычисления, давайте пройдемся по реализации нашего ленивого типа. Начнем с объявления типов, представляющих ленивое значение:

OCaml utop (part 2)

<https://github.com/realworldocaml/examples/blob/v1/code/imperative-programming/lazy.topscript>

```
# type 'a lazy_state =
  | Delayed of (unit -> 'a)
  | Value of 'a
  | Exn of exn
;;
type 'a lazy_state = Delayed of (unit -> 'a) | Value of 'a | Exn of exn
```

Тип `lazy_state` представляет возможные состояния ленивого значения. До того, как ленивое значение будет вычислено, оно имеет состояние `Delayed`, где `Delayed` хранит функцию, вычисляющую фактическое значение. Когда вызов `force` завершится благополучно, ленивое значение переходит в состояние `Value`. Состояние `Exn` наступает, когда вызов `force` завершился исключением. Собственно, ленивое значение — это просто ссылка `ref`, содержащая значение типа `lazy_state`, при этом ссылка `ref` может переходить из начального состояния `Delayed` в состояние `Value` или `Exn`.

Ленивое значение можно создать и с помощью переходника (`think`) — функции, принимающей аргумент типа `unit`. Это еще один способ отложить вычисление выражения:

OCaml utop (part 3)

<https://github.com/realworldocaml/examples/blob/v1/code/imperative-programming/lazy.topscript>

```
# let create_lazy f = ref (Delayed f);;
val create_lazy : (unit -> 'a) -> 'a lazy_state ref = <fun>
```



```
# let v = create_lazy
  (fun () -> print_string "performing lazy computation\n"; sqrt 16.);;
val v : float lazy_state ref = {contents = Delayed <fun>}
```

Теперь нам нужен способ принудительного вычисления ленивого значения. Именно этот способ дает следующая функция:

OCaml utop (part 4)

<https://github.com/realworldocaml/examples/blob/v1/code/imperative-programming/lazy.topscript>

```
# let force v =
  match !v with
  | Value x -> x
  | Exn e -> raise e
  | Delayed f ->
    try
      let x = f () in
      v := Value x;
      x
    with exn ->
      v := Exn exn;
      raise exn
;;
val force : 'a lazy_state ref -> 'a = <fun>
```

которую можно использовать по аналогии с `Lazy.force`:

OCaml utop (part 5)

<https://github.com/realworldocaml/examples/blob/v1/code/imperative-programming/lazy.topscript>

```
# force v;;
performing lazy computation
- : float = 4.
# force v;;
- : float = 4.
```

Единственным отличием между нашей и встроенной реализацией ленивых значений, видимым пользователю, является синтаксис. Вместо `create_lazy (fun () -> sqrt 16.)` то же самое можно записать (с использованием встроенного типа `lazy`) более кратко: `lazy (sqrt 16.)`.

Мемоизация и динамическое программирование

Еще один доброкачественный эффект — *мемоизация* (memoization). Мемоизованная функция запоминает результат предыдущего вызова, благодаря чему получает возможность вернуть его сразу же, не выполняя вычислений, когда будет вызвана в следующий раз с тем же набором аргументов.

Ниже приводится функция, принимающая произвольную функцию одного аргумента и возвращающая мемоизованную версию этой функции. В данном примере мы использовали модуль `Hashtbl` из библиотеки `Core` вместо нашего игрушечного словаря:

OCaml utop (part 1)

<https://github.com/realworldocaml/examples/blob/v1/code/imperative-programming/memo.topscript>

```
# let memoize f =
  let table = Hashtbl.Poly.create () in
  (fun x ->
    match Hashtbl.find table x with
    | Some y -> y
    | None ->
      let y = f x in
      Hashtbl.add_exn table ~key:x ~data:y;
      y
  );;
val memoize : ('a -> 'b) -> 'a -> 'b = <fun>
```

Предыдущий код довольно сложен. Функция `memoize` принимает в аргументе функцию `f`, создает хэш-таблицу (с именем `table`) и возвращает новую функцию – мемоизованную версию функции `f`. При вызове эта новая функция сначала выполняет поиск аргумента в `table`. Если поиск завершился неудачей, она вызывает `f` и сохраняет результат в `table`. Обратите внимание, что `table` остается в области видимости, пока в области видимости остается функция, которую вернула `memoize`.

Мемоизация с успехом может применяться к функциям, выполняющим дорогостоящие вычисления, и вы не против хранения старых значений в кэше неопределенное время. Важно помнить, что мемоизованная функция по своей природе является источником утечек памяти. Пока программа хранит мемоизованную версию функции, она будет также хранить и все результаты ее вызовов.

Мемоизация может также пригодиться для повышения эффективности некоторых рекурсивных алгоритмов. Отличным примером может служить алгоритм вычисления *расстояния редактирования* (также называется *расстоянием Левенштейна*) между двумя строками. Расстояние редактирования – это число односимвольных изменений (включая перемену букв местами, вставку и удаление), необходимых, чтобы одну строку преобразовать в другую. Метрика такого рода может пригодиться для решения различных задач аппроксимации строк, таких как проверка правописания.

Взгляните на следующий код, вычисляющий расстояние редактирования. Понимание алгоритма в данном случае не так важно, но уделите особое внимание структуре рекурсивных вызовов:

OCaml utop (part 2)

<https://github.com/realworldocaml/examples/blob/v1/code/imperative-programming/memo.topscript>

```
# let rec edit_distance s t =
  match String.length s, String.length t with
  | (0,x) | (x,0) -> x
  | (len_s,len_t) ->
    let s' = String.drop_suffix s 1 in
    let t' = String.drop_suffix t 1 in
    let cost_to_drop_both =
```

```

    if s.[len_s - 1] = t.[len_t - 1] then 0 else 1
  in
    List.reduce_exn ~f:Int.min
      [ edit_distance s' t + 1
        ; edit_distance s t' + 1
        ; edit_distance s' t' + cost_to_drop_both
      ]
;;
val edit_distance : string -> string -> int = <fun>
# edit_distance "OCaml" "ocaml";;
- : int = 2

```

Вызов `edit_distance "OCaml" "ocaml"` произведет следующую последовательность рекурсивных вызовов:

Diagram

https://github.com/realworldocaml/examples/blob/v1/code/imperative-programming/edit_distance.asci

```

edit_distance "OCam" "ocaml"
edit_distance "OCaml" "ocam"
edit_distance "OCam" "ocam"

```

А эти вызовы, в свою очередь, произведут следующую последовательность рекурсивных вызовов:

Diagram

https://github.com/realworldocaml/examples/blob/v1/code/imperative-programming/edit_distance2.asci

```

edit_distance "OCam" "ocaml"
  edit_distance "OCa" "ocaml"
    edit_distance "OCam" "ocam"
      edit_distance "OCa" "ocam"
edit_distance "OCaml" "ocam"
  edit_distance "OCam" "ocam"
    edit_distance "OCaml" "oca"
      edit_distance "OCam" "oca"
edit_distance "OCam" "ocam"
  edit_distance "OCa" "ocam"
    edit_distance "OCam" "oca"
      edit_distance "OCa" "oca"

```

Как видите, некоторые вызовы повторяются. Например, выполняются два вызова `edit_distance "OCam" "oca"`. Число избыточных вызовов растет экспоненциально с ростом размеров строк, а это означает, что наша реализация `edit_distance` будет дико «тормозить» на длинных строках. В этом легко убедиться, написав маленькую функцию для хронометража:

OCaml utop (part 3)

<https://github.com/realworldocaml/examples/tree/v1/code/imperative-programming/memo.topscript>

```

# let time f =
  let start = Time.now () in

```

```

let x = f () in
let stop = Time.now () in
printf "Time: %s\n" (Time.Span.to_string (Time.diff stop start));
x ;;
val time : (unit -> 'a) -> 'a = <fun>

```

А теперь измерим производительность на нескольких примерах:

OCaml utop (part 4)

<https://github.com/realworldocaml/examples/tree/v1/code/imperative-programming/memo.topscript>

```

# time (fun () -> edit_distance "OCaml" "ocaml");;
Time: 1.40405ms
- : int = 2
# time (fun () -> edit_distance "OCaml 4.01" "ocaml 4.01");;
Time: 6.79065s
- : int = 2

```

При добавлении всего нескольких символов производительность падает в тысячи раз!

Мемоизация здесь могла бы быть как нельзя кстати, но, чтобы исправить проблему, необходимо мемоизовать вызовы, которые делает `edit_distance` сама. Этот прием иногда называют *динамическим программированием* (dynamic programming). Чтобы увидеть его в действии, давайте оставим пока `edit_distance` и рассмотрим более простой пример: поиск n -го элемента последовательности Фибоначчи. По определению, последовательность Фибоначчи начинается с двух единиц, а каждый следующий элемент является суммой двух предыдущих. Ниже приводится классическая рекурсивная реализация определения чисел Фибоначчи:

OCaml utop (part 1)

<https://github.com/realworldocaml/examples/tree/v1/code/imperative-programming/fib.topscript>

```

# let rec fib i =
  if i <= 1 then 1 else fib (i - 1) + fib (i - 2);;
val fib : int -> int = <fun>

```

Однако эта реализация проявляет экспоненциальное падение производительности по той же причине, что и функция `edit_distance`: выполняется слишком большое число избыточных вызовов `fib`. Это катастрофически сказывается на производительности:

OCaml utop (part 2)

<https://github.com/realworldocaml/examples/blob/v1/code/imperative-programming/fib.topscript>

```

# time (fun () -> fib 20);;
Time: 0.844955ms
- : int = 10946
# time (fun () -> fib 40);;
Time: 12.7751s
- : int = 165580141

```

Как видите, вызов `fib 40` выполняется в тысячи раз дольше вызова `fib 20`.

Так как же прием мемоизации может улучшить ситуацию? Вся хитрость в том, чтобы выполнить мемоизацию перед рекурсивными вызовами `fib`. Мы не можем просто определить `fib` как обычно, мемоизовать ее после этого и ожидать улучшения производительности с первого же вызова:

OCaml utop (part 3)

<https://github.com/realworldocaml/examples/blob/v1/code/imperative-programming/fib.topscript>

```
# let fib = memoize fib;;
val fib : int -> int = <fun>
# time (fun () -> fib 40);;
Time: 12.774s
- : int = 165580141
# time (fun () -> fib 40);;
Time: 0.00309944ms
- : int = 165580141
```

Чтобы ускорить работу `fib`, сначала нужно переписать функцию, реализовав алгоритм раскручивания рекурсии. Следующая версия принимает в первом аргументе (с именем `fib`) функцию, которая будет вызываться вместо обычного рекурсивного вызова:

OCaml utop (part 4)

<https://github.com/realworldocaml/examples/blob/v1/code/imperative-programming/fib.topscript>

```
# let fib_norec fib i =
  if i <= 1 then i
  else fib (i - 1) + fib (i - 2) ;;
val fib_norec : (int -> int) -> int -> int = <fun>
```

Теперь мы можем превратить ее обратно, в обычную функцию вычисления чисел Фибоначчи, замкнув рекурсивную петлю:

OCaml utop (part 5)

<https://github.com/realworldocaml/examples/blob/v1/code/imperative-programming/fib.topscript>

```
# let rec fib i = fib_norec fib i;;
val fib : int -> int = <fun>
# fib 20;;
- : int = 6765
```

Можно даже написать полиморфную функцию, назовем ее `make_rec`, которая будет заключать в рекурсивную петлю любую функцию:

OCaml utop (part 6)

<https://github.com/realworldocaml/examples/blob/v1/code/imperative-programming/fib.topscript>

```
# let make_rec f_norec =
  let rec f x = f_norec f x in
```

```

    f
  ;;
val make_rec : (('a -> 'b) -> 'a -> 'b) -> 'a -> 'b = <fun>
# let fib = make_rec fib_norec;;
val fib : int -> int = <fun>
# fib 20;;
- : int = 6765

```

Это не самая очевидная реализация, и может потребоваться некоторое время, чтобы понять, как она действует. Как и `fib_norec`, функция `f_norec`, что передается в вызов `make_rec`, является нерекурсивной, но она принимает в качестве аргумента функцию, которую должна вызвать. По сути, `make_rec` просто передает `f_norec` самой себе, создавая настоящую рекурсивную функцию.

Довольно необычный ход, но мы всего лишь нашли другой путь реализовать ту же самую медлительную функцию вычисления чисел Фибоначчи. Чтобы заставить ее работать быстрее, необходима такая версия `make_rec`, которая будет выполнять мемоизацию при замыкании рекурсивной петли. Назовем эту функцию `memo_rec`:

OCaml utop (part 7)

<https://github.com/realworldocaml/examples/blob/v1/code/imperative-programming/fib.topscript>

```

# let memo_rec f_norec x =
  let fref = ref (fun _ -> assert false) in
  let f = memoize (fun x -> f_norec !fref x) in
  fref := f;
  f x
;;
val memo_rec : (('a -> 'b) -> 'a -> 'b) -> 'a -> 'b = <fun>

```

Обратите внимание, что `memo_rec` имеет ту же сигнатуру, что и `make_rec`.

Здесь для образования рекурсивной петли вместо `let rec` используется ссылка, потому конструкция `let rec` в данном случае не будет работать по причинам, которые будут описаны позже.

С помощью `memo_rec` мы можем теперь создать эффективную версию `fib`:

OCaml utop (part 8)

<https://github.com/realworldocaml/examples/tree/v1/code/imperative-programming/fib.topscript>

```

# let fib = memo_rec fib_norec;;
val fib : int -> int = <fun>
# time (fun () -> fib 40);;
Time: 0.0591278ms
- : int = 102334155

```

И, как видите, катастрофическое падение производительности осталось в прошлом.

Здесь важное значение имеет особенность использования памяти. Если внимательно взглянуть на определение `memo_rec`, можно подметить, что вызов `memo_rec`

`fib_norec` не приводит к вызову функции `memoize`. Этот вызов выполняется, только когда вызывается сама функция `fib` и, соответственно, функции `memo_rec` передается последний ее аргумент. Результат этого вызова выпадает из области видимости после возврата из `fib`, благодаря чему вызов `memo_rec` для функции не создает утечку памяти – таблица мемоизации утилизируется по завершении вычислений.

Мы можем использовать `memo_rec` как часть общего объявления, которое будет выглядеть чуть сложнее, чем специальная форма `let rec`:

OCaml utop (part 9)

<https://github.com/realworldocaml/examples/tree/v1/code/imperative-programming/fib.topscript>

```
# let fib = memo_rec (fun fib i ->
  if i <= 1 then 1 else fib (i - 1) + fib (i - 2));;
val fib : int -> int = <fun>
```

Прием мемоизации избыточен для реализации алгоритма Фибоначчи, и функция `fib`, определенная выше, в действительности не самая эффективная – она потребляет память прямо пропорционально значению аргумента. Ее легко можно было бы переписать так, чтобы она расходовала постоянный объем памяти.

Но мемоизация отлично подходит для оптимизации `edit_distance`, и мы можем применить к ней тот же подход, что и к функции `fib`. Для этого необходимо изменить `edit_distance` так, чтобы она принимала пару строк в единственном аргументе, потому что `memo_rec` работает только с функциями одного аргумента. (Мы всегда можем вернуться к первоначальному интерфейсу с помощью функции-обертки.) Благодаря этому простому изменению и дополнительному вызову `memo_rec` мы можем получить мемоизованную версию `edit_distance`:

OCaml utop (part 6)

<https://github.com/realworldocaml/examples/tree/v1/code/imperative-programming/memo.topscript>

```
# let edit_distance = memo_rec (fun edit_distance (s,t) ->
  match String.length s, String.length t with
  | (0,x) | (x,0) -> x
  | (len_s,len_t) ->
    let s' = String.drop_suffix s 1 in
    let t' = String.drop_suffix t 1 in
    let cost_to_drop_both =
      if s.[len_s - 1] = t.[len_t - 1] then 0 else 1
    in
    List.reduce_exn ~f:Int.min
      [ edit_distance (s',t) + 1
      ; edit_distance (s ,t') + 1
      ; edit_distance (s',t') + cost_to_drop_both
      ] ;;
val edit_distance : string * string -> int = <fun>
```

Эта новая версия `edit_distance` намного эффективнее первоначальной; следующий вызов выполняется во много тысяч раз быстрее, чем вызов немемоизованной версии:

OCaml utop (part 7)

<https://github.com/realworldocaml/examples/tree/v1/code/imperative-programming/memo.topscript>

```
# time (fun () -> edit_distance ("OCaml 4.01", "ocaml 4.01")); ;
Time: 0.500917ms
- : int = 2
```

Ограничения let rec

Возможно, вас волнует, почему мы не использовали в функции `memo_rec` конструкцию `let rec` для замыкания рекурсивной петли. Взгляните на следующий код, где как раз предпринимается такая попытка:

OCaml utop (part 1)

<https://github.com/realworldocaml/examples/tree/v1/code/imperative-programming/letrec.topscript>

```
# let memo_rec f_norec =
  let rec f = memoize (fun x -> f_norec f x) in
    f
;;
```

Characters 39-69:

Error: This kind of expression is not allowed as right-hand side of 'let rec'
(Ошибка: Выражение такого рода недопустимо справа от 'let rec')

Компилятор отверг определение, потому что OCaml как строгий язык накладывает определенные ограничения на выражения, которые могут появляться справа от `let rec`. В частности, представьте, как был бы скомпилирован следующий фрагмент:

OCaml

https://github.com/realworldocaml/examples/tree/v1/code/imperative-programming/let_rec.ml

```
let rec x = x + 1
```

Обратите внимание, что `x` – это обычное значение, а не функция. Здесь не совсем ясно, как должно обрабатываться это определение компилятором. Можно было бы подумать, что компилятор попадет в бесконечный цикл, но `x` имеет тип `int`, и нет такого типа `int`, которому соответствовал бы бесконечный цикл. Получается, что эту конструкцию нельзя скомпилировать.

Чтобы избежать подобных конфликтов, компилятор позволяет использовать только три конструкции справа от `let rec`: определение функции, вызов конструктора и ключевое слово `lazy`. Это ограничение исключает возможность использования таких вполне законных конструкций, как определение нашей функции `memo_rec`, но также блокирует совершенно бессмысленные конструкции, как определение `x` в примере выше.

Следует отметить, что подобные ограничения отсутствуют в языках, широко использующих отложенные вычисления, таких как Haskell. В действительности мы можем заставить работать определение `x` и в OCaml, задействовав механизм отложенных вычислений:

OCaml utop (part 2)

<https://github.com/realworldocaml/examples/tree/v1/code/imperative-programming/letrec.topscript>

```
# let rec x = lazy (Lazy.force x + 1); ;
val x : int lazy_t = <lazy>
```

Безусловно, попытка вычислить это выражение потерпит неудачу. Ключевое слово `lazy` возбудит исключение, когда «ленивое» значение потребует с помощью `force` вычислить себя в процессе вычисления всего выражения.

OCaml utop (part 3)

<https://github.com/realworldocaml/examples/tree/v1/code/imperative-programming/letrec.topscript>

```
# Lazy.force x;;
Exception: Lazy.Undefined.
```

Однако с помощью такого приема можно создавать практичные рекурсивные функции. Например, механизм отложенных вычислений можно использовать для определения версии `memo_rec`, не изменяющей данных в процессе работы:

OCaml utop (part 5)

<https://github.com/realworldocaml/examples/tree/v1/code/imperative-programming/letrec.topscript>

```
# let lazy_memo_rec f_norec x =
  let rec f = lazy (memoize (fun x -> f_norec (Lazy.force f) x)) in
  (Lazy.force f) x
;;
val lazy_memo_rec : (('a -> 'b) -> 'a -> 'b) -> 'a -> 'b = <fun>
# time (fun () -> lazy_memo_rec fib_norec 40);;
Time: 0.0650883ms
- : int = 102334155
```

Применение отложенных вычислений накладывает еще более строгие ограничения, чем применение изменяемых значений, что позволяет писать более ясный и предсказуемый код.

Ввод и вывод

Императивное программирование – это не только изменение структур данных в памяти. Любая функция, задача которой не сводится к детерминистскому преобразованию аргументов в возвращаемое значение, по своей природе является императивной. Под это определение попадают не только функции, изменяющие данные программы, но также взаимодействующие с внешним миром. Важным примером таких взаимодействий являются операции ввода/вывода, то есть операции чтения/записи с файлами, терминалом и сетевыми сокетами.

Для языка OCaml существует множество библиотек ввода/вывода. В этом разделе мы познакомимся с библиотекой буферизованного ввода/вывода, которую можно использовать через модули `In_channel` и `Out_channel` в библиотеке `Core`. Библиотекой `Core` поддерживаются и другие примитивы ввода/вывода, например посредством модуля `Unix`, а также асинхронный ввод/вывод посредством модуля `Async`, о котором рассказывается в главе 18. Большая часть функциональности, предоставляемой модулями `In_channel` и `Out_channel` (и модулем `Unix`), унаследована из стандартной библиотеки, но в этом разделе мы будем использовать только интерфейсы `Core`.

Терминальный ввод/вывод

Библиотека ввода/вывода в языке OCaml основана на двух типах: `in_channel` (каналы, используемые для чтения) и `out_channel` (каналы, используемые для записи). Модули `In_channel` и `Out_channel` поддерживают только каналы, непосредственно

соответствующие файлам и терминалам; остальные разновидности каналов можно создавать с помощью модуля `Unix`.

Наше обсуждение операций ввода/вывода мы начнем с терминалов. В соответствии с моделью UNIX взаимодействия с терминалами осуществляются посредством трех каналов, которые связаны с тремя стандартными в Unix дескрипторами файлов:

- `In_channel.stdin` – канал «стандартного ввода». По умолчанию данные в этот канал поступают с клавиатуры;
- `Out_channel.stdout` – канал «стандартного вывода». По умолчанию данные, записанные в этот канал, выводятся на экран терминала;
- `Out_channel.stderr` – канал «стандартного вывода ошибок». Действует подобно каналу `stdout`, но предназначен исключительно для вывода сообщений об ошибках.

Значения `stdin`, `stdout` и `stderr` имеют настолько большую практическую ценность, что они сделаны доступными в глобальном пространстве имен, и к ним можно обращаться непосредственно, без использования префиксов `In_channel` и `Out_channel`.

Давайте посмотрим, как они действуют, на примере простого интерактивного приложения. Следующая программа, `time_converter`, запрашивает у пользователя часовой пояс и выводит текущее время в этом часовом поясе. Она определяет часовой пояс с помощью модуля `Zone` из библиотеки `Core`, затем находит поясное время и выводит его:

OCaml

https://github.com/realworldocaml/examples/tree/v1/code/imperative-programming/time_converter.ml

```
open Core.Std
```

```
let () =
  Out_channel.output_string stdout "Pick a timezone: ";
  Out_channel.flush stdout;
  match In_channel.input_line stdin with
  | None -> failwith "No timezone provided"
  | Some zone_string ->
    let zone = Zone.find_exn zone_string in
    let time_string = Time.to_string_abs (Time.now ()) ~zone in
    Out_channel.output_string stdout
      (String.concat
        ["The time in "; Zone.to_string zone; " is "; time_string; "\n"]);
    Out_channel.flush stdout
```

Ниже показано, как скомпилировать эту программу с помощью `corebuild` и запустить ее. После запуска вы увидите приглашение к вводу:

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/imperative-programming/time_converter.out

```
$ corebuild time_converter.byte
$ ./time_converter.byte
Pick a timezone:
```

Теперь можно ввести название часового пояса и нажать клавишу **Return**, а программа выведет текущее время в этом часовом поясе:

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/imperative-programming/time_converter2.out

```
Pick a timezone: Europe/London
```

```
The time in Europe/London is 2013-08-15 00:03:10.666220+01:00.
```

Мы вызвали `Out_channel.flush` для канала `stdout`, чтобы принудительно вытолкнуть его содержимое на экран – каналы типа `out_channel` буферизуются, поэтому их содержимое не выводится при каждом вызове `output_string`. Данные просто будут записываться в буфер, пока в нем достаточно места или пока явно не будет вызвана функция `flush`. Эта особенность значительно увеличивает эффективность процесса вывода за счет уменьшения числа системных вызовов.

Обратите внимание, что `In_channel.input_line` возвращает значение типа `string option`, где `None` указывает, что достигнут конец потока ввода (то есть выполнено условие достижения конца файла). Вызов `Out_channel.output_string` выводит строку результата в буфер, а `Out_channel.flush` выталкивает содержимое буфера на экран. С технической точки зрения заключительный вызов `flush` не требуется, потому что после него программа сразу же завершается и выталкивание всех буферов вывода происходит автоматически, но явное выражение своих намерений – хорошая практика.

Форматированный вывод с помощью printf

Организация вывода с помощью таких функций, как `Out_channel.output_string`, проста и понятна, но иногда она может быть слишком многословна. Поэтому для форматированного вывода часто используется функция `printf`, которая моделирует поведение одноименной функции `printf` из стандартной библиотеки языка C. Функция `printf` принимает *строку формата*, описывающую выводимые значения и их формат, а также аргументы для вывода, соответствующие директивам форматирования в строке формата. Так, например, мы можем написать:

OCaml utop (part 1)

<https://github.com/realworldocaml/examples/tree/v1/code/imperative-programming/printf.topscript>

```
# printf "%i is an integer, %F is a float, \"%s\" is a string\n"
  3 4.5 "five";;
3 is an integer, 4.5 is a float, "five" is a string
- : unit = ()
```

В отличие от своего аналога в языке C, функция `printf` в языке OCaml осуществляет проверку типов аргументов. Например, если передать ей аргумент, тип которого не соответствует спецификатору формата, компилятор выведет сообщение об ошибке:

OCaml utop (part 2)

<https://github.com/realworldocaml/examples/tree/v1/code/imperative-programming/printf.topscript>

```
# printf "An integer: %i\n" 4.5;;
```

Characters 26-29:

```
Error: This expression has type float but an expression was expected of type
      int
```

(Ошибка: Это выражение имеет тип float, тогда как ожидалось выражение типа int)

О строках формата

Как оказывается, строки формата, используемые с функцией `printf`, в корне отличаются от обычных строк. Это различие объясняется поддержкой контроля типов в строках формата, чего нельзя сказать о строках формата в языке C. В частности, компилятор проверяет, значения каких типов описываются строкой формата, и сопоставляет их с типами последующих аргументов функции `printf`.

Для этого содержимое строки формата должно быть проанализировано на этапе компиляции, а это, в свою очередь, означает, что строка формата может быть только литералом. И действительно, если попробовать передать функции `printf` обычную строку, компилятор сообщит об ошибке:

OCaml utop (part 3)

<https://github.com/realworldocaml/examples/tree/v1/code/imperative-programming/printf.topscript>

```
# let fmt = "%i is an integer, %F is a float, \"%s\" is a string\n";;
val fmt : string = "%i is an integer, %F is a float, \"%s\" is a string\n"
# printf fmt 3 4.5 "five";;
```

Characters 9-12:

```
Error: This expression has type string but an expression was expected of type
('a -> 'b -> 'c -> 'd, out_channel, unit) format =
  ('a -> 'b -> 'c -> 'd, out_channel, unit, unit, unit, unit)
  format6
```

(Ошибка: Это выражение имеет тип string, тогда как ожидалось выражение типа ('a -> 'b -> 'c -> 'd, out_channel, unit) format = ('a -> 'b -> 'c -> 'd, out_channel, unit, unit, unit, unit) format6)

Если компилятор обнаружит, что литеральная строка является строкой формата, он проанализирует ее на этапе компиляции и сохранит результаты анализа для сопоставления типов аргументов с найденными директивами форматирования. То есть если добавить в определение строки формата аннотацию, указывающую, что она действительно является строкой формата, компилятор будет интерпретировать ее как строку формата:

OCaml utop (part 4)

<https://github.com/realworldocaml/examples/tree/v1/code/imperative-programming/printf.topscript>

```
# let fmt : ('a, 'b, 'c) format =
  "%i is an integer, %F is a float, \"%s\" is a string\n";;
val fmt : (int -> float -> string -> 'c, 'b, 'c) format = <abstr>
```

И мы сможем передать ее функции `printf`:

OCaml utop (part 5)

<https://github.com/realworldocaml/examples/tree/v1/code/imperative-programming/printf.topscrip>

```
# printf fmt 3 4.5 "five";;
3 is an integer, 4.5 is a float, "five" is a string - : unit = ()
```

Если вам показалось, что строки формата отличаются от всего, что вы видели до сих пор, вы не ошиблись. Строки формата действительно являются особым случаем в системе типов. В большинстве случаев вам не придется беспокоиться об особенностях обработки строк формата – вы можете просто использовать `printf`, не задумываясь о тонкостях. Но дополнительные сведения об этих самых тонкостях вам не помешают.

Теперь посмотрим, как с помощью функции `printf` можно немного сократить программу вывода поясного времени:

OCaml

https://github.com/realworldocaml/examples/tree/v1/code/imperative-programming/time_converter2.ml

open Core.Std

```
let () =
  printf "Pick a timezone: %!";
  match In_channel.input_line stdin with
  | None -> failwith "No timezone provided"
  | Some zone_string ->
    let zone = Zone.find_exn zone_string in
    let time_string = Time.to_string_abs (Time.now ()) ~zone in
    printf "The time in %s is %s.\n%" (Zone.to_string zone) time_string
```

Здесь используются только две директивы форматирования: `%s`, включающая строку, и `%!` , заставляющая функцию `printf` вытолкнуть буфер вывода.

Директивы форматирования, поддерживаемые функцией `printf`, позволяют определять:

- выравнивание и дополнение;
- порядок экранирования специальных символов в строках;
- систему счисления для чисел (десятичную, шестнадцатеричную и двоичную);
- точность представления вещественных чисел.

Существуют также другие `printf`-подобные функции, осуществляющие вывод в иные устройства, отличные от `stdout`:

- `eprintf`, осуществляет вывод в `stderr`;
- `fprintf`, осуществляет вывод в произвольный `out_channel`;
- `sprintf`, возвращает отформатированную строку.

Эту и другую информацию можно найти в документации с описанием API модуля `Printf`, в руководстве по языку OCaml.

Файловый ввод/вывод

Каналы типа `in_channel` и `out_channel` также часто используются для работы с файлами. Ниже демонстрируется пара функций, одна из которых создает файл и заполняет его числами, а вторая читает этот файл и возвращает сумму чисел:

OCaml utop (part 1)

<https://github.com/realworldocaml/examples/tree/v1/code/imperative-programming/file.topscript>

```
# let create_number_file filename numbers =
  let outc = Out_channel.create filename in
  List.iter numbers ~f:(fun x -> fprintf outc "%d\n" x);
  Out_channel.close outc
;;

val create_number_file : string -> int list -> unit = <fun>
# let sum_file filename =
  let file = In_channel.create filename in
  let numbers = List.map ~f:Int.of_string (In_channel.input_lines file) in
  let sum = List.fold ~init:0 ~f:(+) numbers in
  In_channel.close file;
  sum
;;

val sum_file : string -> int = <fun>
# create_number_file "numbers.txt" [1;2;3;4;5];;
- : unit = ()
# sum_file "numbers.txt";;
- : int = 15
```

В обеих функциях используется один и тот же шаблон: сначала создается канал, затем выполняются операции с этим каналом, после чего канал закрывается. Закрывание канала в этом примере играет важную роль, потому что иначе программа не вернет ставшие ненужными ресурсы обратно операционной системе.

Предыдущая реализация имеет одну проблему – если где-то на полпути возникнет исключение, она не закроет файл. Например, если попробовать прочитать файл, содержащий не только числа, мы увидим такую ошибку:

OCaml utop (part 2)

<https://github.com/realworldocaml/examples/tree/v1/code/imperative-programming/file.topscript>

```
# sum_file "/etc/hosts";;
Exception: (Failure "Int.of_string: \"127.0.0.1 localhost\"").
```

А если ошибочная ситуация будет повторяться в цикле снова и снова, в конечном счете программа может исчерпать все доступные в системе дескрипторы:

OCaml utop (part 3)

<https://github.com/realworldocaml/examples/tree/v1/code/imperative-programming/file.topscript>

```
# for i = 1 to 10000 do try ignore (sum_file "/etc/hosts") with _ -> () done;;
- : unit = ()
# sum_file "numbers.txt";;
Exception: (Sys_error "numbers.txt: Too many open files").
```

Теперь вам придется перезапустить интерактивный сеанс, чтобы получить возможность попробовать другие файлы!

Чтобы избавиться от этой проблемы, следует позаботиться об уборке мусора за собой. Сделать это можно с помощью функции `protect`, описанной в главе 7:

OCaml utop (part 1)

<https://github.com/realworldocaml/examples/tree/v1/code/imperative-programming/file2.topscrip>

```
# let sum_file filename =
  let file = In_channel.create filename in
  protect ~f:(fun () ->
    let numbers = List.map ~f:Int.of_string (In_channel.input_lines file) in
    List.fold ~init:0 ~f:(+) numbers)
  ~finally:(fun () -> In_channel.close file)
;;
val sum_file : string -> int = <fun>
```

Теперь утечка дескрипторов файлов устранена:

OCaml utop (part 2)

<https://github.com/realworldocaml/examples/blob/v1/code/imperative-programming/file2.topscrip>

```
# for i = 1 to 10000 do try ignore (sum_file "/etc/hosts") with _ -> () done;;
- : unit = ()
# sum_file "numbers.txt";;
- : int = 15
```

В действительности это был пример более общей проблемы императивного программирования. Программируя в императивном стиле, вы вынуждены постоянно следить за тем, чтобы исключения не оставили программу в противоречивом состоянии.

В модуле `In_channel` имеются функции, позволяющие автоматизировать обработку этих тонкостей. Например, функция `In_channel.with_file` принимает имя файла и функцию для обработки его содержимого, при этом она сама позаботится об открытии и своевременном закрытии файла. Если задействовать эту функцию, тогда пример `sum_file` можно переписать так:

OCaml utop (part 3)

<https://github.com/realworldocaml/examples/blob/v1/code/imperative-programming/file2.topscrip>

```
# let sum_file filename =
  In_channel.with_file filename ~f:(fun file ->
    let numbers = List.map ~f:Int.of_string (In_channel.input_lines file) in
    List.fold ~init:0 ~f:(+) numbers)
;;
val sum_file : string -> int = <fun>
```

Еще одна неприятность, кроющаяся в нашей реализации `sum_file`, заключается в том, что функция сначала считывает файл целиком в память и лишь затем обрабатывает его. Для очень больших файлов эффективнее было бы обрабатывать их построчно. Для этого можно воспользоваться функцией `In_channel.fold_lines`:

OCaml utop (part 4)

<https://github.com/realworldocaml/examples/tree/v1/code/imperative-programming/file2.topscrip>

```
# let sum_file filename =
  In_channel.with_file filename ~f:(fun file ->
```

```
In_channel.fold_lines file ~init:0 ~f:(fun sum line ->
  sum + Int.of_string line))
;;
val sum_file : string -> int = <fun>
```

Мы рассмотрели лишь малую толику возможностей модулей `In_channel` и `Out_channel`. Чтобы получить более полное представление, обращайтесь к документации с описанием API этих модулей.

Порядок вычислений

Порядок вычисления выражений является важной частью определения языка программирования, и это особенно важно при использовании императивного стиля программирования. Большинство языков программирования, с которыми вам наверняка приходилось сталкиваться, являются *строгими* в этом отношении, как и язык OCaml. Когда в строгом языке производится связывание идентификатора с результатом некоторой операции, сначала вычисляется выражение, и только потом его значение связывается с переменной. Аналогично, когда вызывается функция со множеством аргументов, эти аргументы вычисляются до передачи их функции.

Рассмотрим простой пример. У нас имеется несколько углов, и нам требуется определить, есть ли среди них хоть один с отрицательным значением синуса. Ответ на этот вопрос дает следующий фрагмент кода:

OCaml utop (part 1)

<https://github.com/realworldocaml/examples/blob/v1/code/imperative-programming/order.topscript>

```
# let x = sin 120. in
  let y = sin 75. in
  let z = sin 128. in
  List.exists ~f:(fun x -> x < 0.) [x;y;z]
;;
- : bool = true
```

По большому счету, здесь не нужно вычислять `sin 128.`, потому что `sin 75.` возвращает отрицательное значение, и уже на этом этапе можно ответить на поставленный вопрос утвердительно, не вычисляя `sin 128.`

Следовательно, вычисления нужно организовать как-то иначе. Воспользовавшись ключевым словом `lazy`, можно переписать первоначальный пример так, что `sin 128.` вообще не будет вычисляться:

OCaml utop (part 2)

<https://github.com/realworldocaml/examples/blob/v1/code/imperative-programming/order.topscript>

```
# let x = lazy (sin 120.) in
  let y = lazy (sin 75.) in
  let z = lazy (sin 128.) in
  List.exists ~f:(fun x -> Lazy.force x < 0.) [x;y;z]
;;
- : bool = true
```


Убедиться в этом можно, добавив вызовы функции `printf`:

OCaml utop (part 3)

<https://github.com/realworldocaml/examples/blob/v1/code/imperative-programming/order.topscript>

```
# let x = lazy (printf "1\n"; sin 120.) in
  let y = lazy (printf "2\n"; sin 75.) in
  let z = lazy (printf "3\n"; sin 128.) in
  List.exists ~f:(fun x -> Lazy.force x < 0.) [x;y;z]
;;
1
2
- : bool = true
```

OCaml – строгий язык, и тому есть веские основания: отложенные вычисления и императивный стиль программирования обычно плохо смешиваются, потому что применение механизма отложенных вычислений мешает понять, когда должен произойти тот или иной побочный эффект. Знание порядка следования побочных эффектов является основой рассуждений о поведении императивной программы.

В строгом языке точно известно, что выражения в привязках `let`, следующих друг за другом, будут вычислены в том порядке, в каком они следуют в программном коде. Но можно ли утверждать то же самое относительно порядка вычислений в рамках одного выражения? Формально порядок вычислений внутри выражения не определен. Но на практике имеется только один компилятор OCaml, и поэтому его поведение является стандартом де-факто. К сожалению, порядок вычислений в данном случае часто оказывается полностью противоположным в сравнении с тем, что можно было бы ожидать.

Взгляните на следующий пример:

OCaml utop (part 4)

<https://github.com/realworldocaml/examples/blob/v1/code/imperative-programming/order.topscript>

```
# List.exists ~f:(fun x -> x < 0.)
  [ (printf "1\n"; sin 120.);
    (printf "2\n"; sin 75.);
    (printf "3\n"; sin 128.); ]
;;
3
2
1
- : bool = true
```

Здесь можно видеть, что подвыражения, указанные последними, в действительности вычисляются первыми! Это правило является общим для самых разных выражений. Если необходимо гарантировать определенный порядок вычисления разных подвыражений, следует оформить их как последовательность `let`-привязок.

Побочные эффекты и слабый полиморфизм

Взгляните на следующую простую императивную функцию:

OCaml utop (part 1)

<https://github.com/realworldocaml/examples/tree/v1/code/imperative-programming/weak.topscript>

```
# let remember =
  let cache = ref None in
  (fun x ->
    match !cache with
    | Some y -> y
    | None -> cache := Some x; x)
;;
val remember : 'a -> 'a = <fun>
```

Функция `remember` просто кэширует первое переданное ей значение и возвращает его при вызове без аргументов. Это возможно потому, что кэш создается и инициализируется только один раз и затем используется разными вызовами `remember`.

Пользы от функции `remember` не очень много, но, глядя на ее сигнатуру, возникает интересный вопрос: это что еще за тип?

Первый вызов `remember` возвращает то же значение, которое ей было передано, а из этого следует, что тип аргумента и тип возвращаемого значения должны совпадать. Соответственно, `remember` должна иметь сигнатуру `t -> t` для некоторого типа `t`. В определении `remember` нет ничего, что позволило бы судить о конкретном типе `t`, поэтому логично было бы ожидать, что компилятор OCaml сделает обобщение и заменит конкретный тип `t` переменной полиморфного типа. Именно такого рода обобщения являются источником полиморфных типов. Функция `identity`, например, получает полиморфную сигнатуру благодаря такому определению:

OCaml utop (part 2)

<https://github.com/realworldocaml/examples/blob/v1/code/imperative-programming/weak.topscript>

```
# let identity x = x;;
val identity : 'a -> 'a = <fun>
# identity 3;;
- : int = 3
# identity "five";;
- : string = "five"
```

Как видите, полиморфизм функции `identity` позволяет ей оперировать значениями разных типов.

Однако с функцией `remember` произошло нечто иное. Как видно из примеров выше, сигнатура, которую вывел OCaml для `remember`, очень похожа на сигнатуру функции `identity`, но имеет отличия. Приведу ее еще раз:

OCaml

https://github.com/realworldocaml/examples/blob/v1/code/imperative-programming/remember_type.ml

```
val remember : 'a -> 'a = <fun>
```

Подчеркивание в имени переменной типа `'_a` сообщает, что переменная лишь *слабополиморфная* (weakly polymorphic), то есть она может использоваться с любым единственным типом. В этом есть определенный смысл, потому что, в отличие от `identity`, функция `remember` всегда возвращает значение, которое получила при первом вызове, то есть возвращаемое значение всегда будет иметь тот же самый тип.

OCaml превратит слабополиморфную переменную в конкретный тип, как только получит подсказку о том, что это будет за тип:

OCaml utop (part 3)

<https://github.com/realworldocaml/examples/blob/v1/code/imperative-programming/weak.topscript>

```
# let remember_three () = remember 3;;
val remember_three : unit -> int = <fun>
# remember;;
- : int -> int = <fun>
# remember "avocado";;
```

Characters 9-18:

Error: This expression has type string but an expression was expected of type int
(Ошибка: Это выражение имеет тип *string*, тогда как ожидалось выражение типа *int*)

Обратите внимание, что тип функции `remember` был выведен из определения `remember_three`, даже при том, что здесь `remember_three` ни разу не была вызвана!

Ограничение значений

Так когда же компилятор выводит слабополиморфные типы? Как мы видели, потребность в слабополиморфных типах возникает, когда в хранимой изменяемой ячейке запоминается значение неизвестного типа. Поскольку система типов не способна с высокой точностью определить все возможные случаи, OCaml использует приближенное правило, отмечая ситуации, которые не приводят к появлению хранимых изменяемых ячеек, и выводит полиморфные типы только в этих случаях. Это правило называется *ограничением значения* (value restriction).

В основе ограничения значения лежит наблюдение, что некоторые виды выражений, которые мы будем называть *простыми значениями* (simple values), по своей природе не могут вводить хранимые изменяемые ячейки, включая:

- константы (например, целочисленные и вещественные литералы);
- конструкторы, содержащие другие, только простые значения;
- объявления функций, то есть выражения, начинающиеся с ключевого слова `fun` или `function`, или эквивалентные `let`-привязке вида `let f x = ...`;
- `let`-привязки вида `let var = expr1 in expr2`, где `expr1` и `expr2` – это простые значения.

Соответственно, следующее выражение является простым значением, и как результат типы значений, содержащихся в нем, могут быть полиморфными:

OCaml utop (part 1)

https://github.com/realworldocaml/examples/blob/v1/code/imperative-programming/value_restriction.topscript

```
# (fun x -> [x;x]);;
- : 'a -> 'a list = <fun>
```

Но если записать выражение, не являющееся простым значением в соответствии с определением выше, мы получим иной результат. Например, взгляните, что случится, если попытаться мемоизовать функцию, определенную выше.

OCaml utop (part 2)

https://github.com/realworldocaml/examples/blob/v1/code/imperative-programming/value_restriction.topscript

```
# memoize (fun x -> [x;x]);;
- : '_a -> '_a list = <fun>
```

Мемоизованная версия функции фактически была ограничена до простого типа, потому что за кулисами она использует изменяемое состояние, кэшируя значения, возвращаемые предыдущими вызовами. Но OCaml сделал бы точно такие же выводы и в отношении функции, не делающей ничего подобного. Например:

OCaml utop (part 3)

https://github.com/realworldocaml/examples/blob/v1/code/imperative-programming/value_restriction.topscript

```
# identity (fun x -> [x;x]);;
- : '_a -> '_a list = <fun>
```

Здесь вполне возможно было бы вывести полностью полиморфную переменную, но, так как система типов в OCaml не различает чистых функций и функций с побочными эффектами, она оказывается не в состоянии разделить эти две ситуации.

Ограничение значения не требует полного отсутствия изменяемого состояния, единственное требование – отсутствие *хранимого* изменяемого состояния, которое может изменяться между вызовами функции. То есть функция, производящая новую ссылку в каждом вызове, может иметь полностью полиморфный тип (сигнатуру):

OCaml utop (part 4)

https://github.com/realworldocaml/examples/blob/v1/code/imperative-programming/value_restriction.topscript

```
# let f () = ref None;;
val f : unit -> 'a option ref = <fun>
```

Но функция, имеющая изменяемый кэш, сохраняющийся между вызовами, такая как `memoize`, может быть только слабополиморфной.

Частичное применение и ограничение значения

В большинстве случаев, если механизм ограничения значений активируется, значит, для этого есть веские основания. Это может происходить, например, из-за того, что безопасность гарантируется только для значений одного типа. Но иногда механизм ограничения значений активируется и тогда, когда этого не требуется. Наиболее типичным примером такой ситуации могут служить частично примененные функции. Частично примененная функция, подобно любой другой функ-

ции, не является простым значением, из-за этого функции, созданные в результате частичного применения, оказываются менее обобщенными, чем хотелось бы.

Рассмотрим функцию `List.init`, используемую для создания списков, каждый элемент которых создается вызовом функции с индексом этого элемента:

OCaml utop (part 5)

https://github.com/realworldocaml/examples/tree/v1/code/imperative-programming/value_restriction.topscrip

```
# List.init;;
- : int -> f:(int -> 'a) -> 'a list = <fun>
# List.init 10 ~f:Int.to_string;;
- : string list = ["0"; "1"; "2"; "3"; "4"; "5"; "6"; "7"; "8"; "9"]
```

Представьте, что нам потребовалось создать специализированную версию `List.init`, которая всегда создает списки с 10 элементами. Мы могли бы добиться желаемого за счет частичного применения, как показано ниже:

OCaml utop (part 6)

https://github.com/realworldocaml/examples/tree/v1/code/imperative-programming/value_restriction.topscrip

```
# let list_init_10 = List.init 10;;
val list_init_10 : f:(int -> 'a) -> 'a list = <fun>
```

Как видите, для вновь созданной функции компилятор выводит сигнатуру, имеющую слабый полиморфный тип. Это обусловлено отсутствием гарантий, что `List.init` не создаст хранимую ссылку `ref` где-то внутри, которая будет сохраняться между вызовами `list_init_10`. Отказавшись от частичного применения, можно воспрепятствовать этой возможности и одновременно вынудить компилятор выводить полиморфный тип:

OCaml utop (part 7)

https://github.com/realworldocaml/examples/tree/v1/code/imperative-programming/value_restriction.topscrip

```
# let list_init_10 ~f = List.init 10 ~f;;
val list_init_10 : f:(int -> 'a) -> 'a list = <fun>
```

Это преобразование называют *эта-расширением* (*eta expansion*), часто оно оказывается весьма полезным для решения проблем, возникающих из-за ограничения значений.

Ослабление ограничения значений

В действительности механизм вывода типов в языке OCaml действует относительно полиморфных типов чуть лучше, чем было описано выше. Ограничение значений в своей основе является синтаксической проверкой: вы можете использовать определенные операции, которые считаются простыми значениями, а все, что является простым значением, может быть обобщено.

Но в OCaml имеется смягченная версия ограничения значений, способная использовать информацию о типе и выводить полиморфные типы для конструкций, не являющихся простыми значениями.

Например, мы видели, что применение функции, даже такой как функция `identity`, не является простым значением и потому может превратить полиморфное значение в слабополиморфное:

OCaml utop (part 8)

https://github.com/realworldocaml/examples/tree/v1/code/imperative-programming/value_restriction.topscript

```
# identity (fun x -> [x;x]);;
- : 'a -> 'a list = <fun>
```

Но этот результат желателен не всегда. Когда возвращаемое значение имеет неизменяемый тип, OCaml обычно может вывести полностью полиморфный тип:

OCaml utop (part 9)

https://github.com/realworldocaml/examples/blob/v1/code/imperative-programming/value_restriction.topscript

```
# identity [];;
- : 'a list = []
```

С другой стороны, если возвращаемое значение имеет потенциально изменяемый тип, в результате выводится слабополиморфный тип:

OCaml utop (part 10)

https://github.com/realworldocaml/examples/blob/v1/code/imperative-programming/value_restriction.topscript

```
# [[]];;
- : 'a array = [[]]
# identity [[]];;
- : 'a array = [[]]
```

Еще более важным примером может служить определение абстрактных типов. Взгляните на следующую простую структуру данных, представляющую неизменяемый список, поддерживающий постоянное время объединения:

OCaml utop (part 11)

https://github.com/realworldocaml/examples/blob/v1/code/imperative-programming/value_restriction.topscript

```
# module Concat_list : sig
  type 'a t
  val empty : 'a t
  val singleton : 'a -> 'a t
  val concat : 'a t -> 'a t -> 'a t (* постоянное время *)
  val to_list : 'a t -> 'a list      (* линейное время *)
end = struct

  type 'a t = Empty | Singleton of 'a | Concat of 'a t * 'a t

  let empty = Empty
  let singleton x = Singleton x
  let concat x y = Concat (x,y)

  let rec to_list_with_tail t tail =
```

```

    match t with
    | Empty -> tail
    | Singleton x -> x :: tail
    | Concat (x,y) -> to_list_with_tail x (to_list_with_tail y tail)

let to_list t =
  to_list_with_tail t []

end;;
module Concat_list :
sig
  type 'a t
  val empty : 'a t
  val singleton : 'a -> 'a t
  val concat : 'a t -> 'a t -> 'a t
  val to_list : 'a t -> 'a list
end

```

Тонкости реализации в данном случае не важны — важно отметить, что тип `Concat_list.t`, вне всяких сомнений, является неизменяемым значением. Однако когда в игру вступает механизм ограничения значений, OCaml интерпретирует его как изменяемый тип:

OCaml utop (part 12)

https://github.com/realworldocaml/examples/blob/v1/code/imperative-programming/value_restriction.topscript

```

# Concat_list.empty;;
- : 'a Concat_list.t = <abstr>
# identity Concat_list.empty;;
- : '_a Concat_list.t = <abstr>

```

Проблема здесь в том, что абстрактность сигнатуры затмила тот факт, что `Concat_list.t` является неизменяемым типом данных. Мы можем решить эту проблему одним из двух способов: либо сделать тип конкретным (то есть определить реализацию в файле *.mli*), что часто нежелательно; либо сделать переменную типа *ковариантной* (covariant). О ковариантности и контравариантности подробнее будет рассказываться в главе 11, а пока считайте этот прием своеобразной аннотацией, которую можно добавить в интерфейс чистой структуры данных.

В частности, заменив тип `'a t` в интерфейсе типом `+'a t`, мы можем явно обозначить, что структура данных не содержит хранимых ссылок на значения типа `'a`, что поможет компилятору OCaml выводить полиморфные типы для выражений данного типа, который не является простым значением:

OCaml utop

https://github.com/realworldocaml/examples/blob/v1/code/imperative-programming/value_restriction-13.rawscript

```

# module Concat_list : sig
  type +'a t
  val empty : 'a t
  val singleton : 'a -> 'a t

```

```

    val concat : 'a t -> 'a t -> 'a t (* constant time *)
    val to_list : 'a t -> 'a list (* linear time *)
end = struct

    type 'a t = Empty | Singleton of 'a | Concat of 'a t * 'a t
    ...
end;;

module Concat_list :
sig
  type 'a t
  val empty : 'a t
  val singleton : 'a -> 'a t
  val concat : 'a t -> 'a t -> 'a t
  val to_list : 'a t -> 'a list
end

```

Теперь можно применить функцию `identity` к `Concat_list.empty` без потери полиморфизма:

OCaml utop (part 14)

https://github.com/realworldocaml/examples/blob/v1/code/imperative-programming/value_restriction.topscript

```

# identity Concat_list.empty;;
- : 'a Concat_list.t = <abstr>

```

В заключение

В этой главе мы рассмотрели довольно большой объем базовых сведений, в том числе:

- мы обсудили строительные блоки для конструирования изменяемых структур данных, а также основных императивных конструкций, таких как циклы `for` и `while` и оператор `;` последовательного вычисления;
- прошлись по реализациям пары классических императивных структур данных;
- обсудили так называемые благоприятные эффекты (*benign effects*), такие как мемоизация и отложенные вычисления;
- охватили API блочного ввода/вывода в языке OCaml;
- рассмотрели некоторые проблемы уровня языка, такие как порядок вычислений и слабый полиморфизм, влияющие на императивные особенности OCaml.

Масштабы и сложность материала, рассматривавшегося здесь, свидетельствуют о важной роли императивных особенностей в OCaml. Тот факт, что данные в OCaml по умолчанию являются неизменяемыми, не умаляет важности императивного программирования, являющегося фундаментальной частью процесса конструирования любого серьезного приложения, и если вы хотите стать эффективным программистом на OCaml, вам обязательно потребуется понимание императивных приемов программирования на OCaml.

До сих пор в нашем обсуждении модули играли важную, но довольно ограниченную роль. В частности, мы рассматривали их как механизм организации кода в единицы с определенными интерфейсами. Но система модулей в OCaml способна на большее и может служить мощным инструментом создания универсального кода и структурирования крупномасштабных систем. Значительная часть возможностей сосредоточена в функторах.

Функторы, если говорить в общих чертах, – это функции, отображающие модули на модули. Они могут применяться для решения различных проблем структуризации кода, включая следующие:

- **внедрение зависимостей** – обеспечивает возможность замены некоторых компонентов системы. Это может пригодиться, например, для подстановки имитаций компонентов системы во время тестирования;
- **авторасширение модулей** – функторы дают возможность добавлять в имеющиеся модули новые функциональные возможности стандартным способом. Например, кому-то может понадобиться добавить уйму операторов сравнения, основанных на единой базовой функции сравнения. Чтобы сделать это вручную, может потребоваться написать массу повторяющегося кода для каждого типа, а с функторами достаточно будет написать логику только один раз и применить ее к разным типам;
- **создание экземпляров модулей с изменяемым состоянием** – модули могут содержать изменяемые значения, соответственно, может понадобиться создать несколько экземпляров определенного модуля, каждый со своим изменяемым состоянием. Функторы дадут возможность автоматизировать создание таких модулей.

Это лишь некоторые из областей применения функторов. Мы не будем пытаться объять необъятное и привести все возможные примеры использования функторов. Вместо этого в данной главе будет предпринята попытка представить примеры, демонстрирующие языковые особенности и шаблоны проектирования, которые желательно знать, чтобы эффективно применять функторы.

Простейший пример

Давайте создадим функтор, принимающий модуль с единственной целочисленной переменной x и возвращающий новый модуль со значением x на единицу больше. Наша цель – пройти по основным механизмам функторов, несмотря на то что практическая ценность этого примера невысока.

В первую очередь определим сигнатуру модуля, содержащего единственное значение типа `int`:

OCaml utop

<https://github.com/realworldocaml/examples/blob/v1/code/functors/main.topscript>

```
# module type X_int = sig val x : int end;;
module type X_int = sig val x : int end
```

Теперь можно определить функтор. Мы будем использовать тип `X_int` и для определения аргумента функтора, и для определения модуля, возвращаемого функтором:

OCaml utop (part 1)

<https://github.com/realworldocaml/examples/blob/v1/code/functors/main.topscript>

```
# module Increment (M : X_int) : X_int = struct
  let x = M.x + 1
end;;
module Increment : functor (M : X_int) -> X_int
```

Первое, что бросается в глаза, – функторы синтаксически более тяжеловесны, чем обычные функции. С одной стороны, функторы требуют явного использования аннотаций типов, тогда как для функций это не является обязательным. Технически обязательным является только тип входного значения, однако на практике обычно желательно также закреплять тип модуля, возвращаемого функтором, а также использовать файл *.mli*, даже при том что это не является обязательным.

Ниже показано, что случится, если опустить тип модуля, возвращаемого функтором:

OCaml utop (part 2)

<https://github.com/realworldocaml/examples/blob/v1/code/functors/main.topscript>

```
# module Increment (M : X_int) = struct
  let x = M.x + 1
end;;
module Increment : functor (M : X_int) -> sig val x : int end
```

Как видите, тип модуля выводится системой типов буквально, а не как ссылка на именованную сигнатуру `X_int`.

Мы можем использовать функтор `Increment` для определения новых модулей:

OCaml utop (part 3)

<https://github.com/realworldocaml/examples/blob/v1/code/functors/main.topscript>

```
# module Three = struct let x = 3 end;;
module Three : sig val x : int end
# module Four = Increment(Three);;
module Four : sig val x : int end
# Four.x - Three.x;
- : int = 1
```

В данном случае мы применили функтор `Increment` к модулю, сигнатура которого в точности эквивалентна типу `X_int`. Однако функтор `Increment` можно при-

менять к любым модулям, *удовлетворяющим* интерфейсу `X_int` в тех же терминах, в каких содержимое файлов *.ml* удовлетворяет интерфейсам в файлах *.mli*. То есть в типе модуля может отсутствовать некоторая информация, доступная в самом модуле, либо путем исключения некоторых полей, либо путем превращения их в абстрактные поля. Например:

OCaml utop (part 4)

<https://github.com/realworldocaml/examples/blob/v1/code/functors/main.topscript>

```
# module Three_and_more = struct
  let x = 3
  let y = "three"
end;;
module Three_and_more : sig val x : int val y : string end
# module Four = Increment(Three_and_more);;
module Four : sig val x : int end
```

Правила определения соответствия модуля указанной сигнатуре напоминают правила определения соответствия объектов указанному интерфейсу в объектно-ориентированных языках. Как и в объектно-ориентированном контексте, дополнительная информация, не соответствующая указанной сигнатуре (в данном примере – переменная `y`), просто игнорируется.

Более практичный пример: вычисления с применением интервалов

Рассмотрим более практичный пример использования функторов: библиотеку функций для вычислений с применением интервалов. Интервалы часто используются в практике программирования. Они могут иметь разные типы и задействоваться в разных контекстах. Например, может понадобиться организовать вычисления с привлечением интервалов вещественных чисел, строк или значений времени и в каждом из этих случаев выполнять однотипные операции: проверку ширины интервала, проверку попадания в интервал и др.

Давайте посмотрим, как с применением функторов можно реализовать обобщенную библиотеку поддержки интервалов, которую можно было бы использовать с любыми типами, к которым применимо понятие упорядочения.

Прежде всего определим тип модуля, описывающий информацию о конечных точках интервала. Этот интерфейс, который мы назовем `Comparable`, содержит два элемента: функцию сравнения и тип сравниваемых значений:

OCaml utop (part 5)

<https://github.com/realworldocaml/examples/blob/v1/code/functors/main.topscript>

```
# module type Comparable = sig
  type t
  val compare : t -> t -> int
end;;
module type Comparable = sig type t val compare : t -> t -> int end
```

Функция сравнения реализована с применением стандартной идиомы языка OCaml для таких функций: она возвращает 0, если два элемента равны; положительное число, если первый элемент больше второго; и отрицательное число, если первый элемент меньше второго. То есть стандартные функции сравнения можно было бы переписать на основе `compare`, как показано ниже.

OCaml

https://github.com/realworldocaml/examples/blob/v1/code/functors/compare_example.ml

```
compare x y < 0      (* x < y *)
compare x y = 0      (* x = y *)
compare x y > 0      (* x > y *)
```

(Эту идиому можно считать своеобразной исторической ошибкой. Было бы лучше, если бы `compare` возвращала значение вариантного типа с тремя возможными вариантами для понятий «меньше», «больше» и «равно». Но данная идиома настолько утвердилась к настоящему времени, что, скорее всего, уже не изменится.)

Ниже приводится функтор, создающий модуль интервала. Интервал представлен вариантным типом, имеющим два варианта, `Empty` и `Interval (x,y)`, где `x` и `y` представляют границы интервала. В дополнение к типу тело функтора содержит реализации некоторых элементарных функций для взаимодействий с интервалами:

OCaml utop (part 6)

<https://github.com/realworldocaml/examples/blob/v1/code/functors/main.topscript>

```
# module Make_interval(Endpoint : Comparable) = struct

  type t = | Interval of Endpoint.t * Endpoint.t
           | Empty

  (** [create low high] создает новый интервал от [low] до
      [high]. Если [low > high], создается пустой интервал *)
  let create low high =
    if Endpoint.compare low high > 0 then Empty
    else Interval (low,high)

  (** Возвращает true, если интервал пуст *)
  let is_empty = function
    | Empty -> true
    | Interval _ -> false

  (** [contains t x] возвращает true, если [x] попадает в
      интервал [t] *)
  let contains t x =
    match t with
    | Empty -> false
    | Interval (l,h) ->
        Endpoint.compare x l >= 0 && Endpoint.compare x h <= 0

  (** [intersect t1 t2] возвращает пересечение двух
      интервалов *)
```

```

let intersect t1 t2 =
  let min x y = if Endpoint.compare x y <= 0 then x else y in
  let max x y = if Endpoint.compare x y >= 0 then x else y in
  match t1,t2 with
  | Empty, _ | _, Empty -> Empty
  | Interval (l1,h1), Interval (l2,h2) ->
    create (max l1 l2) (min h1 h2)
end ;;

module Make_interval :
  functor (Endpoint : Comparable) ->
    sig
      type t = Interval of Endpoint.t * Endpoint.t | Empty
      val create : Endpoint.t -> Endpoint.t -> t
      val is_empty : t -> bool
      val contains : t -> Endpoint.t -> bool
      val intersect : t -> t -> t
    end

```

Создать экземпляр функтора можно применением его к модулю с подходящей сигнатурой. В следующем фрагменте, вместо того чтобы сначала определить именнованный модуль, а затем применить к нему функтор, мы передаем функтору анонимный модуль:

OCaml utop (part 7)

<https://github.com/realworldocaml/examples/blob/v1/code/functors/main.topscript>

```

# module Int_interval =
  Make_interval(struct
    type t = int
    let compare = Int.compare
  end);;

module Int_interval :
  sig
    type t = Interval of int * int | Empty
    val create : int -> int -> t
    val is_empty : t -> bool
    val contains : t -> int -> bool
    val intersect : t -> t -> t
  end

```

Если интерфейс входного аргумента функтора соответствует стандартным модулям используемых библиотек, можно не определять собственный модуль для передачи его функтору. В следующем примере непосредственно используются модули `Int` и `String`, входящие в состав библиотеки `Core`:

OCaml utop (part 8)

<https://github.com/realworldocaml/examples/blob/v1/code/functors/main.topscript>

```

# module Int_interval = Make_interval(Int) ;;

module Int_interval :
  sig
    type t = Make_interval(Core.Std.Int).t = Interval of int * int | Empty
    val create : int -> int -> t
  end

```

```

    val is_empty : t -> bool
    val contains : t -> int -> bool
    val intersect : t -> t -> t
  end
# module String_interval = Make_interval(String) ;;
module String_interval :
  sig
    type t =
      Make_interval(Core.Std.String).t =
        Interval of string * string
      | Empty
    val create : string -> string -> t
    val is_empty : t -> bool
    val contains : t -> string -> bool
    val intersect : t -> t -> t
  end
end

```

Этот прием действует по той простой причине, что многие модули в библиотеке Core, включая Int и String, соответствуют расширенной версии сигнатуры Comparable, описанной выше. Применение таких стандартизованных сигнатур является общепринятой практикой, во-первых, потому что это упрощает применение функторов, а во-вторых, потому что это способствует стандартизации кодовой базы, которая, в свою очередь, облегчает навигацию по ней.

Вновь объявленный модуль Int_interval можно использовать как любой другой модуль:

OCaml utop (part 9)

<https://github.com/realworldocaml/examples/blob/v1/code/functors/main.topscript>

```

# let i1 = Int_interval.create 3 8;;
val i1 : Int_interval.t = Int_interval.Interval (3, 8)
# let i2 = Int_interval.create 4 10;;
val i2 : Int_interval.t = Int_interval.Interval (4, 10)
# Int_interval.intersect i1 i2;;
- : Int_interval.t = Int_interval.Interval (4, 8)

```

Такая архитектура дает свободу использования любой функции для сравнения значений с границами. Можно, например, создать тип целочисленного интервала с обратным порядком следования, как показано ниже:

OCaml utop (part 10)

<https://github.com/realworldocaml/examples/blob/v1/code/functors/main.topscript>

```

# module Rev_int_interval =
  Make_interval(struct
    type t = int
    let compare x y = Int.compare y x
  end);;
module Rev_int_interval :
  sig
    type t = Interval of int * int | Empty
    val create : int -> int -> t
  end

```

```

val is_empty : t -> bool
val contains : t -> int -> bool
val intersect : t -> t -> t
end

```

Поведение `Rev_int_interval`, разумеется, отличается от поведения `Int_interval`:

OCaml utop (part 11)

<https://github.com/realworldocaml/examples/blob/v1/code/functors/main.topscript>

```

# let interval = Int_interval.create 4 3;;
val interval : Int_interval.t = Int_interval.Empty
# let rev_interval = Rev_int_interval.create 4 3;;
val rev_interval : Rev_int_interval.t = Rev_int_interval.Interval (4, 3)

```

Важно отметить, что `Rev_int_interval.t` – это совершенно иной тип, отличный от `Int_interval.t`, даже при том что они имеют одно и то же физическое представление. Система типов не позволит нам спутать их.

OCaml utop (part 12)

<https://github.com/realworldocaml/examples/blob/v1/code/functors/main.topscript>

```

# Int_interval.contains rev_interval 3;;
Characters 22-34:
Error: This expression has type Rev_int_interval.t
      but an expression was expected of type Int_interval.t
(Ошибка: Это выражение имеет тип Rev_int_interval.t,
      тогда как ожидалось выражение типа Int_interval.t)

```

Это особенно важно потому, что смешивание интервалов двух разновидностей может приводить к семантическим ошибкам. Способность функторов создавать новые типы имеет большую практическую ценность.

Создание абстрактных функторов

Функтор `Make_interval` имеет одну проблему. Программный код, который мы написали, зависит от требования, что верхняя граница должна быть больше нижней, но это требование может быть нарушено. Данное требование устанавливается функцией `create`, но так как тип `Interval.t` не является абстрактным, мы можем обойти функцию `create`:

OCaml utop (part 13)

<https://github.com/realworldocaml/examples/blob/v1/code/functors/main.topscript>

```

# Int_interval.is_empty (* создание с помощью create *)
  (Int_interval.create 4 3) ;;
- : bool = true
# Int_interval.is_empty (* создание в обход create *)
  (Int_interval.Interval (4,3)) ;;
- : bool = false

```

Чтобы сделать `Int_interval.t` абстрактным, необходимо ограничить результат `Make_interval` с помощью интерфейса. Ниже демонстрируется интерфейс, который можно использовать для этой цели:

OCaml utop (part 14)

<https://github.com/realworldocaml/examples/blob/v1/code/functors/main.topscript>

```
# module type Interval_intf = sig
  type t
  type endpoint
  val create : endpoint -> endpoint -> t
  val is_empty : t -> bool
  val contains : t -> endpoint -> bool
  val intersect : t -> t -> t
end;;

module type Interval_intf =
  sig
    type t
    type endpoint
    val create : endpoint -> endpoint -> t
    val is_empty : t -> bool
    val contains : t -> endpoint -> bool
    val intersect : t -> t -> t
  end
```

Этот интерфейс включает тип `endpoint`, чтобы дать возможность ссылаться на тип граничной точки. Опираясь на этот интерфейс, можно дополнить наше определение `Make_interval`. Отметим, что мы добавили тип `endpoint` в реализацию модуля для сопоставления `Interval_intf`:

OCaml utop

<https://github.com/realworldocaml/examples/blob/v1/code/functors/main-15.rawscript>

```
# module Make_interval(Endpoint : Comparable) : Interval_intf = struct
  type endpoint = Endpoint.t
  type t = | Interval of Endpoint.t * Endpoint.t
          | Empty
  ...
end ;;

module Make_interval : functor (Endpoint : Comparable) -> Interval_intf
```

Совместно используемые ограничения

Получившийся модуль является абстрактным, но он, к сожалению, слишком абстрактен. В частности, мы не экспортировали тип `endpoint`, а это означает, что мы не сможем даже создать интервал:

OCaml utop (part 16)

<https://github.com/realworldocaml/examples/blob/v1/code/functors/main.topscript>

```
# module Int_interval = Make_interval(Int);;

module Int_interval :
  sig
    type t = Make_interval(Core.Std.Int).t
    type endpoint = Make_interval(Core.Std.Int).endpoint
    val create : endpoint -> endpoint -> t
    val is_empty : t -> bool
    val contains : t -> endpoint -> bool
    val intersect : t -> t -> t
```



```
end
# Int_interval.create 3 4;;
```

Characters 20-21:

Error: This expression has type int but an expression was expected of type

```
Int_interval.endpoint
```

(Ошибка: Это выражение имеет тип int, тогда как ожидалось выражение типа

```
Int_interval.endpoint)
```

Чтобы исправить эту проблему, следует экспортировать тот факт, что тип `endpoint` эквивалентен типу `Int.t` (или `Endpoint.t`, где `Endpoint` – это аргумент функтора). Обеспечить это можно путем *совместного использования ограничения* (sharing constraint) и тем самым потребовать от компилятора экспортировать факт эквивалентности указанного типа другому типу. Синтаксис совместно используемого ограничения имеет следующий вид:

Syntax

https://github.com/realworldocaml/examples/blob/v1/code/functors/sharing_constraint.syntax

```
<Module_type> with type <type> = <type'>
```

Результатом этого выражения является новая сигнатура, сообщающая, что тип `type`, объявленный внутри определения типа модуля, эквивалентен типу `type'`, объявленному за его пределами. Имеется также возможность применять к одной сигнатуре несколько совместно используемых ограничений:

Syntax

https://github.com/realworldocaml/examples/blob/v1/code/functors/multi_sharing_constraint.syntax

```
<Module_type> with type <type1> = <type1'> and <type2> = <type2'>
```

С помощью совместно используемого ограничения можно создать специализированную версию `Interval_intf` для определения целочисленных интервалов:

OCaml utop (part 17)

<https://github.com/realworldocaml/examples/blob/v1/code/functors/main.topscript>

```
# module type Int_interval_intf =
  Interval_intf with type endpoint = int;;
module type Int_interval_intf =
  sig
    type t
    type endpoint = int
    val create : endpoint -> endpoint -> t
    val is_empty : t -> bool
    val contains : t -> endpoint -> bool
    val intersect : t -> t -> t
  end
```

Совместно используемые ограничения можно также использовать в контексте функтора. Чаще всего ограничения применяются там, где желательно указать, что некоторые типы модуля, сгенерированные функтором, связаны с типами в модуле, переданном функтору.

В данном случае нам хотелось бы экспортировать эквивалентность между типом `endpoint` в новом модуле и типом `Endpoint.t` из модуля `Endpoint`, являющегося аргументом функтора. Делается это следующим образом:

OCaml utop

<https://github.com/realworldocaml/examples/blob/v1/code/functors/main-18.rawscript>

```
# module Make_interval(Endpoint : Comparable)
    : (Interval_intf with type endpoint = Endpoint.t)
= struct

    type endpoint = Endpoint.t
    type t = | Interval of Endpoint.t * Endpoint.t
            | Empty
    ...
end ;;

module Make_interval :
  functor (Endpoint : Comparable) ->
    sig
      type t
      type endpoint = Endpoint.t
      val create : endpoint -> endpoint -> t
      val is_empty : t -> bool
      val contains : t -> endpoint -> bool
      val intersect : t -> t -> t
    end
```

Интерфейс почти не изменился, если не считать, что теперь тип `endpoint` интерпретируется как тип `Endpoint.t`. В результате мы снова можем выполнять операции, такие как конструирование интервалов, требующие известности типа `endpoint`:

OCaml utop (part 19)

<https://github.com/realworldocaml/examples/blob/v1/code/functors/main.topscript>

```
# module Int_interval = Make_interval(Int) ;;
module Int_interval :
  sig
    type t = Make_interval(Core.Std.Int).t
    type endpoint = int
    val create : endpoint -> endpoint -> t
    val is_empty : t -> bool
    val contains : t -> endpoint -> bool
    val intersect : t -> t -> t
  end
# let i = Int_interval.create 3 4;;
val i : Int_interval.t = <abstr>
# Int_interval.contains i 5;;
- : bool = false
```

Деструктивная подстановка

Совместно используемые ограничения в основном справляются с возлагаемым на них заданием, но они имеют некоторые недостатки. В частности, мы теперь столкнулись с бесполезным объявлением типа `endpoint`, которое захламляет и ин-

терфейс, и реализацию. Более удачное решение состоит в том, чтобы изменить сигнатуру `Interval_intf`, заменив `endpoint` на `Endpoint.t`, и удалить определение `endpoint` из сигнатуры. Сделать это можно с применением приема, называемого *деструктивной подстановкой* (destructive substitution). Ниже приводится базовый синтаксис:

Syntax

https://github.com/realworldocaml/examples/blob/v1/code/functors/destructive_sub.syntax

```
<Module_type> with type <type> := <type'>
```

Дальше показано, как использовать этот прием применительно к `Make_interval`:

OCaml utop (part 20)

<https://github.com/realworldocaml/examples/blob/v1/code/functors/main.topscript>

```
# module type Int_interval_intf =
  Interval_intf with type endpoint := int;;
module type Int_interval_intf =
sig
  type t
  val create : int -> int -> t
  val is_empty : t -> bool
  val contains : t -> int -> bool
  val intersect : t -> t -> t
end
```

Как видите, исчезли все упоминания о типе `endpoint` — его заменил тип `int`. Как и при использовании ограничений, деструктивную подстановку также можно использовать в контексте функтора:

OCaml utop

<https://github.com/realworldocaml/examples/blob/v1/code/functors/main-21.rawscript>

```
# module Make_interval(Endpoint : Comparable)
  : Interval_intf with type endpoint := Endpoint.t =
struct

  type t = | Interval of Endpoint.t * Endpoint.t
          | Empty

  ...
end ;;
module Make_interval :
functor (Endpoint : Comparable) ->
sig
  type t
  val create : Endpoint.t -> Endpoint.t -> t
  val is_empty : t -> bool
  val contains : t -> Endpoint.t -> bool
  val intersect : t -> t -> t
end
```

Интерфейс получился в точности таким, каким нам хотелось его видеть: тип `t` является абстрактным, а тип граничной точки экспортируется. Благодаря этому

можно создавать значения типа `Int_interval.t` с помощью функции создания, и исключается возможность прямого применения конструкторов:

OCaml utop (part 22)

<https://github.com/realworldocaml/examples/blob/v1/code/functors/main.topscript>

```
# module Int_interval = Make_interval(Int);;
module Int_interval :
  sig
    type t = Make_interval(Core.Std.Int).t
    val create : int -> int -> t
    val is_empty : t -> bool
    val contains : t -> int -> bool
    val intersect : t -> t -> t
  end
# Int_interval.is_empty
  (Int_interval.create 3 4);;
- : bool = false
# Int_interval.is_empty
  (Int_interval.Interval (4,3));;
Characters 40-48:
Error: Unbound constructor Int_interval.Interval
(Ошибка: Несвязанный конструктор Int_interval.Interval)
```

Исчезновение типа `endpoint` из интерфейса означает также, что больше не нужно определять псевдоним типа `endpoint` в теле модуля.

Следует отметить, что название «деструктивная подстановка» несколько не соответствует действительности – в ней нет ничего деструктивного (разрушительного), она всего лишь является способом создания новой сигнатуры за счет преобразования существующей.

Использование нескольких интерфейсов

Еще одной особенностью, которую было бы желательно иметь в составе модуля реализации интервала, является возможность сериализации, то есть возможность читать и записывать интервалы в виде потоков байтов. В данном случае мы реализуем эту возможность посредством преобразования интервалов в *s*-выражения и обратно, о которых рассказывалось в главе 7. Напомню, что *s*-выражение – это выражение в круглых скобках, атомами в котором являются строки. Этот формат сериализации широко используется в библиотеке `Core`. Например:

OCaml utop (part 23)

<https://github.com/realworldocaml/examples/blob/v1/code/functors/main.topscript>

```
# Sexp.of_string "(This is (an s-expression))";;
- : Sexp.t = (This is (an s-expression))
```

В состав библиотеки `Core` входит расширение синтаксиса `Sexplib`, которое позволяет автоматически генерировать функции преобразования *s*-выражений на основе объявлений типов. Достаточно добавить `with sexp` к определению типа – и расширение сгенерирует необходимые функции преобразования. То есть мы можем записать:

OCaml utop (part 24)

<https://github.com/realworldocaml/examples/blob/v1/code/functors/main.topscript>

```
# type some_type = int * string list with sexp;;
type some_type = int * string list
val some_type_of_sexp : Sexp.t -> int * string list = <fun>
val sexp_of_some_type : int * string list -> Sexp.t = <fun>
# sexp_of_some_type (33, ["one"; "two"]);;
- : Sexp.t = (33 (one two))
# Sexp.of_string "(44 (five six))" |> some_type_of_sexp;;
- : int * string list = (44, ["five"; "six"])
```

Более подробно s-выражения и расширение Sexplib будут обсуждаться в главе 17, а сейчас давайте посмотрим, что случится, если добавить `with sexp` к определению типа `t` внутри функтора:

OCaml utop

<https://github.com/realworldocaml/examples/blob/v1/code/functors/main-25.rawscript>

```
# module Make_interval(Endpoint : Comparable)
  : (Interval_intf with type endpoint := Endpoint.t) = struct

  type t = | Interval of Endpoint.t * Endpoint.t
           | Empty
  with sexp
  ...
end ;;
```

Characters 136-146:

Error: Unbound value Endpoint.t_of_sexp

(Ошибка: Несвязанное значение Endpoint.t_of_sexp)

Проблема в том, что `sexp` добавляет код определения функций преобразования s-выражений, и этот код предполагает, что модуль `Endpoint` имеет соответствующие функции `sexp`-преобразований для `Endpoint.t`. Но мы знаем о `Endpoint` только то, что он удовлетворяет интерфейсу `Comparable`, который ничего не говорит о s-выражениях.

К счастью, в `Core` имеется встроенный интерфейс как раз для этой цели, который называется `Sexprable` и имеет следующее определение:

OCaml

<https://github.com/realworldocaml/examples/blob/v1/code/functors/sexpable.ml>

```
module type Sexprable = sig
  type t
  val sexp_of_t : t -> Sexp.t
  val t_of_sexp : Sexp.t -> t
end
```

Мы можем модифицировать `Make_interval`, задействовав интерфейс `Sexprable` как в объявлениях входных аргументов, так и в объявлении результата. Сначала создадим расширенную версию интерфейса `Interval_intf`, включив в него функции из интерфейса `Sexprable`. Для этого можно использовать деструктивную подстановку в интерфейсе `Sexprable`, чтобы избежать конфликтов между типами:

OCaml utop (part 26)

<https://github.com/realworldocaml/examples/blob/v1/code/functors/main.topscript>

```
# module type Interval_intf_with_sexp = sig
  include Interval_intf
  include Sexpable with type t := t
end;;

module type Interval_intf_with_sexp =
sig
  type t
  type endpoint
  val create : endpoint -> endpoint -> t
  val is_empty : t -> bool
  val contains : t -> endpoint -> bool
  val intersect : t -> t -> t
  val t_of_sexp : Sexp.t -> t
  val sexp_of_t : t -> Sexp.t
end
```

Аналогично можно определить тип `t` в нашем новом модуле и применить деструктивную подстановку ко всем интерфейсам, подключаемым в интерфейсе `Interval_intf`, как показано в следующем примере. Этот прием более очевиден при объединении нескольких интерфейсов, так как он корректно отражает, что все сигнатуры обрабатываются эквивалентно:

OCaml utop (part 27)

<https://github.com/realworldocaml/examples/blob/v1/code/functors/main.topscript>

```
# module type Interval_intf_with_sexp = sig
  type t
  include Interval_intf with type t := t
  include Sexpable with type t := t
end;;

module type Interval_intf_with_sexp =
sig
  type t
  type endpoint
  val create : endpoint -> endpoint -> t
  val is_empty : t -> bool
  val contains : t -> endpoint -> bool
  val intersect : t -> t -> t
  val t_of_sexp : Sexp.t -> t
  val sexp_of_t : t -> Sexp.t
end
```

Теперь можно перейти к самому функтору. Мы тщательно переопределили функции преобразования `sexp`, чтобы гарантировать поддержку инвариантности структуры данных в форме `s`-выражения:

OCaml utop (part 28)

<https://github.com/realworldocaml/examples/blob/v1/code/functors/main.topscript>

```
# module Make_interval(Endpoint : sig
  type t
  include Comparable with type t := t
```

```

        include Sexpable with type t := t
      end)
    : (Interval_intf_with_sexp with type endpoint := Endpoint.t)
= struct

  type t = | Interval of Endpoint.t * Endpoint.t
           | Empty
  with sexp

  (** [create low high] создает новый интервал от [low] до
      [high]. Если [low > high], создается пустой интервал *)
  let create low high =
    if Endpoint.compare low high > 0 then Empty
    else Interval (low,high)

  (* добавляет обертку вокруг автоматически сгенерированного [t_of_sexp],
      чтобы гарантировать инвариантность структуры данных *)
  let t_of_sexp sexp =
    match t_of_sexp sexp with
    | Empty -> Empty
    | Interval (x,y) -> create x y

  (** Возвращает true, если интервал пуст *)
  let is_empty = function
    | Empty -> true
    | Interval _ -> false

  (** [contains t x] возвращает true, если [x] содержится в
      интервале [t] *)
  let contains t x =
    match t with
    | Empty -> false
    | Interval (l,h) ->
      Endpoint.compare x l >= 0 && Endpoint.compare x h <= 0

  (** [intersect t1 t2] возвращает пересечение двух
      интервалов*)
  let intersect t1 t2 =
    let min x y = if Endpoint.compare x y <= 0 then x else y in
    let max x y = if Endpoint.compare x y >= 0 then x else y in
    match t1,t2 with
    | Empty, _ | _, Empty -> Empty
    | Interval (l1,h1), Interval (l2,h2) ->
      create (max l1 l2) (min h1 h2)
end;;

module Make_interval :
functor
  (Endpoint : sig
    type t
    val compare : t -> t -> int
    val t_of_sexp : Sexp.t -> t
    val sexp_of_t : t -> Sexp.t
  end) ->
sig

```

```

type t
val create : Endpoint.t -> Endpoint.t -> t
val is_empty : t -> bool
val contains : t -> Endpoint.t -> bool
val intersect : t -> t -> t
val t_of_sexp : Sexp.t -> t
val sexp_of_t : t -> Sexp.t
end

```

И теперь мы можем использовать функции преобразования, как обычно:

OCaml utop (part 29)

<https://github.com/realworldocaml/examples/blob/v1/code/functors/main.topscript>

```

# module Int_interval = Make_interval(Int) ;;
module Int_interval :
sig
  type t = Make_interval(Core.Std.Int).t
  val create : int -> int -> t
  val is_empty : t -> bool
  val contains : t -> int -> bool
  val intersect : t -> t -> t
  val t_of_sexp : Sexp.t -> t
  val sexp_of_t : t -> Sexp.t
end
# Int_interval.sexp_of_t (Int_interval.create 3 4);;
- : Sexp.t = (Interval 3 4)
# Int_interval.sexp_of_t (Int_interval.create 4 3);;
- : Sexp.t = Empty

```

Расширение модулей

Еще одной типичной областью применения функторов является автоматическое создание функциональности, специфической для типа, стандартизованным способом. Давайте рассмотрим этот прием в контексте функциональной очереди, которая фактически является всего лишь функциональной версией очереди FIFO (first-in, first-out – первым пришел, первым вышел). Так как очередь является функциональной, все операции с ней возвращают новые очереди, а исходная очередь остается неизменной.

Ниже приводится содержимое файла *.mli* с одним из возможных определений интерфейса модуля:

OCaml

<https://github.com/realworldocaml/examples/blob/v1/code/functors/fqueue.mli>

```

type 'a t

val empty : 'a t

(** [enqueue q el] добавляет [el] в конец очереди [q] *)
val enqueue : 'a t -> 'a -> 'a t

(** [dequeue q] возвращает None, если очередь [q] пуста, иначе возвращает

```



```

    первый элемент в очереди и остаток очереди *)
val dequeue : 'a t -> ('a * 'a t) option

(** Выполняет свертку очереди в направлении от начала к концу *)
val fold : 'a t -> init:'acc -> f:('acc -> 'a -> 'acc) -> 'acc

```

Функция `Fqueue.fold` требует дополнительных пояснений. Она следует тому же шаблону, что и функция `List.fold`, описанная в разделе «Эффективное использование модуля `List`» в главе 3. По сути, вызов `Fqueue.fold q ~init ~f` выполнит обход элементов в очереди `q` от первого до последнего, с начальным значением аккумулятора `init`, применит функцию `f` к каждому элементу очереди с накоплением результатов в аккумуляторе и вернет окончательное значение аккумулятора. Как вы увидите далее, функция `fold` является весьма мощной операцией.

Мы реализуем в `Fqueue` хорошо известный прием управления входными и выходными списками, чтобы увеличить эффективность операций постановки в очередь и извлечения из очереди. Если попытаться выполнить извлечение из очереди, когда выходной список пуст, входной список будет переупорядочен и станет новым выходным списком. Ниже приводится реализация модуля:

OCaml

<https://github.com/realworldocaml/examples/blob/v1/code/functors/fqueue.ml>

```

open Core.Std

type 'a t = 'a list * 'a list

let empty = ([], [])

let enqueue (in_list, out_list) x =
  (x :: in_list, out_list)

let dequeue (in_list, out_list) =
  match out_list with
  | hd :: tl -> Some (hd, (in_list, tl))
  | [] ->
    match List.rev in_list with
    | [] -> None
    | hd :: tl -> Some (hd, ([], tl))

let fold (in_list, out_list) ~init ~f =
  let after_out = List.fold ~init ~f out_list in
  List.fold_right ~init:after_out ~f:(fun x acc -> f acc x) in_list

```

Одна из проблем модуля `Fqueue` состоит в том, что его интерфейс является слишком схематичным. В модуле отсутствуют многие вспомогательные функции, которые могли бы иметь большую практическую ценность. В модуле `List`, напротив, имеются функции, такие как `List.iter`, позволяющая применить другую функцию к каждому элементу, и `List.for_all`, возвращающая `true`, если указанный предикат вернет `true` для каждого элемента списка. Такие вспомогательные функции имеются практически у всех контейнерных типов, и их повторная реализация является рутинной задачей.

Так получилось, что многие из этих вспомогательных функций могут быть реализованы на основе имеющейся уже функции `fold`. Вместо того чтобы писать все эти вспомогательные функции вручную для каждого нового контейнерного типа, мы можем добавлять их с помощью функтора в любой контейнерный тип, имеющий функцию `fold`.

Давайте создадим новый модуль `Foldable`, автоматизирующий процесс добавления вспомогательных функций в контейнеры, имеющие функцию `fold`. Как показано ниже, модуль `Foldable` имеет сигнатуру модуля `S`, которая определяет сигнатуру, требующую наличия поддержки операции свертки; и функтор `Extend`, позволяющий расширять любые модули, соответствующие сигнатуре `Foldable.S`:

OCaml

<https://github.com/realworldocaml/examples/blob/v1/code/functors/foldable.ml>

```
open Core.Std

module type S = sig
  type 'a t
  val fold : 'a t -> init:'acc -> f:('acc -> 'a -> 'acc) -> 'acc
end

module type Extension = sig
  type 'a t
  val iter      : 'a t -> f:('a -> unit) -> unit
  val length    : 'a t -> int
  val count     : 'a t -> f:('a -> bool) -> int
  val for_all    : 'a t -> f:('a -> bool) -> bool
  val exists    : 'a t -> f:('a -> bool) -> bool
end

(* Для расширения модуля Foldable *)
module Extend(Arg : S)
  : (Extension with type 'a t := 'a Arg.t) =
struct
  open Arg

  let iter t ~f =
    fold t ~init:() ~f:(fun () a -> f a)

  let length t =
    fold t ~init:0 ~f:(fun acc _ -> acc + 1)

  let count t ~f =
    fold t ~init:0 ~f:(fun count x -> count + if f x then 1 else 0)

  exception Short_circuit

  let for_all c ~f =
    try iter c ~f:(fun x -> if not (f x) then raise Short_circuit); true
    with Short_circuit -> false

  let exists c ~f =
```

```

    try iter c ~f:(fun x -> if f x then raise Short_circuit); false
  with Short_circuit -> true
end

```

Теперь его можно применить к модулю `Fqueue`. Мы можем определить интерфейс расширенной версии модуля `Fqueue`, как показано ниже:

OCaml

https://github.com/realworldocaml/examples/blob/v1/code/functors/extended_fqueue.mli

```

type 'a t
include (module type of Fqueue) with type 'a t := 'a t
include Foldable.Extension with type 'a t := 'a t

```

Чтобы применить функтор, поместим определение `Fqueue` во вложенный модуль с именем `T` и затем вызовом `Foldable.Extend`:

OCaml

https://github.com/realworldocaml/examples/blob/v1/code/functors/extended_fqueue.ml

```

include Fqueue
include Foldable.Extend(Fqueue)

```

В библиотеке `Core` имеется множество функторов для расширения модулей, которые реализованы по тому же шаблону, в их числе:

- `Container.Make` — близко напоминает `Foldable.Extend`;
- `Comparable.Make` — добавляет поддержку функциональности, опирающейся на функцию сравнения, включая поддержку таких контейнеров, как отображения (`maps`) и множества;
- `Hashable.Make` — добавляет поддержку структур данных, основанных на функции хэширования, включая хэш-таблицы, хэш-множества и хэш-кучи (`hash heaps`);
- `Monad.Make` — для так называемых библиотек монад, подобных тем, что описываются в главах 7 и 18. Данный функтор используется для включения коллекции стандартных вспомогательных функций, основанных на операторах `bind` и `return`.

Эти функторы особенно привлекательны в случаях, когда требуется добавить ту же функциональность, что широко доступна в библиотеке `Core`.

В этой главе мы рассмотрели лишь некоторые из областей применения функторов. Функторы в действительности — это очень мощный инструмент организации кода. Проблема лишь в том, что функторы оказываются более тяжеловесными синтаксически, чем остальные конструкции языка, и для их эффективного использования нужно знать и понимать, как решать некоторые сложные проблемы с помощью совместно используемых ограничений и деструктивной подстановки.

Из вышесказанного следует, что для небольших и простых программ широкое использование функторов наверняка будет ошибкой. Но по мере роста сложности ваших программ и востребованности модульной организации кода функторы превращаются в ценнейший инструмент.

Глава 10

Модули первого порядка

Язык OCaml можно представить как состоящий из двух частей: базовый язык, предназначенный для описания значений и типов, и язык модулей, предназначенный для описания модулей и их сигнатур. Эти два подязыка имеют четкую границу, в том смысле что модули могут содержать типы и значения, но обычные значения не могут содержать модулей или типов модулей. Из этого следует, что в OCaml нельзя определить переменную, значением которой будет модуль, или функцию, принимающую модуль в виде аргумента.

Но в OCaml имеется обходное решение, позволяющее преодолевать эту границу, реализованное в форме *модулей первого порядка* (*first-class modules*). Модули первого порядка – это обычные значения, которые можно создавать из обычных модулей и преобразовывать их обратно в обычные модули.

Модули первого порядка представляют довольно сложный прием, и вам потребуется овладеть некоторыми расширенными аспектами языка, чтобы использовать их эффективно. Но преимущества, которые они дают, стоят потраченных усилий, потому что использование модулей как простых значений значительно расширяет круг возможностей и упрощает построение гибких модульных систем.

Приемы работы с модулями первого порядка

Исследование базовых механизмов модулей первого порядка мы начнем со знакомства с небольшими, простыми примерами. Более практичные примеры мы рассмотрим в следующем разделе.

Взгляните на следующую сигнатуру модуля с единственной целочисленной переменной:

OCaml utop

<https://github.com/realworldocaml/examples/blob/v1/code/fcm/main.topscript>

```
# module type X_int = sig val x : int end;;  
module type X_int = sig val x : int end
```

Мы можем также создать модуль, соответствующий этой сигнатуре:

OCaml utop (part 1)

<https://github.com/realworldocaml/examples/blob/v1/code/fcm/main.topscript>

```
# module Three : X_int = struct let x = 3 end;;  
module Three : X_int  
# Three.x;;  
- : int = 3
```

Модуль первого порядка создается в результате упаковки вместе модуля и сигнатуры, которой он соответствует. Делается это с помощью ключевого слова `module`:

Syntax

<https://github.com/realworldocaml/examples/blob/v1/code/fcm/pack.syntax>

```
(module <Модуль> : <Тип_модуля>)
```

То есть преобразование модуля `Three` в модуль первого порядка можно выполнить так:

OCaml utop (part 2)

<https://github.com/realworldocaml/examples/blob/v1/code/fcm/main.topscript>

```
# let three = (module Three : X_int);;
val three : (module X_int) = <module>
```

Тип модуля можно не указывать при создании модуля первого порядка, если его можно вывести. То есть мы можем то же самое выразить так:

OCaml utop (part 3)

<https://github.com/realworldocaml/examples/blob/v1/code/fcm/main.topscript>

```
# module Four = struct let x = 4 end;;
module Four : sig val x : int end
# let numbers = [ three; (module Four) ];;
val numbers : (module X_int) list = [<module>; <module>]
```

Модуль первого порядка можно также создать из анонимного модуля:

OCaml utop (part 4)

<https://github.com/realworldocaml/examples/blob/v1/code/fcm/main.topscript>

```
# let numbers = [three; (module struct let x = 4 end)];;
val numbers : (module X_int) list = [<module>; <module>]
```

Чтобы получить доступ к содержимому модуля первого порядка, его необходимо распаковать в обычный модуль. Сделать это можно с помощью ключевого слова `val`:

Syntax

<https://github.com/realworldocaml/examples/blob/v1/code/fcm/unpack.syntax>

```
(val <модуль_первого_порядка> : <Тип_модуля>)
```

Например:

OCaml utop (part 5)

<https://github.com/realworldocaml/examples/blob/v1/code/fcm/main.topscript>

```
# module New_three = (val three : X_int) ;;
module New_three : X_int
# New_three.x;;
- : int = 3
```

Равенство типов модулей первого порядка

Тип модуля первого порядка, например `(module X_int)`, определяется полностью квалифицированным именем сигнатуры, использованной для его создания. Модуль первого порядка, основанный на сигнатуре с другим именем, даже если это фактически та же самая сигнатура, будет иметь, с точки зрения компилятора, совершенно другой тип:

OCaml utop (part 6)

<https://github.com/realworldocaml/examples/blob/v1/code/fcm/main.topscript>

```
# module type Y_int = X_int;;
module type Y_int = X_int
# let five = (module struct let x = 5 end : Y_int);;
val five : (module Y_int) = <module>
# [three; five];;
Characters 8-12:
Error: This expression has type (module Y_int)
      but an expression was expected of type (module X_int)
(Ошибка: Это выражение имеет тип (module Y_int),
      тогда как ожидалось выражение типа (module X_int))
```

Но даже при том, что типы модулей первого порядка различны, типы модулей, лежащие в их основе, являются совместимыми (в действительности – идентичными), поэтому мы можем унифицировать типы путем распаковывания и переупаковывания модулей:

OCaml utop (part 7)

<https://github.com/realworldocaml/examples/blob/v1/code/fcm/main.topscript>

```
# [three; (module (val five))];;
- : (module X_int) list = [<module>; <module>]
```

Порядок определения равенства типов модулей первого порядка может показаться запутанным. Одной из часто возникающих и проблематичных ситуаций является создание псевдонима типа модуля, объявленного где-то в другом месте. Это часто делается с целью повышения удобочитаемости и может выполняться путем явного объявления типа модуля или неявно, с помощью объявления `include`. В обоих случаях возникает побочный эффект – создание псевдонима типа модуля первого порядка, несовместимого с исходным типом модуля. Чтобы избавиться от этой проблемы, необходимо выработать дисциплину обращения с сигнатурами при конструировании модулей первого порядка.

Можно также написать обычные функции, принимающие и создающие модули первого порядка. Ниже демонстрируются две функции: `to_int`, которая преобразует `(module X_int)` в `int`, и `plus`, которая возвращает сумму двух `(module X_int)`:

OCaml utop (part 8)

<https://github.com/realworldocaml/examples/blob/v1/code/fcm/main.topscript>

```
# let to_int m =
  let module M = (val m : X_int) in
  M.x
;;
val to_int : (module X_int) -> int = <fun>
# let plus m1 m2 =
  (module struct
    let x = to_int m1 + to_int m2
  end : X_int)
;;
val plus : (module X_int) -> (module X_int) -> (module X_int) = <fun>
```

Имея эти функции, можно работать со значениями типа `(module X_int)` более естественным способом, используя преимущества простоты и выразительности базового языка:

OCaml utop (part 9)

<https://github.com/realworldocaml/examples/blob/v1/code/fcm/main.topscript>

```
# let six = plus three three;;
val six : (module X_int) = <module>
# to_int (List.fold ~init:six ~f:plus [three;three]);;
- : int = 12
```

При работе с модулями первого порядка можно использовать некоторые синтаксические сокращения. Одним из наиболее заметных сокращений является возможность преобразования обычных модулей в операциях сопоставления с образцом. То есть функцию `to_int` можно переписать так:

OCaml utop (part 10)

<https://github.com/realworldocaml/examples/blob/v1/code/fcm/main.topscript>

```
# let to_int (module M : X_int) = M.x ;;
val to_int : (module X_int) -> int = <fun>
```

В дополнение к простым значениям модули первого порядка могут содержать типы и функции. Ниже приводится интерфейс, содержащий тип и соответствующую операцию `bump`, которая принимает значение этого типа и производит новое значение:

OCaml utop (part 11)

<https://github.com/realworldocaml/examples/blob/v1/code/fcm/main.topscript>

```
# module type Bumpable = sig
  type t
  val bump : t -> t
end;;
module type Bumpable = sig type t val bump : t -> t end
```

Можно создать несколько экземпляров этого модуля с разными типами, лежащими в основе:

OCaml utop (part 12)

<https://github.com/realworldocaml/examples/blob/v1/code/fcm/main.topscript>

```
# module Int_bumper = struct
  type t = int
  let bump n = n + 1
end;;
module Int_bumper : sig type t = int val bump : t -> t end
# module Float_bumper = struct
  type t = float
  let bump n = n +. 1.
end;;
module Float_bumper : sig type t = float val bump : t -> t end
```

и преобразовать их в модули первого порядка:

OCaml utop (part 13)

<https://github.com/realworldocaml/examples/blob/v1/code/fcm/main.topscript>

```
# let int_bumper = (module Int_bumper : Bumpable);;
val int_bumper : (module Bumpable) = <module>
```

Но с модулем `int_bumper` можно сделать не так много. Так как `int_bumper` является полностью абстрактным, мы не сможем восстановить тот факт, что базовым его типом является тип `int`.

OCaml utop (part 14)

<https://github.com/realworldocaml/examples/blob/v1/code/fcm/main.topscript>

```
# let (module Bumpable) = int_bumper in Bumpable.bump 3;;
```

Characters 52-53:

```
Error: This expression has type int but an expression was expected of type
      Bumpable.t
```

(Ошибка: Это выражение имеет тип int, тогда как ожидалось выражение типа Bumpable.t)

Чтобы сделать `int_bumper` более практичным, необходимо экспортировать тип, как показано ниже:

OCaml utop (part 15)

<https://github.com/realworldocaml/examples/blob/v1/code/fcm/main.topscript>

```
# let int_bumper = (module Int_bumper : Bumpable with type t = int);;
val int_bumper : (module Bumpable with type t = int) = <module>
# let float_bumper = (module Float_bumper : Bumpable with type t = float);;
val float_bumper : (module Bumpable with type t = float) = <module>
```

Совместно используемые ограничения, которые мы добавили выше, делают получившиеся модули первого порядка полиморфными по типу `t`. Как результат теперь мы можем использовать эти значения со значениями соответствующего типа:

OCaml utop (part 16)

<https://github.com/realworldocaml/examples/blob/v1/code/fcm/main.topscript>

```
# let (module Bumpable) = int_bumper in Bumpable.bump 3;;
- : int = 4
# let (module Bumpable) = float_bumper in Bumpable.bump 3.5;;
- : float = 4.5
```

Мы можем также написать функции, использующие такие модули первого порядка полиморфным способом. Следующая функция принимает два аргумента: модуль `Bumpable` и список элементов того же типа, что и тип `t` в модуле:

OCaml utop (part 17)

<https://github.com/realworldocaml/examples/blob/v1/code/fcm/main.topscript>

```
# let bump_list
  (type a)
  (module B : Bumpable with type t = a)
  (l: a list)
=
List.map ~f:B.bump l
```



```
;;
val bump_list : (module Bumpable with type t = 'a) -> 'a list -> 'a list =
  <fun>
```

Здесь мы задействовали особенность языка OCaml, не использовавшуюся нами прежде: *локально-абстрактный тип* (*locally abstract type*). В определении любой функции можно объявить псевдопараметр в форме `(type a)` с любым именем типа, в результате чего будет создан новый тип. В контексте функции этот тип действует подобно абстрактному типу. В примере выше локально-абстрактный тип использовался как часть совместно используемого ограничения, связывающего тип `B.t` с типом элементов входного списка.

В результате функция становится полиморфной и по типу элементов списка, и по типу `Bumpable.t`. Вот как действует такая функция:

OCaml utop (part 18)

<https://github.com/realworldocaml/examples/tree/v1/code/fcm/main.topscript>

```
# bump_list int_bumper [1;2;3];;
- : int list = [2; 3; 4]
# bump_list float_bumper [1.5;2.5;3.5];;
- : float list = [2.5; 3.5; 4.5]
```

Полиморфные модули первого порядка играют важную роль, потому что они позволяют связывать типы, ассоциированные с модулями первого порядка, с типами других значений, участвующих в вычислениях.

Подробнее о локально-абстрактных типах

Одной из ключевых особенностей локально-абстрактных типов является то обстоятельство, что внутри функции они выглядят как абстрактные, а снаружи – как полиморфные. Взгляните на следующий пример:

OCaml utop (part 19)

<https://github.com/realworldocaml/examples/blob/v1/code/fcm/main.topscript>

```
# let wrap_in_list (type a) (x:a) = [x];;
val wrap_in_list : 'a -> 'a list = <fun>
```

Компиляция этой функции проходит успешно, потому что тип `a` используется таким образом, что оказывается совместимым, будучи абстрактным, но система типов выводит тип функции как полиморфный.

Если, с другой стороны, попробовать использовать тип `a` как эквивалент некоторого конкретного типа, такого как `int`, компилятор сообщит об ошибке:

OCaml utop (part 20)

<https://github.com/realworldocaml/examples/tree/v1/code/fcm/main.topscript>

```
# let double_int (type a) (x:a) = x + x;;
Characters 38-39:
```

```
Error: This expression has type a but an expression was expected of type int
(Ошибка: Это выражение имеет тип a, тогда как ожидалось выражение типа int)
```

Локально-абстрактные типы часто используются для создания новых типов, которые можно задействовать при конструировании модулей. Ниже приводится пример использования такого подхода для создания модуля первого порядка:

OCaml utop (part 21)

<https://github.com/realworldocaml/examples/tree/v1/code/fcm/main.topscript>

```
# module type Comparable = sig
  type t
  val compare : t -> t -> int
end ;;

module type Comparable = sig type t val compare : t -> t -> int end
# let create_comparable (type a) compare =
  (module struct
    type t = a
    let compare = compare
    end : Comparable with type t = a)
  ;;

val create_comparable :
  ('a -> 'a -> int) -> (module Comparable with type t = 'a) = <fun>
# create_comparable Int.compare;;
- : (module Comparable with type t = int) = <module>
# create_comparable Float.compare;;
- : (module Comparable with type t = float) = <module>
```

Здесь мы фактически получаем полиморфный тип и внутри модуля экспортируем его как конкретный тип.

Этот прием может использоваться не только при создании модулей первого порядка. Например, этот же подход можно использовать при конструировании локального модуля для передачи его функтору.

Пример: фреймворк обработки запросов

Теперь рассмотрим модули первого порядка в контексте более полного и практического примера. В частности, рассмотрим следующую сигнатуру модуля, реализующего систему обработки пользовательских запросов.

OCaml utop

https://github.com/realworldocaml/examples/tree/v1/code/fcm/query_handler.topscript

```
# module type Query_handler = sig

  (** Конфигурация обработчика запросов. Отметьте, что она может быть
      преобразована в s-выражение и обратно *)
  type config with sexp

  (** Имя службы обработки запросов *)
  val name : string

  (** Состояние обработчика запросов *)
  type t

  (** Создает новый обработчик запросов на основе config *)
  val create : config -> t

  (** Обработывает запрос, где аргументы и возвращаемое значение
      являются s-выражениями *)
  val eval : t -> Sexp.t -> Sexp.t Or_error.t
```

```

end;;
module type Query_handler =
  sig
    type config
    val name : string
    type t
    val create : config -> t
    val eval : t -> Sexp.t -> Sexp.t Or_error.t
    val config_of_sexp : Sexp.t -> config
    val sexp_of_config : config -> Sexp.t
  end
end

```

Здесь мы использовали запросы и ответы в формате s-выражений, а также конфигурацию обработчика запросов. S-выражения обеспечивают простой, гибкий и удобочитаемый формат сериализации и широко используются в библиотеке Core. Но пока достаточно помнить, что это – выражения в круглых скобках, атомарными значениями которых являются строки, то есть (это (самое настоящее) (s-выражение)).

Кроме того, мы использовали расширение Sexplib синтаксиса языка OCaml, добавляющее в него поддержку объявления `with sexp`. При добавлении к типу в сигнатуре объявление `with sexp` добавляет объявления функций преобразования s-выражений, например:

OCaml utop (part 1)

https://github.com/realworldocaml/examples/tree/v1/code/fcm/query_handler.topscript

```

# module type M = sig type t with sexp end;;
module type M =
  sig type t val t_of_sexp : Sexp.t -> t val sexp_of_t : t -> Sexp.t end

```

В модуле объявление `with sexp` добавляет реализации этих функций. То есть мы можем написать:

OCaml utop (part 2)

https://github.com/realworldocaml/examples/tree/v1/code/fcm/query_handler.topscript

```

# type u = { a: int; b: float } with sexp;;
type u = { a : int; b : float; }
val u_of_sexp : Sexp.t -> u = <fun>
val sexp_of_u : u -> Sexp.t = <fun>
# sexp_of_u {a=3;b=7.};;
- : Sexp.t = ((a 3) (b 7))
# u_of_sexp (Sexp.of_string "((a 43) (b 3.4))");;
- : u = {a = 43; b = 3.4}

```

Обо всем этом подробнее рассказывается в главе 17.

Реализация обработчика запросов

Рассмотрим несколько примеров реализации обработчиков запросов, удовлетворяющих интерфейсу `Query_handler`. Первый пример – обработчик, возвращающий уникальные целочисленные идентификаторы. Он действует как внутренний хранимый счетчик, наращиваемый при каждой попытке получить новое значение.

Роль входного запроса в данном случае будет играть простейшее s-выражение: `()`, иначе известное как `Sexp.unit`:

OCaml utop (part 3)

https://github.com/realworldocaml/examples/tree/v1/code/fcm/query_handler.topscript

```
# module Unique = struct
  type config = int with sexp
  type t = { mutable next_id: int }

  let name = "unique"
  let create start_at = { next_id = start_at }

  let eval t sexp =
    match Or_error.try_with (fun () -> unit_of_sexp sexp) with
    | Error _ as err -> err
    | Ok () ->
      let response = Ok (Int.sexp_of_t t.next_id) in
      t.next_id <- t.next_id + 1;
      response
end;;

module Unique :
sig
  type config = int
  val config_of_sexp : Sexp.t -> config
  val sexp_of_config : config -> Sexp.t
  type t = { mutable next_id : config; }
  val name : string
  val create : config -> t
  val eval : t -> Sexp.t -> (Sexp.t, Error.t) Result.t
end
```

С помощью этого модуля можно создать экземпляр обработчика запросов `Unique` и взаимодействовать с ними напрямую:

OCaml utop (part 4)

https://github.com/realworldocaml/examples/tree/v1/code/fcm/query_handler.topscript

```
# let unique = Unique.create 0;;
val unique : Unique.t = {Unique.next_id = 0}
# Unique.eval unique Sexp.unit;;
- : (Sexp.t, Error.t) Result.t = Ok 0
# Unique.eval unique Sexp.unit;;
- : (Sexp.t, Error.t) Result.t = Ok 1
```

Ниже приводится другой пример: обработчик запросов, возвращающий список содержимого каталога. Здесь `config` хранит каталог по умолчанию, относительно которого откладываются все пути, указываемые в запросах:

OCaml utop (part 5)

https://github.com/realworldocaml/examples/tree/v1/code/fcm/query_handler.topscript

```
# module List_dir = struct
  type config = string with sexp
  type t = { cwd: string }
```

```

(** [is_abs p] Возвращает true, если [p] - абсолютный путь *)
let is_abs p =
  String.length p > 0 && p.[0] = '/'

let name = "ls"
let create cwd = { cwd }

let eval t sexp =
  match Or_error.try_with (fun () -> string_of_sexp sexp) with
  | Error _ as err -> err
  | Ok dir ->
    let dir =
      if is_abs dir then dir
      else Filename.concat t.cwd dir
    in
    Ok (Array.sexp_of_t String.sexp_of_t (Sys.readdir dir))
end;;

module List_dir :
sig
  type config = string
  val config_of_sexp : Sexp.t -> config
  val sexp_of_config : config -> Sexp.t
  type t = { cwd : config; }
  val is_abs : config -> bool
  val name : config
  val create : config -> t
  val eval : t -> Sexp.t -> (Sexp.t, Error.t) Result.t
end

```

И снова с помощью этого модуля можно создать экземпляр этого обработчика запросов и взаимодействовать с ними напрямую:

OCaml utop (part 6)

https://github.com/realworldocaml/examples/tree/v1/code/fcm/query_handler.topscript

```

# let list_dir = List_dir.create "/var";;
val list_dir : List_dir.t = {List_dir.cwd = "/var"}
# List_dir.eval list_dir (sexp_of_string ".");;
- : (Sexp.t, Error.t) Result.t =
Ok (lib mail cache www spool run log lock opt local backups tmp)
# List_dir.eval list_dir (sexp_of_string "yp");;
Exception: (Sys_error "/var/yp: No such file or
directory").

```

Диспетчеризация запросов по нескольким обработчикам

А как быть, если потребуется организовать передачу разных запросов разным обработчикам? В идеале было бы неплохо организовать обработчики в простую структуру данных, такую как список. Это невозможно сделать в случае простых модулей и функторов, но легко и просто, когда обработчики представлены модулями первого порядка. Первое, что следует сделать, – определить сигнатуру, объединяющую модуль `Query_handler` с экземпляром обработчика:

OCaml utop (part 7)

https://github.com/realworldocaml/examples/tree/v1/code/fcm/query_handler.topscript

```
# module type Query_handler_instance = sig
  module Query_handler : Query_handler
  val this : Query_handler.t
end;;

module type Query_handler_instance =
  sig module Query_handler : Query_handler val this : Query_handler.t end
```

Имея эту сигнатуру, мы можем создавать модули первого порядка, включающие и экземпляры обработчиков, и операции сопоставления, проверяющие возможность обработки конкретного запроса.

Создать экземпляр можно так:

OCaml utop (part 8)

https://github.com/realworldocaml/examples/tree/v1/code/fcm/query_handler.topscript

```
# let unique_instance =
  (module struct
    module Query_handler = Unique
    let this = Unique.create 0
  end : Query_handler_instance);;

val unique_instance : (module Query_handler_instance) = <module>
```

Подобный способ создания экземпляров выглядит несколько избыточным, но мы можем написать функцию, избавляющую нас от необходимости снова и снова писать шаблонный код. Обратите внимание, что здесь опять используется локально-абстрактный тип:

OCaml utop (part 9)

https://github.com/realworldocaml/examples/tree/v1/code/fcm/query_handler.topscript

```
# let build_instance
  (type a)
  (module Q : Query_handler with type config = a)
  config
=
  (module struct
    module Query_handler = Q
    let this = Q.create config
  end : Query_handler_instance)
;;

val build_instance :
  (module Query_handler with type config = 'a) ->
  'a -> (module Query_handler_instance) = <fun>
```

Функция `build_instance` позволяет выполнить создание нового экземпляра всего одной строкой кода:

OCaml utop (part 10)

https://github.com/realworldocaml/examples/tree/v1/code/fcm/query_handler.topscript

```
# let unique_instance = build_instance (module Unique) 0;;

val unique_instance : (module Query_handler_instance) = <module>
```

```
# let list_dir_instance = build_instance (module List_dir) "/var";;
val list_dir_instance : (module Query_handler_instance) = <module>
```

Теперь можно приступить к коду, осуществляющему выбор из списка экземпляра обработчика, соответствующего запросу. Допустим, что запрос имеет следующий вид:

Scheme

<https://github.com/realworldocaml/examples/tree/v1/code/fcm/query-syntax>

(имя-запроса запрос)

где имя-запроса используется для определения соответствующего обработчика, а запрос — это тело запроса.

Прежде всего нам необходима функция, принимающая список экземпляров обработчиков запросов и конструирующая таблицу диспетчеризации:

OCaml utop (part 11)

https://github.com/realworldocaml/examples/tree/v1/code/fcm/query_handler.topscript

```
# let build_dispatch_table handlers =
  let table = String.Table.create () in
  List.iter handlers
    ~f:(fun (module I : Query_handler_instance) as instance) ->
      Hashtbl.replace table ~key:I.Query_handler.name ~data:instance);
  table
;;

val build_dispatch_table :
  (module Query_handler_instance) list ->
  (module Query_handler_instance) String.Table.t = <fun>
```

Далее нам нужна функция, выбирающая обработчик из таблицы диспетчеризации:

OCaml utop (part 12)

https://github.com/realworldocaml/examples/tree/v1/code/fcm/query_handler.topscript

```
# let dispatch_dispatch_table name_and_query =
  match name_and_query with
  | Sexp.List [Sexp.Atom name; query] ->
    begin match Hashtbl.find dispatch_table name with
    | None ->
      Or_error.error "Could not find matching handler"
        name String.sexp_of_t
    | Some (module I : Query_handler_instance) ->
      I.Query_handler.eval I.this query
    end
  | _ ->
    Or_error.error_string "malformed query"
  ;;

val dispatch :
  (string, (module Query_handler_instance)) Hashtbl.t ->
  Sexp.t -> Sexp.t Or_error.t = <fun>
```

Эта функция распаковывает экземпляр в модуль `I` и затем использует экземпляр обработчика запросов (`I.this`) совместно со связанным модулем (`I.Query_handler`).

Связывание модуля и значения во многом напоминает объектно-ориентированный стиль программирования. Но есть одно важное отличие – модули первого порядка позволяют упаковывать не только простые функции, или методы. Как показано выше, можно также включать типы и даже модули. Мы использовали здесь лишь малую часть возможностей, которые позволяют создавать намного более сложные компоненты, включающие множество взаимозависимых типов и значений.

Теперь объединим все это в полноценный действующий пример, добавив интерфейс командной строки:

OCaml utop (part 13)

https://github.com/realworldocaml/examples/tree/v1/code/fcm/query_handler.topscript

```
# let rec cli dispatch_table =
  printf ">>> %!";
  let result =
    match In_channel.input_line stdin with
    | None -> `Stop
    | Some line ->
      match Or_error.try_with (fun () -> Sexp.of_string line) with
      | Error e -> `Continue (Error.to_string_hum e)
      | Ok (Sexp.Atom "quit") -> `Stop
      | Ok query ->
        begin match dispatch dispatch_table query with
        | Error e -> `Continue (Error.to_string_hum e)
        | Ok s -> `Continue (Sexp.to_string_hum s)
        end;
  in
  match result with
  | `Stop -> ()
  | `Continue msg ->
    printf "%s\n%!" msg;
    cli dispatch_table
;;
val cli : (string, (module Query_handler_instance)) Hashtbl.t -> unit = <fun>
```

Самый эффективный путь запустить этот интерфейс командной строки в работу – поместить код, что приводится выше, в отдельный файл программы вместе со следующей командой, запускающей интерфейс:

OCaml (part 1)

https://github.com/realworldocaml/examples/tree/v1/code/fcm/query_handler.ml

```
let () =
  cli (build_dispatch_table [unique_instance; list_dir_instance])
```

Ниже приводится пример сеанса работы с этой программой:

OCaml utop

https://github.com/realworldocaml/examples/tree/v1/code/fcm/query_example.rawscript

```
$ ./query_handler.byte
>>> (unique ())
0
```



```
>>> (unique ())
1
>>> (ls .)
(agentx at audit backups db empty folders jabberd lib log mail msgs named
 netboot pgsq_socket_alt root rpc run rwho spool tmp vm yp)
>>> (ls vm)
(sleepimage swapfile0 swapfile1 swapfile2 swapfile3 swapfile4 swapfile5
 swapfile6)
```

Загрузка и выгрузка обработчиков запросов

Одно из преимуществ модулей первого порядка заключается в их гибкости и динамизме. Например, совсем несложно изменить нашу реализацию так, чтобы она загружала и выгружала обработчики запросов во время выполнения.

Сделаем это, создав обработчик запроса, с помощью которого можно будет управлять множеством активных обработчиков. Определим модуль `Loader`, конфигурацией которого будет служить список известных модулей `Query_handler`. Ниже приводятся объявления основных типов:

OCaml (part 1)

https://github.com/realworldocaml/examples/tree/v1/code/fcm/query_handler_core.ml

```
module Loader = struct
  type config = (module Query_handler) list sexp_opaque
  with sexp

  type t = { known : (module Query_handler) String.Table.t
            ; active : (module Query_handler_instance) String.Table.t
            }

  let name = "loader"
```

Обратите внимание, что тип `Loader.t` включает две таблицы: одна содержит список всех известных модулей обработчиков запросов, а другая – список активных экземпляров обработчиков. Тип `Loader.t` будет отвечать за создание новых экземпляров и добавление их в таблицу, а также за удаление экземпляров. Обе операции будут выполняться в ответ на запросы пользователя.

Теперь определим функцию, создающую экземпляр `Loader.t`. Она будет принимать список известных модулей обработчиков запросов. Отметьте, что таблица активных модулей обработчиков изначально пуста:

OCaml (part 2)

https://github.com/realworldocaml/examples/tree/v1/code/fcm/query_handler_core.ml

```
let create known_list =
  let active = String.Table.create () in
  let known = String.Table.create () in
  List.iter known_list
    ~f:(fun (module Q : Query_handler) as q) ->
      Hashtbl.replace known ~key:Q.name ~data:q;
  { known; active }
```

Далее реализуем функции управления таблицей активных обработчиков запросов. Начнем с функции загрузки экземпляра. Обратите внимание, что она принимает два аргумента: имя обработчика и конфигурацию для экземпляра этого обработчика в форме *s*-выражения. Эти аргументы используются функцией для создания модуля первого порядка типа `(module Query_handler_instance)`, который затем добавляется в таблицу активных обработчиков:

OCaml (part 3)

https://github.com/realworldocaml/examples/blob/v1/code/fcm/query_handler_core.ml

```
let load t handler_name config =
  if Hashtbl.mem t.active handler_name then
    Or_error.error "Can't re-register an active handler"
    handler_name String.sexp_of_t
  else
    match Hashtbl.find t.known handler_name with
    | None ->
      Or_error.error "Unknown handler" handler_name String.sexp_of_t
    | Some (module Q : Query_handler) ->
      let instance =
        (module struct
          module Query_handler = Q
          let this = Q.create (Q.config_of_sexp config)
          end : Query_handler_instance)
      in
      Hashtbl.replace t.active ~key:handler_name ~data:instance;
      Ok Sexp.unit
```

Функция `load` будет отвергать попытки загрузить уже активные обработчики. Но нам также необходимо иметь возможность выгружать обработчики. Обратите внимание, что обработчик, обрабатывающий запросы управления таблицами, явно отказывается выгрузить самого себя:

OCaml (part 4)

https://github.com/realworldocaml/examples/blob/v1/code/fcm/query_handler_core.ml

```
let unload t handler_name =
  if not (Hashtbl.mem t.active handler_name) then
    Or_error.error "Handler not active" handler_name String.sexp_of_t
  else if handler_name = name then
    Or_error.error_string "It's unwise to unload yourself"
  else (
    Hashtbl.remove t.active handler_name;
    Ok Sexp.unit
  )
```

Наконец, нужно реализовать функцию `eval`, определяющую интерфейс обработки запросов, предоставляемый пользователю. Мы сделаем это, создав альтернативный тип и задействовав функцию преобразования *s*-выражения, сгенерированную для этого типа, для анализа запроса:

OCaml (part 5)

https://github.com/realworldocaml/examples/blob/v1/code/fcm/query_handler_core.ml

```
type request =
  | Load of string * Sexp.t
  | Unload of string
  | Known_services
  | Active_services
with sexp
```

Функция `eval` сама по себе очень проста, она выбирает функцию, соответствующую типу полученного запроса. Обратите внимание, что мы использовали конструкцию `<:sexp_of<string list>>`, чтобы автоматически сгенерировать функцию для преобразования списка строк в s-выражение, которая будет описываться в главе 17.

Эта функция завершается определением модуля `Loader`:

OCaml (part 6)

https://github.com/realworldocaml/examples/blob/v1/code/fcm/query_handler_core.ml

```
let eval t sexp =
  match Or_error.try_with (fun () -> request_of_sexp sexp) with
  | Error _ as err -> err
  | Ok resp ->
    match resp with
    | Load (name, config) -> load t name config
    | Unload name -> unload t name
    | Known_services ->
      Ok (<:sexp_of<string list>> (Hashtbl.keys t.known))
    | Active_services ->
      Ok (<:sexp_of<string list>> (Hashtbl.keys t.active))
end
```

В заключение объединим все вместе с интерфейсом командной строки. Сначала создадим экземпляр загрузчика обработчиков запросов, а потом добавим экземпляр загрузчика в таблицу активных обработчиков. Затем остается только запустить интерфейс командной строки, передав ему таблицу активных обработчиков:

OCaml (part 1)

https://github.com/realworldocaml/examples/blob/v1/code/fcm/query_handler_loader.ml

```
let () =
  let loader = Loader.create [(module Unique); (module List_dir)] in
  let loader_instance =
    (module struct
      module Query_handler = Loader
      let this = loader
    end : Query_handler_instance)
  in
  Hashtbl.replace loader.Loader.active
    ~key:Loader.name ~data:loader_instance;
  cli loader.Loader.active
```

Теперь скомпилируем программу и поэкспериментируем с ней:

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/fcm/build_query_handler_loader.out

```
$ corebuild query_handler_loader.byte
```

Получившийся интерфейс командной строки действует в полном соответствии с нашими ожиданиями: сразу после запуска в нем отсутствуют активные обработчики, но он дает возможность загружать и выгружать их. Ниже приводится пример сеанса работы с программой. Как видно из этого примера, сразу после запуска в нашем распоряжении имеется только один активный обработчик – сам загрузчик loader:

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/fcm/loader_cli1.out

```
$ ./query_handler_loader.byte
>>> (loader known_services)
(ls unique)
>>> (loader active_services)
(loader)
```

Любые попытки использовать неактивные обработчики терпят неудачу:

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/fcm/loader_cli2.out

```
>>> (ls .)
Could not find matching handler: ls
```

Но мы можем загрузить обработчик ls, определив его конфигурацию по своему выбору, после чего он станет доступен для использования, а затем выгрузить его, сделав опять недоступным, и вновь загрузить с другой конфигурацией:

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/fcm/loader_cli3.out

```
>>> (loader (load ls /var))
()
>>> (ls /var)
(agentx at audit backups db empty folders jabberd lib log mail msgs named
netboot pgsql_socket_alt root rpc run rwho spool tmp vm yp)
>>> (loader (unload ls))
()
>>> (ls /var)
Could not find matching handler: ls
```

Самое примечательное, что сам загрузчик не может быть загружен (так как он отсутствует в списке известных обработчиков) и не может быть выгружен:

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/fcm/loader_cli4.out

```
>>> (loader (unload loader))
It's unwise to unload yourself
```

Мы не будем вдаваться в подробное описание тонкостей, тем не менее отметим, что можно получить еще больший динамизм, воспользовавшись средствами динамического связывания, имеющимися в языке OCaml, которые позволяют компилировать и внедрять новый код в действующую программу. Эту процедуру можно автоматизировать с помощью таких библиотек, как `ocaml_plugin`, которые можно установить с помощью инструмента управления пакетами OPAM, автоматизирующего большую часть операций, имеющих отношение к динамическому связыванию.

Жизнь без модулей первого порядка

Справедливости ради следует сказать, что почти все, что возможно с применением модулей первого порядка, возможно и без них, правда, для этого придется пожертвовать ясностью и удобочитаемостью кода. Например, мы могли бы переписать реализацию обработки запросов и без использования модулей первого порядка:

OCaml utop (part 14)

https://github.com/realworldocaml/examples/blob/v1/code/fcm/query_handler.topscript

```
# type query_handler_instance = { name : string
                                ; eval : Sexp.t -> Sexp.t Or_error.t
                                }

type query_handler = Sexp.t -> query_handler_instance
;;

type query_handler_instance = {
  name : string;
  eval : Sexp.t -> Sexp.t Or_error.t;
}

type query_handler = Sexp.t -> query_handler_instance
```

Идея состоит в том, чтобы скрыть истинные типы объектов за интерфейсами функций, хранимых в замыканиях. Так, например, мы могли бы реализовать обработчик `Unique`, как показано ниже:

OCaml utop (part 15)

https://github.com/realworldocaml/examples/blob/v1/code/fcm/query_handler.topscript

```
# let unique_handler config_sexp =
  let config = Unique.config_of_sexp config_sexp in
  let unique = Unique.create config in
  { name = Unique.name
    ; eval = (fun config -> Unique.eval unique config)
  }
;;

val unique_handler : Sexp.t -> query_handler_instance = <fun>
```

С этой точки зрения данный подход выглядит вполне приемлемо, и вполне возможно обойтись и без модулей первого порядка. Но чем больший объем функциональности необходимо будет скрыть в замыканиях и чем сложнее будут взаимоотношения с разными типами, тем более неуклюжим выглядит этот подход и тем предпочтительнее выглядят модули первого порядка.

Мы уже познакомились с некоторыми инструментами OCaml для организации программ, особенно долго обсуждали модули. Однако OCaml поддерживает также объектно-ориентированный стиль программирования. В языке существуют объекты, классы и связанные с ними типы. В этой главе мы познакомим вас с объектами и подтипами в OCaml, а в следующей – с классами и наследованием.

Что есть объектно-ориентированное программирование?

Объектно-ориентированное программирование (часто сокращается до ООП) – это стиль программирования, инкапсулирующий вычисления и данные в логически организованные объекты. Каждый объект содержит некоторые данные в своих полях и имеет методы – функции, выполняющие операции с этими данными (вызов метода также называют «отправкой сообщения» объекту). Определение программного кода, стоящего за объектами, называют классом. Объекты конструируются на основе определений классов вызовом конструктора с данными, которые объект будет использовать для построения самого себя.

ООП имеет пять фундаментальных характеристик, отличающих его от других стилей:

- абстрагирование – тонкости реализации скрываются в недрах объектов, а внешний интерфейс – это всего лишь множество общедоступных методов;
- динамическое связывание – когда объекту посылается сообщение, вызываемый метод определяется реализацией объекта, а не некоторым статическим свойством программы. Иными словами, разные объекты могут реагировать на одно и то же сообщение по-разному;
- поддержка подтипов – если объект *a* обладает всеми функциональными возможностями объекта *b*, тогда мы можем использовать объект *a* в тех же контекстах, что и объект *b*;
- наследование – определение одного вида объектов можно повторно использовать для создания других разновидностей объектов. Новое определение может переопределять некоторые черты поведения, но также может использовать код своего предка;
- открытая рекурсия – методы объекта могут вызывать другие методы того же объекта с использованием специальной переменной (часто с именем *self* или *this*). Когда объекты создаются из классов, эти вызовы используют механизм динамического связывания, что дает возможность из методов, определенных в одном классе, вызывать методы, определенные в другом классе, наследующем первый.

Почти каждый современный язык программирования испытывает влияние ООП, и вы еще будете сталкиваться с этими терминами, если вам доведется использовать C++, Java, C#, Ruby, Python или JavaScript.

Объекты OCaml

Если вы уже знакомы с объектно-ориентированным программированием по таким языкам, как Java или C++, тогда объектная система OCaml станет для вас сюрпризом. Самым необычным является полное отделение объектов и их типов

от системы классов. В таких языках, как Java, имя класса одновременно является и типом объекта, созданного на основе этого класса, а отношения между типами этих объектов соответствуют наследованию. Например, если реализовать класс `Deque` на Java, наследующий класс `Stack`, мы сможем использовать экземпляр класса `Deque` везде, где ожидается экземпляр класса `Stack`.

В языке OCaml дела обстоят совсем иначе. Классы применяются для конструирования объектов и поддерживают наследование, но классы не являются типами. Каждый объект имеет свой *тип объекта*, и если потребуется использовать объекты в программе, вы вообще можете не использовать классы. Ниже приводится пример простого объекта:

OCaml utop (part 1)

<https://github.com/realworldocaml/examples/tree/v1/code/objects/stack.topscript>

```
# let s = object
  val mutable v = [0; 2]

  method pop =
    match v with
    | hd :: tl ->
      v <- tl;
      Some hd
    | [] -> None

  method push hd =
    v <- hd :: v
end ;;
val s : < pop : int option; push : int -> unit > = <obj>
```

Объект имеет значение в виде списка `v` целых чисел, метод `pop`, возвращающий голову (первый элемент) списка `v`, и метод `push`, добавляющий целое число в начало списка `v`.

Тип объекта заключен в угловые скобки `< ... >` и содержит только типы методов. Поля, такие как `v`, не являются частью общедоступного интерфейса объекта. Все взаимодействия с объектом выполняются только посредством его методов. Синтаксис вызова методов основан на использовании символа `#`:

OCaml utop (part 2)

<https://github.com/realworldocaml/examples/tree/v1/code/objects/stack.topscript>

```
# s#pop ;;
- : int option = Some 0
# s#push 4 ;;
- : unit = ()
# s#pop ;;
- : int option = Some 4
```

Обратите внимание, что, в отличие от функций, методы могут иметь нулевое число параметров, потому что вызов объекта производится относительно конкретного экземпляра объекта. Именно поэтому в определении метода `pop` отсутствует аргумент `unit`, который необходим для эквивалентной функциональной версии.

Объекты могут также создаваться с помощью функций. Если необходимо указать начальное значение объекта, можно определить функцию, принимающую значение и возвращающую объект:

OCaml utop (part 3)

<https://github.com/realworldocaml/examples/tree/v1/code/objects/stack.topscript>

```
# let stack init = object
  val mutable v = init

  method pop =
    match v with
    | hd :: tl ->
      v <- tl;
      Some hd
    | [] -> None

  method push hd =
    v <- hd :: v
end ;;

val stack : 'a list -> < pop : 'a option; push : 'a -> unit > = <fun>
# let s = stack [3; 2; 1] ;;
val s : < pop : int option; push : int -> unit > = <obj>
# s#pop ;;
- : int option = Some 3
```

Обратите внимание, что теперь функция `stack` принимает и возвращает значения полиморфного типа `'a`. Когда выполняется вызов `stack` с конкретным значением `[3; 2; 1]`, она возвращает значение того же объектного типа, что и прежде, — со значениями типа `int` на стеке.

Полиморфизм объектов

Подобно полиморфным вариантам, в объявлениях методов можно не указывать типы явно:

OCaml utop (part 1)

<https://github.com/realworldocaml/examples/tree/v1/code/objects/polymorphism.topscript>

```
# let area sq = sq#width * sq#width ;;
val area : < width : int; .. > -> int = <fun>
# let minimize sq : unit = sq#resize 1 ;;
val minimize : < resize : int -> unit; .. > -> unit = <fun>
# let limit sq =
  if (area sq) > 100 then minimize sq ;;
val limit : < resize : int -> unit; width : int; .. > -> unit = <fun>
```

Как видите, типы объектов выводятся автоматически из контекста их вызова.

Если система типов обнаружит несовместимые способы использования одного и того же метода, она сообщит об этом:

OCaml utop (part 2)

<https://github.com/realworldocaml/examples/tree/v1/code/objects/polymorphism.topscript>

```
# let toggle sq b : unit =
  if b then sq#resize `Fullscreen
  else minimize sq ;;
```

Characters 80-82:

```
Error: This expression has type < resize : [> `Fullscreen ] -> unit; .. >
but an expression was expected of type < resize : int -> unit; .. >
Types for method resize are incompatible
(Ошибка: Это выражение имеет тип < resize : [> `Fullscreen ] -> unit; .. >
тогда как ожидалось выражение типа < resize : int -> unit; .. >
Типы для метода resize несовместимы.)
```

Две точки (..) в типе, выведенном компилятором, – это многоточие, обозначающее другие неуказанные методы, которые могут иметься у объекта. Тип `< width : float; .. >` указывает, что объект должен иметь как минимум метод `width` и, возможно, некоторые другие методы. Такие типы объектов называют *открытыми*.

Можно вручную *закрывать* тип объекта, добавив аннотацию типа:

OCaml utop (part 3)

<https://github.com/realworldocaml/examples/tree/v1/code/objects/polymorphism.topscript>

```
# let area_closed (sq: < width : int >) = sq#width * sq#width ;;
val area_closed : < width : int > -> int = <fun>
# let sq = object
  method width = 30
  method name = "sq"
end ;;
val sq : < name : string; width : int > = <obj>
# area_closed sq ;;
```

Characters 12-14:

```
Error: This expression has type < name : string; width : int >
but an expression was expected of type < width : int >
The second object type has no method name
(Ошибка: Это выражение имеет тип < name : string; width : int >
тогда как ожидалось выражение типа < width : int >
Тип второго объекта не имеет метода name.)
```

Элизии полиморфны

Многоточие .. в открытом типе объекта – это элизия (пропуск) и обозначает «возможно, есть еще методы». Может быть это не очевидно, но пропущенный тип объекта фактически является полиморфным. Например, если попытаться записать определение типа, компилятор выдаст сообщение об ошибке «unbound type variable» (несвязанная переменная типа):

OCaml utop (part 4)

<https://github.com/realworldocaml/examples/tree/v1/code/objects/polymorphism.topscript>

```
# type square = < width : int; .. > ;;
```

Characters 5-32:

```
Error: A type variable is unbound in this type declaration.
In type < width : int; .. > as 'a the variable 'a is unbound
(Ошибка: Переменная типа в этом объявлении типа не связана.
В типе < width : int; .. > как 'a переменная 'a не связана.)
```

Это обусловлено тем, что в действительности многоточие `..` является особой разновидностью переменной типа, которая называется рядной переменной (row variable). Такая схема типов с применением строчных переменных называется рядным полиморфизмом (row polymorphism). Рядный полиморфизм используется также в полиморфных вариантных типах, а объекты и полиморфные варианты имеют много общего: объекты соотносятся с записями, как полиморфные варианты с обычными вариантами.

Объект типа `< pop : int option; .. >` может быть любым объектом с методом `pop : int option`, и не важно, как он реализован. При вызове метода `#pop` выбор фактического метода, который должен быть вызван, определяется объектом:

OCaml utop (part 4)

<https://github.com/realworldocaml/examples/tree/v1/code/objects/stack.topscript>

```
# let print_pop st = Option.iter ~f:(printf "Popped: %d\n") st#pop ;;
val print_pop : < pop : int option; .. > -> unit = <fun>
# print_pop (stack [5;4;3;2;1]) ;;
Popped: 5
- : unit = ()
# let t = object
    method pop = Some (Float.to_int (Time.to_float (Time.now ())))
  end ;;
val t : < pop : int option > = <obj>
# print_pop t ;;
Popped: 1376833904
- : unit = ()
```

Неизменяемые объекты

Многие рассматривают объектно-ориентированное программирование как исключительно императивное, когда объекты действуют подобно конечному автомату (state machine). Передача сообщения объекту вызывает изменение его состояния и может приводить к передаче сообщений другим объектам.

В действительности во многих программах такое поведение вполне оправдано, но это не означает, что оно является обязательным. Давайте определим функцию, создающую неизменяемый стек объектов:

OCaml utop (part 1)

<https://github.com/realworldocaml/examples/blob/v1/code/objects/immutable.topscript>

```
# let imm_stack init = object
    val v = init

    method pop =
      match v with
      | hd :: tl -> Some (hd, {< v = tl >})
      | [] -> None

    method push hd =
      {< v = hd :: v >}
  end ;;
val imm_stack :
  'a list -> (< pop : ('a * 'b) option; push : 'a -> 'b > as 'b) = <fun>
```

Ключевыми в этой реализации являются методы `pop` и `push`. Выражение `{< ... >}` возвращает копию текущего объекта того же типа и с измененными значениями указанных полей. Иными словами, метод `push hd` возвращает копию объекта, в котором значение `v` замещается значением `hd :: v`. Исходный объект при этом не изменяется:

OCaml utop (part 2)

<https://github.com/realworldocaml/examples/blob/v1/code/objects/immutable.topscript>

```
# let s = imm_stack [3; 2; 1] ;;
val s : < pop : (int * 'a) option; push : int -> 'a > as 'a = <obj>
# let t = s#push 4 ;;
val t : < pop : (int * 'a) option; push : int -> 'a > as 'a = <obj>
# s#pop ;;
- : (int * (< pop : 'a; push : int -> 'b > as 'b)) option as 'a =
Some (3, <obj>)
# t#pop ;;
- : (int * (< pop : 'a; push : int -> 'b > as 'b)) option as 'a =
Some (4, <obj>)
```

Существуют некоторые ограничения на применение выражения `{< ... >}`. Оно может использоваться только в теле метода и может изменять только значения полей. Реализации методов фиксируются в момент создания объекта – они не могут изменяться динамически.

Когда следует использовать объекты

Кого-то может волновать вопрос: «Когда следует использовать объекты в OCaml?», – в противовес которому есть множество альтернативных механизмов, позволяющих выражать похожие понятия. Модули первого порядка более выразительны (модуль может включать типы, тогда как классы и объекты – нет). Модули, функторы и типы данных также предлагают широкий выбор способов выражения структуры программы. Фактически многие опытные программисты на OCaml редко используют классы и объекты, если вообще используют.

Объекты имеют некоторые преимущества перед записями: они не требуют определения типов, а поддержка рядного полиморфизма увеличивает их гибкость. Однако сложный синтаксис и дополнительные накладные расходы во время выполнения не способствуют использованию объектов взамен записей.

Основные выгоды от применения объектов вытекают из системы классов. Классы поддерживают наследование и открытую рекурсию. Открытая рекурсия позволяет определять взаимозависимые части объекта по отдельности. Это возможно потому, что вызовы между методами объекта определяются после создания экземпляра объекта, то есть имеет место *позднее* связывание. Это делает возможным ссылаться из одного метода на другие методы объекта, не имея представления о том, как они будут реализованы.

В модулях, напротив, используется раннее связывание. Если у вас возникнет желание параметризовать код модуля так, чтобы получить возможность реали-

зовать некоторые его части позже, вам придется написать функцию или функтор. Этот прием является более явным, но и более сложным, чем переопределение методов в классе.

В общем случае можно порекомендовать использовать классы и объекты в ситуациях, когда открытая рекурсия сулит большие выгоды. Отличными примерами могут служить библиотека `Cryptokit`, созданная Ксавье Леруа (Xavier Leroy), включающая различные криптографические примитивы, которые можно комбинировать как строительные блоки, и библиотека `Camlimages`, реализующая операции с различными графическими форматами. Кроме того, в состав `Camlimages` входит также версия той же самой библиотеки на основе модулей, что дает возможность выбора между объектно-ориентированным и функциональным стилями в зависимости от области применения.

С классами и дополнительными примерами использования открытой рекурсии вы познакомитесь в главе 12.

Подтипизация

Подтипы являются центральной концепцией объектно-ориентированного программирования. Она управляет использованием объекта типа *A* в выражениях, где предполагается использовать объект типа *B*. Если такое возможно, говорят, что *A является подтипом B*. Более конкретно: концепция подтипов ограничивает область применения оператора приведения типа `e :> t`. Такое приведение возможно, только если тип объекта *e* является подтипом объекта *t*.

Подтипизация в ширину

В качестве примера определим некоторые простые типы объектов, описывающие геометрические фигуры. Обобщенный тип `shape` имеет метод вычисления площади, а типы `square` и `circle` являются специализированными разновидностями типа `shape`:

OCaml (part 1)

<https://github.com/realworldocaml/examples/blob/v1/code/objects/subtyping.ml>

```
type shape = < area : float >

type square = < area : float; width : int >

let square w = object
  method area = Float.of_int (w * w)
  method width = w
end

type circle = < area : float; radius : int >

let circle r = object
  method area = 3.14 *. (Float.of_int r) ** 2.0
  method radius = r
end
```

Тип `square` имеет метод `area`, как и тип `shape`, и дополнительный метод `width`. Тем не менее мы предполагаем, что квадрат (`square`) является фигурой (`shape`), и это так и есть. Приведение типа `:` должно выполняться явно:

OCaml utop (part 1)

<https://github.com/realworldocaml/examples/blob/v1/code/objects/subtyping.topscript>

```
# let shape w : shape = square w ;;
Characters 22-30:
Error: This expression has type < area : float; width : int >
      but an expression was expected of type shape
      The second object type has no method width
(Ошибка: Это выражение имеет тип < area : float; width : int >
      тогда как ожидалось выражение типа shape
      Тип второго объекта не имеет метода width.)
# let shape w : shape = (square w :> shape) ;;
val shape : int -> shape = <fun>
```

Такого рода подтипизация называется подтипизацией *в ширину*. То есть тип *A* является подтипом *B*, если *A* имеет все методы типа *B* и, возможно, какие-то дополнительные методы. Тип `square` является подтипом типа `shape`, потому что он реализует все методы типа `shape` (метод `area`).

Подтипизация в глубину

Объекты также поддерживают подтипизацию в глубину. Поддержка подтипизации в глубину позволяет приводить типы объектов, если возможно безопасное приведение типов их отдельных методов. Так, тип объекта `<m: t1>` является подтипом типа `<m: t2>`, если `t1` является подтипом `t2`.

Например, можно создать два объекта с методом `shape`:

OCaml utop (part 2)

<https://github.com/realworldocaml/examples/blob/v1/code/objects/subtyping.topscript>

```
# let coin = object
  method shape = circle 5
  method color = "silver"
end ;;
val coin : < color : string; shape : < area : float; radius : int > > = <obj>
# let map = object
  method shape = square 10
end ;;
val map : < shape : < area : float; width : int > > = <obj>
```

Оба объекта имеют метод `shape`, тип которого является подтипом типа `shape`, то есть оба этих объекта можно привести к типу `< shape : shape >`:

OCaml utop (part 3)

<https://github.com/realworldocaml/examples/blob/v1/code/objects/subtyping.topscript>

```
# type item = < shape : shape > ;;
type item = < shape : shape >
# let items = [ (coin :> item) ; (map :> item) ] ;;
val items : item list = [<obj>; <obj>]
```

Полиморфные подтипы вариантов

Подтипизацию можно также использовать для приведения полиморфных вариантных типов к более широким полиморфным вариантным типам. Полиморфный вариантный тип A является подтипом типа B, если теги типа A образуют подмножество тегов типа B:

OCaml utop (part 4)

<https://github.com/realworldocaml/examples/blob/v1/code/objects/subtyping.topscript>

```
# type num = [ `Int of int | `Float of float ] ;;
type num = [ `Float of float | `Int of int ]
# type const = [ num | `String of string ] ;;
type const = [ `Float of float | `Int of int | `String of string ]
# let n : num = `Int 3 ;;
val n : num = `Int 3
# let c : const = (n :> const) ;;
val c : const = `Int 3
```

Вариантность

А что можно сказать о типах, созданных на основе типов объектов? Если исходить из того, что квадрат (square) является фигурой (shape), естественно было бы ожидать, что список квадратов (square list) будет являться списком фигур (shape list). И действительно, OCaml допускает такое приведение:

OCaml utop (part 5)

<https://github.com/realworldocaml/examples/blob/v1/code/objects/subtyping.topscript>

```
# let squares: square list = [ square 10; square 20 ] ;;
val squares : square list = [<obj>; <obj>]
# let shapes: shape list = (squares :> shape list) ;;
val shapes : shape list = [<obj>; <obj>]
```

Отметьте, что данный пример опирается на неизменяемость списков. Было бы небезопасно интерпретировать массив квадратов (square array) как массив фигур (shape array), потому что это позволило бы сохранять фигуры других типов (не square) в контейнере, который должен быть массивом квадратов. OCaml распознает подобные ситуации и не позволяет выполнять подобного приведения:

OCaml utop (part 6)

<https://github.com/realworldocaml/examples/blob/v1/code/objects/subtyping.topscript>

```
# let square_array: square array = [| square 10; square 20 |] ;;
val square_array : square array = [|<obj>; <obj>|]
# let shape_array: shape array = (square_array :> shape array) ;;
Characters 31-60:
```

```
Error: Type square array is not a subtype of shape array
      Type square = < area : float; width : int >
      is not compatible with type shape = < area : float >
      The second object type has no method width
(Ошибка: Тип square array не является подтипом shape array
      Тип square = < area : float; width : int >
      несовместим с типом shape = < area : float >
      Тип второго объекта не имеет метода width)
```

Мы говорим, что тип `'a list` является *ковариантным* (*covariant*) в `'a`, а тип `'a array` — *инвариантным* (*invariant*).

Подтипизация типов функций требует наличия третьего класса вариантности. Функция с типом `square -> string` не может использоваться с типом `shape -> string`, потому что она ожидает, что ее аргумент будет иметь тип `square`, и не знает, например, как поступить с типом `circle`. Однако функция с типом `shape -> string` безопасно может использоваться с типом `square -> string`:

OCaml utop (part 7)

<https://github.com/realworldocaml/examples/blob/v1/code/objects/subtyping.topscript>

```
# let shape_to_string: shape -> string =
  fun s -> sprintf "Shape(%F)" s#area ;;
val shape_to_string : shape -> string = <fun>
# let square_to_string: square -> string =
  (shape_to_string :> square -> string) ;;
val square_to_string : square -> string = <fun>
```

Мы говорим, что `'a -> string` является *контравариантным* (*contravariant*) в `'a`. В общем случае типы функций считаются контравариантными в своих аргументах и ковариантными в результатах.

Аннотации вариантности

OCaml определяет вариантность типа, используя его определение:

OCaml utop (part 8)

<https://github.com/realworldocaml/examples/blob/v1/code/objects/subtyping.topscript>

```
# module Either = struct
  type ('a, 'b) t =
    | Left of 'a
    | Right of 'b
  let left x = Left x
  let right x = Right x
end ;;
module Either :
sig
  type ('a, 'b) t = Left of 'a | Right of 'b
  val left : 'a -> ('a, 'b) t
  val right : 'a -> ('b, 'a) t
end
# (Either.left (square 40) :> (shape, shape) Either.t) ;;
- : (shape, shape) Either.t = Either.Left <obj>
```

Однако если определение скрыто за сигнатурой, тогда OCaml вынужден угадывать вариантность типа:

OCaml utop (part 9)

<https://github.com/realworldocaml/examples/blob/v1/code/objects/subtyping.topscript>

```
# module AbstractEither : sig
  type ('a, 'b) t
  val left: 'a -> ('a, 'b) t
  val right: 'b -> ('a, 'b) t
end = Either ;;
```

```

module AbstractEither :
  sig
    type ('a, 'b) t
    val left : 'a -> ('a, 'b) t
    val right : 'b -> ('a, 'b) t
  end
# (AbstractEither.left (square 40) :> (shape, shape) AbstractEither.t) ;;
Characters 1-32:
Error: This expression cannot be coerced to type
      (shape, shape) AbstractEither.t;
it has type (< area : float; width : int >, 'a) AbstractEither.t
but is here used with type (shape, shape) AbstractEither.t
Type < area : float; width : int > is not compatible with type
shape = < area : float >
The second object type has no method width
(Ошибка: Это выражение не может быть приведено к типу
      (shape, shape) AbstractEither.t;
оно имеет тип (< area : float; width : int >, 'a) AbstractEither.t,
а здесь используется с типом (shape, shape) AbstractEither.t
Тип < area : float; width : int > несовместим с типом
shape = < area : float >
Тип второго объекта не имеет метода width)

```

Исправить эту проблему можно добавлением аннотаций вариантности к параметрам типов в сигнатуре: + for covariance или - for contravariance:

OCaml utop (part 10)

<https://github.com/realworldocaml/examples/blob/v1/code/objects/subtyping.topscript>

```

# module VarEither : sig
  type (+'a, +'b) t
  val left: 'a -> ('a, 'b) t
  val right: 'b -> ('a, 'b) t
end = Either ;;
module VarEither :
  sig
    type (+'a, +'b) t
    val left : 'a -> ('a, 'b) t
    val right : 'b -> ('a, 'b) t
  end
# (VarEither.left (square 40) :> (shape, shape) VarEither.t) ;;
- : (shape, shape) VarEither.t = <abstr>

```

Чтобы получить более конкретное представление о вариантности, давайте создадим несколько стеков с фигурами, применив функцию `stack` к объектам `square` и `circle`:

OCaml (part 2)

<https://github.com/realworldocaml/examples/blob/v1/code/objects/subtyping.ml>

```

type 'a stack = < pop: 'a option; push: 'a -> unit >

let square_stack: square stack = stack [square 30; square 10]

let circle_stack: circle stack = stack [circle 20; circle 40]

```


Если бы потребовалось написать функцию, принимающую список таких стеков и вычисляющую общую площадь всех фигур в них, мы могли бы попробовать такой способ:

OCaml utop (part 11)

<https://github.com/realworldocaml/examples/blob/v1/code/objects/subtyping.topscript>

```
# let total_area (shape_stacks: shape stack list) =
  let stack_area acc st =
    let rec loop acc =
      match st#pop with
      | Some s -> loop (acc +. s#area)
      | None -> acc
    in
    loop acc
  in
  List.fold ~init:0.0 ~f:stack_area shape_stacks ;;
val total_area : shape stack list -> float = <fun>
```

Однако при попытке применить эту функцию к нашим объектам мы получим сообщение об ошибке:

OCaml utop (part 12)

<https://github.com/realworldocaml/examples/blob/v1/code/objects/subtyping.topscript>

```
# total_area [(square_stack :> shape stack); (circle_stack :> shape stack)] ;;
Characters 12-41:
Error: Type square stack = < pop : square option; push : square -> unit >
      is not a subtype of
      shape stack = < pop : shape option; push : shape -> unit >
      Type shape = < area : float > is not a subtype of
      square = < area : float; width : int >
(Ошибка: Тип square stack = < pop : square option; push : square -> unit >
не является подтипом
shape stack = < pop : shape option; push : shape -> unit >
Тип shape = < area : float > не является подтипом
square = < area : float; width : int >)
```

Как видите, типы `square stack` и `circle stack` не являются подтипами `shape stack`. Проблема заключена в методе `push`. Для `shape stack` метод `push` принимает произвольную фигуру (`shape`). Поэтому, если было бы можно привести тип `square stack` к типу `shape stack`, тогда было бы можно втолкнуть произвольную фигуру в стек `square stack`, что определенно является ошибкой.

С другой стороны, тип `< push: 'a -> unit; .. >` является контравариантным в `'a`, поэтому `< push: square -> unit; pop: square option >` не может быть подтипом `< push: shape -> unit; pop: shape option >`.

И все же реализация функции `total_area` вполне возможна. Она не вызывает `push`, поэтому она не порождает данную ошибку. Чтобы помочь ей заработать, необходимо использовать более точный тип, показывающий, что мы не собираемся использовать метод записи. Для этого определим тип `readonly_stack` и подтвердим возможность приведения к нему списка стеков:

OCaml utop (part 13)

<https://github.com/realworldocaml/examples/blob/v1/code/objects/subtyping.topscript>

```
# type 'a readonly_stack = < pop : 'a option > ;;
type 'a readonly_stack = < pop : 'a option >
# let total_area (shape_stacks: shape readonly_stack list) =
  let stack_area acc st =
    let rec loop acc =
      match st#pop with
      | Some s -> loop (acc +. s#area)
      | None -> acc
    in
    loop acc
  in
  List.fold ~init:0.0 ~f:stack_area shape_stacks ;;
val total_area : shape readonly_stack list -> float = <fun>
# total_area [(square_stack :> shape readonly_stack); (circle_stack :>
  shape readonly_stack)] ;;
- : float = 7280.
```

Аспекты, описываемые в этом разделе, могут показаться довольно сложными, но их демонстрация доказывает, что такой подход работает, к тому же аннотации типов играют второстепенную роль. В большинстве объектно-ориентированных языков со строгим контролем типов такие приведения просто невозможны. Например, в C++ тип из STL `list<T>` является инвариантным в `T`, поэтому просто нельзя использовать `list<square>` там, где ожидается `list<shape>` (по крайней мере, безопасно). Аналогичная ситуация наблюдается и в Java, хотя в Java есть «запасный выход», позволяющий программам возвращаться к динамической типизации. Ситуация в OCaml намного лучше: реализация работает, она проходит проверки компилятора, а аннотации очень просты в использовании.

Сужение

Сужение, также называется *приведением вниз* (*down casting*), – это возможность приведения типа объекта к одному из его подтипов. Допустим, что имеется список фигур `shape list`, и мы заранее знаем (не важно, откуда) тип каждой фигуры. Предположим, что все объекты в списке имеют тип `square`. В этом случае *сужение* позволило бы привести тип объекта `shape` к типу `square`. Многие языки программирования поддерживают сужение через динамическое определение типа. Например, в Java приведение `(Square) x` является допустимым, если значение `x` имеет тип `Square` или один из его подтипов; в противном случае попытка приведения типа возбудит исключение.

Сужение недопустимо в OCaml. Точка!

Почему? Тому есть две веские причины. Одна относится к архитектурным принципам, а другая чисто техническая (сложность реализации).

С архитектурной точки зрения, сужение мешает абстракции. Фактически с такой структурной системой типов, как в OCaml, сужение фактически позволило бы перечислять методы объектов. Чтобы проверить, содержит ли объект `obj` некоторый метод `foo : int`, достаточно было бы выполнить приведение `(obj :> < foo : int >)`.

С более практичной точки зрения, сужение поощряет плохой объектно-ориентированный стиль программирования. Взгляните на следующий Java-код, возвращающий имя объекта фигуры:

Java: objects/Shape.java

<https://github.com/realworldocaml/examples/blob/v1/code/objects/Shape.java>

```
String GetShapeName(Shape s) {
    if (s instanceof Square) {
        return "Square";
    } else if (s instanceof Circle) {
        return "Circle";
    } else {
        return "Other";
    }
}
```

Большинство программистов посчитают такой код «неправильным». Вместо анализа типа объекта лучше было бы определить метод, возвращающий имя фигуры. Вместо вызова `GetShapeName(s)` следовало бы вызвать `s.Name()`.

Однако ситуация не всегда так однозначна. Следующий код проверяет, выглядит ли массив фигур как «гантель», то есть состоит ли он из двух объектов `Circle`, разделенных объектом `Line`, и при этом окружности имеют одинаковые радиусы:

Java

<https://github.com/realworldocaml/examples/blob/v1/code/objects/IsBarbell.java>

```
boolean IsBarbell(Shape[] s) {
    return s.length == 3 && (s[0] instanceof Circle) &&
        (s[1] instanceof Line) && (s[2] instanceof Circle) &&
        ((Circle) s[0]).radius() == ((Circle) s[2]).radius();
}
```

В этом случае не так очевидно, как расширить класс `Shape` для поддержки такого рода анализа. Также неочевидно, что объектно-ориентированное программирование хорошо подходит для решения подобных задач. Сопоставление с образцом в данной ситуации выглядит более привлекательно:

OCaml

https://github.com/realworldocaml/examples/blob/v1/code/objects/is_barbell.ml

```
let is_barbell = function
| [Circle r1; Line _; Circle r2] when r1 = r2 -> true
| _ -> false
```

Кроме того, данную ситуацию можно решить с помощью расширения классов вариантами. Можно определить метод `variant`, внедряющий фактический объект в вариантный тип:

OCaml (part 1)

<https://github.com/realworldocaml/examples/blob/v1/code/objects/narrowing.ml>

```
type shape = < variant : repr; area : float>
and circle = < variant : repr; area : float; radius : int >
and line = < variant : repr; area : float; length : int >
```

```

and repr =
  | Circle of circle
  | Line of line;;

let is_barbell = function
| [s1; s2; s3] ->
  (match s1#variant, s2#variant, s3#variant with
   | Circle c1, Line _, Circle c2 when c1#radius = c2#radius -> true
   | _ -> false)
  | _ -> false;;

```

Этот прием работает, однако он имеет некоторые недостатки. В частности, рекурсивное определение типа показывает, что этот подход, по сути, эквивалентен использованию вариантов и что использование объектов не несет значительных преимуществ.

Подтипизация и рядный полиморфизм

Подтипизация и рядный полиморфизм имеют много общего. Оба механизма позволяют писать функции, которые можно применять к объектам разных типов. В этих случаях рядный полиморфизм обычно предпочтительнее подтипов, потому что не требует явного приведения и несет больше информации о типе, давая возможность писать такие функции:

OCaml utop (part 1)

https://github.com/realworldocaml/examples/blob/v1/code/objects/row_polymorphism.topscript

```

# let remove_large l =
  List.filter ~f:(fun s -> s#area <= 100.) l ;;
val remove_large : (< area : float; .. > as 'a) list -> 'a list = <fun>

```

Тип возвращаемого значения этой функции определяется из открытого типа объекта аргумента с учетом любых дополнительных методов, которые могут иметься:

OCaml utop (part 2)

https://github.com/realworldocaml/examples/blob/v1/code/objects/row_polymorphism.topscript

```

# let squares : < area : float; width : int > list =
  [square 5; square 15; square 10] ;;
val squares : < area : float; width : int > list = [<obj>; <obj>; <obj>]
# remove_large squares ;;
- : < area : float; width : int > list = [<obj>; <obj>]

```

Аналогичная функция с закрытым типом, использующая механизм подтипов, не учитывает методов аргумента: о возвращаемом объекте известно только то, что он имеет метод area:

OCaml utop (part 3)

https://github.com/realworldocaml/examples/blob/v1/code/objects/row_polymorphism.topscript

```

# let remove_large (l: < area : float > list) =
  List.filter ~f:(fun s -> s#area <= 100.) l ;;

```

```
val remove_large : < area : float > list -> < area : float > list = <fun>
# remove_large (squares :> < area : float > list ) ;;
- : < area : float > list = [<obj>; <obj>]
```

Однако рядный полиморфизм можно использовать не во всех ситуациях. В частности, рядный полиморфизм нельзя использовать, чтобы поместить объекты разных типов в один контейнер. Например, с помощью рядного полиморфизма нельзя создать список гетерогенных элементов:

OCaml utop (part 4)

https://github.com/realworldocaml/examples/blob/v1/code/objects/row_polymorphism.topscript

```
# let hlist: < area: float; ..> list = [square 10; circle 30] ;;
```

Characters 49-58:

```
Error: This expression has type < area : float; radius : int >
      but an expression was expected of type < area : float; width : int >
      The second object type has no method radius
(Ошибка: Это выражение имеет тип < area : float; radius : int >
  тогда как ожидалось выражение типа < area : float; width : int >
  Тип второго объекта не имеет метода radius.)
```

Аналогично нельзя использовать рядный полиморфизм для сохранения объектов разных типов в одной и той же ссылке:

OCaml utop (part 5)

https://github.com/realworldocaml/examples/blob/v1/code/objects/row_polymorphism.topscript

```
# let shape_ref: < area: float; ..> ref = ref (square 40) ;;
val shape_ref : < area : float; width : int > ref = {contents = <obj>}
# shape_ref := circle 20 ;;
```

Characters 13-22:

```
Error: This expression has type < area : float; radius : int >
      but an expression was expected of type < area : float; width : int >
      The second object type has no method radius
(Ошибка: Это выражение имеет тип < area : float; radius : int >
  тогда как ожидалось выражение типа < area : float; width : int >
  Тип второго объекта не имеет метода radius.)
```

Зато в обоих случаях с успехом можно использовать подтипы:

OCaml utop (part 6)

https://github.com/realworldocaml/examples/blob/v1/code/objects/row_polymorphism.topscript

```
# let hlist: shape list = [(square 10 :> shape); (circle 30 :> shape)] ;;
val hlist : shape list = [<obj>; <obj>]
# let shape_ref: shape ref = ref (square 40 :> shape) ;;
val shape_ref : shape ref = {contents = <obj>}
# shape_ref := (circle 20 :> shape) ;;
- : unit = ()
```

Примечание

Эта глава написана благодаря поддержке Лео Уайт (Leo White).

Глава 12

Классы

Программирование с непосредственным использованием объектов отлично подходит для решения задач, где требуется инкапсуляция, но одной из основных целей объектно-ориентированного программирования является создание кода многократного использования через наследование. Чтобы воспользоваться механизмом наследования, нам нужно познакомиться с *классами*. В объектно-ориентированном программировании класс – это «рецепт» создания объектов. Рецепт может изменяться добавлением новых методов и полей или посредством изменения существующих методов.

Классы в OCaml

В языке OCaml определение класса выполняется на верхнем уровне модуля и начинается с ключевого слова `class`:

OCaml utop

<https://github.com/realworldocaml/examples/tree/v1/code/classes/istack.topscript>

```
# class istack = object
  val mutable v = [0; 2]

  method pop =
    match v with
    | hd :: tl ->
      v <- tl;
      Some hd
    | [] -> None

  method push hd =
    v <- hd :: v
end ;;
class istack :
object
  val mutable v : int list
  method pop : int option
  method push : int -> unit
end
```

Как видно из результатов, инструкция `class istack : object ... end` создала класс `istack` – объект `object ... end` с *типом класса*. Подобно типам модулей, типы классов полностью отделены от обычных типов OCaml (таких как `int`, `string` и `list`), и их не следует путать с типами объектов (такими как `< get : int; .. >`). Тип класса описывает сам класс, а не объекты, создаваемые на основе класса. Из данного

конкретного типа класса видно, что класс `istack` определяет изменяемое поле `v`, метод `pop`, возвращающий значение типа `int option`, и метод `push` с типом `int -> unit`.

Создание объектов на основе класса производится с помощью ключевого слова `new`:

OCaml utop (part 1)

<https://github.com/realworldocaml/examples/tree/v1/code/classes/istack.topscript>

```
# let s = new istack ;;
val s : istack = <obj>
# s#pop ;;
- : int option = Some 0
# s#push 5 ;;
- : unit = ()
# s#pop ;;
- : int option = Some 5
```

В этом примере можно заметить, что объект `s` имеет тип `istack`. «Но постойте, — скажете вы, — мы же особо подчеркивали, что *классы не являются типами*, а что получается на деле?» Не волнуйтесь, все, что было сказано выше, — правда: ни классы, ни имена классов действительно *не являются типами*. Однако для удобства определение класса `istack` также определяет тип объекта `istack` с теми же методами, что и в классе.

Ниже приводится эквивалентное определение типа:

OCaml utop (part 2)

<https://github.com/realworldocaml/examples/blob/v1/code/classes/istack.topscript>

```
# type istack = < pop: int option; push: int -> unit > ;;
type istack = < pop : int option; push : int -> unit >
```

Обратите внимание, что этот тип представляет любые объекты, обладающие указанными методами: объекты, созданные на основе класса `istack`, будут иметь этот тип, но объекты этого типа могут быть созданы не только с помощью класса `istack`.

Параметры класса и полиморфизм

Определение класса служит *конструктором* для класса. В общем случае определение класса может иметь параметры, которые должны передаваться при создании объектов с помощью ключевого слова `new`.

Давайте реализуем вариант класса `istack`, способный хранить любые значения, а не только целые числа. При определении класса параметр типа помещается в квадратные скобки перед именем класса. Мы также добавим параметр `init`, определяющий первоначальное содержимое стека:

OCaml utop

<https://github.com/realworldocaml/examples/tree/v1/code/classes/stack.topscript>

```
# class ['a] stack init = object
  val mutable v : 'a list = init
```

```

method pop =
  match v with
  | hd :: tl ->
    v <- tl;
    Some hd
  | [] -> None

method push hd =
  v <- hd :: v
end ;;
class ['a] stack :
  'a list ->
  object
    val mutable v : 'a list
    method pop : 'a option
    method push : 'a -> unit
  end

```

Отметьте, что параметр типа `['a]` в определении заключен в квадратные скобки, но там, где он используется, квадратные скобки отсутствуют (или замещаются круглыми скобками, если имеется более одного параметра типа).

Аннотация типа в объявлении `v` используется для ограничения свободы механизма вывода типов. Если опустить эту аннотацию, механизм вывода типов сообщит, что класс «слишком полиморфный»: параметр `init` может иметь, к примеру, некоторый тип `'b list`:

OCaml utop (part 1)

<https://github.com/realworldocaml/examples/tree/v1/code/classes/stack.topscript>

```

# class ['a] stack init = object
  val mutable v = init

  method pop =
    match v with
    | hd :: tl ->
      v <- tl;
      Some hd
    | [] -> None

  method push hd =
    v <- hd :: v
end ;;

```

Characters 6-16:

Error: Some type variables are unbound in this type:

```

class ['a] stack :
  'b list ->
  object
    val mutable v : 'b list
    method pop : 'b option
    method push : 'b -> unit
  end

```

The method `pop` has type `'b option` where `'b` is unbound

(Ошибка: Некоторые переменные типа не связаны в следующем типе:

```

class ['a] stack :

```



```
'b list ->
object
  val mutable v : 'b list
  method pop : 'b option
  method push : 'b -> unit
end
Метод pop имеет тип 'b option, где 'b не связан
```

В общем случае необходимо наложить достаточно строгое ограничение, чтобы компилятор смог вывести корректный тип. Мы можем добавить ограничения типов в определениях параметров, полей и методов. Выбор количества ограничений во многом зависит от личных предпочтений. Вы можете добавить ограничения типов во всех трех местах, но лишний текст не способствует ясности кода. Наиболее удобным вариантом, на наш взгляд, является аннотирование полей и/или параметров класса, а ограничения в определениях методов следует добавлять, только когда это действительно необходимо.

Типы объектов и интерфейсы

Рано или поздно у нас может появиться желание выполнить обход элементов в нашем стеке. Для этой цели в объектно-ориентированных языках принято определять класс объекта-итератора. Итератор реализует универсальный механизм исследования и обхода элементов коллекций.

Существуют два основных способа определения абстрактных интерфейсов, подобных этому. В Java итераторы обычно объявляются с применением интерфейса, определяющего набор методов:

Java

<https://github.com/realworldocaml/examples/tree/v1/code/classes/Iterator.java>

```
// Итератор в стиле Java, определяется как интерфейс.
interface <T> iterator {
  T Get();
  boolean HasValue();
  void Next();
};
```

В языках, не поддерживающих интерфейсов, таких как C++, спецификация обычно опирается на *абстрактные классы*, в которых определяются методы без реализации (в C++ используется определение «= 0», означающее «не реализовано»):

C

<https://github.com/realworldocaml/examples/tree/v1/code/classes/citerator.cpp>

```
// Определение абстрактного класса на языке C++.
template<typename T>
class Iterator {
public:
  virtual ~Iterator() {}
  virtual T get() const = 0;
  virtual bool has_value() const = 0;
  virtual void next() = 0;
};
```

OCaml поддерживает оба стиля. Однако OCaml обладает большей гибкостью, чем эти два подхода, благодаря тому что тип объекта может быть реализован любым объектом с соответствующими методами – методы не обязательно должны определяться классом объекта. Оставим пока абстрактные классы и рассмотрим прием на основе типов объектов.

Прежде всего определим тип объектов `iterator`, описывающий методы итератора:

OCaml utop

<https://github.com/realworldocaml/examples/tree/v1/code/classes/iter.topscript>

```
# type 'a iterator = < get : 'a; has_value : bool; next : unit > ;;
type 'a iterator = < get : 'a; has_value : bool; next : unit >
```

Далее определим фактический итератор для списков. Этот итератор можно будет использовать для выполнения итераций по содержимому нашего стека:

OCaml utop (part 1)

<https://github.com/realworldocaml/examples/tree/v1/code/classes/iter.topscript>

```
# class ['a] list_iterator init = object
  val mutable current : 'a list = init

  method has_value = current <> []

  method get =
    match current with
    | hd :: tl -> hd
    | [] -> raise (Invalid_argument "no value")

  method next =
    match current with
    | hd :: tl -> current <- tl
    | [] -> raise (Invalid_argument "no value")
end ;;
class ['a] list_iterator :
  'a list ->
  object
    val mutable current : 'a list
    method get : 'a
    method has_value : bool
    method next : unit
end
```

Наконец, добавим в класс `stack` метод `iterator`, возвращающий итератор. Для этого сконструируем `list_iterator`, ссылающийся на текущее содержимое стека:

OCaml utop (part 2)

<https://github.com/realworldocaml/examples/tree/v1/code/classes/iter.topscript>

```
# class ['a] stack init = object
  val mutable v : 'a list = init

  method pop =
```

```

    match v with
    | hd :: tl ->
        v <- tl;
        Some hd
    | [] -> None

method push hd =
    v <- hd :: v

method iterator : 'a iterator =
    new list_iterator v
end ;;
class ['a] stack :
    'a list ->
    object
        val mutable v : 'a list
        method iterator : 'a iterator
        method pop : 'a option
        method push : 'a -> unit
    end
end

```

Теперь можно создать новый стек, поместить на него несколько значений и выполнить их обход:

OCaml utop (part 3)

<https://github.com/realworldocaml/examples/tree/v1/code/classes/iter.topscript>

```

# let s = new stack [] ;;
val s : 'a stack = <obj>
# s#push 5 ;;
- : unit = ()
# s#push 4 ;;
- : unit = ()
# let it = s#iterator ;;
val it : int iterator = <obj>
# it#get ;;
- : int = 4
# it#next ;;
- : unit = ()
# it#get ;;
- : int = 5
# it#next ;;
- : unit = ()
# it#has_value ;;
- : bool = false

```

Функциональные итераторы

На практике большинство программистов на OCaml стараются избегать объектов-итераторов, предпочитая приемы функционального программирования. Например, ниже приводится альтернативный класс `stack` стека, принимающий функцию `f` и применяющий ее к каждому элементу стека:

OCaml utop (part 4)

<https://github.com/realworldocaml/examples/tree/v1/code/classes/iter.topscript>

```
# class ['a] stack init = object
  val mutable v : 'a list = init

  method pop =
    match v with
    | hd :: tl ->
      v <- tl;
      Some hd
    | [] -> None

  method push hd =
    v <- hd :: v

  method iter f =
    List.iter ~f v
end ;;
class ['a] stack :
  'a list ->
  object
    val mutable v : 'a list
    method iter : ('a -> unit) -> unit
    method pop : 'a option
    method push : 'a -> unit
  end
```

А что можно сказать о функциональных операциях, таких как `map` и `fold`? В общем случае эти методы принимают функцию, возвращающую значение некоторого другого типа, отличного от типа элементов множества.

Например, метод `fold` для нашего класса `['a] stack` должен иметь тип `('b -> 'a -> 'b) -> 'b -> 'b`, где `'b` – полиморфный тип. Чтобы выразить полиморфный тип метода, как в данном случае, необходимо использовать квантификатор типа, демонстрируемый в следующем примере:

OCaml utop (part 5)

<https://github.com/realworldocaml/examples/tree/v1/code/classes/iter.topscript>

```
# class ['a] stack init = object
  val mutable v : 'a list = init

  method pop =
    match v with
    | hd :: tl ->
      v <- tl;
      Some hd
    | [] -> None

  method push hd =
    v <- hd :: v

  method fold : 'b. ('b -> 'a -> 'b) -> 'b -> 'b =
    (fun f init -> List.fold ~f ~init v)
```

```

end ;;
class ['a] stack :
  'a list ->
  object
    val mutable v : 'a list
    method fold : ('b -> 'a -> 'b) -> 'b -> 'b
    method pop : 'a option
    method push : 'a -> unit
  end
end

```

Квантификатор типа 'b. можно прочесть как: «для всех 'b». Квантификаторы типов могут использоваться только *сразу после* имени метода. Это означает, что параметры метода должны выражаться с использованием выражения `fun` или `function`.

Наследование

Наследование выражается в использовании существующего класса для определения нового. Например, следующий класс наследует наш класс `stack` стека для хранения строк и добавляет новый метод `print` для вывода всех строк, находящихся на стеке:

OCaml utop (part 2)

<https://github.com/realworldocaml/examples/tree/v1/code/classes/stack.topscript>

```

# class sstack init = object
  inherit [string] stack init

  method print =
    List.iter ~f:print_string v
end ;;
class sstack :
  string list ->
  object
    val mutable v : string list
    method pop : string option
    method print : unit
    method push : string -> unit
  end
end

```

Класс может переопределять унаследованные методы. Например, следующий класс создает стек целых чисел, который удваивает целочисленное значение, прежде чем положить его на стек:

OCaml utop (part 3)

<https://github.com/realworldocaml/examples/tree/v1/code/classes/stack.topscript>

```

# class double_stack init = object
  inherit [int] stack init as super

  method push hd =
    super#push (hd * 2)
end ;;

```

```
class double_stack :
  int list ->
  object
    val mutable v : int list
    method pop : int option
    method push : int -> unit
  end
```

Инструкция `as super` в этом примере создает специальный объект с именем `super`, который можно использовать для вызова методов суперкласса. Обратите внимание, что `super` — это не настоящий объект, данное имя может использоваться только для вызова методов.

Типы классов

Чтобы позволить коду в другом файле или модуле наследовать класс, необходимо экспортировать его и присвоить ему тип класса. Что такое тип класса?

Давайте для примера завернем наш класс `stack` в модуль (здесь модуль используется исключительно в иллюстративных целях, но суть от этого не меняется, когда потребуется определить файл *.mli*). В соответствии с обычным для модулей стилем мы определим тип `'a t` для представления типа нашего стека:

OCaml

https://github.com/realworldocaml/examples/tree/v1/code/classes/class_types_stack.ml

```
module Stack = struct
  class ['a] stack init = object
    ...
  end

  type 'a t = 'a stack

  let make init = new stack init
end
```

В зависимости от числа реализаций, которые требуется экспортировать, у нас на выбор есть несколько вариантов. Максимально абстрактная сигнатура может полностью скрыть определение класса:

OCaml (part 1)

https://github.com/realworldocaml/examples/tree/v1/code/classes/class_types_stack.ml

```
module AbstractStack : sig
  type 'a t = < pop: 'a option; push: 'a -> unit >

  val make : unit -> 'a t
end = Stack
```

Простота абстрактной сигнатуры достигается за счет игнорирования классов. Но как быть, если необходимо включить классы в сигнатуру, чтобы другие модули могли наследовать их? Для этого нужно определить типы для классов, которые так и называются: *типы классов*.

Понятие типа класса отсутствует в основных объектно-ориентированных языках программирования, поэтому для вас оно может оказаться незнакомым, однако сама концепция чрезвычайно проста. Тип класса определяет тип каждой видимой части класса, включая поля и методы. В точности как и при определении типов модулей, вам не требуется определять типы для всего и вся – все, что будет опущено, окажется скрыто:

OCaml (part 2)

https://github.com/realworldocaml/examples/tree/v1/code/classes/class_types_stack.ml

```
module VisibleStack : sig

  type 'a t = < pop: 'a option; push: 'a -> unit >

  class ['a] stack : object
    val mutable v : 'a list
    method pop : 'a option
    method push : 'a -> unit
  end

  val make : unit -> 'a t
end = Stack
```

В этой сигнатуре мы решили сделать видимым все, что имеется. Тип класса `stack` определяет тип поля `v`, а также типы всех методов.

Открытая рекурсия

Открытая рекурсия дает возможность вызывать методы объекта из других методов того же объекта. Поиск вызываемых методов осуществляется динамически, благодаря чему методы одного класса могут вызывать методы другого класса, если оба класса наследуются одним и тем же объектом. Это позволяет включать в объект взаимно-рекурсивные (*mutually recursive*) части, определяемые по отдельности.

Данная способность определять взаимно-рекурсивные методы в разных компонентах является ключевой особенностью классов: добиться похожей функциональности с применением типов данных или модулей намного сложнее.

Например, рассмотрим реализацию рекурсивных функций для анализа документов простого формата. Этот формат представлен как дерево с узлами трех типов:

OCaml

<https://github.com/realworldocaml/examples/tree/v1/code/classes/doc.ml>

```
type doc =
  | Heading of string
  | Paragraph of text_item list
  | Definition of string list_item list

and text_item =
  | Raw of string
  | Bold of text_item list
  | Enumerate of int list_item list
```

```
| Quote of doc

and 'a list_item =
{ tag: 'a;
  text: text_item list }
```

Мы легко можем написать рекурсивную функцию для обхода узлов дерева. Однако представьте, что потребовалось написать множество схожих рекурсивных функций. Как можно было бы вынести за скобки общие части этих функций, чтобы избежать повторяющегося шаблонного кода?

Самый простой путь – использовать классы и открытую рекурсию. Например, следующий класс определяет объекты, выполняющие свертку данных документа:

OCaml (part 1)

<https://github.com/realworldocaml/examples/tree/v1/code/classes/doc.ml>

```
open Core.Std

class ['a] folder = object(self)
  method doc acc = function
  | Heading _ -> acc
  | Paragraph text -> List.fold ~f:self#text_item ~init:acc text
  | Definition list -> List.fold ~f:self#list_item ~init:acc list

  method list_item: 'b. 'a -> 'b list_item -> 'a =
    fun acc {tag; text} ->
      List.fold ~f:self#text_item ~init:acc text

  method text_item acc = function
  | Raw _ -> acc
  | Bold text -> List.fold ~f:self#text_item ~init:acc text
  | Enumerate list -> List.fold ~f:self#list_item ~init:acc list
  | Quote doc -> self#doc acc doc
end
```

Синтаксис `object (self)` связывает `self` с текущим объектом, что позволяет методам `doc`, `list_item` и `text_item` вызывать друг друга.

Наследуя этот класс, можно создать функции, выполняющие свертку документа. Например, функция `count_doc` подсчитывает число тегов `bold` в документе:

OCaml (part 2)

<https://github.com/realworldocaml/examples/tree/v1/code/classes/doc.ml>

```
class counter = object
  inherit [int] folder as super

  method list_item acc li = acc
  method text_item acc ti =
    let acc = super#text_item acc ti in
    match ti with
    | Bold _ -> acc + 1
    | _ -> acc
end

let count_doc = (new counter)#doc
```


Обратите внимание, как используется специальный объект `super` в `text_item` для вызова метода `text_item` класса `[int] folder`, чтобы выполнить свертку дочерних узлов для узла `text_item`.

Скрытые методы

Методы можно объявлять *скрытыми* (*private*). Скрытые методы могут вызываться подклассами, но они будут недоступны остальному коду (подобно *защищенным* (*protected*) методам в C++).

Например, может потребоваться включить методы в наш класс `folder` для обработки каждого из разных случаев в `doc` и `text_item`, но так, чтобы исключить возможность их экспортирования в подклассах класса `folder`, поскольку они не должны вызываться непосредственно:

OCaml (part 3)

<https://github.com/realworldocaml/examples/blob/v1/code/classes/doc.ml>

```
class ['a] folder2 = object(self)
  method doc acc = function
    | Heading str -> self#heading acc str
    | Paragraph text -> self#paragraph acc text
    | Definition list -> self#definition acc list

  method list_item: 'b. 'a -> 'b list_item -> 'a =
    fun acc {tag; text} ->
      List.fold ~f:self#text_item ~init:acc text

  method text_item acc = function
    | Raw str -> self#raw acc str
    | Bold text -> self#bold acc text
    | Enumerate list -> self#enumerate acc list
    | Quote doc -> self#quote acc doc

  method private heading acc str = acc
  method private paragraph acc text =
    List.fold ~f:self#text_item ~init:acc text
  method private definition acc list =
    List.fold ~f:self#list_item ~init:acc list

  method private raw acc str = acc
  method private bold acc text =
    List.fold ~f:self#text_item ~init:acc text
  method private enumerate acc list =
    List.fold ~f:self#list_item ~init:acc list
  method private quote acc doc = self#doc acc doc
end
let f :
  < doc : int -> doc -> int;
  list_item : 'a . int -> 'a list_item -> int;
  text_item : int -> text_item -> int > = new folder2
```

Последняя инструкция, создающая значение `f`, демонстрирует, что созданный экземпляр объекта `folder2` имеет тип, скрывающий скрытые методы.

Если быть до конца точными, скрытые методы являются частью типа класса, но не частью типа объекта. Это означает, например, что объект `f` не имеет метода `bold`. Однако скрытые методы доступны подклассам: мы можем использовать их, чтобы упростить наш класс `counter`:

OCaml (part 4)

<https://github.com/realworldocaml/examples/blob/v1/code/classes/doc.ml>

```
class counter_with_private_method = object
  inherit [int] folder2 as super

  method list_item acc li = acc

  method private bold acc txt =
    let acc = super#bold acc txt in
    acc + 1
end
```

Доступность только в подклассах является ключевым свойством скрытых методов. Если вам нужны надежные гарантии, что метод будет *по-настоящему* скрытым и недоступным даже в подклассах, можно явно определить тип класса, в котором этот метод отсутствует. В следующем коде скрытые методы явно опущены в типе класса `counter_with_sig` и поэтому не могут вызываться в подклассах:

OCaml (part 5)

<https://github.com/realworldocaml/examples/blob/v1/code/classes/doc.ml>

```
class counter_with_sig : object
  method doc : int -> doc -> int
  method list_item : int -> 'b list_item -> int
  method text_item : int -> text_item -> int
end = object
  inherit [int] folder2 as super

  method list_item acc li = acc

  method private bold acc txt =
    let acc = super#bold acc txt in
    acc + 1
end
```

Бинарные методы

Бинарный метод (binary method) – это метод, принимающий объект типа `self`. Типичным примером бинарных методов может служить метод определения равенства:

OCaml utop

<https://github.com/realworldocaml/examples/blob/v1/code/classes/binary.topscript>

```
# class square w = object(self : 'self)
  method width = w
  method area = Float.of_int (self#width * self#width)
```

```

    method equals (other : 'self) = other#width = self#width
  end ;;
class square :
  int ->
  object ('a)
    method area : float
    method equals : 'a -> bool
    method width : int
  end
# class circle r = object(self : 'self)
  method radius = r
  method area = 3.14 *. (Float.of_int self#radius) ** 2.0
  method equals (other : 'self) = other#radius = self#radius
end ;;
class circle :
  int ->
  object ('a)
    method area : float
    method equals : 'a -> bool
    method radius : int
  end
end

```

Обратите внимание, как можно использовать аннотацию (self: 'self) для получения типа текущего объекта.

Теперь с помощью бинарного метода equals можно проверить равенство экземпляров объектов:

OCaml utop (part 1)

<https://github.com/realworldocaml/examples/blob/v1/code/classes/binary.topscript>

```

# (new square 5)#equals (new square 5) ;;
- : bool = true
# (new circle 10)#equals (new circle 7) ;;
- : bool = false

```

Как видите, прием вполне работоспособен, но в нем кроется одна проблема. Метод equals принимает объект точно типа square или circle. Из-за этого мы не можем определить общий базовый класс shape, который также включал бы метод equals:

OCaml utop (part 2)

<https://github.com/realworldocaml/examples/blob/v1/code/classes/binary.topscript>

```

# type shape = < equals : shape -> bool; area : float > ;;
type shape = < area : float; equals : shape -> bool >
# (new square 5 :> shape) ;;
Characters ~1-23:
Error: Type square = < area : float; equals : square -> bool; width : int >
      is not a subtype of shape = < area : float; equals : shape -> bool >
Type shape = < area : float; equals : shape -> bool >
      is not a subtype of
      square = < area : float; equals : square -> bool; width : int >
(Ошибка: Тип square = < area : float; equals : square -> bool; width : int >
не является подтипом shape = < area : float; equals : shape -> bool >
Тип shape = < area : float; equals : shape -> bool >
не является подтипом
      square = < area : float; equals : square -> bool; width : int >)

```

Проблема в том, что экземпляр квадрата (square) ожидает получить для сравнения так же экземпляр квадрата (square), а не произвольную геометрическую фигуру (shape); то же можно сказать и о circle. Эта проблема относится к ряду фундаментальных. Многие языки решают ее, либо применяя сужение типа (динамическую проверку типа), либо за счет перегрузки методов. Но OCaml не поддерживает ни то, ни другое, так как же быть?

Поскольку проблематичным методом является метод определения равенства, на первый взгляд неплохим решением было бы просто выбросить его из базового типа shape и использовать полиморфное сравнение. Однако встроенное полиморфное сравнение действует вразрез с нашими представлениями о равенстве применительно к объектам:

OCaml utop (part 3)

<https://github.com/realworldocaml/examples/blob/v1/code/classes/binary.topscript>

```
# (object method area = 5 end) = (object method area = 5 end) ;;
- : bool = false
```

Проблема здесь в том, что встроенный полиморфный механизм сравнения считает два объекта равными, если они являются одним и тем же физическим объектом. Существуют и другие доводы против использования встроенного полиморфного механизма сравнения.

Если действительно необходимо определить обобщенный метод определения равенства для геометрических фигур, остается еще одно решение — использовать тот же подход, что применялся, когда мы описывали прием сужения. То есть следует определить *представительский* тип на основе вариантов и реализовать сравнение, опираясь на этот представительский тип:

OCaml utop (part 4)

<https://github.com/realworldocaml/examples/blob/v1/code/classes/binary.topscript>

```
# type shape_repr =
  | Square of int
  | Circle of int ;;
type shape_repr = Square of int | Circle of int
# type shape =
  < repr : shape_repr; equals : shape -> bool; area : float > ;;
type shape = < area : float; equals : shape -> bool; repr : shape_repr >
# class square w = object(self)
  method width = w
  method area = Float.of_int (self#width * self#width)
  method repr = Square self#width
  method equals (other : shape) = other#repr = self#repr
end ;;
class square :
  int ->
  object
    method area : float
    method equals : shape -> bool
    method repr : shape_repr
    method width : int
  end
```

Бинарный метод `equals` теперь реализован в терминах конкретного типа `shape_repr`. При использовании этого решения уже нельзя скрыть метод `repr`, зато можно скрыть определение типа, задействовав систему модулей:

OCaml

https://github.com/realworldocaml/examples/blob/v1/code/classes/binary_module.ml

```
module Shapes : sig
  type shape_repr
  type shape =
    < repr : shape_repr; equals : shape -> bool; area: float >

  class square : int ->
    object
      method width : int
      method area : float
      method repr : shape_repr
      method equals : shape -> bool
    end
end = struct
  type shape_repr =
    | Square of int
    | Circle of int
  ...
end
```

Обратите внимание, что это решение препятствует добавлению новых видов геометрических фигур без добавления новых конструкторов в тип `shape_repr`, что является весьма серьезным ограничением. Кроме того, объекты, созданные с применением этих классов, находятся в отношении «один к одному» с членами представительского типа, из-за чего объекты выглядят несколько избыточными.

Однако реализация сравнения – довольно критический пример применения бинарного метода: ему необходим полный доступ ко всему содержимому другого объекта. Можно привести множество других примеров бинарных методов, где достаточно иметь лишь частичный доступ к объекту. Одним из них может служить метод, сравнивающий размеры фигур:

OCaml

https://github.com/realworldocaml/examples/blob/v1/code/classes/binary_larger.ml

```
class square w = object(self)
  method width = w
  method area = Float.of_int (self#width * self#width)
  method larger other = self#area > other#area
end
```

В данном случае нет соответствия «один к одному» между объектами и их размерами, благодаря чему мы легко можем определять новые виды фигур.

Виртуальные классы и методы

Виртуальный класс – это класс, некоторые методы или поля которого объявлены, но не реализованы. Понятие «виртуальный» не следует путать со словом `virtual` в языке C++. Виртуальные (`virtual`) методы в C++ отличаются тем, что при их вызове выбор реализации осуществляется динамически, во время выполнения, тогда как для обычных, неvirtуальных методов – статически, во время компиляции. В OCaml выбор реализации *любого* метода осуществляется динамически, а ключевое слово `virtual` означает, что данный метод или поле не реализован. Класс, содержащий виртуальные методы, также должен быть помечен ключевым словом `virtual` и не может использоваться для создания экземпляров (то есть на основе этого класса невозможно создавать объекты).

Для дальнейших исследований добавим в наши примеры геометрических фигур поддержку простой интерактивной графики. При этом мы будем использовать библиотеку `Async` параллельных вычислений и библиотеку `Async_graphics`, реализующую асинхронный интерфейс к встроенной в OCaml библиотеке `Graphics`. Исследование приемов конкурентного программирования с применением библиотеки `Async` будет продолжено в главе 18, а пока можно просто игнорировать некоторые детали. Чтобы установить библиотеки, достаточно просто выполнить команду `opam install async_graphics`.

Мы добавим в каждую фигуру метод `draw`, описывающий порядок рисования фигуры с помощью `Async_graphics`:

OCaml

<https://github.com/realworldocaml/examples/blob/v1/code/classes-async/shapes.ml>

```
open Core.Std
open Async.Std
open Async_graphics
```

```
type drawable = < draw: unit >
```

Создание простых фигур

Теперь добавим классы для создания квадратов и окружностей. Мы включили метод `on_click`, реализующий обработку событий в фигурах:

OCaml (part 1)

https://github.com/realworldocaml/examples/tree/v1/code/classes-async/verbose_shapes.ml

```
class square w x y = object(self)
  val mutable x: int = x
  method x = x

  val mutable y: int = y
  method y = y

  val mutable width = w
  method width = width
```

```

method draw = fill_rect x y width width

method private contains x' y' =
  x <= x' && x' <= x + width &&
  y <= y' && y' <= y + width

method on_click ?start ?stop f =
  on_click ?start ?stop
  (fun ev ->
    if self#contains ev.mouse_x ev.mouse_y then
      f ev.mouse_x ev.mouse_y)
end

```

Класс `square` довольно прост, а класс `circle` (ниже) очень похож на него:

OCaml (part 2)

https://github.com/realworldocaml/examples/tree/v1/code/classes-async/verbose_shapes.ml

```

class circle r x y = object(self)
  val mutable x: int = x
  method x = x

  val mutable y: int = y
  method y = y

  val mutable radius = r
  method radius = radius

  method draw = fill_circle x y radius

  method private contains x' y' =
    let dx = abs (x' - x) in
    let dy = abs (y' - y) in
    let dist = sqrt (Float.of_int ((dx * dx) + (dy * dy))) in
    dist <= (Float.of_int radius)

  method on_click ?start ?stop f =
    on_click ?start ?stop
    (fun ev ->
      if self#contains ev.mouse_x ev.mouse_y then
        f ev.mouse_x ev.mouse_y)
end

```

Эти классы имеют много общего, и было бы совсем неплохо вынести общую функциональность в суперкласс. Мы легко можем перенести в суперкласс определения `x` и `y`, но как быть с методом `on_click`? Его определение зависит от конкретного содержимого, разного в разных классах. Решить эту проблему можно с помощью *виртуального* класса. Этот класс объявляет метод `contains`, но оставляет его реализацию за подклассами.

Ниже приводится более краткое определение, начинающееся с виртуального класса `shape`, который реализует методы `on_click` и `on_mousedown`:

OCaml (part 1)

<https://github.com/realworldocaml/examples/tree/v1/code/classes-async/shapes.ml>

```
class virtual shape x y = object(self)
  method virtual private contains: int -> int -> bool

  val mutable x: int = x
  method x = x

  val mutable y: int = y
  method y = y

  method on_click ?start ?stop f =
    on_click ?start ?stop
      (fun ev ->
        if self#contains ev.mouse_x ev.mouse_y then
          f ev.mouse_x ev.mouse_y)

  method on_mousedown ?start ?stop f =
    on_mousedown ?start ?stop
      (fun ev ->
        if self#contains ev.mouse_x ev.mouse_y then
          f ev.mouse_x ev.mouse_y)
end
```

Теперь можно определить классы `square` и `circle`, унаследовав в них класс `shape`:

OCaml (part 2)

<https://github.com/realworldocaml/examples/tree/v1/code/classes-async/shapes.ml>

```
class square w x y = object
  inherit shape x y

  val mutable width = w
  method width = width

  method draw = fill_rect x y width width

  method private contains x' y' =
    x <= x' && x' <= x + width &&
    y <= y' && y' <= y + width
end

class circle r x y = object
  inherit shape x y

  val mutable radius = r
  method radius = radius

  method draw = fill_circle x y radius

  method private contains x' y' =
    let dx = abs (x' - x) in
    let dy = abs (y' - y) in
    let dist = sqrt (Float.of_int ((dx * dx) + (dy * dy))) in
    dist <= (Float.of_int radius)
end
```


Виртуальный класс можно рассматривать как своеобразный функтор, «аргументом» которого является объявление – но не определение – виртуальных методов и полей. Применение функтора происходит через наследование, когда для виртуальных методов указываются конкретные реализации.

Инициализаторы

В процессе создания экземпляров классов можно выполнять выражения, помещая их перед объектным выражением или в качестве начальных значений полей:

OCaml utop

<https://github.com/realworldocaml/examples/tree/v1/code/classes/initializer.topscript>

```
# class obj x =
  let () = printf "Creating obj %d\n" x in
  object
    val field = printf "Initializing field\n"; x
  end ;;

class obj : int -> object val field : int end
# let o = new obj 3 ;;
Creating obj 3
Initializing field
val o : obj = <obj>
```

Однако эти выражения будут выполняться перед созданием объекта и потому не могут ссылаться на его методы. Если потребуется задействовать методы объекта в процессе его создания, следует использовать *инициализатор*. Инициализатор – это выражение, выполняемое сразу после создания объекта.

Например, допустим, что нам требуется дополнить предыдущий модуль с геометрическими фигурами классом `growing_circle` окружностей, которые увеличиваются, если щелкнуть на них мышью. В нем можно унаследовать класс `circle` и использовать унаследованный метод `on_click` для добавления обработчика события щелчка:

OCaml (part 3)

<https://github.com/realworldocaml/examples/tree/v1/code/classes-async/shapes.ml>

```
class growing_circle r x y = object(self)
  inherit circle r x y

  initializer
    self#on_click (fun _x _y -> radius <- radius * 2)
end
```

Множественное наследование

Когда класс наследует поля и методы сразу от нескольких суперклассов, в действие вступает механизм *множественного наследования*. Множественное наследование увеличивает разнообразие способов объединения классов и может оказаться весь-

ма кстати, особенно при наследовании виртуальных классов. Однако оно также может оказаться источником сложностей, особенно когда иерархия наследования имеет форму графа, а не дерева, поэтому множественное наследование следует использовать с большой осторожностью.

Как выполняется разрешение имен

Наибольшие сложности при использовании множественного наследования возникают из-за неоднозначности имен — что получится, если метод или поле с некоторым именем определено более чем в одном классе?

Наследование в OCaml имеет одну важную особенность, о которой нужно помнить всегда: наследование действует подобно текстовому включению. Если существует несколько определений одного и того же имени, использоваться будет самое последнее из них.

Например, взгляните на следующий класс, который наследует класс `square` и определяет новый метод `draw`, использующий для рисования квадрата `draw_rect` вместо `fill_rect`:

OCaml (part 1)

https://github.com/realworldocaml/examples/tree/v1/code/classes-async/multiple_inheritance.ml

```
class square_outline w x y = object
  inherit square w x y
  method draw = draw_rect x y width width
end
```

Так как объявление `inherit` находится перед определением метода, новый метод `draw` «затирает» старый, и квадрат будет рисоваться с помощью `draw_rect`. А что получится, если класс `square_outline` будет определен, как показано ниже?

OCaml (part 1)

https://github.com/realworldocaml/examples/tree/v1/code/classes-async/multiple_inheritance_wrong.ml

```
class square_outline w x y = object
  method draw = draw_rect x y w w
  inherit square w x y
end
```

Здесь объявление `inherit` находится после определения метода, поэтому метод `draw` из класса `square` «затрет» определение нового метода, и квадрат будет рисоваться с использованием `fill_rect`.

Чтобы понять, как будет действовать механизм наследования в том или ином случае, замените каждую директиву `inherit` определением соответствующего класса и определите, какое объявление каждого метода или поля окажется последним. Имейте в виду, что механизм наследования добавляет только поля и методы, перечисленные в типе класса, то есть скрытые методы, отсутствующие в определении типа, не будут включаться в дочерний класс.

Примеси

Когда следует использовать множественное наследование? Если задать этот вопрос десятку человек, вы получите десяток разных ответов (возможно, весьма убедительных). Некоторые утверждают, что множественное наследование слишком сложно; другие убеждены, что наследование само по себе является источником многих проблем и вместо него следует использовать прием композиции объектов. Но с кем бы вы не говорили на эту тему, вы едва ли услышите, что множественное наследование – классная штука и его следует использовать повсеместно.

В любом случае, при использовании объектов существует еще один общий прием применения множественного наследования, который одновременно и полезен, и достаточно прост: этот прием основан на создании *примесей* (*mixin*). В общем случае примесь – это всего лишь виртуальный класс, реализующий некоторую особенность на основе другого виртуального класса. Если у вас имеется класс, реализующий методы *A*, и примесь *M*, предоставляющая методы *B*, тогда вы можете унаследовать *M* – «подмешать» ее, – чтобы получить особенности *B*.

Это объяснение выглядит слишком абстрактным, поэтому давайте разберем несколько примеров, опираясь на наши интерактивные геометрические фигуры. Например, у нас может появиться желание реализовать возможность буксировки фигур мышью. Мы можем определить эту функциональность для любого объекта, имеющего изменяемые поля *x* и *y* и метод `on_mousedown`:

OCaml (part 4)

<https://github.com/realworldocaml/examples/blob/v1/code/classes-async/shapes.ml>

```
class virtual draggable = object(self)
  method virtual on_mousedown:
    ?start:unit Deferred.t ->
    ?stop:unit Deferred.t ->
    (int -> int -> unit) -> unit
  val virtual mutable x: int
  val virtual mutable y: int

  val mutable dragging = false
  method dragging = dragging

  initializer
    self#on_mousedown
      (fun mouse_x mouse_y ->
        let offset_x = x - mouse_x in
        let offset_y = y - mouse_y in
        let mouse_up = Ivar.create () in
        let stop = Ivar.read mouse_up in
        dragging <- true;
        on_mouseup ~stop
          (fun _ ->
            Ivar.fill mouse_up ();
            dragging <- false);
```

```

on_mousemove ~stop
  (fun ev ->
    x <- ev.mouse_x + offset_x;
    y <- ev.mouse_y + offset_y))
end

```

Этот класс позволяет создавать фигуры, поддерживающие возможность буксировки, с применением множественного наследования:

OCaml (part 5)

<https://github.com/realworldocaml/examples/blob/v1/code/classes-async/shapes.ml>

```

class small_square = object
  inherit square 20 40 40
  inherit draggable
end

```

Точно так же примеси можно использовать и для создания анимированных фигур. Каждая анимированная фигура имеет список функций, вызываемых в процессе воспроизведения анимационного эффекта. Мы создадим примесь для поддержки анимации, предоставляющую этот список и гарантирующую вызов функций из списка через регулярные интервалы времени при воспроизведении анимационного эффекта:

OCaml (part 6)

<https://github.com/realworldocaml/examples/blob/v1/code/classes-async/shapes.ml>

```

class virtual animated span = object(self)
  method virtual on_click:
    ?start:unit Deferred.t ->
    ?stop:unit Deferred.t ->
    (int -> int -> unit) -> unit
  val mutable updates: (int -> unit) list = []
  val mutable step = 0
  val mutable running = false

  method running = running

  method animate =
    step <- 0;
    running <- true;
    let stop =
      Clock.after span
      >>| fun () -> running <- false
    in
    Clock.every ~stop (Time.Span.of_sec (1.0 /. 24.0))
      (fun () ->
        step <- step + 1;
        List.iter ~f:(fun f -> f step) updates
      )

  initializer
    self#on_click (fun _x _y -> if not self#running then self#animate)
end

```

Добавление функций в список осуществляется в инициализаторе. Например, этот класс будет производить окружности, смещающиеся вправо в течение секунды после щелчка мышью:

OCaml (part 7)

<https://github.com/realworldocaml/examples/blob/v1/code/classes-async/shapes.ml>

```
class my_circle = object
  inherit circle 20 50 50
  inherit animated Time.Span.second
  initializer updates <- [fun _ -> x <- x + 5]
end
```

Эти инициализаторы можно также добавить с использованием примесей:

OCaml (part 8)

<https://github.com/realworldocaml/examples/blob/v1/code/classes-async/shapes.ml>

```
class virtual linear x' y' = object
  val virtual mutable updates: (int -> unit) list
  val virtual mutable x: int
  val virtual mutable y: int

  initializer
    let update _ =
      x <- x + x';
      y <- y + y'
    in
      updates <- update :: updates
end

let pi = (atan 1.0) *. 4.0

class virtual harmonic offset x' y' = object
  val virtual mutable updates: (int -> unit) list
  val virtual mutable x: int
  val virtual mutable y: int

  initializer
    let update step =
      let m = sin (offset +. ((Float.of_int step) *. (pi /. 64.))) in
      let x' = Float.to_int (m *. Float.of_int x') in
      let y' = Float.to_int (m *. Float.of_int y') in
      x <- x + x';
      y <- y + y'
    in
      updates <- update :: updates
end
```

Так как примеси `linear` и `harmonic` используются исключительно ради побочных эффектов, их можно наследовать многократно в одном и том же объекте, чтобы получить различные анимационные эффекты:

OCaml (part 9)

<https://github.com/realworldocaml/examples/blob/v1/code/classes-async/shapes.ml>

```
class my_square x y = object
  inherit square 40 x y
  inherit draggable
  inherit animated (Time.Span.of_int_sec 5)
  inherit linear 5 0
  inherit harmonic 0.0 7 ~-10
end

let my_circle = object
  inherit circle 30 250 250
  inherit animated (Time.Span.minute)
  inherit harmonic 0.0 10 0
  inherit harmonic (pi /. 2.0) 0 10
end
```

Отображение анимированных фигур

Мы завершим разработку нашего модуля с геометрическими фигурами созданием функции `main`, осуществляющей рисование нескольких фигур на графическом дисплее, и запустим эту функцию с помощью планировщика `Async`:

OCaml (part 10)

<https://github.com/realworldocaml/examples/blob/v1/code/classes-async/shapes.ml>

```
let main () =
  let shapes = [
    (my_circle :> drawable);
    (new my_square 50 350 :> drawable);
    (new my_square 50 200 :> drawable);
    (new growing_circle 20 70 70 :> drawable);
  ] in
  let repaint () =
    clear_graph ();
    List.iter ~f:(fun s -> s#draw) shapes;
    synchronize ()
  in
  open_graph "";
  auto_synchronize false;
  Clock.every (Time.Span.of_sec (1.0 /. 24.0)) repaint

let () = never_returns (Scheduler.go_main ~main ())
```

Наша функция `main` создает список фигур для отображения и определяет функцию `repaint`, которая фактически рисует их на экране. Затем мы открываем графический дисплей и посылаем планировщику `Async` запрос на запуск `repaint` через регулярные интервалы времени.

Наконец, скомпилируем программу и скомпонуем ее с пакетом `async_graphics`:

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/classes-async/build_shapes.out

```
$ corebuild -pkg async_graphics shapes.native
```

После запуска выполняемого файла на экране должно появиться новое графическое окно (в Mac OS X вам предварительно потребуется установить пакет X11). Попробуйте пощелкать мышью на разных фигурах – и удивитесь сложным анимационным эффектам.

Графическая библиотека, представленная здесь, – единственная библиотека этой направленности, встроенная в OCaml. Но она более полезна как инструмент обучения, чем для чего-либо еще. Однако существует несколько сторонних библиотек, реализующих привязки для доступа к более мощным графическим подсистемам:

- Lablgtk – строго типизированный интерфейс к библиотеке виджетов GTK;
- LablGL – интерфейс между OCaml и OpenGL, поддерживает широко используемые стандарты отображения трехмерной графики;
- js_of_ocaml – компилятор кода на OCaml в код на JavaScript и включает интерфейс к WebGL. Это недавно появившийся стандарт отображения трехмерной графики в веб-браузерах.

Инструменты и технологии

Часть II этой книги опирается на основы и демонстрирует более продвинутые инструменты и технологии, которые с успехом можно использовать для создания действующих программ. Далее вы познакомитесь с библиотеками, полезными для создания практичных приложений, а также с шаблонами проектирования, которые помогают комбинировать различные особенности языка для достижения наилучшего результата.

Эта часть начинается со знакомства с двумя наиболее широко используемыми структурами данных в языке OCaml: отображения (maps) и хэш-таблицы. Затем мы узнаем, как конструировать выразительные интерфейсы командной строки, которые могут служить отличным примером искусного использования системы типов для создания библиотек, дружественных по отношению к пользователю.

В некоторых примерах в этой части используется формат JSON обмена данными, поэтому мы сначала покажем, как пользоваться сторонними инструментами для парсинга и создания данных в формате JSON, а также как сконструировать парсер с нуля. Другим форматом обмена данными, широко используемым в OCaml, являются s-выражения, и в этой части мы покажем, как превратить любое значение OCaml в s-выражение с помощью мощного синтаксического процессора *camlp4*.

Эту часть мы закроем туром по библиотеке Async асинхронных взаимодействий, позволяющей создавать масштабируемые, высокопроизводительные сетевые приложения на языке OCaml. К концу этой части вы научитесь обрабатывать веб-запросы с помощью механизма DuckDuckGo и библиотеки Async, а также библиотек JSON и HTTP, написанных на OCaml.

Отображения и хэш-таблицы

Многие задачи программирования требуют работы с данными, организованными в виде пар ключ/значение. Возможно, самый простой способ представления таких данных в OCaml – *ассоциативный список*, который просто является списком пар ключей и значений. Например, отображение 10 цифр в их названия на английском языке можно представить так:

OCaml utop (part 1)

<https://github.com/realworldocaml/examples/blob/v1/code/maps-and-hash-tables/main.topscript>

```
# let digit_alist =
  [ 0, "zero"; 1, "one"; 2, "two" ; 3, "three"; 4, "four"
    ; 5, "five"; 6, "six"; 7, "seven"; 8, "eight"; 9, "nine" ]
;;
val digit_alist : (int * string) list =
  [(0, "zero"); (1, "one"); (2, "two"); (3, "three"); (4, "four");
   (5, "five"); (6, "six"); (7, "seven"); (8, "eight"); (9, "nine")]
```

Для работы с этими данными можно использовать функции из модуля List.Assoc:

OCaml utop (part 2)

<https://github.com/realworldocaml/examples/blob/v1/code/maps-and-hash-tables/main.topscript>

```
# List.Assoc.find digit_alist 6;;
- : string option = Some "six"
# List.Assoc.find digit_alist 22;;
- : string option = None
# List.Assoc.add digit_alist 0 "zilch";;
- : (int, string) List.Assoc.t =
  [(0, "zilch"); (1, "one"); (2, "two"); (3, "three"); (4, "four");
   (5, "five"); (6, "six"); (7, "seven"); (8, "eight"); (9, "nine")]
```

Ассоциативные списки просты и удобны в использовании, но их производительность далека от идеала, потому что практически любая нетривиальная операция с ассоциативным списком требует времени на его сканирование, прямо пропорциональное его длине.

В этой главе мы поговорим о двух более эффективных альтернативах ассоциативным спискам: *отображениях (maps)* и *хэш-таблицах*. Отображение – это

неизменяемая древовидная структура данных, продолжительность большинства операций с которой логарифмически пропорциональна размеру отображения, а хэш-таблица – изменяемая структура данных, продолжительность большинства операций с которой является постоянной величиной. Мы подробно опишем обе эти структуры данных и дадим некоторые рекомендации по выбору той или иной в разных ситуациях.

Отображения

Рассмотрим пример использования отображения на практике. В главе 4 демонстрируется модуль `Counter` для подсчета частоты встречаемости строк во множестве. Вот как выглядит интерфейс этого модуля:

OCaml

<https://github.com/realworldocaml/examples/blob/v1/code/files-modules-and-programs-freq-fast/counter.mli>

```
open Core.Std
```

```
(** Коллекция счетчиков строк *)
type t
```

```
(** Пустая коллекция счетчиков строк *)
val empty : t
```

```
(** Нарастивает счетчик для указанной строки. *)
val touch : t -> string -> t
```

```
(* Преобразует коллекцию счетчиков в ассоциативный список. Строки хранятся
   в единственном экземпляре, а счетчики имеют значения >= 1. *)
val to_list : t -> (string * int) list
```

Как видите, интерфейс весьма прост. `Counter.empty` представляет пустую коллекцию счетчиков строк; метод `touch` увеличивает счетчик для указанной строки на 1; а метод `to_list` возвращает список ненулевых счетчиков.

Ниже приводится реализация модуля:

OCaml

<https://github.com/realworldocaml/examples/blob/v1/code/files-modules-and-programs-freq-fast/counter.ml>

```
open Core.Std
```

```
type t = int String.Map.t
```

```
let empty = String.Map.empty
```

```
let to_list t = Map.to_alist t
```

```
let touch t s =
  let count =
    match Map.find t s with
    | None -> 0
```

```
| Some x -> x
in
Map.add t ~key:s ~data:(count + 1)
```

Обратите внимание, что в одних случаях в предыдущем коде используются ссылки на `String.Map.t`, а в других — на `Map.t`. Это обусловлено тем фактом, что отображения реализованы в виде упорядоченных двоичных деревьев и как таковые нуждаются в некотором способе сравнения ключей.

С этой целью однажды созданное отображение сохраняет необходимую функцию сравнения внутри структуры данных. То есть операции, такие как `Map.find` или `Map.add`, обращающиеся к содержимому отображения или создающие новое отображение на основе существующего, делают это с применением функции сравнения, встроенной в отображение.

Но, чтобы получить отображение, вам необходимо указать, какую функцию сравнения использовать. По этой причине модули, такие как `String`, содержат подмодуль `Map`, имеющий такие значения, как `String.Map.empty` и `String.Map.of_alist`, специализированные для строк и дающие доступ к функции сравнения строк. Такой подмодуль `Map` включается во все модули, удовлетворяющие интерфейсу `Comparable.S` из библиотеки `Core`.

Создание отображений с компараторами

Специализированные подмодули `Map` — это удобно, но это не единственный способ создания `Map.t`. Информация, необходимая для сравнения значений заданного типа, заворачивается в значение под названием *компаратор* (*comparator*), которое можно использовать для создания отображений с использованием модуля `Map` непосредственно:

OCaml utop (part 3)

<https://github.com/realworldocaml/examples/blob/v1/code/maps-and-hash-tables/main.topscript>

```
# let digit_map = Map.of_alist_exn digit_alist
~comparator: Int.comparator;;
val digit_map : (int, string, Int.comparator) Map.t = <abstr>
# Map.find digit_map 3;;
- : string option = Some "three"
```

В этом примере применяются функция `Map.of_alist_exn`, которая создает отображение из ассоциативного списка и возбуждает исключение, если в списке обнаруживаются повторяющиеся ключи.

Компаратор — единственное, что требуется для операций создания отображений с нуля. Операции, изменяющие существующее отображение, просто наследуют компаратор отображения:

OCaml utop (part 4)

<https://github.com/realworldocaml/examples/blob/v1/code/maps-and-hash-tables/main.topscript>

```
# let zilch_map = Map.add digit_map ~key:0 ~data:"zilch";;
val zilch_map : (int, string, Int.comparator) Map.t = <abstr>
```

Тип `Map.t` имеет три параметра типов: один определяет тип ключей, один — тип значений, и один идентифицирует сам компаратор. В действительности тип `'a Int.Map.t` — это всего лишь псевдоним для типа `(int, 'a, Int.comparator) Map.t`.

Включение компаратора в тип играет важную роль, потому что операции, воздействующие сразу на несколько отображений, часто требуют, чтобы отображения совместно использовали одну и ту же функцию сравнения. Взгляните, например, на функцию `Map.symmetric_diff`, которая определяет различия между двумя отображениями:

OCaml utop (part 5)

<https://github.com/realworldocaml/examples/blob/v1/code/maps-and-hash-tables/main.topscript>

```
# let left = String.Map.of_alist_exn ["foo",1; "bar",3; "snoo",0]
  let right = String.Map.of_alist_exn ["foo",0; "snoo",0]
  let diff = Map.symmetric_diff ~data_equal:Int.equal left right
;;
val left : int String.Map.t = <abstr>
val right : int String.Map.t = <abstr>
val diff :
  (string * [ `Left of int | `Right of int | `Unequal of int * int ]) list =
  [("foo", `Unequal (1, 0)); ("bar", `Left 3)]
```

Тип функции `Map.symmetric_diff`, что приводится ниже, требует, чтобы два отображения, которые она сравнивает, имели компаратор одного и того же типа. Каждый компаратор имеет новый абстрактный тип, поэтому тип компаратора уникально идентифицирует этот компаратор:

OCaml utop (part 6)

<https://github.com/realworldocaml/examples/blob/v1/code/maps-and-hash-tables/main.topscript>

```
# Map.symmetric_diff;;
- : ('k, 'v, 'cmp) Map.t ->
  ('k, 'v, 'cmp) Map.t ->
  data_equal:('v -> 'v -> bool) ->
  ('k * [ `Left of 'v | `Right of 'v | `Unequal of 'v * 'v ]) list
= <fun>
```

Это ограничение играет важную роль, потому что алгоритм, используемый функцией `Map.symmetric_diff`, дает корректные результаты, только если оба отображения имеют один и тот же компаратор.

Создать новый компаратор можно с помощью функтора `Comparator.Make`, который принимает модуль, содержащий тип объекта для сравнения, функции `sexp-преобразований` и функцию сравнения. Функции `sexp-преобразований` включаются в компаратор, чтобы обеспечить для пользователей компаратора возможность генерировать более информативные сообщения об ошибках. Например:

OCaml utop (part 7)

<https://github.com/realworldocaml/examples/blob/v1/code/maps-and-hash-tables/main.topscript>

```
# module Reverse = Comparator.Make(struct
  type t = string
```

```

    let sexp_of_t = String.sexp_of_t
    let t_of_sexp = String.t_of_sexp
    let compare x y = String.compare y x
  end);;
module Reverse :
sig
  type t = string
  val compare : t -> t -> int
  val t_of_sexp : Sexp.t -> t
  val sexp_of_t : t -> Sexp.t
  type comparator
  val comparator : (t, comparator) Comparator.t_
end

```

Как видно из следующего примера, оба компаратора, `Reverse.comparator` и `String.comparator`, можно использовать для создания отображений с ключами-строками:

OCaml utop (part 8)

<https://github.com/realworldocaml/examples/blob/v1/code/maps-and-hash-tables/main.topscript>

```

# let alist = ["foo", 0; "snoo", 3];;
val alist : (string * int) list = [("foo", 0); ("snoo", 3)]
# let ord_map = Map.of_alist_exn ~comparator:String.comparator alist;;
val ord_map : (string, int, String.comparator) Map.t = <abstr>
# let rev_map = Map.of_alist_exn ~comparator:Reverse.comparator alist;;
val rev_map : (string, int, Reverse.comparator) Map.t = <abstr>

```

`Map.min_elt` возвращает ключ и значение для наименьшего ключа в отображении. Это позволяет убедиться, что два отображения действительно используют разные функции сравнения:

OCaml utop (part 9)

<https://github.com/realworldocaml/examples/blob/v1/code/maps-and-hash-tables/main.topscript>

```

# Map.min_elt ord_map;;
- : (string * int) option = Some ("foo", 0)
# Map.min_elt rev_map;;
- : (string * int) option = Some ("snoo", 3)

```

Соответственно, если попытаться применить `Map.symmetric_diff` к этим двум отображениям, компилятор выведет сообщение об ошибке:

OCaml utop (part 10)

<https://github.com/realworldocaml/examples/blob/v1/code/maps-and-hash-tables/main.topscript>

```

# Map.symmetric_diff ord_map rev_map;;

```

Characters 27-34:

```

Error: This expression has type (string, int, Reverse.comparator) Map.t
      but an expression was expected of type
      (string, int, String.comparator) Map.t
      Type Reverse.comparator is not compatible with type String.comparator
(Ошибка: Это выражение имеет тип (string, int, Reverse.comparator) Map.t,

```

тогда как ожидалось выражение типа
 (string, int, String.comparator) Map.t
 Тип Reverse.comparator несовместим с типом String.comparator)

Деревья

Как уже отмечалось, отображения содержат в себе компараторы, включаемые в них при их создании. Иногда, часто по надуманным причинам, бывает желательно получить версию отображения, не включающую компаратора. Такое представление данных можно получить с помощью функции `Map.to_tree`, возвращающей дерево, которое является основой отображения, без компаратора:

OCaml utop (part 11)

<https://github.com/realworldocaml/examples/blob/v1/code/maps-and-hash-tables/main.topscript>

```
# let ord_tree = Map.to_tree ord_map;;
val ord_tree : (string, int, String.comparator) Map.Tree.t = <abstr>
```

Даже при том, что значения типа `Map.Tree.t` физически не включают компараторов, сам тип содержит определение компаратора. Это определение известно как *фантомный* тип, потому что отражает некоторые сведения о логике организации рассматриваемого значения, даже при том, что не соответствует какому-либо физическому значению в структуре данных.

Так как компаратор не включается в дерево, его приходится предоставлять явно, когда, например, выполняется поиск ключа, как показано ниже:

OCaml utop (part 12)

<https://github.com/realworldocaml/examples/blob/v1/code/maps-and-hash-tables/main.topscript>

```
# Map.Tree.find ~comparator:String.comparator ord_tree "snoo";;
- : int option = Some 3
```

Алгоритм работы `Map.Tree.find` зависит от соответствия компаратора, используемого для поиска значения, тому, что применялся при его сохранении. Это как раз тот случай, когда в игру вступает фантомный тип. Как показано в следующем примере, применение неверного компаратора приводит к ошибкам компилятора:

OCaml utop (part 13)

<https://github.com/realworldocaml/examples/blob/v1/code/maps-and-hash-tables/main.topscript>

```
# Map.Tree.find ~comparator:Reverse.comparator ord_tree "snoo";;
Characters 45-53:
Error: This expression has type (string, int, String.comparator) Map.Tree.t
      but an expression was expected of type
      (string, int, Reverse.comparator) Map.Tree.t
Type String.comparator is not compatible with type Reverse.comparator
(Ошибка: Это выражение имеет тип (string, int, Reverse.comparator) Map.Tree.t,
тогда как ожидалось выражение типа
(string, int, Reverse.comparator) Map.Tree.t
Тип String.comparator несовместим с типом Reverse.comparator)
```

Полиморфные компараторы

Необязательно создавать специализированные компараторы для каждого типа, построенного на основе отображения. Вместо этого можно использовать компаратор, опирающийся на встроенную полиморфную функцию сравнения, которая обсуждалась в главе 3. Этот компаратор, находящийся в модуле `Comparator.Poly`, позволяет писать такой код:

OCaml utop (part 14)

<https://github.com/realworldocaml/examples/blob/v1/code/maps-and-hash-tables/main.topscript>

```
# Map.of_alist_exn ~comparator:Comparator.Poly.comparator digit_alist;;
- : (int, string, Comparator.Poly.comparator) Map.t = <abstr>
```

Или такой:

OCaml utop (part 15)

<https://github.com/realworldocaml/examples/blob/v1/code/maps-and-hash-tables/main.topscript>

```
# Map.Poly.of_alist_exn digit_alist;;
- : (int, string) Map.Poly.t = <abstr>
```

Обратите внимание, что с точки зрения компилятора отображения, основанные на полиморфном компараторе, не эквивалентны тем, что опираются на компараторы конкретных типов. Например, компилятор отвергнет следующий код:

OCaml utop (part 16)

<https://github.com/realworldocaml/examples/blob/v1/code/maps-and-hash-tables/main.topscript>

```
# Map.symmetric_diff (Map.Poly.singleton 3 "three")
                      (Int.Map.singleton 3 "four" ) ;;
```

Characters 72-99:

```
Error: This expression has type
  string Int.Map.t = (int, string, Int.comparator) Map.t
but an expression was expected of type
  (int, string, Comparator.Poly.comparator) Map.t
Type Int.comparator is not compatible with type
  Comparator.Poly.comparator
```

(Ошибка: Это выражение имеет тип

```
  string Int.Map.t = (int, string, Int.comparator) Map.t
тогда как ожидается выражение типа
  (int, string, Comparator.Poly.comparator) Map.t
Тип Int.comparator несовместим с типом
  Comparator.Poly.comparator)
```

На то есть веские причины: в данном случае нет никакой гарантии, что логика компаратора, ассоциированного с указанным типом, будет соответствовать логике полиморфного компаратора.

Опасности полиморфного сравнения

Полиморфное сравнение очень удобно в использовании, но оно имеет серьезные недостатки и должно применяться с большой осторожностью. В частности, полиморфное

сравнение реализует фиксированный алгоритм сравнения значений любых типов, и иногда этот алгоритм может давать неожиданные результаты.

Чтобы понять, что не так с полиморфным сравнением, необходимо разобраться с особенностями его работы. Полиморфное сравнение выполняется структурно, то есть оно оперирует значениями, представленными во время выполнения, выполняет обход структуры значений, невзирая на их типы.

Это удобно, потому что позволяет сравнивать большинство значений OCaml, и чаще сравнение выполняется в полном соответствии с ожиданиями. Например, для целых и вещественных чисел этот алгоритм действует по аналогии с функцией численного сравнения. Для простых контейнеров, таких как строки, списки и массивы, он выполняет лексикографическое сравнение. И кроме функций и значений, хранящихся за пределами кучи (heap) OCaml, он способен сравнивать значения практически любых типов OCaml. Но иногда структурное сравнение является нежелательным. Отличным примером того могут служить множества. Взгляните на следующие два множества:

OCaml utop (part 18)

<https://github.com/realworldocaml/examples/blob/v1/code/maps-and-hash-tables/main.topscript>

```
# let (s1,s2) = (Int.Set.of_list [1;2],
                Int.Set.of_list [2;1]);;
val s1 : Int.Set.t = <abstr>
val s2 : Int.Set.t = <abstr>
```

Логически эти два множества эквивалентны, и это доказывает результат, возвращаемый функцией `Set.equal`:

OCaml utop (part 19)

<https://github.com/realworldocaml/examples/blob/v1/code/maps-and-hash-tables/main.topscript>

```
# Set.equal s1 s2;;
- : bool = true
```

Но из-за того, что элементы добавлялись в разном порядке, внутренняя структура деревьев, лежащих в основе этих множеств, будет отличаться. Как результат функция структурного сравнения сообщит, что они различны.

Давайте посмотрим, что случится, если для проверки равенства воспользоваться оператором `=`, чтобы задействовать полиморфное сравнение. Непосредственное сравнение двух отображений потерпит неудачу во время выполнения, потому что компараторы, хранящиеся внутри множеств, содержат значения-функции:

OCaml utop (part 20)

<https://github.com/realworldocaml/examples/blob/v1/code/maps-and-hash-tables/main.topscript>

```
# s1 = s2;;
Exception: (Invalid_argument "equal: functional value").
```

Однако мы можем воспользоваться функцией `Set.to_tree`, чтобы выделить внутренние деревья, не имеющие встроенных компараторов:

OCaml utop (part 21)

<https://github.com/realworldocaml/examples/blob/v1/code/maps-and-hash-tables/main.topscript>

```
# Set.to_tree s1 = Set.to_tree s2;;
- : bool = false
```

Данный прием может постоянно вызывать ошибки. Если, к примеру, вы используете отображение с ключами, содержащими множества, тогда отображение, созданное с полиморфным компаратором, будет действовать некорректно, считая по сути одинаковые ключи разными. Хуже того, иногда оно будет работать правильно, а иногда нет – пока множества будут конструироваться в каком-то жестко определенном порядке, отображение будет работать в соответствии с ожиданиями, но как только порядок создания изменится, поведение отображения также изменится.

Множества

Иногда вместо хранения множества пар ключ/значение желательно хранить только множество ключей. Такую структуру данных можно было бы реализовать на основе отображения, представляя значения в нем типом `unit`. Но более идиоматичным (и эффективным) будет использовать тип `set` из библиотеки `Core`, который схож с отображениями, но имеет API, оптимизированный для работы со множествами и потребляющий меньше памяти. Далее приводится простой пример использования этого типа:

OCaml utop (part 17)

<https://github.com/realworldocaml/examples/blob/v1/code/maps-and-hash-tables/main.topscript>

```
# let dedup ~comparator l =
  List.fold l ~init:(Set.empty ~comparator) ~f:Set.add
    |> Set.to_list
;;
val dedup :
  comparator:('a, 'b) Core_kernel.Comparator.t_ -> 'a list -> 'a list = <fun>
# dedup ~comparator:Int.comparator [8;3;2;3;7;8;10];;
- : int list = [2; 3; 7; 8; 10]
```

В дополнение к операторам для работы с отображениями множества поддерживают дополнительные, традиционные для множеств операции, включая объединение, пересечение и разность. Как и в случае с отображениями, мы можем создавать множества с привлечением компараторов конкретного типа или полиморфного компаратора.

Соответствие интерфейсу `Comparable.S`

Интерфейс `Comparable.S` из библиотеки `Core` включает массу полезных функциональных особенностей, включая поддержку операций с отображениями и множествами. В частности, `Comparable.S` требует наличия подмодулей `Map` и `Set`, а также компаратора.

Интерфейсу `Comparable.S` соответствует большинство типов в `Core`, поэтому сразу возникает вопрос: как реализовать поддержку этого интерфейса в своих новых типах. Конечно, реализация всех функциональных особенностей с нуля была бы неподъемным делом.

Модуль `Comparable` содержит ряд функторов, помогающих автоматизировать эту задачу. Простейшим из них является `Comparable.Make`, который принимает любой модуль, удовлетворяющий требованиям следующего интерфейса:

OCaml

<https://github.com/realworldocaml/examples/tree/v1/code/maps-and-hash-tables/comparable.ml>

```
module type Comparable = sig
  type t
  val sexp_of_t : t -> Sexp.t
  val t_of_sexp : Sexp.t -> t
  val compare : t -> t -> int
end
```

Иными словами, он ожидает получить тип с функцией сравнения, а также функции для преобразования в *s-выражения* и обратно. S-выражения – это формат сериализации, широко используемый в библиотеке Core и необходимый для вывода более информативных сообщений об ошибках. Более подробно s-выражения будут обсуждаться в главе 17, а пока будем пользоваться объявлением `sexpr` из расширения синтаксиса `Sexplib`. Это объявление обеспечивает автоматическое создание функций преобразования s-выражений для указанного типа.

Следующий пример демонстрирует, как все это работает, и следует тому же шаблону использования функторов, описывавшемуся в разделе «Расширение модулей» в главе 9:

OCaml utop

<https://github.com/realworldocaml/examples/tree/v1/code/maps-and-hash-tables/main-22.rawscript>

```
# module Foo_and_bar : sig
  type t = { foo: Int.Set.t; bar: string }
  include Comparable.S with type t := t
end = struct
  module T = struct
    type t = { foo: Int.Set.t; bar: string } with sexp
    let compare t1 t2 =
      let c = Int.Set.compare t1.foo t2.foo in
      if c <> 0 then c else String.compare t1.bar t2.bar
    end
    include T
    include Comparable.Make(T)
  end;;
module Foo_and_bar :
sig
  type t = { foo : Int.Set.t; bar : string; }
  val ( >= ) : t -> t -> bool
  val ( <= ) : t -> t -> bool
  val ( = ) : t -> t -> bool
  ...
end
```

Мы не включили полный вывод интерактивной оболочки, потому что он слишком объемный, но и из этого фрагмента видно, что `Foo_and_bar` удовлетворяет требованиям интерфейса `Comparable.S`.

В предыдущем коде мы определили функцию сравнения вручную, но в этом нет строгой необходимости. В состав Core входит расширение синтаксиса с именем `comparelib`, которое создает функцию сравнения, исходя из определения типа. С его помощью предыдущий пример можно записать, как показано ниже:

OCaml utop

<https://github.com/realworldocaml/examples/tree/v1/code/maps-and-hash-tables/main-23.rawscript>

```
# module Foo_and_bar : sig
  type t = { foo: Int.Set.t; bar: string }
  include Comparable.S with type t := t
end = struct
```

```

module T = struct
  type t = { foo: Int.Set.t; bar: string } with sexp, compare
end
include T
include Comparable.Make(T)
end;;

module Foo_and_bar :
sig
  type t = { foo : Int.Set.t; bar : string; }
  val ( >= ) : t -> t -> bool
  val ( <= ) : t -> t -> bool
  val ( = ) : t -> t -> bool
  ...
end

```

Функция сравнения, созданная библиотекой `comparelib` для данного типа, будет обращаться к функциям сравнения для типов компонентов. Как результат поле `foo` будет сравниваться с помощью `Int.Set.compare`. Такой подход отличается от структурного подхода, реализованного полиморфным сравнением, и является более правильным.

Если вам нужна функция сравнения, действующая некоторым особым образом, тогда вам придется самим написать ее; но если вам достаточно всего лишь определить порядок сравнения, необходимый для создания отображений и множеств, библиотека `comparelib` будет отличным выбором.

Удовлетворить требования интерфейса `Comparable.S` можно также с применением полиморфного сравнения:

OCaml utop

<https://github.com/realworldocaml/examples/blob/v1/code/maps-and-hash-tables/main-24.rawscript>

```

# module Foo_and_bar : sig
  type t = { foo: int; bar: string }
  include Comparable.S with type t := t
end = struct
  module T = struct
    type t = { foo: int; bar: string } with sexp
  end
  include T
  include Comparable.Poly(T)
end;;

module Foo_and_bar :
sig
  type t = { foo : int; bar : string; }
  val ( >= ) : t -> t -> bool
  val ( <= ) : t -> t -> bool
  val ( = ) : t -> t -> bool
  ...
end

```

Однако, учитывая причины, описанные выше, полиморфное сравнение следует использовать с осторожностью.

=, == и phys_equal

Если у вас есть опыт программирования на C/C++, вы, вероятно, по привычке будете использовать оператор `==` для проверки равенства значений. В языке OCaml оператор `==` проверяет физическое равенство, а оператор `=` — структурное.

Физически равными считаются структуры, если они находятся по одному и тому же адресу в памяти. Две структуры с идентичным содержимым, но созданные по отдельности, никогда не будут признаны равными оператором `==`.

Оператор `=` структурного равенства рекурсивно исследует каждое поле в двух значениях и сравнивает их по отдельности. Самое важное: если структура данных циклическая (то есть значение рекурсивно ссылается назад, на другое поле в той же структуре), выполнение оператора `=` никогда не завершится, и программа зависнет! Поэтому для сравнения циклических структур следует использовать оператор проверки физического равенства или писать собственные функции сравнения.

Так как операторы `=` и `==` легко перепутать, библиотека Core запрещает оператор `==` и предоставляет вместо него более явную функцию `phys_equal`. Попытавшись использовать оператор `==` в коде, открывающем Core.Std, вы получите сообщение об ошибке:

OCaml utop

https://github.com/realworldocaml/examples/blob/v1/code/maps-and-hash-tables/core_phys_equal.topscript

```
# open Core.Std ;;
# 1 == 2 ;;
Characters -1-1:
Error: This expression has type int but an expression was expected of type
      [ `Consider_using_phys_equal ]
(Ошибка: Это выражение имеет тип int, тогда как ожидается выражение типа
      [ `Consider_using_phys_equal ])
# phys_equal 1 2 ;;
- : bool = false
```

Если вам не терпится подвесить свой интерпретатор OCaml, вы можете сделать это с помощью рекурсивных значений и операции структурной проверки на равенство:

OCaml utop

https://github.com/realworldocaml/examples/blob/v1/code/maps-and-hash-tables/phys_equal.rawscript

```
# type t1 = { foo1:int; bar1:t2 } and t2 = { foo2:int; bar2:t1 } ;;
type t1 = { foo1 : int; bar1 : t2; }
and t2 = { foo2 : int; bar2 : t1; }
# let rec v1 = { foo1=1; bar1=v2 } and v2 = { foo2=2; bar2=v1 } ;;
<lots of text>
# v1 == v1;;
- : bool = true
# phys_equal v1 v1;;
- : bool = true
# v1 = v1 ;;
<press ^Z and kill the process now>
```

Хэш-таблицы

Хэш-таблицы — это императивные родственники отображений. Мы рассмотрели реализацию простой хэш-таблицы в главе 8, поэтому в данном разделе основное внимание будет уделено более практичному модулю `Hashtbl` из библиотеки Core.

Однако мы не будем углубляться в тонкости, как при исследовании отображений, потому что многие понятия остаются неизменными.

Хэш-таблицы имеют несколько ключевых отличий от отображений. Во-первых, хэш-таблицы являются изменяемыми, в том смысле, что добавление пары ключ/значение в хэш-таблицу изменяет существующую таблицу, а не создает новую. Во-вторых, хэш-таблицы обычно имеют более высокую производительность, обеспечивая постоянное время поиска и изменения в противоположность логарифмическому времени у отображений. И наконец, так же как отображения зависят от наличия функции сравнения, применяемой для создания упорядоченного двоичного дерева, лежащего в основе отображения, хэш-таблицы зависят от наличия функции хэширования, то есть функции преобразования ключей в целые числа.

Производительность хэш-таблиц

Постоянное время доступа к элементам, которое обеспечивают хэш-таблицы, скрывает некоторые сложности. Прежде всего все реализации хэш-таблиц в OCaml должны изменять размер таблицы по мере ее наполнения. Для этого требуется распределять в памяти новый массив, используемый для хранения хэш-таблицы, и копировать все записи в новый массив, а это довольно дорогостоящая операция. Это означает, что время добавления нового элемента в таблицу является лишь амортизированным постоянным, то есть оно остается в среднем постоянным в длинной последовательности операций, но некоторые отдельные операции могут оказаться весьма дорогостоящими. Другая скрытая сложность хэш-таблиц имеет отношение к хэш-функции. Если реализация будет включать патологически плохую хэш-функцию, хэширующую любые ключи в одно и то же число, тогда все вставки будут осуществляться в один и тот же блок, вследствие чего время доступа перестанет быть постоянным. Реализация хэш-таблицы в библиотеке Core использует двоичные деревья для организации хранения блоков, поэтому время поиска в таком худшем случае станет логарифмическим, а не линейным, как в традиционных реализациях хэш-таблиц.

Логарифмическая зависимость скорости работы хэш-таблиц из библиотеки Core при наличии хэш-коллизий также помогает защищать против некоторых атак вида «отказ в обслуживании» (Denial-Of-Service, DOS). Одна из хорошо известных атак этого типа заключается в отправке запросов, содержащих специально сконструированные ключи, вызывающие множество коллизий. В комбинации с линейным поведением большинства хэш-таблиц это может привести к отказу службы из-за высокой нагрузки на процессор. Службы на основе хэш-таблиц Core должны быть менее восприимчивы к атакам такого рода, потому что степень деградации производительности будет намного меньше.

При создании хэш-таблицы необходимо предоставить значение типа *хэш-таблицы*, включающее, кроме всего прочего, функцию хэширования ключей данного типа. Такая функция является аналогом компаратора, необходимого для создания отображений:

OCaml utop (part 25)

<https://github.com/realworldocaml/examples/blob/v1/code/maps-and-hash-tables/main.topscrip>

```
# let table = Hashtbl.create ~hashable:String.hashable ();;
val table : (string, 'a) Hashtbl.t = <abstr>
# Hashtbl.replace table ~key:"three" ~data:3;;
```

```
- : unit = ()
# Hashtbl.find table "three";;
- : int option = Some 3
```

Значение `hashable` включается как часть интерфейса `Hashable.S`, которому удовлетворяет большинство типов в библиотеке `Core`. Интерфейс `Hashable.S` также включается подмодулем `Table`, который реализует более удобные функции создания:

OCaml utop (part 26)

<https://github.com/realworldocaml/examples/blob/v1/code/maps-and-hash-tables/main.topscript>

```
# let table = String.Table.create ();;
val table : '_a String.Table.t = <abstr>
```

Имеется также полиморфное значение `hashable`, соответствующее полиморфной хэш-функции, которую предоставляет среда времени выполнения OCaml для случаев, не имеющих собственной функции хэширования:

OCaml utop (part 27)

<https://github.com/realworldocaml/examples/blob/v1/code/maps-and-hash-tables/main.topscript>

```
# let table = Hashtbl.create ~hashable:Hashtbl.Poly.hashable ();;
val table : ('_a, '_b) Hashtbl.t = <abstr>
```

или эквивалентный способ:

OCaml utop (part 28)

<https://github.com/realworldocaml/examples/blob/v1/code/maps-and-hash-tables/main.topscript>

```
# let table = Hashtbl.Poly.create ();;
val table : ('_a, '_b) Hashtbl.t = <abstr>
```

Обратите внимание, что, в отличие от компараторов, используемых с отображениями и множествами, хэш-таблицы отсутствуют в типе `Hashtbl.t`. Это обусловлено тем, что хэш-таблицы не имеют операций, которые воздействовали бы одновременно на несколько хэш-таблиц, зависящих от одной и той же хэш-функции, подобно тому как `Map.symmetric_diff` и `Set.union` зависят от использования одной и той же функции сравнения.

Коллизии при использовании полиморфной хэш-функции

Полиморфная хэш-функция в OCaml действует путем обхода структуры данных сначала в ширину, ограничиваясь определенным числом узлов. По умолчанию функция ограничивается 10 узлами.

Ограничение означает, что хэш-функция может проигнорировать часть структуры данных, что может привести к патологическим ситуациям, когда все такие структуры будут сохраняться с одним и тем же хэш-значением. Эта проблема демонстрируется ниже, с помощью функции `List.range`, используемой для распределения списков целых чисел разной длины:

OCaml utop (part 29)

<https://github.com/realworldocaml/examples/blob/v1/code/maps-and-hash-tables/main.topscript>

```
# Caml.Hashtbl.hash (List.range 0 9);;
- : int = 209331808
# Caml.Hashtbl.hash (List.range 0 10);;
- : int = 182325193
# Caml.Hashtbl.hash (List.range 0 11);;
- : int = 182325193
# Caml.Hashtbl.hash (List.range 0 100);;
- : int = 182325193
```

Как видите, хэш-функция прекращает обход списка после первых 10 элементов. То же самое может происходить с любыми большими структурами данных, включая записи и массивы. Предусматривая возможность хэширования собственных структур данных большого размера, обычно бывает желательно написать собственную хэш-функцию.

Соответствие интерфейсу Hashable.S

Большинство типов в библиотеке Core удовлетворяет требованиям интерфейса Hashable.S, но, как и в случае с интерфейсом Comparable.S, возникает вопрос: как обеспечить соответствие данному интерфейсу при создании новых модулей? Как и в прошлый раз, ответ на этот вопрос: использовать функтор, реализующий всю необходимую функциональность, — в данном случае Hashable.Make. Обратите внимание, что для выполнения «логической» (то есть поразрядной) операции «исключающее ИЛИ» хэш-значений компонентов структуры мы используем оператор `lxor`:

OCaml utop

<https://github.com/realworldocaml/examples/blob/v1/code/maps-and-hash-tables/main-30.rawscript>

```
# module Foo_and_bar : sig
  type t = { foo: int; bar: string }
  include Hashable.S with type t := t
end = struct
  module T = struct
    type t = { foo: int; bar: string } with sexp, compare
    let hash t =
      (Int.hash t.foo) lxor (String.hash t.bar)
    end
  include T
  include Hashable.Make(T)
end;;
module Foo_and_bar :
sig
  type t = { foo : int; bar : string; }
  module Hashable : sig type t = t end
  val hash : t -> int
  val compare : t -> t -> int
  val hashable : t Pooled_hashtbl.Hashable.t

  ...

end
```

Отметьте также, чтобы обеспечить соответствие интерфейсу `Hashable.S`, необходимо реализовать функцию сравнения. Это обусловлено тем, что для хранения хэш-блоков (`hashbuckets`) реализация хэш-таблиц в библиотеке `Core` использует упорядоченные двоичные деревья, обеспечивающие более пологую кривую деградации производительности в случае выбора патологически плохой хэш-функции.

В настоящее время не существует аналога `comparelib` для автоматического создания хэш-функций, поэтому вам придется написать собственную хэш-функцию или использовать встроенную, полиморфную хэш-функцию `Hashtbl.hash`.

Выбор между отображениями и хэш-таблицами

Функциональность отображений и хэш-таблиц во многом перекрывается, из-за чего не всегда очевидно, какую из этих структур выбрать. Вообще говоря, отображения, из-за их неизменяемости, являются в языке OCaml выбором по умолчанию. Однако OCaml имеет также неплохую поддержку императивного программирования, а при программировании в императивном стиле хэш-таблицы часто становятся более естественным выбором.

Помимо применяемых идиом программирования, отображения и хэш-таблицы также существенно отличаются своей производительностью. В коде, где в большом количестве выполняются операции изменения и поиска, хэш-таблицы выглядят более предпочтительными, и чем больше объем данных, тем очевиднее преимущества.

Лучший способ выяснить преимущества, связанные с производительностью, — выполнить хронометраж, так давайте сделаем это прямо сейчас. Следующий тест использует библиотеку `core_bench` и сравнивает производительность отображений и хэш-таблиц при очень простой рабочей нагрузке. Здесь мы создаем 1000 различных целочисленных ключей и затем выполняем их обход и изменение соответствующих им значений. Обратите внимание, что тут для изменения соответствующих структур данных используются функции `Map.change` и `Hashtbl.change`:

OCaml

https://github.com/realworldocaml/examples/blob/v1/code/maps-and-hash-tables/map_vs_hash.ml

```
open Core.Std
open Core_bench.Std

let map_iter ~num_keys ~iterations =
  let rec loop i map =
    if i <= 0 then ()
    else loop (i - 1)
      (Map.change map (i mod num_keys) (fun current ->
        Some (1 + Option.value ~default:0 current)))
  in
  loop iterations Int.Map.empty

let table_iter ~num_keys ~iterations =
```



```

let table = Int.Table.create ~size:num_keys () in
let rec loop i =
  if i <= 0 then ()
  else (
    Hashtbl.change table (i mod num_keys) (fun current ->
      Some (1 + Option.value ~default:0 current));
    loop (i - 1)
  )
in
loop iterations

let tests ~num_keys ~iterations =
  let test name f = Bench.Test.create f ~name in
  [ test "map" (fun () -> map_iter ~num_keys ~iterations)
  ; test "table" (fun () -> table_iter ~num_keys ~iterations)
  ]

let () =
  tests ~num_keys:1000 ~iterations:100_000
  |> Bench.make_command
  |> Command.run

```

Результаты показывают, что версия на основе хэш-таблицы почти в четыре раза быстрее версии на основе отображения:

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/maps-and-hash-tables/run_map_vs_hash.out

```

$ corebuild -pkg core_bench map_vs_hash.native
$ ./map_vs_hash.native -ascii -clear-columns name time speedup
Estimated testing time 20s (change using -quota SECS).

```

Name	Time (ns)	Speedup
map	31_698_313	1.00
table	7_202_631	4.40

Скорость работы также зависит от некоторых других особенностей. Например, она зависит от числа различных ключей. Но в общем случае для кода, обрабатывающего длинные последовательности запросов и часто изменяющего множество пар ключ/значение, хэш-таблицы существенно превосходят отображения.

Но хэш-таблицы не всегда превосходят отображения по скорости. В частности, отображения часто оказываются более производительными в ситуациях, когда требуется хранить в памяти несколько взаимосвязанных версий структур данных в памяти. Это объясняется неизменяемостью отображений, и потому операции, такие как `Map.add`, изменяющие отображение, создают новое отображение, оставляя первоначальное отображение в неприкосновенности. Кроме того, новое и старое отображения совместно используют большую часть физической структуры данных в памяти, поэтому хранение нескольких версий осуществляется более эффективно.

Следующий пример, выполняющий хронометраж, доказывает это. В нем мы создаем список отображений (или хэш-таблиц), который конструируется итеративно, за счет выполнения небольших изменений и с сохранением копий. В версии с отображениями изменения выполняются с помощью функции `Map.change`. В версии с хэш-таблицами – с помощью функции `Hashtbl.change`, но при этом нам приходится также вызывать `Hashtbl.copy`, чтобы создать копию очередной версии таблицы:

OCaml

https://github.com/realworldocaml/examples/blob/v1/code/maps-and-hash-tables/map_vs_hash2.ml

```
open Core.Std
open Core_bench.Std

let create_maps ~num_keys ~iterations =
  let rec loop i map =
    if i <= 0 then []
    else
      let new_map =
        Map.change map (i mod num_keys) (fun current ->
          Some (1 + Option.value ~default:0 current))
        in
      new_map :: loop (i - 1) new_map
  in
  loop iterations Int.Map.empty

let create_tables ~num_keys ~iterations =
  let table = Int.Table.create ~size:num_keys () in
  let rec loop i =
    if i <= 0 then []
    else (
      Hashtbl.change table (i mod num_keys) (fun current ->
        Some (1 + Option.value ~default:0 current));
      let new_table = Hashtbl.copy table in
      new_table :: loop (i - 1)
    )
  in
  loop iterations

let tests ~num_keys ~iterations =
  let test name f = Bench.Test.create f ~name in
  [ test "map" (fun () -> ignore (create_maps ~num_keys ~iterations))
  ; test "table" (fun () -> ignore (create_tables ~num_keys ~iterations))
  ]

let () =
  tests ~num_keys:50 ~iterations:1000
  |> Bench.make_command
  |> Command.run
```

Неудивительно, что в этом случае отображения показывают лучшую, более чем в 10 раз, производительность:

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/maps-and-hash-tables/run_map_vs_hash2.out

```
$ corebuild -pkg core_bench map_vs_hash2.native
$ ./map_vs_hash2.native -ascii -clear-columns name time speedup
Estimated testing time 20s (change using -quota SECS).
```

Name	Time (ns)	Speedup
map	218_581	12.03
table	2_628_423	1.00

Превосходство только растёт с увеличением размеров таблиц или списков.

Как видите, относительная производительность деревьев и отображений в значительной степени зависит от особенностей их использования, и, соответственно, выбор той или иной структуры данных также зависит от особенностей приложения.

Анализ командной строки

Многие программы на OCaml, которые вам доведется создавать, в конечном итоге будут преобразованы в двоичные выполняемые файлы, запускаемые из командной строки. Любая программа командной строки должна поддерживать следующие основные возможности:

- принимать и анализировать аргументы командной строки;
- выводить сообщения об ошибках в ответ на некорректный ввод;
- выводить справку с описанием всех имеющихся параметров командной строки;
- автоматически дополнять ввод пользователя.

Писать все это вновь и вновь в каждой программе не только утомительно, но и чревато ошибками. Библиотека Core включает в себя библиотеку Command, упрощающую реализацию этой задачи, позволяя объявлять все параметры командной строки в одном месте и порождать все упомянутые выше возможности из этих объявлений.

Библиотека Command проста в использовании в простых приложениях и хорошо масштабируется с ростом потребностей. В частности, Command предоставляет поддержку обработки вложенных команд, позволяющую группировать родственные команды в категории. Возможно, кто-то из вас уже знаком с подобным механизмом командной строки по таким программам, как системы управления версиями Git или Mercurial.

В этой главе вы:

- узнаете, как с помощью библиотеки Command конструировать простые и сгруппированные интерфейсы командной строки;
- создадите простые эквиваленты криптографических утилит md5 и shasum;
- увидите, как можно использовать *функциональные комбинаторы* для объявления сложных интерфейсов командной строки безопасным и изящным способом.

Простейший анализ командной строки

Давайте для начала создадим аналог команды `md5sum`, имеющейся в большинстве дистрибутивов Linux (аналогичная команда с именем md5 имеется также в Mac OS X). Следующая функция читает содержимое файла, применяет к данным

несимметричную криптографическую хэш-функцию MD5 и выводит результат в простом текстовом представлении:

OCaml

https://github.com/realworldocaml/examples/blob/v1/code/command-line-parsing/basic_md5.ml

```
open Core.Std
```

```
let do_hash filename =
  In_channel.with_file filename ~f:(fun ic ->
    let open Cryptokit in
    hash_channel (Hash.md5 ()) ic
    |> transform_string (Hexa.encode ())
    |> print_endline
  )
```

Функция `do_hash` принимает параметр `filename` с именем файла и выводит строку с контрольной суммой MD5 на стандартный вывод. Первый шаг на пути превращения этой функции в программу – объявить все возможные параметры командной строки в *спецификации*. Модуль `Command.Spec` определяет комбинаторы, которые можно объединять в цепочки для определения необязательных флагов и позиционных аргументов, а также предусматривать специальные действия (такие как приостановка интерактивного ввода) при вводе определенных комбинаций данных.

Анонимные аргументы

Давайте создадим спецификацию для простого аргумента, который передается непосредственно в командной строке. Такие аргументы часто называют *анонимными аргументами* (*anonymous argument*):

OCaml (part 1)

https://github.com/realworldocaml/examples/blob/v1/code/command-line-parsing/basic_md5.ml

```
let spec =
  let open Command.Spec in
  empty
  +> anon ("filename" %: string)
```

Модуль `Command.Spec` определяет инструменты, необходимые для создания спецификаций командной строки. Спецификация начинается со значения `empty`, к которому затем добавляются другие параметры, с помощью комбинатора `+>`. (Оба значения, использованные в примере, объявлены в модуле `Command.Spec`.)

В данном случае мы определили единственный анонимный аргумент с именем `filename`, который принимает значение типа `string`. Анонимные параметры создаются с помощью оператора `%:`, который связывает текстовое имя (используемое в тексте справки для идентификации параметра) с функцией преобразования, анализирующей фрагменты командной строки и преобразующей их в типы дан-

ных OCaml. В предыдущем примере это – простая функция `Command.Spec.string`, однако далее в этой главе будут представлены более сложные преобразования.

Определение простых команд

После объявления спецификации ее необходимо задействовать для обработки ввода. Самый простой способ – создать интерфейс командной строки непосредственно, с помощью модуля `Command.basic`:

OCaml (part 2)

https://github.com/realworldocaml/examples/blob/v1/code/command-line-parsing/basic_md5.ml

```
let command =
  Command.basic
    ~summary:"Generate an MD5 hash of the input data"
    ~readme:(fun () -> "More detailed information")
    spec
    (fun filename () -> do_hash filename)
```

Модуль `Command.basic` определяет полный интерфейс командной строки, принимающий следующие дополнительные аргументы, помимо тех, что объявлены в спецификации:

- `summary` – обязательное однострочное описание, что выводится в начале текста справки по команде;
- `readme` – для более длинного текста справки, которая вызывается параметром `-help`. Аргумент `readme` – это функция, которая вызывается, только когда текст справки действительно необходим.

Далее в аргументах без меток (в неименованных аргументах) передаются спецификация и функция обратного вызова.

Функция обратного вызова выполняет все необходимые операции по завершении анализа командной строки. Она вызывается с аргументами, содержащими проанализированные значения аргументов командной строки, и превращается в основной поток выполнения приложения. Аргументы передаются функции в том же порядке, в каком они определены в спецификации (с помощью оператора `+>`).

Дополнительный аргумент `unit` функции обратного вызова

Предыдущая функция обратного вызова требует наличия дополнительного аргумента `unit`, следующего за аргументом `filename`. Это гарантирует работоспособность даже пустых спецификаций (то есть имеет значение `Command.Spec.empty`).

Каждая функция в языке OCaml требует передачи как минимум одного аргумента, поэтому заключительный аргумент – `unit`, который не будет вычисляться непосредственно в отсутствие других аргументов.

Выполнение простых команд

Чтобы выполнить простую команду после ее определения, достаточно всего лишь вызвать единственную функцию:

OCaml (part 3)

https://github.com/realworldocaml/examples/blob/v1/code/command-line-parsing/basic_md5.ml

```
let () =
  Command.run ~version:"1.0" ~build_info:"RWO" command
```

Функция `Command.run` принимает пару необязательных аргументов, которые могут пригодиться для идентификации версии выполняемого файла, переданного в промышленную эксплуатацию. Прежде чем опробовать этот пример, вам необходимо установить библиотеку `Cryptokit` командой `opam install cryptokit`. После установки выполните следующую команду, чтобы скомпилировать выполняемый файл:

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/command-line-parsing/build_basic_md5.out

```
$ corebuild -pkg cryptokit basic_md5.native
```

Теперь можно запросить информацию о версии только что скомпилированного выполняемого файла:

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/command-line-parsing/get_basic_md5_version.out

```
$ ./basic_md5.native -version
1.0
$ ./basic_md5.native -build-info
RWO
```

Информация о версии, которую можно видеть в выводе команды, определяется с помощью необязательных аргументов функции `Command.run`. Вы можете оставлять их пустыми в своих программах или получать значения для них непосредственно от системы управления версиями (например, командой `hg id`, генерирующей номер сборки в системе `Mercurial`):

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/command-line-parsing/get_basic_md5_help.out

```
$ ./basic_md5.native
Generate an MD5 hash of the input data
```

```
basic_md5.native FILENAME
```

```
More detailed information
```

```
=== flags ===
```

```
[-build-info] print info about this build and exit
[-version]   print the version of this build and exit
[-help]      print this help text and exit
              (alias: -?)
```

```
missing anonymous argument: FILENAME
```

При запуске выполняемого файла без аргументов он услужливо выводит информацию обо всех доступных параметрах командной строки наряду со стандартным сообщением об ошибке, извещающем о необходимости передать обязательный аргумент `filename`.

Если аргумент `filename` указан, тогда программа вызовет функцию `do_hash` с этим аргументом и выведет контрольную сумму MD5 на стандартный вывод:

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/command-line-parsing/run_basic_md5.out

```
$ ./basic_md5.native ./basic_md5.native
b5ee7de449a2e0c6c01d4f2d898926de
```

И это все, что потребовалось для создания простейшей утилиты вычисления контрольной суммы MD5! Ниже приводится полная версия примера, обсуждавшегося выше, более компактная за счет удаления промежуточных переменных:

OCaml

https://github.com/realworldocaml/examples/tree/v1/code/command-line-parsing/run_basic_md5.out

```
open Core.Std

let do_hash file () =
  In_channel.with_file file ~f:(fun ic ->
    let open Cryptokit in
    hash_channel (Hash.md5 ()) ic
    |> transform_string (Hexa.encode ())
    |> print_endline
  )

let command =
  Command.basic
    ~summary:"Generate an MD5 hash of the input data"
    ~readme:(fun () -> "More detailed information")
    Command.Spec.(empty +> anon ("filename" %: string))
    do_hash

let () =
  Command.run ~version:"1.0" ~build_info:"RWO" command
```

Теперь, после знакомства с основами, в оставшейся части главы мы исследуем некоторые более продвинутые возможности библиотеки `Command`.

Типы аргументов

Механизм парсинга аргументов командной строки может анализировать не только строковые значения. Модуль `Command.Spec` определяет еще несколько функций (перечислены в табл. 14.1), преобразующих входные аргументы в значения различных типов.

Таблица 14.1. Функции преобразования, определенные в модуле *Command.spec*

Тип аргумента	Тип в языке Ocaml	Пример
string	string	foo
int	int	123
float	float	123.01
bool	bool	true
date	Date.t	2013-12-25
time_span	Span.t	5s
file	string	/etc/passwd

Мы можем сжать спецификацию команды до `file`, чтобы отразить тот факт, что аргумент должен быть допустимым именем файла, а не просто произвольной строкой:

OCaml (part 1)

https://github.com/realworldocaml/examples/blob/v1/code/command-line-parsing/basic_md5_as_filename.ml

```
let command =
  Command.basic
    ~summary:"Generate an MD5 hash of the input data"
    ~readme:(fun () -> "More detailed information")
    Command.Spec.(empty +> anon ("filename" %: file))
    do_hash

let () =
  Command.run ~version:"1.0" ~build_info:"RWO" command
```

Попытка выполнить команду с именем несуществующего файла приведет к выводу сообщения об ошибке. Как одно из удобств, в интерактивной командной строке этот тип поддерживает возможность автодополнения (описывается далее в этой главе):

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/command-line-parsing/run_basic_md5_as_filename.out

```
$ ./basic_md5_as_filename.native nonexistent
Uncaught exception:
```

```
(Sys_error "nonexistent: No such file or directory")
```

```
Raised by primitive operation at file "pervasives.ml", line 292, characters 20-46
Called from file "lib/in_channel.ml", line 19, characters 46-65
Called from file "lib/exn.ml", line 87, characters 6-10
```

Определение собственных типов аргументов

Имеется также возможность определить собственные типы аргументов, если предопределенных окажется недостаточно. Например, давайте создадим тип аргумента `regular_file`, гарантирующий, что входным файлом не будет символьное

устройство или иной необычный файл UNIX, который не может быть прочитан полностью:

OCaml

https://github.com/realworldocaml/examples/blob/v1/code/command-line-parsing/basic_md5_with_custom_arg.ml

```
open Core.Std

let do_hash file () =
  In_channel.with_file file ~f:(fun ic ->
    let open Cryptokit in
    hash_channel (Hash.md5 ()) ic
    |> transform_string (Hexa.encode ())
    |> print_endline
  )

let regular_file =
  Command.Spec.Arg_type.create
  (fun filename ->
    match Sys.is_file filename with
    | `Yes -> filename
    | `No | `Unknown ->
      eprintf "'%s' is not a regular file.\n%" filename;
      exit 1
  )

let command =
  Command.basic
  ~summary:"Generate an MD5 hash of the input data"
  ~readme:(fun () -> "More detailed information")
  Command.Spec.(empty +> anon ("filename" %: regular_file))
  do_hash

let () =
  Command.run ~version:"1.0" ~build_info:"RWO" command
```

Функция `regular_file` преобразует строковый параметр `filename` в ту же самую строку, но предварительно проверяет, существует ли файл и является ли он обычным файлом. Собрав и запустив этот код, вы сможете увидеть новые сообщения об ошибках, попытавшись открыть специальное устройство, такое как `/dev/null`:

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/command-line-parsing/run_basic_md5_with_custom_arg.out

```
$ ./basic_md5_with_custom_arg.native /etc/passwd
8cfb68a5622dd12932df658a54698aad
$ ./basic_md5_with_custom_arg.native /dev/null
'/dev/null' is not a regular file.
```

Необязательные аргументы и аргументы по умолчанию

Более жизнеспособная утилита вычисления контрольной суммы MD5 могла бы также читать содержимое со стандартного ввода, если имя файла не указано:

OCaml (part 1)

https://github.com/realworldocaml/examples/blob/v1/code/command-line-parsing/basic_md5_with_optional_file_broken.ml

```
let command =
  Command.basic
    ~summary:"Generate an MD5 hash of the input data"
    ~readme:(fun () -> "More detailed information")
    Command.Spec.(empty +> anon (maybe ("filename" %: string)))
    do_hash

let () =
  Command.run ~version:"1.0" ~build_info:"RWO" command
```

Это всего лишь обертка вокруг объявления аргумента `filename`, представленная функцией `maybe`, указывающей, что аргумент является необязательным. Однако при попытке собрать этот код компилятор сообщит об ошибке:

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/command-line-parsing/build_basic_md5_with_optional_file_broken.out

```
$ corebuild -pkg cryptokit basic_md5_with_optional_file_broken.native
File "basic_md5_with_optional_file_broken.ml", line 18, characters 4-11:
Error: This expression has type string -> unit -> unit
      but an expression was expected of type string option -> unit -> unit
      Type string is not compatible with type string option
Command exited with code 2.
(Ошибка: Это выражение имеет тип string -> unit -> unit
      тогда как ожидалось выражение типа string option -> unit -> unit
      Тип string несовместим с типом string option
Команда завершилась с кодом 2.)
```

Это случилось потому, что изменение типа аргумента повлекло за собой изменение типа функции обратного вызова. Она теперь требует аргумента типа `string option` вместо `string`, потому что значение стало необязательным. Мы можем соответствующим образом изменить пример и организовать чтение со стандартного ввода, если имя файла не было указано при запуске программы:

OCaml

https://github.com/realworldocaml/examples/tree/v1/code/command-line-parsing/basic_md5_with_optional_file.ml

```
open Core.Std

let get_inchan = function
  | None | Some "-" ->
    In_channel.stdin
  | Some filename ->
    In_channel.create ~binary:true filename

let do_hash filename () =
  let open Cryptokit in
  get_inchan filename
  |> hash_channel (Hash.md5 ())
```

```

|> transform_string (Hexa.encode ())
|> print_endline

let command =
  Command.basic
    ~summary:"Generate an MD5 hash of the input data"
    ~readme:(fun () -> "More detailed information")
    Command.Spec.(empty +> anon (maybe ("filename" %: file)))
    do_hash

let () =
  Command.run ~version:"1.0" ~build_info:"RWO" command

```

Параметр `filename` функции `do_hash` теперь имеет тип `string option`. Сейчас он анализируется функцией `get_inchan`, определяющей источник данных – стандартный ввод или файл, – но в остальном команда осталась прежней.

Еще одно возможное решение – передача дефиса в качестве имени файла по умолчанию, если никакое другое имя в командной строке не было указано. Такую подстановку могла бы делать функция `maybe_with_default`. Преимуществом подобного подхода является отсутствие необходимости изменять тип параметра функции обратного вызова (которая могла бы оказаться источником проблем в более сложных приложениях).

Следующий пример действует в точности как предыдущий, но в нем вместо функции `maybe` используется функция `maybe_with_default`:

OCaml

https://github.com/realworldocaml/examples/tree/v1/code/command-line-parsing/basic_md5_with_default_file.ml

```

open Core.Std

let get_inchan = function
  | "-"      -> In_channel.stdin
  | filename -> In_channel.create ~binary:true filename

let do_hash filename () =
  let open Cryptokit in
    get_inchan filename
  |> hash_channel (Hash.md5 ())
  |> transform_string (Hexa.encode ())
  |> print_endline

let command =
  Command.basic
    ~summary:"Generate an MD5 hash of the input data"
    ~readme:(fun () -> "More detailed information")
    Command.Spec.(
      empty
      +> anon (maybe_with_default "-" ("filename" %: file))
    )
    do_hash

let () =
  Command.run ~version:"1.0" ~build_info:"RWO" command

```

Собрав оба примера и применив их к одному и тому же системному файлу, можно убедиться, что они действуют одинаково:

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/command-line-parsing/run_basic_and_default_md5.out

```
$ cat /etc/passwd | ./basic_md5_with_optional_file.native
8cfb68a5622dd12932df658a54698aad
$ cat /etc/passwd | ./basic_md5_with_default_file.native
8cfb68a5622dd12932df658a54698aad
```

Последовательности аргументов

И последнее полезное изменение – обработка целого списка анонимных аргументов. Давайте добавим в нашу утилиту вычисления контрольной суммы MD5 возможность обработки коллекций файлов:

OCaml

https://github.com/realworldocaml/examples/tree/v1/code/command-line-parsing/basic_md5_sequence.ml

```
open Core.Std

let do_hash filename ic =
  let open Cryptokit in
  hash_channel (Hash.md5 ()) ic
  |> transform_string (Hexa.encode ())
  |> fun md5 -> printf "MD5 (%s) = %s\n" filename md5

let command =
  Command.basic
    ~summary:"Generate an MD5 hash of the input data"
    ~readme:(fun () -> "More detailed information")
    Command.Spec.(empty +> anon (sequence ("filename" %: file)))
    (fun files () ->
      match files with
      | [] -> do_hash "-" In_channel.stdin
      | _ ->
        List.iter files ~f:(fun file ->
          In_channel.with_file ~f:(do_hash file) file
        )
    )

let () =
  Command.run ~version:"1.0" ~build_info:"RWO" command
```

Функция обратного вызова получилась немного сложнее, так как теперь в ней предусмотрена обработка дополнительных параметров. Аргумент `files` сейчас является списком строк, и если он пуст, происходит переключение на использование стандартного ввода в качестве источника данных, в точности как в предыдущих примерах, где использовались функции `maybe` и `maybe_with_default`. Если список файлов непуст, выполняются итерации по его элементам, и каждый файл по очереди передается функции `do_hash`.

Добавление поддержки передачи именованных флагов в командной строке

Вы не ограничены только анонимными аргументами командной строки. *Флаг* – это именованное поле, которое может сопровождаться необязательным аргументом. Такие флаги могут следовать в командной строке в любом порядке и даже несколько раз, в зависимости от того, как они объявлены в спецификации.

Давайте добавим в нашу утилиту `md5` поддержку двух аргументов, чтобы обеспечить более точную имитацию версии аналогичной утилиты из Mac OS X. Флаг `-s` определяет строку для хэширования, флаг `-t` вызывает самотестирование утилиты. Ниже приводится законченная реализация:

OCaml

https://github.com/realworldocaml/examples/blob/v1/code/command-line-parsing/basic_md5_with_flags.ml

```
open Core.Std
open Cryptokit

let checksum_from_string buf =
  hash_string (Hash.md5 ()) buf
  |> transform_string (Hexa.encode ())
  |> print_endline

let checksum_from_file filename =
  let ic = match filename with
    | "-" -> In_channel.stdin
    | _ -> In_channel.create ~binary:true filename
  in
  hash_channel (Hash.md5 ()) ic
  |> transform_string (Hexa.encode ())
  |> print_endline

let command =
  Command.basic
    ~summary:"Generate an MD5 hash of the input data"
    Command.Spec.(
      empty
      +> flag "-s" (optional string) ~doc:"string Checksum the given string"
      +> flag "-t" no_arg ~doc:"run a built-in time trial"
      +> anon (maybe_with_default "-" ("filename" %: file))
    )
  (fun use_string trial filename () ->
    match trial with
    | true -> printf "Running time trial\n"
    | false -> begin
      match use_string with
      | Some buf -> checksum_from_string buf
      | None -> checksum_from_file filename
    end
  )

let () = Command.run command
```

Для определения двух новых именованных аргументов командной строки в спецификации используется функция `flag`. Строка `doc` форматируется так, чтобы первое слово служило коротким именем в тексте с описанием порядка использования утилиты, а оставшаяся часть содержала полный текст описания. Отметим, что флаг `-t` не имеет аргументов, и потому сопровождающий его текст справки начинается с пробела. Справочный текст, что выводится предыдущим примером, имеет следующий вид:

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/command-line-parsing/run_basic_md5_flags_help.out

```
$ ./basic_md5_with_flags.native -help
Generate an MD5 hash of the input data

    basic_md5_with_flags.native [FILENAME]

=== flags ===

[-s string]  Checksum the given string
[-t]         run a built-in time trial
[-build-info] print info about this build and exit
[-version]   print the version of this build and exit
[-help]      print this help text and exit
              (alias: -?)

$ ./basic_md5_with_flags.native -s "ocaml rocks"
5a118fe92ac3b6c7854c595ecf6419cb
```

Флаг `-s` в нашей спецификации требует передачи обязательного строкового аргумента. Если указать этот флаг без аргумента, парсер `Command` выведет сообщение об ошибке, подобное тому, что выводится в отсутствие анонимных аргументов в ранних примерах. В табл. 14.2 приводится список некоторых функций, которые могут служить обертками для флагов и управлять порядком их интерпретации.

Таблица 14.2. Функции для обертывания флагов

Функция	Тип в языке Ocaml
<code>required arg</code>	<code>arg</code> и ошибка, если не указан
<code>optional arg</code>	<code>arg option</code>
<code>optional_with_default val arg</code>	<code>arg</code> со значением <code>val</code> по умолчанию
<code>listed arg</code>	<code>arg list</code> , флаг может указываться несколько раз
<code>no_arg</code>	<code>bool</code> , получает значение <code>true</code> , если присутствует

Флаги определяют тип функции обратного вызова так же, как анонимные аргументы. Это позволяет изменять спецификацию и гарантирует, что все функции обратного вызова будут обновлены соответственно, — если этого не сделать, компиляция будет завершаться ошибкой, что препятствует появлению ошибок во время выполнения.

Группировка подкоманд

С помощью флагов и анонимных аргументов можно конструировать весьма сложные интерфейсы командной строки. Однако слишком большое число возможных параметров может сделать программу слишком сложной для начинающих пользователей. Одно из решений этой проблемы – группировка часто используемых операций и придание иерархической организации интерфейсу командной строки.

Вы будете сталкиваться с интерфейсом в этом стиле при использовании инструмента управления пакетами OPAM (или, за пределами OCaml, при работе с командами систем управления версиями Git и Mercurial). OPAM экспортирует команды в следующей форме:

Terminal

<https://github.com/realworldocaml/examples/tree/v1/code/command-line-parsing/opam.out>

```
$ opam config env
$ opam remote list -k git
$ opam install --help
$ opam install cryptokit --verbose
```

Ключевые слова `config`, `remote` и `install` обеспечивают логическую группировку флагов и аргументов. Это предотвращает утечку флагов, специфичных для конкретных подкоманд, в общее конфигурационное пространство.

Обычно этот прием приобретает особенную значимость, только когда рост возможностей приложения достигает некоторого предела. К счастью, его легко реализовать с помощью `Command`: достаточно просто заменить вызов `Command.basic` вызовом функции `Command.group`, которая принимает ассоциативный список спецификации, обрабатывает подкоманды и выводит текст справки:

OCaml utop

<https://github.com/realworldocaml/examples/tree/v1/code/command-line-parsing/group.topscript>

```
# Command.basic ;;
- : summary:string ->
  ?readme:(unit -> string) ->
  ('main, unit -> unit) Command.Spec.t -> 'main -> Command.t
= <fun>
# Command.group ;;
- : summary:string ->
  ?readme:(unit -> string) -> (string * Command.t) list -> Command.t
= <fun>
```

Согласно сигнатуре, функция `group` принимает список простых значений типа `Command.t` и соответствующих им имен. При вызове она отыскивает подкоманды в списке имен и вызывает соответствующий обработчик команды.

Давайте создадим заготовку программы-календаря, выполняющую несколько операций с датами, указанными в командной строке. Для начала необходимо определить команду, которая будет добавлять дни к указанной дате и выводить получившуюся дату:

OCaml

https://github.com/realworldocaml/examples/blob/v1/code/command-line-parsing/cal_add_days.ml

```
open Core.Std

let add =
  Command.basic
    ~summary:"Add [days] to the [base] date and print day"
    Command.Spec.(
      empty
      +> anon ("base" %: date)
      +> anon ("days" %: int)
    )
    (fun base span () ->
      Date.add_days base span
      |> Date.to_string
      |> print_endline
    )

let () = Command.run add
```

Все в этом примере должно быть вам уже знакомо. Проверив и убедившись, что все работает, можно определить новую команду, вычисляющую разность двух дат. Однако вместо создания нового выполняемого файла мы сгруппируем операции в подкоманды с помощью `Command.group`:

OCaml

https://github.com/realworldocaml/examples/blob/v1/code/command-line-parsing/cal_add_sub_days.ml

```
open Core.Std

let add =
  Command.basic ~summary:"Add [days] to the [base] date"
    Command.Spec.(
      empty
      +> anon ("base" %: date)
      +> anon ("days" %: int)
    )
    (fun base span () ->
      Date.add_days base span
      |> Date.to_string
      |> print_endline
    )

let diff =
  Command.basic ~summary:"Show days between [date1] and [date2]"
    Command.Spec.(
      empty
      +> anon ("date1" %: date)
      +> anon ("date2" %: date)
    )
    (fun date1 date2 () ->
      Date.diff date1 date2
    )
```

```

    |> printf "%d days\n"
  )

let command =
  Command.group ~summary:"Manipulate dates"
  [ "add", add; "diff", diff ]
  let () = Command.run command

```

Нам понадобилось всего лишь добавить поддержку подкоманд! Давайте сначала скомпилируем пример и рассмотрим вывод справки с описанием только что добавленных подкоманд.

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/command-line-parsing/build_cal_add_sub_days.out

```

$ corebuild cal_add_sub_days.native
$ ./cal_add_sub_days.native -help
Manipulate dates

```

```
cal_add_sub_days.native SUBCOMMAND
```

```
=== subcommands ===
```

```

add      Add [days] to the [base] date
diff     Show days between [date1] and [date2]
version  print version information
help     explain a given subcommand (perhaps recursively)

```

А теперь опробуем две команды, добавленные нами, чтобы проверить их работу и посмотреть, как действует парсинг дат:

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/command-line-parsing/run_cal_add_sub_days.out

```

$ ./cal_add_sub_days.native add 2012-12-25 40
2013-02-03
$ ./cal_add_sub_days.native diff 2012-12-25 2012-11-01
54 days

```

Расширенное управление парсингом

Действие функций, генерирующих спецификации, может показаться волшебством. В частности, даже зная, как ими пользоваться, не совсем понятно, как они работают, и особенно непонятно, почему типы определяют способ их работы.

Понимание тонкостей совмещения этих спецификаций становится еще более полезным с ростом сложности интерфейсов командной строки, например когда возникает необходимость определить функциональность, единую для нескольких спецификаций, или прервать парсинг для специальной обработки, такой как интерактивный запрос пароля перед продолжением работы. А добиться такого уровня понимания поможет глубокое знание библиотеки `Command`.

В следующих разделах мы рассмотрим логику работы уже представленных комбинаторов и познакомимся с некоторыми новыми комбинаторами, повышающими эффективность использования `Command` еще больше.

Типы в основе `Command.Spec`

Надежность работы модуля `Command` обеспечивается точным соответствием результатов определения спецификаций функциям обратного вызова, которые вызываются главной программой. Чтобы предотвратить какие-либо несоответствия, модуль `Command` использует некоторые интересные особенности механизма системы типов, гарантирующие их согласованность. От вас не требуется досконально разбираться во всем, о чем рассказывается в данном разделе, чтобы пользоваться более сложными комбинаторами, но это поможет вам в отладке ошибок компиляции по мере расширения используемых возможностей модуля `Command`.

Тип `Command.Spec.t` выглядит обманчиво простым: `('a, 'b) t`. Можно подумать, что тип `('a, 'b) t` подобен типу функции `'a -> 'b`, но дополненный информацией о том:

- как выполнять парсинг командной строки;
- что делает команда и как вызывать ее;
- как выполнять автодополнение неполной командной строки.

Тип спецификации трансформирует значение `'a` в `'b`. Например, значение `Spec.t` может иметь тип `(arg1 -> ... -> argN -> 'r, 'r) Spec.t`.

Такое значение трансформирует главную функцию типа `arg1 -> ... -> argN -> 'r` путем передачи всех значений аргументов, в результате чего получается функция, возвращающая значение типа `'r`. Давайте рассмотрим несколько примеров спецификаций и их типов:

OCaml `utop`

https://github.com/realworldocaml/examples/tree/v1/code/command-line-parsing/command_types.topscript

```
# Command.Spec.empty ;;
- : ('m, 'm) Command.Spec.t = <abstr>
# Command.Spec.(empty +> anon ("foo" %: int)) ;;
- : (int -> '_a, '_a) Command.Spec.t = <abstr>
```

Пустая спецификация проста, так как она не добавляет никаких параметров в тип функции обратного вызова. Второй пример добавляет целочисленный анонимный параметр, что сразу отражается на выведенном типе. Он образует команду путем объединения спецификации типа `('main, unit) Spec.t` с функцией типа `'main`. Комбинаторы, что мы видели до сих пор, пошагово наращивают тип `'main` в соответствии с ожидаемыми параметрами командной строки, поэтому окончательный тип `'main` выглядит примерно так: `arg1 -> ... -> argN -> unit`.

Теперь тип функции `Command.basic` должен выглядеть для вас более понятным:

OCaml utop

<https://github.com/realworldocaml/examples/tree/v1/code/command-line-parsing/basic.topscript>

```
# Command.basic ;;
- : summary:string ->
  ?readme:(unit -> string) ->
    ('main, unit -> unit) Command.Spec.t -> 'main -> Command.t
= <fun>
```

Параметры в `Spec.t` здесь играют важную роль. Они показывают, что функция обратного вызова для спецификации должна принимать идентичные аргументы, передаваемые главной функции, за исключением дополнительного аргумента `unit`. Заключительный аргумент `unit` здесь гарантирует, что функция обратного вызова будет интерпретироваться как функция, потому что в противном случае при нулевом количестве аргументов командной строки (то есть `Spec.empty`) функция обратного вызова не имела бы никаких аргументов и была бы вызвана немедленно. Именно поэтому потребовалось добавлять дополнительные скобки `()` в функцию обратного вызова во всех предыдущих примерах.

Объединение фрагментов спецификаций

Если вы хотите выделить общие операции командной строки, вам на помощь придет оператор `++`, который объединяет вместе две спецификации. Чтобы продемонстрировать применение этого оператора, давайте добавим в наше приложение календаря флаги отладки и вывода более подробного отчета о работе.

OCaml

https://github.com/realworldocaml/examples/blob/v1/code/command-line-parsing/cal_append.ml

```
open Core.Std
```

```
let add ~common =
  Command.basic ~summary:"Add [days] to the [base] date"
    Command.Spec.(
      empty
      +> anon ("base" %: date)
      +> anon ("days" %: int)
      ++ common
    )
  (fun base span debug verbose () ->
    Date.add_days base span
    |> Date.to_string
    |> print_endline
  )

let diff ~common =
  Command.basic ~summary:"Show days between [date2] and [date1]"
    Command.Spec.(
      empty
      +> anon ("date1" %: date)
      +> anon ("date2" %: date)
```

```

    ++ common
  )
  (fun date1 date2 debug verbose () ->
    Date.diff date1 date2
    |> printf "%d days\n"
  )

```

Определения спецификаций близко напоминают предыдущие примеры, за исключением дополнительного параметра `common` в конце каждой спецификации. Мы можем передать эти флаги при определении групп.

OCaml (part 1)

https://github.com/realworldocaml/examples/blob/v1/code/command-line-parsing/cal_append.ml

```

let () =
  let common =
    Command.Spec.(
      empty
      +> flag "-d" (optional_with_default false bool) ~doc:" Debug mode"
      +> flag "-v" (optional_with_default false bool) ~doc:" Verbose output"
    )
  in
  List.map ~f:(fun (name, cmd) -> (name, cmd ~common))
    [ "add", add; "diff", diff ]
  |> Command.group ~summary:"Manipulate dates"
  |> Command.run

```

Оба флага теперь передаются всем функциям обратного вызова. Это упрощает рефакторинг кода при использовании компилятора, который автоматически обнаруживает все места, где используются команды. Достаточно добавить параметр в общее определение, запустить компилятор и исправить обнаруженные им ошибки.

Например, если удалить флаг `verbose` и выполнить компиляцию, мы получим подробное и выразительное сообщение об ошибке:

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/command-line-parsing/build_cal_append_broken.out

```
$ corebuild cal_append_broken.native
```

```
File "cal_append_broken.ml", line 38, characters 45-52:
```

```
Error: This expression has type
```

```
  (bool -> unit -> unit -> unit, unit -> unit -> unit) Command.Spec.t
```

```
but an expression was expected of type
```

```
  (bool -> unit -> unit -> unit, unit -> unit) Command.Spec.t
```

```
Type unit -> unit is not compatible with type unit
```

```
Command exited with code 2.
```

```
(Ошибка: Это выражение имеет тип
```

```
  (bool -> unit -> unit -> unit, unit -> unit -> unit) Command.Spec.t
```

```
тогда как ожидалось выражение типа
```

```
  (bool -> unit -> unit -> unit, unit -> unit) Command.Spec.t
```

```
Тип unit -> unit несовместим с типом unit
```

```
Команда завершилась с кодом 2.)
```

Несмотря на устрашающий текст, вся проблема кроется в последней его строке, где сообщается, что вы передали функции обратного вызова слишком много аргументов (`unit -> unit` вместо `unit`). Если вы сделали это единственное изменение в отлаженной программе, то обычно даже не требуется ломать голову над текстом ошибки — достаточно имени файла и номера строки, чтобы понять, в чем проблема, и исправить ее.

Интерактивный запрос ввода

Комбинатор `step` позволяет управлять нормальным ходом парсинга и указывать функцию, отображающую аргументы функции обратного вызова в новое множество значений. Например, давайте вернемся к первому варианту программы-календаря, которая добавляет указанное число дней к заданной дате:

OCaml

https://github.com/realworldocaml/examples/blob/v1/code/command-line-parsing/cal_add_days.ml

```
open Core.Std
```

```
let add =
  Command.basic
    ~summary:"Add [days] to the [base] date and print day"
    Command.Spec.(
      empty
      +> anon ("base" %: date)
      +> anon ("days" %: int)
    )
    (fun base span () ->
      Date.add_days base span
      |> Date.to_string
      |> print_endline
    )
```

```
let () = Command.run add
```

Эта версия программы требует, чтобы вы указали начальную дату и число дней, которое требуется добавить к ней. Если число дней не указано в командной строке, выводится сообщение об ошибке. Теперь давайте изменим программу так, чтобы она выводила интерактивный запрос на ввод числа дней, если при вызове программы была указана только начальная дата:

OCaml

https://github.com/realworldocaml/examples/blob/v1/code/command-line-parsing/cal_add_interactive.ml

```
open Core.Std
```

```
let add_days base span () =
  Date.add_days base span
  |> Date.to_string
  |> print_endline
```

```

let add =
  Command.basic
    ~summary:"Add [days] to the [base] date and print day"
    Command.Spec.(
      step
        (fun m base days ->
          match days with
          | Some days ->
            m base days
          | None ->
            print_endline "enter days: ";
            read_int ()
            |> m base
        )
      +> anon ("base" %: date)
      +> anon (maybe ("days" %: int))
    )
  add_days

let () = Command.run add

```

Теперь целочисленный анонимный аргумент `days` объявлен необязательным, и нам требуется преобразовать его в обязательное значение перед вызовом функции `add_days`. Комбинатор `step` дает возможность выполнить это преобразование применением указанной функции обратного вызова. В данном примере функция обратного вызова сначала проверяет, определен ли аргумент `days`. И если не определен, интерактивно читает целое число со стандартного ввода.

Первый аргумент `m` в функции обратного вызова, что передается комбинатору `step`, — это следующая функция обратного вызова в цепочке. Преобразование завершается вызовом `m base days` с новыми значениями. Значение `days`, которое передается следующей функции обратного вызова, теперь является обязательным значением типа `int`:

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/command-line-parsing/build_and_run_cal_add_interactive.out

```

$ ocamlbuild -use-ocamlfind -tag thread -pkg core cal_add_interactive.native
$ ./cal_add_interactive.native 2013-12-01
enter days:
35
2014-01-05

```

Такое преобразование позволяет сохранить прежнее определение функции `add days: Date.t -> int -> unit`. Функция `step` преобразует аргумент типа `int option` в обычный аргумент типа `int`, пригодный для передачи функции `add_days`. Данное преобразование явно отражается в типе возвращаемого значения функции `step`:

OCaml utop

<https://github.com/realworldocaml/examples/blob/v1/code/command-line-parsing/step.topscrip>

```

# open Command.Spec ;;
# step (fun m (base:Date.t) days ->

```

```

match days with
| Some days -> m base days
| None ->
    print_endline "enter days: ";
    m base (read_int ()) ;;
- : (Date.t -> int -> '_a, Date.t -> int option -> '_a) t = <abstr>

```

Первая половина `Spec.t` показывает, что функция обратного вызова имеет тип `Date.t -> int`, а возвращаемое значение, ожидаемое от спецификации, следующей далее в цепочке, имеет тип `Date.t -> int option`.

Добавление аргументов с метками в функции обратного вызова

Комбинатор `step` позволяет легко управлять типами функций обратного вызова. Это может пригодиться для согласования с имеющимися интерфейсами или сделать код более явным за счет добавления аргументов с метками (именованных аргументов):

OCaml

https://github.com/realworldocaml/examples/blob/v1/code/command-line-parsing/cal_add_labels.ml

```
open Core.Std
```

```

let add_days ~base_date ~num_days () =
  Date.add_days base_date num_days
  |> Date.to_string
  |> print_endline

let add =
  Command.basic
    ~summary:"Add [days] to the [base] date and print day"
    Command.Spec.(
      step (fun m base days -> m ~base_date:base ~num_days:days)
      +> anon ("base" %: date)
      +> anon ("days" %: int)
    )
    add_days

let () = Command.run add

```

Этот пример `cal_add_labels` возвращает нас назад к неинтерактивной версии программы-календаря, но теперь главная функция `add_days` принимает аргументы с метками. Функция `step` в спецификации просто преобразует аргументы по умолчанию `base` и `days` в аргументы с метками.

Аргументы с метками требуют больше ввода, зато они способны предотвратить ошибки, когда имеется несколько аргументов командной строки похожих типов, но с разными именами и предназначениями. Метки хорошо использовать, когда в противном случае пришлось бы обрабатывать множество анонимных аргументов с типами `int` и `string`.

Автодополнение командной строки средствами Bash

Современные командные оболочки UNIX обычно поддерживают возможность автодополнения при нажатии на клавишу табуляции, помогающую в интерактивном режиме сконструировать командную строку. Она активируется нажатием клавиши **Tab** в середине команды и предлагает на выбор возможные варианты продолжения. Чаще всего данная возможность используется для поиска файлов в текущем каталоге, но на самом деле ее действие можно распространить и на другие части команды.

Точная механика работы функции автодополнения отличается в разных командных оболочках, но далее мы будем предполагать, что используется наиболее популярная: *bash*. Она по умолчанию применяется в большинстве дистрибутивов Linux и Mac OS X, но в других операционных системах, таких как *BSD или Windows (при использовании Cygwin), вам может потребоваться явно переключиться на нее. Одним словом, в оставшейся части этого раздела будет предполагаться, что вы используете *bash*.

Функция автодополнения в Bash не всегда устанавливается по умолчанию, поэтому проверьте с помощью системного диспетчера пакетов ее присутствие в системе.

Таблица 14.3. Диспетчеры пакетов и пакеты с функцией автодополнения

Операционная система	Диспетчер пакетов	Пакет
Debian Linux	apt	bash-completion
Mac OS X	Homebrew	bash-completion
FreeBSD	Система портов	<i>/usr/ports/shells/bash-completion</i>

После установки и настройки пакета с реализацией функции автодополнения для *bash* проверьте ее работоспособность, введя команду *ssh* и нажав клавишу **Tab**. В результате должен появиться список хостов, хранящийся в вашем файле *~/.ssh/known_hosts*. Если вы увидите список хостов, к которым подключались недавно, можно продолжать дальше. Если это будет список файлов в текущем каталоге, обратитесь к документации для вашей операционной системы и выясните, как правильно настроить функцию автодополнения.

И еще, вам потребуется выяснить, где находится каталог *bash_completion.d*. Там хранятся все фрагменты, содержащие логику автодополнения. В Linux этот каталог часто хранится в пути */etc/bash_completion.d*, а в Mac OS X – в пути */usr/local/etc/bash_completion.d*.

Создание фрагментов автодополнения

Библиотека Command имеет декларативное описание всех допустимых параметров, используя которое, можно сгенерировать сценарий командной оболочки для поддержки автодополнения нашей команды. Чтобы создать фрагмент, просто за-

пустите команду с переменной окружения `COMMAND_OUTPUT_INSTALLATION_BASH`, установленной в любое произвольное значение.

Например, давайте попробуем сгенерировать фрагмент для примера вычисления контрольной суммы MD5, обсуждавшегося выше, при этом будем предполагать, что выполняемый файл имеет имя *basic_md5_with_flags* и находится в текущем каталоге:

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/command-line-parsing/md5_completion.out

```
$ env COMMAND_OUTPUT_INSTALLATION_BASH=1 ./basic_md5_with_flags.native
function _jsautocom_23343 {
  export COMP_CWORD
  COMP_WORDS[0]=./basic_md5_with_flags.native
  COMPREPLY=( $("${COMP_WORDS[@]}") )
}
complete -F _jsautocom_23343 ./basic_md5_with_flags.native
```

Напомню, что тип аргумента мы определили как `Arg_type.file`. В результате мы получили логику автодополнения и теперь можем просто нажимать клавишу **Tab**, чтобы обеспечить подстановку имен файлов из текущего каталога.

Установка фрагмента автодополнения

Не следует стараться вникать в суть фрагмента, полученного выше (если только вы не испытываете очарования внутренней красотой сценариев на языке командной оболочки). Вместо этого просто перенаправьте вывод в файл в вашем текущем каталоге и импортируйте его в текущей командной оболочке:

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/command-line-parsing/cal_completion.out

```
$ env COMMAND_OUTPUT_INSTALLATION_BASH=1 ./cal_add_sub_days.native > cal.cmd
$ . cal.cmd
$ ./cal_add_sub_days.native <tab>
add      diff      help      version
```

Поддержка автодополнения команд действует и для флагов, и для сгруппированных команд, что очень удобно для построения сложных интерфейсов командной строки. Не забудьте установить фрагмент в общесистемный каталог *bash_completion.d*, если хотите, чтобы он автоматически загружался при каждом входе в систему.

Установка универсального обработчика автодополнения

К большому сожалению, командная оболочка `bash` не поддерживает установку универсального обработчика для приложений, основанных на библиотеке `Command`. Это означает, что вам придется устанавливать сценарий с логикой автодополнения для каждого приложения в отдельности. Впрочем, эту процедуру можно автоматизировать, перепоручив ее системе сборки.

С ее помощью можно проверить, как другие приложения устанавливают свои фрагменты для функции автодополнения, и следовать тем же путем, потому что в разных системах порядок установки может отличаться.

Альтернативные парсеры командной строки

Этим разделом завершается наш тур по библиотеке `Command`. Вы должны знать, что использование данной библиотеки – не единственный способ организовать анализ аргументов командной строки; существует еще несколько альтернатив, доступных в ОРАМ. Три самые известные из них перечислены ниже:

- модуль `Arg` – входит в состав стандартной библиотеки `OCaml` и используется самим компилятором для обработки аргументов командной строки. Вообще говоря, библиотека `Command` более функциональна, чем модуль `Arg` (основное ее преимущество – поддержка подкоманд, комбинатора `step` и возможность определения текста справки), но нет ничего плохого, если вы решите использовать `Arg`.
Вы можете использовать функцию `Command.Spec.flags_of_args_exn` для преобразования спецификаций `Arg` в спецификации, совместимые с `Command`. Такая возможность довольно часто используется на практике, когда требуется перенести программный код на платформу стандартной библиотеки `Core`;
- `ocaml-getopt` – библиотека `ocaml-getopt` обеспечивает поддержку универсального синтаксиса командной строки, реализованного в утилитах GNU `getopt` и `getopt_long`. Эти утилиты широко используются в мире открытого программного обеспечения, и эта библиотека позволит вашим программам на `OCaml` следовать тем же правилам;
- `Cmdliner` – это смесь библиотек `Command` и `Getopt`. Она дает возможность декларативного определения интерфейсов командной строки, но экспортирует более `getopt`-подобный интерфейс. Она также помогает автоматизировать создание страниц справочного руководства для UNIX (man-страниц). Парсер `Cmdliner` используется инструментом ОРАМ для анализа аргументов командной строки.

Обработка данных JSON

Сериализация данных, то есть преобразование их в последовательность байтов и обратно, которую можно было бы записать на диск или передать по сети, относится к важнейшим и часто встречающимся задачам программирования. Вам нередко придется обеспечивать поддержку сторонних форматов данных (таких как XML). Иногда вам будет необходим формат, обеспечивающий высокую эффективность, иногда – удобочитаемый, который допускал бы простую возможность редактирования человеком. Библиотеки OCaml предоставляют несколько решений для сериализации данных, подходящих для разных ситуаций, в зависимости от решаемых задач.

В этой главе мы познакомимся с поддержкой простого и популярного формата JSON представления данных, а в последующих – с другими форматами сериализации. Здесь будет представлена пара новых приемов, объединяющих основные идеи из первой части книги, с использованием:

- *полиморфных вариантов* для создания легко расширяемых библиотек и протоколов;
- *функциональных комбинаторов* для конструирования общих, типизированных операций со структурами данных;
- внешних инструментов для создания заготовок модулей на языке OCaml и сигнатур на основе внешних файлов спецификаций.

Основы JSON

JSON – это легковесный формат обмена данными, часто используемый веб-службами и браузерами. Он описан в RFC 4627 и намного проще в парсинге и создании, чем альтернативы, такие как XML. Вы часто будете сталкиваться с форматом JSON при работе с современными веб-API, поэтому в данной главе мы рассмотрим несколько разных способов его использования.

Данные в формате JSON представлены всего двумя простыми структурами: неупорядоченными коллекциями пар ключ/значение и упорядоченными списками значений. Значения могут быть строками, логическими величинами, вещественными и целыми числами или null. Давайте для примера посмотрим, как может выглядеть в формате JSON информация об этой книге:

```
{
  "title": "Real World OCaml",
  "tags" : [ "functional programming", "ocaml", "algorithms" ],
  "pages": 450,
  "authors": [
    { "name": "Jason Hickey", "affiliation": "Google" },
```

```
{ "name": "Anil Madhavapeddy", "affiliation": "Cambridge"},
{ "name": "Yaron Minsky", "affiliation": "Jane Street"}
],
"is_online": true
}
```

Самым внешним значением в формате JSON обычно является запись (заключается в фигурные скобки), содержащая неупорядоченное множество пар ключ/значение. Ключи могут быть только строками, а значения могут быть любого типа, поддерживаемого форматом JSON. В предыдущем примере поле `tags` – это список строк, тогда как поле `authors` – список записей. В отличие от списков в языке OCaml, списки в формате JSON могут содержать значения разных типов.

Такая свободная форма представления типов JSON является и благословением, и проклятием. Преобразование данных в формат JSON выполняется очень легко, но код, выполняющий парсинг, должен учитывать разные возможные вариации представления значений. Например, как быть, если поле `pages` в примере выше в действительности имеет строковое значение "450", а не целочисленное?

Наша первая задача – преобразовать данные в формате JSON в более структурированное представление на языке OCaml, чтобы можно было использовать преимущества статической типизации. При анализе данных JSON в Python или Ruby можно написать модульные тесты, проверяющие корректность обработки необычных исходных данных. В языке OCaml предпочтение отдается статической проверке во время компиляции. Например, операция сопоставления с образцом предупредит вас, если вы не учли, что значение может хранить не только фактическую величину, но и `Null`.

Установка библиотеки Yojson

Для языка OCaml имеется несколько библиотек обработки формата JSON. Для нужд обсуждения в этой главе мы выбрали библиотеку `Yojson1`, созданную Мартином Джамбоном (Martin Jambon). Ее можно установить с помощью инструмента управления пакетами OPAM, выполнив команду `opam install yojson`. Если диспетчер пакетов OPAM у вас еще не установлен, обращайтесь за инструкциями по адресу: <http://realworldocaml.org/install>. После установки библиотеку можно открыть в интерактивной оболочке:

OCaml utopt

<https://github.com/realworldocaml/examples/blob/v1/code/json/install.topscript>

```
# #require "yojson" ;;
# open Yojson ;;
```

Парсинг данных в формате JSON с помощью Yojson

Спецификация формата JSON определяет всего несколько типов данных, поэтому типа `Yojson.Basic.json`, что приводится ниже, вполне достаточно для представления JSON с любой допустимой структурой:

¹ <http://mjambon.com/yojson.html>.

OCaml

https://github.com/realworldocaml/examples/blob/v1/code/json/yojson_basic.mli

```
type json = [
  | `Assoc of (string * json) list
  | `Bool of bool
  | `Float of float
  | `Int of int
  | `List of json list
  | `Null
  | `String of string
]
```

Обратите внимание на следующие интересные особенности, которые можно наблюдать в этом определении.

- Тип `json` является *рекурсивным*, а это означает, что некоторые теги ссылаются обратно на полный тип `json`. Например, типы `Assoc` и `List` могут содержать ссылки на значения JSON разных типов. Этим они отличаются от списков OCaml, элементы в которых могут быть только одного типа.
- Определение явно включает вариант `Null` для пустых полей. По умолчанию OCaml не допускает пустых значений, поэтому данную ситуацию приходится определять явно.
- В определении типа используются не обычные, а полиморфные варианты. Важность этого станет понятна позднее, когда мы приступим к добавлению собственных расширений в формат JSON.

Давайте попробуем преобразовать пример фрагмента в формате JSON, представленный выше, в значение этого типа. Первую остановку мы совершим в документации к `Yojson.Basic`, где описываются следующие функции:

OCaml (part 1)

https://github.com/realworldocaml/examples/blob/v1/code/json/yojson_basic.mli

```
val from_string : ?buf:Bi_outbuf.t -> ?fname:string -> ?lnum:int ->
  string -> json
(* Читает значение JSON из строки.
   [buf]      : использует этот буфер в процессе анализа вместо
                создания нового.
   [fname]    : имя файла с данными, которое будет указываться в сообщениях
                об ошибках. Не обязательно должно быть именем существующего
                файла.
   [lnum]     : номер первой строки в исходных данных. По умолчанию 1. *)

val from_file : ?buf:Bi_outbuf.t -> ?fname:string -> ?lnum:int ->
  string -> json
(* Читает значение JSON из файла. Назначение необязательных аргументов
   см. в описании [from_string]. *)

val from_channel : ?buf:Bi_outbuf.t -> ?fname:string -> ?lnum:int ->
  in_channel -> json
(** Читает значение JSON из канала.
    Назначение необязательных аргументов см. в описании [from_string]. *)
```

При первом знакомстве с этими интерфейсами можно игнорировать необязательные аргументы (которые отмечены знаками вопроса в сигнатуре), поскольку они имеют вполне разумные значения по умолчанию. В сигнатурах выше необязательные аргументы обеспечивают возможность более точного управления выделением буфера в памяти и сообщениями об ошибках, возникающих при попытке парсинга некорректного формата JSON.

Если из сигнатур этих функций убрать необязательные элементы, их назначение станет более очевидным. Они реализуют три способа парсинга данных в формате JSON непосредственно из строки, из файла или из буферизованного канала ввода:

OCaml

https://github.com/realworldocaml/examples/blob/v1/code/json/yojson_basic_simple.mli

```
val from_string : string -> json
val from_file   : string -> json
val from_channel : in_channel -> json
```

Следующий пример демонстрирует применение функций парсинга содержимого строки и файла. В последнем случае предполагается, что данные хранятся в файле с именем *book.json*:

OCaml

https://github.com/realworldocaml/examples/blob/v1/code/json/read_json.ml

```
open Core.Std
let () =
  (* Прочитать файл JSON в строку OCaml *)
  let buf = In_channel.read_all "book.json" in
  (* Использовать конструктор строки JSON *)
  let json1 = Yojson.Basic.from_string buf in
  (* Использовать конструктор файла JSON *)
  let json2 = Yojson.Basic.from_file "book.json" in
  (* Сравнить два значения *)
  print_endline (if json1 = json2 then "OK" else "FAIL")
```

Соберем эту программу вызовом *corebuild*:

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/json/run_read_json.out

```
$ corebuild -pkg yojson read_json.native
$ ./read_json.native
OK
```

Функция `from_file` принимает имя входного файла и сама открывает и закрывает его. Однако для конструирования значений JSON чаще используется функция `from_string`, потому что исходные данные обычно поступают в программу в виде строк через сетевые соединения (подробнее об этом рассказывается в главе 18) или из баз данных. В конце пример сравнивает результаты, полученные разными механизмами.

Выборка значений из структур JSON

Теперь, когда мы увидели, как выполнить парсинг примера данных в формате JSON в значение OCaml, давайте попробуем поэкспериментировать с этим значением и извлечем из него значения отдельных полей:

OCaml

https://github.com/realworldocaml/examples/blob/v1/code/json/parse_book.ml

open Core.Std

```
let () =
  (* Прочитать файл JSON *)
  let json = Yojson.Basic.from_file "book.json" in

  (* Открыть локально функции для работы со значениями JSON *)
  let open Yojson.Basic.Util in
  let title = json |> member "title" |> to_string in
  let tags = json |> member "tags" |> to_list |> filter_string in
  let pages = json |> member "pages" |> to_int in
  let is_online = json |> member "is_online" |> to_bool_option in
  let is_translated = json |> member "is_translated" |> to_bool_option in
  let authors = json |> member "authors" |> to_list in
  let names = List.map authors ~f:(fun json -> member "name" json |> to_string) in

  (* Вывести результаты парсинга *)
  printf "Title: %s (%d)\n" title pages;
  printf "Authors: %s\n" (String.concat ~sep:", " names);
  printf "Tags: %s\n" (String.concat ~sep:", " tags);
  let string_of_bool_option =
    function
    | None -> "<unknown>"
    | Some true -> "yes"
    | Some false -> "no" in
  printf "Online: %s\n" (string_of_bool_option is_online);
  printf "Translated: %s\n" (string_of_bool_option is_translated)
```

Теперь скомпилируем пример так же, как было показано выше:

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/json/run_parse_book.out

```
$ corebuild -pkg yojson parse_book.native
$ ./parse_book.native
Title: Real World OCaml (450)
Authors: Jason Hickey, Anil Madhavapeddy, Yaron Minsky
Tags: functional programming, ocaml, algorithms
Online: yes
Translated: <unknown>
```

В этом примере в дело вводится модуль `Yojson.Basic.Util`, который содержит *функции-комбинаторы*, помогающие отображать объекты JSON в более строго типизированные значения OCaml.

Функциональные комбинаторы

Комбинаторы – это шаблон проектирования, который часто применяется в функциональном программировании. Джон Хьюз (John Hughes) определяет их как «функции, конструирующие фрагменты программ из фрагментов программ». В функциональном языке под комбинаторами обычно подразумеваются функции высшего порядка, комбинирующие другие функции для применения преобразований к значениям.

Вы уже встречались с несколькими подобными функциями из модуля `List`:

OCaml

https://github.com/realworldocaml/examples/blob/v1/code/json/list_excerpt.mli

```
val map : 'a list -> f:('a -> 'b) -> 'b list
val fold : 'a list -> init:'accum -> f:('accum -> 'a -> 'accum) -> 'accum
```

Функции `map` и `fold` являются самыми, пожалуй, распространенными комбинаторами, преобразующими исходный список за счет применения указанной функции к каждому значению в списке. Функция `map` является простейшим комбинатором, возвращающим получившийся список непосредственно. Функция `fold` применяет к каждому значению в списке функцию, накапливающую результат в виде единственного значения:

OCaml (part 1)

https://github.com/realworldocaml/examples/blob/v1/code/json/list_excerpt.mli

```
val iter : 'a list -> f:('a -> unit) -> unit
```

Функция `iter` – более специализированный комбинатор, который может пригодиться, только в императивном коде. Входная функция применяется к каждому значению, но не возвращает никакого результата. Вместо этого функция должна производить некоторый побочный эффект, такой как изменение поля записи или вывод информации на экран.

Библиотека `Yojson` предоставляет в модуле `Yojson.Basic.Util` несколько комбинаторов, часть которых перечислена в табл. 15.1.

Таблица 15.1. Комбинаторы из библиотеки `Yojson`

Функция	Тип	Назначение
<code>member</code>	<code>string -> json -> json</code>	Выбирает указанное поле в записи JSON
<code>to_string</code>	<code>json -> string</code>	Преобразует значение JSON в значение OCaml типа <code>string</code> . Возбуждает исключение, если это невозможно
<code>to_int</code>	<code>json -> int</code>	Преобразует значение JSON в значение OCaml типа <code>int</code> . Возбуждает исключение, если это невозможно
<code>filter_string</code>	<code>json list -> string list</code>	Извлекает допустимые строки из списка полей JSON и возвращает их в виде списка строк

Мы рассмотрим каждый из них по очереди. В следующих примерах также используется прямой конвейерный оператор (pipe-forward operator) `|>`, описывавшийся в главе 2. Он позволяет составлять цепочки из функций обработки значений JSON и передавать результаты от одной к другой, без необходимости создавать промежуточные `let`-привязки.

Рассмотрим для начала выборку единственного поля `title`:

OCaml utop (part 1)

https://github.com/realworldocaml/examples/blob/v1/code/json/parse_book.topscript

```
# open Yojson.Basic.Util ;;
# let title = json |> member "title" |> to_string ;;
val title : string = "Real World OCaml"
```

Функция-член принимает объект JSON и ключ, и возвращает поле JSON, связанное с этим ключом, или Null. Так как известно, что значение поля `title` всегда имеет тип `string` в нашем примере, мы можем смело преобразовывать это значение в строку OCaml. Функция `to_string` выполняет это преобразование и возбуждает исключение, если значение имеет какой-то другой тип JSON. Оператор `|>` обеспечивает удобный способ объединения операций:

OCaml utop (part 2)

https://github.com/realworldocaml/examples/blob/v1/code/json/parse_book.topscript

```
# let tags = json |> member "tags" |> to_list |> filter_string ;;
val tags : string list = ["functional programming"; "ocaml"; "algorithms"]
# let pages = json |> member "pages" |> to_int ;;
val pages : int = 450
```

Поле `tags` подобно полю `title`, но в нем хранится список строк, а не одна строка. Преобразование этого поля в список строк OCaml (`string list`) выполняется в два этапа. Сначала тип `List JSON` преобразуется в список (`list`) OCaml значений JSON, а затем отфильтровываются значения `String`, в результате чего получается список строк (`string list`) OCaml. Не забывайте, что списки в языке OCaml могут содержать только значения одного типа, поэтому любые значения JSON, которые не могут быть преобразованы в значения типа `string`, будут пропущены функцией `filter_string`:

OCaml utop (part 3)

https://github.com/realworldocaml/examples/blob/v1/code/json/parse_book.topscript

```
# let is_online = json |> member "is_online" |> to_bool_option ;;
val is_online : bool option = Some true
# let is_translated = json |> member "is_translated" |> to_bool_option ;;
val is_translated : bool option = None
```

Поля `is_online` и `is_translated` являются необязательными, поэтому их отсутствие не должно приводить к ошибке. Это обстоятельство можно выразить с помощью типа `bool option` и извлечь его с помощью `to_bool_option`. В нашем примере данных в формате JSON присутствует только поле `is_online`, а поле `is_translated` отсутствует, поэтому для последнего возвращается `None`:

OCaml utop (part 4)

https://github.com/realworldocaml/examples/blob/v1/code/json/parse_book.topscript

```
# let authors = json |> member "authors" |> to_list ;;
val authors : Yojson.Basic.json list =
  [ `Assoc
```

```
[("name", `String "Jason Hickey"); ("affiliation", `String "Google")];
`Assoc
[("name", `String "Anil Madhavapeddy");
 ("affiliation", `String "Cambridge")];
`Assoc
[("name", `String "Yaron Minsky");
 ("affiliation", `String "Jane Street")]]
```

Последний пример использования комбинаторов JSON – извлечения всех полей `name` с именами из списка авторов `authors`. Сначала мы сконструировали список авторов `author list`, а затем преобразовали его в список строк `string list`. Обратите внимание, что здесь список явно связывается с переменной `authors`. То же самое можно было бы записать более кратко, применив прямой конвейерный оператор:

OCaml utop (part 5)

https://github.com/realworldocaml/examples/blob/v1/code/json/parse_book.topscript

```
# let names =
  json |> member "authors" |> to_list
  |> List.map ~f:(fun json -> member "name" json |> to_string) ;;
val names : string list =
  ["Jason Hickey"; "Anil Madhavapeddy"; "Yaron Minsky"]
```

Такой стиль программирования, без использования промежуточных переменных и с составлением цепочек из функций, называется *комбинаторным программированием (point-free programming)*¹. Это краткий стиль, но им не следует злоупотреблять из-за увеличенной сложности отладки промежуточных значений. Если на каждом этапе преобразования промежуточному значению будет присваиваться явное имя, отладчику будет проще представить поток выполнения программисту.

Такой прием использования статически типизированных функций в сочетании с системой типов OCaml дает очень мощную комбинацию. Многие ошибки, бессмысленные во время выполнения (например, смешивание списков и объектов), будут выловлены компилятором.

Конструирование значений JSON

Создание и вывод значений JSON с помощью типа `Yojson.Basic.json` являются достаточно простой задачей. Вы можете просто конструировать значения типа `json` и применять к ним функцию `to_string`. Давайте вспомним, как выглядит тип `Yojson.Basic.json`:

OCaml

https://github.com/realworldocaml/examples/blob/v1/code/json/yojson_basic.mli

```
type json = [
  | `Assoc of (string * json) list
  | `Bool of bool
```

¹ Также часто можно встретить названия «бесточечное программирование» и «программирование, свободное от указателей». — *Прим. перев.*

```

| `Float of float
| `Int   of int
| `List  of json list
| `Null
| `String of string
]

```

Вы можете использовать этот тип непосредственно для создания значения и затем вызвать функцию форматированного вывода из модуля `Yojson.Basic`:

OCaml utop (part 1)

https://github.com/realworldocaml/examples/blob/v1/code/json/build_json.topscript

```

# let person = `Assoc [ ("name", `String "Anil") ] ;;
val person : [> `Assoc of (string * [> `String of string ]) list ] =
  `Assoc [("name", `String "Anil")]

```

В этом примере конструируется простой объект JSON, представляющий информацию о единственном человеке. Нам не пришлось определять для этой цели явный тип, поскольку мы положились на волшебство полиморфных вариантов.

Система типов языка OCaml вывела тип `person`, опираясь на порядок конструирования этого значения. В данном случае для определения записи использовались только варианты `Assoc` и `String`, а так как здесь нет никакой информации о других возможных вариантах (таких как `Int` или `Null`), допустимых в записях JSON, выведенный тип содержит только эти поля:

OCaml utop (part 2)

https://github.com/realworldocaml/examples/blob/v1/code/json/build_json.topscript

```

# Yojson.Basic.pretty_to_string ;;
- : ?std:bool -> Yojson.Basic.json -> string = <fun>

```

Функция `pretty_to_string` имеет более явную сигнатуру и требует аргумента типа `Yojson.Basic.json`. При применении `pretty_to_string` к значению `person` тип `person` статически сопоставляется со структурой типа `json`, что гарантирует их совместимость:

OCaml utop (part 3)

https://github.com/realworldocaml/examples/blob/v1/code/json/build_json.topscript

```

# Yojson.Basic.pretty_to_string person ;;
- : string = "{ \"name\": \"Anil\" }"
# Yojson.Basic.pretty_to_channel stdout person ;;
{ "name": "Anil" }
- : unit = ()

```

В данном случае никаких проблем не возникает. Наше значение `person` имеет выведенный тип, который интерпретируется как допустимый подтип типа `json`, и поэтому преобразование в строку проходит без ошибок и без необходимости явно указывать тип для `person`. Автоматическое определение типов дает возможность писать более краткий код, не жертвуя надежностью кода во время выполнения,

так как все случаи использования полиморфных вариантов все еще проверяются на этапе компиляции.

Полиморфные варианты и упрощение проверки типов

Одна из сложностей, часто возникающих на практике, – излишняя подробность сообщений об ошибках при использовании полиморфных вариантов. Например, представьте, что вы конструируете значение типа `Assoc` и по ошибке указали единственное фактическое значение вместо списка ключей:

OCaml utop (part 4)

https://github.com/realworldocaml/examples/blob/v1/code/json/build_json.topscript

```
# let person = `Assoc ("name", `String "Anil");;
val person : [> `Assoc of string * [> `String of string ] ] =
  `Assoc ("name", `String "Anil")
# Yojson.Basic.pretty_to_string person ;;
Characters 30-36:
Error: This expression has type
  [> `Assoc of string * [> `String of string ] ]
  but an expression was expected of type Yojson.Basic.json
Types for tag `Assoc are incompatible
(Ошибка: Это выражение имеет тип
  [> `Assoc of string * [> `String of string ] ]
  тогда как ожидалось выражение типа Yojson.Basic.json
  Типы для тега `Assoc несовместимы)
```

Сообщение об ошибке излишне подробно и может вызывать сложности при наличии большого числа значений. Вы можете помочь компилятору сократить сообщение об этой ошибке, явно добавив аннотацию типа и сообщив ему о своих намерениях:

OCaml utop (part 5)

https://github.com/realworldocaml/examples/blob/v1/code/json/build_json.topscript

```
# let (person : Yojson.Basic.json) =
  `Assoc ("name", `String "Anil");;
Characters 37-68:
Error: This expression has type 'a * 'b
  but an expression was expected of type
    (string * Yojson.Basic.json) list
(Ошибка: Это выражение имеет тип 'a * 'b
  тогда как ожидается выражение типа
    (string * Yojson.Basic.json) list)
```

Мы указали, что значение `person` имеет тип `Yojson.Basic.json`, и, как результат, компилятор обнаружил, что аргумент варианта `Assoc` имеет некорректный тип. Данный пример иллюстрирует сильные и слабые стороны полиморфных вариантов: они легковесны и гибки, но сообщения об ошибках могут сбивать с толку. Однако явное аннотирование значений значительно облегчает эту проблему.

Другие приемы, которые, подобно этому, помогают интерпретировать сообщения об ошибках, мы обсудим в главе 22.

Использование нестандартных расширений JSON

Стандартные типы JSON действительно очень просты – типы OCaml намного более выразительны. Библиотека `Yojson` поддерживает расширенный формат JSON для случаев, когда не требуется взаимодействовать с внешними системами, а все-

го лишь необходим простой, удобочитаемый формат представления данных для внутренних нужд. Тип `Yojson.Safe.json` является надмножеством полиморфного варианта `Basic` и имеет следующее определение:

OCaml

https://github.com/realworldocaml/examples/blob/v1/code/json/yojson_safe.mli

```
type json = [
  | `Assoc of (string * json) list
  | `Bool of bool
  | `Float of float
  | `Floatlit of string
  | `Int of int
  | `Intlit of string
  | `List of json list
  | `Null
  | `String of string
  | `Stringlit of string
  | `Tuple of json list
  | `Variant of string * json option
]
```

Тип `Safe.json` включает все варианты из `Basic.json` и добавляет еще несколько. Стандартный тип JSON, такой как `String`, можно использовать и с модулем `Basic`, и с нестандартным модулем `Safe`. Однако если попытаться использовать значения дополнительных типов с модулем `Basic`, компилятор отвергнет такой код, пока вы не сделаете его совместимым с переносимым подмножеством JSON.

Библиотека `Yojson` поддерживает следующие расширения JSON:

- суффикс `lit` – отмечает значения, хранящиеся в виде строк JSON. Например, значение типа `Floatlit` будет сохранено как `"1.234"` вместо `1.234`;
- тип `Tuple` – сохраняется как `("abc", 123)` вместо списка;
- тип `Variant` – позволяет более явно кодировать варианты OCaml, как `<"Foo">` или `<"Bar":123>` для варианта с параметрами.

Единственное назначение этих расширений – дать более полный контроль над представлением значений OCaml в формате JSON (например, сохранять вещественные числа в виде строк JSON).

Вывод все еще соответствует тому же стандартному формату, применяя который, легко можно организовать обмен данными с программами, написанными на других языках.

Преобразовать значение типа `Safe.json` в значение типа `Basic.json` можно с помощью функции `to_basic`:

OCaml (part 1)

https://github.com/realworldocaml/examples/blob/v1/code/json/yojson_safe.mli

```
val to_basic : json -> Yojson.Basic.json
(** Кортежи преобразуются в массивы JSON, Variants преобразуются в
строки JSON или массивы строк (конструктор) и значение json
(аргумент). Длинные целые преобразуются в строки JSON.
```

Примеры:

```
`Tuple [ `Int 1; `Float 2.3 ] -> `List [ `Int 1; `Float 2.3 ]
`Variant ("A", None)         -> `String "A"
`Variant ("B", Some x)       -> `List [ `String "B", x ]
`Intlit "12345678901234567890" -> `String "12345678901234567890"
```

*)

Автоматическое отображение JSON в типы OCaml

Комбинаторы, описанные выше, упрощают создание функций для извлечения полей из записей JSON, однако писать их все еще приходится вручную. При реализации больших спецификаций намного проще генерировать отображения из схем JSON в значения OCaml механистическим способом, чем писать функции преобразования по отдельности.

Далее мы рассмотрим альтернативный метод обработки JSON с использованием инструментов OCaml. В результате у нас получится наш первый предметно-ориентированный язык (Domain Specific Language), компилирующий спецификации JSON в модули OCaml, который затем может использоваться повсюду в приложении.

Установка библиотеки и инструмента ATDgen

Библиотека и инструмент ATDgen включают в себя несколько библиотек OCaml, реализующих интерфейс к библиотеке Yojson, а также инструмент командной строки, генерирующий программный код. Установить весь комплект можно с помощью диспетчера пакетов OPAM:

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/json/install_atdgen.out

```
$ opam install atdgen
$ atdgen -version
1.2.3
```

Инструмент командной строки устанавливается в каталог `~/.opam` и уже должен находиться в пути поиска `PATH` после запуска команды `opam config env`. Если у вас что-то не получилось, обращайтесь за инструкциями по адресу: <https://github.com/realworldocaml/book/wiki/Installation-Instructions>.

Основы ATD

В основе ATD лежит следующая идея: в отдельном файле определяется формат JSON данных, затем запускается компилятор (*atdgen*), который генерирует программный код на языке OCaml, способный конструировать и анализировать значения JSON. Это означает, что вам вообще не потребуется писать код на OCaml, выполняющий парсинг, так как он будет сгенерирован автоматически.

Давайте на конкретном примере посмотрим, как это работает, задействовав небольшую часть GitHub API. GitHub – популярная веб-служба для хостинга IT-проектов и их совместной разработки, предоставляющая API на основе формата JSON. Следующий фрагмент кода ATD описывает API авторизации веб-службы GitHub (опирающийся на псевдостандартный веб-протокол, известный как OAuth):

OCaml

<https://github.com/realworldocaml/examples/blob/v1/code/json/github.atd>

```

type scope = [
  User <json name="user">
  | Public_repo <json name="public_repo">
  | Repo <json name="repo">
  | Repo_status <json name="repo_status">
  | Delete_repo <json name="delete_repo">
  | Gist <json name="gist">
]

type app = {
  name: string;
  url: string;
} <ocaml field_prefix="app_">

type authorization_request = {
  scopes: scope list;
  note: string;
} <ocaml field_prefix="auth_req_">

type authorization_response = {
  scopes: scope list;
  token: string;
  app: app;
  url: string;
  id: int;
  ?note: string option;
  ?note_url: string option;
}

```

Синтаксис спецификации ATD очень похож на синтаксис определения типов OCaml. Каждой записи JSON присваивается имя типа (например, `app` в предыдущем примере). Можно также определять варианты, напоминающие вариантные типы OCaml (как, например, `scope` в примере выше).

Аннотации ATD

Основные отличия синтаксиса ATD от синтаксиса OCaml обусловлены поддержкой аннотаций внутри определений. Аннотации могут уточнять код, который должен быть сгенерирован для конкретной цели (что представляет для нас наибольший интерес).

Например, поле `scope`, объявленное в примере выше как вариантный тип, а имя каждого варианта начинается с буквы верхнего регистра, в соответствии с соглашениями, принятыми для вариантов в языке OCaml. Однако значения JSON, возвращаемые веб-службой GitHub, в действительности имеют имена, начинающиеся с букв нижнего регистра, и потому не имеют точного соответствия с именами вариантов.

Аннотация `<json name="user">` сообщает компилятору, что поле имеет JSON-значение `user`, но соответствующий вариант на языке OCaml носит имя `User`. Такие

аннотации часто помогают отображать значения JSON в зарезервированные слова OCaml (такие как `type`).

Компиляция спецификаций ATD в код на OCaml

Спецификация ATD, представленная выше, может быть скомпилирована в программный код на OCaml с помощью инструмента командной строки *atdgen*. Давайте запустим компилятор дважды, чтобы сгенерировать определения типов на OCaml и модуль сериализации JSON, преобразующий исходные данные в эти типы и обратно.

Инструмент *atdgen* создаст несколько новых файлов в текущем каталоге. Файлы *github_t.ml* и *github_t.mli* будут содержать модуль OCaml с объявлениями типов, соответствующих объявлениям в файле ATD:

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/json/build_github_atd.out

```
$ atdgen -t github.atd
$ atdgen -j github.atd
$ ocamlfind ocamlc -package atd -i github_t.mli
type scope =
  [ `Delete_repo | `Gist | `Public_repo | `Repo | `Repo_status | `User ]
type app = { app_name : string; app_url : string; }
type authorization_request = {
  auth_req_scopes : scope list;
  auth_req_note : string;
}
type authorization_response = {
  scopes : scope list;
  token : string;
  app : app;
  url : string;
  id : int;
  note : string option;
  note_url : string option;
}
```

Соответствие определению ATD достаточно очевидно. Обратите внимание, что имена полей в записях на языке OCaml в одном и том же модуле не могут затенять друг друга, и поэтому мы требуем от ATDgen добавлять префикс к имени каждого поля, чтобы отличать их от полей в других записях, объявленных в том же модуле. Например, `<ocaml field_prefix="auth_req">` в спецификации ATD вынудит компилятор добавить префикс `auth_req_` к имени каждого поля в записи `authorization_request`.

Модуль *Github_t* содержит только определения типов, а модуль *Github_j* — функции сериализации в формат JSON и обратно. С полным интерфейсом можно ознакомиться в файле *github_j.mli*, но наибольший интерес для нас представляют функции преобразования в строки и обратно. Для нашего предыдущего примера они имеют следующий вид:

OCaml

https://github.com/realworldocaml/examples/blob/v1/code/json/github_j_excerpt.mli

```
val string_of_authorization_request :
  ?len:int -> authorization_request -> string
  (** Сериализует значение типа {!authorization_request}
      в строку JSON.
      @param len определяет начальную длину
          буфера для внутреннего использования.
          По умолчанию: 1024. *)

val string_of_authorization_response :
  ?len:int -> authorization_response -> string
  (** Сериализует значение типа {!authorization_response}
      в строку JSON.
      @param len определяет начальную длину
          буфера для внутреннего использования.
          По умолчанию: 1024. *)
```

Это очень удобно! Теперь мы можем написать единственный файл ATD, и весь программный код на OCaml, реализующий преобразование в формат JSON и обратно, будет сгенерирован автоматически. Существует возможность управлять некоторыми аспектами сериализации, передавая флаги инструменту *atdgen*. Наиболее важными для нас являются:

- `-j-std` – преобразует кортежи и варианты в стандартные значения JSON и отвергает вывод значений NaN и бесконечных значений (infinities). Этот флаг следует указывать, если предполагается взаимодействовать со службами, не использующими ATD;
- `-j-custom-fields FUNCTION` – для каждого неопознанного поля вместо возбуждения исключения вызывает указанную функцию *FUNCTION*;
- `-j-defaults` – всегда явно выводит значение JSON, если возможно. Требуется, чтобы в спецификации ATD для поля было определено значение по умолчанию.

Полная спецификация ATD весьма сложна, и полное ее описание можно найти по адресу: <http://mjambon.com/atdgen/atdgen-manual.html>. Компилятор ATD способен также обрабатывать цели, отличные от JSON, и выводить код на других языках программирования (таких как Java), если это необходимо.

Кроме того, существует несколько похожих проектов, позволяющих автоматизировать процесс создания программного кода. Например, проект *Piqi*¹ поддерживает преобразования между форматами XML, JSON и Google Protocol Buffers², а проект *Thrift*³ поддерживает множество других языков программирования, включая OCaml.

Пример: запрос информации об организации в GitHub

Давайте завершим эту главу примером парсинга данных в формате JSON, получаемым от действующего сервера GitHub, и создадим инструмент, запрашиваю-

¹ <http://piqi.org/>.

² http://ru.wikipedia.org/wiki/Protocol_Buffers. – *Прим. перев.*

³ <http://thrift.apache.org/>.

ший информацию об организации в репозитории GitHub посредством имеющегося API. Для начала загляните в электронную документацию с описанием API GitHub, чтобы увидеть, как выглядит схема JSON с информацией об организации.

Теперь создадим файл ATD, описывающий все необходимые нам поля. Любые дополнительные поля, присутствующие в ответе, будут игнорироваться парсером ATD, поэтому нам не требуется обеспечивать полный охват всех полей, которые сервер GitHub может посылать в ответе:

OCaml

https://github.com/realworldocaml/examples/tree/v1/code/json/github_org.atd

```
type org = {
  login: string;
  id: int;
  url: string;
  ?name: string option;
  ?blog: string option;
  ?email: string option;
  public_repos: int
}
```

Создадим объявление типа OCaml перед применением `atdgen -t` к файлу спецификации:

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/json/generate_github_org_types.out

```
$ atdgen -t github_org.atd
$ cat github_org_t.mli
(* Сгенерирован автоматически на основе "github_org.atd" *)
```

```
type org = {
  login: string;
  id: int;
  url: string;
  name: string option;
  blog: string option;
  email: string option;
  public_repos: int
}
```

Тип OCaml имеет очевидное соответствие со спецификацией ATD, но нам все еще необходима логика, которая выполняет преобразования данных JSON в этот тип и обратно. Вызов `atdgen -j` сгенерирует весь необходимый код и сохранит его в новом файле `github_org_j.ml`:

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/json/generate_github_org_json.out

```
$ atdgen -j github_org.atd
$ cat github_org_j.mli
(* Сгенерирован автоматически на основе "github_org.atd" *)
```

```

type org = Github_org_t.org = {
  login: string;
  id: int;
  url: string;
  name: string option;
  blog: string option;
  email: string option;
  public_repos: int
}

val write_org :
  Bi_outbuf.t -> org -> unit
  (** Выводит значение JSON типа {!org}. *)

val string_of_org :
  ?len:int -> org -> string
  (** Сериализует значение типа {!org}
      в строку JSON.
      @param len определяет начальную длину
      буфера для внутреннего использования.
      По умолчанию: 1024. *)

val read_org :
  Yojson.Safe.lexer_state -> Lexing.lexbuf -> org
  (** Вводит данные JSON типа {!org}. *)

val org_of_string :
  string -> org
  (** Десериализует данные JSON типа {!org}. *)

```

Интерфейс `Github_org_j` сериализации содержит все необходимое для отображения в типы OCaml и JSON. Проще всего взаимодействовать с этим интерфейсом через функции `string_of_org` и `org_of_string`, но существуют также более мощные, низкоуровневые функции для работы с буферами, которые можно использовать в ситуациях, когда требуется высокая производительность (но мы не будем рассматривать их в этом примере).

Чтобы закончить пример, нам необходимо написать программу на OCaml, которая будет извлекать данные в формате JSON и с помощью этих модулей выводить сводную информацию. Именно это и делает наш следующий пример.

Программа, представленная ниже, вызывает утилиту `curl` командной строки посредством интерфейса `Core_extended.Shell` и перехватывает ее вывод. Перед опробованием примера убедитесь, что утилита `curl` установлена в вашей системе. Вам может также понадобиться выполнить команду `opam install core_extended`, если данная библиотека еще не установлена:

OCaml

https://github.com/realworldocaml/examples/tree/v1/code/json/github_org_info.ml

```

open Core.Std

let print_org file () =
  let url = sprintf "https://api.github.com/orgs/%s" file in
  Core_extended.Shell.run_full "curl" [url]
  |> Github_org_j.org_of_string

```

```
|> fun org ->
let open Github_org_t in
let name = Option.value ~default:"???" org.name in
printf "%s (%d) with %d public repos\n"
  name org.id org.public_repos

let () =
  Command.basic ~summary:"Print Github organization information"
    Command.Spec.(empty +> anon ("organization" %: string))
    print_org
|> Command.run
```

Следующий короткий сценарий на языке командной оболочки генерирует весь необходимый программный код и создает окончательный выполняемый файл:

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/json/build_github_org.out

```
$ atdgen -t github_org.atd
$ atdgen -j github_org.atd
$ corebuild -pkg core_extended,yojson,atdgen github_org_info.native
```

Теперь можно попробовать запустить инструмент командной строки с единственным аргументом, определяющим имя организации. Он должен извлечь данные в формате JSON из Сети, проанализировать их и вывести сводную информацию в консоль:

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/json/run_github_org.out

```
$ ./github_org_info.native mirage
Mirage account (131943) with 37 public repos
$ ./github_org_info.native janestreet
??? (3384712) with 34 public repos
```

В данных JSON, возвращаемых в ответ на запрос информации об организации *janestreet*, отсутствует название организации, но такая возможность явно отражена в типе OCaml, потому что в спецификации ATD поле *name* отмечено как обязательное. Наш код на OCaml явно обрабатывает эту ситуацию, и потому нам можно не беспокоиться об исключениях, возникающих при ссылке по пустому указателю. Аналогично целочисленное поле *id* в схеме JSON отображено в целочисленное значение OCaml.

Наш пример очень прост, тем не менее он наглядно демонстрирует возможность определения необязательных полей и значений по умолчанию. Загляните в полное описание спецификации ATD для GitHub API, которое можно найти по адресу <http://github.com/avsm/ocaml-github>, где вы обнаружите множество ухищрений, так типичных для API действующих веб-служб.

Наш пример основан на использовании утилиты командной строки *curl*, с помощью которой он получает данные в формате JSON, что является не самым эффективным решением. Поэтому в главе 18 мы познакомимся с возможностью взаимодействий по протоколу HTTP непосредственно из программ на OCaml.

Парсинг с помощью OCamllex и Menhir

Многие задачи программирования начинаются с интерпретации текстовых данных в той или иной форме. *Парсинг (parsing)* – это процесс преобразования таких текстовых данных в структуры, которыми легко может оперировать программный код. Для большей простоты часто используются узкоспециализированные подходы к парсингу данных, например данные сначала разбиваются на строки, а затем с помощью регулярных выражений из этих строк извлекаются отдельные компоненты.

Но такой упрощенный подход малопригоден для парсинга сложных данных, и особенно данных с рекурсивной структурой, которые широко распространены в языках программирования, или форматах, таких как JSON и XML. Точный и эффективный парсинг подобных форматов с поддержкой информативных сообщений об ошибках является непростой задачей.

Часто можно найти уже готовую библиотеку, решающую все эти проблемы. Кроме того, существуют инструменты в форме *генераторов парсеров*, упрощающие задачу создания собственных парсеров. Генератор парсеров создает парсер, опираясь на спецификацию формата данных, который требуется анализировать.

Генераторы парсеров имеют длинную историю, и в их число входят такие инструменты, как *lex* и *yacc*, появившиеся в начале 1970-х годов. В языке OCaml имеются собственные альтернативы: *ocamllex*, служащая заменой *lex*, а также *ocamlyacc* и *menhir*, служащие заменой *yacc*. Мы исследуем эти инструменты на примере реализации парсера для сериализации формата JSON, обсуждавшегося в главе 15.

Парсинг – довольно сложная и часто запутанная тема, поэтому мы не будем рассказывать здесь обо всех теоретических тонкостях, а представим практическое введение в создание парсеров на языке OCaml.

Menhir и ocamlyacc

Menhir – альтернативный генератор парсеров, который в общем случае обладает более широкими возможностями в сравнении с генератором *ocamlyacc*, имеющим уже довольно почтенный возраст. Menhir в значительной степени совместим с грамматикой *ocamlyacc*, благодаря чему переход на использование Menhir обычно не вызывает проблем (старый код часто не требует изменений или требует незначительных изменений, описанных в руководстве по Menhir).

Самое большое преимущество генератора Menhir состоит в том, что его сообщения об ошибках в целом более информативны и менее запутаны, а генерируемые им парсеры поддерживают возможность повторного использования и проще параметризуются

в модулях OCaml. Мы рекомендуем в новых разработках использовать Menhir вместо `ocamlyacc`.

Генератор Menhir не распространяется в составе OCaml, но он доступен для установки с помощью диспетчера пакетов OPAM: чтобы установить его, выполните команду `opam install menhir`.

Лексический анализ и парсинг

Парсинг (или синтаксический анализ) традиционно делится на два этапа: *лексический анализ* (*lexical analysis*), являющийся разновидностью упрощенной фазы парсинга, в ходе которой поток символов преобразуется в поток логических лексем; и окончательный парсинг, в процессе которого поток лексем преобразуется в окончательное представление, часто — древовидную структуру данных, называемую *абстрактным синтаксическим деревом* (*Abstract Syntax Tree, AST*).

Тот факт, что термин *парсинг* (*parsing*) одновременно обозначает и весь процесс преобразования текстовых данных в структурированную форму, и вторую его фазу преобразования потока лексем в дерево AST, вносит определенную путаницу. Поэтому с этого момента мы будем использовать термин *парсинг* для обозначения второй фазы синтаксического анализа.

Давайте рассмотрим особенности лексического анализа и парсинга в контексте формата JSON. Ниже представлен фрагмент текста, содержащий описание объекта JSON, который состоит из строкового поля с именем `title` и массива `cities`, содержащего два объекта с названиями городов и массивами почтовых индексов:

JSON

<https://github.com/realworldocaml/examples/tree/v1/code/parsing/example.json>

```
{
  "title": "Cities",
  "cities": [
    { "name": "Chicago", "zip": [60601] },
    { "name": "New York", "zip": [10004] }
  ]
}
```

С точки зрения синтаксиса этот фрагмент можно рассматривать как последовательность простых логических единиц: фигурных скобок, квадратных скобок, запятых, двоеточий, идентификаторов, чисел и строк в кавычках. То есть текст в формате JSON можно интерпретировать как последовательность лексем следующего типа:

OCaml

https://github.com/realworldocaml/examples/tree/v1/code/parsing/manual_token_type.ml

```
type token =
  | NULL
  | TRUE
  | FALSE
  | STRING of string
  | INT of int
```

```
| FLOAT of float
| ID of string
| LEFT_BRACK
| RIGHT_BRACK
| LEFT_BRACE
| RIGHT_BRACE
| COMMA
| COLON
| EOF
```

Обратите внимание, что в данном представлении отсутствует некоторая информация об оригинальном тексте. Например, здесь никак не представлены пробельные символы. Для анализа потока лексем вполне типично и даже полезно скрывать некоторые тонкости, касающиеся исходного текста, которые не требуются, чтобы понять его назначение.

Если предыдущий пример текста в формате JSON преобразовать в список таких лексем, он мог бы выглядеть как-то так:

OCaml

<https://github.com/realworldocaml/examples/tree/v1/code/parsing/tokens.ml>

```
[ LEFT_BRACE; ID("title"); COLON; STRING("Cities"); COMMA; ID("cities"); ...
```

С данными в таком представлении проще работать, чем с исходным текстом, поскольку в нем отсутствуют некоторые неважные синтаксические детали и добавлена структура. Но даже в таком виде данные остаются слишком низкоуровневыми, в сравнении с простым деревом AST, использовавшимся в главе 15 для представления данных в формате JSON:

OCaml

<https://github.com/realworldocaml/examples/tree/v1/code/parsing/json.ml>

```
type value = [
  | `Assoc of (string * value) list
  | `Bool of bool
  | `Float of float
  | `Int of int
  | `List of value list
  | `Null
  | `String of string
]
```

Такое представление гораздо удобнее в обращении, чем поток лексем, так как отражает тот факт, что значения JSON могут вкладываться друг в друга и могут иметь разные типы, такие как числа, строки, массивы и объекты. Парсер, который мы напишем, будет преобразовывать поток лексем в значение типа AST, представленное ниже и соответствующее нашему примеру данных в формате JSON:

OCaml

https://github.com/realworldocaml/examples/tree/v1/code/parsing/parsed_example.ml

```
`Assoc
  ["title", `String "Cities";
```



```
"cities", `List
[`Assoc ["name", `String "Chicago"; "zip", `List [`Int 60601]];
`Assoc ["name", `String "New York"; "zip", `List [`Int 10004]]]]
```

Определение парсера

Файл с определением парсера имеет расширение *.mly* и состоит из двух разделов, которые разбиты разделительной строкой, состоящей из символов `%`. Первый раздел файла содержит объявления, включая определения лексем и типов, директивы предшествования и другие директивы. Второй раздел отводится для описания грамматики анализируемого языка.

Начнем с объявления списка лексем. Объявление любой лексемы имеет вид `%token <type> uid`, где поле `<type>` является необязательным, а поле `uid` содержит идентификатор, состоящий только из заглавных букв. Для анализа формата JSON нам потребуются лексемы, соответствующие числам, строкам, идентификаторам и знакам пунктуации:

OCaml

<https://github.com/realworldocaml/examples/tree/v1/code/parsing/parser.mly>

```
%token <int> INT
%token <float> FLOAT
%token <string> ID
%token <string> STRING
%token TRUE
%token FALSE
%token NULL
%token LEFT_BRACE
%token RIGHT_BRACE
%token LEFT_BRACK
%token RIGHT_BRACK
%token COLON
%token COMMA
%token EOF
```

Наличие поля `<type>` в определении означает, что лексема несет в себе некоторое значение. Лексема `INT`, например, несет целое число, `FLOAT` — вещественное число, а `STRING` — строку. Остальные лексемы, такие как `TRUE`, `FALSE` и знаки пунктуации, не связаны ни с каким значением, и поэтому для них определение `<type>` не указывается.

Описание грамматики

Теперь необходимо описать грамматику выражения JSON. Утилита *menhir*, подобно большинству генераторов парсеров, использует *контекстно-свободные грамматики* (*context-free grammars*). (Если говорить точнее, утилита *menhir* поддерживает грамматики LR(1), но мы не будем обращать внимания на эти технические тонкости.) Контекстно-свободную грамматику можно рассматривать как множество аб-

страктных имен, называемых *нетерминальными символами* (*nonterminal symbols*), в комплексе с коллекцией правил преобразования нетерминальных символов в последовательности лексем и нетерминальных символов. Последовательность лексем может быть разобрана с помощью грамматики, если есть возможность применить правила этой грамматики, чтобы произвести последовательность преобразований, начиная с определенного *начального символа*, открывающего последовательность лексем.

Описание грамматики JSON мы начнем с объявления начального нетерминального символа `prog`, встретив который, парсер должен преобразовать его значение в значение OCaml типа `Json.value option`. После этого мы закроем раздел объявлений строкой с символами `%%`:

OCaml (part 1)

<https://github.com/realworldocaml/examples/tree/v1/code/parsing/parser.mly>

```
%start <Json.value option> prog
%%
```

Теперь можно начинать определять порядок создания парсера. В терминах *menhir* порядок задается набором правил, где каждое правило перечисляет все возможные результаты анализа того или иного нетерминального символа. Ниже приводится пример правила для символа `prog`:

OCaml (part 2)

<https://github.com/realworldocaml/examples/tree/v1/code/parsing/parser.mly>

```
prog:
  | EOF          { None }
  | v = value { Some v }
;
```

Синтаксис определения правил напоминает синтаксис инструкции `match` в языке OCaml. Символами вертикальной черты начинаются описания отдельных результатов, а фигурные скобки содержат описание семантики действия: программный код на языке OCaml, генерирующий значение OCaml, соответствующее данному случаю. Это могут быть любые допустимые выражения на языке OCaml, которые будут вычисляться на этапе парсинга и производить значения, присоединенные к нетерминальным символам.

Для символа `prog` возможны два случая. Первый: `EOF` — когда текст в формате JSON отсутствует и, соответственно, отсутствует значение JSON, поэтому в данном случае возвращается значение `None`. Второй: когда имеется экземпляр значения нетерминального символа `value`, соответствующий правильно оформленному значению JSON, и в этом случае мы заворачиваем значение `Json.value` в `ter Some`. Обратите внимание, что здесь мы связываем значение `value` с переменной `v`, которая затем используется в фигурных скобках.

Теперь перейдем к более сложному примеру и рассмотрим правило для символа `value`:

OCaml (part 3)

<https://github.com/realworldocaml/examples/tree/v1/code/parsing/parser.mly>

```
value:
| LEFT_BRACE; obj = object_fields; RIGHT_BRACE
  { `Assoc obj }
| LEFT_BRACK; vl = array_values; RIGHT_BRACK
  { `List vl }
| s = STRING
  { `String s }
| i = INT
  { `Int i }
| x = FLOAT
  { `Float x }
| TRUE
  { `Bool true }
| FALSE
  { `Bool false }
| NULL
  { `Null }
;
```

Согласно этим правилам, значение JSON value может быть:

- объектом, заключенным в фигурные скобки;
- массивом, заключенным в квадратные скобки;
- строкой, целым или вещественным числом, логическим или пустым (null) значением.

В каждом из этих случаев программный код на OCaml в фигурных скобках показывает, как преобразуется рассматриваемый объект. Отметим, что здесь присутствуют два нетерминала, которые мы пока не определили: `object_fields` и `array_values`. Парсинг этих символов мы рассмотрим далее.

Парсинг последовательностей

Следующее правило для `object_fields` в действительности является лишь тонкой оберткой, изменяющей порядок следования элементов на обратный в списке, возвращаемом следующим далее правилом для `rev_object_fields`. Обратите внимание, что первый случай в `rev_object_fields` обрабатывает ситуацию с пустым значением слева, соответствующую пустой последовательности лексем. Комментарий (* пустая *) добавляет ясности:

OCaml (part 4)

<https://github.com/realworldocaml/examples/tree/v1/code/parsing/parser.mly>

```
object_fields: obj = rev_object_fields { List.rev obj };

rev_object_fields:
| (* пустая *) { [] }
| obj = rev_object_fields; COMMA; k = ID; COLON; v = value
  { (k, v) :: obj }
;
```

Правила структурированы именно так, потому что *menhir* генерирует *леворекурсивные парсеры (left-recursive parsers)* – магазинные автоматы, потребляющие меньше памяти. Следующее праворекурсивное правило (*right-recursive rule*) принимает те же исходные данные, но в процессе парсинга потребляет больше памяти, необходимой для чтения определения полей объектов:

OCaml (part 4)

https://github.com/realworldocaml/examples/tree/v1/code/parsing/right_rec_rule.mly

```
(* Неэффективное праворекурсивное правило *)
object_fields:
| (* пустая *) { [] }
| k = ID; COLON; v = value; COMMA; obj = object_fields
  { (k, v) :: obj }
```

При желании можно сохранить леворекурсивное определение и просто конструировать возвращаемое значение слева направо. Но этот прием еще менее эффективен, потому что время конструирования списка увеличивается с его длиной и при таком подходе находится в квадратичной зависимости:

OCaml (part 4)

https://github.com/realworldocaml/examples/tree/v1/code/parsing/quadratic_rule.mly

```
(* Квадратичное леворекурсивное правило *)
object_fields:
| (* empty *) { [] }
| obj = object_fields; COMMA; k = ID; COLON; v = value
  { obj @ [k, v] }
;
```

Сборка подобных списков является распространенной задачей в большинстве грамматик, и подобные правила, показанные выше, выглядят избыточными (хотя знакомство с ними полезно для понимания того, как выполняется парсинг). Утилита *Menhir* включает расширенную библиотеку встроенных правил, упрощающих обработку подобных ситуаций. Эти правила подробно описаны в руководстве к *Menhir* и включают необязательные значения, пары значений с необязательным разделителем, а также списки элементов (также с необязательным разделителем).

Ниже приводится версия грамматики JSON, в которой используются более краткие правила *Menhir*. Обратите внимание, что правило *separated_list* применяется одновременно и для анализа объектов JSON, и для анализа списков:

OCaml (part 1)

https://github.com/realworldocaml/examples/tree/v1/code/parsing/short_parser.mly

```
prog:
| v = value { Some v }
| EOF { None } ;

value:
| LEFT_BRACE; obj = obj_fields; RIGHT_BRACE { `Assoc obj }
| LEFT_BRACK; vl = list_fields; RIGHT_BRACK { `List vl }
| s = STRING { `String s }
| i = INT { `Int i }
```

```

| x = FLOAT                { `Float x }
| TRUE                     { `Bool true }
| FALSE                   { `Bool false }
| NULL                    { `Null } ;

obj_fields:
  obj = separated_list(COMMA, obj_field)    { obj } ;

obj_field:
  k = STRING; COLON; v = value              { (k, v) } ;

list_fields:
  vl = separated_list(COMMA, value)         { vl } ;

```

Вызвать утилиту *menhir* можно с помощью *corebuild*, указав флаг `-use-menhir`. Он сообщает системе сборки, что для обработки файлов с расширением *.mly* она должна использовать *menhir* вместо *ocamlyacc*:

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/parsing/build_short_parser.out
 \$ `corebuild -use-menhir short_parser.mli`

Определение лексического анализатора

Теперь с помощью *ocamllex* можно определить лексический анализатор, который будет преобразовывать исходный текст в поток лексем. Определение лексического анализатора должно храниться в файле с расширением *.mll*.

Вступление

Будем рассматривать определение лексического анализатора по разделам. Первый раздел – необязательный фрагмент программного кода на языке OCaml, заключенный в фигурные скобки:

OCaml

<https://github.com/realworldocaml/examples/tree/v1/code/parsing/lexer.mll>

```

{
open Lexing
open Parser

exception SyntaxError of string

let next_line lexbuf =
  let pos = lexbuf.lex_curr_p in
  lexbuf.lex_curr_p <-
    { pos with pos_bol = lexbuf.lex_curr_pos;
      pos_lnum = pos.pos_lnum + 1
    }
}

```

Этот код определяет вспомогательные функции для использования в последующих фрагментах кода на OCaml, а также выполняет настройку окружения, открывая необходимые модули и определяя исключение `SyntaxError`.

Вспомогательная функция `next_line` служит для определения местоположения лексем. Модуль `Lexing` определяет структуру `lexbuf`, где хранится состояние лексического анализатора, включая текущее местоположение в исходном файле. Функция `next_line` просто обращается к полю `lex_curr_pos` с информацией о текущем местоположении и обновляет номер строки.

Регулярные выражения

Следующий раздел в файле с лексическим анализатором содержит коллекцию именованных регулярных выражений. Синтаксически эти определения очень похожи на `let`-привязки в языке OCaml, но в действительности они являются специализированными синтаксическими конструкциями объявления регулярных выражений. Например:

OCaml (part 1)

<https://github.com/realworldocaml/examples/tree/v1/code/parsing/lexer.mll>

```
let int = '-'? ['0'-'9'] ['0'-'9']*
```

Данный синтаксис являет собой смесь синтаксиса языка OCaml и традиционного синтаксиса регулярных выражений. Регулярное выражение `int` определяет необязательный ведущий знак «минус» (`-`), за которым следует цифра от 0 до 9, за которой, в свою очередь, может следовать произвольное число цифр то 0 до 9. Знак вопроса указывает, что предшествующий ему компонент регулярного выражения является необязательным; квадратные скобки используются для определения диапазонов; а оператор `*` указывает, что предшествующий компонент может повторяться произвольное число раз (в том числе и 0 раз).

Вещественные числа определяются аналогично, с той лишь разницей, что здесь необходимо предусмотреть возможность присутствия десятичной точки и экспоненты. Мы сделали регулярное выражение более удобочитаемым, реализовав его как последовательность именованных регулярных выражений:

OCaml (part 2)

<https://github.com/realworldocaml/examples/tree/v1/code/parsing/lexer.mll>

```
let digit = ['0'-'9']
let frac = '.' digit*
let exp = ['e' 'E'] ['- ' '+']? digit+
let float = digit* frac? exp?
```

В конце определяются пробельные символы, символы перевода строки и идентификаторы:

OCaml (part 3)

<https://github.com/realworldocaml/examples/tree/v1/code/parsing/lexer.mll>

```
let white = [' ' '\t']+
let newline = '\\r' | '\\n' | "\\r\\n"
let id = ['a'-'z' 'A'-'Z' '_' ] ['a'-'z' 'A'-'Z' '0'-'9' '_' ]*
```

В выражении `newline` присутствует оператор `|`, который в регулярных выражениях позволяет определять несколько альтернатив (в данном случае – различные комбинации из символов возврата каретки и перевода строки: CR, LF и CRLF).

Лексические правила

Лексические правила, по сути, являются функциями, получающими данные и возвращающими выражения на языке OCaml, которые дают в результате лексемы. Эти выражения могут быть весьма сложными, производить побочные эффекты и вызывать другие правила. Рассмотрим правило `read` для парсинга выражения JSON:

OCaml (part 4)

<https://github.com/realworldocaml/examples/tree/v1/code/parsing/lexer.mll>

```
rule read =
  parse
  | white { read lexbuf }
  | newline { next_line lexbuf; read lexbuf }
  | int { INT (int_of_string (Lexing.lexeme lexbuf)) }
  | float { FLOAT (float_of_string (Lexing.lexeme lexbuf)) }
  | "true" { TRUE }
  | "false" { FALSE }
  | "null" { NULL }
  | '"' { read_string (Buffer.create 17) lexbuf }
  | '{' { LEFT_BRACE }
  | '}' { RIGHT_BRACE }
  | '[' { LEFT_BRACK }
  | ']' { RIGHT_BRACK }
  | ':' { COLON }
  | ',' { COMMA }
  | _ { raise (SyntaxError ("Unexpected char: " ^ Lexing.lexeme lexbuf)) }
  | eof { EOF }
```

Правила структурированы подобно операции сопоставления с образцом, с той лишь разницей, что варианты замещаются регулярными выражениями слева. Справа находится возвращаемый результат. Код на языке OCaml получает параметр с именем `lexbuf`, содержащий позицию в исходном файле, а также текст, совпавший с регулярным выражением.

Первый вариант `white { read lexbuf }` рекурсивно вызывает лексический анализатор. То есть он пропускает пробельный символ и возвращает следующую лексему. Вариант `newline { next_line lexbuf; read lexbuf }` действует аналогично, но дополнительно увеличивает номер строки с помощью вспомогательной функции, объявленной в начале файла. Перейдем к третьему варианту:

OCaml

https://github.com/realworldocaml/examples/tree/v1/code/parsing/lexer_int_fragment.mll

```
| int { INT (int_of_string (Lexing.lexeme lexbuf)) }
```

Он указывает, что при обнаружении совпадения с регулярным выражением `int` лексический анализатор должен вернуть выражение `INT (int_of_string (Lexing.lexeme lexbuf))`. Выражение `Lexing.lexeme lexbuf` возвращает полную строку совпадения с регулярным выражением. В данном случае – строковое представление числа, поэтому мы вызываем функцию `int_of_string` для преобразования этой строки в число.

Далее следуют варианты для всех остальных разновидностей лексем. Строковое выражение, такое как `"true" { TRUE }`, применяется к ключевым словам, также определены варианты для специальных символов, такие как `'{' { LEFT_BRACE }`.

Некоторые из вариантов перекрывают друг друга. Например, регулярное выражение `"true"` также соответствует образцу `id`. Для устранения неоднозначности, когда обнаруживается совпадение с несколькими образцами, *ocamllex* использует следующие правила.

- Побеждает всегда самое длинное совпадение. Например, для строки `trueX`: 167 регулярное выражение `"true"` найдет совпадение с четырьмя символами, а `id` найдет совпадение с пятью символами. В состязании победит более длинное совпадение, и в результате будет возвращено значение ID `"trueX"`.
- Если все совпадения имеют одинаковую длину, побеждает вариант, стоящий выше. Например, для строки `true: 167` оба варианта, `"true"` и `id`, совпадут с первыми четырьмя символами, но вариант `"true"` стоит выше, поэтому будет возвращено значение `TRUE`.

Рекурсивные правила

В отличие от многих других генераторов лексических анализаторов, *ocamllex* позволяет определять в одном файле сразу несколько лексических анализаторов, и эти определения могут быть рекурсивными. В данном примере мы используем рекурсию для анализа строковых литералов:

OCaml (part 5)

<https://github.com/realworldocaml/examples/tree/v1/code/parsing/lexer.mll>

```
and read_string buf =
  parse
  | '"' { STRING (Buffer.contents buf) }
  | '\\' '/' { Buffer.add_char buf '/'; read_string buf lexbuf }
  | '\\' '\\' { Buffer.add_char buf '\\'; read_string buf lexbuf }
  | '\\' 'b' { Buffer.add_char buf '\b'; read_string buf lexbuf }
  | '\\' 'f' { Buffer.add_char buf '\012'; read_string buf lexbuf }
  | '\\' 'n' { Buffer.add_char buf '\n'; read_string buf lexbuf }
  | '\\' 'r' { Buffer.add_char buf '\r'; read_string buf lexbuf }
  | '\\' 't' { Buffer.add_char buf '\t'; read_string buf lexbuf }
  | [^ '"' '\\']+
    { Buffer.add_string buf (Lexing.lexeme lexbuf);
      read_string buf lexbuf
    }
  | _ { raise
      (SyntaxError("Illegal string character: " ^ Lexing.lexeme lexbuf)) }
  | eof { raise (SyntaxError ("String is not terminated")) }
```

Это правило получает аргумент `buf : Buffer.t`. Если достигнута закрывающая кавычка `"`, мы возвращаем содержимое буфера как `STRING`.

Остальные варианты служат для обработки содержимого строк. Вариант `[^ '"' '\\']+ { ... }` соответствует обычному исходному тексту без двойных кавычек и символов обратной косой черты. Варианты, начинающиеся с обратной косой чер-

ты (`\`), определяют реакцию на экранированные последовательности. Во всех этих вариантах в конце выполняется рекурсивный вызов лексического анализатора.

Это все, что касается лексического анализатора. Далее нам нужно объединить лексический анализатор с парсером.

Обработка Юникода

В дискуссии выше мы умолчали об одной важной детали: парсинг символов Юникода, который помог бы обрабатывать полный спектр систем письменности, существующих в мире. Для OCaml имеется несколько сторонних решений для работы с Юникодом, имеющих разные степени гибкости и сложности.

- Camomile¹ поддерживает весь спектр символов Юникода, порядка 200 кодировок, а также правила сортировки с учетом национальных требований.
- Ulex² – генератор лексических анализаторов для Юникода, который может служить как замена для `osamllex` с поддержкой Юникода.
- Utf³ – неблокирующий потоковый кодек Юникода для OCaml, доступный как отдельная библиотека. В его состав входят библиотеки: `Uunf`⁴ – для нормализации текста и `Uucd`⁵ – база данных символов Юникода. В Интернете можно также найти парсер JSON⁶, иллюстрирующий применение Utf.

Все эти библиотеки доступны для установки с помощью OPAM под соответствующими именами.

Объединяем все вместе

В заключительном разделе этой главы объединим лексический анализатор с парсером. Как мы видели в определении типа в файле *parser.mli*, функция, выполняющая парсинг, ожидает получить лексический анализатор типа `Lexing.lexbuf -> token` и `lexbuf`:

OCaml

<https://github.com/realworldocaml/examples/blob/v1/code/parsing/prog.mli>

```
val prog: (Lexing.lexbuf -> token) -> Lexing.lexbuf -> Json.value option
```

Прежде чем приступить к лексическому анализу, давайте сначала определим функции для обработки ошибок парсинга. В настоящий момент поддерживаются две ошибки: `Parser.Error` и `Lexer.SyntaxError`. Проще всего при встрече ошибки вывести ее и завершить работу:

OCaml

<https://github.com/realworldocaml/examples/blob/v1/code/parsing-test/test.ml>

```
open Core.Std
open Lexer
open Lexing
```

¹ <http://camomile.sourceforge.net/>.

² <http://www.cduce.org/ulex>.

³ <http://erratique.ch/software/uutf>.

⁴ <http://erratique.ch/software/uunf>.

⁵ <http://erratique.ch/software/uucd>.

⁶ <http://erratique.ch/software/jsonm>.

```

let print_position outx lexbuf =
  let pos = lexbuf.lex_curr_p in
  fprintf outx "%s:%d:%d" pos.pos_fname
    pos.pos_lnum (pos.pos_cnum - pos.pos_bol + 1)

let parse_with_error lexbuf =
  try Parser.prog Lexer.read lexbuf with
  | SyntaxError msg ->
    fprintf stderr "%a: %s\n" print_position lexbuf msg;
    None
  | Parser.Error ->
    fprintf stderr "%a: syntax error\n" print_position lexbuf;
    exit (-1)

```

Подход «завершить при первой же ошибке» самый простой в реализации, но не самый дружелюбный. Обработка ошибок часто оказывается весьма сложной задачей, поэтому мы не будем обсуждать пути ее решения здесь. Однако отметим, что генератор *Menhir* определяет дополнительные механизмы, которые можно попробовать использовать для обработки ошибок. Они подробно описываются в справочном руководстве.

Стандартная библиотека поддержки лексического анализа *Lexing* предоставляет функцию `from_channel` для чтения исходного текста из канала. Следующая функция описывает структуру, где функция `Lexing.from_channel` используется для создания значения `lexbuf`, которое вместе с функцией `Lexer.read` передается функции парсера `Parser.prog`. `Parsing.prog` возвращает `None` по достижении конца файла. Для вывода `Json.value` мы определили функцию `Json.output_value`, не показанную здесь:

OCaml (part 1)

<https://github.com/realworldocaml/examples/blob/v1/code/parsing-test/test.ml>

```

let rec parse_and_print lexbuf =
  match parse_with_error lexbuf with
  | Some value ->
    printf "%a\n" Json.output_value value;
    parse_and_print lexbuf
  | None -> ()

let loop filename () =
  let inx = In_channel.create filename in
  let lexbuf = Lexing.from_channel inx in
  lexbuf.lex_curr_p <- { lexbuf.lex_curr_p with pos_fname = filename };
  parse_and_print lexbuf;
  In_channel.close inx

```

Ниже приводится содержимое исходного файла для тестирования кода:

JSON

<https://github.com/realworldocaml/examples/blob/v1/code/parsing-test/test1.json>

```

true
false
null
[1, 2, 3., 4.0, .5, 5.5e5, 6.3]

```

```
"Hello World"
{ "field1": "Hello",
  "field2": 17e13,
  "field3": [1, 2, 3],
  "field4": { "fieldA": 1, "fieldB": "Hello" }
}
```

Теперь скомпилируем и запустим пример, передав ему этот файл, чтобы посмотреть, как действует парсер:

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/parsing-test/build_test.out

```
$ ocamlbuild -use-menhir -tag thread -use-ocamlfind -quiet -pkg core test.native
$ ./test.native test1.json
true
false
null
[1, 2, 3.000000, 4.000000, 0.500000, 550000.000000, 6.300000]
"Hello World"
{ "field1": "Hello",
  "field2": 1700000000000000.000000,
  "field3": [1, 2, 3],
  "field4": { "fieldA": 1,
    "fieldB": "Hello" } }
```

В нашей реализации любые ошибки являются фатальными и вызывают завершение программы с ненулевым кодом выхода:

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/parsing-test/run_broken_test.out

```
$ cat test2.json
{ "name": "Chicago",
  "zips": [12345,
]
{ "name": "New York",
  "zips": [10004]
}
$ ./test.native test2.json
test2.json:3:2: syntax error
```

На этом мы завершаем обсуждение вопросов, связанных с созданием парсеров. В качестве заключительного замечания отметим, что полиморфный вариантный тип JSON, который был определен в этой главе, в действительности структурно совместим с представлением Yojson, описанным в главе 15. Это означает, что данный парсер можно использовать совместно со вспомогательными функциями из библиотеки Yojson для построения более сложных приложений.

Сериализация данных с применением s-выражений

S-выражения — это вложенные выражения в круглых скобках, атомарными значениями которых являются строки. Свою популярность они обрели благодаря языку программирования Lisp в 1960-х годах. S-выражения остаются одним из самых простых и наиболее эффективных средств представления структурированных данных в удобочитаемой форме.

Полное определение s-выражений доступно в электронном виде по адресу: <http://people.csail.mit.edu/rivest/Sexpr.txt>. Ниже приводится пример s-выражения:

Scheme

<https://github.com/realworldocaml/examples/blob/v1/code/sexpr/basic.scm>

(Это (самое настоящее) (s-выражение))

S-выражения играют важную роль в библиотеке Core, выступая в качестве эффективного формата сериализации по умолчанию. В действительности мы уже встречались с s-выражениями в главах 7, 9 и 10.

В этой главе мы займемся более глубоким исследованием s-выражений. В частности, мы обсудим:

- особенности формата s-выражений, включая его парсинг с выводом информативных сообщений об ошибках при обработке исходных данных с ошибками;
- порядок генерации s-выражений на основе произвольных типов OCaml;
- использование аннотаций типов для управления поведением преобразователей s-выражений;
- порядок интеграции s-выражений в ваши интерфейсы, в частности порядок добавления преобразователей s-выражений в модуль без нарушения границ абстракций.

Мы объединим все это в конце главы, в примере конфигурационного файла для веб-сервера.

ОСНОВЫ ИСПОЛЬЗОВАНИЯ

Тип, представляющий s-выражение, очень прост:

OCaml

<https://github.com/realworldocaml/examples/blob/v1/code/sexpr/sexp.mli>

```
module Sexp : sig
  type t =
    | Atom of string
    | List of t list
end
```

S-выражение можно представить в виде дерева, каждый узел которого представляет собой список дочерних узлов, а листьями являются строки. Библиотека Core обеспечивает отличную поддержку s-выражений в виде модуля `Sexp`, включающего функции для преобразования s-выражений в строки и обратно. Давайте перепишем наш пример s-выражения в терминах этого типа:

OCaml utop

https://github.com/realworldocaml/examples/blob/v1/code/sexpr/print_sexp.topscript

```
# Sexp.List [
  Sexp.Atom "Это";
  Sexp.List [ Sexp.Atom "самое"; Sexp.Atom "настоящее" ];
  Sexp.List [ Sexp.Atom "s"; Sexp.Atom "выражение" ];
];;
- : Sexp.t = (Это (самое настоящее) (s-выражение))
```

Выведенный результат выглядит так опрятно потому, что библиотека Core регистрирует свой механизм форматированного вывода для интерактивной оболочки. Этот механизм основан на функциях в модуле `Sexp` преобразования s-выражений в строки и обратно:

OCaml utop

https://github.com/realworldocaml/examples/blob/v1/code/sexpr/sexp_printer.topscript

```
# Sexp.to_string (Sexp.List [Sexp.Atom "1"; Sexp.Atom "2"]);;
- : string = "(1 2)"
# Sexp.of_string ("(1 2 (3 4))") ;;
- : Sexp.t = (1 2 (3 4))
```

В дополнение к модулю `Sexp` большинство базовых типов в библиотеке Core поддерживает возможность преобразования в s-выражения и обратно. Например, в соответствующих модулях имеются функции преобразования целых чисел, строк и исключений:

OCaml utop

https://github.com/realworldocaml/examples/blob/v1/code/sexpr/to_from_sexp.topscript

```
# Int.sexp_of_t 3;;
- : Sexp.t = 3
# String.sexp_of_t "hello";;
- : Sexp.t = hello
```

```
# Exn.sexp_of_t (Invalid_argument "foo");;
- : Sexp.t = (Invalid_argument foo)
```

Имеется также возможность преобразования более сложных типов, таких как списки или массивы, полиморфные по отношению к типам, содержащихся в них значений:

OCaml utop (part 1)

https://github.com/realworldocaml/examples/blob/v1/code/sexpr/to_from_sexp.topscript

```
# List.sexp_of_t;;
- : ('a -> Sexp.t) -> 'a list -> Sexp.t = <fun>
# List.sexp_of_t Int.sexp_of_t [1; 2; 3];;
- : Sexp.t = (1 2 3)
```

Обратите внимание, что функция `List.sexp_of_t` является полиморфной и принимает в первом аргументе другую функцию преобразования для обработки элементов преобразуемого списка. Библиотека `Core` широко использует эту схему определения преобразователей s-выражений для полиморфных типов.

Функции, выполняющие преобразования в обратном направлении, то есть восстанавливающие значения OCaml из s-выражений, по сути, используют тот же трюк для обработки полиморфных типов, что представлен в следующем примере. Обратите внимание, что эти функции будут возбуждать исключения, если s-выражение не соответствует рассматриваемому типу OCaml.

OCaml utop (part 2)

https://github.com/realworldocaml/examples/blob/v1/code/sexpr/to_from_sexp.topscript

```
# List.t_of_sexp;;
- : (Sexp.t -> 'a) -> Sexp.t -> 'a list = <fun>
# List.t_of_sexp Int.t_of_sexp (Sexp.of_string "(1 2 3)");;
- : int list = [1; 2; 3]
# List.t_of_sexp Int.t_of_sexp (Sexp.of_string "(1 2 three)");;
Exception:
(Sexplib.Conv.Of_sexp_error (Failure "int_of_sexp: (Failure int_of_string)"
three)).
```

Подробнее о функции вывода в интерактивной оболочке

Значения s-выражений, созданные нами, выводятся в интерактивной оболочке именно как s-выражения, а не как варианты `Atom` и `List`, каковыми они являются в действительности.

Это объясняется возможностью в языке OCaml устанавливать нестандартные средства форматированного вывода для интерактивной оболочки, способные выводить некоторые значения в более дружелюбном представлении. В общем случае они устанавливаются как пакеты `ocamlfind` с именами, оканчивающимися на `top`:

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/sexpr/list_top_packages.out

```
$ ocamlfind list | grep top
compiler-libs.toplevel (version: [distributed with Ocaml])
core.top                (version: 109.37.00)
ctypes.top              (version: 0.1)
lwt.simple-top          (version: 2.4.3)
```

```
num.top          (version: 1.3.3)
sexplib.top      (version: 109.20.00)
uri.top          (version: 1.3.8)
```

Пакет `core.top`, который должен загружаться по умолчанию в файле `.ocamlinit`, уже загружает средства форматированного вывода из библиотеки `Core`, поэтому вам не требуется предпринимать каких-либо специальных действий для организации форматированного вывода `s`-выражений.

Преобразование типов OCaml в `s`-выражения

А что, если потребуется функция для преобразования совершенно нового типа в `s`-выражение? Разумеется, вы можете написать ее вручную, например:

OCaml `utop`

https://github.com/realworldocaml/examples/tree/v1/code/sexpr/manually_making_sexp.topscript

```
# type t = { foo: int; bar: float } ;;
type t = { foo : int; bar : float; }
# let sexp_of_t t =
  let a x = Sexp.Atom x and l x = Sexp.List x in
  l [ l [a "foo"; Int.sexp_of_t t.foo ];
    l [a "bar"; Float.sexp_of_t t.bar]; ] ;;
val sexp_of_t : t -> Sexp.t = <fun>
# sexp_of_t { foo = 3; bar = -5.5 } ;;
- : Sexp.t = ((foo 3) (bar -5.5))
```

Писать такие функции довольно утомительно. Еще более утомительно писать функции, выполняющие обратное преобразование, которые потребуются, когда дело дойдет до парсера, то есть `t_of_sexp`, которые гораздо сложнее в реализации. Создание вручную программного кода, осуществляющего парсинг и вывод, требует выполнить массу чисто механической работы, из-за чего увеличивается вероятность появления ошибок.

Учитывая, насколько шаблонный код требуется писать, вы могли бы подумать о создании программы, которая исследовала бы определение типа и автоматически создавала функции преобразования. Как оказывается, пакет `Sexplib` уже реализует такую возможность. Пакет `Sexplib`, входящий в состав библиотеки `Core`, включает библиотеку для выполнения операций с `s`-выражениями и расширение синтаксиса для создания таких функций преобразования. При наличии данного расширения любой тип, включающий аннотацию `with sexp`, будет автоматически снабжаться необходимыми функциями:

OCaml `utop`

https://github.com/realworldocaml/examples/tree/v1/code/sexpr/auto_making_sexp.topscript

```
# type t = { foo: int; bar: float } with sexp ;;
type t = { foo : int; bar : float; }
val t_of_sexp : Sexp.t -> t = <fun>
val sexp_of_t : t -> Sexp.t = <fun>
# t_of_sexp (Sexp.of_string "((bar 35) (foo 3))") ;;
- : t = {foo = 3; bar = 35.}
```

Расширение синтаксиса можно также использовать за пределами объявлений типов. Как отмечалось в главе 7, аннотация `with sexpr` может добавляться к объявлениям исключений, чтобы дать библиотеке Core возможность генерировать более дружелюбные строковые представления значений:

OCaml utop (part 1)

https://github.com/realworldocaml/examples/tree/v1/code/sexpr/auto_making_sexpr.topscript

```
# exception Bad_message of string list ;;
exception Bad_message of string list
# Exn.to_string (Bad_message ["1";"2";"3"]) ;;
- : string = ("\"Bad_message(_)\"")
# exception Good_message of string list with sexpr;;
exception Good_message of string list
# Exn.to_string (Good_message ["1";"2";"3"]) ;;
- : string = ("(/toplevel//.Good_message (1 2 3))")
```

Для создания функций преобразования s-выражений необязательно объявлять именованные типы. Ниже демонстрируется прием, который дает возможность создавать функции прямо в коде, внутри больших выражений:

OCaml utop

https://github.com/realworldocaml/examples/tree/v1/code/sexpr/inline_sexpr.topscript

```
# let l = [(1,"one"); (2,"two")] ;;
val l : (int * string) list = [(1, "one"); (2, "two")]
# List.iter l ~f:(fun x ->
  <:sexpr_of<int * string>> x
  |> Sexp.to_string
  |> print_endline) ;;
(1 one)
(2 two)
- : unit = ()
```

Объявление `<:sexpr_of<int * string>>` просто генерирует `sexpr`-преобразователь для типа `int * string`. Это может пригодиться, когда потребуется организовать `sexpr`-преобразования анонимных типов.

Почти все расширения синтаксиса, входящие в состав Core, имеют одну и ту же базовую структуру: они автоматически генерируют код на основе определений типов, реализующий функциональность, которую теоретически можно было бы написать вручную, но требуют от программиста несоизмеримо меньше усилий.

Расширения синтаксиса, Camlp4 и Type_conv

Язык OCaml не поддерживает непосредственно возможности автоматического создания кода на основе определений типов. Но он предоставляет мощный механизм расширения синтаксиса, известный как Camlp4, который дает возможность расширять грамматику языка. Механизм Camlp4 тесно интегрирован в комплект инструментов OCaml, может активироваться в интерактивной оболочке и подключаться на этапе компиляции флагом `-pp` компилятора.

Пакет Sexplib является частью семейства расширений синтаксиса, включающего также пакет Comparelib, описанный в главе 13, и пакет Fieldslib, описанный в главе 5, генерирующие код, опираясь на объявления типов, и все они основаны на общей библиотеке

с именем `Type_conv`. Эта библиотека обеспечивает поддержку обобщенного языка аннотирования типов (например, аннотации `with`) и утилиты для работы с определениями типов. Если вам понадобится создать собственное расширение синтаксиса, осуществляющее анализ типов, необходимо будет подумать об использовании `Type_conv` в качестве основы.

Формат Sexpr

Текстовое представление s-выражений выглядит очень просто. Любое s-выражение записывается как последовательность вложенных друг в друга выражений в круглых скобках, состоящих из строковых атомов, разделенных пробелами. Атомы, содержащие круглые скобки или пробелы, заключаются в кавычки. Символ обратной косой черты играет роль экранирующего символа, а точка с запятой начинается однострочные комментарии. То есть следующий файл *example.scm*:

Scheme

<https://github.com/realworldocaml/examples/blob/v1/code/sexpr/example.scm>

```
;; example.scm
```

```
((foo 3.3) ;; Это комментарий
 (bar "это () \" атом"))
```

можно загрузить с помощью `Sexplib`. Как видно в примере ниже, комментарии не входят в получающееся s-выражение:

OCaml utop

https://github.com/realworldocaml/examples/blob/v1/code/sexpr/example_load.topscript

```
# Sexp.load_sexp "example.scm" ;;
- : Sexp.t = ((foo 3.3) (bar "this is () an \" атом"))
```

В общем и целом формат s-выражений поддерживает три типа комментариев:

```
;
```

простирается до конца строки;

```
#|
|#
```

многострочный комментарий;

```
##;
```

простирается до конца первого законченного s-выражения.

Все эти комментарии демонстрируются в следующем примере:

Scheme

https://github.com/realworldocaml/examples/blob/v1/code/sexpr/comment_heavy.scm

```
;; comment_heavy_example.scm
((это выражение включается)
 ; (это комментарий
 (это останется))
```

```
#; (все это будет
    закомментировано (даже на нескольких строках.))
(и #| блочные комментарии #| могут быть вложенными |#
  и располагаться
  на нескольких строках))) |#
на этом все
))
```

И снова, если загрузить этот файл как s-выражение, все комментарии будут отброшены:

OCaml utop (part 1)

https://github.com/realworldocaml/examples/blob/v1/code/sexpr/example_load.topscript

```
# Sexp.load_sexp "comment_heavy.scm" ;;
- : Sexp.t = ((это выражение включается) (это останется) (на этом все))
```

Если внести ошибку в наше s-выражение из файла *example.scm* (опустить открывающую скобку перед *bar*) и сохранить его, например, в файле *broken_example.scm*, при попытке загрузить его будет выведено сообщение об ошибке:

OCaml utop (part 2)

https://github.com/realworldocaml/examples/blob/v1/code/sexpr/example_load.topscript

```
# Exn.handle_uncaught ~exit:false (fun () ->
  ignore (Sexp.load_sexp "example_broken.scm")) ;;
Uncaught exception:
(Sexplib.Sexp.Parse_error
  ((location parse) (err_msg "unexpected character: ' '") (text_line 4)
   (text_char 29) (global_offset 78) (buf_pos 78)))
- : unit = ()
```

В этом примере мы использовали функцию `Exn.handle_uncaught`, чтобы обеспечить вывод полной информации об исключении. Обычно все программы на основе библиотеки `Core` желательно заключать в вызов этого обработчика, чтобы получать максимально информативные сообщения о неожиданных исключениях.

Сохранение инвариантов

Наиболее важной функциональной особенностью, предоставляемой пакетом `Sexplib`, является автоматическое создание функций преобразования для новых типов. Мы уже видели, как действует этот механизм, тем не менее давайте рассмотрим еще один пример. Ниже приводится исходный программный код простой библиотеки, реализующей представление целочисленных интервалов, очень похожей на описывавшуюся в главе 9:

OCaml

https://github.com/realworldocaml/examples/blob/v1/code/sexpr/int_interval.ml

```
(* Модуль для представления закрытых целочисленных интервалов *)
open Core.Std

(* Инвариант: для любого Range (x,y), y >= x *)
```

```
type t =
  | Range of int * int
  | Empty
with sexp

let is_empty =
  function
  | Empty -> true
  | Range _ -> false

let create x y =
  if x > y then
    Empty
  else
    Range (x,y)

let contains i x =
  match i with
  | Empty -> false
  | Range (low,high) -> x >= low && x <= high
```

Далее показано, как можно использовать этот модуль:

OCaml

https://github.com/realworldocaml/examples/blob/v1/code/sexpr/test_interval.ml

```
open Core.Std

let intervals =
  let module I = Int_interval in
  [ I.create 3 4;
    I.create 5 4; (* должен получиться пустым *)
    I.create 2 3;
    I.create 1 6;
  ]

let () =
  intervals
  |> List.sexp_of_t Int_interval.sexp_of_t
  |> Sexp.to_string_hum
  |> print_endline
```

Но мы кое-что опустили: мы не создали сигнатуру *.mli* для `Int_interval`. Обратите внимание, что нам следует явно экспортировать функции преобразования s-выражений, созданные внутри файла *.ml*. Например, ниже представлен интерфейс, не экспортирующий функций:

OCaml

https://github.com/realworldocaml/examples/blob/v1/code/sexpr/int_interval_nosexp.mli

```
type t

val is_empty : t -> bool
val create : int -> int -> t
val contains : t -> int -> bool
```

В результате компилятор сообщит об ошибке:

Terminal

```
https://github.com/realworldocaml/examples/tree/v1/code/sexpr/
build_test_interval_nosexp.out
```

```
$ corebuild test_interval_nosexp.native
File "test_interval_nosexp.ml", line 14, characters 20-42:
Error: Unbound value Int_interval.sexp_of_t
Command exited with code 2.
(Ошибка: Несвязанное значение Int_interval.sexp_of_t
Команда завершилась с кодом 2.)
```

Мы могли бы экспортировать типы вручную, добавив определения сигнатур дополнительных функций, сгенерированных пакетом Sexplib:

OCaml

```
https://github.com/realworldocaml/examples/blob/v1/code/sexpr/
int_interval_manual_sexp.mli
open Core.Std
```

```
type t
val t_of_sexp : Sexp.t -> t
val sexp_of_t : t -> Sexp.t

val is_empty : t -> bool
val create : int -> int -> t
val contains : t -> int -> bool
```

Но это не самое лучшее решение, так как придется вручную добавлять сигнатуры всех дополнительных функций для всех значений, которые может понадобиться сериализовать. Пакет Sexplib решает эту проблему, предоставляя то же самое расширение синтаксиса для использования в определениях сигнатур внутри файлов *.mli*. Ниже приводится окончательная версия сигнатуры:

OCaml

```
https://github.com/realworldocaml/examples/blob/v1/code/sexpr/int_interval.mli
type t with sexp
```

```
val is_empty : t -> bool
val create : int -> int -> t
val contains : t -> int -> bool
```

Теперь снова скомпилируем файл *test_interval.ml* и запустим получившуюся программу. В результате мы получим следующее:

Terminal

```
https://github.com/realworldocaml/examples/tree/v1/code/sexpr/build_test_interval.out
$ corebuild test_interval.native
$ ./test_interval.native
((Range 3 4) Empty (Range 2 3) (Range 1 6))
```

Применяя функции преобразования `sexpr`, легко допустить ошибку, обусловленную тем, что они могут нарушать инвариантность кода. Например, проверка `is_empty` в модуле `Int_interval` опирается на тот факт, что для любого значения `Range (x,y)` значение `y` больше и равно значению `x`. Функция `create` обеспечивает соблюдение этой инвариантности, а функция `t_of_sexpr` – нет.

Эту проблему можно исправить, переопределив автоматически сгенерированную функцию и написав собственный `sexpr`-преобразователь, вызывающий автоматически сгенерированную функцию после всех необходимых проверок:

OCaml

https://github.com/realworldocaml/examples/tree/v1/code/sexpr/sexpr_override.ml

```
type t =
  | Range of int * int
  | Empty
with sexpr

let create x y =
  if x > y then Empty else Range (x,y)

let t_of_sexpr sexpr =
  let t = t_of_sexpr sexpr in
  begin match t with
  | Empty -> ()
  | Range (x,y) ->
      if y < x then of_sexpr_error "Upper and lower bound of Range swapped" sexpr
  end;
  t
```

Этот трюк переопределения существующей функции новой вполне допустим в языке OCaml. Так как функция `t_of_sexpr` определена как обычная привязка `let`, а не `let rec`, вызов `t_of_sexpr` обращается к версии, сгенерированной расширением `Sexplib`, а не рекурсивно к новой версии.

Другой важный аспект нашего определения – вызов функции `of_sexpr_error` для возбуждения исключения при неудаче, возникшей в процессе парсинга. Это повышает информативность сообщения об ошибке в сравнении с тем, что выводит пакет `Sexplib`, которое будет продемонстрировано в следующем разделе.

Вывод информативных сообщений об ошибках

Десериализация типа из `s`-выражения выполняется в два этапа: сначала байты из файла преобразуются в `s`-выражение, а затем `s`-выражение преобразуется в тип. Одна из проблем, сопутствующих этому процессу, заключается в сложности локализации ошибок. Взгляните на следующий пример:

OCaml

https://github.com/realworldocaml/examples/blob/v1/code/sexpr/read_foo.ml

```
open Core.Std

type t = {
  a: string;
```

```

b: int;
c: float option
} with sexp

let run () =
  let t =
    Sexp.load_sexp "foo_broken_example.scm"
    |> t_of_sexp
  in
  printf "b is: %d\n%" t.b

let () =
  Exn.handle_uncaught ~exit:true run

```

Если передать этой программе файл, содержащий ошибку, например такой, как показано ниже:

Scheme

https://github.com/realworldocaml/examples/blob/v1/code/sexpr/foo_broken_example.scm

```

((a "not-an-integer")
 (b "not-an-integer")
 (c 1.0))

```

она выведет:

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/sexpr/build_read_foo.out

```

$ corebuild read_foo.native
$ ./read_foo.native foo_example_broken.scm
Uncaught exception:

(Sexplib.Conv.Of_sexp_error
 (Failure "int_of_sexp: (Failure int_of_string)") not-an-integer)

Raised at file "lib/conv.ml", line 281, characters 36-72
Called from file "lib/core_int.ml", line 6, characters 7-14
Called from file "lib/std_internal.ml", line 115, characters 7-33
Called from file "read_foo.ml", line 5, characters 2-8
Called from file "read_foo.ml", line 4, characters 2-40
Called from file "read_foo.ml", line 11, characters 4-60
Called from file "lib/exn.ml", line 87, characters 6-10

```

Такое сообщение об ошибке несет мало полезной информации. В частности, из текста сообщения видно, что ошибка парсинга возникла в атоме «not-an-integer», но не понятно, в каком из них! Если подобная ошибка возникнет при парсинге большого файла, сообщение о ней вообще станет бесполезным.

Однако не отчаивайтесь! Лишь немного изменив код, мы можем существенно повысить информативность сообщений об ошибках:

OCaml

https://github.com/realworldocaml/examples/blob/v1/code/sexpr/read_foo_better_errors.ml

```
open Core.Std
```

```
type t = {
```

```

a: string;
b: int;
c: float option
} with sexp

let run () =
  let t = Sexp.load_sexp_conv_exn "foo_broken_example.scm" t_of_sexp in
  printf "b is: %d\n%" t.b

let () =
  Exn.handle_uncaught ~exit:true run

```

Если снова попробовать загрузить тот же файл, мы увидим более конкретное сообщение:

Terminal

```

https://github.com/realworldocaml/examples/tree/v1/code/sexpr/
build_read_foo_better_errors.out

```

```

$ corebuild read_foo_better_errors.native
$ ./read_foo_better_errors.native foo_example_broken.scm
Uncaught exception:

```

```

(Sexplib.Conv.Of_sexp_error
 (Sexplib.Sexp.Annotated.Conv_exn foo_broken_example.scm:2:5
  (Failure "int_of_sexp: (Failure int_of_string)"))
 not-an-integer)

```

```

Raised at file "lib/pre_sexp.ml", line 1145, characters 12-58
Called from file "read_foo_better_errors.ml", line 10, characters 10-68
Called from file "lib/exn.ml", line 87, characters 6-10

```

Здесь текст `foo_broken_example.scm:2:5` сообщает нам, что ошибка возникла в файле `foo_broken_example.scm`, в строке 2, в символе 5. Эта информация поможет быстро понять, в чем проблема. Возможность определять точное местоположение ошибки зависит от использования функции `of_sexp_error` в `sexp`-преобразователях. В функциях преобразования, сгенерированных пакетом `Sexplib`, это уже делается, но вы должны обеспечить то же самое в собственных функциях преобразования.

Директивы `sexp`-преобразований

Пакет `Sexplib` поддерживает коллекцию директив для изменения поведения по умолчанию функций преобразования, сгенерированных автоматически. Эти директивы позволяют определять способ представления типов в виде `s`-выражений без необходимости писать собственные функции преобразования.

Обратите внимание, что эти директивы не являются частью стандартного синтаксиса OCaml, а добавляются расширением синтаксиса `Sexplib`. Однако, так как расширение `Sexplib` широко используется в библиотеке `Core` и является частью стандартного пакета расширений, активируемого системой сборки `corebuild`, вы можете использовать их в своем коде, использующем библиотеку `Core`, без лишних усилий.

sexpr_opaque

Чаще других на практике используется директива `sexpr_opaque`, назначение которой – пометить неконвертируемые компоненты типов. Все, что помечено директивой `sexpr_opaque`, будет представлено в виде атома `<opaque>` функцией преобразования в s-выражение и вызывать исключение в функции преобразования из s-выражения.

Обратите внимание, что тип компонента, помеченного как неконвертируемый, может не иметь функций преобразования. В этом случае, если тип, который определен без `sexpr`-преобразователей, попытаться использовать в типе с `sexpr`-преобразователями, компилятор выведет сообщение об ошибке:

OCaml utop

https://github.com/realworldocaml/examples/blob/v1/code/sexpr/sexpr_opaque.topscript

```
# type no_converter = int * int ;;
type no_converter = int * int
# type t = { a: no_converter; b: string } with sexpr ;;
Characters 14-26:
Error: Unbound value no_converter_of_sexpr
(Ошибка: Несвязанное значение no_converter_of_sexpr)
```

Директива `sexpr_opaque` позволяет внедрить неконвертируемый тип `no_converter` в определение другого типа.

OCaml utop (part 1)

https://github.com/realworldocaml/examples/blob/v1/code/sexpr/sexpr_opaque.topscript

```
# type t = { a: no_converter sexpr_opaque; b: string } with sexpr ;;
type t = { a : no_converter; b : string; }
val t_of_sexpr : Sexp.t -> t = <fun>
val sexpr_of_t : t -> Sexp.t = <fun>
```

Если теперь попытаться преобразовать значение этого типа в s-выражение, можно увидеть, что содержимое поля помечено как неконвертируемое (или «непрозрачное» – `opaque`):

OCaml utop (part 2)

https://github.com/realworldocaml/examples/blob/v1/code/sexpr/sexpr_opaque.topscript

```
# sexpr_of_t { a = (3,4); b = "foo" } ;;
- : Sexp.t = ((a <opaque>) (b foo))
```

Обратите внимание, что функция `t_of_sexpr` для неконвертируемого типа все-таки генерируется, но при попытке вызвать ее возбуждает исключение:

OCaml utop (part 3)

https://github.com/realworldocaml/examples/blob/v1/code/sexpr/sexpr_opaque.topscript

```
# t_of_sexpr (Sexp.of_string "((a whatever) (b foo))") ;;
Exception:
(Sexplib.Conv.Of_sexpr_error
 (Failure "opaque_of_sexpr: cannot convert opaque values") whatever).
```


Сделано это с целью дать возможность определять функции преобразования для типов, включающих компоненты неконвертируемых типов. Очень удобно, потому что получающиеся функции преобразования необязательно будут терпеть неудачу на всех исходных данных. Например, представьте запись, содержащую `no_converter list`, тогда функция `t_of_sexp` будет завершаться успехом, если список пуст:

OCaml utop (part 4)

https://github.com/realworldocaml/examples/blob/v1/code/sexpr/sexp_opaque.topscript

```
# type t = { a : no_converter sexp_opaque list; b : string } with sexp ;;
type t = { a : no_converter list; b : string; }
val t_of_sexp : Sexp.t -> t = <fun>
val sexp_of_t : t -> Sexp.t = <fun>
# t_of_sexp (Sexp.of_string "((a ()) (b foo))") ;;
- : t = {a = []; b = "foo"}
```

Если для нормальной работы достаточно функции преобразования только в каком-то одном направлении, этого можно добиться, используя аннотацию `with sexp_of` или `with of_sexp` вместо `with sexp`:

OCaml utop (part 5)

https://github.com/realworldocaml/examples/blob/v1/code/sexpr/sexp_opaque.topscript

```
# type t = { a : no_converter sexp_opaque; b : string } with sexp_of ;;
type t = { a : no_converter; b : string; }
val sexp_of_t : t -> Sexp.t = <fun>
# type t = { a : no_converter sexp_opaque; b : string } with of_sexp ;;
type t = { a : no_converter; b : string; }
val t_of_sexp : Sexp.t -> t = <fun>
```

sexp_list

Иногда `sexp`-преобразователи имеют больше круглых скобок, чем хотелось бы. Взгляните, например, на следующий вариантный тип:

OCaml utop

https://github.com/realworldocaml/examples/tree/v1/code/sexpr/sexp_list.topscript

```
# type compatible_versions =
  | Specific of string list
  | All with sexp ;;
type compatible_versions = Specific of string list | All
val compatible_versions_of_sexp : Sexp.t -> compatible_versions = <fun>
val sexp_of_compatible_versions : compatible_versions -> Sexp.t = <fun>
# sexp_of_compatible_versions
  (Specific ["3.12.0"; "3.12.1"; "3.13.0"]) ;;
- : Sexp.t = (Specific (3.12.0 3.12.1 3.13.0))
```

Кто-то предпочел бы, чтобы синтаксис был меньше перегружен скобками, окружающими списки. Мы можем поменять `string list` в объявлении типа на `string sexp_list`, чтобы получить альтернативный синтаксис:

OCaml utop (part 1)

https://github.com/realworldocaml/examples/tree/v1/code/sexpr/sexp_list.topscript

```
# type compatible_versions =
  | Specific of string sexp_list
  | All with sexp ;;
type compatible_versions = Specific of string list | All
val compatible_versions_of_sexp : Sexp.t -> compatible_versions = <fun>
val sexp_of_compatible_versions : compatible_versions -> Sexp.t = <fun>
# sexp_of_compatible_versions
  (Specific ["3.12.0"; "3.12.1"; "3.13.0"]) ;;
- : Sexp.t = (Specific 3.12.0 3.12.1 3.13.0)
```

sexp_option

Также часто используется директива `sexp_option`, которая делает необязательными поля записей в s-выражениях. Обычно необязательные значения представляются либо как `()` (для `None`), либо как `(x)` (для `Some x`), и поле записи, содержащее необязательное значение, отображается соответственно. Например:

OCaml utop

https://github.com/realworldocaml/examples/tree/v1/code/sexpr/sexp_option.topscript

```
# type t = { a: int option; b: string } with sexp ;;
type t = { a : int option; b : string; }
val t_of_sexp : Sexp.t -> t = <fun>
val sexp_of_t : t -> Sexp.t = <fun>
# sexp_of_t { a = None; b = "hello" } ;;
- : Sexp.t = ((a ()) (b hello))
# sexp_of_t { a = Some 3; b = "hello" } ;;
- : Sexp.t = ((a (3)) (b hello))
```

Но что, если нам необходимо сделать необязательным само поле, то есть чтобы поле вообще можно было бы опустить? В этом случае можно воспользоваться директивой `sexp_option`:

OCaml utop (part 1)

https://github.com/realworldocaml/examples/tree/v1/code/sexpr/sexp_option.topscript

```
# type t = { a: int sexp_option; b: string } with sexp ;;
type t = { a : int option; b : string; }
val t_of_sexp : Sexp.t -> t = <fun>
val sexp_of_t : t -> Sexp.t = <fun>
# sexp_of_t { a = Some 3; b = "hello" } ;;
- : Sexp.t = ((a 3) (b hello))
# sexp_of_t { a = None; b = "hello" } ;;
- : Sexp.t = ((b hello))
```

Определение значений по умолчанию

Объявление `sexp_option` в действительности является лишь частным случаем определения поведения по умолчанию, имеющего отношение к обработке неуказанных полей. В частности, `sexp_option` заполняет отсутствующие поля значением `None`. Но иногда бывает желательно определить другие значения по умолчанию.

Взгляните на следующий тип, представляющий конфигурацию простого веб-сервера:

OCaml utop

https://github.com/realworldocaml/examples/tree/v1/code/sexpr/sexp_default.topscrip

```
# type http_server_config = {
  web_root: string;
  port: int;
  addr: string;
} with sexp ;;

type http_server_config = { web_root : string; port : int; addr : string; }
val http_server_config_of_sexp : Sexp.t -> http_server_config = <fun>
val sexp_of_http_server_config : http_server_config -> Sexp.t = <fun>
```

У кого-то может возникнуть вполне естественное желание назначить некоторым из этих параметров значения по умолчанию. Например, параметру `port` можно назначить значение по умолчанию 80, а параметру `addr` – значение `localhost`. Реализовать это можно следующим способом:

OCaml utop (part 1)

https://github.com/realworldocaml/examples/tree/v1/code/sexpr/sexp_default.topscrip

```
# type http_server_config = {
  web_root: string;
  port: int with default(80);
  addr: string with default("localhost");
} with sexp ;;

type http_server_config = { web_root : string; port : int; addr : string; }
val http_server_config_of_sexp : Sexp.t -> http_server_config = <fun>
val sexp_of_http_server_config : http_server_config -> Sexp.t = <fun>
```

Если теперь попробовать преобразовать `s`-выражение, содержащее только значение `web_root`, можно увидеть, что остальные параметры получили желаемые значения по умолчанию:

OCaml utop (part 2)

https://github.com/realworldocaml/examples/tree/v1/code/sexpr/sexp_default.topscrip

```
# let cfg = http_server_config_of_sexp
  (Sexp.of_string "((web_root /var/www/html))") ;;
val cfg : http_server_config =
  {web_root = "/var/www/html"; port = 80; addr = "localhost"}
```

Если преобразовать конфигурацию обратно в `s`-выражение, можно заметить, что в нем присутствуют все поля, даже при том, что в этом нет необходимости:

OCaml utop (part 3)

https://github.com/realworldocaml/examples/tree/v1/code/sexpr/sexp_default.topscrip

```
# sexp_of_http_server_config cfg ;;
- : Sexp.t = ((web_root /var/www/html) (port 80) (addr localhost))
```

Избежать экспортирования значений по умолчанию можно с помощью директивы `sexp_drop_default`:

OCaml utop (part 4)

https://github.com/realworldocaml/examples/tree/v1/code/sexpr/sexp_default.topscript

```
# type http_server_config = {
  web_root: string;
  port: int with default(80), sexp_drop_default;
  addr: string with default("localhost"), sexp_drop_default;
} with sexp ;;

type http_server_config = { web_root : string; port : int; addr : string; }
val http_server_config_of_sexp : Sexp.t -> http_server_config = <fun>
val sexp_of_http_server_config : http_server_config -> Sexp.t = <fun>
# let cfg = http_server_config_of_sexp
  (Sexp.of_string "((web_root /var/www/html))") ;;
val cfg : http_server_config =
  {web_root = "/var/www/html"; port = 80; addr = "localhost"}
# sexp_of_http_server_config cfg ;;
- : Sexp.t = ((web_root /var/www/html))
```

Как видите, поля, имеющие значения по умолчанию, просто не попали в s-выражение. С другой стороны, если попробовать преобразовать конфигурацию с другими значениями параметров, эти значения будут включены в s-выражение:

OCaml utop (part 5)

https://github.com/realworldocaml/examples/tree/v1/code/sexpr/sexp_default.topscript

```
# sexp_of_http_server_config { cfg with port = 8080 } ;;
- : Sexp.t = ((web_root /var/www/html) (port 8080))
# sexp_of_http_server_config
  { cfg with port = 8080; addr = "192.168.0.1" } ;;
- : Sexp.t = ((web_root /var/www/html) (port 8080) (addr 192.168.0.1))
```

Это может пригодиться при проектировании форматов конфигурационных файлов — компактных и простых в создании и сопровождении. Кроме того, данная возможность может пригодиться для обеспечения обратной совместимости: если в запись `config` добавить новое поле и сделать его необязательным, тогда вы сможете использовать старые конфигурационные файлы.

Конкурентное программирование с помощью Async

Логика работы программ, взаимодействующих с внешним миром, часто предполагает ожидание: ожидание щелчка мышью, ожидание завершения операции чтения данных с диска или ожидание освобождения места в выходном сетевом буфере. Даже в не самых сложных интерактивных приложениях с успехом можно использовать приемы конкурентного программирования, например для ожидания наступления нескольких событий или немедленной реакции на первое наступившее событие.

Один из подходов к организации конкурентного выполнения заключается в использовании системных потоков выполнения. Данный подход является доминирующим в таких языках программирования, как Java или C#. В этой модели для каждой задачи, которой может потребоваться приостановиться в ожидании некоторого события, выделяется отдельный поток выполнения, который можно заблокировать, не останавливая работу самой программы.

Другой подход применяется в однопоточных программах и заключается в использовании *цикла событий*, в рамках которого реализуется реакция на внешние события, такие как тайм-ауты или щелчки мышью, путем вызова функций, специально зарегистрированных для этого. Этот подход часто используется в языках программирования, подобных языку JavaScript, имеющему однопоточную среду выполнения, а также во многих библиотеках поддержки графического интерфейса пользователя.

Каждый из данных механизмов имеет собственные достоинства и недостатки. Система потоков требует значительного объема памяти и других системных ресурсов. Кроме того, операционная система может произвольно прерывать выполнение одного потока, чтобы передать управление другому, что требует от программиста проявлять особую осторожность и защищать совместно используемые ресурсы с применением блокировок и условных переменных, вследствие чего легко допустить ошибку.

Однопоточные системы, управляемые событиями, с другой стороны, в каждый момент времени решают только одну задачу и не требуют применения сложных

механизмов синхронизации. Однако перевернутая организация программ, управляемых событиями, часто приводит к тому, что вам нередко приходится прибегать к весьма неуклюжим уловкам, чтобы обеспечить некоторое подобие конкурентного выполнения в цикле событий, что может завести в труднопроходимый лабиринт разнообразных ситуаций обработки событий.

В этой главе мы познакомимся с библиотекой Async, предлагающей гибридную модель, цель которой состоит в том, чтобы дать вам все самое лучшее из двух миров, избежать потери производительности и проблем синхронизации, свойственных многопоточной модели выполнения, и утраты контроля над выполнением, так характерной для систем, управляемых событиями.

Основы Async

Давайте вспомним, как обычно реализуются операции ввода/вывода в библиотеке Core:

OCaml utop (part 1)

<https://github.com/realworldocaml/examples/tree/v1/code/async/main.topscript>

```
# In_channel.read_all;;
- : string -> string = <fun>
# Out_channel.write_all "test.txt" ~data:"This is only a test.";;
- : unit = ()
# In_channel.read_all "test.txt";;
- : string = "This is only a test."
```

Взглянув на тип функции `In_channel.read_all`, можно заключить, что она выполняет блокирующую операцию. В частности, тот факт, что она возвращает конкретную строку, означает, что она не может вернуть управление, пока не закончит чтения. Проще говоря, выполнение программы не может быть продолжено, пока функция не завершит чтения.

Добропорядочные функции из библиотеки Async никогда не блокируют выполнения. Вместо этого они возвращают значение типа `Deferred.t`, действующее как контейнер, который когда-нибудь будет заполнен результатами. В качестве примера рассмотрим сигнатуру эквивалента функции `In_channel.read_all` из библиотеки Async:

OCaml utop (part 3)

<https://github.com/realworldocaml/examples/tree/v1/code/async/main.topscript>

```
# #require "async";;
# open Async.Std;;
# Reader.file_contents;;
- : string -> string Deferred.t = <fun>
```

Сначала мы загружаем пакет Async директивой `#require`, а затем открываем модуль `Async.Std`, который добавляет ряд новых идентификаторов и модулей в окружение, делая использование возможностей Async более удобным. В программах, использующих возможности Async, общепринято открывать модуль `Async.Std`,

подобно тому, как общепринято открывать модуль `Core.Std` при использовании библиотеки `Core`.

Отложенный результат (`deferred`), по сути, является дескриптором для доступа к значению, которое может быть вычислено когда-то в будущем. Поэтому сразу после вызова `Reader.file_contents` отложенный результат будет содержать пустое значение, в чем можно убедиться, вызвав метод `Deferred.peek` отложенного результата:

OCaml utop (part 4)

<https://github.com/realworldocaml/examples/blob/v1/code/async/main.topscript>

```
# let contents = Reader.file_contents "test.txt";;
val contents : string Deferred.t = <abstr>
# Deferred.peek contents;;
- : string option = None
```

Значение `contents` в данный момент еще не определено, отчасти потому, что пока не были выполнены необходимые операции ввода/вывода. При использовании библиотеки `Async` обработка ввода/вывода и других событий выполняется планировщиком `Async`. В автономных программах планировщик необходимо запускать явно, но *интерактивная оболочка* знает о существовании планировщика и запускает его автоматически. Кроме того, *интерактивная оболочка* знает, как обращаться с отложенными результатами, и когда вы вводите выражение с типом `Deferred.t`, она убедится, что планировщик запущен, и заблокирует свое выполнение до момента, пока результат не будет получен. То есть можно записать:

OCaml utop (part 5)

<https://github.com/realworldocaml/examples/blob/v1/code/async/main.topscript>

```
# contents;;
- : string = "This is only a test."
```

Если теперь снова вызвать метод `peek`, можно будет убедиться, что значение `contents` определено:

OCaml utop (part 6)

<https://github.com/realworldocaml/examples/blob/v1/code/async/main.topscript>

```
# Deferred.peek contents;;
- : string option = Some "This is only a test."
```

Чтобы включить отложенные результаты в работу, необходима возможность организовать ожидание завершения отложенных вычислений. Такую возможность дает функция `Deferred.bind`. Давайте сначала познакомимся с сигнатурой этой функции:

OCaml utop (part 7)

<https://github.com/realworldocaml/examples/blob/v1/code/async/main.topscript>

```
# Deferred.bind ;;
- : 'a Deferred.t -> ('a -> 'b Deferred.t) -> 'b Deferred.t = <fun>
```

`Deferred.bind d f` принимает значение отложенного результата `d` и функцию `f`, которая должна быть вызвана, как только значение `d` будет определено. Функцию `Deferred.bind` можно рассматривать как разновидность оператора составления последовательности операций. По сути, мы берем асинхронную операцию `d` и связываем ее со следующей стадией обработки – функцией `f`.

Если говорить более конкретно, вызов `Deferred.bind` возвращает новый отложенный результат, который получает определенное значение после возврата из функции `f`. Она также неявно регистрирует новое *задание* для планировщика `Async`, ответственное за вызов `f`, когда `d` получит определенное значение.

Ниже приводится простой пример использования функции `bind` для подключения другой функции, которая преобразует в верхний регистр все символы, содержащиеся в файле:

OCaml utop (part 8)

<https://github.com/realworldocaml/examples/blob/v1/code/async/main.topscript>

```
# let uppercase_file filename =
  Deferred.bind (Reader.file_contents filename)
    (fun text ->
      Writer.save filename ~contents:(String.uppercase text))
;;
val uppercase_file : string -> unit Deferred.t = <fun>
# uppercase_file "test.txt";;
- : unit = ()
# Reader.file_contents "test.txt";;
- : string = "THIS IS ONLY A TEST."
```

Непосредственное использование функции `Deferred.bind` может показаться утомительным, поэтому в состав `Async.Std` был включен инфиксный оператор `>>=`. Используя этот оператор, функцию `uppercase_file` можно записать иначе:

OCaml utop (part 9)

<https://github.com/realworldocaml/examples/blob/v1/code/async/main.topscript>

```
# let uppercase_file filename =
  Reader.file_contents filename
  >>= fun text ->
    Writer.save filename ~contents:(String.uppercase text)
;;
val uppercase_file : string -> unit Deferred.t = <fun>
```

В этом фрагменте мы отбросили круглые скобки, окружающие функцию справа, и не стали добавлять отступ для тела этой функции. Это – стандартная практика при использовании оператора `bind`.

Теперь рассмотрим еще один пример использования `bind`. В данном случае напишем функцию, подсчитывающую количество строк в файле:

OCaml utop (part 10)

<https://github.com/realworldocaml/examples/blob/v1/code/async/main.topscript>

```
# let count_lines filename =
  Reader.file_contents filename
```



```
>>= fun text ->
  List.length (String.split text ~on:'\n')
;;
```

Characters 85-125:

Error: This expression has type int but an expression was expected of type
'a Deferred.t

(Ошибка: Это выражение имеет тип int, тогда как ожидалось выражение типа
'a Deferred.t)

Код выглядит достаточно безупречно, но, как видите, компилятору он не понравился. Проблема здесь в том, что `bind` ожидает получить функцию, возвращающую отложенный результат, а мы передали функцию, возвращающую конкретное значение. Чтобы обеспечить соответствие сигнатуры предъявляемым требованиям, нам необходима функция, которая заворачивала бы конкретное значение в отложенный результат. Такая функция имеется в пакете `Async` и называется `return`:

OCaml utop (part 11)

<https://github.com/realworldocaml/examples/blob/v1/code/async/main.topscript>

```
# return;;
- : 'a -> 'a Deferred.t = <fun>
# let three = return 3;;
val three : int Deferred.t = <abstr>
# three;;
- : int = 3
```

Применив функцию `return`, мы можем благополучно скомпилировать `count_lines`:

OCaml utop (part 12)

<https://github.com/realworldocaml/examples/blob/v1/code/async/main.topscript>

```
# let count_lines filename =
  Reader.file_contents filename
  >>= fun text ->
    return (List.length (String.split text ~on:'\n'))
;;
val count_lines : string -> int Deferred.t = <fun>
```

Пара `bind` и `return` образует типичный шаблон функционального программирования, известный как *монада* (*monad*). Его можно встретить во многих приложениях, даже в тех, где не используются параллельные вычисления. В действительности мы уже сталкивались с монадами в разделе «Функция `bind` и другие идиомы обработки ошибок» в главе 7.

Прием использования `bind` и `return` в паре настолько распространен, что для него была определена сокращенная версия — функция `Deferred.map`, имеющая следующую сигнатуру:

OCaml utop (part 13)

<https://github.com/realworldocaml/examples/blob/v1/code/async/main.topscript>

```
# Deferred.map;;
- : 'a Deferred.t -> f:('a -> 'b) -> 'b Deferred.t = <fun>
```

а также отдельный инфиксный оператор `>>|`. С его помощью можно еще более кратко записать функцию `count_lines`:

OCaml utop (part 14)

<https://github.com/realworldocaml/examples/blob/v1/code/async/main.topscript>

```
# let count_lines filename =
  Reader.file_contents filename
  >>| fun text ->
    List.length (String.split text ~on:'\n')
;;
val count_lines : string -> int Deferred.t = <fun>
# count_lines "/etc/hosts";;
- : int = 12
```

Имейте в виду, что `count_lines` возвращает отложенный результат, а интерактивная оболочка ожидает, пока он не получит определенного значения, и выводит его.

Ivar и upon

Отложенные вычисления часто реализуются с применением комбинаторов `bind`, `map` и `return`, но иногда бывает желательно реализовать отложенные вычисления, явно выполняемые в пользовательском коде. Делается это с помощью модуля `Ivar`. (Название `Ivar` корнями уходит в язык Concurrent ML, разработанный Джоном Реппи (John Reppy) в начале 90-х. Буква «i» в названии *ivar* означает *incremental* (пошаговый).)

Существуют три фундаментальные операции для работы с пошаговыми вычислениями `ivar`: создание с использованием `Ivar.create`; прочитать отложенный результат, соответствующий пошаговым вычислениям, можно с помощью `Ivar.read`; а записать конкретное значение и тем самым сделать определенным соответствующий отложенный результат — с помощью `Ivar.fill`. Следующий пример иллюстрирует эти операции:

OCaml utop (part 15)

<https://github.com/realworldocaml/examples/blob/v1/code/async/main.topscript>

```
# let ivar = Ivar.create ();;
val ivar : '_a Ivar.t = <abstr>
# let def = Ivar.read ivar;;
val def : '_a Deferred.t = <abstr>
# Deferred.peek def;;
- : '_a option = None
# Ivar.fill ivar "Hello";;
- : unit = ()
# Deferred.peek def;;
- : string option = Some "Hello"
```

Иногда механизм пошаговых вычислений оказывается слишком низкоуровневым. Операторы, такие как `map`, `bind` и `return`, обычно проще в обращении. Однако пошаговые вычисления могут пригодиться, когда возникнет потребность реализовать какой-нибудь шаблон синхронизации, пока не имеющий хорошей поддержки.

Например, представьте, что необходимо запланировать последовательность операций, выполняемых с фиксированной задержкой, и при этом нужны гарантии, что эти операции будут выполняться в том же порядке, в каком были переданы планировщику. Эту идею демонстрирует следующая сигнатура:

OCaml utop (part 16)

<https://github.com/realworldocaml/examples/blob/v1/code/async/main.topscript>

```
# module type Delayer_intf = sig
  type t
  val create : Time.Span.t -> t
  val schedule : t -> (unit -> 'a Deferred.t) -> 'a Deferred.t
end;;

module type Delayer_intf =
sig
  type t
  val create : Core.Span.t -> t
  val schedule : t -> (unit -> 'a Deferred.t) -> 'a Deferred.t
end
```

Операции обрабатываются функцией `schedule` в форме переходников (think, то есть функций, принимающих единственный аргумент типа `unit`). Вызывающему коду возвращается отложенный результат, который в конечном итоге будет заполнен содержимым отложенного результата, возвращаемого функцией-переходником. Для реализации этого подхода мы будем использовать оператор `upon`, имеющий следующую сигнатуру:

OCaml utop (part 17)

<https://github.com/realworldocaml/examples/blob/v1/code/async/main.topscript>

```
# upon;;
- : 'a Deferred.t -> ('a -> unit) -> unit = <fun>
```

Подобно операторам `bind` и `return`, `upon` вызовет указанную функцию, когда получит отложенный результат с конкретным значением; но, в отличие от них, `upon` не создает нового отложенного результата.

Наша реализация выполнения операций с задержкой основана на очереди функций-переходников, в которой каждый вызов `schedule` добавляет указанную функцию в очередь и создает отложенное задание, которое извлечет функцию из очереди и выполнит ее. Сама задержка будет реализована с помощью функции `after`, принимающей интервал времени и возвращающей отложенный результат, который становится определенным через указанный интервал времени:

OCaml utop (part 18)

<https://github.com/realworldocaml/examples/blob/v1/code/async/main.topscript>

```
# module Delayer : Delayer_intf = struct
  type t = { delay: Time.Span.t;
             jobs: (unit -> unit) Queue.t;
           }

  let create delay =
```

```

    { delay; jobs = Queue.create () }

let schedule t thunk =
  let ivar = Ivar.create () in
  Queue.enqueue t.jobs (fun () ->
    upon (thunk ()) (fun x -> Ivar.fill ivar x));
  upon (after t.delay) (fun () ->
    let job = Queue.dequeue_exn t.jobs in
    job ());
  Ivar.read ivar
end;;

module Delayer : Delayer_intf

```

Код получился не особенно длинным, но достаточно сложным. В частности, обратите внимание, как применение очереди гарантирует определенную последовательность выполнения операций, даже при том что функции, запланированные оператором `upon`, вызываются без соблюдения этого порядка. Такого рода тонкости характерны для кода, использующего пошаговые вычисления, и вследствие этого часто бывает предпочтительнее применение более простого стиля на основе `map/bind/return`.

Примеры: эхо-сервер

Теперь, разобравшись с основами использования пакета `Async`, напомним небольшую автономную программу – эхо-сервер, то есть программу, принимающую соединения от клиентов и возвращающую обратно все, что будет ей передано.

Для начала напомним функцию, которая читает входные данные и выводит их обратно. Для этого воспользуемся модулями `Reader` и `Writer` из пакета `Async`, реализующими удобные абстракции для работы с каналами ввода/вывода:

OCaml

<https://github.com/realworldocaml/examples/blob/v1/code/async/echo.ml>

```

open Core.Std
open Async.Std

(* Копирует данные из канала чтения в канал записи, используя
   указанный промежуточный буфер *)
let rec copy_blocks buffer r w =
  Reader.read r buffer
  >>= function
  | `Eof -> return ()
  | `Ok bytes_read ->
    Writer.write w buffer ~len:bytes_read;
    Writer.flushed w
  >>= fun () ->
    copy_blocks buffer r w

```

Для последовательного выполнения действий в этом коде используется оператор `bind`: сначала вызовом `Reader.read` извлекается блок входных данных. Когда операция чтения завершится и вернет новый блок, выполняется его запись с по-

мощью `Writer.write`. В заключение вызовом функции `Writer.flushed`, возвращающей отложенный результат, выполняется задержка до момента, пока буфер с записанными данными не будет вытолкнут, после чего осуществляется рекурсивный вызов. Если обнаружится признак конца файла, цикл прекращается. Отложенный результат, возвращаемый функцией `copy_blocks`, становится определенным только после обнаружения признака конца файла.

Особо обратите внимание, что здесь используется *эффект блокировки*, то есть если запись остановится, следующая операция чтения не начнется. Для серверов это очень важно, потому что в противном случае прекращение приема данных на стороне клиента может вызвать утечку памяти на сервере из-за необходимости выделять все новые блоки памяти для читаемых данных, которые не могут быть отправлены обратно.

Кому-то может показаться, что цепочка отложенных результатов, получающаяся в ходе выполнения цикла, также может привести к утечке памяти. В конце концов, этот код строит постоянно растущую цепочку из привязок, каждая из которых создает отложенный результат. Однако в данном случае все отложенные результаты станут определенными, только когда последний отложенный результат получит определенное значение, то есть когда будет встречен признак конца файла. Поэтому все эти отложенные результаты можно заменить единственным отложенным результатом. Пакет `Async` предусматривает такую логику выполнения и тем самым препятствует появлению утечек памяти. По сути, это разновидность оптимизации хвостовой рекурсии.

Функция `copy_blocks` обеспечивает логику обработки соединения с клиентом, но нам необходим еще сервер, который будет принимать эти соединения и вызывать `copy_blocks`. Для реализации сервера мы воспользуемся модулем `Tcp` из пакета `Async`, содержащим коллекцию утилит для создания клиентов и серверов, действующих по протоколу TCP:

OCaml (part 1)

<https://github.com/realworldocaml/examples/blob/v1/code/async/echo.ml>

```
(** Запускает TCP-сервер, прослушивающий указанный порт и вызывающий
    copy_blocks для каждого клиента. *)
let run () =
  let host_and_port =
    Tcp.Server.create
      ~on_handler_error:`Raise
      (Tcp.on_port 8765)
      (fun _addr r w ->
        let buffer = String.create (16 * 1024) in
        copy_blocks buffer r w)
  in
  ignore (host_and_port : (Socket.Address.Inet.t, int) Tcp.Server.t Deferred.t)
```

Функция `Tcp.Server.create` возвращает значение типа `Tcp.Server.t` — дескриптор сервера, с помощью которого можно остановить действующий сервер. В этом примере мы не используем данную возможность, поэтому явно игнорируем воз-

возвращаемое значение, чтобы подавить вывод сообщения о неиспользуемой переменной. Мы также заключили игнорируемое значение в аннотацию типа, чтобы сделать природу игнорируемого значения более явной.

Последний аргумент функции `Tcp.Server.create` является наиболее важным – это дескриптор соединения с клиентом. Отметим, что в этом примере отсутствует явная операция, которая закрывала бы соединение с клиентом. Это объясняется тем, что сервер закроет соединение автоматически, как только обработчик вернет определенное значение в отложенном результате.

В заключение нам нужно инициировать сервер и запустить планировщик Async:

OCaml (part 2)

<https://github.com/realworldocaml/examples/blob/v1/code/async/echo.ml>

```
(* Вызывает [run] и затем запускает планировщик *)
let () =
  run ();
  never_returns (Scheduler.go ())
```

Часто те, кто только начинает осваивать Async, допускают одну распространенную ошибку – они забывают запускать планировщик. Эта ошибка может вызывать недоумение, потому что без планировщика ваша программа вообще ничего не будет делать; даже банальный вызов `printf` ничего не будет выводить в окно терминала.

Следует также отметить, что даже при том, что мы не затратили никаких усилий на реализацию поддержки нескольких клиентов, этот сервер способен одновременно обрабатывать множество клиентов без каких-либо дополнений.

Теперь, когда у нас появился эхо-сервер, можно попробовать подключиться к нему с помощью утилиты *netcat*, которая вызывается командой `nc`:

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/async/run_echo.out

```
$ ./echo.native &
$ nc 127.0.0.1 8765
This is an echo server
This is an echo server
It repeats whatever I write.
It repeats whatever I write.
```

Функции, никогда не возвращающие управления

Кого-то может заинтересовать, что происходит при вызове `never_returns`. Идиома `never_returns` пришла из библиотеки *Core*, где используется для обозначения функций, которые никогда не возвращают управления. Обычно тип возвращаемого значения такой функции определяется как `'a`:

OCaml utop (part 19)

<https://github.com/realworldocaml/examples/blob/v1/code/async/main.topscript>

```
# let rec loop_forever () = loop_forever ();;
val loop_forever : unit -> 'a = <fun>
# let always_fail () = assert false;;
val always_fail : unit -> 'a = <fun>
```

Может показаться странным, что такую функцию можно вызывать там, где ожидается возвращаемое значение типа `unit`. Механизм контроля типов необязательно будет сообщать об ошибке в подобных ситуациях:

OCaml utoп (part 20)

<https://github.com/realworldocaml/examples/blob/v1/code/async/main.topscript>

```
# let do_stuff n =
  let x = 3 in
  if n > 0 then loop_forever ();
  x + n
;;
val do_stuff : int -> int = <fun>
```

Имя `loop_forever` говорит само за себя. Но в других случаях, как, например, с функцией `Scheduler.go`, тот факт, что она никогда не возвращает управления, очевиден, поэтому мы используем систему типов, чтобы сделать это обстоятельство более явным, обозначая тип возвращаемого значения как `never_returns`. Попробуем применить тот же трюк к `loop_forever`:

OCaml utoп (part 21)

<https://github.com/realworldocaml/examples/blob/v1/code/async/main.topscript>

```
# let rec loop_forever () : never_returns = loop_forever ();;
val loop_forever : unit -> never_returns = <fun>
```

Тип `never_returns` является пустым, поэтому функция не может вернуть значение типа `never_returns`, а это означает, что тип `never_returns` возвращаемого значения может иметь только функция, никогда не возвращающая управления! Если теперь переписать нашу функцию `do_stuff`, мы получим ожидаемое сообщение об ошибке:

OCaml utoп (part 22)

<https://github.com/realworldocaml/examples/blob/v1/code/async/main.topscript>

```
# let do_stuff n =
  let x = 3 in
  if n > 0 then loop_forever ();
  x + n
;;
```

Characters 38-67:

```
Error: This expression has type unit but an expression was expected of type
      never_returns
```

(Ошибка: Это выражение имеет тип `unit`, тогда как ожидалось выражение типа `never_returns`)

Исправить эту ошибку можно вызовом функции `never_returns`:

OCaml utoп (part 23)

<https://github.com/realworldocaml/examples/blob/v1/code/async/main.topscript>

```
# never_returns;;
- : never_returns -> 'a = <fun>
# let do_stuff n =
  let x = 3 in
  if n > 0 then never_returns (loop_forever ());
  x + n
;;
val do_stuff : int -> int = <fun>
```

Мы обеспечили успешную компиляцию, явно указав в исходном коде, что функция `loop_forever` никогда не возвращает управления.

Усовершенствование эхо-сервера

Давайте попробуем пойти немного дальше и добавим несколько усовершенствований в наш эхо-сервер. В частности:

- добавим интерфейс командной строки с помощью модуля `Command`;
- добавим флаг, с помощью которого можно будет указывать номер прослушиваемого порта, при подключении к которому клиент будет получать обратно текст, состоящий только из символов верхнего регистра;
- упростим код, задействовав интерфейс `Pipe` из пакета `Async`.

Все это реализовано в следующем коде:

OCaml

https://github.com/realworldocaml/examples/blob/v1/code/async/better_echo.ml

```
open Core.Std
open Async.Std

let run ~uppercase ~port =
  let host_and_port =
    Tcp.Server.create
      ~on_handler_error:`Raise
      (Tcp.on_port port)
      (fun _addr r w ->
        Pipe.transfer (Reader.pipe r) (Writer.pipe w)
          ~f:(if uppercase then String.uppercase else Fn.id))
  in
  ignore (host_and_port : (Socket.Address.Inet.t, int) Tcp.Server.t Deferred.t);
  Deferred.never ()

let () =
  Command.async_basic
    ~summary:"Start an echo server"
    Command.Spec.(
      empty
      +> flag "-uppercase" no_arg
        ~doc:" Convert to uppercase before echoing back"
      +> flag "-port" (optional_with_default 8765 int)
        ~doc:" Port to listen on (default 8765)"
    )
    (fun uppercase port () -> run ~uppercase ~port)
  |> Command.run
```

Обратите внимание на вызов `Deferred.never` в функции `run`. Как можно догадаться по имени, `Deferred.never` возвращает отложенный результат, который никогда не получит определенного значения. В данном случае это служит признаком, что эхо-сервер никогда не завершится.

Самое значительное изменение, по сравнению с предыдущей версией, — использование `Pipe`. `Pipe` — это асинхронный канал обмена данными, применяемый для соединения различных частей программы. Его можно рассматривать как очередь типа поставщик/потребитель, в которой готовность к записи или чтению определяется по отложенным результатам. В нашем сервере каналы нашли довольно

узкое применение, но вообще они являются важной частью пакета Async, поэтому есть смысл обсудить их подробнее.

Каналы создаются в виде пар дескрипторов для чтения и записи:

OCaml utop (part 24)

<https://github.com/realworldocaml/examples/blob/v1/code/async/main.topscript>

```
# let (r,w) = Pipe.create ();;
val r : '_a Pipe.Reader.t = <abstr>
val w : '_a Pipe.Writer.t = <abstr>
```

`r` и `w` — это дескрипторы одного и того же объекта. Обратите внимание, что `r` и `w` имеют слабополиморфные типы, о которых рассказывается в разделе «Побочные эффекты и слабый полиморфизм» в главе 8, и поэтому могут хранить только значения единственного простого типа.

Если попытаться выполнить запись в интерактивной оболочке, она просто заблокируется. Вы можете прервать ожидание, нажав комбинацию **Control-C**:

OCaml utop

https://github.com/realworldocaml/examples/blob/v1/code/async/pipe_write_break.rawscript

```
# Pipe.write w "Hello World!";;
Interrupted.
```

Отложенный результат, возвращаемый функцией записи, получит определенное значение, как только значение, записанное в канал, будет прочитано:

OCaml utop (part 25)

<https://github.com/realworldocaml/examples/blob/v1/code/async/main.topscript>

```
# let (r,w) = Pipe.create ();;
val r : '_a Pipe.Reader.t = <abstr>
val w : '_a Pipe.Writer.t = <abstr>
# let write_complete = Pipe.write w "Hello World!";;
val write_complete : unit Deferred.t = <abstr>
# Pipe.read r;;
- : [ `Eof | `Ok of string ] = `Ok "Hello World!"
# write_complete;;
- : unit = ()
```

В функции `run` мы используем одну из множества вспомогательных функций из модуля `Pipe`. В частности, мы используем `Pipe.transfer` для организации чтения данных из канала и записи их в канал. Вот как выглядит сигнатура `Pipe.transfer`:

OCaml utop (part 26)

<https://github.com/realworldocaml/examples/blob/v1/code/async/main.topscript>

```
# Pipe.transfer;;
- : 'a Pipe.Reader.t -> 'b Pipe.Writer.t -> f:('a -> 'b) -> unit Deferred.t =
<fun>
```

Два подключенных канала генерируются вызовами `Reader.pipe` и `Writer.pipe` соответственно. Обратите внимание, что здесь также возникает эффект блокировки,

то есть если пишущий конец окажется заблокирован, чтение автоматически будет остановлено, что не позволит прочитать дополнительные данные.

Важно также отметить, что отложенный результат, возвращаемый функцией `Pipe.transfer`, получает определенное значение, как только будет прочитан последний элемент данных из входного канала и передан для записи в выходной канал. Сразу после этого сервер закроет соединение с клиентом. Соответственно, когда клиент будет отключен, остальные завершающие действия будут выполнены автоматически.

Парсинг командной строки для этой программы реализован на основе библиотеки `Command`, представленной в главе 14. Открытие `Async.Std` затеняет модуль `Command` расширенной версией, содержащей функцию `async_basic`:

OCaml utop (part 27)

<https://github.com/realworldocaml/examples/blob/v1/code/async/main.topscript>

```
# Command.async_basic;;
- : summary:string ->
  ?readme:(unit -> string) ->
  ('a, unit -> unit Deferred.t) Command.Spec.t -> 'a -> Command.t
= <fun>
```

Она отличается от обычной функции `Command.basic` тем, что требует от главной функции вернуть значение типа `Deferred.t`, и попытка выполнить команду (с помощью `Command.run`) автоматически запускает планировщик `Async`, избавляя от необходимости явно вызывать `Scheduler.go`.

Пример: поиск определений с помощью DuckDuckGo

DuckDuckGo – это поисковая система со свободно доступным интерфейсом. В этом разделе мы попробуем использовать пакет `Async` для создания небольшой утилиты командной строки, которая будет посылать запросы системе DuckDuckGo и извлекать результаты.

В своем программном коде мы задействуем еще несколько других библиотек, каждую из которых можно установить с помощью OPAM. За справочной информацией по установке обращайтесь по адресу: <https://github.com/realworldocaml/book/wiki/Installation-Instructions>. Ниже перечислены библиотеки, которые нам потребуются:

- `textwrap` – библиотека, реализующая перенос длинных строк. Она будет применяться для вывода результатов;
- `uri` – библиотека для обработки URI (Uniform Resource Identifiers – универсальные идентификаторы ресурсов), примерами которых могут служить HTTP;
- `yojson` – библиотека для парсинга JSON, описанная в главе 15;
- `cohttp` – библиотека для создания клиентов и серверов HTTP. Нам потребуется поддержка `Async`, реализованная в виде пакета `cohttp.async`.

А теперь приступим к реализации.

Обработка URI

Адреса HTTP URL, идентифицирующие конечные точки в Сети, в действительности являются частью более обширного семейства, известного как Uniform Resource Identifiers (URI). Полное описание спецификации URI приводится в документе RFC 3986¹ и содержит массу технических тонкостей, разобраться в которых очень непросто. К счастью, имеется библиотека `uri`, предоставляющая строго типизированный интерфейс, решающий большую часть проблем.

Нам потребуется функция, генерирующая адреса URI, которые будут использоваться для обращения к поисковой системе DuckDuckGo:

OCaml

<https://github.com/realworldocaml/examples/blob/v1/code/async/search.ml>

```
open Core.Std
open Async.Std
```

```
(* Генерирует поисковый URI службы DuckDuckGo *)
let query_uri query =
  let base_uri = Uri.of_string "http://api.duckduckgo.com/?format=json" in
  Uri.add_query_param base_uri ("q", [query])
```

Значение типа `Uri.t` конструируется с помощью функции `Uri.of_string`, к которому затем добавляется параметр запроса `q` с желаемым критерием поиска. Библиотека сама заботится о надлежащем кодировании URI при выводе по сетевому протоколу.

Парсинг строк JSON

HTTP-ответ от службы DuckDuckGo поступает в формате JSON, распространенном (и, к счастью, простом) и определяемом в документе RFC 4627². Парсинг данных в формате JSON мы будем выполнять с помощью библиотеки `Yojson`, представленной в главе 15.

Как ожидается, ответ службы DuckDuckGo имеет вид записи JSON, которая представлена тегом `Assoc` вариантного типа JSON в `Yojson`. Сам результат поиска будет возвращаться службой либо в виде значения ключа «Abstract», либо в виде значения ключа «Definition». Поэтому следующий код проверяет оба ключа и возвращает первое непустое значение:

OCaml (part 1)

<https://github.com/realworldocaml/examples/blob/v1/code/async/search.ml>

```
(* Извлекает поле "Definition" или "Abstract" из ответа DuckDuckGo *)
let get_definition_from_json json =
  match Yojson.Safe.from_string json with
```

¹ <http://tools.ietf.org/html/rfc3986>.

² <http://www.ietf.org/rfc/rfc4627.txt>.

```
| `Assoc kv_list ->
let find key =
  begin match List.Assoc.find kv_list key with
  | None | Some (`String "") -> None
  | Some s -> Some (Yojson.Safe.to_string s)
  end
in
begin match find "Abstract" with
| Some _ as x -> x
| None -> find "Definition"
end
| _ -> None
```

Выполнение запроса HTTP

Теперь рассмотрим код, отправляющий поисковый запрос по протоколу HTTP, используя для этого библиотеку Cohttp:

OCaml (part 2)

<https://github.com/realworldocaml/examples/blob/v1/code/async/search.ml>

```
(* Выполняет поисковый запрос к службе DuckDuckGo *)
let get_definition word =
  Cohttp_async.Client.get (query_uri word)
>=> fun (_, body) ->
  Pipe.to_list body
>>| fun strings ->
  (word, get_definition_from_json (String.concat strings))
```

Чтобы лучше разобраться в происходящем, взглянем на тип функции `Cohttp_async.Client.get`, что можно сделать, не покидая интерактивной оболочки:

OCaml utop (part 28)

<https://github.com/realworldocaml/examples/blob/v1/code/async/main.topscript>

```
# #require "cohttp.async";;
# Cohttp_async.Client.get;;
- : ?interrupt:unit Deferred.t ->
  ?headers:Cohttp.Header.t ->
  Uri.t -> (Cohttp.Response.t * string Pipe.Reader.t) Deferred.t
= <fun>
```

Функция `get` принимает обязательный аргумент с адресом URI и возвращает отложенный результат со значением типа `Cohttp.Response.t` (который мы игнорируем) и объект чтения из канала, в который будет записан запрос.

В данном случае тело HTTP-ответа получится не очень большим, поэтому мы вызываем `Pipe.to_list` для выборки строк из канала в единый список. Затем эти строки объединяются вызовом `String.concat`, и результат передается функции парсинга.

Выполнение единственного запроса малоинтересно с точки зрения конкуренции, поэтому напомним код, который одновременно будет выполнять множество запросов. Но прежде напомним средство форматирования и вывода результатов поиска:

OCaml (part 3)

<https://github.com/realworldocaml/examples/blob/v1/code/async/search.ml>

```
(* Выводит пару слово/определение *)
let print_result (word, definition) =
  printf "%s\n%s\n\n"
    word
    (String.init (String.length word) ~f:(fun _ -> '-'))
  (match definition with
  | None -> "No definition found"
  | Some def ->
    String.concat ~sep:"\n"
      (Wrapper.wrap (Wrapper.make 70) def))
```

Для переноса строк мы использовали модуль `Wrapper` из пакета `textwrap`. Возможно, не очевидно, что эта процедура использует `Async`, но это так: версия `printf`, которая вызывается здесь, в действительности является специализированной функцией форматированного вывода, действующей через планировщик `Async`. Оригинальное определение `printf` затеняется новой функцией при открытии `Async.Std`. Важным побочным эффектом этого является отсутствие какого-либо вывода при вызове функций, таких как `printf`, если в программе, основанной на использовании `Async`, забыть запустить планировщик!

Следующая функция выполняет несколько параллельных запросов, дожидается результатов и затем выводит их:

OCaml (part 4)

<https://github.com/realworldocaml/examples/blob/v1/code/async/search.ml>

```
(* Выполняет несколько параллельных запросов и после их выполнения
   выводит результаты. *)
let search_and_print words =
  Deferred.all (List.map words ~f:get_definition)
  >>| fun results ->
    List.iter results ~f:print_result
```

Мы используем `List.map`, чтобы вызвать `get_definition` для каждого слова, и `Deferred.all`, чтобы дождаться получения всех результатов. Ниже приводится тип функции `Deferred.all`:

OCaml utop (part 29)

<https://github.com/realworldocaml/examples/blob/v1/code/async/main.topscript>

```
# Deferred.all;;
- : 'a Deferred.t list -> 'a list Deferred.t = <fun>
```

Обратите внимание, что список, возвращаемый функцией `Deferred.all`, отражает порядок следования отложенных результатов, переданных ей. По этой причине определения будут выведены в том же порядке, что и искомые слова, независимо от порядка получения ответов. Мы могли бы реализовать вывод результатов в порядке их поступления (обычно случайном), как показано ниже:

OCaml (part 1)

https://github.com/realworldocaml/examples/blob/v1/code/async/search_out_of_order.ml

```
(* Выполняет несколько параллельных запросов и
    выводит результаты в порядке их поступления *)
let search_and_print words =
  Deferred.all_unit (List.map words ~f:(fun word ->
    get_definition word >>| print_result))
```

Разница в том, что выполнение запросов и вывод результатов производится в замыкании, передаваемом функции `map`, вместо того чтобы дожидаться получения всех результатов и затем вывести их все вместе. Здесь использована функция `Deferred.all_unit`, принимающая список `unit` отложенных результатов и возвращающая единственное значение `unit` отложенного результата, которое становится определенным, когда все отложенные результаты во входном списке будут определены. Тип этой функции можно посмотреть в интерактивной оболочке:

OCaml utop (part 30)

<https://github.com/realworldocaml/examples/blob/v1/code/async/main.topscript>

```
# Deferred.all_unit;;
- : unit Deferred.t list -> unit Deferred.t = <fun>
```

Наконец, реализуем интерфейс командной строки с использованием `Command.async_basic`:

OCaml (part 5)

<https://github.com/realworldocaml/examples/blob/v1/code/async/search.ml>

```
let () =
  Command.async_basic
    ~summary:"Retrieve definitions from duckduckgo search engine"
    Command.Spec.(
      empty
      +> anon (sequence ("word" %: string))
    )
    (fun words () -> search_and_print words)
|> Command.run
```

Это было все, что необходимо для создания простого и удобного инструмента поиска:

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/async/run_search.out

```
$ corebuild -pkg cohttp.async,yojson,textwrap search.native
$ ./search.native "Concurrent Programming" "OCaml"
Concurrent Programming
-----
```

```
"Concurrent computing is a form of computing in which programs are
designed as collections of interacting computational processes that
may be executed in parallel."
```

OCaml

"OCaml, originally known as Objective Caml, is the main implementation of the Caml programming language, created by Xavier Leroy, Jérôme Vouillon, Damien Doligez, Didier Rémy and others in 1996."

Обработка исключений

При обращении к внешним ресурсам ошибки могут возникнуть везде: от непредсказуемого поведения сервера до отключения сети и исчерпания локальных ресурсов, что может привести к ошибкам во время выполнения. В программах на языке OCaml некоторые из этих ошибок обозначаются как особые возвращаемые значения, а некоторые — как исключения. Мы уже рассматривали вопросы обработки исключений в программном коде на OCaml, в разделе «Исключения» в главе 7, но, как будет показано далее, обработка исключений в конкурентных программах сопряжена с некоторыми новыми для вас сложностями.

Давайте поближе познакомимся с обработкой исключений в Async, реализовав асинхронные вычисления, которые (иногда) будут возбуждать исключение. Функция `maybe_raise` блокируется на полсекунды и затем или возбуждает исключение, или возвращает `unit`, чередуя эти два варианта поведения в последовательности вызовов:

OCaml utop (part 31)

<https://github.com/realworldocaml/examples/blob/v1/code/async/main.topscript>

```
# let maybe_raise =
  let should_fail = ref false in
  fun () ->
    let will_fail = !should_fail in
    should_fail := not will_fail;
    after (Time.Span.of_sec 0.5)
    >>= fun () ->
      if will_fail then raise Exit else return ()
;;

val maybe_raise : unit -> unit Deferred.t = <fun>
# maybe_raise ();;
- : unit = ()
# maybe_raise ();;
Exception:
(lib/monitor.ml.Error_
 ((exn Exit) (backtrace ("")
 (monitor
 ((name block_on_async) (here ()) (id 55) (has_seen_error true)
 (someone_is_listening true) (kill_index 0))
 ((name main) (here ()) (id 1) (has_seen_error false)
 (someone_is_listening false) (kill_index 0)))))).
```

В интерактивной оболочке исключение, возбужденное вызовом `maybe_raise ()`, завершает вычисление только данного выражения, но в программе необработанное исключение может вызвать завершение всего процесса.

Итак, как же организовать обработку таких исключений? Можно попробовать использовать для этих целей конструкцию `try/with`, но, как показано ниже, она не всегда гарантирует успех:

OCaml utop (part 32)

<https://github.com/realworldocaml/examples/blob/v1/code/async/main.topscript>

```
# let handle_error () =
  try
    maybe_raise ()
    >>| fun () -> "success"
  with _ -> return "failure"
;;

val handle_error : unit -> string Deferred.t = <fun>
# handle_error ();;
- : string = "success"
# handle_error ();;
Exception:
(lib/monitor.ml.Error_
 ((exn Exit) (backtrace (""))
 (monitor
  ((name block_on_async) (here ()) (id 59) (has_seen_error true)
   (someone_is_listening true) (kill_index 0))
  ((name main) (here ()) (id 1) (has_seen_error false)
   (someone_is_listening false) (kill_index 0)))))).
```

Причина неудачи в том, что конструкция `try/with` может перехватывать исключения только в коде, непосредственно выполняемом внутри нее, тогда как `maybe_raise` всего лишь ставит в план задание `Async` для выполнения в будущем, а само исключение возбуждается внутри задания.

Такие асинхронные ошибки можно перехватывать с помощью функции `try_with` из пакета `Async`:

OCaml utop (part 33)

<https://github.com/realworldocaml/examples/blob/v1/code/async/main.topscript>

```
# let handle_error () =
  try_with (fun () -> maybe_raise ())
  >>| function
    | Ok () -> "success"
    | Error _ -> "failure"
;;

val handle_error : unit -> string Deferred.t = <fun>
# handle_error ();;
- : string = "success"
# handle_error ();;
- : string = "failure"
```

`try_with f` принимает как аргумент функцию, возвращающую отложенный результат, и сама возвращает отложенный результат, который принимает значение `Ok`, если `f` что-то вернула, или `Error exn`, если `f` возбудила исключение, прежде чем возвращаемое ею значение стало определенным.

Мониторы

`try_with` – отличный способ обработки исключений, возникающих в пакете `Async`, но это еще не все. Все механизмы обработки исключений, имеющиеся в `Async`, включая `try_with`, основаны на системе *мониторов*, созданных по образу и подобию одноименного механизма обработки ошибок в языке `Erlang`. Мониторы – это достаточно низкоуровневый механизм, они редко используются непосредственно, тем не менее знание принципа их работы не будет лишним.

Монитор в `Async` – это контекст, определяющий, что делать при появлении не-обработанного исключения. Каждое задание `Async` выполняется в контексте не-которого монитора, который в момент выполнения задания называется *текущим монитором*. Когда планируется новое задание `Async`, например с помощью функции `bind` или `map`, оно наследует текущий монитор задания-родителя.

Мониторы образуют древовидную структуру – когда создается новый монитор (например, вызовом `Monitor.create`), он становится потомком текущего монитора. Имеется возможность явно запускать задание внутри определенного монитора: с помощью функции `within`, которая принимает функцию-переходник и возвращает обычное (неотложенное) значение, или с помощью функции `within'`, которая принимает функцию-переходник и возвращает отложенный результат. Например:

OCaml utop (part 34)

<https://github.com/realworldocaml/examples/blob/v1/code/async/main.topscript>

```
# let blow_up () =
  let monitor = Monitor.create ~name:"blow up monitor" () in
  within' ~monitor maybe_raise
;;
val blow_up : unit -> unit Deferred.t = <fun>
# blow_up ();;
- : unit = ()
# blow_up ();;
Exception:
(lib/monitor.ml.Error_
((exn Exit) (backtrace (""))
(monitor
((name "blow up monitor") (here ()) (id 71) (has_seen_error true)
(someone_is_listening false) (kill_index 0))
((name block_on_async) (here ()) (id 70) (has_seen_error false)
(someone_is_listening true) (kill_index 0))
(name main) (here ()) (id 1) (has_seen_error false)
(someone_is_listening false) (kill_index 0)))))).
```

В случае появления исключения вдобавок к обычной трассировке стека выводится также трассировка мониторов, начиная с монитора, созданного нами, который называют «аварийным монитором» (`blow up monitor`). Остальные мониторы, присутствующие в выводе, были порождены механизмом обработки отложенных результатов в интерактивной оболочке.

Мониторы обладают гораздо более широкими возможностями, чем трассировка исключений. Их можно также использовать для явной обработки ошибок. Од-

ной из наиболее важных в этом отношении является функция `Monitor.errors`. Она отсоединяет монитор от его родителя и передает ему поток ошибок, который иначе был бы доставлен родительскому монитору. Это позволяет организовать собственную обработку ошибок, включая передачу их родителю. Ниже приводится очень простой пример функции, перехватывающей ошибки, которые могут возникать в порождаемых ею процессах:

OCaml utop

<https://github.com/realworldocaml/examples/blob/v1/code/async/main-35.rawscript>

```
# let swallow_error () =
  let monitor = Monitor.create () in
  Stream.iter (Monitor.errors monitor) ~f:(fun _exn ->
    printf "an error happened\n");
  within' ~monitor (fun () ->
    after (Time.Span.of_sec 0.5) >>= fun () -> failwith "Kaboom!")
;;
val swallow_error : unit -> 'a Deferred.t = <fun>
# swallow_error ();;
an error happened
```

Сообщение «an error happened» выводится, но отложенный результат, возвращаемый функцией `swallow_error`, никогда не получит определенного значения. В этом есть определенный смысл, так как вычисления фактически никогда не завершаются, поэтому нет никакого значения, которое можно было бы вернуть. Прервать выполнение в интерактивной оболочке можно нажатием комбинации **Control+C**.

Ниже приводится пример монитора, обрабатывающего одни исключения и передающего своему родителю другие. Исключения передаются родительскому монитору, который определяется с помощью `Monitor.current`, вызовом `Monitor.send_exn`:

OCaml utop (part 36)

<https://github.com/realworldocaml/examples/blob/v1/code/async/main.topscript>

```
# exception Ignore_me;;
exception Ignore_me
# let swallow_some_errors exn_to_raise =
  let child_monitor = Monitor.create () in
  let parent_monitor = Monitor.current () in
  Stream.iter (Monitor.errors child_monitor) ~f:(fun error ->
    match Monitor.extract_exn error with
    | Ignore_me -> printf "ignoring exn\n"
    | _ -> Monitor.send_exn parent_monitor error);
  within' ~monitor:child_monitor (fun () ->
    after (Time.Span.of_sec 0.5)
    >>= fun () -> raise exn_to_raise)
;;
val swallow_some_errors : exn -> 'a Deferred.t = <fun>
```

Обратите внимание, что для извлечения первоначального исключения используется функция `Monitor.extract_exn`. Пакет `Async` оборачивает перехваченные им

исключения дополнительной информацией, включая трассировку мониторов, поэтому для сопоставления необходимо извлечь исходное исключение.

Если передать функции `swallow_some_errors` другое исключение, отличное от `Ignore_me`, например встроенное исключение `Not_found`, оно будет передано родительскому монитору, как обычно:

OCaml utop (part 37)

<https://github.com/realworldocaml/examples/blob/v1/code/async/main.topscript>

```
# swallow_some_errors Not_found;;
Exception:
(lib/monitor.ml.Error_
((exn Not_found) (backtrace (""))
(monitor
((name (id 75)) (here ()) (id 75) (has_seen_error true)
(someone_is_listening true) (kill_index 0))
(name block_on_async) (here ()) (id 74) (has_seen_error true)
(someone_is_listening true) (kill_index 0))
(name main) (here ()) (id 1) (has_seen_error false)
(someone_is_listening false) (kill_index 0)))))).
```

Если передать исключение `Ignore_me`, оно будет проигнорировано, и отложенный результат никогда не получит определенного значения:

OCaml utop

<https://github.com/realworldocaml/examples/blob/v1/code/async/main-38.rawscript>

```
# swallow_some_errors Ignore_me;;
ignoring exn
```

На практике следует как можно реже использовать мониторы непосредственно, а вместо них применять функции, такие как `try_with` и `Monitor.protect`, основанные на мониторах. Примером библиотек, использующих мониторы непосредственно, является `Tcp.Server.create`, которая перехватывает исключения, возбуждаемые логикой обработки сетевых соединений и функциями обратного вызова обработки отдельных запросов. В обоих случаях она закрывает соединение. Именно в таких ситуациях, когда требуется организовать собственную обработку ошибок, мониторы могут сослужить добрую службу.

Пример: обработка исключений при работе с DuckDuckGo

Давайте вернемся немного назад и улучшим обработку исключений в нашем клиенте, взаимодействующем со службой DuckDuckGo. В частности, мы изменим обработку ошибки, возникшей в одном запросе, так чтобы она не препятствовала выполнению других запросов.

Поскольку ошибки в поисковых запросах возникают слишком редко, добавим возможность искусственного создания ошибок. Для этого реализуем отправку запросов нескольким серверам и предусмотрим обработку ошибок, возникающих при обращении к серверам с ошибочными адресами.

Прежде всего изменим функцию `query_uri`, добавив в нее аргумент, определяющий адрес сервера:

OCaml (part 1)

https://github.com/realworldocaml/examples/blob/v1/code/async/search_with_configurable_server.ml

```
(* Генерирует поисковый URI службы DuckDuckGo *)
let query_uri ~server query =
  let base_uri =
    Uri.of_string (String.concat ["http://";server;"/?format=json"])
  in
  Uri.add_query_param base_uri ("q", [query])
```

Кроме того, мы внесем необходимые изменения, чтобы получить возможность передавать список серверов из командной строки, и реализуем круговое распределение запросов между ними. Теперь давайте посмотрим, что произойдет, если пересобрать приложение и передать ему список серверов, часть из которых не будет отвечать на запросы:

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/async/run_search_with_configurable_server.out

```
$ corebuild -pkg cohttp.async,yojson,textwrap \
  search_with_configurable_server.native
$ ./search_with_configurable_server.native \
  -servers localhost,api.duckduckgo.com \
  "Concurrent Programming" OCaml
("unhandled exception"
 ((lib/monitor.ml.Error_
  ((exn (Unix.Unix_error "Connection refused" connect 127.0.0.1:80))
   (backtrace
    ("Raised by primitive operation at file \"lib/unix_syscalls.ml\",
     line 797, characters 12-69"
     "Called from file \"lib/deferred.ml\", line 20, characters 62-65"
     "Called from file \"lib/scheduler.ml\", line 125, characters 6-17"
     "Called from file \"lib/jobs.ml\", line 65, characters 8-13" ""))
   (monitor
    ((name Tcp.close_sock_on_error) (here ()) (id 5) (has_seen_error true)
     (someone_is_listening true) (kill_index 0))
    ((name main) (here ()) (id 1) (has_seen_error true)
     (someone_is_listening false) (kill_index 0))))))
 (Pid 15971)))
```

Как видите, мы получили ошибку «Connection refused» (соединение отвергнуто), которая вызвала аварийное завершение всей программы, даже при том, что один из двух запросов мог бы быть выполнен успешно. Мы могли бы обрабатывать ошибки для каждого соединения в отдельности, задействовав функцию `try_with` внутри каждого вызова `get_definition`, как показано ниже:

OCaml (part 1)

https://github.com/realworldocaml/examples/blob/v1/code/async/search_with_error_handling.ml

```
(* Выполняет поисковый запрос к службе DuckDuckGo *)
let get_definition ~server word =
  try_with (fun () ->
```

```

Cohttp_async.Client.get (query_uri ~server word)
>>= fun (_, body) ->
  Pipe.to_list body
>>| fun strings ->
  (word, get_definition_from_json (String.concat strings)))
>>| function
| Ok (word,result) -> (word, Ok result)
| Error _         -> (word, Error "Unexpected failure")

```

Здесь мы сначала вызываем `try_with`, чтобы перехватить возможное исключение, и затем с помощью `map` (оператор `>>|`) преобразуем ошибку в требуемую нам форму: пару значений, первое из которых является искомым словом, а второе – результатом поиска (может быть ошибкой).

Теперь нам осталось лишь изменить реализацию функции `print_result`, чтобы она могла обрабатывать новый тип:

OCaml (part 2)

https://github.com/realworldocaml/examples/blob/v1/code/async/search_with_error_handling.ml

```

(* Выводит пару слово/определение *)
let print_result (word,definition) =
  printf "%s\n%s\n\n%s\n\n"
    word
    (String.init (String.length word) ~f:(fun _ -> '-'))
    (match definition with
    | Error s -> "DuckDuckGo query failed: " ^ s
    | Ok None -> "No definition found"
    | Ok (Some def) ->
      String.concat ~sep:"\n"
        (Wrapper.wrap (Wrapper.make 70) def))

```

Если теперь попытаться вызвать тот же запрос, мы получим обработку ошибок для каждого соединения в отдельности:

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/async/run_search_with_error_handling.out

```

$ corebuild -pkg cohttp.async,yojson,textwrap \
  search_with_error_handling.native
$ ./search_with_error_handling.native \
  -servers localhost,api.duckduckgo.com \
  "Concurrent Programming" OCaml

```

Concurrent Programming

DuckDuckGo query failed: Unexpected failure

OCaml

"OCaml, originally known as Objective Caml, is the main implementation of the Caml programming language, created by Xavier Leroy, Jérôme Vouillon, Damien Doligez, Didier Rémy and others in 1996."

Теперь ошибка возникла только в запросе, отправленном серверу localhost.

Отметьте, что в этом коде мы опираемся на особенность функции `Cohttp_async.Client.get`, которая автоматически освобождает ресурсы после появления исключения, в частности она закрывает дескрипторы файлов. Если потребуется реализовать такую функциональность непосредственно, для этого можно воспользоваться функцией `Monitor.protect`, которая является аналогом функции `protect`, описанной в разделе «Восстановление работоспособности после исключений» в главе 7.

Тайм-ауты, отмена и выбор

В конкурентных программах часто бывает необходимо объединять результаты выполнения нескольких параллельных заданий. Мы уже столкнулись с такой необходимостью в примере, демонстрирующем выполнение запросов к службе DuckDuckGo, где использовали `Deferred.all` и `Deferred.all_unit`, чтобы дождаться момента, когда все отложенные результаты получают определенное значение. Другим полезным примитивом является `Deferred.both`, который дает возможность дождаться вычисления двух отложенных результатов разных типов и возвращает оба значения в виде кортежа. В следующем фрагменте мы используем функцию `sec` для определения интервала времени, выраженного в секундах:

OCaml utop (part 39)

<https://github.com/realworldocaml/examples/blob/v1/code/async/main.topscript>

```
# let string_and_float = Deferred.both
  (after (sec 0.5) >>| fun () -> "A")
  (after (sec 0.25) >>| fun () -> 32.33);;
val string_and_float : (string * float) Deferred.t = <abstr>
# string_and_float;;
- : string * float = ("A", 32.33)
```

Однако иногда достаточно дождаться только первого из возможных событий, что может пригодиться для организации тайм-аута. В следующем примере мы используем функцию `Deferred.any`, которая принимает список отложенных результатов и возвращает один из них, который первым получит определенное значение:

OCaml utop (part 40)

<https://github.com/realworldocaml/examples/blob/v1/code/async/main.topscript>

```
# Deferred.any [ (after (sec 0.5) >>| fun () -> "half a second")
  ; (after (sec 10.) >>| fun () -> "ten seconds") ] ;;
- : string = "half a second"
```

Давайте используем этот прием для добавления тайм-аута в нашего клиента, осуществляющего поиск с помощью службы DuckDuckGo. Следующий код служит оберткой для функции `get_definition`. Он принимает величину тайм-аута (в виде значения типа `Time.Span.t`) и возвращает либо результаты поиска, либо ошибку, если ожидание оказалось слишком долгим:

OCaml (part 1)

https://github.com/realworldocaml/examples/blob/v1/code/async/search_with_timeout.ml

```
let get_definition_with_timeout ~server ~timeout word =
  Deferred.any
    [ (after timeout >>| fun () -> (word, Error "Timed out"))
    ; (get_definition ~server word
      >>| fun (word, result) ->
        let result' = match result with
          | Ok _ as x -> x
          | Error _ -> Error "Unexpected failure"
        in
        (word, result')
      )
    ]
```

Для преобразования отложенных результатов, вычисления которых ожидает программа, мы использовали оператор `>>|`, чтобы `Deferred.any` смогла выбирать между значениями одного типа.

Проблема в этом коде заключается в том, что HTTP-запрос, запущенный вызовом `get_definition`, в действительности не завершается после тайм-аута. Как результат в `get_definition_with_timeout` может возникнуть утечка открытых соединений. К счастью, `Cohttp` дает возможность прерывать выполнение запросов. Вы можете передать отложенный результат под меткой `interrupt` функции `Cohttp_async.Client.get`. Определив наличие метки `interrupt`, функция закроет соединение.

Следующий фрагмент демонстрирует, как можно изменить `get_definition` и `get_definition_with_timeout`, чтобы прервать выполнение вызова `get` в случае истечения тайм-аута:

OCaml (part 1)

https://github.com/realworldocaml/examples/blob/v1/code/async/search_with_timeout_no_leak_simple.ml

```
(* Выполняет поисковый запрос к службе DuckDuckGo *)
let get_definition ~server ~interrupt word =
  try_with (fun () ->
    Cohttp_async.Client.get ~interrupt (query_uri ~server word)
    >>= fun (_, body) ->
      Pipe.to_list body
      >>| fun strings ->
        (word, get_definition_from_json (String.concat strings)))
  >>| function
  | Ok (word, result) -> (word, Ok result)
  | Error exn -> (word, Error exn)
```

Далее добавим в `get_definition_with_timeout` создание отложенного результата для передачи функции `get_definition`. Этот результат будет получать определенное значение после истечения тайм-аута:

OCaml (part 2)

https://github.com/realworldocaml/examples/blob/v1/code/async/search_with_timeout_no_leak_simple.ml

```
let get_definition_with_timeout ~server ~timeout word =
  get_definition ~server ~interrupt:(after timeout) word
>>| fun (word,result) ->
  let result' = match result with
    | Ok _ as x -> x
    | Error _ -> Error "Unexpected failure"
  in
  (word,result')
```

Это будет вызывать закрытие соединения по истечении предельного времени ожидания. Но теперь наш код не знает, истекло время ожидания или был получен нормальный ответ от службы. В частности, при завершении по тайм-ауту теперь будет выводиться сообщение "Unexpected failure" (Неожиданная ошибка) вместо "Timed out" (Время ожидания истекло), выводившегося в предыдущей реализации.

Мы можем добиться большей точности, обрабатывая тайм-аут с помощью функции `choose` из пакета `Async`. Функция `choose` позволяет выбирать из коллекции отложенных результатов, чтобы обеспечить более точную реакцию на происходящее. С помощью функции `choice` можно связать каждый отложенный результат с функцией, которая должна быть вызвана при выборе данного отложенного результата. Ниже приводятся сигнатуры функций `choice` и `choose`:

OCaml utop (part 41)

<https://github.com/realworldocaml/examples/blob/v1/code/async/main.topscript>

```
# choice;;
- : 'a Deferred.t -> ('a -> 'b) -> 'b Deferred.choice = <fun>
# choose;;
- : 'a Deferred.choice list -> 'a Deferred.t = <fun>
```

Обратите внимание: нет никакой гарантии, что будет выбран отложенный результат, который действительно первым получил определенное значение. Функция `choose` гарантирует лишь единственность выбора и вызов соответствующей функции.

В следующем примере мы используем функцию `choose`, чтобы гарантировать получение определенного значения отложенным результатом `interrupt`, только если истекло время ожидания:

OCaml (part 2)

https://github.com/realworldocaml/examples/blob/v1/code/async/search_with_timeout_no_leak.ml

```
let get_definition_with_timeout ~server ~timeout word =
  let interrupt = Ivar.create () in
  choose
    [ choice (after timeout) (fun () ->
      Ivar.fill interrupt ();
      (word,Error "Timed out"))
```



```

; choice (get_definition ~server ~interrupt:(Ivar.read interrupt) word)
  (fun (word,result) ->
    let result' = match result with
    | Ok _ as x -> x
    | Error _ -> Error "Unexpected failure"
    in
    (word,result')
  )
]

```

Если теперь запустить программу с достаточно маленьким временем тайм-аута, мы увидим, что один запрос завершился успешно, а остальные были прерваны по тайм-ауту:

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/async/run_search_with_timeout_no_leak.out

```

$ corebuild -pkg cohttp.async,yojson,textwrap \
  search_with_timeout_no_leak.native
$ ./search_with_timeout_no_leak.native \
  "concurrent programming" ocaml -timeout 0.2s
concurrent programming
-----

```

DuckDuckGo query failed: Timed out

```

ocaml
-----

```

"OCaml or Objective Caml, is the main implementation of the Caml programming language, created by Xavier Leroy, Jérôme Vouillon, Damien Doligez, Didier Rémy and others in 1996."

Работа с системными потоками

Хотя до сих пор мы еще не работали с настоящими системными потоками выполнения (system threads), то есть потоками, переключение между которыми осуществляется самой операционной системой, тем не менее в языке OCaml имеется встроенная их поддержка. В начале этой главы говорилось, почему пакет Async обычно предпочтительнее, чем системные потоки выполнения, но даже если в большинстве случаев вы будете пользоваться Async, иногда вам все же придется обращаться к поддержке системных потоков в OCaml, поэтому есть смысл поближе познакомиться с ними.

Самое поразительное в поддержке системных потоков выполнения в языке OCaml — она не дает доступа к физическому параллелизму. Это обусловлено тем, что среда выполнения OCaml имеет единственную глобальную блокировку, которую может удерживать только один поток в каждый конкретный момент времени.

Проще говоря, поддержка потоков выполнения не обеспечивает настоящего параллелизма, но тогда какой смысл вообще говорить о них?

Самая распространенная причина использования системных потоков – в том, что в операционных системах некоторые системные вызовы не имеют неблокирующих альтернатив. То есть невозможно вызывать их непосредственно с применением таких систем, как Async, не блокируя при этом выполнения программы. По этой причине Async поддерживает пул потоков для выполнения подобных системных вызовов. В большинстве случаев, как пользователю Async, вам не придется задумываться об этом, но знать об этом следует.

Другая причина в пользу использования системных потоков – применение библиотек, не имеющих отношения к языку OCaml, реализующих собственные циклы событий или по иным причинам нуждающихся в отдельных потоках выполнения. В этом случае иногда полезно выполнять некоторый код на OCaml в другом потоке. Подробнее интерфейс OCaml вызова внешних функций обсуждается в главе 19.

Еще один случай использования системных потоков – улучшение взаимодействия с кодом на OCaml, выполняющем интенсивные вычисления. В Async при выполнении длительных вычислений, в ходе которых не вызывается функция `bind` или `map`, эти вычисления могут заблокировать среду выполнения Async до их завершения.

Один из способов избежать этого – явно разбить решение большой задачи на более мелкие подзадачи. Но иногда это невозможно или слишком сложно из-за необходимости вносить изменения в уже существующий код. Поэтому часто прибегают к другому решению: выполняют код в отдельном потоке. Модуль `In_thread` из пакета Async предоставляет множество средств для этого, простейшим из которых является функция `In_thread.run`. Например, можно написать следующий код:

OCaml utop (part 42)

<https://github.com/realworldocaml/examples/blob/v1/code/async/main.topscript>

```
# let def = In_thread.run (fun () -> List.range 1 10);;
val def : int list Deferred.t = <abstr>
# def;;
- : int list = [1; 2; 3; 4; 5; 6; 7; 8; 9]
```

чтобы обеспечить выполнение `List.range 1 10` в одном из рабочих потоков Async. По завершении вычислений результат будет помещен в отложенный результат, откуда он сможет быть извлечен обычным способом с помощью инструментов из Async.

Для взаимодействия с системными потоками в Async используется довольно сложный алгоритм. Взгляните на следующую функцию, проверяющую отзывчивость Async. Она принимает функцию-переходник (`thunk`), возвращающую отложенный результат, затем вызывает переданную ей функцию, после чего возобновляет свое выполнение через каждые 100 миллисекунд и выводит разницу между временем запуска и текущим временем, пока отложенный результат не получит определенного значения. В заключение она в последний раз выводит разницу во времени:

OCaml utop (part 43)

<https://github.com/realworldocaml/examples/blob/v1/code/async/main.topscript>

```
# let log_delays thunk =
  let start = Time.now () in
  let print_time () =
    let diff = Time.diff (Time.now ()) start in
    printf "%s, " (Time.Span.to_string diff)
  in
  let d = thunk () in
  Clock.every (sec 0.1) ~stop:d print_time;
  d >>| fun () -> print_time (); printf "\n"
;;
val log_delays : (unit -> unit Deferred.t) -> unit Deferred.t = <fun>
```

Если передать этой функции простой отложенный результат, определяющий тайм-аут, она будет работать в полном соответствии с нашими ожиданиями, возобновляя выполнение примерно каждые 100 миллисекунд:

OCaml utop

<https://github.com/realworldocaml/examples/tree/v1/code/async/main-44.rawscript>

```
# log_delays (fun () -> after (sec 0.5));;
0.154972ms, 102.126ms, 203.658ms, 305.73ms, 407.903ms, 501.563ms,
- : unit = ()
```

Теперь посмотрим, что произойдет, если вместо тайм-аута попробовать дождаться окончания вычислительного цикла:

OCaml utop

<https://github.com/realworldocaml/examples/tree/v1/code/async/main-45.rawscript>

```
# let busy_loop () =
  let x = ref None in
  for i = 1 to 100_000_000 do x := Some i done
;;
val busy_loop : unit -> unit = <fun>
# log_delays (fun () -> return (busy_loop ()));;
19.2185s,
- : unit = ()
```

Как видите, вместо того чтобы возобновить выполнение 10 раз в секунду, `log_delays` полностью блокируется на все время выполнения `busy_loop`.

Если, напротив, воспользоваться функцией `In_thread.run`, чтобы обеспечить выполнение функции в другом потоке, мы получим иное поведение:

OCaml utop

<https://github.com/realworldocaml/examples/tree/v1/code/async/main-46.rawscript>

```
# log_delays (fun () -> In_thread.run busy_loop);;
0.332117ms, 16.6319s, 18.8722s,
- : unit = ()
```

Теперь функция `log_delays` получит возможность поработать, хотя и реже, чем раз в 100 миллисекунд. Причина такого поведения – в том, что, используя систем-

ные потоки выполнения, мы полностью отдаемся во власть операционной системы, которая сама решает, когда выделить квант времени тому или иному потоку. Поведение потоков выполнения в значительной мере зависит от настроек операционной системы.

Другая особенность работы с потоками в OCaml связана с распределением памяти. При компиляции в машинный код потоки OCaml получают возможность приобрести блокировку среды времени выполнения, только когда они взаимодействуют с механизмом распределения памяти. То есть если поток вообще не выделяет память, он не позволит другим потокам OCaml захватить блокировку и приступить к выполнению. Байт-код действует иначе, то есть тот же цикл, скомпилированный в байт-код, не будет препятствовать работе других потоков:

OCaml utop

<https://github.com/realworldocaml/examples/tree/v1/code/async/main-47.rawscript>

```
# let noalloc_busy_loop () =
  for i = 0 to 100_000_000 do () done
;;
val noalloc_busy_loop : unit -> unit = <fun>
# log_delays (fun () -> In_thread.run noalloc_busy_loop);;
0.169039ms, 4.58345s, 4.77866s, 4.87957s, 12.4723s, 15.0134s,
- : unit = ()
```

Но если этот же цикл скомпилировать в машинный код, он заблокирует все другие потоки:

Terminal

[https://github.com/realworldocaml/examples/tree/v1/code/async/](https://github.com/realworldocaml/examples/tree/v1/code/async/run_native_code_log_delays.out)

`run_native_code_log_delays.out`

```
$ corebuild -pkg async native_code_log_delays.native
$ ./native_code_log_delays.native
15.5686s,
$
```

Из всех этих примеров следует, что взаимодействие между потоками полно непредсказуемых неожиданностей. Конечно, пакет Async имеет собственные ограничения, но, оставаясь в его пределах, вы будете получать более предсказуемое поведение.

Защищенность данных в потоках и блокировки

Приступая к использованию системных потоков выполнения, необходимо с особым вниманием отнестись к изменяемым структурам данных. Большинство изменяемых структур данных в OCaml не имеют четко определенной семантики в многопоточной среде выполнения. Поэтому неаккуратное их использование может приводить к самым разным неприятностям, от повреждения данных до исключений времени выполнения, а в некоторых редких случаях и к исключениям доступа к памяти (segfaults). Это означает, что при совместном использовании изменяемых данных несколькими потоками выполнения доступ к этим данным

должен выполняться под защитой мьютексов. Даже структуры данных, которые с виду кажутся вполне безопасными, могут содержать изменяемые компоненты внутри. Например, «ленивые» значения могут оказаться непредсказуемыми в многопоточной среде.

В программах на OCaml в основном используются две реализации мьютексов: модуль `Mutex`, входящий в состав стандартной библиотеки и являющийся всего лишь оберткой вокруг системных мьютексов, и `Nano_mutex` – более эффективная альтернатива, использующая механизм блокировок в среде выполнения OCaml и позволяющая избежать дорогостоящего создания системных мьютексов. Как результат создание мьютекса `Nano_mutex.t` происходит в 20 раз быстрее, чем создание мьютекса `Mutex.t`, а приобретение – на 40% быстрее.

В общем случае объединение `Async` и потоков выполнения является весьма хитрой задачей, но она вполне решаема при соблюдении следующих условий:

- потоки не имеют совместно используемых изменяемых данных;
- вычисления, производимые с применением `In_thread.run`, не вызывают никаких функций из библиотеки `Async`.

Иногда вполне безопасно можно использовать потоки выполнения и с нарушением этих условий. В частности, внешние потоки выполнения могут приобретать блокировку `Async` с помощью модуля `Thread_safe` из библиотеки `Async` и тем самым обеспечивать безопасность вычислений. Это очень гибкий способ внедрения потоков выполнения в мир `Async`, но это достаточно сложный случай использования, рассмотрение которого выходит далеко за рамки данной главы.

Система времени выполнения

Написание хорошего программного кода на OCaml – это лишь часть типичного процесса разработки программного обеспечения. Вам также необходимо понимать, как OCaml выполняет этот код и как отлаживать и профилировать приложения. Третья часть книги целиком будет посвящена исследованию инструментов компилятора и системы времени выполнения языка OCaml. Эта система удивительно проста, если сравнивать ее с аналогичными системами в других языках, таких как Java или .NET CLR, поэтому данная глава будет понятна даже тем, кто случайно пришел в OCaml.

Для начала мы пройдемся по библиотеке `Stdlib`, обеспечивающей связь программного кода на OCaml с внешними библиотеками, написанными на C. Наиболее интересные особенности библиотеки будет демонстрироваться на примере функций `POSIX` и с использованием терминала.

Значения OCaml имеют простую организацию в памяти, в чем вы убедитесь в следующей главе, в процессе исследования различных типов данных. Затем мы продемонстрируем, как в языке OCaml выполняется сборка мусора, помогающая избежать утечек памяти.

В заключение этой части в двух длинных главах мы исследуем инструменты, входящие в состав компилятора OCaml, разбив их на две логические группы. В первой из этих глав мы познакомимся с парсером и механизмом контроля типов и дадим различные советы по решению типичных проблем в исходном коде. Во второй – расскажем об особенностях компиляции в байт-код и машинный код, а также объясним, как отлаживать и профилировать выполняемые двоичные файлы.

Интерфейс внешних функций

В языке OCaml имеется несколько способов взаимодействий с программным кодом, написанным на других языках. Компилятор способен связывать приложение с внешними библиотеками посредством кода на C, а также производить объектные файлы, которые могут встраиваться в приложения на других языках.

Механизм, посредством которого код, написанный на одном языке программирования, может вызывать код на другом языке программирования, называется *интерфейсом внешних функций* (Foreign Function Interface, FFI). В этой главе мы:

- увидим, как вызывать функции из библиотек на языке C;
- узнаем, как конструировать высокоуровневые абстракции на языке OCaml, основанные на низкоуровневых библиотеках на языке C;
- исследуем несколько законченных примеров доступа к интерфейсу терминала и функциям для работы с датой/временем в UNIX.

Простейший интерфейс внешних функций в OCaml даже не требует писать код на C! Библиотека `Stypes` дает возможность определять интерфейс исключительно на языке OCaml и берет на себя все хлопоты по загрузке символов из C-библиотеки и вызову внешних функций.

Давайте сразу перейдем к практическому примеру и посмотрим, как выглядит библиотека. В этом примере мы организуем взаимодействие с библиотекой `Ncurses`, доступной во многих системах и не влекущей за собой сложных зависимостей.

Установка библиотеки `Stypes`

Чтобы получить возможность использовать библиотеку `Stypes`, необходимо предварительно установить библиотеку `libffi`¹. Это очень популярная библиотека, и она должна быть доступна диспетчеру пакетов в вашей ОС.

Примечание для пользователей Mac OS: версия `libffi`, установленная по умолчанию в Mac OS X 10.8, слишком устарела и непригодна для работы с некоторыми функциональными возможностями `Stypes`. Воспользуйтесь инструментом установки пакетов Homebrew (`brew`) и с его помощью получите последнюю версию `libffi` до установки библиотеки для OCaml.

После этого можно установить библиотеку `Stypes` с помощью `OPAM`, как обычно:

Terminal

```
https://github.com/realworldocaml/examples/tree/v1/code/ffi/install.out
```

```
$ brew install libffi # для пользователей MacOS X
```

```
$ opam install ctypes
```

¹ <https://github.com/atgreen/libffi>.

```
$ utop
# require "ctypes.foreign" ;;
```

Чтобы опробовать первый пример, вам также потребуется библиотека Ncurses. Она входит в стандартный комплект установки большинства операционных систем, таких как Mac OS X, и в Debian Linux поставляется в виде пакета `libncurses5-dev`.

Пример: интерфейс к терминалу

Ncurses – это библиотека, упрощающая разработку текстовых интерфейсов, независимых от типа терминала. Она используется консольными клиентами электронной почты, такими как Mutt и Pine, и консольными веб-браузерами, такими как Lynx.

Полное описание интерфейса на языке C можно найти по адресу: <http://www.gnu.org/software/ncurses/>. В своем примере мы будем использовать лишь малую часть имеющихся возможностей, поскольку наша задача состоит в том, чтобы просто продемонстрировать особенности использования библиотеки Ctypes:

C

<https://github.com/realworldocaml/examples/tree/v1/code/ffi/ncurses.h>

```
typedef struct _win_st WINDOW;
typedef unsigned int chtype;

WINDOW *initscr      (void);
WINDOW *newwin        (int, int, int, int);
void     endwin        (void);
void     refresh        (void);
void     wrefresh       (WINDOW *);
void     addstr         (const char *);
int      mvwaddch       (WINDOW *, int, int, const chtype);
void     mvwaddstr      (WINDOW *, int, int, char *);
void     box            (WINDOW *, chtype, chtype);
int      cbreak         (void);
```

Функции в библиотеке Ncurses оперируют либо текущим псевдотерминалом, либо окном, созданным с помощью функции `newwin`. Структура `WINDOW` хранит внутреннее состояние библиотеки и за пределами Ncurses должна интерпретироваться как нечто абстрактное. Клиентам Ncurses достаточно сохранить указатель и передавать его функциям библиотеки, которые позаботятся обо всем остальном.

В библиотеке Ncurses имеется более 200 функций, но в нашем примере мы коснемся лишь некоторых из них. Функции `initscr` и `newwin` создают указатель на структуру `WINDOW` для всего окна терминала или его части соответственно. Функция `mvwaddstr` принимает указатель на структуру, смещения по координатам `x` и `y` и строку для вывода на экран в указанную позицию. Обновление терминала происходит только после вызова `refresh` или `wrefresh`.

Библиотека Ctypes реализует интерфейс, с помощью которого программный код на OCaml может отображать функции на языке C в эквивалентные функции на OCaml. Библиотека сама заботится о преобразовании вызовов функций на

OCaml и их аргументов в вызовы функций на языке C, о вызове внешних функций в библиотеках на языке C и о возврате результатов в виде значений OCaml.

Начнем с самого простого – определения основных значений и, в частности, с определения указателя на структуру `WINDOW`:

OCaml

<https://github.com/realworldocaml/examples/blob/v1/code/ffi/ncurses.ml>

```
open Ctypes
type window = unit ptr
let window : window typ = ptr void
```

Нам неизвестна организация самой структуры `WINDOW`, поэтому мы будем интерпретировать указатель на эту структуру как указатель типа `void`. Мы исправим данное определение ниже, в этой главе, а пока будем довольствоваться этим. Вторая инструкция определяет значение OCaml, представляющее указатель на структуру `WINDOW`. Это значение будет использоваться далее, в определениях функций Ctypes:

OCaml (part 1)

<https://github.com/realworldocaml/examples/blob/v1/code/ffi/ncurses.ml>

```
open Foreign

let initscr =
  foreign "initscr" (void @-> returning window)
```

Это все, что нам потребуется для вызова нашей первой функции `initscr`, выполняющей инициализацию терминала. Функция `foreign` принимает два параметра:

- имя функции на языке C, поиск которой будет выполнен с помощью POSIX-функции `dlsym`;
- значение, определяющее полный комплект аргументов функции на языке C и тип возвращаемого значения; оператор `@->` добавляет аргумент в список параметров, а `returning` завершает список типом возвращаемого значения.

Остальной интерфейс к библиотеке `Ncurses` просто дополняет эти определения:

OCaml (part 2)

<https://github.com/realworldocaml/examples/blob/v1/code/ffi/ncurses.ml>

```
let newwin =
  foreign "newwin"
    (int @-> int @-> int @-> int @-> returning window)

let endwin =
  foreign "endwin" (void @-> returning void)

let refresh =
  foreign "refresh" (void @-> returning void)

let wrefresh =
  foreign "wrefresh" (window @-> returning void)

let addstr =
```

```
foreign "addstr" (string @-> returning void)

let mvwaddch =
  foreign "mvwaddch"
    (window @-> int @-> int @-> char @-> returning void)

let mvwaddstr =
  foreign "mvwaddstr"
    (window @-> int @-> int @-> string @-> returning void)

let box =
  foreign "box" (window @-> char @-> char @-> returning void)

let cbreak =
  foreign "cbreak" (void @-> returning int)
```

Все эти определения являются прямыми отображениями объявлений на языке C в заголовочном файле библиотеки Ncurses. Отметьте, что значения `string` и `int` здесь не имеют отношения к объявлениям типов OCaml, — эти значения импортируются при открытии модуля `Ctypes` в начале файла.

Большинство параметров в примере использования Ncurses представляют обычные скалярные типы языка C. Исключение составляют `window` (указатель на состояние библиотеки) и `string` (отображает строки OCaml, хранящие в себе значение длины, в буферы символов, длина которых определяется по завершающему нулевому символу, следующему непосредственно за данными).

Сигнатура `ncurses.mli` модуля выглядит как самая обычная сигнатура на языке OCaml. Ее можно вывести непосредственно из файла `ncurses.ml`, указав специальную цель сборки:

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/ffi/infer_ncurses.out

```
$ corebuild -pkg ctypes.foreign ncurses.inferred.mli
$ cp _build/ncurses.inferred.mli .
```

Цель `inferred.mli` предписывает компилятору сгенерировать сигнатуру по умолчанию для файла модуля и сохранить ее в каталоге `_build`. В большинстве случаев бывает желательно скопировать этот файл в каталог с исходным кодом и изменить его для безопасности вызывающего кода, путем сокрытия некоторых внутренних особенностей за абстракциями.

Ниже приводится преобразованный интерфейс, который можно безопасно использовать из других библиотек:

OCaml

<https://github.com/realworldocaml/examples/blob/v1/code/ffi/ncurses.mli>

```
type window
val window : window Ctypes.typ
val initscr : unit -> window
val endwin : unit -> unit
val refresh : unit -> unit
```

```

val wrefresh : window -> unit
val newwin : int -> int -> int -> int -> window
val mvwaddch : window -> int -> int -> char -> unit
val addstr : string -> unit
val mvwaddstr : window -> int -> int -> string -> unit
val box : window -> char -> char -> unit
val cbreak : unit -> int

```

Тип `window` стал абстрактным, чтобы гарантировать создание указателей этого типа только посредством вызова функции `Ncurses.initscr`. Это предотвратит возможность передачи по ошибке указателей `void`, полученных из других источников, функциям из библиотеки `Ncurses`.

Теперь скомпилируем программу «hello world», осуществляющую вывод в терминал:

OCaml

<https://github.com/realworldocaml/examples/blob/v1/code/ffi/hello.ml>

```
open Ncurses
```

```

let () =
  let main_window = initscr () in
  ignore(cbreak ());
  let small_window = newwin 10 10 5 5 in
  mvwaddstr main_window 1 2 "Hello";
  mvwaddstr small_window 2 2 "World";
  box small_window '\000' '\000';
  refresh ();
  Unix.sleep 1;
  wrefresh small_window;
  Unix.sleep 5;
  endwin ()

```

Выполняемый файл `hello` компонуется с пакетом `ctypes.foreign`:

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/ffi/build_hello.out

```
$ corebuild -pkg ctypes.foreign -lflags -cclib,-lncurses hello.native
```

Теперь, если запустить программу командой `./hello.native`, она должна вывести «Hello World» в терминал!

О директивах сборки при использовании библиотеки Ctypes

Пример команды сборки, представленный выше, включает несколько дополнительных директив компоновки. В частности, директива `-lflags` предписывает компилятору `ocamlbuild` передать следующий за ней список аргументов, разделенных запятыми, команде `ocaml`, выполняющей компоновку двоичного файла. Утилита `ocaml`, в свою очередь, использует `-cclib` для передачи директив дальше, системному компилятору (обычно `gcc` или `clang`). В первую очередь нам нужно скомпоновать программу с библиотекой `ncurses` на языке C, чтобы обеспечить доступность символов в `Ctypes`, что и делает комбинация аргументов `-cclib,-lncurses`.

В некоторых дистрибутивах Linux, таких как Ubuntu 11.10 и выше, необходимо также добавить в директиву `-lflags` аргументы `-cclib,-Xlinker` и `--no-as-needed`. Аргумент `-Xlinker`

интерпретируется компилятором как директива для системного компоновщика `ld`, которому также передается флаг `--no-as-needed`. В некоторых современных дистрибутивах Linux (таких как Ubuntu 11.10 и выше) системный компоновщик настроен так, что он связывает с выполняемым файлом только библиотеки, содержащие символы, непосредственно используемые программой. Однако при использовании `Ctypes` обращение к символам производится только во время выполнения, что приводит к появлению исключений из-за недоступности библиотеки.

Флаг `--no-as-needed` отключает такое поведение компоновщика и гарантирует компоновку со всеми указанными библиотеками, независимо от наличия прямых ссылок на символы в них. К сожалению, флаг работает не везде (примечательным исключением является Mac OS X, где нельзя передать данный флаг).

Библиотека не была бы столь полезной, если бы она ограничивалась лишь возможностью определять простые C-типы. Она обеспечивает полную поддержку арифметики указателей языка C, преобразование указателей, операции чтения/записи с указателями с использованием функций на OCaml как указателей на функции в коде на языке C, а также определения `struct` и `union`.

Некоторые из этих возможностей мы подробно рассмотрим в оставшейся части главы, когда будем знакомиться с примерами использования POSIX-функций для работы с датами.

Простые скалярные типы языка C

В первую очередь посмотрим, как определяются простые скалярные типы. Все типы языка C имеют свои эквиваленты в языке OCaml, представленные через общее определение:

OCaml

<https://github.com/realworldocaml/examples/blob/v1/code/ctypes/ctypes.mli>

```
type 'a typ
```

`Ctypes.typ` – это тип значений, представляющих типы C в OCaml. С каждым экземпляром типа `typ` связаны два типа:

- тип C, используемый для хранения и передачи значений внешней библиотеке;
- соответствующий тип OCaml; параметр `'a` типа содержит тип OCaml – такой, что значение типа `t typ` используется для чтения и записи значений OCaml типа `t`.

В `Ctypes` можно увидеть и другие примеры использования значений `typ`, такие как:

- конструирование типов функций для связывания с внешними функциями;
- конструирование указателей для чтения и записи данных в контейнеры на языке C;
- описание компонентов структур, объединений и массивов.

Ниже приводятся определения большинства скалярных типов, соответствующих стандарту C99, включая некоторые платформозависимые:

OCaml (part 1)

<https://github.com/realworldocaml/examples/blob/v1/code/ctypes/ctypes.mli>

```

val void      : unit typ
val char      : char typ
val schar     : int typ
val short     : int typ
val int       : int typ
val long      : long typ
val llong     : llong typ
val nativeint : nativeint typ

val int8_t    : int typ
val int16_t   : int typ
val int32_t   : int32 typ
val int64_t   : int64 typ
val uchar     : uchar typ
val uchar     : uchar typ
val uint8_t   : uint8 typ
val uint16_t  : uint16 typ
val uint32_t  : uint32 typ
val uint64_t  : uint64 typ
val size_t    : size_t typ
val ushort    : ushort typ
val uint      : uint typ
val ulong     : ulong typ
val ullong    : ullong typ

val float     : float typ
val double    : float typ

val complex32 : Complex.t typ
val complex64 : Complex.t typ

```

Все эти значения имеют тип `'a typ`, где имя значения (например, `void`) сообщает соответствующий тип в языке C, тип компонента `'a` (например, `unit`) отражает представление этого типа в языке OCaml. Большинство отображений выглядит просто, но некоторые из них требуют дополнительных пояснений.

- Значения `void` в языке OCaml имеют тип `unit`. Использование этого значения в виде типа аргумента или возвращаемого значения приводит к созданию функции на OCaml, принимающей или возвращающей `unit`. Попытка разыменовать указатель типа `void` считается ошибкой, как и в языке C, и возбуждает исключение `IncompleteType`.
- Тип `size_t` в языке C является псевдонимом одного из беззнаковых целочисленных типов. Фактический размер значений этого типа и правила выравнивания при размещении в памяти различаются на разных платформах. Библиотека `Stypes` определяет тип `size_t` на языке OCaml как псевдоним подходящего целочисленного типа.
- В OCaml поддерживаются только вещественные числа двойной точности, поэтому оба типа `float` и `double` в языке C отображаются в тип `float` языка OCaml, а типы `float complex` и `double complex` — в тип `Complex.t` двойной точности.

Указатели и массивы

Создание программ на языке С немыслимо без применения указателей, поэтому они являются обязательной частью библиотеки `Stypes`, которая реализует полноценную поддержку арифметики указателей, преобразования указателей, чтения и записи по указателям, а также передачу указателей в функции и возврат их из функций.

Мы уже видели простой случай использования указателей в примере `Ncurses`. А теперь давайте рассмотрим новый пример, в котором попробуем использовать следующие функции POSIX:

С

https://github.com/realworldocaml/examples/tree/v1/code/ffi/posix_headers.h

```
time_t time(time_t *);
double difftime(time_t, time_t);
char *ctime(const time_t *timep);
```

Функция `time` возвращает текущее календарное время, а так как она является наиболее простой, начнем с нее. Сначала откроем некоторые модули из библиотеки `Stypes`:

- `Ctypes` – модуль `Ctypes` определяет функции, описывающие типы языка С на OCaml;
- `PosixTypes` – модуль `PosixTypes` включает объявления некоторых дополнительных типов POSIX (таких как `time_t`);
- `Foreign` – модуль `Foreign` экспортирует функцию `foreign`, обеспечивающую возможность вызова функций на языке С.

Теперь прямо в интерактивной оболочке можно написать интерфейс к функции `time`.

OCaml utop

<https://github.com/realworldocaml/examples/tree/v1/code/ffi/posix.topscript>

```
# #require "ctypes.foreign" ;;
# #require "ctypes.top" ;;
# open Ctypes ;;
# open PosixTypes ;;
# open Foreign ;;
# let time = foreign "time" (ptr time_t @-> returning time_t) ;;
val time : time_t ptr -> time_t = <fun>
```

Функция `foreign` является главным связующим звеном между языками OCaml и С. Она принимает два аргумента: имя функции на С и значение, описывающее тип этой функции. В данном случае функция `time` принимает единственный аргумент типа `ptr time_t` и возвращает значение типа `time_t`.

Теперь, не покидая интерактивную оболочку, можно вызвать получившуюся интерфейсную функцию. Ее аргумент является необязательным, поэтому мы просто передадим ей пустой указатель, который автоматически будет приведен к типу указателя на `time_t`:

OCaml utop (part 1)

<https://github.com/realworldocaml/examples/tree/v1/code/ffi/posix.topscript>

```
# let cur_time = time (from_voidp time_t null) ;;
val cur_time : time_t = 1376834134
```

Так как мы собираемся вызывать `time` неоднократно, напомним функцию-обертку, которая будет передавать пустой указатель:

OCaml utop (part 2)

<https://github.com/realworldocaml/examples/tree/v1/code/ffi/posix.topscript>

```
# let time' () = time (from_voidp time_t null) ;;
val time' : unit -> time_t = <fun>
```

Поскольку `time_t` является абстрактным типом, мы не можем использовать значение этого типа непосредственно. Нам нужно написать интерфейс ко второй функции в списке прототипов (`difftime`), чтобы можно было выполнять какие-нибудь полезные операции со значениями, возвращаемыми функцией `time`:

OCaml utop (part 3)

<https://github.com/realworldocaml/examples/tree/v1/code/ffi/posix.topscript>

```
# let difftime =
  foreign "difftime" (time_t @-> time_t @-> returning double) ;;
val difftime : time_t -> time_t -> float = <fun>
# let t1 =
  time' () in
  Unix.sleep 2;
  let t2 = time' () in
    difftime t2 t1 ;;
- : float = 2.
```

Интерфейс к функции `difftime`, представленный выше, позволяет сравнивать два значения `time_t`.

Выделение памяти для указателей

Рассмотрим чуть более сложный пример, где мы будем передавать функции непустой указатель. Продолжим пример, начатый выше, и напомним интерфейс к функции `ctime`, которая преобразует значение типа `time_t` в удобочитаемую строку:

OCaml utop (part 4)

<https://github.com/realworldocaml/examples/tree/v1/code/ffi/posix.topscript>

```
# let ctime = foreign "ctime" (ptr time_t @-> returning string) ;;
val ctime : time_t ptr -> string = <fun>
```

Новую интерфейсную функцию мы также определили в интерактивной оболочке, наращивая нашу коллекцию. Однако мы не можем просто передать ей результат вызова `time`:

OCaml utop (part 5)

<https://github.com/realworldocaml/examples/tree/v1/code/ffi/posix.topscript>

```
# ctime (time' ()) ;;
Characters 7-15:
```

```
Error: This expression has type time_t but an expression was expected of type
```

```
time_t ptr
(Ошибка: Это выражение имеет тип time_t, тогда как ожидается выражение типа
time_t ptr)
```

Это объясняется тем, что `ctime` ожидает получить указатель на значение `time_t`, а не само значение. Чтобы исправить проблему, нужно выделить память для значения типа `time_t` и получить ее адрес:

OCaml utop (part 6)

<https://github.com/realworldocaml/examples/tree/v1/code/ffi/posix.topscript>

```
# let t_ptr = allocate time_t (time' ()) ;;
val t_ptr : time_t ptr = (int64_t*) 0x238ac30
```

Функция `allocate` принимает тип значения и его содержимое и возвращает указатель соответствующего типа. Теперь можно вызвать `ctime` и передать ей указатель:

OCaml utop (part 7)

<https://github.com/realworldocaml/examples/tree/v1/code/ffi/posix.topscript>

```
# ctime t_ptr ;;
- : string = "Sun Aug 18 14:55:36 2013\n"
```

Использование представлений для отображения составных значений

Скалярные типы `C` отображаются в типы `OCaml`, что называется «один в один», однако с составными типами `C` дело обстоит несколько сложнее. Представления (`views`) создают новые описания типов `C`, обладающие особым поведением, при использовании их для чтения и записи значений языка `C`.

Одно такое представление мы уже использовали в определении функции `ctime` выше. Представление `string` служит оберткой для типа `char *` языка `C` (на языке `OCaml` записывается как `ptr char`) и выполняет преобразование строковых представлений между языками `C` и `OCaml` при чтении и записи значения.

Ниже приводится сигнатура функции `Ctypes.view`:

OCaml (part 2)

<https://github.com/realworldocaml/examples/tree/v1/code/ctypes/ctypes.mli>

```
val view :
  read:('a -> 'b) ->
  write:('b -> 'a) ->
  'a typ -> 'b typ
```

Библиотека `Ctypes` содержит пару низкоуровневых функций преобразования, отображающих строковые значения на языке `OCaml` в буфер символов на языке `C` и обратно, копируя содержимое соответствующих структур данных. Они имеют следующие сигнатуры:

OCaml (part 3)

<https://github.com/realworldocaml/examples/tree/v1/code/ctypes/ctypes.mli>

```
val string_of_char_ptr : char ptr -> string
val char_ptr_of_string : string -> char ptr
```


Благодаря наличию этих функций определение значения `Ctypes.string`, использующего представления, выглядит очень просто:

OCaml

https://github.com/realworldocaml/examples/tree/v1/code/ctypes/ctypes_impl.ml

```
let string =
  view (char ptr)
    ~read:string_of_char_ptr
    ~write:char_ptr_of_string
```

Тип этой функции `string` выглядит как обычный тип `typ`, без внешних признаков использования функции `view`:

OCaml (part 4)

<https://github.com/realworldocaml/examples/tree/v1/code/ctypes/ctypes.mli>

```
val string : string.typ
```

Строки в OCaml и буферы символов в C

С точки зрения интерфейса, строки в OCaml выглядят похожими на буферы символов в C, однако, с точки зрения размещения в памяти, они существенно отличаются.

Строки в языке OCaml хранятся в динамической памяти и имеют заголовок, определяющий их длину. Буферы в языке C также имеют фиксированную длину, но в соответствии с соглашениями строки в языке C завершаются нулевым (байт `\0`) символом. Строковые функции в языке C вычисляют длину строки, сканируя буфер, пока не встретится первый нулевой символ.

Из этого следует, что нужно быть осторожными при передаче строк OCaml функциям на языке C, которые могут содержать нулевые символы, так как первый такой символ будет интерпретироваться как конец строки. Кроме того, библиотека Ctypes предоставляет интерфейс копирования по умолчанию для строк, а это означает, что его не следует использовать, когда желательно вносить изменения в буфер на месте. В таких случаях используйте поддержку типа `Bigarray` для передачи блоков памяти по ссылке.

Структуры и объединения

Конструкции `struct` и `union` в языке C позволяют создавать новые типы из существующих. Библиотека Ctypes содержит аналогичные конструкции, действующие подобным образом.

Определение структуры

Давайте добавим еще одну функцию для работы со временем к написанным выше. POSIX-функция `gettimeofday` возвращает время с точностью до микросекунды. Ее сигнатура, включая определение структуры, приводится ниже:

C

https://github.com/realworldocaml/examples/tree/v1/code/ffi/timeval_headers.h

```
struct timeval {
  long tv_sec;
  long tv_usec;
};
```

```
int gettimeofday(struct timeval *, struct timezone *tv);
```

С помощью `Stypes` этот тип можно описать, как показано далее. Продолжим накапливать нашу коллекцию в интерактивной оболочке:

OCaml utop (part 8)

<https://github.com/realworldocaml/examples/tree/v1/code/ffi/posix.topscript>

```
# type timeval ;;
type timeval
# let timeval : timeval structure typ = structure "timeval" ;;
val timeval : timeval structure typ = struct timeval
```

Первая команда определяет новый тип `timeval` на языке OCaml, который мы будем использовать для создания OCaml-версий структуры. Это *фантомный тип*, объявленный с целью дать возможность отличать соответствующий тип на языке C от других типов указателей. Теперь структура `timeval` имеет тип, отличающий ее от других структур, которые могут быть определены где-то еще, и помогает избежать путаницы между ними.

Вторая команда вызывает функцию `structure` для создания нового составного типа. На данный момент тип структуры определен не полностью: мы можем добавлять в него поля, но не можем использовать его в вызовах внешних функций или для создания значений.

Добавление полей в структуры

В определении структуры `timeval` пока отсутствуют какие-либо поля, поэтому их нужно добавить:

OCaml utop (part 9)

<https://github.com/realworldocaml/examples/tree/v1/code/ffi/posix.topscript>

```
# let tv_sec = field timeval "tv_sec" long ;;
val tv_sec : (Signed.long, (timeval, [ `Struct ]) structured) field = <abstr>
# let tv_usec = field timeval "tv_usec" long ;;
val tv_usec : (Signed.long, (timeval, [ `Struct ]) structured) field =
<abstr>
# seal timeval ;;
- : unit = ()
```

Функция `field` добавляет поле в конец структуры, как показано выше, на примере полей `tv_sec` и `tv_usec`. Поля структуры являются типизированными функциями доступа, ассоциированными с определенной структурой, и соответствуют меткам в языке C.

Операция добавления поля изменяет переменную-структуру и записывает новый размер (точное значение которого зависит от типа только что добавленного поля). После вызова функции `seal`, фиксирующей определение структуры, появляется возможность создавать экземпляры структуры, но попытки добавить новые поля будут вызывать ошибку.

Незавершенные определения структур

Функция `gettimeofday` требует передачи во втором аргументе указателя типа `struct timezone`, поэтому нам также необходимо определить еще один составной тип:

OCaml utop (part 10)

<https://github.com/realworldocaml/examples/tree/v1/code/ffi/posix.topscript>

```
# type timezone ;;
type timezone
# let timezone : timezone structure typ = structure "timezone" ;;
val timezone : timezone structure typ = struct timezone
```

Нам не потребуется создавать значения `struct timezone`, поэтому определение данной структуры можно оставить незавершенным, без включения каких-либо полей и фиксации вызовом функции `seal`. Если попытаться использовать такую структуру в ситуации, где необходимо знать ее размер, библиотека возбудит исключение `IncompleteType`.

Теперь все готово к реализации интерфейса для функции `gettimeofday`:

OCaml utop (part 11)

<https://github.com/realworldocaml/examples/tree/v1/code/ffi/posix.topscript>

```
# let gettimeofday = foreign "gettimeofday"
  (ptr timeval @-> ptr timezone @-> returning_checking_errno int) ;;
val gettimeofday : timeval structure ptr -> timezone structure ptr -> int =
  <fun>
```

Здесь появилась еще одна новая особенность: функция `returning_checking_errno`, действующая подобно функции `returning`, за исключением того, что она проверяет признак ошибки, возвращаемый функцией на C. Изменения в переменной `errno` отображаются в исключение `OCaml Unix.Unix_error`, как это делают функции в стандартной библиотеке.

Как и прежде, можно создать функцию-обертку, упрощающую использование `gettimeofday`. Функции `make`, `addr` и `getf` создают экземпляр структуры, получают его адрес и извлекают значение поля:

OCaml utop (part 12)

<https://github.com/realworldocaml/examples/blob/v1/code/ffi/posix.topscript>

```
# let gettimeofday' () =
  let tv = make timeval in
  ignore(gettimeofday (addr tv) (from_voidp timezone null));
  let secs = Signed.Long.(to_int (getf tv tv_sec)) in
  let usecs = Signed.Long.(to_int (getf tv tv_usec)) in
  Pervasives.(float secs +. float usecs /. 1000000.0) ;;
val gettimeofday' : unit -> float = <fun>
# gettimeofday' () ;;
- : float = 1376834137.14
```

Необходимо проявлять дополнительную осторожность, чтобы в попытках не смешать все модули, используемые здесь. Оба модуля, `Pervasives` и `Ctypes`, определяют разные функции `float`. Модуль `Ctypes`, что был открыт ранее, переопределяет функцию из модуля `Pervasives`. Поэтому нам потребовалось открыть модуль `Pervasives` локально, чтобы получить доступ к его функции `float`.

Команда вывода времени

В предыдущем разделе мы определили множество интерфейсных функций, а теперь давайте объединим их в одном законченном примере, открывающем доступ к ним через интерфейс командной строки:

OCaml

<https://github.com/realworldocaml/examples/blob/v1/code/ffi/datetime.ml>

```
open Core.Std
open Ctypes
open PosixTypes
open Foreign

let time      = foreign "time" (ptr time_t @-> returning time_t)
let difftime = foreign "difftime" (time_t @-> time_t @-> returning double)
let ctime     = foreign "ctime" (ptr time_t @-> returning string)

type timeval
let timeval : timeval structure typ = structure "timeval"
let tv_sec  = field timeval "tv_sec" long
let tv_usec = field timeval "tv_usec" long
let ()      = seal timeval

type timezone
let timezone : timezone structure typ = structure "timezone"

let gettimeofday = foreign "gettimeofday"
  (ptr timeval @-> ptr timezone @-> returning_checking_errno int)

let time' () = time (from_voidp time_t null)

let gettimeofday' () =
  let tv = make timeval in
  ignore(gettimeofday (addr tv) (from_voidp timezone null));
  let secs = Signed.Long.(to_int (getf tv tv_sec)) in
  let usecs = Signed.Long.(to_int (getf tv tv_usec)) in
  Pervasives.(float secs +. float usecs /. 1_000_000.)

let float_time () = printf "%f!\n" (gettimeofday' ())

let ascii_time () =
  let t_ptr = allocate time_t (time' ()) in
  printf "%s!" (ctime t_ptr)

let () =
  let open Command in
  basic ~summary:"Display the current time in various formats"
    Spec.(empty +> flag "-a" no_arg ~doc:" Human-readable output format")
    (fun human -> if human then ascii_time else float_time)
  |> Command.run
```

Его можно скомпилировать и запустить, как обычно:

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/ffi/build_datetime.out

```
$ corebuild -pkg ctypes.foreign datetime.native
$ ./datetime.native
1376833554.984496
$ ./datetime.native -a
Sun Aug 18 14:45:55 2013
```

Зачем нужна функция returning?

Любопытным читателям может быть интересно узнать, почему все эти определения функций завершаются вызовом функции `returning`:

OCaml

https://github.com/realworldocaml/examples/blob/v1/code/ffi/return_frag.ml

```
(* правильные типы *)
val time: ptr time_t @-> returning time_t
val difftime: time_t @-> time_t @-> returning double
```

Вызов функции `returning` кому-то может показаться лишним. Почему бы просто не определить типы, как показано ниже?

OCaml (part 1)

https://github.com/realworldocaml/examples/blob/v1/code/ffi/return_frag.ml

```
(* неправильные типы *)
val time: ptr time_t @-> time_t
val difftime: time_t @-> time_t @-> double
```

Причина заключена в типах высшего порядка (higher types) и в двух различиях интерпретации функций в OCaml и C. Во-первых, функции в языке OCaml являются значениями первого порядка (first-class values), а в языке C – нет. Например, в C можно вернуть указатель на функцию из другой функции, но нельзя вернуть саму функцию. Во-вторых, функции в языке OCaml обычно определяются в каррированном стиле – сигнатура функции с двумя аргументами записывается, как показано ниже:

OCaml (part 2)

https://github.com/realworldocaml/examples/blob/v1/code/ffi/return_frag.ml

```
val curried : int -> int -> int
```

что в действительности означает:

OCaml (part 3)

https://github.com/realworldocaml/examples/blob/v1/code/ffi/return_frag.ml

```
val curried : int -> (int -> int)
```

а аргументы могут передаваться по одному за раз, в результате чего будут создаваться замыкания. В языке C, напротив, функции принимают все аргументы сразу. Эквивалентная функция на языке C имеет следующий тип:

C

https://github.com/realworldocaml/examples/blob/v1/code/ffi/return_c_frag.h

```
int uncurried_C(int, int);
```

и аргументы всегда должны передаваться ей все вместе:

C

https://github.com/realworldocaml/examples/blob/v1/code/ffi/return_c_frag.c

```
uncurried_C(3, 4);
```

Функция на языке C, записанная в каррированном стиле, выглядит совершенно иначе:

C

https://github.com/realworldocaml/examples/blob/v1/code/ffi/return_c_uncurried.c

```
/* Функция, принимающая аргумент типа int и возвращающая
   указатель на функцию, принимающую второй аргумент типа int. */
typedef int (function_t)(int);
function_t *curried_C(int);

/* передача двух аргументов сразу */
curried_C(3)(4);

/* передача аргументов по одному */
function_t *f = curried_C(3); f(4);
```

Интерфейсная функция на языке OCaml, вызывающая `uncurried_C` с помощью `Ctypes`, будет иметь тип `int -> int -> int`: это функция двух аргументов. Интерфейсная функция на языке OCaml, вызывающая `curried_C`, будет иметь тип `int -> (int -> int)`: это функция одного аргумента, возвращающая функцию одного аргумента.

Конечно, в OCaml эти функции абсолютно эквивалентны. Но, так как типы в OCaml и C одни и те же, а семантика различна, нам необходим некоторый признак для различения этих случаев. Роль такого признака как раз и играет вызов `returning` в определениях функций.

Определение массивов

Массивы в C являются непрерывными блоками памяти, хранящими значения одного типа. Любой простой тип из числа объявленных выше может использоваться для распределения блоков памяти с помощью модуля `Array`:

OCaml (part 5)

<https://github.com/realworldocaml/examples/blob/v1/code/ctypes/ctypes.mli>

```
module Array : sig
  type 'a t = 'a array

  val get : 'a t -> int -> 'a
  val set : 'a t -> int -> 'a -> unit
  val of_list : 'a typ -> 'a list -> 'a t
  val to_list : 'a t -> 'a list
  val length : 'a t -> int
  val start : 'a t -> 'a ptr
  val from_ptr : 'a ptr -> int -> 'a t
  val make : 'a typ -> ?initial:'a -> int -> 'a t
end
```

Функции для работы с массивами похожи на те, что определены в стандартном модуле `Array`, за исключением того, что они оперируют массивами, хранящимися в представлении, принятом в языке C, а не в представлении языка OCaml, о котором рассказывается в главе 20.

Как и в случае со стандартными массивами OCaml, операции преобразований между массивами и списками сопряжены с копированием значений и могут оказаться весьма дорогостоящими для больших структур данных. Следует также отметить возможность преобразования массива в указатель `ptr` на начало буфера в памяти, что может пригодиться в ситуациях, когда потребуется передать указатель на массив и размер массива в функцию на языке C.

Объединения (unions) в языке С – это именованные структуры, которые могут отображаться в одну и ту же область памяти. Они также полностью поддерживаются библиотекой Ctypes, но мы не будем углубляться в их обсуждение.

Операторы разыменования указателей и арифметических операций с ними

Библиотека Ctypes определяет ряд операторов, дающих возможность манипулировать указателями и массивами так же, как в языке С. Однако эквиваленты в Ctypes пользуются некоторыми дополнительными преимуществами, которые дает более строгая система типов в OCaml (см. табл. 19.1).

Таблица 19.1. Операторы для работы с указателями и массивами

Оператор	Назначение
<code>!@ p</code>	Разыменование указателя <code>p</code>
<code>p <-@ v</code>	Запись значения <code>v</code> по адресу <code>p</code>
<code>p +@ n</code>	Если <code>p</code> указывает на элемент массива, вычисляет адрес следующего <code>n</code> -го элемента
<code>p -@ n</code>	Если <code>p</code> указывает на элемент массива, вычисляет адрес предыдущего <code>n</code> -го элемента

Существуют и другие функции, не являющиеся операторами (за дополнительной информацией обращайтесь к документации Ctypes), для сравнения указателей.

Передача функций в код на С

В код на языке С можно передавать не только простые значения, но и функции на языке OCaml. Функция `qsort` из стандартной библиотеки С сортирует массивы элементов, используя функцию сравнения, переданную в виде указателя. Вот как выглядит сигнатура `qsort`:

С

<https://github.com/realworldocaml/examples/blob/v1/code/ffi/qsort.h>

```
void qsort(void *base, size_t nmemb, size_t size,
          int(*compar)(const void *, const void *));
```

Программисты на С часто используют `typedef` для определения типов указателей на функции, чтобы упростить чтение кода. С применением `typedef` тип функции `qsort` можно выразить проще:

С

https://github.com/realworldocaml/examples/blob/v1/code/ffi/qsort_typedef.h

```
typedef int(compare_t)(const void *, const void *);

void qsort(void *base, size_t nmemb, size_t size, compare_t *);
```

Как ни странно, но это определение очень похоже на соответствующее определение в библиотеке Ctypes. Так как описаниями типов являются обычные значения, мы легко можем использовать `let` вместо `typedef` и получить действующий интерфейс к `qsort` на языке OCaml:

OCaml utop

<https://github.com/realworldocaml/examples/blob/v1/code/ffi/qsrt.topscript>

```
# #require "ctypes.foreign" ;;
# open Ctypes ;;
# open PosixTypes ;;
# open Foreign ;;
# let compare_t = ptr void @-> ptr void @-> returning int ;;
val compare_t : (unit ptr -> unit ptr -> int) fn = <abstr>
# let qsrt = foreign "qsrt"
  (ptr void @-> size_t @-> size_t @->
   funptr compare_t @-> returning void) ;;
val qsrt :
  unit ptr -> size_t -> size_t -> (unit ptr -> unit ptr -> int) -> unit =
  <fun>
```

Тип `compare_t` используется только однажды (в определении `qsrt`), поэтому его можно, при желании, встроить непосредственно в код на OCaml. Как видно из примера выше, получившееся значение `qsrt` – это функция высшего порядка, потому что в четвертом аргументе ей передается другая функция. Как и прежде, давайте определим функцию-обертку, упрощающую использование `qsrt`. Второй и третий аргументы функции `qsrt` определяют длину (число элементов) массива и размер одного элемента.

Массивы, создаваемые с помощью `Ctypes`, имеют более богатую поддержку времени выполнения, чем массивы на языке C, поэтому нам не требуется передавать информацию о размерах. Кроме того, вместо небезопасного типа `void ptr` можно использовать полиморфизм OCaml.

Пример: быстрая сортировка в командной строке

Ниже приводится исходный код утилиты командной строки, использующей функцию `qsrt` для сортировки всех целых чисел, полученных со стандартного ввода:

OCaml

<https://github.com/realworldocaml/examples/tree/v1/code/ffi/qsrt.ml>

```
open Core.Std
open Ctypes
open PosixTypes
open Foreign

let compare_t = ptr void @-> ptr void @-> returning int

let qsrt = foreign "qsrt"
  (ptr void @-> size_t @-> size_t @-> funptr compare_t @->
   returning void)

let qsrt' cmp arr =
  let open Unsigned.Size_t in
  let ty = Array.element_type arr in
  let len = of_int (Array.length arr) in
  let elsize = of_int (sizeof ty) in
  let start = to_voidp (Array.start arr) in
  let compare l r = cmp (!@ (from_voidp ty l)) (!@ (from_voidp ty r)) in
```



```

qsort start len elsize compare;
arr

let sort_stdin () =
  In_channel.input_lines stdin
  |> List.map ~f:int_of_string
  |> Array.of_list int
  |> qsort' Int.compare
  |> Array.to_list
  |> List.iter ~f:(fun a -> printf "%d\n" a)

let () =
  Command.basic ~summary:"Sort integers on standard input"
    Command.Spec.empty sort_stdin
  |> Command.run

```

Скомпилируем ее с помощью *corebuild*, как обычно, и проверим, передав ей некоторые данные, а заодно и сохраним выведенный интерфейс для дальнейшего исследования:

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/ffi/build_qsort.out

```

$ corebuild -pkg ctypes.foreign qsort.native
$ cat input.txt
5
3
2
1
4
$ ./qsort.native < input.txt
1
2
3
4
5
$ corebuild -pkg ctypes.foreign qsort.inferred.mli
$ cp _build/qsort.inferred.mli qsort.mli

```

В выведенном интерфейсе можно видеть типы основной функции `qsort`, служащей интерфейсом к одноименной функции на C, а также функции-обертки `qsort'`:

OCaml

<https://github.com/realworldocaml/examples/tree/v1/code/ffi/qsort.mli>

```

val compare_t : (unit Ctypes.ptr -> unit Ctypes.ptr -> int) Ctypes.fn
val qsort :
  unit Ctypes.ptr ->
  PosixTypes.size_t ->
  PosixTypes.size_t -> (unit Ctypes.ptr -> unit Ctypes.ptr -> int) -> unit
val qsort' : ('a -> 'a -> int) -> 'a Ctypes.array -> 'a Ctypes.array
val sort_stdin : unit -> unit

```

Обертка `qsort'` имеет более канонический интерфейс OCaml, чем функция `qsort`. Она принимает функцию сравнения и массив `Ctypes.array` и возвращает все

тот же массив `Ctypes.array`. Строго говоря, от функции не требуется, чтобы она возвращала массив, потому что исходный массив сортируется на месте, но такой подход позволяет вставлять эту функцию в цепочку с помощью оператора `|>` (как это делается в функции `sort_stdin` в примере).

Использование `qsort`¹ для сортировки массивов не вызывает никаких сложностей. Наш пример читает исходные данные со стандартного ввода в список, преобразует его в массив `C`, передает функции `qsort` и выводит результат в стандартный вывод. И снова: не путайте модули `Ctypes.Array` и `Core.Std.Array`: первый из них становится доступен только после открытия `Ctypes` в начале файла.

Срок жизни значений Ctypes

Значения, размещенные в памяти с помощью `Ctypes` (например, вызовом `allocate`, `Array.make` и другими подобными функциями), не будут утилизироваться механизмом сборки мусора, пока они остаются доступными из значений OCaml. Память, занимаемая ими, освобождается только после того, как они станут недоступны программному коду, вызовом функции финализации, зарегистрированной сборщиком мусора (Garbage Collector, GC).

Однако понятие доступности для значений `Ctypes` несколько отличается от понятия доступности для обычных значений OCaml. Функции, создающие значения `Ctypes`, возвращают указатель на значение, управляемый средой выполнения OCaml, и пока любые производные от этого указателя остаются доступными сборщику мусора (GC), значение не будет утилизировано.

Под «производными» здесь понимаются указатели, полученные из исходного путем применения арифметики указателей. То есть доступность ссылки на элемент массива или поле структуры будет препятствовать утилизации всего массива или структуры.

Из предыдущего правила следует, что указатели, хранящиеся в динамической памяти `C`, никак не отражаются на достижимости или недостижимости. Например, если имеется `C`-массив указателей на структуры, тогда вам придется что-то предпринять, чтобы защитить эти структуры от утилизации сборщиком мусора. Добиться этого можно, например, с помощью глобального массива значений на стороне OCaml, где продолжать хранить структуры, пока они нужны.

Функции, которые передаются программному коду на `C`, подчиняются похожему правилу. Функция, созданная на стороне OCaml во время выполнения, может быть утилизирована, как только станет недоступной (выйдет из области видимости). Как мы уже видели, функции на OCaml, передаваемые программному коду на `C`, преобразуются в указатели, которые записываются в память на стороне `C`. Эти указатели никак не влияют на доступность функций OCaml. В случае с примером применения `qsort` все просто: функция сравнения нужна только на время вызова `qsort`. Однако другие библиотеки на `C` могут сохранять указатели на функции в своих глобальных переменных или где-то еще, поэтому вам придется позаботиться о таких функциях на стороне OCaml, чтобы они не оказались преждевременно утилизированы сборщиком мусора.

Дополнительная информация о взаимодействии с кодом на C

Дистрибутив библиотеки `Ctypes`¹ содержит несколько крупных примеров, включая:

- интерфейс к POSIX FTS API, демонстрирующий использование функций обратного вызова на `C`;

¹ <http://github.com/ocaml-labs/ocaml-ctypes>.

- более полный интерфейс к библиотеке Ncurses, чем приводился в начале этой главы;
- исчерпывающий комплект тестов, охватывающий тестированием всю библиотеку, где вы сможете подсмотреть немало интересного.

Вообще говоря, сведения, что приводятся в этой главе, не нужны для понимания внутренних особенностей языка OCaml. Разработчики библиотеки Stypes приложили все усилия, чтобы максимально упростить создание интерфейсных функций, но в оставшихся главах в этой части обсуждаются действительно важные темы. В главе 20 вы узнаете, как хранятся в памяти значения OCaml, а в главе 21 познакомитесь с механизмом автоматического управления памятью.

Библиотека Stypes дает программам на OCaml возможность обращаться к значениям, реализованным на языке C, ограждая вас от тонкостей, связанных с особенностями хранения данных в OCaml, и вводит уровень абстракции, скрывающий детали механизма вызова внешних функций. Хотя она охватывает большинство ситуаций, иногда все же бывает необходимо заглянуть за ширму абстракций, чтобы обеспечить более полное управление взаимодействиями между двумя языками.

Дополнительную информацию об интерфейсе с языком C можно получить в нескольких местах.

- Стандартный интерфейс внешних функций в OCaml позволяет также связывать программный код на OCaml и C с другой стороны границы и создавать функции на языке C, оперирующие значениями на стороне OCaml. Подробное описание стандартного интерфейса можно найти в руководстве по языку OCaml¹ и в книге «Developing Applications with Objective Caml»².
- Флоран Моннье (Florent Monnier) написал великолепный справочник³, где приводятся примеры вызова функций на языке OCaml из программного кода на C. В нем рассматривается обширное множество типов данных OCaml и приводятся более сложные примеры вызовов между C и OCaml.
- SWIG⁴ – инструмент, позволяющий связывать программы, написанные на C/C++, с различными языками высокого уровня, включая OCaml. В руководстве по SWIG имеются примеры преобразования библиотечных спецификаций в интерфейсные функции на OCaml.

Организация структур в памяти

Язык C дает определенную свободу выбора в организации структур в памяти. В структуры могут добавляться пустые промежутки между полями и в конце, чтобы соответствовать требованиям, предъявляемым аппаратной платформой к выравниванию элементов структур. Библиотека Stypes использует платформозависимые правила определения размеров и выравнивания, чтобы восстановить

¹ <http://caml.inria.fr/pub/docs/manual-ocaml-4.00/manual033.html>.

² <http://caml.inria.fr/pub/docs/oreilly-book/ocaml-ora-book.pdf>.

³ <http://www.linux-nantes.org/~fmonnier/ocaml/ocaml-wrapping-c.html>.

⁴ <http://www.swig.org/>.

организацию структур в памяти. Программный код на OCaml и C будет получать одинаковые представления об организации структур, при условии что поля структур в описаниях по обе стороны следуют в одном и том же порядке.

Однако такой подход может вызывать сложности, если поля структуры не полностью определены в интерфейсе библиотеки. Интерфейс может перечислять поля без соблюдения порядка их следования, или делать доступными некоторые поля только на определенных платформах, или добавлять недокументированные поля из соображений производительности. Например, определение `struct timeval`, использовавшееся в этой главе, точно описывает организацию структуры для большинства платформ, но реализации для некоторых необычных архитектур могут включать дополнительные члены, что будет приводить к странностям в поведении примеров.

Для решения этой проблемы был создан пакет `Cstubs`, вложенный в библиотеку `Ctypes`. Вместо того чтобы строить предположения, исходя из определений структур, указанных пользователем и отражающих фактические определения структур в библиотеках на C, пакет `Cstubs` генерирует код, который использует заголовочные файлы на языке C для точного определения организации структур. Одно из достоинств такого подхода заключается в том, что вам практически не придется изменять свой код. Пакет `Cstubs` предоставляет альтернативные реализации функций `field` и `seal`, которые мы уже использовали для описания структуры `struct timeval`; вместо вычисления смещений и размеров членов структуры для текущей платформы эти реализации получают их непосредственно из языка C.

Описание пакета `Cstubs` можно найти в электронной документации¹, где также приводятся инструкции по интеграции с *autoconf*.

¹ <http://ocaml-labs.github.io/ocaml-ctypes/>.

Представление значений в памяти

Интерфейс FFI, с которым мы познакомились в главе 19, скрывает детали обмена данными между библиотеками на языке C и средой выполнения OCaml. На то есть веская причина: непосредственное использование этого интерфейса является весьма деликатным делом и требует полного понимания некоторых механизмов, вовлеченных в работу. Прежде всего необходимо знать организацию данных OCaml в памяти во время выполнения. Кроме того, необходимо гарантировать правильность взаимодействий вашего кода с механизмом управления динамической памятью.

Однако знание внутренних особенностей OCaml может пригодиться не только при использовании интерфейса внешних функций. Когда вы начнете создавать более сложные приложения на OCaml, вам обязательно придется пользоваться различными внешними системными инструментами. Например, инструменты профилирования выводят отчеты, опираясь на организацию памяти во время выполнения, а отладчики выполняют двоичные файлы, вообще ничего не зная о статических типах OCaml. Для эффективного использования этих инструментов необходимо иметь представление о том, как выполняются некоторые преобразования между мирами OCaml и C.

К счастью, инструменты OCaml отличаются весьма предсказуемым поведением. Компилятор старается избегать волшебства оптимизации и вместо этого полагается на линейную модель выполнения. С обретением опыта вы сможете быстро обнаруживать блоки кода, где программа на OCaml проводит большую часть времени.

Куда исчезают типы OCaml во время выполнения?

Компилятор OCaml выполняет компиляцию в несколько этапов. На первом этапе выполняется проверка синтаксиса и конструируется абстрактное синтаксическое дерево (Abstract Syntax Tree, AST). На следующем этапе выполняется проверка типов в AST. В правильно построенной программе функция не должна применяться к значению ошибочного типа. Например, функции `print_endline` должен передаваться единственный строковый аргумент, а попытка передать целое число должна приводить к ошибке.

Так как OCaml проверяет типы на этапе компиляции, ему не нужно помнить большой объем информации во время выполнения. Поэтому на более поздних этапах компилятор может упростить объявления типов до минимально необходимого подмножества, действительно необходимого для поддержания полиморфных значений во время выполнения. В этом заключается основной выигрыш в производительности в сравнении с такими языками, как Java или .NET, где для вызова метода во время выполнения не-

обходимо найти конкретный экземпляр объекта и выполнить вызов требуемого метода. Дороговизна такого способа вызова метода в этих языках частично компенсируется действиями динамического компилятора (Just-in-Time), но в OCaml предпочтение было отдано упрощению на этапе выполнения.

Подробнее о процессе компиляции мы расскажем в главах 22 и 23.

В данной главе мы узнаем, как отображаются типы OCaml в значения времени выполнения, и пройдемся по ним в интерактивной оболочке. В главе 21 вы также узнаете, как реализовано управление этими значениями во время выполнения.

Блоки и значения OCaml

Для представления значений во время выполнения, таких как кортежи, записи, замыкания или массивы, OCaml использует блоки памяти (то есть непрерывные последовательности слов в ОЗУ). Программа неявно выделяет эти блоки памяти в момент создания таких значений:

OCaml utop

https://github.com/realworldocaml/examples/tree/v1/code/memory-repr/simple_record.topscrip

```
# type t = { foo: int; bar: int } ;;
type t = { foo : int; bar : int; }
# let x = { foo = 13; bar = 14 } ;;
val x : t = {foo = 13; bar = 14}
```

Объявление типа `t` не занимает ни байта памяти во время выполнения, но последующая инструкция `let` выделяет новый блок памяти с двумя словами. Одно слово хранит значение поля `foo`, а другое — значение поля `bar`. Компилятор OCaml транслирует такие выражения в явные инструкции выделения блоков памяти.

В OCaml используется однородное представление памяти (*uniform memory representation*), в котором каждая переменная OCaml хранится как *значение*. Значение в OCaml — это одно слово в памяти, которое является либо целочисленным значением, либо указателем на некоторую другую область в памяти. Среда выполнения OCaml следит за всеми значениями, благодаря чему она может освобождать их сразу же, как только они выходят из употребления. Как следствие она должна различать целые числа и указатели, используемые для поиска других значений, чтобы не пытаться интерпретировать целые числа как указатели.

Различение целых чисел и указателей во время выполнения

Обертывание простых типов (таких как целые числа) структурами данных, хранящими дополнительные метаданные с информацией об обернутом значении, называется *упаковкой* (*boxing*). Упаковка удешевляет сборку мусора, но она же удорожает доступ к данным, хранящимся в упакованных значениях из-за наличия дополнительного уровня косвенности.

В языке OCaml не все значения подвергаются упаковке во время выполнения. Для различения целых чисел и указателей во время выполнения используется

единственный битовый признак в слове. Значение считается целым числом, если младший бит в слове заголовка блока не равен нулю, в противном случае значение интерпретируется как указатель. Некоторые типы OCaml – включая `bool`, `int`, пустой список, `unit` и варианты без конструкторов – отображаются непосредственно в целочисленное представление.

Такой подход означает, что целые числа хранятся как неупакованные значения и для них не требуется выделять блоки памяти. Они могут передаваться непосредственно другим функциям в регистрах процессора и вообще являются самыми дешевыми и быстрыми в обработке значениями.

Значение интерпретируется как указатель, если его младший бит равен нулю. Указатель при этом может храниться в неизменном виде, поскольку гарантирует, что указатели ссылаются только на адреса на границах слов (то есть младший бит указателя всегда равен нулю).

Единственная проблема указателей – различие указателей на значения OCaml (по которым следует сборщик мусора) и указателей на значения C (которые не должны обслуживаться сборщиком мусора).

Однако все гораздо проще, чем могло бы показаться, потому что среда выполнения следит за адресами блоков памяти, выделенных для хранения значений OCaml. Если указатель ссылается на адрес внутри блока динамической памяти, помеченного как управляемый средой выполнения OCaml, считается, что он указывает на значение OCaml. Если указатель ссылается за пределы области памяти, занятой средой выполнения OCaml, он интерпретируется как указатель на некоторый системный ресурс.

Немного о выравнивании указателей OCaml по границам слов

Осторожный читатель может задаться вопросом: как OCaml гарантирует выравнивание всех указателей по границам слов? В прошлом, когда RISC-процессоры, такие как Sparc, MIPS и Alpha, имели большое распространение, доступ к памяти не по границам слов запрещался на аппаратном уровне, а подобные попытки могли приводить к аппаратным исключениям и безусловному завершению программы. То есть исторически все указатели округлялись до границы адреса аппаратного слова (обычно имеющего размер 32 или 64 бита).

Современные CISC-процессоры, такие как Intel x86, поддерживают доступ к адресам не по границам слов, но они все равно работают намного быстрее, если обращение к памяти производится по границам слов. Поэтому компилятор OCaml просто округляет все указатели, гарантируя тем самым обнуление одного-двух младших битов в любых допустимых указателях. Установка младшего бита в ненулевое значение – простой способ пометить значение как целочисленное, хотя и за счет потери значимого разряда.

Еще более осторожный читатель наверняка обеспокоится вопросом производительности арифметики с целыми числами. Так как целые числа в OCaml снабжаются дополнительным признаком в младшем бите, перед выполнением любой операции необходимо выполнять сдвиг вправо, чтобы восстановить «правильное» значение. Да, это так, но компилятор OCaml генерирует высокоэффективный ассемблерный код x86, использующий инструкции современных процессоров для сокрытия дополнительных сдвигов везде, где только возможно. Например, сложение таких чисел выполняется единственной инструкцией `LEA`, вычитание может быть выполнено двумя инструкциями, а умножение – всего парой инструкций больше.

Блоки и значения

Блок в OCaml – это элементарная единица памяти в куче (heap). Блок включает одно слово заголовка (32 или 64 бита, в зависимости от аппаратной архитектуры), за которым следует область памяти переменной длины, хранящая либо неинтерпретируемые двоичные данные, либо массив *полей*. Заголовок содержит многоцелевой байт признаков, определяющих порядок интерпретации данных.

Механизм сборки мусора никогда не исследует неинтерпретируемые, двоичные данные. Если установлен признак, что данные являются массивом полей OCaml, их содержимое обрабатывается как допустимые значения OCaml. Механизм сборки мусора всегда исследует поля в процессе сборки мусора.



Рис. 20.1 ❖ Структура блока памяти

Поле размера хранит длину блока памяти в словах. Это поле имеет размер 22 бита на 32-разрядных архитектурах, что объясняет ограничение в 16 Мбайт на длину строк в OCaml на этих архитектурах. Если вам потребуются более длинные строки, либо переходите на 64-разрядную архитектуру, либо используйте модуль `Bigarray`.

2-битное поле цвета используется механизмом сборки мусора для хранения информации о своем состоянии на этапе «маркировки и чистки». Мы еще вернемся к этому полю в главе 21. Это поле недоступно исходному коду на OCaml.

Байт признаков служит нескольким разным целям и указывает, какие данные хранит блок: неинтерпретируемые двоичные данные или массив полей. Если этот байт содержит значение, большее или равное `No_scan_tag` (251), все данные в блоке считаются неинтерпретируемыми двоичными данными и не исследуются сборщиком мусора. Чаще всего такие блоки имеют тип `string`, о чем подробнее рассказывается далее в этой главе.

Точное представление значений внутри блока зависит от статического типа OCaml. Все типы в OCaml упрощаются до значений, перечисленных в табл. 20.1.

Таблица 20.1. Значения OCaml

Значение OCaml	Представление
<code>int</code> или <code>char</code>	Непосредственное значение со сдвигом влево на 1 разряд / Младший значащий бит установлен в 1
<code>unit</code> , <code>[]</code> , <code>false</code>	Как значение <code>int 0</code>
<code>true</code>	Как значение <code>int 1</code>
<code>Foo Bar</code>	Как возрастающие значения <code>int</code> , начиная с 0
<code>Foo Bar of int</code>	Варианты с параметрами являются упакованными, а варианты без параметров – неупакованными

Таблица 20.1 (окончание)

Значение OCaml	Представление
Полиморфные варианты	Область переменной длины в зависимости от числа параметров
Вещественные числа	Как блок с единственным полем, содержащим вещественное число двойной точности
Строки	Массив байтов с явным значением длины
[1; 2; 3]	Как 1::2::3::[], где [] представлен как int, а h::t – блок с признаком 0 и двумя параметрами
Кортежи, записи и массивы	Массив значений. Массивы могут иметь переменный размер, а кортежи и записи – фиксированный
Записи или массивы, в которых все элементы являются вещественными числами	Специальный признак неупакованного массива вещественных чисел или записи, включающей только поля типа float

Целые числа, символы и другие простые типы

Значения многих простых типов эффективнее хранить в неупакованном виде. Наиболее ярким их представителем является тип `int`, который, впрочем, теряет один бит, что отводится под признак. Другие значения атомарных типов, такие как `unit` и пустой список `[]`, хранятся в виде целочисленных констант. Булевы (логические) значения выражаются числами 1 и 0 – `true` и `false` соответственно.

Эти простые типы, такие как пустые списки и `unit`, весьма эффективны в использовании, потому что целые числа никогда не сохраняются в куче. Они могут передаваться функциям не на стеке, а в регистрах процессора, если, конечно, функция имеет не слишком много параметров. Современные аппаратные архитектуры, такие как `x86_64`, имеют множество запасных регистров, что еще больше увеличивает эффективность использования неупакованных целых чисел.

Кортежи, записи и массивы

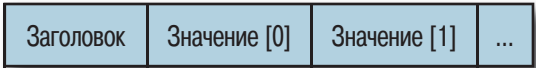


Рис. 20.2 ❖ Структура блока памяти, занимаемой кортежем, записью или массивом

Кортежи, записи и массивы имеют идентичное представление в памяти во время выполнения – в виде блока со значением признака 0. Кортежи и записи имеют постоянный размер, определяемый на этапе компиляции, тогда как массивы могут изменяться в размерах. С другой стороны, массивы могут хранить только элементы одного типа, однако это ограничение не связано с представлением данных в памяти.

Увидеть различия между блоками и целочисленными значениями можно с помощью модуля `Obj`, который экспортирует внутреннее представление значений в код OCaml:

OCaml utop

<https://github.com/realworldocaml/examples/blob/v1/code/memory-repr/reprs.topscript>

```
# Obj.is_block (Obj.repr (1,2,3)) ;;
- : bool = true
# Obj.is_block (Obj.repr 1) ;;
- : bool = false
```

Функция `Obj.repr` возвращает представление времени выполнения любого значения OCaml. `Obj.is_block` проверяет младший бит, чтобы определить, является ли значение заголовком блока или упакованным целочисленным значением.

Вещественные числа и массивы

Вещественные числа в OCaml всегда хранятся как вещественные числа двойной точности. Отдельные вещественные числа хранятся как блоки с единственным полем, содержащим само число. Такой блок имеет значение `Double_tag` в байте признаков, сигнализирующее сборщику мусора, что это значение не требуется исследовать:

OCaml utop (part 1)

<https://github.com/realworldocaml/examples/blob/v1/code/memory-repr/reprs.topscript>

```
# Obj.tag (Obj.repr 1.0) ;;
- : int = 253
# Obj.double_tag ;;
- : int = 253
```

Так как каждое вещественное число упаковано в отдельный блок памяти, эффективность обработки больших массивов вещественных чисел может оказаться значительно ниже в сравнении с массивами упакованных целых чисел. Поэтому в OCaml предусмотрено специальное представление для записей и массивов, содержащих только вещественные числа. Они организованы в виде блоков памяти, хранящих массивы вещественных чисел в упакованном виде. Такие блоки отмечены байтом признаков со значением `Double_array_tag`, сигнализирующим сборщику мусора, что этот блок не содержит значений OCaml.

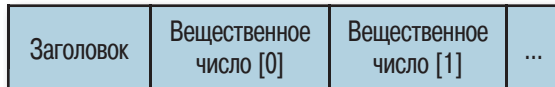


Рис. 20.3 ❖ Структура блока памяти, занимаемой массивом вещественных чисел

Для начала проверим, действительно ли массивы вещественных чисел снабжаются другим значением признака, отличным от признака, которым помечаются обычные вещественные значения:

OCaml utop (part 2)

<https://github.com/realworldocaml/examples/blob/v1/code/memory-repr/reprs.topscript>

```
# Obj.double_tag ;;
- : int = 253
```

```
# Obj.double_array_tag ;;
- : int = 254
```

Как видите, массивы вещественных чисел имеют признак со значением 254. А теперь исследуем некоторые типовые значения, проверив с помощью функции `Obj.tag` признаки для некоторых видов блоков, и попробуем использовать `Obj.double_field` для извлечения вещественного числа из блока:

OCaml utop (part 3)

<https://github.com/realworldocaml/examples/blob/v1/code/memory-repr/reprs.topscript>

```
# Obj.tag (Obj.repr [| 1.0; 2.0; 3.0 |]) ;;
- : int = 254
# Obj.tag (Obj.repr (1.0, 2.0, 3.0) ) ;;
- : int = 0
# Obj.double_field (Obj.repr [| 1.1; 2.2; 3.3 |]) 1 ;;
- : float = 2.2
# Obj.double_field (Obj.repr 1.234) 0 ;;
- : float = 1.234
```

Сначала мы проверили, действительно ли массив вещественных чисел имеет признак, соответствующий массиву неупакованных вещественных чисел (254). В следующей строке проверяется кортеж вещественных чисел, и, как можно видеть, он *не* оптимизирован и имеет признак обычного кортежа (0).

Только к записям и массивам применяется оптимизация массивов вещественных чисел.

Варианты и списки

Простые вариантыные типы без дополнительных параметров для любых их ветвей хранятся как целочисленные значения, начиная с 0 для первого варианта и далее в возрастающем порядке:

OCaml utop (part 4)

<https://github.com/realworldocaml/examples/blob/v1/code/memory-repr/reprs.topscript>

```
# type t = Apple | Orange | Pear ;;
type t = Apple | Orange | Pear
# ((Obj.magic (Obj.repr Apple)) : int) ;;
- : int = 0
# ((Obj.magic (Obj.repr Pear)) : int) ;;
- : int = 2
# Obj.is_block (Obj.repr Apple) ;;
- : bool = false
```

Функция `Obj.magic` выполняет простое приведение между двумя типами OCaml. В данном примере указание на тип `int` требует извлечь целочисленное значение из памяти. Функция `Obj.is_block` подтверждает, что значение является простым целым числом, а не более сложным блоком.

Варианты с параметрами устроены сложнее. Они хранятся как блоки, со значениями байтов-признаков, возрастающими от 0 (отсчет начинается с самого левого варианта, имеющего параметр). Параметры хранятся как слова в блоке:

OCaml utop (part 5)

<https://github.com/realworldocaml/examples/blob/v1/code/memory-repr/reprs.topscript>

```
# type t = Apple | Orange of int | Pear of string | Kiwi ;;
type t = Apple | Orange of int | Pear of string | Kiwi
# Obj.is_block (Obj.repr (Orange 1234)) ;;
- : bool = true
# Obj.tag (Obj.repr (Orange 1234)) ;;
- : int = 0
# Obj.tag (Obj.repr (Pear "xyz")) ;;
- : int = 1
# (Obj.magic (Obj.field (Obj.repr (Orange 1234)) 0) : int) ;;
- : int = 1234
# (Obj.magic (Obj.field (Obj.repr (Pear "xyz")) 0) : string) ;;
- : string = "xyz"
```

В этом примере значения Apple и Kiwi хранятся как обычные целые числа со значениями 0 и 1 соответственно. Два других значения — Orange и Pear — имеют параметры и хранятся в виде блоков, с байтами-признаками, имеющими значения, возрастающие от 0 (соответственно, Pear получает значение признака 1, что и подтверждает вызов Obj.tag). Наконец, параметры являются полями, хранящими значения в блоках, для извлечения которых можно использовать Obj.field.

Списки хранятся в представлении, как если бы список был записан в виде вариатного типа со значениями Nil и Cons. Пустой список [] хранится как целое число 0, а последующие блоки получают признак 0 и два параметра: блок с текущим значением и указатель на оставшуюся часть списка.

Используйте модуль Obj с осторожностью

Obj — это недокументированный модуль, экспортирующий внутреннюю организацию компилятора и среды выполнения OCaml. Его с успехом можно использовать для исследования особенностей поведения программного кода во время выполнения, но он никогда не должен применяться в промышленном окружении, если только вы не понимаете всех последствий такого применения. Этот модуль действует в обход системы типов OCaml, и некорректное его использование может стать причиной повреждения данных в памяти и исключений нарушения прав доступа к памяти (segmentation faults). Некоторые программы доказательства теорем, такие как Coq, генерируют код, фактически использующий модуль Obj, но внешние сигнатуры модулей никогда не экспортируют его. Если вы не занимаетесь машинными доказательствами теорем, оправдывающими применение модуля Obj, используйте его исключительно для нужд отладки!

Учитывая такой способ кодирования признаков, легко понять, почему в каждом определении типа может содержаться не более 240 вариантов с параметрами, хотя количество вариантов без параметров ограничивается лишь размером аппаратного целого числа (31 или 63 бита). Это ограничение обусловлено размером байта-признака и тем обстоятельством, что некоторые верхние значения зарезервированы для других нужд.

Полиморфные варианты

Полиморфные варианты более гибкие, чем обычные варианты, но они менее эффективны, с точки зрения производительности. Это обусловлено недостатком статической информации на этапе компиляции, с помощью которой можно было бы оптимизировать размещение значений в памяти.

Полиморфные варианты без параметров хранятся как неупакованные целые и занимают единственное слово в памяти, так же как обычные варианты. Это целочисленное значение определяется применением хэш-функции к *имени* варианта. Хэш-функция не экспортируется компилятором, но вы можете найти альтернативную ее реализацию в библиотеке `type_conv`:

OCaml utop (part 6)

<https://github.com/realworldocaml/examples/blob/v1/code/memory-repr/reprs.topscript>

```
# Pa_type_conv.hash_variant "Foo" ;;
- : int = 3505894
# (Obj.magic (Obj.repr `Foo) : int) ;;
- : int = 3505894
```

Хэш-функция реализована так, что она возвращает одинаковые значения и на 32-разрядных, и на 64-разрядных платформах, поэтому представление в памяти остается неизменным на разных платформах и аппаратных архитектурах.

Полиморфные варианты занимают больше места в памяти, чем обычные варианты, когда в конструкторы типов данных включаются дополнительные параметры. Обычные варианты используют байт-признак для кодирования значения варианта и хранят поля для содержимого, но единственного байта недостаточно для кодирования хэш-значений полиморфных вариантов. Поэтому для них выделяется новый блок памяти (с признаком 0), куда и сохраняется значение. Как следствие полиморфные варианты с конструкторами занимают памяти на одно слово больше, чем обычные варианты с конструкторами.

Эффективность снижается еще больше в сравнении с обычными вариантами, когда конструктор полиморфного варианта имеет более одного параметра. Обычные варианты хранят параметры в единственном блоке с несколькими полями, но для полиморфных вариантов приходится использовать более гибкую схему организации, потому что они могут повторно применяться в разных контекстах в разных единицах компиляции. Для параметров выделяется блок кортежа, элементы которого ссылаются на поля с аргументами варианта. Соответственно, для хранения таких вариантов расходуются еще три слова памяти плюс дополнительная память для хранения ссылки на кортеж.

Вообще говоря, дополнительный расход памяти в типичном приложении оказывается несущественным, зато полиморфные варианты обладают большей гибкостью, чем обычные варианты. Однако если вы пишете код, для которого важна высокая производительность или он должен выполняться в среде с ограниченным объемом памяти, вы будете точно знать, как будут организованы данные в памяти, руководствуясь описанными здесь правилами. Компилятор OCaml никогда не переключается между представлениями с целью оптимизации.

Строковые значения

Строки являются стандартными блоками с заголовком, хранящим размер строки в машинных словах. Значение `String_tag` (252) больше значения `No_scan_tag`, поэтому сборщик мусора не будет анализировать содержимое такого блока, где хранятся символы, составляющие строку, с дополнительными байтами, обеспечивающими выравнивание границ блока.



Рис. 20.4 ❖ Структура блока памяти, занимаемой строкой

На 32-разрядных архитектурах количество байтов для выравнивания определяется как остаток от деления длины строки на размер слова. На 64-разрядных архитектурах число байтов для выравнивания может достигать семи, а в 32-разрядных – трех (см. табл. 20.2).

Таблица 20.2. Длина строки и число байтов для выравнивания

Остаток от деления длины строки на 4	Байты для выравнивания
0	00 00 00 03
1	00 00 02
2	00 01
3	00

Такая организация строк в памяти гарантирует, что любая строка будет заканчиваться нулевым байтом и обеспечивает эффективное вычисление длины строки без необходимости сканировать ее содержимое. Упомянутые вычисления выполняются по следующей формуле:

$$\text{длина} = \text{число_слов_в_блоке} * \text{размер_слова} - (\text{последний_байт_в_блоке} - 1)$$

Гарантированное присутствие нулевого символа особенно оценят те, кому требуется передавать строки в программный код на языке C, но учтите, что в C длина строк вычисляется иначе, чем в OCaml. В OCaml строки могут содержать нулевые символы в любом месте, а в C – нет.

В противном случае необходимо использовать функции C, которые способны принимать подобные буферы с произвольным содержимым, а не только C-строки. Например, функции `memcpy` и `memmove` из стандартной библиотеки C способны оперировать произвольными данными, а `strlen` и `strcpy` – только буферами, заканчивающимися нулевым символом, и ни одна из них не поддерживает нулевых символов внутри строк.

Нестандартные блоки памяти

OCaml поддерживает также возможность создания *нестандартных* блоков памяти с признаком `Custom_tag`, давая возможность выполнять над значениями OCaml операции, определяемые пользователем. Нестандартные блоки сохраняются в куче OCaml, подобно обычным блокам, и могут иметь произвольный размер. Признак `Custom_tag` (255) имеет значение больше `No_scan_tag`, поэтому блоки с этим признаком не будут исследоваться сборщиком мусора.

Первое слово данных в нестандартном блоке должно быть указателем на структуру операций. Нестандартные блоки не могут хранить указатели на другие блоки OCaml и не обслуживаются сборщиком мусора:

C

https://github.com/realworldocaml/examples/tree/v1/code/memory-repr/custom_ops.c

```
struct custom_operations {
    char *identifier;
    void (*finalize)(value v);
    int (*compare)(value v1, value v2);
    intnat (*hash)(value v);
    void (*serialize)(value v,
                      /*out*/ uintnat * wsize_32 /*размер в байтах*/,
                      /*out*/ uintnat * wsize_64 /*размер в байтах */);
    uintnat (*deserialize)(void * dst);
    int (*compare_ext)(value v1, value v2);
};
```

Операции определяют, как среда выполнения должна производить полиморфное сравнение, хэширование и двоичный маршалинг. В число операций может также входить необязательная функция *финализации*, которая будет вызываться средой выполнения непосредственно перед утилизацией блока. Нестандартные функции финализации не имеют ничего общего с обычными функциями финализации в OCaml (которые создаются функцией `Gc.finalize` и описываются в главе 21). Они часто используются для вызова функций освобождения ресурсов на языке C, таких как `free`.

Управление внешней памятью средствами Bigarray

Часто нестандартные блоки применяются для управления внешней памятью непосредственно из программного кода на OCaml. Первоначально интерфейс `Bigarray` предназначался для обмена данными с программным кодом на Fortran и представления блока системной памяти как многомерного массива, к которому можно обращаться из OCaml. Операции, поддерживаемые интерфейсом `Bigarray`, воздействуют непосредственно на внешнюю память, без копирования данных в память OCaml (что для больших массивов является довольно дорогим удовольствием).

Интерфейс `Bigarray` можно использовать не только для научных вычислений – в составе `Core` имеется несколько библиотек, использующих его для нужд ввода/вывода:

- `Iobuf` – модуль `Iobuf` отображает буферы ввода/вывода в одномерные массивы байтов. Он реализует интерфейс скользящего окна, дающий возможность процессу-получателю читать данные из буфера прямо в ходе его заполнения процессом-производителем. Это позволяет программам на `OCaml` использовать буферы ввода/вывода, выделенные во внешней памяти самой операционной системой, не прибегая к промежуточному копированию данных.
- `Bigstring` – модуль `Bigstring` реализует интерфейс, напоминающий модуль `String`, но использующий `Bigarray`. Он конструирует расширяемые строковые буферы, которые могут целиком храниться во внешней системной памяти.
- `Lacaml` – библиотека `Lacaml`¹ не является частью библиотеки `Core`, но предоставляет интерфейсы для популярных математических библиотек `BLAS` и `LAPACK`, написанных на языке `Fortran`. Она позволяет разработчикам писать высокопроизводительные вычислительные приложения, использующие методы линейной алгебры. Она поддерживает векторы и матрицы большого размера, а статическая система типов языка `OCaml` упрощает написание надежных алгоритмов.

¹ <https://bitbucket.org/mmottl/lacaml>.

Сборка мусора

В главе 20 мы познакомились с особенностями хранения в памяти отдельных переменных OCaml. После запуска программы среда выполнения OCaml управляет жизненным циклом этих переменных, периодически сканируя их и утилизируя, когда они становятся ненужными. Это, в свою очередь, означает отсутствие необходимости управлять памятью вручную, что существенно снижает вероятность утечек памяти в вашем коде.

Среда выполнения OCaml – это библиотека, написанная на языке C, которая предоставляет подпрограммы для вызова из программ на языке OCaml. Среда выполнения управляет кучей (областью динамической памяти), которая представляет собой коллекцию блоков памяти, полученных от операционной системы. Она использует ее для хранения блоков, заполненных значениями OCaml, созданными в процессе работы программы.

Алгоритм сборки мусора

Когда памяти оказывается недостаточно, чтобы удовлетворить запрос на выделение нового блока, среда времени выполнения вызывает сборщика мусора (Garbage Collector, GC). Программы на языке OCaml не могут явно освобождать значения, закончив работу с ними. Вместо этого сборщик мусора регулярно проверяет, какие значения продолжают использоваться, а какие вышли из употребления и их можно удалить. Неиспользуемые значения собираются вместе, и занимаемая ими память освобождается для повторного использования приложением.

Сборщик мусора не непрерывно следит за значениями, созданными программой, а проверяет их через регулярные интервалы времени, начиная со множества *корневых* значений, к которым у приложения всегда есть доступ (таким как стек). Сборщик мусора обслуживает ориентированный граф, в котором роль узлов играют блоки, а роль ребер – указатели, как, например, в случае, когда поле в блоке `b1` ссылается на блок `b2`.

Все блоки, доступные из корневых значений, при следовании к ним по ребрам должны оставаться в неприкосновенности, а недостижимые блоки можно освободить для повторного использования. Алгоритм, используемый средой выполнения OCaml для обхода значений в куче, часто называют алгоритмом маркировки и чистки (mark and sweep), и мы расскажем о нем далее.

Сборка мусора с разделением на поколения

Обычно программы на языке OCaml создают множество маленьких переменных, которые используются короткий промежуток времени и затем покидают область видимости навсегда. Разработчики OCaml учли это обстоятельство и, чтобы повысить производительность сборщика мусора, реализовали алгоритм с разделением переменных на поколения.

Сборщик мусора поддерживает несколько непересекающихся областей памяти для хранения блоков, размещая их в этих областях в зависимости от продолжительности жизни. Если говорить более конкретно, куча OCaml делится на две такие области:

- маленькая вспомогательная куча фиксированного размера, куда первоначально помещается большинство блоков;
- большая основная куча переменного размера, куда помещаются блоки-долгожители.

В типичном функциональном стиле программирования предполагается, что «молодые» блоки имеют тенденцию «умирать» в юности, а «старые» блоки «живут» долго. Это предположение часто называют *гипотезой о поколениях* (*generational hypothesis*).

В OCaml используются разные организации памяти и алгоритмы обслуживания основной и вспомогательной куч, учитывающие различия между поколениями. Далее мы подробно объясним суть этих различий.

Модуль Gc и переменная OCAMLRUNPARAM

OCaml поддерживает несколько механизмов определения и изменения поведения среды выполнения. Модуль Gc предоставляет возможность взаимодействовать с этими механизмами из программного кода на OCaml, и мы часто будем упоминать его в оставшейся части главы. По аналогии с некоторыми другими модулями библиотека Core несколько изменяет интерфейс модуля Gc из стандартной библиотеки OCaml. В своих пояснениях далее мы будем предполагать, что вы открыли Core.Std.

Управлять поведением среды выполнения OCaml можно также посредством переменной окружения OCAMLRUNPARAM, устанавливая в ней требуемые параметры перед запуском приложения. В ней можно определять параметры настройки сборщика мусора, например чтобы исследовать их влияние на производительность программы. Формат значения переменной OCAMLRUNPARAM описан в руководстве по языку OCaml: <http://caml.inria.fr/pub/docs/manual-ocaml-4.01/runtime.html#sec255>.

Быстрая вспомогательная куча

Вспомогательная куча – место, где хранится большинство короткоживущих объектов. Это одна непрерывная область виртуальной памяти, содержащей последовательность блоков со значениями OCaml. Если в этой куче есть свободное место, запрос на выделение памяти для нового блока удовлетворяется очень быстро – за фиксированное время, за счет выполнения всего пары машинных инструкций.

В процессе сборки мусора во вспомогательной куче OCaml *перемещает* все «живые» блоки из вспомогательной кучи в основную. Эта работа выполняется за

период времени, пропорциональный числу перемещаемых блоков, которое обычно невелико, согласно гипотезе о поколениях. Сборка мусора во вспомогательной куче *останавливает вращение Земли* (то есть приостанавливает выполнение приложения), пока она не закончится, именно поэтому так важно, чтобы сборка мусора протекала максимально быстро.

Выделение памяти во вспомогательной куче

Вспомогательная куча – это непрерывная область виртуальной памяти размером обычно всего в несколько мегабайт, чтобы ее можно было быстро просканировать.

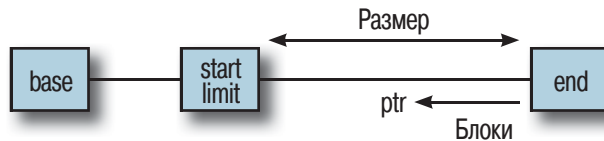


Рис. 21.1 ❖ Организация вспомогательной кучи

Начало и конец вспомогательной кучи среда выполнения хранит в двух указателях: на начало кучи ссылается указатель `caml_young_start`, а на конец – указатель `caml_young_end` (в дальнейших объяснениях мы будем опускать префикс `caml_young` для краткости). Указатель `base` – это адрес в памяти, возвращаемый системной функцией `malloc`, а `start` – адрес ближайшей к `base` границы (такое решение упрощает сохранение значений ОСaml).

Во вновь созданной вспомогательной куче указатель `limit` совпадает с указателем `start`, а указатель `ptr` текущей вершины кучи совпадает с указателем `end`. По мере добавления новых блоков указатель `ptr` уменьшается, пока не достигнет значения указателя `limit`, – в этот момент запускается сборка мусора.

Перед размещением нового блока во вспомогательной куче значение указателя `ptr` уменьшается на размер блока (включая размер слова заголовка), и проверяется, не достиг ли он значения указателя `limit`. Если памяти оказывается недостаточно, запускается сборка мусора. Она производится очень быстро (алгоритм почти не содержит ветвлений).

У кого-то может появиться вопрос: «Зачем нужен указатель `limit`, если похоже, что он всегда совпадает с указателем `start`?» Дело в том, что он упрощает принудительный запуск сборки мусора во вспомогательной куче, так как для этого среде выполнения достаточно всего лишь присвоить ему значение указателя `end` – при первой же попытке выделить память будет обнаружена ее нехватка и произойдет вызов сборщика мусора. Принудительная сборка может потребоваться по самым разным причинам, например при обработке сигналов UNIX.

Установка размера вспомогательной кучи

Размер вспомогательной кучи по умолчанию устанавливается равным 2 Мбайт на 64-разрядных платформах, но может быть увеличен до 8 Мбайт при использовании библиотеки `Core` (которая обычно изменяет настройки по умолчанию для достижения большей производительности, хотя и ценой большего потребления памяти). Эту на-

строjkу можно переопределить с помощью параметра `s=<число_слов>` в переменной окружения `OCAMLRUNPARAM`. После запуска программы это значение можно изменить вызовом функции `Gc.set`:

OCaml utop

<https://github.com/realworldocaml/examples/blob/v1/code/gc/tune.topscript>

```
# let c = Gc.get () ;;
val c : Gc.control =
  {Core.Std.Gc.Control.minor_heap_size = 1000000;
   major_heap_increment = 1000448; space_overhead = 100; verbose = 0;
   max_overhead = 500; stack_limit = 1048576; allocation_policy = 0}
# Gc.tune ~minor_heap_size:(262144 * 2) () ;;
- : unit = ()
```

Изменение размера вспомогательной кучи немедленно вызывает принудительную сборку мусора в ней. Обратите внимание, что библиотека `Core` увеличивает размер вспомогательной кучи весьма существенно в сравнении со стандартными настройками, поэтому при работе в среде с ограниченными ресурсами вам может потребоваться уменьшить этот размер.

Основная куча долгоживущих блоков

Основная куча — место, где хранится основная масса долгоживущих и объемных значений. Она состоит из произвольного числа фрагментов виртуальной памяти, каждый из которых хранит блоки попеременно с областями свободной памяти. Среда выполнения поддерживает список свободных фрагментов, в котором хранятся адреса всех свободных участков памяти, которые выделяются в ответ на запросы программы.

Основная куча обычно намного больше вспомогательной и может достигать нескольких гигабайтов в размерах. Сборка мусора в ней производится с применением алгоритма маркировки и чистки, состоящего из нескольких этапов:

- на этапе *маркировки* (*mark phase*) производится обход блоков в графе, и все «живые» блоки помечаются установкой бита в байте-признаке заголовка блока (этот признак называют *цветом* (*color*));
- на этапе *чистки* (*sweep phase*) последовательно просматриваются все фрагменты кучи и выявляются «умершие» (то есть вышедшие из употребления) блоки, которые не были помечены на предыдущем этапе;
- на этапе *компактификации* (*compact phase*) производится перемещение блоков во вновь выделенную кучу с целью избавиться от промежутков свободной памяти между занятыми блоками — этот этап предотвращает фрагментацию кучи в долгоживущих программах и обычно выполняется гораздо реже этапов маркировки и чистки.

В процессе сборки мусора в основной куче также приходится приостанавливать работу приложения, чтобы процедура перемещения блоков не оказала отрицательного влияния на работу программы. Этапы маркировки выполняются инкрементально, на небольших областях кучи, чтобы не вызывать длительных пауз в работе приложения, и перед сканированием каждой области предварительно

производится чистка вспомогательной кучи. Только этап компактификации выполняется для всей кучи целиком, поэтому он выполняется достаточно редко.

Выделение памяти в основной куче

Поддержка основной кучи осуществляется с помощью единого связанного списка непрерывных фрагментов памяти, отсортированного по их виртуальным адресам. Каждый фрагмент – это отдельная область памяти, выделенная вызовом функции *malloc(3)* и состоящая из заголовка и области данных, в которой содержатся блоки со значениями OCaml. Заголовок фрагмента включает в себя следующую информацию:

- виртуальный адрес фрагмента;
- размер области данных в байтах;
- размер в байтах, используемый на этапе компактификации для слияния маленьких блоков;
- ссылка на следующий фрагмент в списке.

Область данных каждого фрагмента начинается на границе страницы, а ее размер кратен размеру страницы (4 Кбайта). Она содержит непрерывную последовательность блоков размером в одну-две страницы, но обычно размер блоков составляет 1 Мбайт (или 512 Кбайт на 32-разрядных архитектурах).

Управление ростом основной кучи

Для управления ростом основной кучи модуль *Gc* использует значение *major_heap_increment*. Это значение определяет число слов, на которое увеличивается основная куча. Это – единственная операция выделения памяти, которая наблюдается операционной системой со стороны среды выполнения OCaml после запуска приложения (так как вспомогательная куча имеет фиксированный размер).

Если предполагается, что программа будет выделять память для очень больших значений или большого количества маленьких значений за короткий промежуток времени, можно увеличить это значение, чтобы повысить производительность за счет уменьшения числа операций перераспределения памяти для кучи. Маленькое значение может привести к созданию большого числа небольших фрагментов кучи, разбросанных по разным областям виртуальной памяти, и потребовать выполнения большего числа операций для поддержания их списка:

OCaml *utop* (part 1)

<https://github.com/realworldocaml/examples/blob/v1/code/gc/tune.topscript>

```
# Gc.tune ~major_heap_increment:(1000448 * 4) () ;;
- : unit = ()
```

При выделении памяти в основной куче для значения сначала проверяется список свободных блоков на наличие области, подходящей по размеру. Если такая область отсутствует, среда выполнения увеличивает размер основной кучи, распределяя фрагмент достаточно большого размера. Этот фрагмент затем добавляется в список свободных областей, и попытка выделить память выполняется снова (и на этот раз успешно).

Не забывайте, что размещение новых блоков в основной куче выполняется при их перемещении из вспомогательной кучи, и только если они «выживают» после сборки мусора во вспомогательной куче. Единственное исключение составляют

значения размером больше 256 слов (то есть больше 2 Кбайт на 64-разрядных платформах). Такие значения сразу помещаются в основную кучу, поскольку размещение подобного блока во вспомогательной куче вызвало бы сборку мусора в ней и копирование этого блока в основную кучу.

Стратегии распределения памяти

Реализация поддержки основной кучи оптимизирована до предела и опирается на процедуру компактификации, гарантирующую отсутствие сильной фрагментации памяти. Политика выделения памяти, используемая по умолчанию, прекрасно подходит для большинства приложений, но следует знать, что существуют и другие варианты.

Список свободных блоков всегда проверяется перед распределением нового блока в основной куче. По умолчанию поиск в списке свободных блоков ведется в соответствии со стратегией *следующего подходящего соответствия*, однако доступна также стратегия *первого подходящего совпадения*.

Стратегия следующего подходящего совпадения

При использовании стратегии следующего подходящего совпадения среда выполнения сохраняет указатель на блок в списке свободных блоков, который был использован последним. Когда поступает новый запрос на выделение памяти, среда выполнения начинает искать следующий блок, двигаясь в направлении конца списка, и затем, при необходимости, продолжает поиск с начала списка.

Стратегия следующего подходящего совпадения используется по умолчанию. Она является достаточно недорогой, поскольку позволяет снова и снова использовать один и тот же блок в потоке запросов. Дополнительным преимуществом этой стратегии является автоматическая оптимизация использования внутреннего кэша процессора.

Стратегия первого подходящего совпадения

Если в процессе выполнения программы выделяются блоки для значений самых разных размеров, список свободных блоков может оказаться слишком фрагментированным. В этой ситуации сборщик мусора вынужден будет тратить значительное время на компактификацию, несмотря на наличие большого числа свободных фрагментов, из-за того что ни один не подходит по размеру, чтобы удовлетворить запрос.

Основной целью стратегии первого подходящего совпадения является уменьшение фрагментации памяти (и, соответственно, числа операций компактификации), но она более медлительна. Для каждого запроса приходится просматривать список свободных блоков с самого начала, в поисках подходящего свободного фрагмента, вместо того чтобы использовать последний использовавшийся, как предусматривает стратегия следующего подходящего совпадения.

В некоторых ситуациях, когда требуется высокая производительность, выгоды от уменьшения частоты компактификации кучи перевешивают более высокую стоимость операции выделения памяти.

Управление политикой распределения памяти

Изменить политику распределения памяти можно с помощью поля `Gc.allocation_policy`. Значение 0 (по умолчанию) активирует стратегию следующего подходящего совпадения, а значение 1 – стратегию первого подходящего совпадения.

Того же эффекта можно добиться, определяя параметры `a=0` и `a=1` в переменной окружения `OCAMLRUNPARAM`.

Маркировка и сканирование кучи

Процесс маркировки всей кучи целиком может занимать продолжительное время и вызывать заметные паузы в работе приложения. Поэтому маркировка выполняется инкрементально, то есть куча маркируется фрагментами. Каждое значение в куче имеет 2-битное поле *цвета*, в котором хранится информация о том, проверялось ли это значение сборщиком мусора, чтобы ему проще было возобновить сканирование после перерыва.

Таблица 21.1. Значения поля цвета

Цвет	Значение
Синий	В списке свободных блоков и в данный момент не используется
Белый (на этапе маркировки)	Пока недостижим, но, возможно, будет определен как достижимый
Белый (на этапе чистки)	Недостижим и может быть освобожден
Серый	Достижим, но поля пока не проверены
Черный	Достижим, и поля проверены

Признак цвета в заголовках хранит большую часть информации о процессе маркировки, что дает возможность приостанавливать его и возобновлять позднее. Фактически в процессе маркировки чередуется выполнение сборщика мусора и приложения. Среда выполнения OCaml вычисляет размер фрагмента для маркировки, опираясь на частоту следования операций распределения памяти и объем доступной памяти.

Процесс маркировки запускается со множества *корневых* значений, которые никогда не выходят из употребления (таких как стек приложения). Все значения в куче первоначально помечены как «белые, возможно достижимые, но еще не проверенные». Затем сборщик мусора рекурсивно обходит все поля в корневых значениях с поиском «сначала в глубину» и вталкивает вновь найденные белые блоки в промежуточный стек *серых блоков* на время, пока занимается исследованием их полей. После обхода всех полей исследованное значение получает черный цвет и выталкивается со стека.

Этот процесс продолжается, пока стек серых значений не опустеет и не останется значений для исследования. Однако в этом процессе есть одно краевое состояние. Стек серых значений может расти только до определенного размера, после чего сборщик не может выполнять рекурсивного обхода промежуточных значений, так как их нигде будет сохранять на время обхода их полей. В этом случае куча отмечается как «грязная», и после обработки имеющихся серых значений инициализируется более дорогостоящая проверка.

Чтобы пометить кучу как «грязную», сборщик мусора сначала помечает ее как «чистую» и обходит ее полностью, блок за блоком, в порядке возрастания адресов. Обнаружив серый блок, он добавляет его в список серых блоков и рекурсивно помечает все блоки, на которые он ссылается, используя обычную стратегию для «чистой» кучи. По завершении обхода кучи этап маркировки повторяется снова, и проверяется, стала ли куча снова «грязной». Этот процесс продолжается, пока куча не станет чистой. Полное сканирование кучи продолжается, пока сканирование не завершится успехом, без переполнения стека серых блоков.

Управление чисткой основной кучи

Вы можете вручную инициировать обход одного фрагмента основной кучи вызовом `major_slice`. При этом сначала будет выполнена сборка мусора во вспомогательной куче, а затем произведен обход фрагмента основной кучи. Размер фрагмента обычно вычисляется автоматически самим сборщиком мусора и возвращается, чтобы вы могли изменить его в будущем:

OCaml utop (part 2)

<https://github.com/realworldocaml/examples/blob/v1/code/gc/tune.topscript>

```
# Gc.major_slice 0 ;;
- : int = 260440
# Gc.full_major () ;;
- : unit = ()
```

Параметр `space_overhead` управляет агрессивностью сборщика мусора в отношении выбора размера фрагмента, исследуемого за один проход. Он определяет пропорцию объема памяти для хранения «живых» данных, которая может быть «потрачена впустую» из-за того, что сборщик мусора будет откладывать утилизацию недостижимых блоков. Библиотека Core устанавливает значение 100, соответствующее типичной системе, не испытывающей острой нехватки памяти. Установите большее значение, если в вашей системе большой объем памяти, или меньшее, чтобы заставить сборщик мусора трудиться усерднее и утилизировать освободившиеся блоки чаще за счет большего потребления процессорного времени.

Компактификация кучи

С каждым циклом работы сборщика мусора может увеличиваться фрагментация кучи из-за того, что блоки будут утилизироваться не в том порядке, в каком создавались. Фрагментация затрудняет поиск непрерывных блоков памяти для удовлетворения последующих запросов на выделение памяти и может вызывать неоправданное увеличение размера кучи.

Проведение компактификации кучи помогает избежать этого за счет перемещения всех значений, находящихся в куче, в свежую кучу и расположения их по порядку. Безыскусная реализация алгоритма компактификации могла бы потребовать выделения дополнительной памяти под новую (свежую) кучу, однако сборщик мусора в OCaml выполняет компактификацию на месте, внутри существующей кучи.

Управление частотой компактификации

Параметр `max_overhead` в модуле Gc определяет отношение между объемом свободной и занятой памяти, после которого активируется этап компактификации.

Значение 0 вызывает компактификацию после каждого цикла сборки мусора в основной куче, тогда как максимальное значение 1000000 запрещает компактификацию полностью. Значение по умолчанию отлично подходит для большинства ситуаций, если только у вас не используются какие-то необычные схемы выделения памяти, вызывающие компактификацию чаще обычного:

OCaml utop (part 3)

<https://github.com/realworldocaml/examples/blob/v1/code/gc/tune.topscript>

```
# Gc.tune ~max_overhead:0 () ;;
- : unit = ()
```

Указатели между поколениями

Одна из сложностей, связанных со сборкой мусора с учетом поколений, происходит из того, что сборка мусора во вспомогательной куче производится намного чаще, чем в основной куче. Чтобы определить, какие блоки во вспомогательной куче продолжают использоваться, сборщику мусора приходится следить за тем, какие блоки в основной куче ссылаются на блоки во вспомогательной куче. Без этой информации каждый цикл сборки мусора во вспомогательной куче потребовал бы сканирования большой основной кучи.

Чтобы избежать такой зависимости между сборкой мусора в основной и вспомогательной кучах, среда выполнения OCaml поддерживает множество *указателей между поколениями*. Компилятор устанавливает барьер записи, чтобы обновлять это множество всякий раз, когда в значении внутри основной кучи сохраняется указатель на блок во вспомогательной куче.

Барьер записи

Барьер записи может оказывать существенное влияние на структуру программного кода. Это является одной из основных причин использования неизменяемых структур данных, а создание новой копии с изменениями иногда может оказаться более быстрым решением, чем изменение записи на месте.

Компилятор OCaml следит за всеми изменяемыми типами и добавляет в генерируемый им код вызов функции `caml_modify` перед выполнением любых изменений. Она проверяет адрес записи с записываемым значением и гарантирует непротиворечивость хранящегося множества. Несмотря на высокую эффективность реализации барьера записи, иногда он может стать причиной дополнительных циклов сборки мусора и вызывать нежелательное замедление распределения памяти для свежего значения во вспомогательной куче.

Давайте убедимся в этом на примере простой тестовой программы. Перед компиляцией этого кода следует установить комплект инструментов для проведения хронометража, входящий в состав Core, выполнив команду `opam install core_bench`:

OCaml

https://github.com/realworldocaml/examples/blob/v1/code/gc/barrier_bench.ml

```
open Core.Std
open Core_bench.Std
```

```
type t1 = { mutable iters1: int; mutable count1: float }
```

```

type t2 = { iters2: int; count2: float }

let rec test_mutable t1 =
  match t1.iters1 with
  | 0 -> ()
  | _ ->
    t1.iters1 <- t1.iters1 - 1;
    t1.count1 <- t1.count1 +. 1.0;
    test_mutable t1

let rec test_immutable t2 =
  match t2.iters2 with
  | 0 -> ()
  | n ->
    let iters2 = n - 1 in
    let count2 = t2.count2 +. 1.0 in
    test_immutable { iters2; count2 }

let () =
  let iters = 1000000 in
  let tests = [
    Bench.Test.create ~name:"mutable"
      (fun () -> test_mutable { iters1=iters; count1=0.0 });
    Bench.Test.create ~name:"immutable"
      (fun () -> test_immutable { iters2=iters; count2=0.0 })
  ] in
  Bench.make_command tests |> Command.run

```

Эта программа определяет изменяемый тип `t1` и неизменяемый тип `t2`. Испытательный цикл выполняет обход обоих полей и наращивает счетчик. Скомпилируем и выполним эту программу с дополнительными параметрами, чтобы увидеть, сколько циклов сборки мусора было произведено в ходе выполнения:

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/gc/run_barrier_bench.out

```

$ corebuild -pkg core_bench barrier_bench.native
$ ./barrier_bench.native -ascii name alloc
Estimated testing time 20s (change using -quota SECS).

```

Name	Time (ns)	Minor	Major	Promoted	% of max
mutable	6_304_612	2_000_004	9.05	9.05	100.00
immutable	4_775_718	5_000_005	0.03	0.03	75.75

Здесь можно явно видеть, как за счет увеличения потребления памяти достигается увеличение производительности. Версия с изменяемым значением выполняется существенно дольше, чем неизменяемая, но расходует памяти значительно меньше. Выделение памяти во вспомогательной куче выполняется очень быстро, поэтому часто лучше пользоваться неизменяемыми структурами данных. С другой стороны, если значение изменяется достаточно редко, барьер записи будет преодолеваться быстрее, чем распределение нового блока.

Единственный способ достоверно узнать, какой вариант будет работать быстрее, — провести хронометраж в реальных условиях с помощью `core_bench` и поэкс-

периментировать с разными возможностями. Выполняемые файлы, снабженные средствами хронометража, поддерживают несколько полезных параметров, влияющих на поведение сборщика мусора:

Terminal

```
https://github.com/realworldocaml/examples/tree/v1/code/gc/
show_barrier_bench_help.out
```

```
$ ./barrier_bench.native -help
```

Benchmark for mutable, immutable

```
barrier_bench.native [COLUMN ...]
```

Columns that can be specified are:

name	- Name of the test.
cycles	- Number of CPU cycles (RDTSC) taken.
cycles-err	- 95% confidence interval and R^2 error for cycles.
~cycles	- Cycles taken excluding major GC costs.
time	- Number of nano secs taken.
time-err	- 95% confidence interval and R^2 error for time.
~time	- Time (ns) taken excluding major GC costs.
alloc	- Allocation of major, minor and promoted words.
gc	- Show major and minor collections per 1000 runs.
percentage	- Relative execution time as a percentage.
speedup	- Relative execution cost as a speedup.
samples	- Number of samples collected for profiling.

R^2 error indicates how noisy the benchmark data is. A value of 1.0 means the amortized cost of benchmark is almost exactly predicated and 0.0 means the reported values are not reliable at all.

Also see: http://en.wikipedia.org/wiki/Coefficient_of_determination

Major and Minor GC stats indicate how many collections happen per 1000 runs of the benchmarked function.

The following columns will be displayed by default:

```
+name time percentage
```

To specify that a column should be displayed only if it has a non-trivial value, prefix the column name with a '+'.
 To specify that a column should be displayed only if it has a non-trivial value, prefix the column name with a '+'.
 To specify that a column should be displayed only if it has a non-trivial value, prefix the column name with a '+'.

```
=== flags ===
```

[-ascii]	Display data in simple ascii based tables.
[-clear-columns]	Don't display default columns. Only show user specified ones.
[-display STYLE]	Table style (short, tall, line, blank or column). Default short.
[-geometric SCALE]	Use geometric sampling. (default 1.01)
[-linear INCREMENT]	Use linear sampling to explore number of runs, example 1.
[-no-compactions]	Disable GC compactions.
[-quota SECS]	Time quota allowed per test (default 10s).
[-save]	Save benchmark data to <test name>.txt files.
[-stabilize-gc]	Stabilize GC between each sample capture.

```

[-v]                High verbosity level.
[-width WIDTH]      width limit on column display (default 150).
[-build-info]        print info about this build and exit
[-version]           print the version of this build and exit
[-help]             print this help text and exit
                    (alias: -?)

```

Параметры `-no-compactions` и `-stabilize-gc` могут помочь принудительно вызывать фрагментацию памяти приложения. Это позволит сымитировать поведение долгоживущего приложения, не ожидая слишком долго, чтобы провести тестирование производительности.

Подключение функций-финализаторов к значениям

Механизм автоматического управления памятью в OCaml гарантирует, что память, занимаемая значением, будет рано или поздно освобождена после выхода этого значения из использования либо как результат работы сборщика мусора, либо в результате завершения программы. Иногда бывает желательно выполнить некоторый код сразу после утилизации значения сборщиком мусора, например чтобы гарантировать закрытие дескриптора файла или запись сообщения в журнал.

Какие значения можно финализировать?

Некоторые значения не допускают возможности подключения финализаторов, потому что для них не выделяется память в куче. Примерами таких значений могут служить целые числа, конструкторы констант, логические значения, пустые массивы, пустые списки и значение `unit`. Точный перечень значений, для которых выделяется или не выделяется память в куче, зависит от реализации. Именно поэтому в библиотеку Core включен модуль `Heap_block`, выполняющий проверку перед подключением финализатора.

Некоторые константы могут размещаться в куче, но никогда не утилизироваться в течение всего времени работы программы, например список целочисленных констант. Модуль `Heap_block` явно проверяет, находится ли значение в основной или вспомогательной куче, и отвергает большинство констант. Оптимизации, выполняемые компилятором, могут также предусматривать дублирование некоторых неизменяемых значений, таких как вещественные числа в массивах. Это может привести к вызову финализатора в то время, как другая копия этого же значения используется программой.

По этой причине подключайте финализаторы только к значениям, для которых есть уверенность, что они размещаются в куче и не являются неизменяемыми. Часто финализаторы подключаются к дескрипторам файлов, чтобы гарантировать их закрытие. Однако финализатор не должен рассматриваться как основной инструмент закрытия файлов, потому что момент запуска финализатора зависит от того, когда сборщик мусора утилизирует значение. В высоконагруженной системе легко можно исчерпать ограниченный ресурс, такой как дескрипторы файлов, до того как сборщик мусора успеет возобновить его.

Библиотека Core предоставляет модуль `Heap_block`, который динамически проверяет, подходит ли указанное значение для финализации. Затем блок передается функции `Gc.add_finalizer` из пакета `Async`, которая планирует вызов финализатора с учетом наличия в программе параллельно выполняющихся потоков.

Давайте исследуем небольшой пример финализации значений разных типов, из которых одни размещаются в куче, а другие являются константами, генерируемыми на этапе компиляции:

OCaml

<https://github.com/realworldocaml/examples/blob/v1/code/gc/finalizer.ml>

```
open Core.Std
open Async.Std

let attach_finalizer n v =
  match Heap_block.create v with
  | None -> printf "%20s: FAIL\n%" n
  | Some hb ->
    let final _ = printf "%20s: OK\n%" n in
    Gc.add_finalizer hb final

type t = { foo: bool }

let main () =
  let allocated_float = Unix.gettimeofday () in
  let allocated_bool = allocated_float > 0.0 in
  let allocated_string = String.create 4 in
  attach_finalizer "immediate int" 1;
  attach_finalizer "immediate float" 1.0;
  attach_finalizer "immediate variant" (`Foo "hello");
  attach_finalizer "immediate string" "hello world";
  attach_finalizer "immediate record" { foo=false };
  attach_finalizer "allocated float" allocated_float;
  attach_finalizer "allocated bool" allocated_bool;
  attach_finalizer "allocated variant" (`Foo allocated_bool);
  attach_finalizer "allocated string" allocated_string;
  attach_finalizer "allocated record" { foo=allocated_bool };
  Gc.compact ();
  return ()

let () =
  Command.async_basic ~summary:"Testing finalizers"
    Command.Spec.empty main
  |> Command.run
```

Если скомпилировать и запустить эту программу, она должна вывести примерно следующее:

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/gc/run_finalizer.out

```
$ corebuild -pkg async finalizer.native
$ ./finalizer.native
    immediate int: FAIL
    immediate float: FAIL
immediate variant: FAIL
immediate string: FAIL
immediate record: FAIL
```

```
allocated bool: FAIL
allocated record: OK
allocated string: OK
allocated variant: OK
allocated float: OK
```

Функция финализации вызывается сборщиком мусора сразу после утилизации значения. Если в одном цикле сборки мусора утилизируется сразу несколько значений, функции финализации будут вызваны в порядке, обратном порядку вызова функций `add_finalizer`. Каждый вызов `add_finalizer` добавляет новую функцию во множество функций, которые должны вызываться при утилизации значений. Допускается устанавливать несколько функций финализации для одного и того же блока.

После того как сборщик мусора определит, что блок `b` вышел из употребления и более недоступен программе, он удаляет из множества финализаторов все функции, связанные с блоком `b`, и последовательно применяет каждую из них к этому блоку. То есть каждая функция финализации будет вызвана не более одного раза. Однако завершение программы не вызывает запуска всех финализаторов.

В функции финализации можно использовать все возможности языка OCaml, включая операции присваивания, которые вновь могут сделать значение достижимым и тем самым предотвратить утилизацию. Однако это может вызвать бесконечный цикл чередования вызовов других финализаторов с этим.

Компиляторы: парсинг и контроль типов

Компиляция исходного кода в выполняемую программу осуществляется целым комплексом библиотек, компоновщиков и ассемблеров. Важно понимать, как они совмещаются друг с другом, помогая вам решать повседневные задачи по разработке, отладке и разворачиванию приложений.

Язык OCaml является строго типизированным языком и отвергает исходный код, не соответствующий его требованиям. Достигается это за счет последовательности проверок и трансформаций, выполняемых компилятором. На каждой стадии работы компилятора решается своя задача (например, проверка типов, оптимизация или создание выполняемого кода) и отбрасывается некоторая доля информации, полученной на предыдущей стадии. На выходе получается машинный код, ничего не подозревающий о модулях или объектах OCaml, с которых начиналась компиляция.

Разумеется, от вас не требуется делать что-то из перечисленного вручную. Достаточно запустить команды вызова компилятора (`ocamlc` и `ocamlopt`) из командной строки, и они автоматически выполнят всю последовательность стадий. Однако иногда бывает желательно погрузиться в процесс компиляции глубже обычного, чтобы отловить ошибку или попытаться решить проблему производительности. В этой главе подробно описывается последовательность компиляции, чтобы вы при необходимости могли эффективно использовать инструменты командной строки.

В данной главе рассматриваются следующие темы:

- последовательность стадий компиляции и какие действия выполняются на каждой стадии;
- предварительная обработка исходного кода с помощью `Camlp4` и промежуточные формы;
- процедура проверки типов, включая разрешение модулей;

Подробности, касающиеся компиляции в выполняемый код, приводятся далее, в главе 23.

Обзор инструментов компилятора

Инструменты компилятора OCaml принимают исходный программный код в текстовом виде, различая модули и сигнатуры по расширениям имен файлов `.ml` и `.mli` соответственно. Основы процесса компиляции уже описывались в главе 4, поэто-

му далее мы будем предполагать, что к настоящему моменту вы уже скомпилировали несколько программ на OCaml.

Каждый файл с исходным кодом представляет *единицу компиляции* (*compilation unit*) и компилируется независимо от других файлов. Компилятор генерирует промежуточные файлы с разными расширениями имен файлов для использования на последующих этапах компиляции. Компоновщик (*linker*) принимает коллекцию скомпилированных файлов и производит автономный выполняемый файл или библиотеку, которую можно использовать в других приложениях.

В общем виде процесс компиляции выглядит, как показано на рис. 22.1.

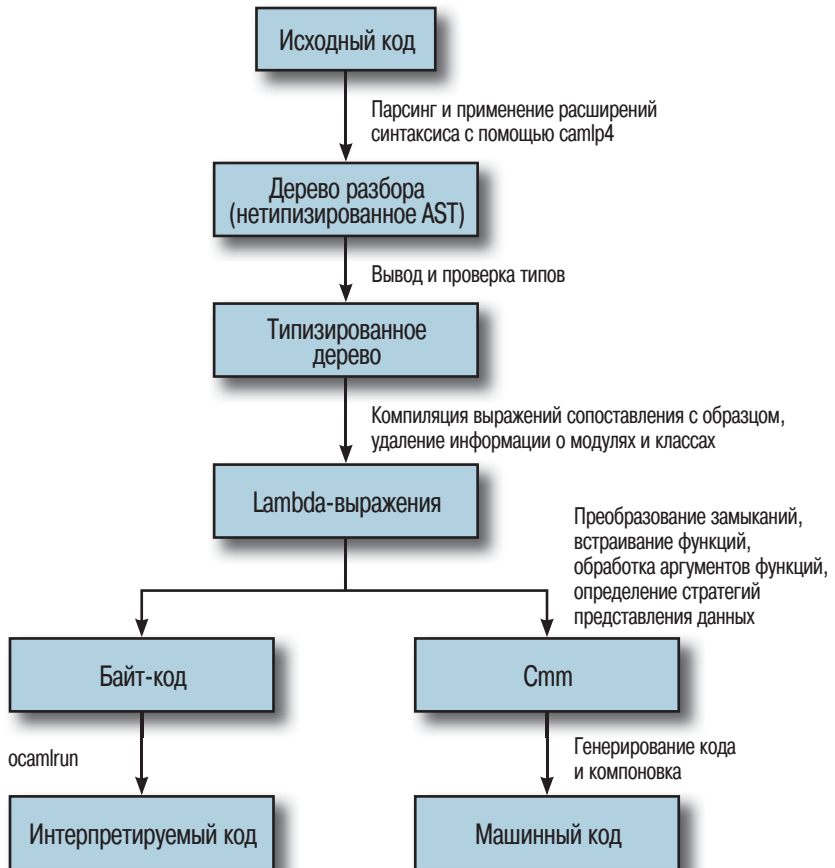


Рис. 22.1 ❖ Схема процесса компиляции

Обратите внимание на разветвление процесса компиляции ближе к концу. В OCaml имеется несколько компиляторов, использующих результаты, полученные на предыдущих этапах, но производящих разный код. Байт-код можно выполнять под управлением переносимого интерпретатора и даже преобразовывать

в исходный код на языке JavaScript (с помощью `js_of_ocaml`) или на языке C (с помощью `OCamlCC`). Компилятор машинного кода генерирует специализированные двоичные выполняемые файлы, обеспечивающие максимальную скорость работы приложений.

Как получить исходный код компилятора

Хотя это и не требуется для понимания примеров, у кого-то из вас может появиться желание получить исходные тексты OCaml и заглядывать в них в процессе чтения данной главы. Исходные тексты доступны в разных местах:

- стабильные версии в виде *zip*- и *tar*-архивов можно загрузить на сайте OCaml¹;
- исходные тексты для разработчиков доступны в анонимном зеркале репозитория Subversion²;
- все ветви с полной историей разработки доступны в Git-зеркале репозитория Subversion на сайте GitHub³.

Дерево каталогов с исходными текстами разбито на подкаталоги. Основной компилятор:

- `config/` – директивы настройки процедуры сборки OCaml для разных операционных систем и архитектур;
- `bytecomp/` и `byterun/` – компилятор и среда выполнения байт-кода, включая сборщик мусора;
- `asmcomp/` и `asmrun/` – компилятор и среда выполнения машинного кода. Символические ссылки на различные модули из каталога `byterun`, обеспечивающие совместное их использование, наиболее заметным из которых является модуль GC;
- `parsing/` – лексический анализатор OCaml, парсер и библиотеки, необходимые для их работы;
- `typing/` – реализация статической проверки и поддержка определения типов;
- `camlp4/` – исходный код препроцессора макросов;
- `driver/` – интерфейсы командной строки для инструментов компилятора.

Вместе с компилятором создаются также следующие инструменты и сценарии:

- `debugger/` – интерактивный отладчик байт-кода;
- `toplevel/` – интерактивная оболочка;
- `emacs/` – поддержка языка OCaml (*ocaml-mode*) для текстового редактора Emacs;
- `stdlib/` – компилятор стандартной библиотеки, включая модуль `Pervasives`;
- `ocamlbuild/` – система сборки, автоматизирующая поддержку основных режимов компиляции исходного кода на OCaml;
- `otherlibs/` – дополнительные библиотеки, например для поддержки ОС Unix и графики;
- `tools/` – утилиты командной строки, такие как `ocamldep`, устанавливаемые вместе с компилятором;
- `testsuite/` – регрессионные тесты для компилятора.

Теперь пройдемся по всем этапам компиляции и посмотрим, чем они могут быть полезны в повседневной работе программиста на OCaml.

Парсинг исходного кода

Когда файл с исходным кодом передается компилятору OCaml, он сначала преобразует текст в более структурированное абстрактное синтаксическое дерево (Ab-

¹ <http://caml.inria.fr/download.en.html>.

² <http://caml.inria.fr/ocaml/anonsvn.en.html>.

³ <https://github.com/ocaml/ocaml>.

stract Syntax Tree, AST). Логика парсинга реализована с применением приемов, описанных в главе 16. Правила лексического и синтаксического анализа можно найти в каталоге *parsing*, в дереве с исходными текстами компилятора.

Синтаксические ошибки

Цель парсера OCaml – подготовить структуру данных AST к следующему этапу компиляции. Поэтому для любого исходного кода, не соответствующего основным синтаксическим требованиям, компилятор генерирует *синтаксические* ошибки с текстом, содержащим имя файла и номер строки, где предположительно находится ошибка.

Вот пример исходного кода, вызывающего синтаксическую ошибку из-за попытки применить к модулю обычную инструкцию присваивания вместо let-привязки:

OCaml

https://github.com/realworldocaml/examples/blob/v1/code/front-end/broken_module.ml

```
let () =
  module MyString = String;
  ()
```

Ниже приводится текст сообщения об ошибке, генерируемого компилятором:

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/front-end/build_broken_module.out

```
$ ocamlc -c broken_module.ml
File "broken_module.ml", line 2, characters 2-8:
Error: Syntax error
```

Правильная версия, которая создает модуль MyString, открывая его локально, компилируется вполне благополучно:

OCaml

https://github.com/realworldocaml/examples/blob/v1/code/front-end/fixed_module.ml

```
let () =
  let module MyString = String in
  ()
```

В тексте сообщения о любой синтаксической ошибке указываются номер строки и номер символа в строке, соответствующего первой лексеме, на которой «споткнулся» парсер. В примере выше причиной ошибки стало ключевое слово `module`, недопустимое в этом месте, поэтому информация о местоположении верна.

Автоматическое оформление отступов в исходном коде

К сожалению, точность сообщений о синтаксических ошибках во многом зависит от их характера. Попробуйте определить причину ошибки в следующей функции:

OCaml

https://github.com/realworldocaml/examples/blob/v1/code/front-end/follow_on_function.ml

```
let concat_and_print x y =
  let v = x ^ y in
  print_endline v;
  v;

let add_and_print x y =
  let v = x + y in
  print_endline (string_of_int v);
  v

let () =
  let _x = add_and_print 1 2 in
  let _y = concat_and_print "a" "b" in
  ()
```

При попытке скомпилировать этот файл компилятор выведет следующее сообщение:

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/front-end/build_follow_on_function.out

```
$ ocamlc -c follow_on_function.ml
File "follow_on_function.ml", line 11, characters 0-3:
Error: Syntax error
```

Номер строки в сообщении указывает на конец функции `add_and_print`, но в действительности ошибка находится в конце определения *первой* функции. Здесь присутствует лишняя точка с запятой, из-за которой определение второй функции оказывается частью первой `let`-привязки. Это и вызывает ошибку парсинга в конце второй функции.

Данный класс ошибок (вызванных единственным ошибочным символом) трудно поддается диагностике в больших объемах исходного кода. К счастью, в ОРАМ существует замечательный инструмент, который называется *ocp-indent*. Он оформляет отступы строк в исходном коде в соответствии с его структурой. Применение этого инструмента не только помогает улучшить оформление исходного кода, но также существенно упрощает поиск причин появления синтаксических ошибок.

Давайте обработаем ошибочный фрагмент с помощью *ocp-indent* и посмотрим, что получится в результате:

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/front-end/indent_follow_on_function.out

```
$ ocp-indent follow_on_function.ml
let concat_and_print x y =
  let v = x ^ y in
  print_endline v;
  v;
```

```

let add_and_print x y =
  let v = x + y in
  print_endline (string_of_int v);
  v

let () =
  let _x = add_and_print 1 2 in
  let _y = concat_and_print "a" "b" in
  ()

```

Как видите, определение функции `add_and_print` было дополнено отступами, как если бы оно было частью определения первой функции `concat_and_print`. Теперь лишнюю точку с запятой, ставшую причиной ошибки, заметить намного проще. Достаточно просто удалить эту точку с запятой и повторно запустить *ocp-indent*, чтобы убедиться в корректности синтаксиса:

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/front-end/indent_follow_on_function_fixed.out

```

$ ocp-indent follow_on_function_fixed.ml
let concat_and_print x y =
  let v = x ^ y in
  print_endline v;
  v

let add_and_print x y =
  let v = x + y in
  print_endline (string_of_int v);
  v

let () =
  let _x = add_and_print 1 2 in
  let _y = concat_and_print "a" "b" in
  ()

```

На домашней странице проекта *ocp-indent*¹ рассказывается, как подружить эту утилиту с различными текстовыми редакторами. Исходные тексты всех библиотек, входящих в состав Coq, отформатированы с помощью данного инструмента, чтобы гарантировать единообразие оформления. Этот пример достоин подражания, особенно если вы собираетесь публиковать собственный исходный код в Интернете.

Автоматическое создание документации на основе интерфейсов

В процессе парсинга из исходных текстов удаляются лишние пробелы и комментарии, которые не играют определяющей роли в семантике программы. Однако в состав дистрибутива OCaml входят дополнительные инструменты, уделяющие комментариям особое внимание.

¹ <https://github.com/OCamlPro/ocp-indent>.

Инструмент *ocamldoc* отыскивает в исходном коде специально отформатированные комментарии и на их основе создает пакет документации. Эти комментарии, как правило, объединяются с определениями функций и сигнатур и выводятся в виде отформатированного описания. Данная утилита может генерировать страницы HTML, документы в форматах LaTeX и PDF, страницы справочного руководства UNIX и даже диаграммы зависимостей модулей для визуализации средствами Graphviz¹.

Ниже приводится пример исходного кода, снабженного комментариями, которые понимает утилита *ocamldoc*:

OCaml

<https://github.com/realworldocaml/examples/tree/v1/code/front-end/doc.ml>

```
(** example.ml: Первый специальный комментарий в файле представляет
    описание всего модуля. *)

(** Комментарий с описанием исключения My_exception. *)
exception My_exception of (int -> int) * int

(** Комментарий с описанием типа [weather] *)
type weather =
  | Rain of int (** Комментарий с описанием конструктора Rain *)
  | Sun          (** Комментарий с описанием конструктора Sun *)

(** Поиск информации о текущей погоде в стране
    @author Anil Madhavapeddy
    @param location Страна, для которой выполняется поиск информации о погоде.
*)
let what_is_the_weather_in location =
  match location with
  | `Cambridge -> Rain 100
  | `New_york -> Rain 20
  | `California -> Sun
```

Комментарии *ocamldoc* отличаются наличием двух символов звездочки в начале. Существуют также определенные правила по оформлению метаданных в комментариях. Например, поле вида @tag определяет определенные свойства, такие как автор данного фрагмента кода.

Попробуем скомпилировать документацию в форматах HTML и страниц справочного руководства UNIX:

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/front-end/build_ocamldoc.out

```
$ mkdir -p html man/man3
$ ocamldoc -html -d html doc.ml
$ ocamldoc -man -d man/man3 doc.ml
$ man -M man Doc
```

В результате в каталоге *html/* должны появиться файлы HTML, а в каталоге *man/man3* – страницы справочного руководства UNIX. Разнообразие форматов

¹ <http://www.graphviz.org/>.

комментариев и параметров компиляции не очень велико. Полный их список можно найти в руководстве по языку OCaml¹.

Использование других генераторов документации

Оформление по умолчанию страниц HTML, генерируемых утилитой `ocamldoc`, выглядит весьма скромно и поддерживает только особенности Web 1.0. Однако инструмент поддерживает возможность подключения других генераторов документации, обеспечивающих более привлекательное оформление или генерирующих более подробное описание.

- `Argot`¹ – улучшенный генератор документации в формате HTML, поддерживающий функции свертки кода и поиска определения по имени или типу.
- Генераторы `ocamldoc`² добавляют поддержку ссылок Bibtex внутри комментариев и генерируют документацию, включающую фрагменты кода.
- Имеется также генератор, генерирующий документацию в формате JSON³.

Препроцессинг исходного кода

Одной из мощнейших особенностей OCaml является поддержка возможности расширения грамматики языка без внесения изменений в компилятор. Ее можно сравнить с препроцессором `cpr`, выполняющим обработку директив условной компиляции в программах на языках C/C++.

В состав дистрибутива OCaml входит система под названием `Camlp4`, предназначенная для создания расширяемых парсеров. Она состоит из ряда библиотек, используемых для определения грамматик, а также динамически загружаемых расширений синтаксиса для таких грамматик. Модули из `Camlp4` регистрируют новые ключевые слова и позднее преобразуют их (в действительности целые фрагменты программ) в обычный исходный код на языке OCaml, который может быть скомпилирован стандартным компилятором.

Мы уже познакомились с некоторыми библиотеками из состава `Core`, использующими `Camlp4`:

- `Fieldslib` – генерирует значения первого порядка, представляющие поля записей;
- `Sexplib` – преобразует определения типов в `s`-выражения;
- `Bin_prot` – обеспечивает эффективные операции преобразования двоичных данных.

Все эти библиотеки весьма несущественно расширяют язык, добавляя поддержку ключевого слова `with` в объявлениях типов, и обеспечивают создание дополнительного кода на основе этого объявления. Ниже приводится простейший пример использования `Sexplib` и `Fieldslib`:

OCaml

https://github.com/realworldocaml/examples/blob/v1/code/front-end/type_conv_example.ml
`open Sexplib.Std`

```
type t = {
```

¹ <http://caml.inria.fr/pub/docs/manual-ocaml/manual029.html>.

² <http://argot.x9c.fr/>.

³ <https://gitorious.org/ocamldoc-generators/ocamldoc-generators>.

⁴ <https://github.com/xen-org/ocamldoc-json>.

```
foo: int;
bar: string
} with sexp, fields
```

При попытке скомпилировать этот код без привлечения Camlp4 вы получите сообщение об ошибке, поскольку ключевое слово `with` обычно не может использоваться после определения типа:

Terminal

```
https://github.com/realworldocaml/examples/tree/v1/code/front-end/
build_type_conv_without_camlp4.out
```

```
$ ocamlfind ocamlc -c type_conv_example.ml
File "type_conv_example.ml", line 6, characters 2-6:
Error: Syntax error
```

Теперь добавим пакеты `Fieldslib` и `Sexplib`, и программа благополучно скомпилируется:

Terminal

```
https://github.com/realworldocaml/examples/tree/v1/code/front-end/
build_type_conv_with_camlp4.out
```

```
$ ocamlfind ocamlc -c -syntax camlp4o -package sexplib.syntax \
  -package fieldslib.syntax type_conv_example.ml
```

Мы указали здесь пару дополнительных флагов. Флаг `-syntax` предписывает утилите `ocamlfind` добавить флаг `-pp` в командную строку вызова компилятора. Он сообщает компилятору, что тот должен вызвать препроцессор на этапе синтаксического анализа.

Флаг `-package` импортирует другие библиотеки OCaml. Расширение `.syntax` в имени пакета указывает, что эти библиотеки являются препроцессорами, которые должны запускаться на этапе синтаксического анализа. Модули расширений синтаксиса динамически загружаются командой `camlp4o` и преобразуют исходный программный код в обычный код на OCaml, не содержащий новых ключевых слов. Затем компилятор компилирует этот трансформированный код, ничего не подозревая о манипуляциях, произведенных препроцессором.

Оба расширения, `Fieldslib` и `Sexplib`, обрабатывают новое ключевое слово `with`, но они не могут быть одновременно зарегистрированы для обработки одного и того же расширения. Поэтому они используют в качестве основы библиотеку `Type_conv`, реализующую обобщенную поддержку расширений. Библиотека `Type_conv` регистрирует расширение грамматики в Camlp4, а OCamlfind гарантирует, что она будет загружена до расширения `Fieldslib` или `Sexplib`.

Расширения генерируют шаблонный код на OCaml прямо во время компиляции, опираясь на определение типа. Это позволяет избежать потерь производительности, вызванных динамическим созданием кода во время выполнения и применением динамического компилятора, которые, в свою очередь, могут стать источником непредсказуемости. Вместо этого весь дополнительный код генерируется на этапе компиляции с помощью Camlp4, а информация о типах может быть исключена из выполняемого образа.

Расширения синтаксиса получают исходное дерево AST и возвращают модифицированную версию. Но как узнать, какие изменения были выполнены, если мы незнакомы с модулем `Camlp4`? Самый простой путь – прочитать документацию, сопровождающую расширение. Другой путь – исследовать поведение расширения с помощью интерактивной оболочки или запустить `Camlp4` вручную. Мы покажем оба способа далее.

Использование `Camlp4` в интерактивной оболочке

Интерактивная оболочка *utop* может автоматически пропускать вводимые вами фразы через *camlp4*. Для этого вам следует добавить следующие строки в свой файл `~/.ocamlinit` (подробности см. по адресу: <http://realworldocaml.org/install>)¹:

OCaml *utop*

https://github.com/realworldocaml/examples/tree/v1/code/front-end/camlp4_toplevel.topscript

```
# #use "topfind" ;;
- : unit = ()
Findlib has been successfully loaded. Additional directives:
#require "package";;      to load a package
#list;;                   to list the available packages
#camlp4o;;                to load camlp4 (standard syntax)
#camlp4r;;                to load camlp4 (revised syntax)
#predicates "p,q,...";;   to set these predicates
Topfind.reset();;         to force that packages will be reloaded
#thread;;                 to enable threads

- : unit = ()
# #camlp4o ;;
```

Первая директива загружает интерфейс *ocamlfind* верхнего уровня, позволяющий загружать пакеты *ocamlfind* (вместе со всеми зависимостями). Вторая директива сообщает интерактивной оболочке, что она должна предварительно пропускать все фразы через `Camlp4`. Теперь можно запустить интерактивную оболочку *utop* и загрузить в нее расширения синтаксиса. В дальнейших наших экспериментах мы будем использовать расширение синтаксиса *comparelib*.

В языке OCaml имеется встроенный оператор, выполняющий полиморфное сравнение, который исследует два значения во время выполнения для ответа на вопрос о равенстве. Как отмечалось в главе 13, полиморфное сравнение менее эффективно, чем явное определение функций сравнения. Однако определение таких функций для сложных типов быстро превращается в утомительную рутину.

¹ Автор продемонстрировал действие директив в интерактивной оболочке, в действительности же в указанный файл требуется добавить всего две строки:

```
#use "topfind" ;;
#camlp4o ;;
```

– Прим. перев.

Давайте посмотрим, как `comparelib` решает эту проблему, на примерах в интерактивной оболочке:

OCaml utop (part 1)

https://github.com/realworldocaml/examples/blob/v1/code/front-end/camlp4_toplevel.topscript

```
# #require "comparelib.syntax" ;;
# type t = { foo: string; bar : t } ;;
type t = { foo : string; bar : t; }
# type t = { foo: string; bar: t } with compare ;;
type t = { foo : string; bar : t; }
val compare : t -> t -> int = <fun>
val compare_t : t -> t -> int = <fun>
```

Первое определение `t` является стандартным определением типа на языке OCaml, и результат его выполнения вполне ожидаем. Вторая фраза включает директиву `with compare`. Она перехватывается расширением синтаксиса `comparelib` и преобразуется в обычное определение типа с двумя новыми функциями.

Запуск Camlp4 из командной строки

Интерактивная оболочка позволяет быстро исследовать сигнатуры, сгенерированные расширениями, но как узнать, что в действительности делают эти новые функции? Этого нельзя сделать непосредственно в интерактивной оболочке, поскольку она автоматически вызывает Camlp4.

Давайте вернемся в системную командную строку, чтобы получить результаты трансформации, выполняемой расширением `comparelib`. Создайте файл со следующим определением типа:

OCaml

https://github.com/realworldocaml/examples/blob/v1/code/front-end/comparelib_test.ml

```
type t = {
  foo: string;
  bar: t
} with compare
```

Нам нужно запустить выполняемый файл Camlp4, указав пути поиска, в которых находятся `Comparelib` и `Type_conv`. Напишем для этого небольшой сценарий командной оболочки:

Shell script

https://github.com/realworldocaml/examples/blob/v1/code/front-end/camlp4_dump.cmd

```
#!/bin/sh
```

```
OCAMLFIND="ocamlfind query -predicates syntax,preprocessor -r"
INCLUDE=`$OCAMLFIND -i-format comparelib.syntax`
ARCHIVES=`$OCAMLFIND -a-format comparelib.syntax`
camlp4o -printer o $INCLUDE $ARCHIVES $1
```

Сначала сценарий перечисляет пути поиска подключаемых файлов и библиотек, необходимых расширению `comparelib`, с использованием диспетчера пакетов

ocamlfind. Затем он вызывает препроцессор `camlp4o`, передает ему эти пути и выводит получившееся дерево AST на стандартный вывод:

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/front-end/process_comparelib_test.out

```
$ sh camlp4_dump.cmd comparelib_test.ml
type t = { foo : string; bar : t }

let _ = fun ( _ : t) -> ()

let rec compare : t -> t -> int =
  fun a__001_ b__002_ ->
    if Pervasives.( == ) a__001_ b__002_
    then 0
    else
      (let ret =
         (Pervasives.compare : string -> string -> int) a__001_.foo
          b__002_.foo
       in
        if Pervasives.( <> ) ret 0
        then ret
        else compare a__001_.bar b__002_.bar)

let _ = compare

let compare_t = compare

let _ = compare_t
```

Вывод содержит определение оригинального типа, сопровождаемое некоторым кодом, сгенерированным автоматически и реализующим функцию, которая явно сравнивает все поля в записи. Если это расширение использовать в командной строке компилятора, сгенерированный код будет скомпилирован, как если бы вы ввели его сами.

Обратите внимание: хотя сгенерированный код использует `Pervasives.compare`, он также аннотирован типом `string`, что позволяет компилятору использовать специальную функцию сравнения строк вместо функции полиморфного сравнения. Это имеет определенные следствия: вспомните, как в главе 13 говорилось, что `comparelib` предоставляет надежные функции сравнения для обработки значений, которые логически являются одинаковыми, но имеют разное внутреннее представление (такие как `Int.Set.t`).

Примечания к стилю: шаблонные символы в `let`-привязках

Внимательные читатели могли заметить конструкцию `let _ = fun` в автоматически сгенерированном коде выше. Символ подчеркивания в `let`-привязке действует подобно шаблонному символу подчеркивания в операции сопоставления и сообщает компилятору, что он должен получить любое возвращаемое значение и тут же отбросить его. Этот прием с успехом используется в коде, механически сгенерированном библиотекой `Type_conv`, но в коде, который пишется вручную, его следует избегать. Если выра-

жение возвращает `unit`, используйте явную привязку `unit`. Такая привязка будет генерировать ошибку, если выражение изменит свой тип в будущем (например, в результате рефакторинга кода):

Syntax

https://github.com/realworldocaml/examples/tree/v1/code/front-end/let_unit.syntax

```
let () = <expr>
```

Если выражение имеет другой тип, используйте этот тип явно:

OCaml

https://github.com/realworldocaml/examples/tree/v1/code/front-end/let_notunit.ml

```
let (_:some_type) = <expr>
let () = ignore (<expr> : some_type)
)(* Если выражение возвращает unit Deferred.t *)
let () = don't_wait_for (<expr>
```

Последняя инструкция позволяет игнорировать выражения `Async`, которые должны выполняться в фоновом режиме, не блокируя работу текущего потока выполнения.

Еще одна существенная причина использования символов подчеркивания – связывание некоторого значения с именем переменной, которое, как предполагается, будет использоваться в еще не написанном коде. В обычной ситуации такая операция присваивания будет приводить к выводу предупреждающего сообщения компилятора о неиспользуемом значении. Подобные предупреждения не выводятся для переменных, имена которых начинаются с символа подчеркивания:

OCaml

https://github.com/realworldocaml/examples/tree/v1/code/front-end/unused_var.ml

```
let fn x y =
  let _z = x + y in
  ()
```

Даже при том, что переменная `_z` не используется в этом коде, компилятор не будет генерировать сообщение, предупреждающее о присутствии неиспользуемой переменной.

Препроцессинг сигнатур модулей

Другой полезной особенностью `type_conv` является автоматическое создание сигнатур модулей. Скопируйте предыдущее определение типа в файл `comparelib_test.mli`, чтобы получилось следующее:

OCaml

https://github.com/realworldocaml/examples/tree/v1/code/front-end/comparelib_test.mli

```
type t = {
  foo: string;
  bar: t
} with compare
```

Если теперь еще раз запустить сценарий `camlp4_dump.cmd`, вы увидите, что для файла сигнатуры был сгенерирован иной код:

Terminal

```
https://github.com/realworldocaml/examples/tree/v1/code/front-end/
process_comparelib_interface.out
```

```
$ sh camlp4_dump.cmd comparelib_test.mli
```

```
type t = { foo : string; bar : t }
```

```
val compare : t -> t -> int
```

Внешняя сигнатура, сгенерированная расширением `comparelib`, оказалась намного проще фактического кода. Непосредственное применение инструмента `Camlp4` к оригинальному исходному коду позволит воочию увидеть все эти преобразования.

Не злоупотребляйте расширениями синтаксиса

Расширения синтаксиса – очень мощный инструмент, способный до неузнаваемости изменить исходный код. Библиотека `Core` включает ряд довольно консервативных расширений, вносящих минимальные изменения в синтаксис. Существуют также сторонние библиотеки, намного более амбициозные. Одни из них реализуют поддержку чувствительности к отступам, другие реализуют совершенно новые языки, основывающиеся на `OCaml`, а третьи включают поддержку условной компиляции для макросов или возможность журналирования.

Несмотря на заманчивость возможности упаковать весь шаблонный код в расширения `Camlp4`, такой подход может сделать исходный код более сложным для чтения и понимания. Расширения в библиотеке `Core` в основном занимаются манипуляциями с типами с помощью фреймворка `type_conv` и фактически не изменяют синтаксиса языка `OCaml`. Кроме того, прежде чем приступить к созданию собственного расширения, следует также подумать о совместимости с другими расширениями. Два разных расширения могут создать грамматический клинч, ведущий к появлению странных синтаксических и трудновоспроизводимых ошибок. Именно поэтому большая часть расширений синтаксиса в библиотеке `Core` опирается в своей работе на фреймворк `type_conv`, который играет роль единой точки расширения грамматики через ключевое слово `with`.

Дополнительные источники информации о Camlp4

Мы преднамеренно показали здесь лишь порядок использования расширений `Camlp4` и ничего не сказали о том, как создавать собственные. Подробное описание принципов разработки новых расширений – слишком трудоемкая задача, это потребовало бы написать целую книгу.

Поэтому мы лишь перечислим некоторые источники дополнительной информации, откуда вы сможете начать свои изыскания:

- серия статей Джейка Донхама (Jake Donham) с описанием внутреннего устройства `Camlp4` и механизма расширений синтаксиса: <http://ambassador.to.the.computers.blogspot.co.uk/p/reading-camlp4.html>;
- вики-страница `Camlp4`: <http://brion.inria.fr/gallium/index.php/Camlp4>;
- исходные тексты `Camlp4`, которые можно установить с помощью `OPAM`.

Статическая проверка типов

После получения действительного абстрактного синтаксического дерева компилятор должен убедиться, что код соответствует правилам, установленным систе-

мой типов OCaml. Код, синтаксически корректный, но неправильно использующий значения, должен быть отвергнут с выводом описания проблемы.

Несмотря на то что проверка типов выполняется за один проход, в действительности она делится на три разных этапа, выполняемых одновременно:

- *автоматический вывод типов* – алгоритм, определяющий типы автоматически в отсутствие аннотаций, добавленных вручную;
- *система модулей* – объединяет программные компоненты с явными сигнатурами типов;
- *явная подтипизация* – выявляет объекты и полиморфные варианты.

Механизм автоматического вывода типов позволяет писать краткий код, решающий конкретную задачу, и помогает компилятору убедиться в безошибочном использовании переменных.

Действие этого механизма не распространяется на большие объемы кода, которые могут храниться в нескольких файлах и компилироваться отдельно. Небольшое изменение в одном модуле может затронуть определения во множестве других файлов и библиотек и потребовать их перекомпиляции. Данная проблема решается системой модулей за счет поддержки объединения и управления явными сигнатурами типов для модулей внутри единого проекта, а также повторного использования их через функторы и модули первого порядка.

Подтипизация в объектах OCaml всегда выполняется явно (с помощью оператора `>`). Благодаря такому подходу упрощается конструкция механизма автоматического вывода типов и появляется возможность выполнения отдельных проверок.

Демонстрация типов, выводимых компилятором

Мы уже знаем, как увидеть действие механизма автоматического вывода типов в интерактивной оболочке. Однако имеется дополнительная возможность получить сигнатуры типов для всего файла. Создайте файл с единственным определением типа и значением:

OCaml

<https://github.com/realworldocaml/examples/tree/v1/code/front-end/typedef.ml>

```
type t = Foo | Bar
let v = Foo
```

Теперь запустите компилятор с флагом `-i`, чтобы получить сигнатуру типа для этого файла. Компилятор произведет проверку типов и остановится после вывода интерфейса на стандартный вывод:

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/front-end/infer_typedef.out

```
$ ocamlc -i typedef.ml
type t = Foo | Bar
val v : t
```

В результате вы увидите сигнатуру для модуля, представленного исходным файлом. Эта возможность часто используется на практике для получения файла *.mli*, играющего роль начальной заготовки для последующей правки вручную.

Компилятор сохраняет скомпилированную версию интерфейса в файле с расширением *.cmi*. Этот интерфейс получается либо путем компиляции имеющегося файла сигнатуры модуля с расширением *.mli*, либо путем компиляции выведенных типов, если компилятору доступен только файл реализации *.ml*.

Компилятор проверяет совместимость сигнатур файлов *.ml* и *.mli*, и если они несовместимы, механизм проверки типов выводит сообщение об ошибке:

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/front-end/conflicting_interfaces.out

```
$ echo type t = Foo > test.ml
$ echo type t = Bar > test.mli
$ ocamlc -c test.mli test.ml
File "test.ml", line 1:
Error: The implementation test.ml does not match the interface test.cmi:
  Type declarations do not match:
    type t = Foo
  is not included in
    type t = Bar
File "test.ml", line 1, characters 5-12: Actual declaration
Fields number 1 have different names, Foo and Bar.
(Ошибка: Реализация test.ml не соответствует интерфейсу test.cmi:
  Не совпадают объявления типов:
    type t = Foo
  не включен в
    type t = Bar
  Файл "test.ml", строка 1, символы 5-12: Фактические объявления
  полей с номерами 1 имеют разные имена, Foo и Bar.)
```

Что должно создаваться первым, ml или mli?

Существуют две школы, исповедующие разный порядок разработки программного кода на языке OCaml. Намного проще начинать разработку с файла с расширением *.ml* и руководствоваться механизмом автоматического вывода типов при создании своих функций. После этого можно сгенерировать файл *.mli*, как рассказывалось выше, откорректировать его вручную, оставив в нем описание только экспортируемых функций. Если вы пишете код, разбросанный по нескольким файлам, иногда проще начать с создания сигнатур *.mli* и проверки их совместимости между собой. Когда сигнатуры будут готовы, можно приступить к созданию реализаций, пребывая в уверенности, что они будут совместимы по типам и не будут иметь циклических зависимостей.

Как это часто бывает с подобными стилистическими дебатами, чтобы выбрать ту или другую точку зрения, необходимо на практике попробовать обе системы и понять, какая из них лучше подходит вам лично. Но, несмотря на разногласия, обе школы едины во мнении, что независимо от выбранного порядка промышленный код всегда должен включать явное определение сигнатуры в файле *mli* для каждого файла *ml*, имеющегося в проекте. Допускается также иметь файлы *mli* без соответствующих им файлов *ml*, если вы объявляете только сигнатуры (например, типы модулей).

Файлы сигнатур считаются отличным местом для включения краткой документации и помогают скрывать внутренние тонкости, которые не должны экспортироваться. Поддержание отдельных файлов сигнатур позволяет также увеличить скорость инкрементальной компиляции больших объемов кода, потому что повторная компиляция сигнатуры *mli* выполняется намного быстрее, чем полная компиляция реализации.

Вывод типов

Вывод типов – это процесс определения типов выражений, исходя из их использования. Данная возможность частично реализована во многих других языках, таких как Haskell и Scala, но в OCaml она возведена в ранг фундаментальной особенности, лежащей в основе самого языка.

Механизм вывода типов в OCaml основан на алгоритме Хиндли-Милнера (Hindley-Milner), который известен своей способностью выводить наиболее общие типы для выражений без необходимости явно указывать аннотации типов. Алгоритм способен выводить разные типы для одного и того же выражения и поддерживает понятие *главного типа* (*principal type*), являющегося наиболее общим среди возможных. Аннотации типов, добавленные вручную, могут уточнять типы, но в их отсутствие выбираются наиболее общие типы.

В OCaml имеются некоторые расширения языка, требующие явно определять главный тип, но в большинстве случаев вы легко сможете обойтись без каких-либо аннотаций (хотя иногда они помогают компилятору сформулировать более точные сообщения об ошибках).

Добавление аннотаций типов для поиска ошибок

Часто можно услышать мнение, что самое сложное в программировании на OCaml – это удовлетворить механизм проверки типов, но как только код скомпилируется без ошибок, он будет работать правильно с первой же попытки! Конечно же, это преувеличение, но оно очень похоже на истину, особенно для тех, кто имеет опыт работы с динамическими языками. Система статических типов в языке OCaml защищает вас от целых классов ошибок, таких как ошибки обращения к памяти и нарушения абстракций, отвергая ошибочный код на этапе компиляции и предохраняя от появления ошибок во время выполнения. Знание особенностей работы механизма контроля типов во время компиляции является ключом к созданию надежных библиотек и приложений, использующих все преимущества статической проверки типов.

Существует несколько хитростей, упрощающих поиск ошибок в коде, связанных с типами. Первая – добавление аннотаций, сужающих источник ошибок. Эти аннотации не должны изменять типов данных и могут быть удалены после исправления программного кода. Однако они действуют как опорные точки, помогающие находить ошибки в процессе разработки программы.

Аннотации особенно полезны при использовании большого числа полиморфных вариантов или объектов. Механизм вывода типов в комбинации с рядным полиморфизмом (row polymorphism) может генерировать просто огромные сигнатуры, а ошибки имеют свойство более широкого распространения, чем при использовании явно типизированных вариантов или классов.

Например, взгляните на следующий ошибочный пример, выражающий некоторые простые алгебраические операции над целыми числами:

OCaml

https://github.com/realworldocaml/examples/blob/v1/code/front-end/broken_poly.ml

```
let rec algebra =
  function
  | `Add (x,y) -> (algebra x) + (algebra y)
  | `Sub (x,y) -> (algebra x) - (algebra y)
  | `Mul (x,y) -> (algebra x) * (algebra y)
  | `Num x -> x

let _ =
  algebra (
    `Add (
      (`Num 0),
      (`Sub (
        (`Num 1),
        (`Mul (
          (`Nu 3), (`Num 2)
        ))
      ))
    ))
  ))
```

Здесь присутствует единственная опечатка, где вместо имени Num используется имя Nu, в результате чего генерируется весьма внушительное сообщение об ошибке:

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/front-end/build_broken_poly.out

```
$ ocamlc -c broken_poly.ml
```

```
File "broken_poly.ml", line 9, characters 10-154:
```

```
Error: This expression has type
```

```
[> `Add of
```

```
  ([< `Add of 'a * 'a
    | `Mul of 'a * 'a
    | `Num of int
    | `Sub of 'a * 'a
    > `Num ]
```

```
as 'a) *
```

```
[> `Sub of 'a * [> `Mul of [> `Nu of int ] * [> `Num of int ] ] ] ]
```

```
but an expression was expected of type 'a
```

```
The second variant type does not allow tag(s) `Nu
```

(Ошибка: Это выражение имеет тип

```
[> `Add of
```

```
  ([< `Add of 'a * 'a
    | `Mul of 'a * 'a
    | `Num of int
    | `Sub of 'a * 'a
    > `Num ]
```

```
as 'a) *
```

```
[> `Sub of 'a * [> `Mul of [> `Nu of int ] * [> `Num of int ] ] ] ]
```

тогда как ожидалось выражение типа 'a

Тип второго варианта не допускает тег(и) `Nu)

В общем и целом сообщение содержит точную информацию, но оно слишком подробное и содержит номер строки, не соответствующий точному местоположению неправильного имени варианта. Лучшее, что смог сделать компилятор, – направить нас в вызов функции `algebra`.

Это объясняется тем, что механизм проверки типов не обладает достаточной информацией для сопоставления типа, выведенного из определения функции `algebra`, с ее применением несколькими строками ниже. Он определяет типы для обоих выражений по отдельности, а когда они не совпадают, лучшее, что он может сделать, – вывести различия.

Давайте посмотрим, что произойдет при добавлении аннотации типа, чтобы помочь компилятору:

OCaml

https://github.com/realworldocaml/examples/blob/v1/code/front-end/broken_poly_with_annot.ml

```
type t = [
  | `Add of t * t
  | `Sub of t * t
  | `Mul of t * t
  | `Num of int
]

let rec algebra (x:t) =
  match x with
  | `Add (x,y) -> (algebra x) + (algebra y)
  | `Sub (x,y) -> (algebra x) - (algebra y)
  | `Mul (x,y) -> (algebra x) * (algebra y)
  | `Num x -> x

let _ =
  algebra (
    `Add (
      (`Num 0),
      (`Sub (
        (`Num 1),
        (`Mul (
          (`Nu 3), (`Num 2)
        ))
      ))
    ))
  )
```

Этот код содержит ту же самую ошибку, но на этот раз в него добавлены закрытое определение типа полиморфных вариантов и аннотация типа в определение функции `algebra`. В результате сообщение компилятора об ошибке стало более понятным:

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/front-end/build_broken_poly_with_annot.out

```
$ ocamlc -i broken_poly_with_annot.ml
File "broken_poly_with_annot.ml", line 22, characters 14-21:
Error: This expression has type [> `Nu of int ]
```

```

but an expression was expected of type t
The second variant type does not allow tag(s) `Nu
(Ошибка: Это выражение имеет тип [> `Nu of int ],
        тогда как ожидалось выражение типа t
        Тип второго варианта не допускает тег(и) `Nu)

```

Это сообщение об ошибке указывает непосредственный номер строки, содержащей опечатку. Устранив проблему, можно убрать аннотации, если вы предпочитаете более краткий код. Разумеется, вы можете оставить аннотации на месте, чтобы помочь в рефакторинге и отладке в будущем.

Явное определение главных типов

Компилятор может также работать в режиме строгого *контроля главных типов* (*principal type checking*), который активируется флагом `-principal`. При работе в этом режиме компилятор выводит предупреждения о рискованном использовании информации о типах, чтобы гарантировать наличие у механизма вывода типов единого главного типа. Тип считается рискованным, если успех или неудача механизма вывода типов зависит от порядка определения типов подвыражений.

Эта проверка затрагивает лишь некоторые особенности языка:

- полиморфные методы объектов;
- изменение порядка следования аргументов с метками в функции в сравнении с определениями их типов;
- исключение необязательных аргументов с метками;
- обобщенные алгебраические типы данных (Generalized Algebraic Data Types, GADT), поддерживаемые в версии OCaml 4.0 и выше;
- автоматическое устранение неоднозначности между именами полей и конструкторов в записях (начиная с версии OCaml 4.1).

Ниже приводится пример вывода предупреждения о рискованном типе, вызванном устранением неоднозначности.

OCaml

https://github.com/realworldocaml/examples/blob/v1/code/front-end/non_principal.ml

```

type s = { foo: int; bar: unit }
type t = { foo: int }

```

```

let f x =
  x.bar;
  x.foo

```

Попытка вывести сигнатуру с добавлением ключа `-principal` выведет новое предупреждение:

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/front-end/build_non_principal.out

```

$ ocamlc -i -principal non_principal.ml
File "non_principal.ml", line 6, characters 4-7:
Warning 18: this type-based field disambiguation is not principal.

```

(Предупреждение 18: устранение неоднозначности типа на основе поля ненадежно.)

```
type s = { foo : int; bar : unit; }
type t = { foo : int; }
val f : s -> int
```

Этот пример страдает ненадежностью, потому что вывод типа для `x.foo` был выполнен на основе типа `x.bar`, тогда как для надежного определения типа требуется, чтобы тип каждого подвыражения мог определяться независимо. Если из определения функции `f` удалить `x.bar`, тип аргумента будет определен `t`, а не `s`.

Исправить проблему можно либо перестановкой объявлений типов, либо добавлением явной аннотации типа:

OCaml

<https://github.com/realworldocaml/examples/blob/v1/code/front-end/principal.ml>

```
type s = { foo : int; bar : unit }
type t = { foo : int }

let f (x:s) =
  x.bar;
  x.foo
```

Теперь неоднозначность в выводе типов отсутствует, потому что мы явно указали тип аргумента, и результат вывода типов подвыражений больше не зависит от порядка их следования.

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/front-end/build_principal.out

```
$ ocamlc -i -principal principal.ml
type s = { foo : int; bar : unit; }
type t = { foo : int; }
val f : s -> int
```

Утилита *ocamlbuild* поддерживает эквивалентный тег `principal`. Фактически сценарий-обертка *corebuild* добавляет его по умолчанию, но ничего страшного не случится, если указать его явно:

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/principal/build_principal.out

```
$ corebuild -tag principal principal.cmi non_principal.cmi
File "non_principal.ml", line 6, characters 4-7:
Warning 18: this type-based field disambiguation is not principal.
```

(Предупреждение 18: устранение неоднозначности типа на основе поля ненадежно.)

В идеале весь код должен компилироваться с флагом `-principal`. Это уменьшает вероятность появления неоднозначностей при выводе типов и обеспечивает поддержку понятия единственного известного типа. Следует, однако, упомянуть и о недостатках данного режима: он замедляет работу механизма вывода типов и приводит к увеличению размеров файлов *.cmi*. Вообще говоря, это может превратиться в проблему только при интенсивном использовании объектов, которые обычно имеют большие сигнатуры, включающие все их методы.

Если компиляция с флагом `-principal` выполняется без ошибок, это может служить гарантией, что программа так же успешно будет проходить проверку типов и в обычном режиме компиляции. По этой причине сценарий-обертка *corebuild* использует флаг `-principal` компилятора по умолчанию, отдавая предпочтение более строгому выводу типов, пусть и ценой незначительной потери скорости компиляции и увеличенного расхода дискового пространства.

Имейте в виду, что файлы *.cmi*, полученные при компиляции с флагом `-principal`, отличаются от полученных в обычном режиме. Поэтому старайтесь обеспечивать компиляцию всего проекта целиком в каком-то одном режиме. Смешивание файлов не приведет к ухудшению безопасности типов, но может приводить к неожиданным ошибкам механизма контроля типов. В таких ситуациях всегда пробуйте сначала перекомпилировать все дерево исходных текстов в одном режиме.

Модули и раздельная компиляция

Система модулей OCaml позволяет повторно использовать небольшие программные компоненты в более крупных проектах, сохраняя при этом все преимущества статической системы типов. Мы познакомились с основами модулей в главе 4. Язык модулей, позволяющий оперировать их сигнатурами, распространяется также на функторы и модули первого порядка, описанные в главах 9 и 10 соответственно.

В этом разделе мы подробнее обсудим реализацию поддержки модулей в компиляторе. Модули являются основой больших проектов, которые могут состоять из большого числа файлов с исходным кодом (также известных как *единицы компиляции* (*compilation units*)). Было бы весьма непрактично, например, перекомпилировать оба файла с исходным кодом при изменении только одного из них, и система модулей минимизирует такие повторные компиляции, способствуя тем самым повторному использованию программных компонентов.

Соответствие между файлами и модулями

Поддержка раздельных единиц компиляции дает удобную возможность разбивать большие иерархии модулей на коллекции файлов. Опишем отношения между файлами и модулями непосредственно в терминах системы модулей.

Создайте файл с именем *alice.ml* и следующим содержимым:

OCaml

<https://github.com/realworldocaml/examples/tree/v1/code/front-end/alice.ml>

```
let friends = [ Bob.name ]
```

и соответствующий файл сигнатуры:

OCaml

<https://github.com/realworldocaml/examples/tree/v1/code/front-end/alice.mli>

```
val friends : Bob.t list
```

Эти два файла в точности эквивалентны включению следующего кода в некоторый другой модуль под видом модуля Alice:

OCaml

https://github.com/realworldocaml/examples/tree/v1/code/front-end/alice_combined.ml

```
module Alice : sig
  val friends : Bob.t list
end = struct
  let friends = [ Bob.name ]
end
```

Определение пути поиска модулей

В предыдущем примере модуль `Alice` имеет ссылки на другой модуль `Bob`. Поэтому, чтобы убедиться в допустимости типа модуля `Alice`, компилятору требуется также проверить, содержит ли модуль `Bob` значение `Bob.name` и определение типа `Bob.t`.

Механизм контроля типов преобразует такие ссылки между модулями в конкретные структуры и сигнатуры, чтобы обеспечить унификацию типов за границами модулей. С этой целью он просматривает список каталогов и отыскивает в них файлы скомпилированных интерфейсов с именами, совпадающими с именами модулей. Например, он попытается найти файлы `alice.cmi` и `bob.cmi` и будет использовать как интерфейсы для `Alice` и `Bob` первые, встретившиеся ему на пути.

Путь поиска модулей определяется с помощью флагов `-I` компилятора, в которых указываются пути к каталогам, содержащим файлы `.cmi`. Вручную передавать эти флаги было бы слишком утомительно, особенно при использовании в проекте большого числа библиотек. Именно это послужило поводом к созданию утилиты `OCamlfind`. Она автоматизирует процесс включения имен сторонних пакетов и встраивает необходимые флаги в команду вызова компилятора.

По умолчанию поиск файлов `.cmi` выполняется только в текущем каталоге и в каталогах стандартной библиотеки `OCaml`. Кроме того, в каждой единице компиляции по умолчанию открывается модуль `Pervasives`. Местоположение стандартной библиотеки определяется командой `ocamlc -where` и может быть переопределено установкой переменной окружения `CAMLLIB`. Разумеется, не следует переопределять путь по умолчанию без веских на то причин (как, например, необходимость настройки окружения кросс-компиляции).

Исследование единиц компиляции с помощью `ocamlobjinfo`

Чтобы обеспечить раздельную компиляцию, необходимо гарантировать неизменность всех файлов `.cmi` между компиляциями модуля, используемых для проверки его типа. В противном случае проверка типов двух модулей может быть выполнена с участием разных сигнатур одного и того же общего модуля. Что, в свою очередь, может привести к полному разрушению системы типов и, как следствие, повреждению данных в памяти во время выполнения и краху приложения.

В `OCaml` предусмотрены меры, препятствующие появлению подобных ситуаций, которые выражаются в сохранении контрольной суммы MD5 в каждом файле `.cmi`. Давайте рассмотрим поближе предыдущий наш файл `typedef.ml`:

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/front-end/typedef_objinfo.out

```
$ ocamlc -c typedef.ml
$ ocamlobjinfo typedef.cmi
File typedef.cmi
```

```
Unit name: Typedef
Interfaces imported:
  bd274dc132ce5c3d8b6774d19cd373a6 Typedef
  36b5bc8227dc9914c6d9fd9bdcfadb45 Pervasives
```

Утилита `ocamlbobjinfo` исследует скомпилированный интерфейс и сообщает, от каких других единиц компиляции он зависит. В данном случае мы не использовали никаких внешних модулей, кроме `Pervasives`. Каждый модуль по умолчанию зависит от `Pervasives`, если при компиляции не использовался флаг `-nopervasives` (однако такой подход – скорее исключение из правил и используется достаточно редко).

Длинные алфавитно-цифровые идентификаторы, предшествующие именам модулей, – это хэши, вычисленные на основе всех типов и значений, экспортируемых единицей компиляции. Они используются на этапах проверки типов и компоновки, чтобы убедиться в непротиворечивости всех единиц компиляции. Различия в хэшах означают, что единица компиляции с одним и тем же именем модуля может конфликтовать с сигнатурами в разных модулях. Компилятор отвергнет такую программу с сообщением об ошибке, как показано ниже:

Terminal

```
https://github.com/realworldocaml/examples/tree/v1/code/front-end/
inconsistent_compilation_units.out

$ ocamlc -c foo.ml
File "foo.ml", line 1, characters 0-1:
Error: The files /home/build/bar.cmi
      and /usr/lib/ocaml/map.cmi make inconsistent assumptions
      over interface Map
(Ошибка: Файлы /home/build/bar.cmi
      и /usr/lib/ocaml/map.cmi делают противоречивые предположения
      об интерфейсе Map)
```

Такая проверка хэшей является анахронизмом, но она достаточно надежно гарантирует безопасность раздельной компиляции на всем пути, вплоть до окончательной фазы компоновки. Существующая система сборки гарантирует, что вы никогда не увидите подобных сообщений об ошибках, но если это все же случится, просто удалите все промежуточные файлы и выполните компиляцию «с нуля».

Упаковка модулей вместе

Правило отображения модулей в файлы, описанное выше, требует прямого соответствия между модулями верхнего уровня и файлами. Однако часто бывает удобнее разбить большой модуль на несколько файлов, чтобы упростить его правку, но компилировать их как единый модуль `OCaml`.

Ключ `-pack` компилятора принимает список скомпилированных объектных файлов (*.sto* – для байт-кода и *.cmx* – для машинного кода) с соответствующими им скомпилированными интерфейсами *.cmi* и объединяет их в единый модуль. Такая процедура упаковки генерирует совершенно новые файлы *.sto* (или *.cmx*) и *.cmi*, включающие исходные модули.

При упаковке машинного кода выдвигаются дополнительные требования: модули, предназначенные для упаковки, должны компилироваться с ключом `-for-pack`, определяющим конечное имя пакета. Самый простой способ организовать упаковку – позволить утилите `ocamlbuild` самой определить аргументы командной строки. Поэтому рассмотрим его на следующем простом примере.

Сначала создадим пару игрушечных модулей с именами *A.ml* и *B.ml*, содержащих по одному значению. Нам также потребуется файл *_tags*, который будет добавлять флаг *-for-pack* для файлов *.cmx* (обратите внимание, что в нем нужно явно исключить саму цель упаковки). И наконец, файл *X.mlpack*, содержащий список модулей, подлежащих упаковке в единый модуль *X*. Существуют специальные правила в *ocamlbuild*, которые определяют порядок отображения файлов *%.mlpack* в упакованные эквиваленты *%.cmx* или *%.cmo*:

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/packing/show_files.out

```
$ cat A.ml
let v = "hello"
$ cat B.ml
let w = 42
$ cat _tags
<*.cmx> and not "X.cmx": for-pack(X)
$ cat X.mlpack
A
B
```

Теперь можно напрямую вызвать утилиту *corebuild* для сборки файла *X.cmx*, но давайте для полноты примера создадим новый модуль, который будет скомпонован с модулем *X*:

OCaml

<https://github.com/realworldocaml/examples/blob/v1/code/packing/test.ml>

```
let v = X.A.v
let w = X.B.w
```

Теперь можно скомпилировать этот тестовый модуль и посмотреть, какой интерфейс будет выведен в результате упаковки содержимого *X*. Затем мы исследуем интерфейс, импортируемый модулем *Test*, и убедимся, что ни один из двух модулей, *A* и *B*, нигде не упоминается – используется только упакованный модуль *X*:

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/packing/build_test.out

```
$ corebuild test.inferred.mli test.cmi
$ cat _build/test.inferred.mli
val v : string
val w : int
$ ocamlobjinfo _build/test.cmi
File _build/test.cmi
Unit name: Test
Interfaces imported:
  906fc1b74451f0c24ceaa085e0f26e5f Test
  36b5bc8227dc9914c6d9fd9bdcfad45 Pervasives
  25f4b4e10ec64c56b2987f5900045fec X
```

Упаковка и пути поиска

Одной из распространенных ошибок, возникающих при упаковке, является ошибочная сборка упакованных файлов *.cmi* в каталогах с вложенными модулями. Добавляя эти

каталоги в путь поиска, вы делаете видимыми и вложенные модули. Соответственно, если вы забудете включить имя модуля верхнего уровня в качестве префикса (например, `X.A`) и будете использовать вложенный модуль (`A`) непосредственно, программа будет скомпилирована и скомпонована без каких-либо ошибок.

Однако типы `A` и `X.A` не становятся эквивалентными автоматически, поэтому механизм контроля типов будет жаловаться на попытки смешивания упакованных и неупакованных версий библиотеки.

В основном подобные проблемы случаются с модульными тестами, которые компилируются вместе с библиотеками. Их можно избежать, помня о необходимости открывать в тестах только упакованные модули, или использовать библиотеки только после их установки (и, соответственно, без экспортирования промежуточных модулей).

Сокращение путей к модулям в сообщениях об ошибках

Библиотека `Core` широко использует систему модулей `OCaml`, чтобы обеспечить полноценную замену стандартной библиотеки. Она собирает все свои модули в единый модуль `Std`, открыть который достаточно, чтобы получить доступ ко всем замещающим модулям и функциям.

Впрочем, у такого подхода есть один недостаток: сообщения об ошибках получаются более многословными, чем хотелось бы.

Увидеть их можно, если запустить «родную» интерактивную оболочку `OCaml` (не *utor*).

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/front-end/short_paths_1.out

```
$ ocaml
# List.map print_endline "" ;;
Error: This expression has type string but an expression was expected of type
      string list
(Ошибка: Это выражение имеет тип string, тогда как ожидалось выражение типа
      string list)
```

Без использования `Core.Std` сообщение выглядит достаточно очевидным. Но стоит подключить библиотеку `Core`, и сообщение становится более многословным:

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/front-end/short_paths_2.out

```
$ ocaml
# open Core.Std ;;
# List.map ~f:print_endline "" ;;
Error: This expression has type string but an expression was expected of type
      'a Core.Std.List.t = 'a list
(Ошибка: Это выражение имеет тип string, тогда как ожидалось выражение типа
      'a Core.Std.List.t = 'a list)
```

Стандартный модуль `List` переопределяется модулем `Core.Std.List`. Компилятор старается максимально отразить эквивалентность типов, что и приводит к удлинению сообщений об ошибках.

Исправить этот недостаток можно за счет применения так называемых эвристик сокращения путей. Это заставит компилятор отыскивать все псевдонимы типов с более короткими путями к модулям и использовать их при выводе сообщений.

Данный режим активируется передачей компилятору ключа `-short-paths` и может также применяться в интерактивной оболочке.

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/front-end/short_paths_3.out

```
$ ocaml -short-paths
# open Core.Std;;
# List.map ~f:print_endline "foo";;
Error: This expression has type string but an expression was expected of type 'a list
(Ошибка: Это выражение имеет тип string, тогда как ожидалось выражение типа 'a list)
```

Улучшенная интерактивная оболочка *utop* активирует этот режим по умолчанию, именно поэтому мы не обращали вашего внимания на данную особенность раньше. Однако сам компилятор не использует эвристику сокращения путей по умолчанию, так как в некоторых ситуациях бывает полезно видеть полную информацию о типе.

Вам придется самим определить для себя наиболее предпочтительный режим компиляции: с выводом коротких путей или по умолчанию и передавать или не передавать флаг `-short-paths` компилятору.

Типизированное синтаксическое дерево

После успешного завершения процедуры проверки типов информация о них встраивается в AST, в результате чего получается *типизированное абстрактное синтаксическое дерево* (*typed abstract syntax tree*). Оно содержит полную информацию о местоположении и конкретном типе каждой лексемы в исходном файле.

Компилятор может вывести это дерево в виде скомпилированных файлов `.cmt` и `.cmi` для реализации и сигнатуры единицы компиляции соответственно. Эта операция активируется передачей компилятору ключа `-bin-annot`.

Файлы `.cmt` особенно полезны для инструментов в интегрированных средах разработки, поддерживающих возможность сопоставления исходного кода на OCaml в определенном местоположении с выведенными или внешними типами.

Использование `ocp-index` для поддержки автодополнения

Одним из таких инструментов командной строки, который можно использовать в текстовых редакторах для поддержки функции автодополнения, является *ocp-index*. Установить его можно из OPAM, как показано ниже:

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/front-end/install_ocp_index.out

```
$ opam install ocp-index
$ ocp-index
```

Давайте вернемся назад к нашему примеру реализации интерфейса к библиотеке Ncurses, который рассматривался в главе 19. В нем мы определили модуль для доступа к библиотеке Ncurses. Для начала скомпилируем его интерфейс с ключом

-bin-annot, чтобы получить файлы *.cmt* и *.cmti*, а затем опробуем утилиту *ocp-index* в режиме дополнения:

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/ocp-index/index_ncurses.out

```
$ corebuild -pkg ctypes.foreign -tag bin_annot ncurses.cmi
$ ocp-index complete -I . Ncur
Ncurses module
$ ocp-index complete -I . Ncurses.a
Ncurses.addstr val string -> unit
$ ocp-index complete -I . Ncurses.
Ncurses.window val Ncurses.window Ctypes.typ
Ncurses.wrefresh val Ncurses.window -> unit
Ncurses.initscr val unit -> Ncurses.window
Ncurses.endwin val unit -> unit
Ncurses.refresh val unit -> unit
Ncurses.newwin val int -> int -> int -> int -> Ncurses.window
Ncurses.mvwaddch val Ncurses.window -> int -> int -> char -> unit
Ncurses.mvwaddstr val Ncurses.window -> int -> int -> string -> unit
Ncurses.addstr val string -> unit
Ncurses.box val Ncurses.window -> char -> char -> unit
Ncurses.cbreak val unit -> int
```

Утилите *ocp-index* нужно передать список каталогов, где искать файлы *.cmt*, и фрагмент текста для автодополнения. Как вы наверняка понимаете, автодополнение — ценная функция при работе с большими проектами. Загляните на домашнюю страницу проекта *ocp-index*, где приводится дополнительная информация о встраивании этой утилиты в текстовые редакторы¹.

Непосредственное исследование типизированного синтаксического дерева

Компилятор поддерживает два дополнительных флага, передав которые, можно вывести дерево AST во внутреннем представлении. Не следует рассчитывать, что разные версии компилятора будут выводить одну и ту же информацию при вызове с этими флагами. Тем не менее они являются очень полезным инструментом исследований.

Воспользуемся вновь нашим игрушечным модулем *typedef.ml*:

OCaml

<https://github.com/realworldocaml/examples/blob/v1/code/front-end/typedef.ml>

```
type t = Foo | Bar
let v = Foo
```

Для начала посмотрим, как выглядит нетипизированное синтаксическое дерево, сгенерированное на этапе парсинга:

¹ <https://github.com/ocamlpro/ocp-index>.

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/front-end/parsetree_typedef.out

```
$ ocamlc -dparsetree typedef.ml 2>&1
[
  structure_item (typedef.ml[1,0+0]..[1,0+18])
    Pstr_type
    [
      "t" (typedef.ml[1,0+5]..[1,0+6])
        type_declaration (typedef.ml[1,0+5]..[1,0+18])
          ptype_params =
            []
          ptype_cstrs =
            []
          ptype_kind =
            Ptype_variant
            [
              (typedef.ml[1,0+9]..[1,0+12])
                "Foo" (typedef.ml[1,0+9]..[1,0+12])
                []
                None
              (typedef.ml[1,0+15]..[1,0+18])
                "Bar" (typedef.ml[1,0+15]..[1,0+18])
                []
                None
            ]
          ptype_private = Public
          ptype_manifest =
            None
    ]
  structure_item (typedef.ml[2,19+0]..[2,19+11])
    Pstr_value Nonrec
    [
      <def>
        pattern (typedef.ml[2,19+4]..[2,19+5])
          Ppat_var "v" (typedef.ml[2,19+4]..[2,19+5])
        expression (typedef.ml[2,19+8]..[2,19+11])
          Pexp_construct "Foo" (typedef.ml[2,19+8]..[2,19+11])
          None
          false
    ]
  ]
]
```

Получился довольно большой объем вывода для простой двухстрочной программы, но он наглядно показывает, какие объемные структуры генерирует парсер OCaml даже для коротких файлов с исходным кодом.

Каждый фрагмент AST снабжен точной информацией о местоположении (включая имя файла и местоположение лексемы в этом файле). Этот код еще не был подвергнут проверке системой типов, поэтому в него включены все лексемы.

Типизированное дерево AST, которое обычно выводится в файл *.cmt* в скомпилированном виде, можно получить в более удобочитаемом виде с помощью ключа `-dtypedtree`:

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/front-end/typedtree_typedef.out

```
$ ocamlc -dtypedtree typedef.ml 2>&1
[
  structure_item (typedef.ml[1,0+0]..typedef.ml[1,0+18])
    Pstr_type
    [
      t/1008
      type_declaration (typedef.ml[1,0+5]..typedef.ml[1,0+18])
        ptype_params =
          []
        ptype_cstrs =
          []
        ptype_kind =
          Ptype_variant
          [
            "Foo/1009"
            []
            "Bar/1010"
            []
          ]
        ptype_private = Public
        ptype_manifest =
          None
    ]
  structure_item (typedef.ml[2,19+0]..typedef.ml[2,19+11])
    Pstr_value Nonrec
    [
      <def>
        pattern (typedef.ml[2,19+4]..typedef.ml[2,19+5])
          Ppat_var "v/1011"
        expression (typedef.ml[2,19+8]..typedef.ml[2,19+11])
          Pexp_construct "Foo"
          []
          false
    ]
  ]
]
```

Типизированное дерево AST более явное, чем нетипизированное. Например, объявление типа получило уникальное имя (*t/1008*), как и значение *v* (*v/1011*).

Вам редко придется рассматривать такие деревья, полученные от компилятора, если только вы не собираетесь заниматься созданием собственных инструментов разработки, таких как *ocp-index*, или расширений для самого компилятора. Однако знание о существовании подобной промежуточной формы будет полезным,

поскольку оно пригодится, когда мы начнем погружение в процесс создания выполняемого кода, освещаемого в следующей главе.

Существует несколько новых инструментов, позволяющих организовать использование этих типизированных деревьев AST в широко известных редакторах, таких как Emacs или Vim. В число лучших из них входит Merlin¹, добавляющий поддержку автодополнения, отображающий выведенные типы и способный выводить сообщения об ошибках непосредственно в текстовом редакторе. Все необходимые инструкции по встраиванию инструмента Merlin в текстовые редакторы вы найдете на домашней странице проекта.

¹ <https://github.com/the-lambda-church/merlin>.

Компиляторы: байт-код и машинный код

После выполнения этапа проверки типов компилятор OCaml может остановиться и вывести сообщения об ошибках, если таковые обнаружены. Затем он начинает процесс компиляции модулей, не содержащих ошибок, в выполняемый код.

В этой главе мы рассмотрим следующие темы:

- нетипизированный промежуточный *lambda*-код с оптимизированными операциями сопоставления;
- компилятор байт-кода *ocamlc* и интерпретатор *ocamlrun*;
- генератор машинного кода *ocamlopt*, отладка и профилирование машинного кода.

Нетипизированная *lambda*-форма

На первом этапе создания выполняемого кода удаляется вся информация о статических типах и создается промежуточный код в упрощенной *lambda*-форме. В этой форме отсутствуют какие-либо высокоуровневые конструкции, такие как модули и объекты, которые замещаются более простыми значениями, такими как записи и указатели на функции. Также выполняются анализ операций сопоставления с образцом и их компиляция в высокооптимизированные автоматы.

Этап создания *lambda*-формы является ключевой стадией, на которой происходит удаление информации о типах, а исходный код отображается в модель памяти времени выполнения, описанную в главе 20. На этой стадии также выполняются некоторые оптимизации, наиболее значимая из которых – преобразование инструкций сопоставления с образцом в более оптимальные низкоуровневые инструкции.

Оптимизация сопоставлений с образцом

Если передать компилятору ключ `-dlambda`, он выведет *lambda*-код с применением синтаксиса *s*-выражений. Давайте воспользуемся этой возможностью, чтобы больше узнать о том, как действует механизм сопоставления с образцом в языке OCaml, для чего напомним три разные инструкции сопоставления и сравним их *lambda*-код.

Сначала напомним простое и исчерпывающее сопоставление с тремя обычными вариантами:

OCaml

https://github.com/realworldocaml/examples/blob/v1/code/back-end/pattern_monomorphic_large.ml

```
type t = | Alice | Bob | Charlie | David
```

```
let test v =
  match v with
  | Alice   -> 100
  | Bob     -> 101
  | Charlie -> 102
  | David   -> 103
```

Ниже показан lambda-код этого фрагмента:

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/back-end/lambda_for_pattern_monomorphic_large.out

```
$ ocamlc -dlambda -c pattern_monomorphic_large.ml 2>&1
(setglobal Pattern_monomorphic_large!
  (let
    (test/1013
      (function v/1014
        (switch* v/1014
          case int 0: 100
          case int 1: 101
          case int 2: 102
          case int 3: 103)))
    (makeblock 0 test/1013)))
```

Не следует стремиться понять все тонкости внутреннего представления, которое официально не документировано, так как может изменяться между версиями компилятора. Несмотря на это, рассмотрим все же некоторые наиболее интересные моменты.

- Здесь полностью отсутствуют понятия модулей или типов. Глобальные значения создаются с помощью `setglobal`, а значения OCaml конструируются с помощью `makeblock`. Блоки — это значения времени выполнения, о которых рассказывалось в главе 20.
- Сопоставление с образцом превратилось в инструкцию выбора, выполняющую переход непосредственно к нужному варианту, исходя из тега заголовка `v`. Напомним, что варианты без параметров хранятся в памяти в виде целых чисел в том порядке, в каком они определены в программе. Механизм сопоставления с образцом знает об этом и трансформирует образец в эффективную таблицу переходов.
- Значения адресуются уникальными именами, позволяющими отличать затененные значения за счет добавления числа в конец имени (например, `v/1014`). Безопасность типов, проверенная на более раннем этапе компиляции, гарантирует, что низкоуровневые операции доступа не будут приводить к разрушению данных во время выполнения, поэтому на данном уровне не требуется выполнять каких-либо дополнительных проверок. Однако

неразумное использование небезопасных инструментов, таких как модуль `Obj.magic`, все еще может вызывать аварийные ситуации.

Компилятор конструирует таблицу переходов, включающую все четыре варианта. Если уменьшить число вариантов до двух, тогда отпадет необходимость в сложностях, связанных с вычислением такой таблицы:

OCaml

https://github.com/realworldocaml/examples/blob/v1/code/back-end/pattern_monomorphic_small.ml

```
type t = | Alice | Bob
```

```
let test v =
  match v with
  | Alice -> 100
  | Bob -> 101
```

Lambda-код этого фрагмента выглядит совсем иначе:

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/back-end/lambda_for_pattern_monomorphic_small.out

```
$ ocamlc -dlambda -c pattern_monomorphic_small.ml 2>&1
(setglobal Pattern_monomorphic_small!
  (let (test/1011 (function v/1012 (if (!= v/1012 0) 101 100)))
    (makeblock 0 test/1011)))
```

Вместо таблицы переходов компилятор генерирует простой условный переход, потому что он статически определил, что диапазон возможных вариантов достаточно мал для этого. Наконец, давайте посмотрим на тот же код, но уже с полиморфными вариантами:

OCaml

https://github.com/realworldocaml/examples/blob/v1/code/back-end/pattern_polymorphic.ml

```
let test v =
  match v with
  | `Alice -> 100
  | `Bob -> 101
  | `Charlie -> 102
  | `David -> 103
  | `Eve -> 104
```

Lambda-представление этого кода также демонстрирует, как выглядят полиморфные варианты во время выполнения:

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/back-end/lambda_for_pattern_polymorphic.out

```
$ ocamlc -dlambda -c pattern_polymorphic.ml 2>&1
(setglobal Pattern_polymorphic!
  (let
```



```

(test/1008
  (function v/1009
    (if (!= v/1009 3306965)
      (if (>= v/1009 482771474) (if (>= v/1009 884917024) 100 102)
        (if (>= v/1009 3457716) 104 103))
      101)))
(makeblock 0 test/1008)))

```

В главе 6 уже упоминалось, что использование полиморфных вариантов в операциях сопоставления с образцом менее эффективно, и теперь вам должно быть понятно – почему. Полиморфные варианты имеют значения времени выполнения, которые вычисляются путем хэширования имен вариантов, из-за чего компилятор не может использовать таблицу переходов, как при использовании обычных вариантов. Вместо этого он создает дерево решений, с помощью которого сравнивает хэш-значения с исходной переменной за наименьшее возможное число сравнений.

Дополнительные сведения о компиляции операций сопоставления с образцом

Сопоставление с образцом является одной из важнейших операций в программировании на языке OCaml. Вы часто будете сталкиваться с глубоко вложенными операциями сопоставления со сложными структурами данных. Отличное описание фундаментальных алгоритмов, реализованных в компиляторе OCaml, можно найти в статье «Optimizing pattern matching» Фабриса ле Фессанта (Fabrice Le Fessant) и Люка Мароже (Luc Marandet).

В статье описывается алгоритм поиска с возвратом (backtracking algorithm), используемый в классических реализациях компиляции сопоставлений с образцом, а также некоторые оптимизации, специфичные для OCaml, такие как использование информации о полноте вариантов и управление потоком выполнения через статические исключения. Разумеется, для простого использования сопоставления с образцом не требуется полного понимания внутренних механизмов этой операции, но эти знания помогут понять, почему данная операция настолько проста в использовании в языке OCaml.

Оценка производительности сопоставления с образцом

Давайте исследуем производительность этих трех приемов сопоставления, чтобы точнее оценить их стоимость во время выполнения. Модуль `Core_bench` выполняет тестовый фрагмент несколько тысяч раз и вычисляет также статистический разброс результатов. Выполните команду `opam install core_bench`, чтобы установить библиотеку:

OCaml

https://github.com/realworldocaml/examples/blob/v1/code/back-end-bench/bench_patterns.ml

```

open Core.Std
open Core_bench.Std

```

```

type t = | Alice | Bob
type s = | A | B | C | D | E

```

```

let polymorphic_pattern () =
  let test v =
    match v with

```

```

    | `Alice   -> 100
    | `Bob     -> 101
    | `Charlie -> 102
    | `David   -> 103
    | `Eve     -> 104
  in
  List.iter ~f:(fun v -> ignore(test v))
    [ `Alice; `Bob; `Charlie; `David ]

let monomorphic_pattern_small () =
  let test v =
    match v with
    | Alice -> 100
    | Bob   -> 101
  in
  List.iter ~f:(fun v -> ignore(test v))
    [ Alice; Bob ]

let monomorphic_pattern_large () =
  let test v =
    match v with
    | A -> 100
    | B -> 101
    | C -> 102
    | D -> 103
    | E -> 104
  in
  List.iter ~f:(fun v -> ignore(test v))
    [ A; B; C; D ]

let tests = [
  "Polymorphic pattern", polymorphic_pattern;
  "Monomorphic larger pattern", monomorphic_pattern_large;
  "Monomorphic small pattern", monomorphic_pattern_small;
]

let () =
  List.map tests ~f:(fun (name,test) -> Bench.Test.create ~name test)
  |> Bench.make_command
  |> Command.run

```

После сборки и запуска этот пример будет работать примерно 30 секунд, после чего вы увидите результаты, сведенные в таблицу:

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/back-end-bench/run_bench_patterns.out

```

$ corebuild -pkg core_bench bench_patterns.native
$ ./bench_patterns.native -ascii
Estimated testing time 30s (change using -quota SECS).

```

Name	Time (ns)	% of max
Polymorphic pattern	31.51	100.00
Monomorphic larger pattern	29.19	92.62
Monomorphic small pattern	16.25	51.57

Эти результаты подтверждают предположения о производительности, сделанные выше на основе анализа `lambda`-кода. Самое маленькое время выполнения показывает операция сопоставления, скомпилированная как короткая условная инструкция, а инструкция сопоставления с полиморфными вариантами оказалась самой медленной. Эти примеры не показывают каких-то огромных различий, но те же приемы вы можете использовать в своих программах для выявления узких мест производительности.

`Lambda`-код является в первую очередь переходным мостиком к выполняемому байт-коду, о котором рассказывается далее. Часто гораздо проще исследовать результаты, полученные на этом этапе, чем копаться в ассемблерном коде, полученном из скомпилированных выполняемых файлов.

Переносимый байт-код

После того как компилятор сгенерирует `lambda`-код, он оказывается на расстоянии вытянутой руки от выполняемого кода. В этой точке комплект инструментов `OCaml` делится на два отдельных компилятора. Сначала мы познакомимся с компилятором байт-кода, который состоит из двух компонентов:

- `ocamlc` – компилирует файлы в байт-код, близко отражающий структуру `lambda`-кода;
- `ocamlrun` – переносимый интерпретатор, выполняющий байт-код.

Самые большие преимущества байт-кода: простота, переносимость и скорость компиляции. Преобразование `lambda`-кода в байт-код выполняется очень просто, в результате чего получается выполняемый код с предсказуемой (хотя и невысокой) скоростью выполнения.

Интерпретатор байт-кода реализует виртуальную машину на основе стека. Стек `OCaml` и связанный с ним аккумулятор способны хранить следующие значения:

- *длинные целые* – эти значения соответствуют типу `int` языка `OCaml`;
- *блоки* – значения, состоящие из заголовка блока и адреса в памяти, где хранятся поля данных, содержащие фактические значения `OCaml`, индексированные целыми числами;
- *смещения кода* – значения, представляющие относительные адреса кода.

Интерпретатор виртуальной машины имеет всего семь регистров: программный счетчик, указатель стека, аккумулятор, указатели на исключение и аргумент, а также указатели на переменные окружения и глобальные данные. Вывести инструкции байт-кода в текстовом виде можно с помощью ключа `-dinstr`. Попробуем проделать это с одним из предыдущих примеров сопоставления с образцом:

Terminal

```
https://github.com/realworldocaml/examples/tree/v1/code/back-end/
instr_for_pattern_monomorphic_small.out
```

```
$ ocamlc -dinstr pattern_monomorphic_small.ml 2>&1
      branch L2
L1: acc 0
      push
```

```

    const 0
    negint
    branchifnot L3
    const 101
    return 1
L3: const 100
    return 1
L2: closure L1, 0
    push
    acc 0
    makeblock 1, 0
    pop 1
    setglobal Pattern_monomorphic_small!

```

Предыдущий байт-код является результатом преобразования lambda-кода в последовательность простых инструкций, выполняемых интерпретатором.

Всего существует около 140 инструкций, но большинство из них являются лишь вариантами типичных операций (например, применение функции к определенному числу операндов). Подробное описание полного набора инструкций можно найти по адресу: <http://cadmium.x9c.fr/distrib/caml-instructions.pdf>.

Как определялся набор инструкций байт-кода?

Интерпретация байт-кода производится существенно медленнее, чем выполнение машинного кода, и все же интерпретатор OCaml входит в число самых высокопроизводительных интерпретаторов, не использующих динамическую (JIT) компиляцию. Его история восходит к инновационному труду Ксавье Леруа (Xavier Leroy) «The ZINC experiment: An Economical Implementation of the ML Language»¹.

Эта статья закладывает теоретические основы для реализации набора инструкций строго функционального языка программирования, такого как OCaml. Интерпретатор байт-кода в современном OCaml до сих пор основывается на модели ZINC. Компилятор машинного кода использует иную модель, потому что при вызове функций он может передавать аргументы через регистры процессора, а не на стеке, как это вынужден делать интерпретатор байт-кода.

Изучение причин различий между реализациями интерпретатора байт-кода и компилятора машинного кода является очень важным упражнением для любого начинающего специалиста по языкам программирования.

Компиляция и компоновка байт-кода

Команда *ocamlc* компилирует отдельные файлы *.ml* в файлы *.cmo* с байт-кодом. Скомпилированные файлы с байт-кодом соответствуют одноименным интерфейсам *.cmi*, посредством которых экспортируют сигнатуры типов для использования в других единицах компиляции.

Типичная библиотека OCaml состоит из нескольких файлов с исходным кодом, в результате компиляции которых образуется несколько файлов *.cmo*, которые необходимо передавать в командной строке при использовании библиотеки в другом коде. Компилятор может объединить несколько файлов в более удобный единый архивный файл, если передать ему ключ *-a*. Архивы с байт-кодом отличаются расширением *.cma*.

¹ <http://hal.inria.fr/docs/00/07/00/49/PS/RT-0117.ps>.

Отдельные объекты в библиотеке компонуются как обычные файлы *.сто* в порядке, который был определен при создании файла библиотеки. Если файл из библиотеки нигде не используется в программе, он не включается в окончательный выполняемый файл, если только не указан ключ *-linkall*, вынуждающий включать все объекты. Аналогичным образом обрабатываются объектные файлы и архивы в языке С (*.о* и *.а* соответственно).

Затем файлы с байт-кодом объединяются (компонуются) со стандартной библиотекой *OCaml* для создания выполняемой программы. Порядок следования аргументов *.сто* в командной строке определяет порядок, в каком соответствующие единицы компиляции будут инициализироваться во время выполнения. Не забывайте, что в *OCaml* нет единственной функции *main*, как в языке С, поэтому порядок компоновки файлов в *OCaml* играет более важную роль, чем в С.

Выполнение байт-кода

Среда выполнения байт-кода состоит из трех основных компонентов: интерпретатора байт-кода, сборщика мусора и множества функций на языке С, реализующих простейшие операции. Байт-код содержит инструкции для вызова этих функций на С, когда это необходимо.

Компоновщик *OCaml* производит байт-код, по умолчанию предназначенный для работы под управлением стандартной среды выполнения *OCaml*, поэтому он должен знать обо всех функциях на С, вызываемых из других библиотек, не загружаемых по умолчанию.

Информация об этих дополнительных библиотеках может быть указана при компоновке байт-кода в архивы:

Terminal

```
https://github.com/realworldocaml/examples/tree/v1/code/back-end-embed/link_dllib.out
$ ocamlc -a -o mylib.cma a.cmo b.cmo -dllib -lmylib
```

Флаг *dllib* внедряет аргументы в файл архива. При компоновке любых пакетов с этим архивом в последующем также будут включаться дополнительные директивы компоновки с кодом на С. Это позволит интерпретатору динамически загрузить символы внешней библиотеки при выполнении байт-кода.

Можно также сгенерировать автономный выполняемый файл, включающий интерпретатор *ocamlrun* и байт-код в виде единственной библиотеки. Такие файлы создаются следующим образом:

Terminal

```
https://github.com/realworldocaml/examples/tree/v1/code/back-end-embed/link_custom.out
$ ocamlc -a -o mylib.cma -custom a.cmo b.cmo -cclib -lmylib
```

Утилита *OCamlbuild* заботится о многих из этих тонкостей в соответствии со встроенными в нее правилами. Правило *%.byte*, использовавшееся на протяжении всей книги, обеспечивает создание выполняемого байт-кода, а добавление тега *custom* приводит к встраиванию интерпретатора в выполняемый файл.

Создание автономного выполняемого файла с байт-кодом близко напоминает компиляцию в машинный код, так как в обоих случаях создаются автономные выполняемые файлы. Существует также множество других флагов и ключей управления компиляцией в байт-код (особенно примечательными из них являются флаги, управляющие компоновкой с разделяемыми библиотеками и созданием автономных выполняемых файлов). Подробное их описание можно найти по адресу: <http://caml.inria.fr/pub/docs/manual-ocaml/comp.html#sec243>.

Встраивание байт-кода OCaml в программы на C

За компиляцией в байт-код обычно следует заключительный этап компоновки, выполняемый утилитой *ocamlc*. Однако иногда может быть необходимо встроить код на OCaml в существующее приложение на C. Этот режим работы компилятора поддерживается с помощью директивы `-output-obj`.

Эта директива вынуждает *ocamlc* сохранить результат своей работы в объектном файле вместе с функцией `caml_startup`. Все модули OCaml будут скомпонованы в этот объектный файл в форме байт-кода, как если бы составляли обычную автономную программу на OCaml.

Этот объектный файл может компоноваться с программами на C с помощью стандартного компилятора C и нуждается только в библиотеке поддержки байт-кода (устанавливается как *libcamlrn.a*). Чтобы создать выполняемый файл, достаточно включить в компоновку программы библиотеку времени выполнения и объектный файл с байт-кодом. Ниже приводится пример, демонстрирующий возможность встраивания кода на OCaml в код на C.

Создайте два файла с исходным кодом на OCaml, содержащих вывод простых строк:

OCaml

https://github.com/realworldocaml/examples/blob/v1/code/back-end-embed/embed_me1.ml

```
let () = print_endline "hello embedded world 1"
```

OCaml

https://github.com/realworldocaml/examples/blob/v1/code/back-end-embed/embed_me2.ml

```
let () = print_endline "hello embedded world 2"
```

Затем создайте файл с исходным кодом на C, который будет служить главной точкой входа в программу:

C

<https://github.com/realworldocaml/examples/blob/v1/code/back-end-embed/main.c>

```
#include <stdio.h>
#include <caml/alloc.h>
#include <caml/mlvalues.h>
#include <caml/memory.h>
#include <caml/callback.h>
```

```
int
main (int argc, char **argv)
{
    printf("Before calling OCaml\n");
    fflush(stdout);
    caml_startup (argv);
    printf("After calling OCaml\n");
    return 0;
}
```

Теперь скомпилируйте файлы OCaml в отдельный объектный файл:

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/back-end-embed/build_embed.out

```
$ rm -f embed_out.c
$ ocamlc -output-obj -o embed_out.o embed_me1.ml embed_me2.ml
```

После этого компилятор OCaml больше не нужен, так как файл *embed_out.o* содержит весь необходимый код в скомпилированном виде. Скомпилируйте конечный выполняемый файл с помощью *gcc*:

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/back-end-embed/build_embed_binary.out

```
$ gcc -fPIC -Wall -I`ocamlc -where` -L`ocamlc -where` -ltermcap -lm -ldl \
  -o finalbc.native main.c embed_out.o -lcamlrun
$ ./finalbc.native
Before calling OCaml
hello embedded world 1
hello embedded world 2
After calling OCaml
```

Вызвав компилятор *ocamlc* с ключом *-verbose*, можно увидеть, какие аргументы командной строки передаются компилятору GCC. При создании объектного файла можно даже получить исходный код на языке C, указав в директиве *-output-obj* имя файла с расширением *.c* вместо *.o*:

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/back-end-embed/build_embed_c.out

```
$ ocamlc -output-obj -o embed_out.c embed_me1.ml embed_me2.ml
```

Поддержка встраивания кода на OCaml, продемонстрированная выше, позволяет писать код, способный взаимодействовать с любыми окружениями, поддерживаемыми компилятором C. Есть даже возможность вызывать отдельные функции на OCaml из кода на C, для чего с помощью модуля *Callback* следует зарегистрировать именованные точки входа в код на OCaml. Подробнее об этом рассказывается в разделе «Interfacing C with OCaml» руководства по языку OCaml¹.

¹ <http://caml.inria.fr/pub/docs/manual-ocaml/intfc.html>.

Компиляция быстрого машинного кода

Компилятор в машинный код является тем инструментом, с помощью которого создается большая часть промышленного кода OCaml. Он компилирует lambda-код в быстрый машинный код, выполняя межмодульное встраивание (cross-module inlining) и дополнительные оптимизации, которые не выполняются компилятором байт-кода. Особое внимание уделяется совместимости со средой выполнения байт-кода, в том смысле что один и тот же код должен действовать одинаково, независимо от того, какой комплект инструментов использовался для компиляции.

Команда *ocamlopt*, являющаяся интерфейсом к компилятору в машинный код, близко напоминает компилятор *ocamlc*. Она также принимает файлы *.ml* и *.mli*, но на их основе создает:

- файлы *.o* с объектным кодом;
- файлы *.cmx* с дополнительной информацией для компоновки и оптимизации;
- файлы *.cmi* скомпилированного интерфейса – фактически те же, что создает компилятор байт-кода.

Когда компилятор компоует модули в выполняемый файл, он использует содержимое файлов *.cmx* для межмодульного встраивания (cross-module inlining) единиц компиляции. Это может значительно повысить скорость выполнения функций из стандартной библиотеки, которые часто используются за пределами своего модуля.

Коллекции файлов *.cmx* и *.o* могут также упаковываться в архивы *.cmxa*, для чего компилятору следует передать флаг *-a*. Однако, в отличие от версии, создающей байт-код, файлы *.cmx* должны оставаться в пути поиска компилятора, чтобы они были доступны механизму межмодульного встраивания (cross-module inlining). Если этого не сделать, компиляция все еще будет выполняться успешно, но некоторые важнейшие оптимизации не будут выполнены, и вы получите выполняемые файлы, действующие медленнее.

Исследование ассемблерного кода

Компилятор машинного кода генерирует код на языке Ассемблера, который затем передается системному транслятору для его компиляции в объектные файлы. Чтобы получить код на языке Ассемблера, достаточно передать компилятору *ocamlopt* флаг *-S*.

Ассемблерный код очень тесно связан с аппаратной архитектурой, поэтому в дальнейшем обсуждении предполагается, что компиляция выполняется на 64-разрядной архитектуре Intel или AMD. Пример кода был сгенерирован с использованием флагов *-inline20* и *-nodynlink*, обеспечивающих создание ассемблерного кода со всеми оптимизациями, поддерживаемыми компилятором. Даже при том, что эти оптимизации немного ухудшают читаемость кода, их применение позволит вам точно увидеть, какой код выполняет процессор. Не забывайте, что для получения более общей картины можно использовать lambda-код, если вдруг почувствуете, что заблудились в ассемблерном лесу.

Влияние полиморфного сравнения

Мы предупреждали вас в главе 13, что использование полиморфного сравнения, с одной стороны, удобно, с другой – рискованно. Давайте теперь рассмотрим точные отличия на уровне языка Ассемблера.

Прежде всего напишем функцию сравнения, которую снабдим аннотациями типов, чтобы компилятор знал, что она будет использоваться только для сравнения целых чисел:

OCaml

https://github.com/realworldocaml/examples/blob/v1/code/back-end/compare_mono.ml

```
let cmp (a:int) (b:int) =
  if a > b then a else b
```

Теперь скомпилируем ее в ассемблерный код, в файле *compare_mono.S*, и посмотрим, что получилось. В некоторых системах, таких как Linux, расширение этого файла может быть символом в нижнем регистре:

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/back-end/asm_from_compare_mono.out

```
$ ocamlc -inline 20 -nodynlink -S compare_mono.ml
```

Если прежде вам не доводилось сталкиваться с ассемблерным кодом, он может показаться вам немного пугающим. Чтобы понять этот код, может потребоваться изучить язык Ассемблера x86, поэтому мы постараемся рассказать вам о некоторых инструкциях подробнее, чтобы вы могли увидеть некоторые шаблоны. Фрагмент, соответствующий реализации функции `cmp`, приводится ниже:

Assembly

<https://github.com/realworldocaml/examples/blob/v1/code/back-end/cmp.S>

```
_camlCompare_mono_cmp_1008:
    .cfi_startproc
.L101:
    cmpq    %rbx, %rax
    jle     .L100
    ret
    .align  2
.L100:
    movq    %rbx, %rax
    ret
    .cfi_endproc
```

Здесь `_camlCompare_mono_cmp_1008` – это ассемблерная метка, которая была получена путем объединения имени модуля (`Compare_mono`) с именем функции (`cmp_1008`). Числовое окончание в имени функции получено из `lambda`-кода (который можно получить с помощью ключа `-dlambda`, но в данном случае в этом нет необходимости).

Аргументы передаются функции `cmp` в регистрах `%rbx` и `%rax`, а сравнение выполняется с помощью инструкции `jle` «jump if less than or equal» (перейти, если мень-

ше или равно). Для этого требуется, чтобы оба аргумента были целыми числами со знаком. Теперь давайте посмотрим, что случится, если убрать аннотации из кода на языке OCaml, чтобы выполнялось полиморфное сравнение:

OCaml

https://github.com/realworldocaml/examples/blob/v1/code/back-end/compare_poly.ml

```
let cmp a b =
  if a > b then a else b
```

Результат компиляции этого кода с флагом `-S` получается существенно сложнее:

Assembly

https://github.com/realworldocaml/examples/blob/v1/code/back-end/compare_poly_asm.S

```
_camlCompare_poly_cmp_1008:
    .cfi_startproc
    subq    $24, %rsp
    .cfi_adjust_cfa_offset 24
.L101:
    movq    %rax, 8(%rsp)
    movq    %rbx, 0(%rsp)
    movq    %rax, %rdi
    movq    %rbx, %rsi
    leaq    _caml_greaterthan(%rip), %rax
    call    _caml_c_call
.L102:
    leaq    _caml_young_ptr(%rip), %r11
    movq    (%r11), %r15
    cmpq    $1, %rax
    je      .L100
    movq    8(%rsp), %rax
    addq    $24, %rsp
    .cfi_adjust_cfa_offset -24
    ret
    .cfi_adjust_cfa_offset 24
    .align 2
.L100:
    movq    0(%rsp), %rax
    addq    $24, %rsp
    .cfi_adjust_cfa_offset -24
    ret
    .cfi_adjust_cfa_offset 24
    .cfi_endproc
```

Директивы `.cfi` – это подсказки Ассемблера с информацией о кадре стека вызовов (Call Frame Information), которые позволяют отладчику получать более осмысленную информацию о вызовах. Они не оказывают влияния на производительность во время выполнения. Обратите внимание, что остальная реализация больше не основывается на простом сравнении регистров. Вместо этого аргументы помещаются на стек (регистр `%rsp`) и выполняется вызов функции на языке C, в ходе которого указатель на `caml_greaterthan` помещается в регистр `%rax` и осуществляется переход к `caml_c_call`.

Реализация OCaml для архитектуры x86_64 кэширует адрес вспомогательной кучи в регистре `%r15`, так как функции на OCaml обращаются к ней очень часто. Указатель на вспомогательную кучу может также изменяться вызываемым кодом на C (например, когда он создает новые значения OCaml), поэтому после возврата из `caml_greaterthan` выполняется восстановление регистра `%r15`. Наконец, результат сравнения выталкивается со стека и возвращается вызывающему коду.

Тестирование производительности полиморфного сравнения

Не нужно быть знатоком в языке Ассемблера, чтобы понять, что такое полиморфное сравнение гораздо тяжеловеснее простого целочисленного сравнения, представленного ранее. Давайте подтвердим этот теоретический вывод практикой, написав тест `Core_bench`, измеряющий производительность обеих функций:

OCaml

https://github.com/realworldocaml/examples/blob/v1/code/back-end-bench/bench_poly_and_mono.ml

```
open Core.Std
open Core_bench.Std

let polymorphic_compare () =
  let cmp a b = if a > b then a else b in
  for i = 0 to 1000 do
    ignore(cmp 0 i)
  done

let monomorphic_compare () =
  let cmp (a:int) (b:int) =
    if a > b then a else b in
  for i = 0 to 1000 do
    ignore(cmp 0 i)
  done

let tests =
  [ "Polymorphic comparison", polymorphic_compare;
    "Monomorphic comparison", monomorphic_compare ]

let () =
  List.map tests ~f:(fun (name,test) -> Bench.Test.create ~name test)
  |> Bench.make_command
  |> Command.run
```

Результаты тестирования показывают значительную разницу в производительности:

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/back-end-bench/run_bench_poly_and_mono.out

```
$ corebuild -pkg core_bench bench_poly_and_mono.native
$ ./bench_poly_and_mono.native -asci
Estimated testing time 20s (change using -quota SECS).
```

Name	Time (ns)	% of max
Polymorphic comparison	13_919	100.00
Monomorphic comparison	815	5.86

Как видите, полиморфное сравнение почти в 20 раз медленнее! К этим результатам нельзя относиться слишком серьезно, так как этот тест очень ограничен, подобно любым другим микротестам, — он не является представительным для больших программ. Однако если вы занимаетесь созданием кода, выполняющего массивные вычисления с глубокими и вложенными циклами, есть смысл заглянуть в ассемблерный код в поисках мест, где можно применить дополнительные оптимизации вручную.

Отладка двоичных выполняемых файлов

Компилятор машинного кода создает выполняемые файлы, которые можно отлаживать с помощью обычного системного отладчика, такого как GNU *gdb*. Вам нужно лишь скомпилировать свои библиотеки с ключом `-g`, чтобы добавить в файлы отладочную информацию.

Дополнительная отладочная информация вставляется также в вывод на языке Ассемблера. В состав этой информации входят директивы CFI, на которых мы заостряли ваше внимание выше (например, `.cfi_start_proc` и `.cfi_end_proc`, обозначающие границы функций на OCaml).

Имена функций в отладчике

Итак, как же обращаться к функциям на языке OCaml в интерактивном отладчике, таком как *gdb*? Прежде всего необходимо разобраться, как имена функций OCaml отображаются в символические имена в скомпилированных объектных файлах.

При компиляции любого файла с исходным кодом на языке OCaml в объектный файл должны экспортироваться уникальные символические имена, совместимые с двоичным интерфейсом языка C. Это означает, что любые значения OCaml, которые могут использоваться другими единицами компиляции, требуется отобразить в символические имена. При этом должны учитываться особенности языка OCaml, такие как возможность вложения модулей друг в друга, определения анонимных функций и затенения имен переменных.

При преобразовании имен переменных и функций компилятор следует простым правилам:

- все символические имена начинаются с префикса `caml`, за которым следует локальное имя модуля (точки замещаются символами подчеркивания);
- далее следуют два символа подчеркивания (`_`) и имя переменной;
- к именам переменных также добавляются в конец еще один символ подчеркивания и номер, который вычисляется на этапе компиляции в `lambda`-код, когда имена переменных замещаются уникальными (в пределах модуля) значениями; увидеть эти номера можно, исследовав вывод утилиты *ocamlpt* при вызове с ключом `-dlambda`.

Символические имена, соответствующие анонимным функциям, менее предсказуемы, если не заглядывать в промежуточные файлы, производимые компилятором. Когда необходимо отладить анонимную функцию, обычно проще изменить исходный код, объявив функцию через обычную `let`-привязку.

Точки останова в отладчике GNU

Давайте посмотрим, как определяются символические имена, воспользовавшись для этого интерактивным отладчиком GNU *gdb*.

Будьте внимательны при использовании *gdb* в Mac OS X

Примеры, следующие ниже, предполагают, что вы пользуетесь отладчиком *gdb* в ОС Linux или FreeBSD. В Mac OS X 10.8 уже имеется предустановленный отладчик *gdb*, но он недостаточно надежно интерпретирует отладочную информацию, содержащуюся в двоичных выполняемых файлах. В результате он может представлять функции по их низкоуровневым именам, таким как `_L101`, игнорируя более удобочитаемые имена.

При использовании OCaml 4.1 рекомендуется выполнять отладку машинного кода на альтернативных платформах, например в Linux, или вручную дизассемблировать машинный код с применением дизассемблера, отображающего символические имена на функции OCaml.

В MacOS 10.9 отладчик *gdb* был убран полностью, и вместо него используется отладчик *lldb* из проекта LLVM. Многие рекомендации, что приводятся здесь, в равной степени применимы к этому, как и к любому другому DWARF-совместимому (Debug With Arbitrary Record Format – отладка в формате с произвольными записями), отладчику, который способен правильно интерпретировать отладочную информацию, имеющуюся в двоичных выполняемых файлах, но имейте в виду, что *lldb* имеет иной интерфейс командной строки, чем *gdb*. За дополнительной информацией обращайтесь к руководству по *lldb*.

Давайте напишем пару взаимно рекурсивных функций, выбирающих альтернативные значения из списка. Такого рода рекурсия не является хвостовой, поэтому стек вызовов будет расти с каждой итерацией:

OCaml

https://github.com/realworldocaml/examples/tree/v1/code/back-end/alternate_list.ml

```
open Core.Std
```

```
let rec take =
  function
  | [] -> []
  | hd::tl -> hd :: (skip tl)
and skip =
  function
  | [] -> []
  | _::tl -> take tl

let () =
  take [1;2;3;4;5;6;7;8;9]
  |> List.map ~f:string_of_int
  |> String.concat ~sep:", "
  |> print_endline
```

Скомпилируем этот код с отладочной информацией и выполним его. В результате должен появиться следующий вывод:

Terminal

```
https://github.com/realworldocaml/examples/tree/v1/code/back-end-bench/
run_alternate_list.out
$ corebuild -tag debug alternate_list.native
$ ./alternate_list.native -ascii
1,3,5,7,9
```

Теперь выполним этот же код под управлением *gdb*:

Terminal

```
https://github.com/realworldocaml/examples/tree/v1/code/back-end/gdb_alternate0.out
$ gdb ./alternate_list.native
GNU gdb (GDB) 7.4.1-debian
Copyright (C) 2012 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law. Type "show copying"
and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>...
Reading symbols from /home/avsm/alternate_list.native...done.
(gdb)
```

Отладчик приглашает ввести директиву отладки. Попробуем установить точку останова непосредственно перед вызовом *take*:

Terminal

```
https://github.com/realworldocaml/examples/tree/v1/code/back-end/gdb_alternate1.out
(gdb) break camlAlternate_list__take_69242
Breakpoint 1 at 0x5658d0: file alternate_list.ml, line 5.
```

Мы использовали здесь правила определения символических имен, описанные выше. В подобных случаях удобно пользоваться функцией автодополнения, которая срабатывает при нажатии клавиши табуляции: просто введите часть имени и нажмите клавишу **Tab**, чтобы увидеть список возможных вариантов.

Установив точку останова, запустите программу:

Terminal

```
https://github.com/realworldocaml/examples/tree/v1/code/back-end/gdb_alternate2.out
(gdb) run
Starting program: /home/avsm/alternate_list.native
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, camlAlternate_list__take_69242 () at alternate_list.ml:5
4         function
```

Отладчик будет выполнять программу, пока не встретит первого вызова `take`, после чего остановится в ожидании дальнейших инструкций. Отладчик *gdb* имеет массу возможностей, воспользуемся ими и посмотрим, как выглядит стек вызовов через пару рекурсивных вызовов:

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/back-end/gdb_alternate3.out

```
(gdb) cont
Continuing.

Breakpoint 1, camlAlternate_list__take_69242 () at alternate_list.ml:5
4      function
(gdb) cont
Continuing.

Breakpoint 1, camlAlternate_list__take_69242 () at alternate_list.ml:5
4      function
(gdb) bt
#0 camlAlternate_list__take_69242 () at alternate_list.ml:4
#1 0x00000000005658e7 in camlAlternate_list__take_69242 () at alternate_list.ml:6
#2 0x00000000005658e7 in camlAlternate_list__take_69242 () at alternate_list.ml:6
#3 0x00000000005659f7 in camlAlternate_list__entry () at alternate_list.ml:14
#4 0x0000000000560029 in caml_program ()
#5 0x000000000080984a in caml_start_program ()
#6 0x00000000008099a0 in ?? ()
#7 0x0000000000000000 in ?? ()
(gdb) clear camlAlternate_list__take_69242
Deleted breakpoint 1
(gdb) cont
Continuing.
1,3,5,7,9
[Inferior 1 (process 3546) exited normally]
```

Команда `cont` возобновляет выполнение программы после останова. Команда `bt` выводит трассировку стека. И команда `clear` удаляет точку останова, после чего программа может продолжать работу до полного завершения. Отладчик *gdb* имеет массу других возможностей, которые мы не будем рассматривать здесь, но вы можете заглянуть в видеоруководство Марка Шинвелла (Mark Shinwell) «Real-world debugging in OCaml» по адресу: <http://www.youtube.com/watch?v=NF2WpWnB-nk>.

Одной из замечательных особенностей машинного кода, получаемого в результате компиляции программ на языке OCaml, является совместное использование стека программным кодом на C. Это означает, что *gdb* возвращает объединенную трассировку стека, позволяющую судить о происходящем в вашей программе и в библиотеке времени выполнения. Благодаря этому вы сможете отслеживать все вызовы библиотек на языке C и даже обратные вызовы функций на OCaml из библиотек на C, находясь в окружении, включающем среду времени выполнения OCaml как библиотеку.

Профилирование машинного кода

Запись информации о том, где программа проводит большую часть времени, и ее анализ называются *профилированием производительности* (*performance profiling*). Двоичные выполняемые файлы, производимые компилятором OCaml, могут профилироваться точно так же, как двоичные выполняемые файлы, производимые компилятором C, — с использованием правил составления символических имен, описанных выше, и вывода профилировщика.

Большинство инструментов профилирования пользуется возможностью встраивания дополнительного кода в выполняемые файлы. OCaml поддерживает два таких инструмента:

- GNU *gprof*, измеряющий время выполнения и составляющий дерево вызовов;
- фреймворк профилирования *Perf*¹, включаемый в современные версии Linux.

Имейте в виду, что другие инструменты, такие как Valgrind, также прекрасно справляются с профилированием скомпилированного кода OCaml, при условии что программа скомпонована с ключом `-g`, встраивающим отладочную информацию.

Gprof

Профилировщик *gprof* создает профиль выполнения программ, записывая дерево вызовов функций и время, проведенное программой в этих функциях.

Чтобы профилировщик *gprof* мог дать максимально точную информацию, выполняемый код должен компилироваться *и* компоноваться с флагом `-p`. При компиляции с этим флагом генерируется дополнительный код, который в процессе выполнения программы записывает информацию профилирования в файл с именем *gmon.out*. Эта информация затем исследуется с помощью *gprof*.

Perf

Perf — более современная альтернатива *gprof*. Данный профилировщик не требует внедрения дополнительного кода. Вместо этого он использует аппаратные счетчики и отладочную информацию в выполняемом файле программы.

Запустите сначала *Perf*, передав ему файл программы, чтобы он мог собрать первичную информацию. В следующем примере мы попытались выполнить профилирование нашего теста, выполняющего тестирование производительности барьера записи, обсуждавшегося ранее:

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/back-end/perf_record.out

```
$ perf record -g ./barrier_bench.native
```

```
Estimated testing time 20s (change using -quota SECS).
```

Name	Time (ns)	Time 95ci	Percentage
----	-----	-----	-----
mutable	7_306_219	7_250_234-7_372_469	96.83
immutable	7_545_126	7_537_837-7_551_193	100.00

¹ <https://perf.wiki.kernel.org/>.


```
[ perf record: Woken up 11 times to write data ]
[ perf record: Captured and wrote 2.722 MB perf.data (~118926 samples) ]
perf record -g ./barrier.native
Estimated testing time 20s (change using -quota SECS).
```

Name	Time (ns)	Time 95ci	Percentage
----	-----	-----	-----
mutable	7_306_219	7_250_234-7_372_469	96.83
immutable	7_545_126	7_537_837-7_551_193	100.00

```
[ perf record: Woken up 11 times to write data ]
[ perf record: Captured and wrote 2.722 MB perf.data (~118926 samples) ]
```

По окончании можно приступить к исследованию результатов:

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/back-end/perf_report.out

```
$ perf report -g
+ 48.86% barrier.native barrier.native [.] camlBarrier__test_immutable_69282
+ 30.22% barrier.native barrier.native [.] camlBarrier__test_mutable_69279
+ 20.22% barrier.native barrier.native [.] caml_modify
```

Эти результаты отражают ход выполнения самого теста. Тестирование случая использования изменяемого значения состоит из вызовов функций `test_mutable` и `the caml_modify`. На выполнение этих функций уходит чуть больше половины всего времени работы приложения.

Профилировщик Perf имеет постоянно растущий набор команд, с помощью которых можно архивировать результаты сеансов профилирования и сравнивать их между собой. Подробнее об этом можно узнать на домашней странице проекта по адресу: https://perf.wiki.kernel.org/index.php/Main_Page.

Использование указателя на кадр для получения более точных результатов

Несмотря на то что Perf не требует добавления в выполняемые файлы дополнительного кода, осуществляющего зондирование, ему все же необходимо средство отслеживания вызовов функций, чтобы ядро могло обеспечить точную трассировку функций для каждого события.

Кадры стека OCaml слишком сложны для Perf, поэтому компиляция должна выполняться с использованием тех же соглашений, которые применяются для оформления вызовов функций на C. В 64-разрядных архитектурах Intel это означает, что для записи истории вызовов функций должен использоваться регистр указателя на кадр стека (frame pointer).

Такое применение указателя на кадр стека предполагает уменьшение скорости выполнения (обычно на 3–5%), из-за чего данная особенность не получила широкого распространения. По этой причине в OCaml 4.1 поддержка указателя на кадр стека является необязательной, но может включаться для увеличения точности результатов, собираемых профилировщиком Perf.

В OPAM имеется компилятор `switch`, компилирующий исходный код на OCaml с активной поддержкой указателя на кадр стека:

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/back-end/opam_switch.out

```
$ opam switch 4.01.0+fp
```

Использование указателя на кадр стека изменяет соглашения о вызовах функций OCaml, но OPAM позаботится о перекомпиляции всех ваших выполняемых файлов под новый интерфейс. Подробнее об этом можно прочитать в блоге OCamlPro: <http://www.ocamlpro.com/blog/2012/08/08/profile-native-code.html>.

Встраивание машинного кода в программы на C

Обычно компилятор машинного кода создает законченные выполняемые файлы, но может также производить автономные объектные файлы по аналогии с компилятором байт-кода. Такой объектный файл не имеет никаких зависимостей от OCaml, кроме библиотеки времени выполнения.

Среда времени выполнения для машинного кода – это другая библиотека, отличная от среды выполнения байт-кода, и устанавливается она в виде *libasmrun.a* в каталог стандартной библиотеки OCaml.

Попробуем скомпоновать те же исходные файлы, что приводились в примере встраивания байт-кода выше в этой главе:

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/back-end-embed/build_embed_native.out

```
$ ocamlpt -output-obj -o embed_native.o embed_me1.ml embed_me2.ml
$ gcc -Wall -I `ocamlc -where` -o final.native embed_native.o main.c \
  -L `ocamlc -where` -lasmrun -ltermcap -lm -ldl
$ ./final.native
Before calling OCaml
hello embedded world 1
hello embedded world 2
After calling OCaml
```

Файл *embed_native.o* – это автономный объектный файл, не имеющий ссылок в какой-либо другой код OCaml, кроме библиотеки времени выполнения, аналогичной среде времени выполнения байт-кода. Помните, что порядок компоновки библиотек имеет важное значение для современных инструментальных средств GNU, которые выполняют разрешение символических имен слева направо за один проход.

Отладочная среда времени выполнения

Несмотря на все ваши старания, легко можно внести ошибку в отдельные компоненты, такие как интерфейс к программному коду на C, которые могут вызвать повреждения в динамической памяти. OCaml включает библиотеку *libasmrund.a* – разновидность библиотеки времени выполнения, – скомпилированную с дополнительными отладочными проверками целостности памяти, которые выполняются в каждом цикле сборки мусора. Эти проверки прервут выполнение программы, оказавшейся на грани повреждения данных в памяти, и помогут локализовать ошибку в коде C.

Чтобы задействовать отладочную библиотеку, просто скомпонуйте свою программу с флагом `-runtime-variant d`:

Terminal

https://github.com/realworldocaml/examples/tree/v1/code/back-end-embed/run_debug_hello.out

```
$ ocamlpt -runtime-variant d -verbose -o hello.native hello.ml
$ ./hello.native
```

```
### OCaml runtime: debug mode ###
Initial minor heap size: 2048k bytes
Initial major heap size: 992k bytes
Initial space overhead: 80%
Initial max overhead: 500%
Initial heap increment: 992k bytes
Initial allocation policy: 0
Hello OCaml World!
```

Если вы получите сообщение об ошибке, что файл `libasmrund.a` не найден, это может быть обусловлено тем, что вы пользуетесь версией OCaml 4.00, а не 4.01. Она устанавливается по умолчанию только в самой последней версии, которую и следует использовать.

Сводка по расширениям имен файлов

Мы видели, как компилятор использует промежуточные файлы для хранения результатов различных этапов компиляции. В этом разделе приводится перечень всех этих файлов с краткими описаниями.

В табл. 23.1 перечислены промежуточные файлы, создаваемые компилятором *ocamlc*.

Таблица 23.1. Промежуточные файлы, создаваемые компилятором OCaml в байт-код

Расширение	Назначение
.ml	Файлы с исходным кодом реализаций модулей
.mli	Файлы с исходным кодом интерфейсов модулей. Если отсутствуют, генерируются автоматически из файлов .ml
.cmi	Файлы с интерфейсами модулей, скомпилированными из файлов .mli
.cmo	Объектные файлы со скомпилированным байт-кодом реализаций модулей
.cma	Библиотеки объектных файлов, упакованные в единые файлы
.o	Объектные файлы, скомпилированные из файлов с исходным кодом на C
.cmt	Реализации модулей в виде типизированных абстрактных синтаксических деревьев
.cmti	Интерфейсы модулей в виде типизированных абстрактных синтаксических деревьев
.annot	Файлы аннотаций старого образца, ныне замененные файлами .cmt

Компилятор машинного кода производит некоторые дополнительные файлы (см. табл. 23.2).

Таблица 23.2. Промежуточные файлы, создаваемые компилятором OCaml в машинный код

Расширение	Назначение
.o	Скомпилированные объектные файлы с реализациями модулей
.cmx	Содержат дополнительную информацию для компоновки и оптимизации
.cmxa и .a	Библиотеки из файлов .cmx и .o, сохраняются в файлах с расширениями .cmxa и .a соответственно. Эти файлы всегда создаются парами
.S или .s	Код на языке Ассемблера, который генерируется при компиляции с флагом <code>-S</code>

Алфавитный указатель

Символы

`%`.byte, правило, 508
`.cmi`, файлы, 511
`.cmx`, файлы, 511
`=`, оператор сравнения (структурного), 307
`==`, оператор сравнения (физическое равенство), 307

А

Argot, генератор HTML, 477
Array, модуль
 Array.blit, функция, 182
 Array.set, функция, 182
assert, директива, 168
AST (Abstract Syntax Tree – абстрактное синтаксическое дерево), 358
Async, библиотека
 Ivar, модуль, 393
 и системные потоки выполнения, 416
 обработка исключений, 406
 основы, 389
 преимущества, 389
 пример использования DuckDuckGo, 401
 пример эхо-сервера, 395
 тайм-ауты и отмена, 413
ATDgen, библиотека
 аннотации, 351
 компиляция спецификаций в код OCaml, 352
 основы, 350
 пример, 354
 установка, 350

В

bash, автодополнение, 336
Bigarray, интерфейс, 454
Bigstring, модуль, 455
bind, функция, 163, 391
Bin_prot, библиотека, 477
BLAS, математическая библиотека, 455

С

Camlimages, библиотека, 259

Camlp4, механизм расширений синтаксиса, 477
 вызов из командной строки, 480
Camlp4, механизм расширения синтаксиса, 375
Camomile, парсер с поддержкой Юникода, 368
cohttp, библиотека, 401
Command.basic, модуль, 317
Command.group, функция, 327
Command, модуль, 330
Comparable, модуль
 Comparable.Make, функтор, 234, 304
 Comparable.S, интерфейс, 304
Comparator.Poly, модуль, 302
config, ключевое слово, 327
Container.Make, функтор, 234
corebuild, утилита, 100
Core.Std, библиотека, 23
Core, библиотека, 23
Core, стандартная библиотека императивные словари, 178
 поиск с помощью ocamlfind, 100
Cryptokit, библиотека, 259
Ctypes, библиотека
 директивы сборки, 426
 массивы, 437
 организация структур в памяти, 443
 передача функций в код на C, 438
 пример интерфейса к терминалу, 423
 простые скалярные типы языка C, 427
 срок жизни значений Ctypes, 441
 структуры и объединения, 432
 указатели и массивы, 429
 установка, 422

Д

Deferred.any, функция, 413
Deferred.bind, функция, 390
Deferred.both, функция, 413
Deferred.never, функция, 399
Deferred.peek, функция, 390
Deferred.t, тип, 389

destutter, функция, 93
 Doubly_linked, модуль, 189
 DuckDuckGo, пример
 дополнительные библиотеки, 401
 клиентские HTTP-запросы, 402
 обработка исключений, 410
 парсинг строк JSON, 402

Е

eprintf, функция, 204
 Error.of_list, функция, 163
 Error.tag, функция, 163
 Error.t, тип, 161
 Exn, модуль
 backtrace, функция, 172
 set_recording, функция, 173
 exn, тип, 166

F

failwith, функция, 167
 FFI (Foreign Function Interface – интерфейс внешних функций), 184
 fieldslib, библиотека, 134
 Fieldslib, расширение, 477
 Field, модуль
 Field.fset, функция, 134
 Field.get, функция, 134
 Field.name, функция, 134
 Field.setter, функция, 134
 FIFO (first-in, first-out – первым пришел, первым вышел), очереди, 231
 filter_string, функция, 344
 find_exn, функция, 166
 for, циклы, 46, 185
 fprintf, функция, 204
 function, ключевое слово, 65, 94
 fun, ключевое слово
 анонимные функции, 55
 синтаксис каррирования, 58
 функции нескольких аргументов, 57

G

Gc, модуль, 457
 gdb, отладчик, 515
 GitHub API, 350, 354
 gprof, профилировщик, 519

H

Hashable.Make, функтор, 234, 310
 Hashable.S, интерфейс, 310
 Hashtbl, модуль, 307
 Heap_block, модуль, 467

I

In_channel, модуль, 200
 install, ключевое слово, 327
 In_thread.run, функция, 417
 In_thread, модуль, 417
 Iobuf, модуль, 455
 Ivar, модуль, 393

J

-j-custom-fields FUNCTION, флаг, 353
 -j-defaults, флаг, 353
 js_of_ocaml, компилятор, 294
 JSON, формат данных
 автоматическое отображение, 350
 выборка значений, 343
 достоинства и недостатки, 340
 конструирование значений, 346
 нестандартные расширения, 349
 основы, 339
 парсинг с помощью библиотеки Yojson, 340
 -j-std, флаг, 353

L

LablGL, библиотека, 294
 Lablgtk, библиотека, 294
 Lacaml, библиотека, 455
 lambda-код
 основы, 501
 оптимизация сопоставления
 с образцом, 501
 оценка производительности
 сопоставления с образцом, 504
 LAPACK, библиотека связи, 184
 LAPACK, математическая библиотека, 455
 lazy, ключевое слово, 191
 let rec, 188, 197, 199
 let (), объявление, 99
 let-привязки, рекурсивные
 и нерекурсивные функции, 60
 let-синтаксис
 вложенные привязки, 52

- и сопоставление с образцом, 53
- и функции, 56
- привязки верхнего уровня, 50
- let, синтаксис, вложенные let-привязки, 38
- libasmrun.a, библиотека, 521
- libffi, библиотека, 422
- linkpkg, флаг, 100
- List.Assoc, модуль
 - List.Assoc.add, функция, 98
 - List.Assoc.find, функция, 98
- List.dedup, функция, 63
- List, модуль, 32, 84
 - List.append, функция, 90
 - List.filter_map, функция, 89
 - List.filter, функция, 88
 - List.fold, функция, 85
 - List.init, функция, 92
 - List.map2_exn, функция, 85
 - List.map, функция, 84
 - List.partition_tf, функция, 89
 - List.reduce, функция, 88
 - и функция String.concat, 86
 - создание таблиц, 84
- lit, суффикс, 349
- loop_forever, функция, 397
- LR(1), грамматики, 360

M

- main, функция, 99
- malloc(3), функция, 460
- Map, модуль
 - Map.of_alist_exn, функция, 298
 - Map.to_tree, функция, 301
- match, инструкции, 83
- match, инструкция, 78
- member, функция, 344
- Menhir, генератор парсеров
 - в сравнении с ocaml yacc, 357
 - встроенные правила, 363
 - запуск, 364
 - леворекурсивные определения, 363
- module, ключевое слово, 236
- Monad.Make, функтор, 234

N

- Ncurses, библиотека, 423
- never_returns, функция, 398

O

- OAuth, веб-протокол, 350
- Obj, модуль, 451
- ocaml-getopt, библиотека, 338
- OCAMLRUNPARAM, переменная окружения, 173, 457
- ocaml yacc, генератор парсеров, 357
- OCaml, инструменты
 - ocamlbuild, 100
 - ocamlc, 100, 506
 - ocamlfind, 100
 - ocamlopt, 100, 511
 - ocamlrun, 506
- OCaml, комплект инструментов
 - ocamlc, 470
 - ocamlopt, 470
 - обзор, 471
- OCaml, комплект инструментов
 - ocamldoc, 476
- OCaml, язык программирования
 - введение, 23
 - основы, 23
- ocp-index, утилита, 496
- OPAM, инструмент управления пакетами, 327
- Out_channel, модуль, 200
 - stderr, канал, 201
 - stdin, канал, 201
 - stdout, канал, 201

P

- Perf, профилировщик, 519
- phys_equal, функция, 307
- POSIX, функции, 429
- printf, функция, 202

Q

- qsort, функция, 438

R

- Random, модуль, 46
- Reader, модуль, 395
- rec, ключевое слово, 59
- remote, ключевое слово, 327
- Result.t, тип, 161
- returning, функция, 436
- return, функция, 392

RFC 4627, документ, 402
RFC 4627, описание формата JSON, 339

S

Scheduler.go, функция, 397
Sexplib, пакет, 162, 374, 382, 477
sexp_list, директива, 384
sexp_opaque, директива, 383
sexp_option, директива, 385
sexp-преобразования, 299
Type_conv, библиотека, 375
расширение синтаксиса, 374
sexp_list, директива, 384
sexp_opaque, директива, 383
sexp_option, директива, 385
sexp, объявление, 242
sexp, объявления, 167, 227
sprintf, функция, 204
String.concat, функция, 86
s-выражения, 162
десериализация, 380
запросы и ответы, 242
изменение поведения по умолчанию, 381
определение значений по умолчанию, 385
основы использования, 372
отладка, 380
преобразование типов OCaml, 374
пример, 227
проверка инвариантности, 380
формат, 376
сохранение инвариантов, 377

T

TCP, серверы и клиенты, 396
textwrap, библиотека, 401
-thread, флаг, 100
to_int, функция, 344
to_string, функция, 344
Type_conv, библиотека, 375

U

Ulex, генератор лексических анализаторов, 368
uri, библиотека, 401
-use-menhir, флаг, 364
utor, интерактивная оболочка, 23
Utf, кодек Юникода, 368

W

while, циклы, 46, 185
Writer, модуль, 395

Y

yjson, библиотека, 401
Yjson, библиотека
комбинаторы, 344
парсинг данных в формате JSON, 340
парсинг строк JSON, 402
поддержка вариантных типов, 349
поддержка расширений формата JSON, 349
установка, 340

A

Абстрактное синтаксическое дерево (Abstract Syntax Tree, AST), 358, 444, 473
Абстрактное синтаксическое дерево (typed abstract syntax tree), типизированное, 496
Абстрактные типы, 103, 240
Автодополнение, 336, 496
обработчики, 337
Автоматический вывод типов, 484
Алгебраические типы данных, 141
Анализ командной строки
Command, библиотека, 315
автодополнение средствами bash, 336
альтернативы библиотеке Command, 338
аргументы по умолчанию, 321
группировка подкоманд, 327
именованные аргументы, 325
интерактивный ввод аргументов, 333
необязательные аргументы, 321
определение собственных типов аргументов, 321
последовательности аргументов, 324
простейший, 316
расширенное управление парсингом, 329
типы аргументов, 319
Анимационные эффекты
отображение, 293
с помощью примесей, 291
Аннотации типов, 73, 486
Анонимные аргументы, 316
Анонимные функции, 42, 55

Аргументы

- анонимные, 316
 - аргумент unit функции обратного вызова, 317
 - аргументы с меткой, 32
 - вывод типов, 72
 - группировка подкоманд, 327
 - именованные аргументы, 325
 - интерактивный ввод, 333
 - необязательные, 69, 71, 321
 - определение собственных типов аргументов, 321
 - последовательности аргументов, 324
 - по умолчанию, 321
 - расширенное управление парсингом, 329
 - с метками, 66, 68, 335
 - с метками и функции высшего порядка, 68
 - типы аргументов, 319
- Аргументы с меткой, 32
- Ассоциативные списки, 296

Б

- Байт-кода компилятор
- инструменты, 506
 - компиляция и компоновка байт-кода, 507
 - набор инструкций, 507
- Барьеры записи, 464
- Безопасность, проблемы, модуль Obj, 451
- Белые значения, 462
- Бинарные методы, 281
- Благоприятные эффекты
- memoизация, 190, 192
 - отложенные вычисления, 190
- Блоки (памяти), 445
- Блокировка, 389
- Большие массивы, 183

В

- Вариантные типы, 137
- базовый синтаксис, 137
 - и записи, 141
 - и рекурсивные структуры данных, 145
 - особенности, 137
 - полиморфные, 149
 - универсальные образцы и рефакторинг, 139
- Варианты, 41
- представление в памяти, 450

Ввод/вывод, 200

- копирование данных, 395
 - терминальный ввод/вывод, 200
 - файлы, 204
 - форматированный, 202
- Верхнего уровня, привязки, 50
- Вещественные числа, 449
- Виртуальные классы, 285
- Вложенные let-привязки, 52
- Вложенные модули, 107
- Внешние библиотеки, взаимодействие, 184
- Внешняя память, 454
- Восстановление работоспособности после исключений, 169
- Вспомогательная куча, 457
- выделение памяти, 457
 - установка размера, 458
- Вывод типов, 25, 27
- Выравнивание указателей, 446
- Выравнивание, форматирование в функции printf, 204
- Выражения, порядок вычислений, 207
- Вычисления с применением интервалов, 218
- функции сравнения, 219
- Вычитания, оператор, 62

Г

- Генераторы парсеров, 357
- Геометрические фигуры, 259, 285
- Гипотеза о поколениях (generational hypothesis), 457
- Грамматики
- LR(1), 360
 - контекстно-свободные, 360
- Графические библиотеки, 294

Д

- Двоичные числа, форматирование в функции printf, 204
- Двусвязные списки, 186
- Деструктивная подстановка, 226
- Десятичные числа, форматирование в функции printf, 204
- Динамическое программирование, 192, 477
- Документация, автоматическое создание на основе интерфейсов, 475
- Дубликатов, удаление, 63

Е

Единицы компиляции (compilation units), 471, 491

З

Записи, 40

базовый синтаксис, 120

и варианты типы, 141

изменяемые поля, 43, 131

и уплотнение полей, 124

конструирование, 124

повторное использование имен полей, 125

поля первого порядка, 132

представление в памяти, 448

сопоставление с образцом и полнота, 122

функциональные обновления, 130

Затенение, 51

Защищенные методы, 280

Значения

в структурах JSON, 346

в формате JSON, 339

выборка из структур JSON, 343

и финализаторы, 467

корневые, 456

отображение составных значений, 431

представление в памяти, 444

упаковка (boxing), 445

фильтрация с помощью List.filter, 88

И

Идентификаторы

добавление в модули, 111

и открытие модулей, 109

Изменяемые данные, 182

поля записей/объектов и ссылочные ячейки, 183

Изменяемые поля записей, 43, 131

Императивное программирование, 42

ввод/вывод, 200

двусвязные списки, 186

и благоприятные эффекты, 190

императивные словари, 178

итеративные функции, 189

массивы, 43

недостатки, 189

обзор, 215

побочные эффекты, 209

порядок вычислений, 207

преимущества, 177

слабый полиморфизм, 209

ссылки, 45

циклы for и while, 46

элементарные изменяемые данные, 182

Инвариантность (invariant), 262

Инициализаторы, 288

Интерактивная оболочка Caml4p, 479

Интерактивный ввод аргументов, 333

Интерфейс внешних функций

массивы, 437

организация структур в памяти, 443

основы, 422

передача функций в код на C, 438

пример интерфейса к терминалу, 423

простые скалярные типы языка C, 427

структуры и объединения, 432

указатели и массивы, 429

Интерфейс внешних функций (Foreign Function Interface, FFI), 184

Интерфейсы

Comparable.S, интерфейс, 304

Hashable.S, 310

автоматическое создание

документации, 475

интерфейс внешних функций (Foreign

Function Interface, FFI), 422

как типы объектов, 272

сокрытие реализации, 103

Инфиксные операторы, 60

Исключения, 165

асинхронные ошибки, 407

в конкурентном программировании, 406

восстановление работоспособности

после исключений, 169

в поисковых системах, 410

вспомогательные функции, 167

и ошибки, 29

комбинирование исключений и типов

с информацией об ошибках, 175

обработка, 169, 406

обработчики исключений, 169

перехват исключений, 170

текстовое представление, 167

трассировка стека, 172

Исходный код

автоматическое оформление отступов, 473

парсинг, 473
 препроцессинг, 477
 Итеративные функции, 189
 Итераторы, 272
 функциональные, 274

К

Кадры стека, 92
 Каналы, 399
 Каррированные функции, 57
 Квадратные скобки, 362
 Классы
 базовый синтаксис, 269
 бинарные методы, 281
 виртуальные, 285
 и инициализаторы, 288
 и наследование, 276
 и открытая рекурсия, 278
 множественное наследование, 289
 параметры классов и полиморфизм, 270
 преимущества, 259
 примеси, 290
 скрытые методы, 280
 типы классов, 277
 типы объектов и интерфейсы, 272
 Ключ/значение, пары, 296, 339
 Ковариантность (covariant), 262
 Комбинаторы
 в библиотеке Yojson, 344
 функциональные, 315
 Комбинаторы, функциональные, 344
 Компактификация, 463
 управление частотой, 463
 Компараторы, создание отображений, 298
 Компилятор байт-кода и компилятор
 машинного кода, 101
 Компиляторы
 байт-код и машинный код, 471
 в байт-код и в машинный код, 101
 включение/выключение
 предупреждений, 123
 диаграмма работы, 471
 исходный код, 472
 комплект инструментов, 471
 порядок генерации кода, 485
 Компиляции процесс
 быстрый машинный код, 511
 комплект инструментов, 471

нетипизированная lambda-форма, 501
 парсинг исходного кода, 473
 переносимый байт-код, 506
 препроцессинг исходного кода, 477
 расширения имен файлов, 522
 статическая проверка типов, 483
 Компиляция, этапы процесса, 444
 Компиляция, разделяемая, 491
 Конкретные типы, 106
 Конкурентное программирование, 388, 406
 Контекстно-свободные грамматики, 360
 Контравариантность (contravariant), 262
 Корневые значения, 456
 Кorteжи, 30, 349
 представление в памяти, 448
 Кучи Heap_block, модуль, 467

Л

Лексические анализаторы
 коллекция регулярных выражений, 365
 необязательный фрагмент кода, 364
 поддержка Юникода, 368
 правила, 366
 рекурсивные правила, 367
 спецификация, 364
 Лексический анализ, 358
 Линейная алгебра, 455
 Локально абстрактные типы, 240

М

Маркировка и чистка, сборка мусора, 456
 Массивы, 43, 179
 большие, 183
 и императивное программирование, 43
 и указатели, 429
 определение, 437
 представление в памяти, 448
 Машинный код
 встраивание в программы на C, 521
 исследование машинного кода, 511
 отладка, 515
 преимущества, 511
 профилирование, 519
 Мемоизация
 преимущества и недостатки, 192
 пример, 193
 функций, 192

Методы

- бинарные, 281
- и виртуальные классы, 285
- скрытые, 280

Механизм форматированного вывода, 372

Множественное наследование, примеси, 290

Модули

- вложенные, 107
- именование, 103
- локальное открытие, 110
- модули первого порядка, 235
- основы, 101
- открытие, 109
- подключение, 111
- проектирование с применением модулей, 117
- пути поиска, 492
- раздельная компиляция, 491
- рекомендации по проектированию, 117
- сокрытие реализации, 103
- соотношения с файлами, 491
- типы, 103
- упаковка, 493
- циклические зависимости, 115

Модули первого порядка

- альтернативные решения, 252
- и объекты, 258
- и полиморфизм, 240
- использование, 235
- равенство типов, 237
- фреймворк обработки запросов, 241

Модульные тесты, 340

Монады, 392

Мониторы, 408

Мьютексы, 420

Н

Наследование, 276, 289

Начальный символ, 361

Неинтерпретируемые, двоичные данные, 447

Необязательные аргументы, 69, 321
явная передача, 71

Необязательные значения, 38

Нетерминальные символы, 361

Неупакованные целые числа, 448

О

Области видимости, 50

Обобщенные типы, 28

Обработка запросов

- диспетчеризация, 244
- загрузка и выгрузка обработчиков, 248
- и модули первого порядка, 241
- реализация, 242

Обработка исключений, 406

Обработка ошибок, 368

- восстановление работоспособности после исключений, 169
- вспомогательные функции для возбуждения исключений, 167
- выбор стратегии, 175
- исключения, 165
- комбинирование исключений и типов с информацией об ошибках, 175
- обработка исключений, 169
- обработчики исключений, 169
- перехват исключений, 170
- преобразование ошибок, 163
- типы возвращаемых значений с признаком ошибки, 159
- трассировка стека, 172

Обработчики, автодополнения, 337

Объектно-ориентированное программирование (ООП), 253, 269

Объектные файлы C, 509

Объекты

- в OCaml, 253
- в объектно-ориентированном программировании, 253
- достоинства и недостатки, 258
- и модули первого порядка, 258
- и полиморфизм, 255
- и сужение, 265
- неизменяемые, 257
- типы объектов и интерфейсы, 272

Ограничение значения, 210

Однозначные образцы, 122

Однородное представление памяти (uniform memory representation), 445

Операторы

- оператор ::, 33, 77
- оператор вычитания, 62
- оператор отрицания, 62

оператор последовательного выполнения, 63
 операторы полиморфного сравнения, 95
 префиксные и инфиксные, 60
 управления указателями, 438
 Основная куча, 457
 выделение памяти, 460
 для долгоживущих блоков, 459
 компактификация, 463
 стратегии выделения памяти, 461
 указатели между поколениями, 464
 управление ростом, 460
 управление чисткой, 463
 частота компактификации, 463
 этап компактификации, 459
 этап маркировки и чистки, 459
 Отказ в обслуживании (Denial-Of-Service, DOS), атаки, 308
 Открытая рекурсия, 253, 259, 278
 Открытая схема хэширования, 178
 Открытие модулей, 109
 локальное, 110
 Открытые типы объектов, 256
 Отладка
 s-выражений, 380
 двоичных выполняемых файлов, 515
 имена функций в отладчике, 515
 интерактивные отладчики, 515
 отладочная среда выполнения, 521
 трассировка стека, 172
 Отложенные вычисления, 190
 Отображение
 JSON в типы OCaml, 350
 составных значений, 431
 Отображение анимационных эффектов, 293
 Отображения, 297
 Comparable.S, интерфейс, 304
 древовидная структура, 301
 основы, 296
 полиморфное сравнение, 302
 создание с помощью компараторов, 298
 сравнение с хэш-таблицами, 311
 Отрицания, оператор, 62
 Ошибки
 вывод сообщений с помощью s-выражений, 380
 и исключения, 29
 несовпадение типов, 113

несоответствие определений типов, 115
 определение с помощью инструкции match, 83
 отсутствие определений, 114
 предупреждения компилятора, 123
 преобразование, 163
 синтаксические ошибки, 473
 тайм-ауты и отмена, 413
 универсальные образцы и рефакторинг, 139
 циклические зависимости, 115

П

Память

 выделение для указателей, 430
 организация структур, 443
 стратегии выделения памяти, 461
 строки и буферы символов, 432
 уменьшение фрагментации, 461
 управление внешней памятью, 454
 управление памятью, 456
 управление политикой выделения памяти, 462

Параметрический полиморфизм, 29

Парсинг

 генераторы парсеров, 357
 исходного кода, 473
 обработка ошибок, 368
 определение, 357
 определение лексического анализатора, 364
 определение парсеров, 360
 последовательностей, 362

Пары ключ/значение, 339

Первого порядка, поля, 132

Переменные

 затенение, 51
 неизменяемые, 53
 область видимости, 50

Переменные типа, 209

Переходники (thunk), 162

Побочные эффекты, 209

Повторяющиеся значения, удаление, 89, 93

Подключение модулей, 111

Подкоманды, группировка, 327

Подтипизация, 259

 в глубину, 260

 в ширину, 259

- и вариантность, 261
 - и рядный полиморфизм, 267
 - основы, 259
 - Позднее связывание, 258
 - Полиморфизм
 - в локально абстрактных типах, 240
 - в модулях первого порядка, 240
 - в объектах, 255
 - в хэш-функциях, 309
 - и параметры классов, 270
 - рядный полиморфизм, 257
 - слабый, 209
 - Полиморфное сравнение, 95, 302, 479, 512
 - Полиморфные вариантные типы, 149
 - автоматический вывод, 150
 - в данных JSON, 341
 - верхняя и нижняя границы, 150
 - в сравнении с обычными вариантами, 151, 157
 - недостатки, 157
 - проверка типов, 348
 - универсальные образцы, 153
 - Полиморфные варианты, представление в памяти, 452
 - Полиморфные подтипы вариантов, 261
 - Поля
 - добавление в структуры, 433
 - изменяемые, 131, 178, 183
 - первого порядка, 132
 - повторное использование имен полей, 125
 - уплотнение полей, 124
 - уплотнения полей, 40
 - функции доступа, 133
 - функциональные обновления, 130
 - Порядок вычислений, 207
 - Потоки выполнения, системные, 388, 416
 - и блокировки, 420
 - преимущества, 416
 - Представительские типы, 283
 - Представление памяти во время выполнения
 - блоки и значения, 447
 - важность знания, 444
 - варианты, 450
 - записи, 448
 - кортежи, 448
 - массивы, 448
 - нестандартные блоки, 454
 - полиморфные варианты, 452
 - списки, 450
 - строковые значения, 453
 - Префиксные операторы, 60
 - Приведение вниз (down casting), 265
 - Привязки
 - верхнего уровня, 50
 - область видимости, 50
 - шаблонные символы в let-привязках, 481
 - Примеси, 290
 - Проблемы безопасности
 - модуль Obj, 451
 - отказ в обслуживании (Denial-Of-Service, DOS), атаки, 308
 - Проверка инвариантности, 380
 - Проверка типов, 348
 - Программирование
 - динамическое, 192
 - законченная программа, 48
 - императивное, 177
 - конкурентное с применением Async, 389
 - межъязыковые интерфейсы, 422
 - объектно-ориентированное, 269
 - объектно-ориентированное (ООП), 253
 - функциональное и императивное, 177
 - Программы
 - в единственном файле, 98
 - из нескольких файлов, 101
 - Проектирование с применением модулей, 117
 - Профилирование машинного кода, 519
 - Пустые значения, 39
- ## Р
- Работоспособность, восстановление после исключений, 169
 - Равенство
 - структурное, 307
 - физическое, 307
 - Равенство, проверка, 307
 - Раздельная компиляция, 491
 - Раннее связывание, 258
 - Расстояние Левенштейна, 193
 - Расстояние редактирования, 193
 - Расширения синтаксиса
 - злоупотребления, 483
 - Расширяемые парсеры, 477
 - Регулярные выражения, 365
 - Рекурсивные структуры данных, 145

Рекурсивные функции, 59

для списков, 36

определение, 59

Рекурсия

в лексических анализаторах, 367

в типах JSON, 341

открытая, 278

хвостовая, 91

Рефакторинг, 139, 333

Рискованные типы, 489

Рядный полиморфизм, 257

и подтипизация, 267

С

Сборка мусора, 447, 456

алгоритм, 456

вспомогательная куча, 457

для долгоживущих блоков, 459

и указатели между поколениями, 464

маркировка и сканирование кучи, 462

маркировка и чистка, 456

основная куча, 457

с разделением на поколения, 457

срок жизни значений Ctypes, 441

финализаторы, 467

этап компактификации, 459

этап маркировки и чистки, 459

Сериализация данных

в формат JSON, 339

с применением s-выражений, 371

Серые значения, 462

Сигнатуры

абстрактные типы, 103

конкретные типы, 106

Синие значения, 462

Синтаксические ошибки, 473

Система модулей, 484

Системные потоки выполнения, 388, 416

и блокировки, 420

преимущества, 416

Скалярные типы языка C, 427

Скрытые методы, 280

Слабый полиморфизм, 209

Словари, императивные, 178

Службы поиска, 402

Совместно используемые ограничения, 223

Сопоставление с образцом

в списках, 34

и let, 53

извлечение данных, 78

и полнота, 122

компактность и скорость, 93

оптимизация в lambda-коде, 501

оценка производительности, 504

структуры данных, 80

универсальные образцы, 139, 153

фундаментальные алгоритмы, 504

Списки, 31

List, модуль, 84

ассоциативные списки, 296

двусвязные списки, 186

деление с помощью List.partition_tf, 89

добавление новых привязок, 98

извлечение данных из, 78

комбинирование, 90

объединение элементов, 88

оператор ::, 33, 77

определение длины, 91

поиск по ключу, 98

представление в памяти, 450

рекурсивные функции для списков, 36

создание, 77

сопоставление с образцом, 34

структура, 78

удаление дубликатов, 63

удаление повторяющихся значений,

89, 93

фильтрация значений, 88

Ссылки, 45

Ссылочные ячейки, 183

Статическая проверка во время

компиляции, 340, 470

Сторонние библиотеки поддержки

графики, 294

Стратегии выделения памяти, 461

первый подходящий, 461

следующий подходящий, 461

Стратегия обработки ошибок, 175

Строки

и буферы символов, 432

конкатенация, 86

представление в памяти, 453

Строки формата, 202

Строковые значения, представление

в памяти, 453

Структурное равенство, 307

Структуры данных

- варианты, 41
- записи, 40
- изменяемые поля записей, 43
- и сопоставление с образцом, 80
- кортежи, 30
- массивы, 43
- неизменяемые, 43
- необязательные значения, 38
- пары ключ/значение, 296
- пустые значения, 39
- рекурсивные, 145
- списки, 31
- ссылки, 45
- циклические, 187
- элементарные изменяемые данные, 182

Структуры и объединения

- добавление полей, 433
- команда вывода времени, 435
- незавершенные определения, 433
- определение структур, 432
- организация структур в памяти, 443

Сужение, 265

Суммарные типы, 141

Т

Тайм-ауты, 413

Типы абстрактные, 103

Типы возвращаемых значений с признаком ошибки, 159

Типы данных

- абстрактные, 240
- алгебраические, 141
- вариантные типы, 137
- варианты, 40
- записи, 40
- ковариантные, 214
- контравариантные, 214
- локально абстрактные типы, 240
- новые, 40
- полиморфные вариантные типы, 149
- с фиксированной и переменной структурой, 122

Типы классов, 277

Типы произведений, 141

Точки с запятой и запятые, 33

Трассировка стека, 172

У

Удаление дубликатов, 63

Указатели, 445

- выделение памяти для, 430
- выравнивание по границам слов, 446
- и массивы, 429
- между поколениями, 464
- операторы управления, 438

Указатели между поколениями, 464

Указатель на кадр стека (frame pointer), 520

Универсальные образцы, 153

Упаковка значений (boxing), 445

Упаковка модулей, 493

Уплотнения полей, 40

Управление политикой выделения памяти, 462

Ф

Файлы

- .mli, 103
- .mll, 364
- .mly, 360
- программы в единственном файле, 98
- программы из нескольких файлов, 101

Фигурные скобки, 362

Физическое равенство, 307

Финализаторы, 467

Флаги, 325, 353, 496

Форматированный ввод/вывод, 202

Форматированный вывод, 372

Функторы

- абстрактные, 222
- вычисления с применением интервалов, 218
- деструктивная подстановка, 226
- множественные интерфейсы, 227
- основные механизмы, 216
- преимущества, 216
- расширение модулей, 231
- совместно используемые ограничения, 223

Функции, 25, 54

- filter_string, функция, 344
- member, функция, 344
- to_int, функция, 344
- to_string, функция, 344
- анонимные, 55
- анонимные функции, 42

аргументы с метками, 66, 68
 вывод типов аргументов, 72
 высшего порядка и аргументы
 с метками, 68
 и исключения, 166, 167
 итеративные, 189
 каррированные, 57
 не возвращающие управления, 397
 необязательные аргументы, 69, 71
 нескольких аргументов, 57
 обратного вызова, 317
 объявление с ключевым словом
 function, 65
 определение, 65
 префиксные и инфиксные операторы, 60
 рекурсивные, 59
 финализации, 467
 хэш-функции, 308
 частичное применение, 58
 Функции доступа к полям, 133
 Функциональное и императивное
 программирование, 42
 Функциональные итераторы, 274
 Функциональные комбинаторы, 344
 Функциональные обновления, 130
 Функциональный стиль, 42

Х

Хвостовая рекурсия, 91
 Хэш-таблицы, 307
 основы, 296, 307
 полиморфизм в хэш-функциях, 309
 производительность, 308
 соответствие интерфейсу Hashable.S, 310
 сравнение с отображениями, 311
 Хиндли-Милнера (Hindley-Milner),
 алгоритм, 486

Ц

Целые числа, 445, 448
 Циклические зависимости, 115
 Циклические структуры данных, 187
 Циклы, 185
 Циклы событий, 388

Ч

Частичное применение, 58, 64, 211
 Черные значения, 462

Ш

Шаблонные символы в let-привязках, 481
 Шестнадцатеричные числа,
 форматирование в функции printf, 204

Э

Элементарные изменяемые данные, 182
 поля записей/объектов и ссылочные
 ячейки, 183
 Элементы
 вставка в список, 188
 деление с помощью List.partition_tf, 89
 обход с помощью итераторов, 272
 объединение с помощью List.reduce, 88
 определение новых, 188
 Элизии (...), 256
 Эхо-серверы, 395

Я

Явная подтипизация, 484
 Явное определение главных типов, 489

Книги издательства «ДМК Пресс» можно заказать
в торгово-издательском холдинге «Планета Альянс» наложенным платежом,
выслав открытку или письмо по почтовому адресу:

115487, г. Москва, 2-й Нагатинский пр-д, д. 6А.

При оформлении заказа следует указать адрес (полностью), по которому должны быть
высланы книги; фамилию, имя и отчество получателя.

Желательно также указать свой телефон и электронный адрес.

Эти книги вы можете заказать и в интернет-магазине: **www.aliants-kniga.ru**.

Оптовые закупки: тел. **(499) 782-38-89**.

Электронный адрес: **books@aliants-kniga.ru**.

Ярон Мински, Анил Мадхавapedди и Джейсон Хикки

Программирование на языке OCaml

Главный редактор *Мовчан Д. А.*
dmkpress@gmail.com

Перевод *Киселев А. Н.*

Корректор *Синяева Г. И.*

Верстка *Чаннова А. А.*

Дизайн обложки *Мовчан А. Г.*

Формат 70×100 1/16 .

Гарнитура «Петербург». Печать офсетная.

Усл. печ. л. 32. Тираж 200 экз.

№

Веб-сайт издательства: www.dmk.ru

Эта книга введет вас в мир OCaml, надежный язык программирования, обладающий большой выразительностью, безопасностью и быстродействием. Пройдя через множество примеров, вы быстро поймете, что OCaml – это превосходный инструмент, позволяющий писать быстрый, компактный и надежный системный код.

Вы познакомитесь с основными понятиями языка, узнаете о приемах и инструментах, помогающих превратить OCaml в эффективное средство разработки практических приложений. В конце книги вы сможете углубиться в изучение тонких особенностей инструментов компилятора и среды выполнения OCaml.

Включённые в книгу упражнения позволяют вам:

- познакомиться с основами языка, такими как функции высшего порядка, алгебраические типы данных и модули;
- исследовать расширенные особенности, такие как функторы, модули первого порядка и объекты;
- узнать, как пользоваться библиотекой Core – всеобъемлющей, универсальной стандартной библиотекой для OCaml;
- научиться проектировать эффективные библиотеки многократного использования с применением подходов к абстрагированию и модульности, характерных для OCaml;
- увидеть способы решения практических задач программирования – от анализа аргументов командной строки до реализации асинхронных сетевых операций;
- изучить приемы профилирования и отладки с такими инструментами, как GNU gdb.

«Эта книга давно ожидалась в сообществе пользователей OCaml. В ней не только дается превосходное введение в программирование на языке OCaml, она также знакомит с библиотеками и инструментами, повышающими эффективность программирования на этом языке.»

Ксавье Леруа (Xavier Leroy),

автор и первый разработчик языка OCaml, старший научный сотрудник института INRIA

«Программисты – это цифровые хореографы, осторожно балансирующие между правильностью, модульностью, производительностью и параллельным выполнением. Данная книга покажет вам, как легко и просто сохранить этот баланс.»

Мариус Эриксен (Marius Eriksen),

Ведущий инженер программист в компании Twitter

Internet-магазин:

www.dmkpress.com

Книга – почтой:

e-mail: orders@alians-kniga.ru

Оптовая продажа:

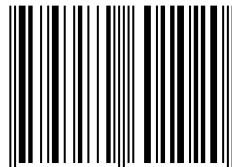
«Альянс-книга»

Тел./факс: (499) 782-3889

e-mail: books@alians-kniga.ru



ISBN 978-5-97060-102-0



9 785970 601020 >